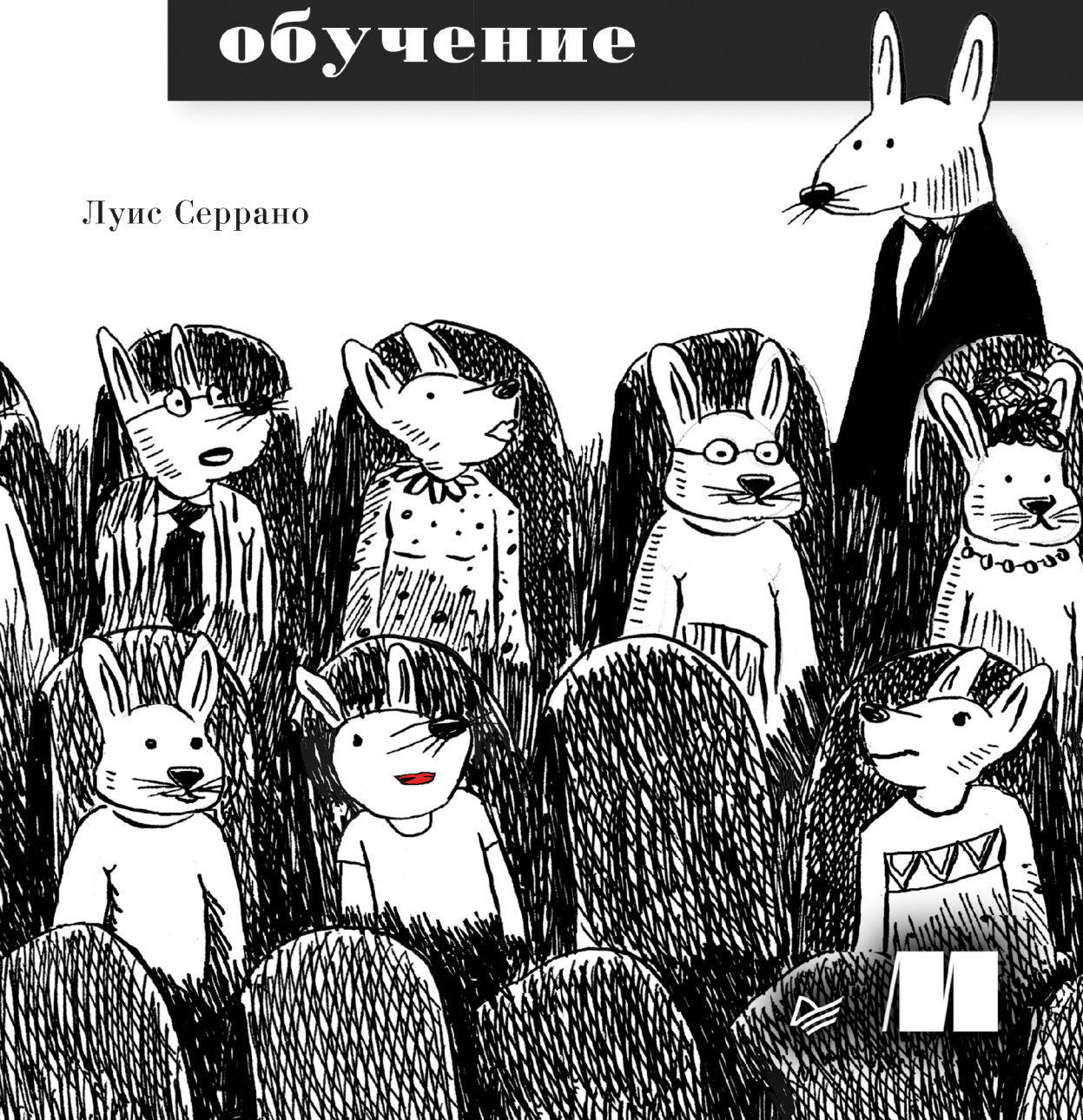


грокаем

Машинное обучение

Луис Сerrано



grokking
Machine
Learning

Luis C. Serrano

Foreword by Sebastian Thrun


MANNING
SHELTER ISLAND

играем

Машинное обучение



Санкт-Петербург • Москва • Минск

2024

ББК 32.813+32.973.23-018
УДК 004.89
С32

Серрано Луис

С32 Грокаем машинное обучение. — СПб.: Питер, 2024. — 512 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-1923-3

Машинное обучение — это набор методов анализа данных, основанных на алгоритмах, которые дают все более точные результаты по мере поступления новых данных. Машинное обучение лежит в основе систем рекомендаций, программ распознавания лиц, «умных» колонок и даже беспилотных автомобилей. Эта уникальная книга объясняет основные понятия машинного обучения на простых и доступных примерах, увлекательных упражнениях и запоминающихся иллюстрациях.

Здесь нет зубодробительного академического жаргона, для понимания объяснений достаточно знаний основ алгебры. По мере чтения вы будете создавать модели для идентификации спама и распознавания изображений и другие интересные проекты на языке Python.

Откройте для себя мощные методы машинного обучения, для понимания и применения которых достаточно знаний математики на уровне средней школы!

Для читателей, знающих основы языка Python. Знаний в области машинного обучения не требуется.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.813+32.973.23-018
УДК 004.89

Права на издание получены по соглашению с Manning Publications.

Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги.

В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др.

Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 9781617295911 англ.
ISBN 978-5-4461-1923-3

© 2021 by Manning Publications Co. All rights reserved
© Перевод на русский язык ООО «Прогресс книга», 2023
© Издание на русском языке, оформление
ООО «Прогресс книга», 2023
© Серия «Библиотека программиста», 2023

Оглавление



Предисловие	11
Вступление	13
Благодарности.....	15
О книге	17
Как организована книга: дорожная карта	17
О коде.....	19
От издательства.....	20
Об авторе.....	21
Глава 1. Что такое машинное обучение? Это здравый смысл, проявляемый компьютером	22
Я с радостью присоединяюсь к вам на пути к знаниям!	23
Машинное обучение повсюду.....	23
Нужен ли большой опыт в математике и программировании, чтобы понять машинное обучение?	24
Так что же такое машинное обучение?	25
Как заставить машины принимать решения с помощью данных? Концепция вспоминания — формулирования — прогнозирования	28
Резюме.....	37
Глава 2. Типы машинного обучения	38
В чем разница между размеченными и неразмеченными данными.....	40
Контролируемое обучение — раздел машинного обучения, который работает с размеченными данными.....	42
Неконтролируемое обучение: раздел машинного обучения, который работает с неразмеченными данными.....	46

6 Грокаем машинное обучение

Что такое обучение с подкреплением.....	56
Резюме.....	58
Упражнения.....	58
Глава 3. Прямая вплотную к точкам. Линейная регрессия.....	60
Задача: спрогнозировать цену дома	62
Решение: построить регрессионную модель цен на жилье.....	63
Как заставить компьютер провести прямую.	
Алгоритм линейной регрессии	70
Как мы измеряем результаты? Функция ошибок	86
Реальное применение. Использование Turi Create для прогнозирования цен на жилье в Индии.....	94
Что делать, если данные не расположены на прямой?	
Полиномиальная регрессия	96
Параметры и гиперпараметры.....	98
Применение регрессии	99
Резюме.....	100
Упражнения.....	101
Глава 4. Оптимизация процесса обучения. Недообучение, переобучение, тестирование и регуляризация	103
Пример недообучения и переобучения с использованием полиномиальной регрессии	106
Как заставить компьютер выбрать правильную модель?	
Тестированием!	108
Где нарушено золотое правило и как это исправить?	
Контрольный набор.....	111
Численный способ определить нужную сложность модели.	
График сложности модели	112
Еще один способ избежать переобучения — регуляризация	114
Полиномиальная регрессия, тестирование и регуляризация с помощью Turi Create	124
Резюме.....	128
Упражнения.....	129
Глава 5. Использование прямых для разделения точек.	
Алгоритм персептрона.....	131
Проблема: мы на чужой планете и не знаем местного языка!	134

Как определить, хорошо ли работает классификатор?	
Функция ошибки	149
Как найти хороший классификатор? Алгоритм персептрона	157
Кодирование алгоритма персептрона	166
Применение алгоритма персептрона	171
Резюме	173
Упражнения	174
Глава 6. Непрерывный подход к разделению точек.	
Логистические классификаторы	176
Логистические классификаторы — непрерывная версия персептронных классификаторов.	178
Как найти хороший логистический классификатор?	
Алгоритм логистической регрессии	190
Кодирование алгоритма логистической регрессии	197
Реальное применение. Классификация обзоров IMDB с помощью Turi Create	201
Классификация по нескольким классам. Логистическая функция со многими переменными	204
Резюме	206
Упражнения	206
Глава 7. Как оценивать классификационные модели?	
Достоверность и ее друзья	208
Достоверность. Как часто модель оказывается права?	210
Как устранить проблему с достоверностью? Определение различных типов ошибок и способы их измерения	211
Полезный инструмент для оценки модели — кривая рабочих характеристик приемника	223
Резюме	235
Упражнения	237
Глава 8. Как использовать вероятность по максимуму.	
Наивная байесовская модель	239
Болен или здоров? История с теоремой Байеса в качестве героя	241
Пример использования — модель спам-фильтра	246
Построение модели спам-фильтра на основе реальных данных	260
Резюме	264
Упражнения	264

Глава 9. Разбиение данных согласно ответам на вопросы.	
Деревья решений.....	267
Задача: необходимо рекомендовать пользователям приложения, которые они, скорее всего, скачают.....	274
Решение: создание системы рекомендации приложений.....	276
Помимо вопросов типа «да/нет».....	292
Графическая граница деревьев решений.....	296
Реальное применение. Моделирование приема студентов с помощью Scikit-Learn.....	300
Деревья решений для регрессии.....	304
Сфера применения.....	308
Резюме.....	309
Упражнения.....	310
Глава 10. Комбинирование ради усиления. Нейронные сети.....	312
Нейронные сети на примере. Более сложная чужая планета.....	314
Обучение нейронных сетей.....	328
Кодирование нейронных сетей в Keras.....	336
Нейронные сети для регрессии.....	344
Другие архитектуры для более сложных наборов данных.....	346
Резюме.....	349
Упражнения.....	350
Глава 11. Нахождение границ со стилем. Машины опорных векторов и метод ядра.....	352
Использование новой функции ошибок для создания более совершенных классификаторов.....	355
Кодирование машин с опорными векторами в Scikit-Learn.....	362
Обучение SVM с нелинейными границами. Метод ядра.....	364
Резюме.....	387
Упражнения.....	388
Глава 12. Комбинирование моделей для достижения максимальных результатов. Ансамблевое обучение.....	390
С небольшой помощью наших друзей.....	392
Бэггинг. Случайное объединение нескольких слабых моделей для создания сильной.....	394

AdaBoost. Разумное объединение слабых алгоритмов для формирования сильного	399
Градиентный бустинг. Использование деревьев решений для формирования сильных обучающих алгоритмов	408
XGBoost — экстремальный метод градиентного бустинга.....	413
Применение методов ансамбля	423
Резюме.....	423
Упражнения.....	424
Глава 13. Применение изученного на практике. Реальный пример разработки данных и машинного обучения	425
Набор данных «Титаник»	427
Очистка набора данных. Отсутствующие значения и как с ними бороться.....	431
Конструирование признаков. Преобразование признаков в наборе данных перед обучением моделей	433
Обучение моделей.....	439
Настройка гиперпараметров для поиска наилучшей модели. Поиск по сетке	444
Применение k-кратной перекрестной проверки для повторного использования данных в качестве обучающих и проверочных	447
Резюме.....	448
Упражнения.....	449
Приложение А. Решения к упражнениям	450
Глава 2. Типы машинного обучения	450
Глава 3. Прямая вплотную к точкам. Линейная регрессия	453
Глава 4. Оптимизация процесса обучения. Недообучение, переобучение, тестирование и регуляризация	456
Глава 5. Использование прямых для разделения точек. Алгоритм персептрона.....	458
Глава 6. Непрерывный подход к разделению точек. Логистические классификаторы	461
Глава 7. Как оценивать классификационные модели? Достоверность и ее друзья.....	463
Глава 8. Как использовать вероятность по максимуму. Наивная байесовская модель	468
Глава 9. Разбиение данных согласно ответам на вопросы. Деревья решений.....	473

10 Грокаем машинное обучение

Глава 10. Комбинирование ради усиления. Нейронные сети	478
Глава 11. Нахождение границ со стилем. Машины опорных векторов и метод ядра	481
Глава 12. Комбинирование моделей для достижения максимальных результатов. Ансамблевое обучение	484
Глава 13. Применение изученного на практике. Реальный пример разработки данных и машинного обучения.	485
Приложение Б. Математика, лежащая в основе градиентного спуска. Спуск с горы с использованием производных и склонов.	486
Использование градиентного спуска для уменьшения функций	487
Использование градиентного спуска для обучения моделей	489
Использование градиентного спуска для регуляризации	502
Застревание на локальных минимумах: как оно происходит и как его обойти	503
Приложение В. Ссылки	505

Предисловие



Правда ли, что машинное обучение — это сложно и его трудно освоить? Конечно же, нет! Просто прочтите эту книгу!

Луис Серрано — настоящий волшебник, когда дело доходит до объяснения простыми словами. Я познакомился с ним, когда он преподавал машинное обучение на Udacity. Он заставлял наших студентов понять, что все машинное обучение так же просто, как сложение или вычитание. И самое главное, он делал материал интересным. Видеоролики, которые он создал для Udacity, были невероятно увлекательными и остаются одними из самых популярных материалов, предлагаемых на платформе.

Но эта книга еще лучше! Даже самые пугливые получают удовольствие от чтения, поскольку Серрано раскрывает некоторые из самых сокровенных секретов сообщества машинного обучения. Он шаг за шагом проведет вас через каждый важный алгоритм и метод в этой области. Вы можете стать фанатом машинного обучения, даже если вам не нравится математика. Серрано минимизирует математическую тарабарщину и вместо этого полагается на интуицию и практические объяснения.

Истинная цель книги — дать вам возможность самостоятельно овладеть этими методами. Так что книга полна забавных упражнений, в которых вы сможете сами опробовать эти загадочные (а теперь и разоблаченные) техники. Что бы вы предпочли — проглотить последнее телешоу Netflix или потратить время на применение машинного обучения к проблемам компьютерного зрения и понимания естественного языка? Если последнее, то эта книга для вас. Я не могу выразить, насколько это увлекательно — играть с новейшими разработками в области машинного обучения и видеть, как компьютер творит чудеса под вашим чутким руководством.

А поскольку машинное обучение едва ли не самая популярная технология последних лет, вы сможете использовать новообретенные навыки и в своей

работе. Несколько лет назад газета New York Times объявила, что в мире насчитывается всего 10 000 экспертов по машинному обучению и миллионы открытых вакансий. Так обстоит дело и сегодня! Изучите эту книгу и станьте профессиональным инженером по машинному обучению. Вы гарантированно овладеете одним из самых востребованных в современном мире навыков.

В этой книге Луис Серрано проделал превосходную работу, объяснив сложные алгоритмы и сделав их доступными практически для всех. Но он сделал это не в ущерб глубине. Вместо этого он сфокусировался на расширении возможностей читателя с помощью цепочки понятных проектов и упражнений. В этом смысле это далеко не пассивное чтение. Чтобы в полной мере извлечь пользу из книги, вам придется потрудиться. У нас в Udacity есть поговорка: вы не похудеете, глядя за тем, как тренируются другие. Чтобы использовать машинное обучение, вы должны научиться применять его для решения реальных задач. Если вы готовы, то эта книга для вас — кем бы вы ни были!

*Себастьян Тран,
PhD, основатель Udacity, адъюнкт-профессор
Стэнфордского университета*

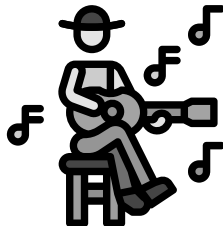
Вступление



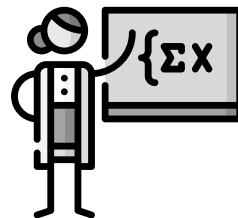
Будущее уже здесь, и у этого будущего есть имя — машинное обучение. Благодаря его применению практически во всех отраслях, от медицины до банковского дела, от запуска беспилотных автомобилей до заказа кофе, интерес к нему растет лавинообразно. Но *что такое* машинное обучение?

Чаще всего, читая книгу или слушая лекцию по машинному обучению, я вижу либо море сложных формул, либо море строк кода. Долго я думал, что это и есть машинное обучение и что оно предназначено только для тех, кто обладает солидными знаниями как в математике, так и в информатике.

Однако позже я начал сравнивать машинное обучение с другими предметами, такими как музыка. Музыкальная теория и практика — сложные предметы. Но когда мы размышляем о музыке, то думаем не о партитурах и гаммах, а о песнях и мелодиях. И тогда я задался вопросом: а не может ли машинное обучение быть тем же самым? Действительно ли это лишь набор формул и кода или за этим есть мелодия?



Музыка



Машинное обучение

Рис. В.1. Музыка — это не только гаммы и ноты. За техническими деталями скрывается мелодия. Точно так же машинное обучение — это не только формулы и код. Есть и мелодия, и в этой книге мы ее исполним

С этой мыслью я и отправился в путешествие с целью понять мелодию машинного обучения. Я месяцами пялился на формулы и код. Нарисовал множество диаграмм. Царапал рисунки на салфетках и показывал их своей семье, друзьям и коллегам. Я обучал модели на небольших и огромных наборах данных. Экспериментировал. Через некоторое время я начал слышать мелодию машинного обучения. Внезапно в моем сознании начали формироваться прекрасные образы. Я начал писать истории, которые соответствуют всем этим концепциям машинного обучения. Мелодии, картинки, истории — вот как мне нравится изучать любую тему, и именно этими мелодиями, картинками и историями я делюсь с вами в книге. Моя цель — сделать машинное обучение на 100 % понятным каждому, и эта книга — шаг в этом путешествии, шаг, который я счастлив делать вместе с вами!

Благодарности



Прежде всего я хотел бы поблагодарить моего редактора Марину Майклс, без которой книга не появилась бы на свет. Благодарю Марджан Бейс, Берта Бейтса и остальных членов команды издательства Manning за их поддержку, профессионализм, отличные идеи и терпение. Благодарю технических корректоров Ширли Яп и Карстена Стребека, редактора по развитию Криса Ати и рецензентов за то, что они дали отличные отзывы и исправили многие мои ошибки. Я благодарю выпускающего редактора Кери Хейлз, редактора Памелу Хант, графического редактора Дженнифер Хоул, корректора Джейсона Эверетта и всю производственную команду за отличную работу по воплощению этой книги в реальность. Благодарю Лауру Монтойю за ее помощь с инклюзивным языком и этикой искусственного интеллекта, Диего Эрнандеса — за ценные дополнения к коду и Кристиана Пикона — за огромную помощь с техническими аспектами репозитория и библиотек.

Я благодарен Себастьяну Трану за прекрасную работу по демократизации образования. Udacity была платформой, которая впервые подарила мне возможность учить мир, и я хотел бы поблагодарить замечательных коллег и студентов, которых там встретил. Алехандро Пердомо и команда Zapata Computing заслуживают благодарности за то, что познакомили меня с миром квантового машинного обучения. Спасибо многим замечательным руководителям и коллегам, с которыми я познакомился в Google и Apple, — они сыграли важную роль в моей карьере. Особая благодарность Роберто Чиприани и команде Paper Inc. за то, что позволили мне стать частью своей семьи, и за превосходную работу, которую они выполняют в образовательном сообществе.

Я хотел бы поблагодарить своих многочисленных университетских преподавателей, которые сформировали мою карьеру и мой образ мышления: Мэри Фальк де Лосада и ее команду на Колумбийских математических олимпиадах, где я полюбил математику и имел возможность встретиться с великими

наставниками и завести дружбу, которая длилась всю жизнь; научного руководителя Сергея Фомина, который сыграл важную роль в моем математическом образовании и стиле преподавания; советника моего учителя Яна Гулдена; Нантель и Франсуа Бержерон, Брюса Сагана и Федерико Ардила, а также многих профессоров и коллег, с которыми я имел возможность работать, в частности, в университетах Ватерлоо, Мичигана, Квебека в Монреале и Йорке, и, наконец, Ричарда Хосино, команду и студентов университета Квест, которые помогли мне протестировать и улучшить материал этой книги.

Спасибо всем рецензентам: Элу Пежевски, Альберту Ногесу Сабатеру, Амиту Ламбе, Биллу Митчеллу, Борко Джурковичу, Даниэле Андреису, Эрику Саперу, Хао Лю, Джереми Р. Лошайдеру, Хуану Габриэлю Боно, Кей Энгельхардт, Кшиштофу Камычеку, Мэтью Марголису, Маттиасу Бушу, Майклу Брайту, Милладу Дагдони, Полине Кесельман, Тони Холдройд и Валери Парэм-Томпсон, ваши предложения помогли сделать эту книгу лучше.

Я хотел бы поблагодарить свою жену Каролину Лассо, которая всегда поддерживала меня; маму Сесилию Эрреру, которая воспитывала меня с любовью и поощряла следовать за своими мечтами; бабушку — она тот ангел, что смотрит на меня с небес; лучшего друга Алехандро Моралеса за то, что всегда был рядом, и друзей, которые осветили мой путь и скрасили мою жизнь. Я благодарю вас и люблю всем сердцем.

YouTube, блоги, подкасты и социальные сети дали мне возможность общаться с тысячами потрясающих людей по всему миру. Любознательные умы с бесконечной страстью к обучению, коллеги-преподаватели, которые щедро делятся своими знаниями и прозрениями, образуют электронное племя, которое вдохновляет меня каждый день и дает силы продолжать преподавать и учиться. Я благодарю всех, кто делится своими знаниями с миром или стремится учиться каждый день.

Я благодарю всех, кто стремится сделать этот мир более справедливым и мирным. Всех, кто борется за справедливость, мир, окружающую среду и за равные возможности для всех людей на Земле независимо от расы, пола, места рождения, условий и выбора, я благодарю от всего сердца.

И последнее, но, конечно, не менее важное: эта книга посвящается тебе, читатель. Ты выбрал путь обучения, путь совершенствования, путь ощущения комфорта в условиях дискомфорта, и это достойно восхищения. Я надеюсь, что эта книга станет важным шагом на пути к твоим мечтам и созданию лучшего мира.



Эта книга научит двум вещам: моделям машинного обучения и тому, как их использовать. Модели машинного обучения бывают разных типов. Некоторые из них возвращают детерминированный ответ, такой как «да» или «нет», а другие — ответ в виде вероятности. Одни из них используют уравнения, другие — операторы `if`. Их общая черта заключается в том, что все они возвращают ответ или прогноз. Отрасль машинного обучения, которая включает в себя модели, возвращающие прогноз, удачно названа *прогностическим машинным обучением*. На этом типе машинного обучения мы и сосредоточимся.

Как организована книга: дорожная карта

Типы глав

Эта книга состоит из глав двух типов. Большинство из них — главы 3, 5, 6, 8–12 — посвящены одному типу моделей машинного обучения. В каждой главе модель подробно разбирается, в том числе рассматриваются примеры, формулы, код и самостоятельные упражнения. А главы 4, 7 и 13 содержат полезные методы, которые можно использовать для обучения, оценки и улучшения моделей машинного обучения. В частности, в главе 13 сквозной пример исследуется на реальном наборе данных, здесь вы сможете применить все знания, полученные в предыдущих главах.

Рекомендуемые пути обучения

Вы можете работать с книгой двумя способами. Я рекомендую знакомиться с ней последовательно, глава за главой, потому что такое чередование моделей

и методов их обучения весьма полезно. Другой путь заключается в том, чтобы сначала изучить все модели (главы 3, 5, 6, 8–12), а затем — методы их обучения (главы 4, 7 и 13). И, конечно же, поскольку все мы учимся по-разному, вы можете следовать собственному пути!

Приложения

Книга имеет три приложения. В приложении А опубликованы решения упражнений, описанных в каждой главе. Приложение Б содержит некоторые полезные формальные математические выводы, более технически сложные, чем остальная книга. Приложение В — это список литературы и ресурсов, с которыми я рекомендую ознакомиться, если вы хотите углубить свое понимание темы.

Требования и цели обучения

Книга даст вам прочную основу в области прогностического машинного обучения. Чтобы извлечь из нее максимальную пользу, вы должны обладать визуальным мышлением и хорошо разбираться в элементарной математике. Будет полезно (хотя и не обязательно) знать, как писать код, особенно на Python, потому что на протяжении всей книги вам предоставляется возможность реализовать и применить несколько моделей в реальных наборах данных. Прочитав эту книгу, вы сможете сделать следующее.

- Описать наиболее важные модели в прогностическом машинном обучении и то, как они работают, включая линейную и логистическую регрессию, наивный байесовский алгоритм, деревья решений, нейронные сети, машины опорных векторов и методы ансамблей.
- Определить их сильные и слабые стороны и параметры, которые они применяют.
- Определить, как эти модели используются в реальном мире, и сформулировать потенциальные способы применения машинного обучения к любой конкретной задаче, которую вы хотели бы решить.
- Узнать, как оптимизировать, сравнивать и улучшать эти модели, чтобы создавать лучшие из возможных моделей машинного обучения.
- Кодировать модели вручную или с помощью существующего пакета и использовать их для прогнозирования реальных наборов данных.

Если у вас на примете есть конкретный набор данных или проблема, я приглашаю вас подумать о том, как применить к ним то, что вы узнаете из этой книги, и использовать ее в качестве отправной точки для внедрения и экспериментов с вашими собственными моделями.

Я очень рад начать это путешествие вместе с вами и надеюсь, что вы рады не меньше!

Другие ресурсы

Данная книга самодостаточна. Это означает, что помимо требований, описанных ранее, в ней представлены все необходимые нам концепции. Тем не менее я включаю много ссылок, которые пригодятся, если вы захотите понять концепции более глубоко или изучить дополнительные темы. Все ссылки размещены в приложении В, а также здесь: <http://serrano.academy/grokking-machine-learning>.

Некоторые из моих собственных ресурсов тоже дополняют материалы этой книги. На странице в <http://serrano.academy> вы найдете множество материалов в форме видео, постов и кода. Видео есть и на моем канале YouTube www.youtube.com/c/LuisSerrano, с которым я рекомендую ознакомиться. И к большинству глав этой книги прилагается видео, которое стоит посмотреть во время чтения.

Мы будем писать код

Мы будем писать код на Python. Однако, если вы планируете изучать предложенные концепции без кода, можете игнорировать его. Тем не менее я рекомендую вам хотя бы взглянуть на него в ознакомительных целях.

Книга поставляется с репозиторием кода, и большинство глав дадут вам возможность кодировать алгоритмы с нуля или использовать некоторые распространенные пакеты Python для построения моделей, соответствующих заданному набору данных. Репозиторий GitHub — это www.github.com/luisguiserrano/mapping, и я указываю соответствующие блокноты по всей книге. В README репозитория вы найдете инструкции по установке пакетов для успешного запуска кода.

Основными пакетами Python, которые мы используем в книге, являются:

- NumPy — для хранения массивов и выполнения сложных математических вычислений;
- Pandas — для хранения, обработки и анализа больших наборов данных;
- Matplotlib — для построения графиков данных;
- Turi Create — для хранения и обработки данных, а также обучения моделей;
- Scikit-Learn — для обучения моделей машинного обучения;
- Keras (TensorFlow) — для обучения нейронных сетей.

О коде

Эта книга содержит много примеров исходного кода, внедренных в обычный текст. В обоих случаях исходный код форматируется таким **моноширинным шрифтом**, чтобы отделить его от обычного текста. Иногда код также обозначен **жирным шрифтом**, чтобы выделить код, который изменился по сравнению с предыдущими шагами в главе, например, когда новая функция добавляется к уже существующей строке кода.

Во многих случаях исходный код был переформатирован: мы добавили разрывы строк и переработали отступы, чтобы более полно использовать доступное пространство на страницах книги. Кроме того, комментарии, имеющиеся в исходном коде, часто удалялись, если код описывался в тексте. Многие листинги сопровождаются аннотациями к коду, уточняющими важные концепции.

Код примеров, приводимых в книге, доступен для скачивания на веб-сайте Manning (<https://www.manning.com/books/grokking-machine-learning>), а также с GitHub на www.github.com/luiguiserrano/manning.

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция). Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

Об авторе



Луис Дж. Серрано — ученый-исследователь, работающий в области квантового искусственного интеллекта в Zapata Computing. Ранее он был инженером по машинному обучению в Google, ведущим преподавателем искусственного интеллекта в Apple и руководителем отдела контента в области искусственного интеллекта и науки о данных в Udacity. Луис получил докторскую степень по математике в Мичиганском университете, степень бакалавра и магистра математики в университете Ватерлоо и после этого работал исследователем в лаборатории комбинаторной и информационной математики Квебекского университета в Монреале. Луис ведет популярный YouTube-канал о машинном обучении с более чем 85 000 подписчиков и более чем 4 млн просмотров, часто выступает с докладами на конференциях по искусственному интеллекту и науке о данных.

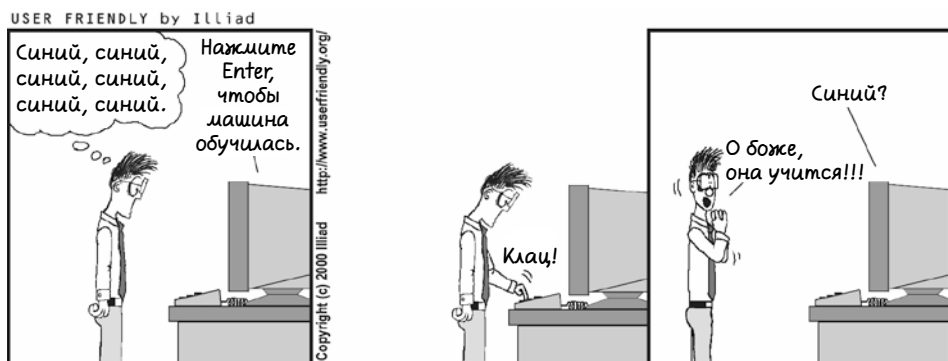
1

Что такое машинное обучение? Это здравый смысл, проявляемый компьютером



В этой главе

- ✓ Что такое машинное обучение.
 - ✓ Сложно ли машинное обучение (спойлер: нет).
 - ✓ Чему мы научимся в этой книге.
 - ✓ Что такое искусственный интеллект и чем он отличается от машинного обучения.
 - ✓ Как мыслят люди и как можно внедрить эти идеи в машину.
 - ✓ Некоторые базовые примеры машинного обучения из реальной жизни.
-



Я с радостью присоединяюсь к вам на пути к знаниям!

Приветствую вас на страницах этой книги! Я искренне рад присоединиться к вам на пути к пониманию машинного обучения. На высоком уровне машинное обучение — это процесс, в ходе которого компьютер решает проблемы и принимает решения. Почти как человек.

Через эту книгу я хочу донести до вас одну мысль: машинное обучение — это просто! Чтобы разобраться с ним, не нужно быть великим математиком или программистом. Базовая математика, конечно, понадобится, но основными составляющими послужат здравый смысл, хорошая визуальная интуиция и желание изучать и применять указанные методы ко всему, чем вы увлечены, и в области, где хотите что-то улучшить. Я получил огромное удовольствие от написания книги, так как мне понравилось углубляться в данную тему, и я надеюсь, что вы получите огромное удовольствие, прочитав ее и погрузившись в машинное обучение!

Машинное обучение повсюду

Машинное обучение можно встретить везде. Это утверждение с каждым днем кажется все более верным. Мне трудно представить себе хоть один аспект жизни, который нельзя было бы так или иначе улучшить с помощью машинного обучения. В любом деле, требующем повторения, анализа данных и сбора выходных данных, вам поможет машинное обучение. За последние несколько лет благодаря достижениям в области вычислительных мощностей и повсеместному сбору данных в сфере машинного обучения наблюдался поистине взрывной рост. Назову лишь несколько областей применения машинного обучения: системы рекомендаций, распознавание изображений, обработка текста, автопилот в автомобилях, выявление спама, постановка медицинских диагнозов... Список можно продолжать бесконечно. Возможно, у вас есть задача или область,

в которой требуется ощутимый прогресс. Весьма вероятно, что машинное обучение вполне применимо в этой области — и именно это привело вас к этой книге. Давайте вместе с этим разберемся!

Нужен ли большой опыт в математике и программировании, чтобы понять машинное обучение?

Нет. Машинное обучение требует воображения, творческого подхода и визуального мышления. Оно заключается в том, чтобы улавливать закономерности, которые появляются в мире, и использовать их для прогнозирования событий будущего. Если вам нравится находить закономерности и выявлять корреляции, тогда машинное обучение — ваша область. Если бы я сказал вам, что бросил курить, стал есть больше овощей и занялся спортом, то что, согласно вашему прогнозу, произойдет с моим здоровьем через год? Возможно, оно улучшится. Если бы я сказал вам, что перехожу с красных свитеров на зеленые, то что, по-вашему, случится с моим здоровьем через год? Скорее всего, мало что изменится (может, случатся грандиозные перемены, но не из-за того, о чем я сообщил). Выявление таких корреляций и закономерностей — вот что такое машинное обучение. Единственное различие машинного обучения состоит в том, что здесь мы привязываем к этим шаблонам формулы и цифры, чтобы компьютеры могли их распознавать.

Для изучения машинного обучения необходимы некоторые знания математики и программирования, но экспертом быть вовсе не обязательно. Если вы *уже* эксперт в любой из этих областей (или даже в обеих), то эти навыки наверняка вам пригодятся. Но если пока это не так, вы все равно сможете изучить машинное обучение и восполнить пробелы в математике и программировании по ходу дела. В этой книге мы представим все необходимые математические концепции в тот момент, когда они нам понадобятся. Если говорить о кодировании, то объем кода, который потребуется в машинном обучении, зависит только от вас. В машинном обучении есть задания как для тех, кто программирует весь день напролет, так и для тех, кто не программирует вообще. Многие пакеты, API и инструменты помогают нам реализовать машинное обучение с минимальным объемом кода. С каждым днем машинное обучение становится все более общедоступным, и я рад, что вы не остались в стороне!

Формулы и код — это весело, если их рассматривать как язык

В большинстве книг по машинному обучению алгоритмы объясняются математически с использованием формул, производных и т. д. Хотя такие точные описания методов хорошо работают на практике, сама по себе формула может быть скорее запутанной, чем иллюстративной. Однако, подобно музыкальной партитуре, за запутанностью формулы может скрываться чудесная мелодия.

Например, взглянем на формулу $\sum_{i=1}^4 i$. На первый взгляд она выглядит некрасиво, но представляет собой простейшее суммирование, а именно $1 + 2 + 3 + 4$.

А как насчет $\sum_{i=1}^n w_i$? Это просто сумма множества (n) чисел. Но когда я думаю о сумме многих чисел, то представляю что-то вроде $3 + 2 + 4 + 27$, а не $\sum_{i=1}^n w_i$. Всякий раз, когда я вижу формулу, мне сразу же нужен небольшой пример, и тогда картина в моем сознании проясняется. Что приходит на ум, когда я вижу что-то вроде $P(A|B)$? Это условная вероятность, поэтому я думаю о чем-то вроде: «Вероятность того, что событие A произойдет, с учетом того, что другое событие, B , уже произошло». Например, если A представляет собой сегодняшний дождь, а B — жизнь в тропическом лесу Амазонии, то формула $P(A|B) = 0,8$ попросту означает: «Вероятность того, что сегодня пойдет дождь, учитывая, что мы живем в тропическом лесу Амазонии, составляет 80 %».

Если вы действительно любите формулы, не волнуйтесь — в книге их достаточно. Но появляться они будут сразу после примера, который их иллюстрирует.

То же самое будет и с кодом. Если мы смотрим на код со стороны, он может казаться сложным, и бывает трудно поверить, что кто-то может уместить все это в своей голове. Однако код — это лишь последовательность шагов, и, как правило, каждый из них прост. В этой книге мы будем писать код, но разобьем его на простые шаги, каждый из которых подробно объясним на примерах либо с помощью иллюстраций. В первых нескольких главах мы будем кодировать свои модели с нуля, чтобы лучше понимать, как они работают. Однако в последующих главах модели усложнятся. Для них мы станем использовать такие пакеты, как Scikit-Learn, Turi Create или Keras, в которых точно и эффективно реализовано большинство алгоритмов машинного обучения.

Так что же такое машинное обучение?

Чтобы дать определение машинному обучению, сначала определим более общий термин — искусственный интеллект (ИИ).

Что такое искусственный интеллект

Это общий термин, который мы определяем так.

ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ — набор всех задач, в которых компьютер может принимать решения.

Во многих случаях компьютер принимает эти решения, имитируя принятие решения человеком. Также он может имитировать эволюционные, генетические или физические процессы. Но в целом каждый раз, когда мы видим, что компьютер решает проблему самостоятельно, будь то вождение автомобиля, поиск маршрута между двумя точками, постановка диагноза пациенту или рекомендация фильма, мы наблюдаем работу искусственного интеллекта.

Что такое машинное обучение

Машинное обучение похоже на искусственный интеллект, и часто их определения путают.

Машинное обучение (МО) является частью искусственного интеллекта, определяют его следующим образом.

МАШИННОЕ ОБУЧЕНИЕ — набор всех задач, в которых компьютер может принимать решения на *основе данных*.

Что это значит? Позвольте проиллюстрировать на рис. 1.1.



Рис. 1.1. Машинное обучение — это область искусственного интеллекта

Вернемся к рассмотрению того, как именно люди принимают решения. В общих чертах мы принимаем решения двумя путями:

- с помощью логики и рассуждений;
- задействуя собственный опыт.

Представьте, что мы принимаем решение о покупке автомобиля. Можно внимательно изучить его характеристики, такие как цена, расход топлива и навигация, после чего попытаться определить наилучшую комбинацию, которая соответствует нашему бюджету. Это использование логики и принятие разумного решения. Если же вместо этого мы спросим всех своих друзей, какими автомобилями они владеют и что им в них нравится или не нравится, составим список аргументов за и против, после чего с его помощью примем решение, то это будет применением опыта — в данном случае опыта друзей.

Машинное обучение представляет собой второй метод — принятие решений с использованием своего опыта. На компьютерном языке термин, обозначающий *опыт*, звучит как *данные*. Следовательно, при машинном обучении компьютеры принимают решения на основе данных. Таким образом, каждый раз, когда мы заставляем компьютер решать задачи или принимать решения с помощью только данных, мы занимаемся машинным обучением. Простыми словами машинное обучение можно описать так — это рассудительность, за исключением того факта, что проявляет ее компьютер.

Переход от решения проблем с использованием любых необходимых средств к решению проблем с применением только данных может показаться небольшим шагом для компьютера, но это огромный шаг для человечества (рис. 1.2). Когда-то давно, если мы хотели заставить компьютер выполнять какую-то задачу, нам нужно было писать программу, а именно целый набор инструкций, которым компьютер должен следовать. Этот процесс хорош для простых задач, но некоторые из них слишком сложны для подобной концепции. Например, рассмотрим задачу по определению того, есть ли на изображении яблоко. Начав писать компьютерную программу для решения этой задачи, мы быстро поймем, что это крайне сложно.

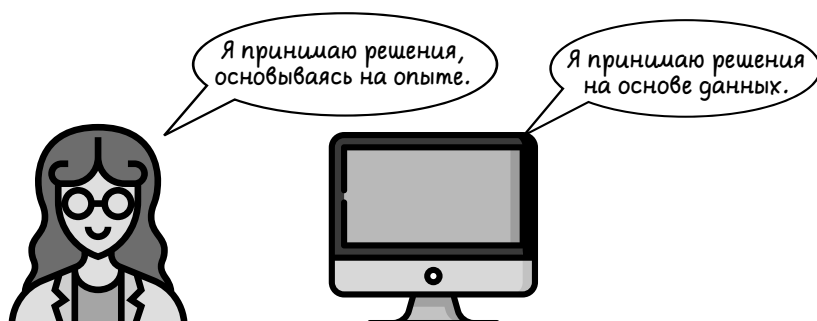


Рис. 1.2. Машинное обучение охватывает все задачи, в которых компьютеры принимают решения на основе данных

Давайте сделаем шаг назад и зададим себе вопрос: как мы, люди, узнали, как выглядит яблоко? Большинство слов мы выучили не благодаря тому, что кто-то объяснил нам, что они означают, а путем повторения. В детстве мы видели много предметов, и взрослые сообщали нам, что это за предметы. Прежде чем узнать, что такое яблоко, мы видели множество яблок на протяжении многих лет и слышали слово «яблоко», пока однажды в голове что-то не щелкнуло и мы не поняли, что такое яблоко. Это как раз то, что мы заставляем делать компьютер в рамках машинного обучения. Мы демонстрируем ему множество изображений и сообщаем, какие из них содержат яблоко (что составляет наши данные). Мы повторяем этот процесс до тех пор, пока компьютер не начнет улавливать верные формы и атрибуты, которые и составляют яблоко. В конце процесса, когда мы передаем компьютеру новое изображение, он уже может использовать эти шаблоны, чтобы определить, содержит ли изображение яблоко. То есть точно так же, как люди принимают решения на основе предыдущего опыта, компьютеры могут принимать решения на основе предыдущих данных. Конечно, нам все равно нужно запрограммировать компьютер так, чтобы он мог улавливать эти закономерности. В этом поможет ряд приемов, которым мы обучимся в книге.

Ну и раз уж мы сделали первый шаг — что такое глубокое обучение?

Точно так же, как машинное обучение является частью искусственного интеллекта, глубокое обучение — часть машинного обучения. В предыдущем разделе мы узнали, что в нашем распоряжении есть несколько методов, чтобы заставить компьютер учиться на основе данных. Один из этих методов работает настолько хорошо, что обзавелся собственной областью исследований, называемой *глубоким обучением* (ГО) (рис. 1.3), которую мы определяем следующим образом.

ГЛУБОКОЕ ОБУЧЕНИЕ — область машинного обучения, использующая объекты, называемые *нейронными сетями*.



Рис. 1.3. Глубокое обучение является частью машинного обучения

Что такое нейронные сети? О них мы узнаем в главе 10. Глубокое обучение — это, возможно, наиболее часто используемый тип машинного обучения, потому что он действительно хорошо работает. Видя какое-то из передовых приложений, таких как распознавание изображений, генерация текста, игра в го или беспилотные автомобили, скорее всего, мы в той или иной степени наблюдаем глубокое обучение.

Другими словами, глубокое обучение — это часть машинного обучения, которое, в свою очередь, является частью искусственного интеллекта. Если бы эта книга была о транспорте, то ИИ был бы совокупностью транспортных средств, МО — автомобилями, а ГО — Ferrari.

Как заставить машины принимать решения с помощью данных? Концепция вспоминания — формулирования — прогнозирования

В предыдущем разделе говорилось, что машинное обучение состоит из набора методов, которые мы используем, чтобы заставить компьютер принимать

решения на основе данных. В этом разделе узнаем, что подразумевается под принятием решений на основе данных и как работают некоторые из таких методов. Для этого еще раз проанализируем процесс, с помощью которого люди принимают решения, основанные на опыте. Это то, что называется *концепцией воспоминания — формулирования — прогнозирования* (рис. 1.4). Составляющие ее шаги сформулированы в следующем разделе. Цель машинного обучения состоит в том, чтобы научить компьютеры мыслить одинаково, следуя одним и тем же принципам.



Рис. 1.4. Вспоминание — формулирование — прогнозирование — основная концепция, используемая в книге

Как мыслят люди?

Когда нам, людям, нужно принять решение, основанное на собственном опыте, мы обычно задействуем следующую концепцию.

1. **Вспоминаем** похожие ситуации в прошлом.
2. **Формулируем** общее правило.
3. Используем его, чтобы **спрогнозировать** то, что может произойти в будущем.

Например, если вопрос звучит так: «Будет ли сегодня дождь?», то процесс предсказания состоит в следующем.

1. Мы **вспоминаем**, что на прошлой неделе большую часть времени шел дождь.
2. Мы **формулируем** правило о том, что в этом месте большую часть времени идет дождь.
3. И **прогнозируем**, что сегодня будет дождь.

Мы можем быть правы или неправы, но по крайней мере пытаемся сделать максимально точный прогноз, основываясь на имеющейся информации.

Некоторые лингвомодели и алгоритмы машинного обучения

Прежде чем обратиться к дополнительным примерам, иллюстрирующим методы, используемые в машинном обучении, дадим определения кое-каких полезных терминов, которые будем применять на протяжении всей книги. Мы знаем, что при машинном обучении заставляем компьютер учиться решать проблему с помощью данных, используя их для построения *модели*. Что такое модель? Определим ее следующим образом.

МОДЕЛЬ — набор правил, которые представляют наши данные и могут быть задействованы для прогнозирования.

Мы можем думать о модели как о представлении реальности, использующем набор правил, максимально точно имитирующих существующие данные. В примере с дождем в предыдущем разделе нашим представлением реальности была модель, описывающая мир, где большую часть времени идет дождь. Это простой мир с одним правилом: большую часть времени идет дождь. Представление может быть точным, а может — нет, но, согласно нашим данным, это наиболее точное представление реальности, которое мы способны сформулировать. Позже мы применяем это правило для составления прогнозов на основе неявных данных.

Алгоритм — это процесс, который использовался для построения модели. В текущем примере он прост: мы посмотрели, сколько дней шел дождь, и поняли, что большинство. Конечно, алгоритмы машинного обучения могут быть намного сложнее, но в итоге они всегда состоят из набора шагов. Наше определение алгоритма таково.

АЛГОРИТМ — процедура или набор шагов, используемых для решения задачи или выполнения вычисления. Здесь цель алгоритма — построение модели.

Короче говоря, модель — это то, что мы применяем для прогнозирования, а алгоритм — то, с помощью чего строим модель. Эти два определения легко спутать, и часто они взаимозаменяемы, так что для ясности рассмотрим несколько примеров.

Примеры используемых людьми моделей

В этом разделе мы сосредоточимся на распространенном применении машинного обучения — обнаружении спама. В следующих примерах будем выявлять как спам, так и письма, им не являющиеся. Письма, не являющиеся спамом, называются также *полезными*¹.

СПАМ И ПОЛЕЗНЫЕ ПИСЬМА

Спам — это общий термин для обозначения ненужной или нежелательной электронной почты, такой как письма-рассылки, рекламные акции и пр.

¹ В оригинале ham — антоним к слову spam. — *Примеч. пер.*

Термин берет начало в скетче Монти Пайтона 1972 года, в котором каждый пункт меню ресторана содержал спам в качестве ингредиента. Разработчики используют термин «ветчина» (ham) для обозначения электронных писем, не являющихся спамом.

Пример 1. Раздражающий друг

В этом примере мы встречаемся со своим другом Бобом, любителем отправлять электронные письма. Многие его письма — это спам в виде писем счастья. Они уже начинают раздражать. Сегодня суббота, и мы только что получили очередное уведомление по электронной почте от Боба. Можно ли угадать, спам ли это письмо, не глядя на него?

Попробуем использовать метод вспоминания — формулирования — прогнозирования. Во-первых, **вспомним**, скажем, последние десять электронных писем, полученных от Боба. Это наши данные. Мы помним, что шесть из них были спамом, а остальные четыре — полезными. Исходя из этой информации, можем **сформулировать** следующую модель.

Модель 1. Шесть из каждых десяти электронных писем от Боба — спам.

Это правило станет нашей моделью. Обратите внимание на то, что правило не обязательно должно быть истинным. Более того, оно может быть бессовестно неверным. Но, учитывая наши данные, это лучшее, что мы можем придумать, так что придется работать с тем, что есть. Далее в этой книге мы узнаем, как оценивать модели и улучшать их, когда это необходимо.

Теперь, когда у нас есть правило, можем использовать его, чтобы **спрогнозировать**, является ли электронное письмо спамом. Если шесть из десяти писем Боба — спам, то можно предположить, что новое электронное письмо с вероятностью 60 % будет спамом и с вероятностью 40 % — нет. Согласно этому правилу, с несколько большей вероятностью электронное письмо является спамом. Таким образом, прогнозируем, что электронное письмо — спам (рис. 1.5).



Рис. 1.5. Очень простая модель машинного обучения

Повторюсь, наше предсказание может оказаться неверным. Мы можем открыть электронное письмо и понять, что оно все-таки полезное. Но мы сделали прогноз на основе *известных данных*. Вот в чем суть машинного обучения! Возможно, вы уже думаете о том, можно ли внести в процесс какие-то улучшения. Мы оцениваем каждое электронное письмо от Боба сходным образом, но можно ли получить больше информации, которая поможет отделить спам? Попробуем немного глубже проанализировать эти электронные письма. Например, можно выяснить, когда именно Боб отправлял их, и попробовать выявить закономерность.

Пример 2. Раздражающий друг по расписанию

Давайте более внимательно рассмотрим электронные письма, которые Боб отправлял нам месяцем ранее. Точнее, посмотрим, в какой именно день он это делал. Вот электронные письма с названием дня недели и информацией о том, являются ли они спамом:

- понедельник — полезное;
- вторник — полезное;
- суббота — спам;
- воскресенье — спам;
- воскресенье — спам;
- среда — полезное;
- пятница — полезное;
- суббота — спам;
- вторник — полезное;
- четверг — полезное.

Все заиграло новыми красками, не так ли? Закономерность налицо! Похоже, что каждое электронное письмо, отправленное Бобом в течение недели, полезное, а отправленное в выходные — спам. В этом есть логика: возможно, в течение недели он отправляет нам рабочую электронную почту, а в выходные у него есть время для рассылки спама, и он решает ни в чем себе не отказывать. Итак, мы можем **сформулировать** более обоснованное правило или модель.

Модель 2. Каждое электронное письмо, которое Боб отправляет в течение недели, — это полезное письмо, а отправленное в выходные — спам.

Теперь посмотрим, какой сегодня день. Если воскресенье и мы только что получили электронное письмо от Боба, то с большой долей уверенности можем **спрогнозировать**, что это спам (рис. 1.6). Сделав прогноз, сразу отправляем его в мусорную корзину и продолжаем наш день.



Рис. 1.6. Несколько более сложная модель машинного обучения

Пример 3. Все усложняется!

Теперь предположим, что мы продолжаем следовать этому правилу и однажды встречаем Боба на улице. Он спрашивает: «Почему ты не пришел на мой день рождения?» Мы понятия не имеем, о чем он говорит. Оказывается, в прошлое воскресенье он рассылал приглашения на свой день рождения, а мы его пропустили! Почему? Потому что он отправил письмо в выходной и мы предположили, что это спам. Похоже, нам требуется модель получше. Давайте вернемся к просмотру электронных писем Боба — тому шагу, когда мы **вспоминаем**, — и выясним, нет ли там других закономерностей:

- 1 Кбайт — полезное;
- 2 Кбайт — полезное;
- 16 Кбайт — спам;
- 20 Кбайт — спам;
- 18 Кбайт — спам;
- 3 Кбайт — полезное;
- 5 Кбайт — полезное;
- 25 Кбайт — спам;
- 1 Кбайт — полезное;
- 3 Кбайт — полезное.

Что же мы видим? Похоже, что объемные электронные письма обычно являются спамом, в то время как более мелкие — как правило, нет. Это вполне логично, потому что спам-письма часто содержат большие вложения.

Итак, мы можем **сформулировать** следующее правило.

Модель 3. Любое электронное письмо размером 10 Кбайт или больше — это спам, а менее 10 Кбайт — нет.

Теперь, когда правило сформулировано, можем сделать **прогноз**. Мы смотрим на электронное письмо, которое получили сегодня от Боба, видим, что его размер составляет 19 Кбайт, и закономерно приходим к выводу — это спам (рис. 1.7).



Рис. 1.7. Еще одна чуть более сложная модель машинного обучения

Ну что, задача *решена*? Как бы не так!

Прежде всего обратите внимание на то, что для составления прогнозов мы использовали день недели и размер электронного письма. Это примеры *признаков* — одного из самых важных понятий в книге.

ПРИЗНАК — любое свойство или характеристика данных, которые модель может применять для прогнозирования.

Легко догадаться, что существует еще множество признаков, которые могут указывать на то, является электронное письмо спамом или нет. Есть идеи? В следующих разделах мы выявим еще несколько.

Пример 4. Еще?

Наши два классификатора вполне рабочие, потому что исключают большие электронные письма и электронные письма, отправленные в выходные дни. Каждый из них использует только один из этих двух признаков. Но что, если нам нужно правило, которое работало бы с обоими признаками? Вполне допустимы правила, подобные следующим.

Модель 4. Если размер электронного письма более 10 Кбайт или оно было отправлено в выходные дни, то оно классифицируется как спам. В противном случае — как полезное.

Модель 5. Если электронное письмо отправлено в течение недели, то оно должно быть больше 15 Кбайт, чтобы попасть в категорию спама. Если оно отправлено в выходные дни, то должно быть больше 5 Кбайт, чтобы попасть в категорию спама. В противном случае оно классифицируется как полезное.

Можно усложнять и дальше.

Модель 6. Присвоим дням недели порядковые номера: понедельник — 0, вторник — 1, среда — 2, четверг — 3, пятница — 4, суббота — 5, воскресенье — 6. Если мы сложим номер дня и размер электронного письма (в килобайтах) и в результате получим 12 или более, то электронное письмо классифицируется как спам (рис. 1.8). В противном случае — как полезное.



Рис. 1.8. Еще более сложная модель машинного обучения

Все это рабочие модели. Мы можем продолжать создавать все больше и больше моделей, добавляя уровни сложности или рассматривая все новые признаки. Теперь вопрос в том, какая же модель будет лучшей. Вот тут-то нам и понадобится помощь компьютера.

Примеры моделей, используемых машинами

Наша задача заключается в том, чтобы заставить компьютер думать так, как думаем мы, а именно применять структуру воспоминания — формулирования — прогнозирования. Если упростить, то вот что должен сделать компьютер на каждом из этапов.

- **Вспомнить** — обратиться к огромной таблице данных.
- **Сформулировать** — создать модели, просматривая множество правил и формул, и проверить, какая модель лучше всего соответствует данным.
- **Спрогнозировать** — использовать эту модель для прогнозирования будущих данных.

Этот процесс немногим отличается от того, что мы проделывали в предыдущем разделе. Улучшением здесь является то, что компьютер может быстро создавать модели, просматривая множество формул и комбинаций правил до тех пор, пока не обнаружит то, что хорошо соответствует имеющимся данным. Например, мы можем создать классификатор спама с такими признаками, как

отправитель, дата и время суток, количество слов, количество орфографических ошибок и появление определенных слов, таких как «купить» или «выиграть». Модель вполне может выглядеть как следующее логическое утверждение.

Модель 7

- Если в электронном письме есть две орфографические ошибки или более, то оно классифицируется как спам.
- Если оно содержит вложение размером более 10 Кбайт, оно классифицируется как спам.
- Если отправителя нет в нашем списке контактов, письмо классифицируется как спам.
- Если в нем есть слова «купи» и «выиграй», оно классифицируется как спам.
- В противном случае оно классифицируется как полезное.

Все это можно записать следующей формулой.

Модель 8. Если (размер) + 10 (количество орфографических ошибок) – (количество появлений слова «мама») + 4 (количество появлений слова «купить») > 10, то мы классифицируем сообщение как спам (рис. 1.9). В противном случае классифицируем его как «ветчину».

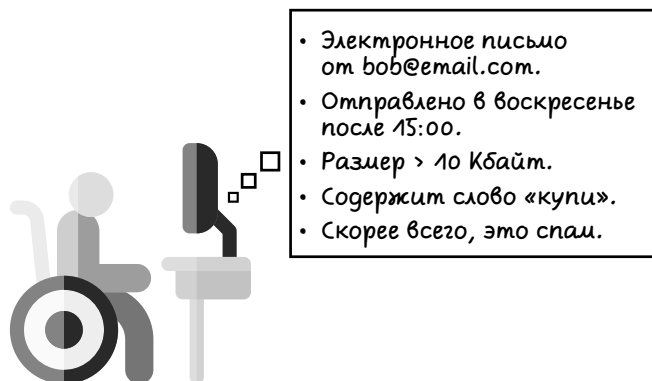


Рис. 1.9. Гораздо более сложная модель машинного обучения, выработанная компьютером

Теперь вопрос: какое правило лучшее? Быстрый ответ: то, которое лучше всего соответствует данным. А настоящий ответ: то, которое лучше всего обобщает новые данные. В конце концов, мы можем прийти к сложному правилу, но компьютер способен сформулировать его и использовать для быстрого принятия решений. Следующий вопрос: как создать наилучшую модель? Именно об этом и пойдет речь в книге.

Резюме

- Машинное обучение — это просто! Любой желающий может изучить и применять его независимо от своего опыта. Все, что нужно, — это желание учиться и отличные идеи для реализации!
- Машинное обучение чрезвычайно полезно, поэтому используется в большинстве дисциплин. В науке, технологиях, социальной сфере и медицине оно оказывает значимое влияние и будет оказывать его впредь.
- Машинное обучение — это здравый смысл, реализуемый компьютером. Оно имитирует мышление людей, что позволяет принимать решения быстро и точно.
- Точно так же, как люди принимают решения на основе опыта, компьютеры способны принимать решения на основе предыдущих данных. Вот в чем суть машинного обучения!
- Машинное обучение использует структуру воспоминания — формулирования — прогнозирования следующим образом:
 - **вспомнить** — обратиться к предыдущим данным;
 - **сформулировать** — построить модель или правило на основе этих данных;
 - **спрогнозировать** — применить эту модель для прогнозирования будущих данных.

**В этой главе**

- ✓ Три различных типа машинного обучения: контролируемое, неконтролируемое и обучение с подкреплением.
 - ✓ Разница между размеченными и неразмеченными данными.
 - ✓ Разница между регрессией и классификацией, а также их использованием.
-



Как мы узнали из главы 1, машинное обучение — это здравый смысл для компьютера. Принимая решения на основе предыдущих данных, он примерно имитирует процесс, посредством которого люди принимают решения на основе опыта. Естественно, программирование компьютеров для имитации процесса человеческого мышления — сложная задача, потому что компьютеры созданы для хранения и обработки чисел, а не для принятия решений. Ее и стремится решить машинное обучение. Оно делится на несколько ветвей в зависимости от типа принимаемого решения. В этой главе мы рассмотрим некоторые наиболее важные из них.

Машинное обучение нашло применение во многих областях, например таких, как:

- прогнозирование цен на жилье на основе размера дома, количества комнат и местоположения;
- прогнозирование сегодняшних цен на фондовом рынке на основе вчерашних цен и других рыночных факторов;
- обнаружение спама на основе слов, употребленных в электронном письме, и данных отправителя;
- распознавание изображений, например лиц или животных, на основе составляющих их пикселей;
- обработка длинных текстовых документов и создание их резюме;
- рекомендации пользователю видеороликов или фильмов, например, на YouTube или Netflix;
- создание чат-ботов, которые взаимодействуют с людьми и отвечают на вопросы;
- обучение автомобилей с автопилотом самостоятельно двигаться по городу;
- постановка диагноза людям и разделение их на больных и здоровых;
- сегментация рынка на аналогичные группы в зависимости от местоположения, покупательской способности и интересов;
- игры, подобные шахматам или го.

Попробуйте представить, как можно использовать машинное обучение в каждой из этих областей. Обратите внимание на то, что некоторые из этих применений отличаются друг от друга, но задачи могут решать аналогичные. Например, прогнозировать цены на жилье и на акции можно одинаковыми методами. Прогноз того, является ли электронное письмо спамом или является ли транзакция по кредитной карте мошеннической, также можно сделать с помощью сходных методов. А как насчет группировки пользователей приложения на основе их сходства? Это звучит иначе, чем прогнозирование цен на жилье, но выполнить это можно аналогично группировке газетных статей по темам. А как насчет игры в шахматы? Не похоже на все предыдущие приложения, но зато может иметь сходство с игрой в го.

Модели машинного обучения подразделяются на типы в зависимости от того, как они работают.

Итак, три основных семейства моделей машинного обучения:

- контролируемое обучение;
- неконтролируемое обучение;
- обучение с подкреплением.

В этой главе рассмотрим все три. Однако в рамках книги сосредоточимся только на контролируемом обучении, так как это самый естественный способ начать обучение и, возможно, наиболее широко используемый в настоящее время. О других типах можно самостоятельно узнать из литературы, потому что все они интересны и полезны! Среди ресурсов, собранных в приложении В, имеется несколько интересных ссылок, в том числе на несколько видеороликов, созданных автором.

В чем разница между размеченными и неразмеченными данными

Что такое данные

Мы говорили о данных в главе 1, но прежде чем двигаться дальше, дадим четкое определение того, что в этой книге подразумевается под *данными*. Данные — это всего лишь информация. Всегда, когда у нас есть таблица с информацией, у нас есть данные. Обычно каждая строка в таблице представляет собой точку данных. Допустим, у нас есть набор данных о домашних животных. В этом случае каждая строка представляет отдельного четвероногого. Каждое домашнее животное в таблице описывается определенными признаками.

Что же это за признаки

В главе 1 мы определили признаки как свойства или характеристики данных. Если наши данные содержатся в таблице, то признаками являются столбцы таблицы. В примере с домашним животным признаками могут быть размер, имя, тип или вес. Признаками могут служить даже цвета пикселей на изображении домашнего животного. Это то, что описывает наши данные. Однако некоторые признаки — особенные, мы называем их *метками*.

Метки?

Этот вариант чуть менее точен, так как зависит от контекста проблемы, которую мы пытаемся решить. Как правило, если мы хотим спрогнозировать определенный признак на основе других, этот признак служит меткой. Если пытаемся спрогнозировать тип домашнего животного (например, кошки или собаки) на основе информации о нем, то метка — это тип домашнего животного (кошка/собака). Если стремимся спрогнозировать, болен питомец или здоров, основываясь на симптомах и другой информации, то метка — это состояние здоровья (болен/здоров). Если пытаемся предсказать возраст питомца, то метка — это возраст (число).

Прогнозы

Мы уже использовали концепцию свободного прогнозирования, теперь давайте определим ее. Цель прогностической модели машинного обучения состоит в том, чтобы угадать метки в данных. Предположение, которое делает модель, называется *прогнозом*.

Теперь, зная, что такое метки, мы можем допустить, что существуют два основных типа данных: *размеченные* и *неразмеченные*.

Размеченные и неразмеченные данные

Размеченные данные — такие, которые поставляются с тегом или меткой (метка может быть как типом, так и числом). Неразмеченные данные — те, что поставляются без меток. Примером размеченных данных может служить набор данных, относящихся к электронным письмам, в котором есть столбец, где записано, является ли письмо спамом, или столбец, где указано, связано ли оно с работой. Пример неразмеченных данных — набор электронных писем, где нет конкретного столбца, который мы хотели бы спрогнозировать.

На рис. 2.1 мы видим три набора данных, содержащих изображения домашних животных. В первом наборе есть столбец с указанием типа домашнего животного, во втором — столбец, где приведен его вес. Это два примера размеченных данных. Третий набор состоит из изображений без меток, что делает его неразмеченными данными.



Рис. 2.1. Наборы данных: *слева* — размечен, меткой служит тип домашнего животного (собака/кошка); *посередине* — размечен, меткой служит вес (в килограммах); *справа* — не размечен

Конечно, это определение содержит некоторую двусмысленность, потому что в зависимости от проблемы мы определяем, подходит ли конкретный признак в качестве метки. Таким образом, определение того, размечены данные или нет, во многих случаях зависит от решаемой проблемы.

Размеченные и неразмеченные данные дают две ветви машинного обучения — *контролируемое* и *неконтролируемое*. Их определение мы найдем в следующих трех разделах.

Контролируемое обучение — раздел машинного обучения, который работает с размеченными данными

Мы можем обнаружить контролируемое обучение в некоторых наиболее распространенных современных приложениях, включая распознавание изображений, различные формы обработки текста и системы рекомендаций. Контролируемое обучение — это тип машинного обучения, при котором используются размеченные данные. Короче говоря, цель модели контролируемого обучения состоит в том, чтобы спрогнозировать (угадать) метки.

В примере на рис. 2.1 набор данных слева содержит изображения собак и кошек, а метки — «собака» и «кошка». Для этого набора данных модель машинного обучения будет использовать предыдущие данные и прогнозировать метки новых точек данных. Это означает, что, если мы введем новое изображение *без* метки, модель будет угадывать, кто это — собака или кошка, прогнозируя таким образом метку точки данных (рис. 2.2). Здесь точка данных соответствует собаке, и алгоритм контролируемого обучения обучается прогнозировать, что она действительно соответствует собаке.



Рис. 2.2. Модель контролируемого обучения прогнозирует метку новой точки данных

Если вы помните из главы 1, основа для принятия решения — это принцип вспоминания — формулирования — прогнозирования. Именно так работает контролируемое обучение. Модель сначала **вспоминает** набор данных о собаках и кошках. Затем **формулирует** модель или правило для всего, что, по ее мнению, представляет собой собаку и кошку. Наконец, когда появляется новое изображение, модель делает **прогноз** о том, что, по ее мнению, является меткой изображения — собака или кошка (рис. 2.3).

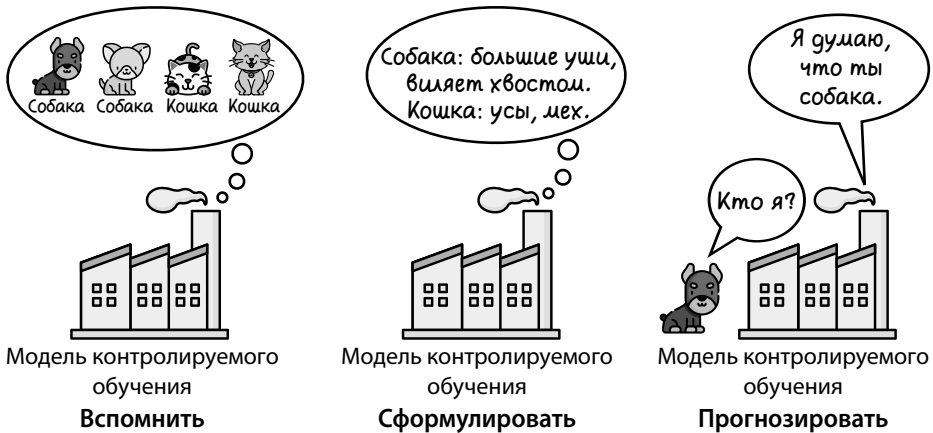


Рис. 2.3. Модель контролируемого обучения соответствует схеме вспоминания — формулирования — прогнозирования

Теперь обратите внимание на то, что на рис. 2.1 два типа размеченных наборов данных. В том, что в середине, каждая точка данных размечена весом

животного. Здесь метки представляют собой числа. В наборе данных слева каждая точка данных размечена типом животного — собака или кошка. В нем метки — это состояния. Числа и состояния — это два типа данных, с которыми мы столкнемся в моделях контролируемого обучения. Первый тип называется *числовыми данными*, второй — *категориальными*.

ЧИСЛОВЫЕ ДАННЫЕ — это любой тип данных, в которых используются такие числа, как 4, 2,35 или -199. Примерами числовых данных могут служить цены, размеры или веса.

КАТЕГОРИАЛЬНЫЕ ДАННЫЕ — это любой тип данных, который использует категории или состояния, такие как «мужчина/женщина» или «кошка/собака/птица». Для этого типа данных у нас имеется конечный набор категорий, которые нужно связать с каждой из точек данных.

Это приводит к появлению следующих двух типов моделей контролируемого обучения.

РЕГРЕССИОННЫЕ МОДЕЛИ — это типы моделей, которые прогнозируют **числовые данные**. Результатом регрессионной модели выступает *число*, такое как вес животного.

КЛАССИФИКАЦИОННЫЕ МОДЕЛИ — это типы моделей, которые прогнозируют **категориальные данные**. Результатом классификационной модели служит *категория* или *состояние*, например тип животного — кошка или собака.

Рассмотрим два примера моделей контролируемого обучения: одну регрессию и одну классификацию.

Модель 1. Модель прогнозирования цен на жилье (регрессия). Здесь каждая точка данных представляет собой дом. Метка каждого дома — это его цена. Наша цель — добиться того, чтобы, когда на рынке появится новый дом (точка данных), мы могли бы спрогнозировать его метку, то есть цену.

Модель 2. Модель обнаружения спама в электронной почте (классификация). Здесь каждая точка данных представляет собой электронное письмо. Метка каждого из них — является ли оно спамом. Наша цель в том, чтобы при поступлении нового электронного письма (точки данных) мы могли спрогнозировать его метку, а именно, является ли оно спамом.

Обратите внимание на разницу между моделями 1 и 2.

- Модель прогнозирования цен на жилье может возвращать число из множества возможных значений, таких как 100 долларов, 250 000 долларов или 3 125 672,33 доллара. Это делает ее *регрессионной* моделью.
- Модель обнаружения спама может возвращать только два значения: спам или «ветчина». Таким образом, это *классификационная* модель.

В следующих подразделах мы подробнее остановимся на регрессии и классификации.

Регрессионные модели прогнозируют числа

Как говорилось ранее, регрессионные модели — это те, где метка, которую мы хотим получить, представляет собой число. Оно прогнозируется на основе признаков. В примере с жильем признаками могут быть любые характеристики, описывающие дом: размер, количество комнат, расстояние до ближайшей школы или уровень преступности в районе.

Далее приведены другие области, где можно использовать регрессионные модели.

- **Фондовый рынок** — прогнозирование цены определенной акции на основе других цен на акции и иных рыночных сигналов.
- **Медицина** — прогнозирование ожидаемой продолжительности жизни пациента или ожидаемого времени выздоровления на основе симптомов и истории болезни.
- **Продажи** — прогнозирование ожидаемой суммы, которую потратит клиент, на основе его демографических данных и поведения при покупках в прошлом.
- **Рекомендации видео** — прогнозирование ожидаемого количества времени, в течение которого пользователь будет смотреть видео, на основе демографических данных пользователя и других просмотренных им видео.

Наиболее распространенным методом регрессии выступает линейная регрессия. Для составления прогнозов в ней применяются линейные функции (прямые или аналоги) на основе признаков. Мы изучим линейную регрессию в главе 3. Другими популярными методами, с помощью которых выполняется регрессия, выступают регрессия дерева решений (ее мы изучим в главе 9) и несколько методов ансамбля, таких как случайные леса, AdaBoost, деревья с градиентным усилением и XGBoost (см. главу 12).

Классификационные модели предсказывают состояние

Классификационные модели — это модели, в которых метка, которую мы хотим спрогнозировать, представляет собой состояние, принадлежащее конечному набору состояний. Наиболее распространенные модели классификации выдают прогноз в виде «да» или «нет», но многие другие модели используют и больший набор состояний. На рис. 2.3 мы видели пример классификации, поскольку он предсказывает тип домашнего животного — кошка или собака.

В примере распознавания спама среди электронной почты модель предсказывает состояние электронного письма (а именно спам или «ветчина») на основе его признаков. В этом случае особенностями электронного письма могут быть содержащиеся в нем слова, количество орфографических ошибок, отправитель или что-либо еще, что его характеризует.

Другим распространенным применением классификации выступает распознавание изображений. Наиболее популярные модели распознавания

изображений принимают в качестве входных данных пиксели изображения и выдают прогноз о том, что на нем представлено. Два наиболее известных набора данных для распознавания изображений — это MNIST и CIFAR-10. MNIST содержит приблизительно 60 000 черно-белых изображений размером 28×28 пикселей с написанными от руки цифрами, которые размечены как 0–9. Изображения взяты из нескольких источников, включая Американское бюро переписи населения и хранилище рукописных цифр (их авторы — американские старшеклассники). Набор данных MNIST можно найти по ссылке <http://yann.lecun.com/exdb/mnist/>. Набор данных CIFAR-10 содержит 60 000 цветных изображений различных объектов размером 32×32 пиксела. Они размечены десятью различными объектами (это объясняет число 10 в его названии): самолетами, автомобилями, птицами, кошками, оленями, собаками, лягушками, лошадьми, кораблями и грузовиками. Эта база данных поддерживается Канадским институтом перспективных исследований (CIFAR), ее можно найти по ссылке <https://www.cs.toronto.edu/~kriz/cifar.html>.

Далее приведены дополнительные полезные применения классификационных моделей.

- **Анализ настроений** — прогнозирование того, будет ли рецензия на фильм положительной или отрицательной, на основе имеющихся в ней слов.
- **Посещаемость сайта** — прогнозирование того, нажмет пользователь на ссылку или нет, на основе его демографических данных и взаимодействия с сайтом в прошлом.
- **Социальные сети** — прогнозирование того, будет ли пользователь дружить или взаимодействовать с другим пользователем, на основе их демографии, истории и общих друзей.
- **Рекомендации по видео** — прогнозирование того, будет ли пользователь смотреть видео, на основе демографических данных и других просмотренных видео.

Основная часть книги (главы 5, 6, 8–12) охватывает классификационные модели. В них мы изучаем перцептроны (глава 5), логистические классификаторы (глава 6), наивный байесовский алгоритм (глава 8), деревья решений (глава 9), нейронные сети (глава 10), машины опорных векторов (глава 11) и методы ансамбля (глава 12).

Неконтролируемое обучение: раздел машинного обучения, который работает с неразмеченными данными

Неконтролируемое обучение помимо прочего является распространенным типом машинного обучения. От контролируемого обучения его отличает то, что данные для него не размечены. Другими словами, цель модели

машинного обучения состоит в том, чтобы извлечь как можно больше информации из набора данных, который не имеет меток или целей для прогнозирования.

Каким может быть такой набор данных и что мы можем с ним делать? В целом несколько меньше, чем с размеченным набором, так как у нас нет меток для прогнозирования. Однако мы способны извлечь достаточно информации и из неразмеченного набора данных. Вернемся к примеру с кошками и собаками в крайнем правом наборе данных на рис. 2.1. Он состоит из изображений кошек и собак, но в нем нет меток. Поэтому мы не знаем, какой тип домашнего животного представляет каждое изображение, и не можем спрогнозировать, соответствует новое изображение собаке или кошке. Однако можем выполнять другие задачи, например, определять, похожи ли две картинки друг на друга. Это то, на что способны алгоритмы неконтролируемого обучения. Такой алгоритм может группировать изображения на основе сходства, даже не зная, что представляет собой каждая группа (рис. 2.4). Если все сделано правильно, алгоритм сможет отделить изображения собак от изображений кошек и даже сгруппировать их по породам!



Рис. 2.4. Алгоритм неконтролируемого обучения вполне может извлекать информацию из данных. Например, он способен группировать похожие элементы

На самом деле, даже если метки есть, мы все равно можем использовать методы неконтролируемого обучения для своих данных, чтобы предварительно обработать их и более эффективно задействовать методы контролируемого обучения.

Основные направления неконтролируемого обучения — это кластеризация, понижение размерности и генеративное обучение.

АЛГОРИТМЫ КЛАСТЕРИЗАЦИИ — алгоритмы, которые группируют данные в кластеры на основе сходства.

АЛГОРИТМЫ ПОНИЖЕНИЯ РАЗМЕРНОСТИ — алгоритмы, которые упрощают данные и достоверно описывают их с меньшим количеством признаков.

ГЕНЕРАТИВНЫЕ АЛГОРИТМЫ — алгоритмы, которые способны генерировать новые точки данных, похожие на существующие.

В следующих трех подразделах мы более подробно изучим эти три ответвления.

Алгоритмы кластеризации разбивают набор данных на группы похожих элементов

Как говорилось ранее, алгоритмы кластеризации — это алгоритмы, которые разбивают набор данных на группы схожих элементов. Чтобы проиллюстрировать это, вернемся к двум наборам данных, упомянутых в разделе «Контролируемое обучение...», — о жилье и спаме. Но теперь представьте, что у них нет меток. Это означает, что в наборе данных о жилье нет цен, а в наборе данных о почте нет информации о том, что электронные письма являются спамом. Начнем с набора данных о жилье. Что можно с ним сделать? Вот вам одна идея: мы могли бы каким-то образом сгруппировать дома по сходству. Например, можно сгруппировать их по местоположению, цене, размеру или сочетанию этих факторов. Такой процесс называется *кластеризацией*. Кластеризация — это ответвление неконтролируемого машинного обучения, занимающееся задачами группировки элементов в наборе данных, разбивая их на кластеры, где все точки данных похожи.

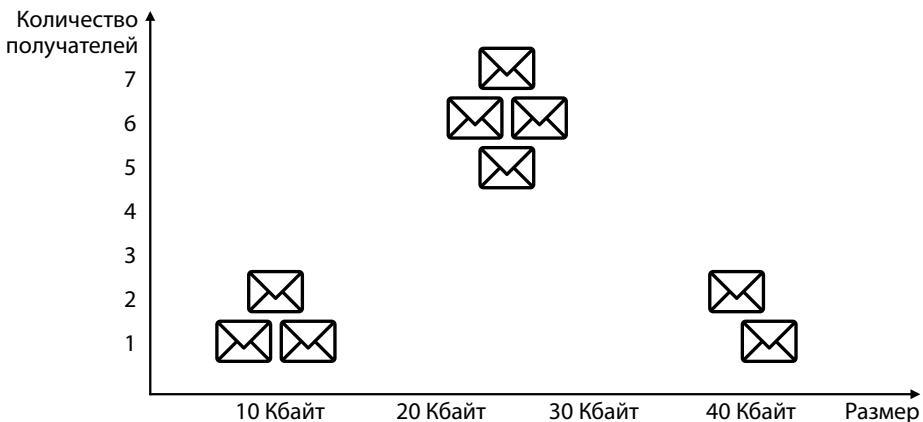
Теперь рассмотрим второй пример — набор данных электронных писем. Поскольку он не размечен, мы не знаем, является ли то или иное электронное письмо спамом. Однако можем выполнить кластеризацию набора. Алгоритм кластеризации разбивает изображения на несколько групп в зависимости от различных признаков электронного письма. Ими могут быть слова, использованные в сообщении, отправитель, количество и размер вложений или типы ссылок внутри письма. После кластеризации набора данных человек (или совместно человек и алгоритм контролируемого обучения) может обозначить эти кластеры такими категориями, как «Личные», «Социальные» и «Рекламные акции».

В качестве примера рассмотрим набор данных, содержащий девять электронных писем, которые нужно сгруппировать (табл. 2.1). Признаки набора данных — размер электронного письма и количество получателей.

Таблица 2.1. Таблица электронных писем с указанием их размера и количества получателей

Электронная почта	Размер	Получатели
1	8	1
2	12	1
3	43	1
4	10	2
5	40	2
6	25	5
7	23	6
8	28	6
9	26	7

Сразу становится очевидно, что мы могли бы сгруппировать электронные письма по количеству получателей. Это создаст два кластера: один с электронными письмами, у которых два или менее получателей, и один с электронными письмами, у которых пять или более получателей. Мы могли бы также попытаться сгруппировать их в три группы по размеру. Но по мере того, как таблица будет становиться все больше, следить за группами станет все труднее и труднее. А что если построить график? Представим электронные письма в виде графика, где на горизонтальной оси указан размер, а на вертикальной — количество получателей. В итоге получим график, приведенный на рис. 2.5.

**Рис. 2.5.** График набора данных электронной почты

Здесь мы видим три четко определенных кластера (рис. 2.6).

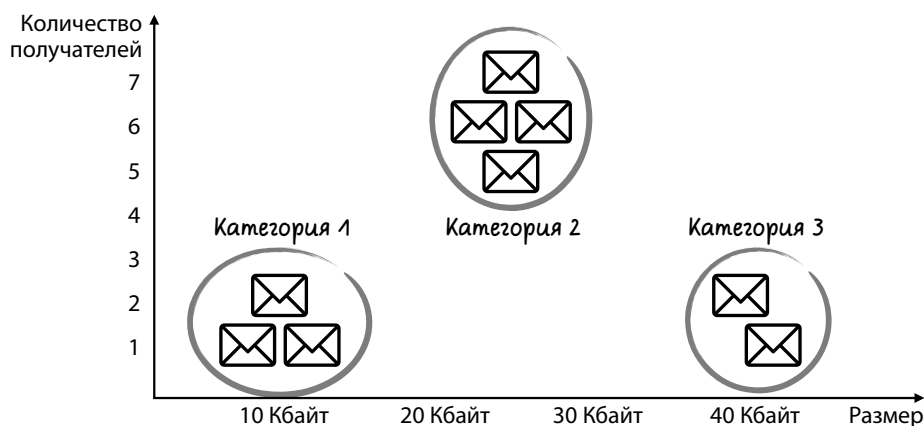


Рис. 2.6. Можно сгруппировать электронные письма в три категории в зависимости от размера и количества получателей

Этот последний шаг отражает всю суть кластеризации. Конечно, нам, людям, легко работать с тремя группами, если они представлены графически. Но для компьютера эта задача не из легких. Кроме того, представьте, что данные содержат миллионы точек с сотнями или тысячами признаков. При наличии более чем трех объектов люди уже не смогут увидеть скопления, потому что будут не в силах визуализировать их размеры. К счастью, компьютеры могут выполнять такой тип кластеризации для огромных наборов данных с несколькими строками и столбцами.

Другие области применения кластеризации.

- **Сегментация рынка** — разделение покупателей на группы на основе демографических данных и предыдущего покупательского поведения для создания различных групповых маркетинговых стратегий.
- **Генетика** — объединение видов в группы на основе сходства генов.
- **Медицинская визуализация** — разделение изображения на части для изучения различных типов тканей.
- **Рекомендации по видео** — разделение пользователей на группы на основе демографических данных и просмотренных ранее видео и применение этого приема для рекомендации пользователю роликов, которые смотрели другие члены их группы.

Подробнее о моделях неконтролируемого обучения

В дальнейшем мы не будем рассматривать неконтролируемое обучение. Однако я настоятельно рекомендую изучить эту сферу самостоятельно. Вот некоторые из наиболее важных алгоритмов кластеризации. В приложении В перечислены

другие ресурсы, включая несколько моих видеороликов, с помощью которых вы можете подробно изучить эти алгоритмы.

- **Кластеризация К-средних.** Этот алгоритм группирует точки, выбирая некоторые случайные центры масс и перемещая их все ближе и ближе к точкам, пока они не окажутся в нужных местах.
- **Иерархическая кластеризация.** Алгоритм начинается с группировки ближайших точек и продолжает работу, пока не получится нескольких четко определенных групп.
- **Пространственная кластеризация на основе плотности (DBSCAN).** Этот алгоритм начинает группировать точки в местах с высокой плотностью, помечая изолированные точки как шум.
- **Гауссовы модели смещения.** Алгоритм не присваивает точку одному кластеру, а присваивает доли точки каждому из существующих кластеров. Например, если есть три кластера, А, В и С, то алгоритм может определить, что 60 % точки принадлежит группе А, 25 % — группе В и 15 % — группе С.

Понижение размерности упрощает обработку данных без потери слишком большого объема информации

Понижение размерности — это полезный шаг в ходе предварительной обработки данных, который можно применять для их значительного упрощения, прежде чем переходить к другим методам. В качестве примера вернемся к набору данных о жилье. Представим, что в наборе присутствуют следующие признаки:

- размер;
- количество спален;
- количество ванных комнат;
- уровень преступности в районе;
- расстояние до ближайшей школы.

Этот набор данных состоит из пяти столбцов. А если нам требуется превратить его в более простой, с меньшим количеством столбцов, не теряя при этом большого количества информации? Давайте сделаем это, руководствуясь здравым смыслом. Присмотритесь повнимательнее к этим пяти признакам. Виден ли какой-то способ упрощения — возможно, способ объединить более мелкие категории в более общие?

При внимательном рассмотрении становится ясно, что первые три характеристики схожи, так как связаны с размером дома. Аналогично, четвертый и пятый признаки похожи друг на друга, поскольку связаны с качеством жизни в районе. Мы могли бы объединить первые три признака в больший признак «размер», а четвертый и пятый — в «благополучие района». Как же уплотнить признаки, касающиеся размера? Мы могли бы опустить данные о комнатах и спальнях и учитывать только размер или, возможно, взять какую-то другую

комбинацию этих трех признаков. Таким же образом можно сжать признаки, касающиеся благополучия района. Алгоритмы понижения размерности ищут эффективные способы уплотнить первичные признаки (рис. 2.7, *слева*), потеряв при этом как можно меньше информации, сохранив данные как можно более нетронутыми и одновременно сумев упростить их для обработки и хранения (рис. 2.7, *справа*).

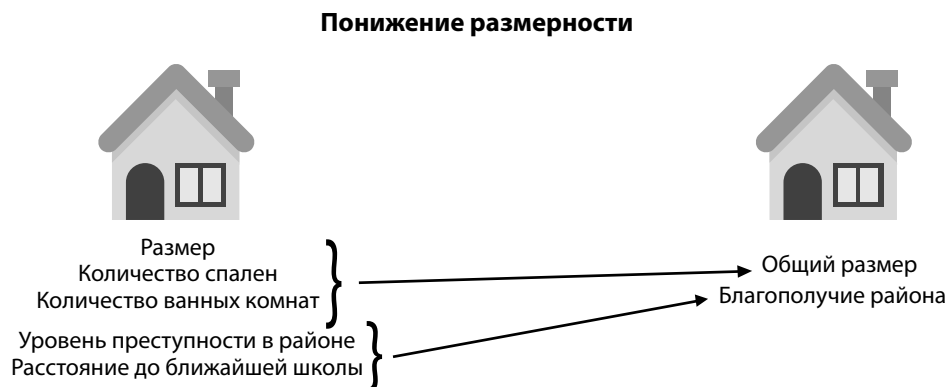


Рис. 2.7. Алгоритмы понижения размерности помогают упрощать данные

Почему этот процесс называется понижением размерности, если мы всего лишь уменьшаем количество столбцов в данных? Модное словцо для обозначения количества столбцов в наборе данных — *измерение*. Подумайте вот о чем: если данные состоят из одного столбца, то каждая точка данных представляет собой одно число. Набор чисел может быть нанесен на график как набор точек на линии, которая имеет ровно одно измерение. Если данные состоят из двух столбцов, то каждая точка данных формируется парой чисел. Мы можем представить набор пар чисел как набор точек в городе, где первое число — это номер улицы, а второе — номер дома. Адреса на карте являются двумерными, потому что располагаются на плоскости. Что происходит, когда наши данные состоят из трех столбцов? В этом случае каждая точка данных формируется тремя числами. Мы можем представить, что если каждый адрес в городе — это здание, то первое и второе числа — это улица и номер здания, а третье — этаж. Это больше похоже на трехмерный город. Можно продолжать и дальше. А как насчет четырех чисел? Мы не можем по-настоящему это визуализировать, но если бы могли, то набор точек выглядел бы как набор мест в четырехмерном городе и т. д. Лучший способ представить себе четырехмерный город — это представить таблицу с четырьмя столбцами. А как насчет 100-мерного города? Это будет таблица со 100 столбцами, в которой у каждого человека есть адрес, состоящий из 100 значений. Картина, которую

можно было бы вообразить, думая о более высоких измерениях, показана на рис. 2.8. Это придуманный город, в котором адреса имеют столько координат, сколько нам нужно. Поэтому, перейдя от пяти измерений к двум, мы превратили наш пятимерный город в двумерный. Вот почему это называется понижением размерности.

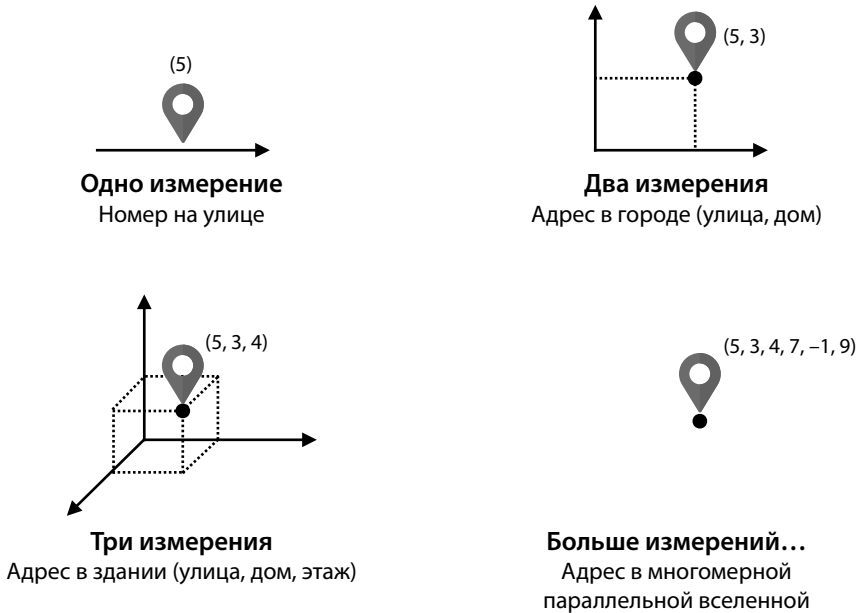


Рис. 2.8. Представление пространства более высоких измерений

Другие способы упрощения данных. Матричная факторизация и разложение по сингулярным значениям

Кажется, что кластеризация и понижение размерности совсем не похожи друг на друга, но на деле они не так уж сильно различаются. Предположим, у нас имеется таблица, полная данных, каждая строка соответствует точке данных, а каждый столбец — признаку. Следовательно, мы можем использовать кластеризацию для уменьшения количества строк в наборе данных (рис. 2.9), а понижение размерности — для уменьшения количества столбцов (рис. 2.10).

Возможно, вам интересно, есть ли способ, которым можно сократить количество строк и столбцов одновременно? И ответом будет — да! Два распространенных способа это проделать — разложение *матрицы на множители* и разложение *по сингулярным значениям*. Эти два алгоритма преобразуют большую матрицу данных в произведение меньших матриц.

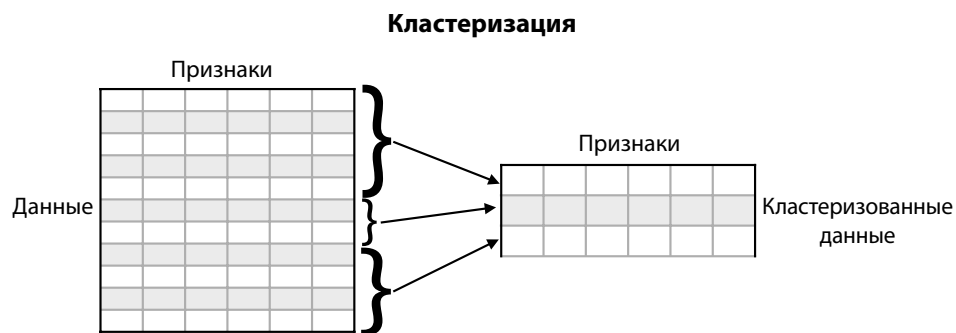


Рис. 2.9. Кластеризация может быть использована для упрощения данных за счет уменьшения количества строк в наборе данных путем группировки нескольких строк в одну

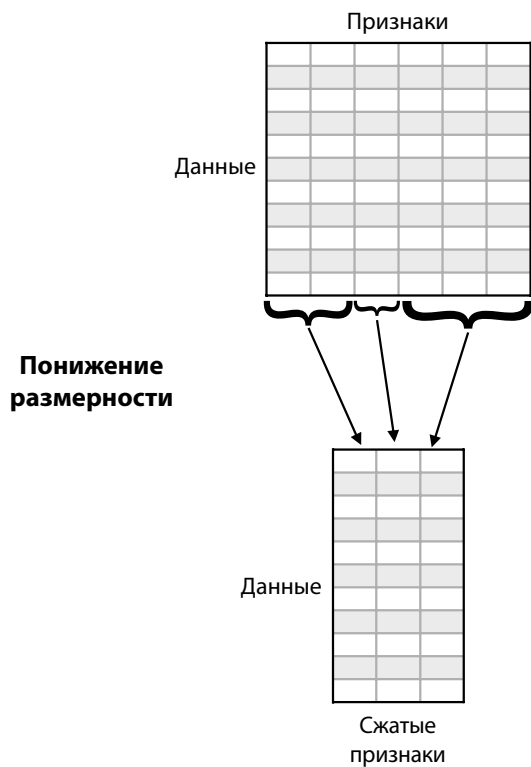


Рис. 2.10. Понижение размерности можно применять для упрощения данных за счет сокращения количества столбцов в наборе данных

Такие компании, как Netflix, широко задействуют матричную факторизацию для выработки рекомендаций. Представьте себе большую таблицу, где каждая строка соответствует пользователю, каждый столбец — фильму, а каждая запись в матрице — это рейтинг, который пользователь дал фильму. С помощью матричной факторизации можно извлечь определенные признаки, такие как тип фильма, актеры, появляющиеся в нем, и др., а также предсказать рейтинг, который пользователь даст ленте, на основе этих признаков.

При сжатии изображений применяется разложение по сингулярным значениям. Например, черно-белое изображение можно рассматривать как большую таблицу данных, где каждая запись содержит интенсивность соответствующего пиксела. Разложение по сингулярным значениям использует методы линейной алгебры для упрощения этой таблицы данных, что позволяет нам упростить изображение и сохранить его облегченную версию с помощью меньшего количества записей.

Генеративное машинное обучение

Генеративное машинное обучение — одна из самых удивительных областей машинного обучения. Если вы видели ультрареалистичные лица, изображения или видео, созданные компьютерами, значит, вы видели генеративное машинное обучение в действии.

Область генеративного обучения состоит из моделей, которые, учитывая набор данных, могут выводить новые точки данных, выглядящие как образцы из исходного набора. Эти алгоритмы изучают, как выглядят данные, после чего создают аналогичные точки данных. Например, если набор данных содержит изображения лиц, то алгоритм будет создавать реалистично выглядящие лица. Генеративные алгоритмы способны создавать чрезвычайно реалистичные изображения, картины и т. д. Они также создают видео, музыку, рассказы, стихи и много других замечательных штук. Наиболее популярный генеративный алгоритм — это генеративные состязательные сети (GAN), разработанные Яном Гудфеллоу с соавторами. Другие полезные и популярные генеративные алгоритмы: вариационные автокодеры, созданные Кингмой и Веллингом, и ограниченные машины Больцмана (RBMs), разработанные Джефффри Хинтоном.

Вы наверняка уже поняли, что генеративное обучение — довольно сложная штука. Человеку гораздо легче определить, изображена ли на картинке собака, чем нарисовать ее. Эта задача не менее трудна и для компьютеров. Так что алгоритмы генеративного обучения сложны, и для их эффективной работы требуется много данных и значительные вычислительные мощности. Поскольку наша книга посвящена контролируемому обучению, мы не будем подробно останавливаться на генеративном обучении, но в главе 10 получим представление о том, как работают некоторые из таких алгоритмов, поскольку они, как правило, используют нейронные сети. Приложение В содержит рекомендации по ресурсам, включая видео автора, которые помогут вам, если вы захотите продолжить изучение этой темы.

Что такое обучение с подкреплением

Обучение с подкреплением — это еще один тип машинного обучения, при котором изначально данные не передаются и мы должны заставить компьютер выполнить задачу. Вместо данных модель получает среду и агента, который должен в ней перемещаться. У агента есть цель или набор целей.

В окружающей среде есть награды и наказания, которые помогают агенту принимать правильные решения для достижения цели. Все это звучит несколько абстрактно, так что давайте рассмотрим пример.

Сетчатый мир

На рис. 2.11 мы видим сетчатый мир с роботом в левом нижнем углу. Это наш агент. Его цель состоит в том, чтобы добраться до сундука с сокровищами, расположенного в правом верхнем углу сетки. В ней также имеется гора. Это означает, что робот не сможет пройти через этот квадрат, потому что не способен взбираться на горы. Мы также видим дракона, который нападет на робота, если тот осмелится остановиться на его позиции. Это значит, что часть цели состоит в том, чтобы там не останавливаться. Вот и вся игра. И чтобы дать роботу информацию о том, как действовать дальше, мы отслеживаем результат. Счет начинается с нуля. Если робот доберется до сундука с сокровищами, то мы наберем 100 очков. Если он доберется до дракона, то потеряем 50 очков. И чтобы наш робот двигался быстро, мы можем задать условие, что за каждый шаг, который делает робот, мы теряем 1 очко, потому что он теряет энергию при ходьбе.

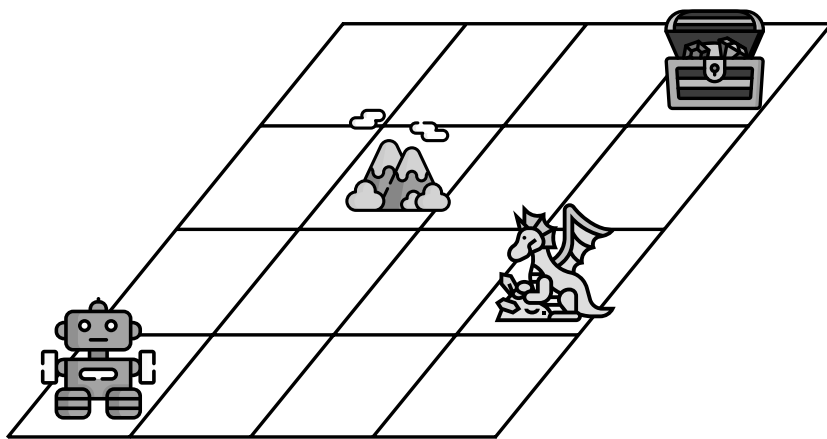


Рис. 2.11. Сетчатый мир, агентом в котором выступает робот

Если кратко, то способ обучения такого алгоритма следующий: робот начинает ходить, записывая свой результат и запоминая, какие шаги он сделал. Через какое-то время он может встретиться с драконом и потерять много очков. Таким образом он учится ассоциировать квадрат дракона и близкие к нему квадраты с понижением счета. В какой-то момент он может наткнуться на сундук с сокровищами и научиться связывать этот квадрат и соседние с ним с высокой наградой. Поиграв в эту игру в течение длительного времени, робот получит хорошее представление о том, насколько ценен каждый квадрат. После чего он сможет пройти путь по квадратам, дойдя до сундука с сокровищами. На рис. 2.12 показан возможный путь. Он не идеален, поскольку проходит слишком близко к дракону. Сможете проложить путь получше?

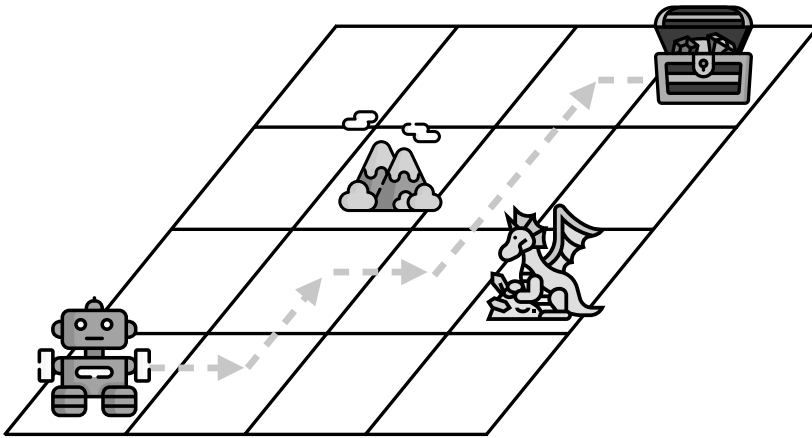


Рис. 2.12. Путь, по которому робот мог бы пройти, чтобы найти сундук с сокровищами

Конечно, это очень краткое объяснение, обучение с подкреплением — это нечто гораздо большее. В приложении В представлены некоторые ресурсы для дальнейшего изучения, включая видео про обучение с глубоким подкреплением.

Обучение с подкреплением применяется во множестве передовых областей. В их числе:

- **игры.** Компьютеры научились побеждать в таких играх, как го или шахматы, с помощью именно обучения с подкреплением. Кроме того, агентов научили побеждать в играх Atari, таких как *Breakout* или *Super Mario*;
- **робототехника.** Обучение с подкреплением широко используется, чтобы помочь роботам выполнять такие задачи, как сбор коробок, уборка комнаты или даже танцы;
- **беспилотные автомобили.** Методы обучения с подкреплением применяются для того, чтобы помочь автомобилю выполнять задачи планирования маршрута или поведения в определенных условиях.

Резюме

- Существует несколько типов машинного обучения, включая контролируемое обучение, неконтролируемое обучение и обучение с подкреплением.
- Данные могут быть размеченными или неразмеченными. Размеченные данные содержат специальный признак, или метку, который мы стремимся предсказать. Неразмеченные данные не содержат этого признака.
- Контролируемое обучение используется для размеченных данных и заключается в построении моделей, предсказывающих метки для неявных данных.
- Неконтролируемое обучение применяется для неразмеченных данных и состоит из алгоритмов, которые упрощают данные без потери большого количества информации. Неконтролируемое обучение часто служит этапом предварительной обработки.
- Два распространенных типа алгоритмов контролируемого обучения называются регрессией и классификацией.
 - Регрессионные модели — это те, в которых ответом служит любое число.
 - Классификационные модели — те, в которых ответ относится к типу или классу.
- Два распространенных типа алгоритмов неконтролируемого обучения — это кластеризация и понижение размерности.
 - Кластеризация используется для группировки похожих данных в кластеры для извлечения информации или упрощения ее обработки.
 - Понижение размерности — это способ упростить данные, объединив некоторые схожие признаки и потеряв как можно меньше информации.
 - Матричная факторизация и разложение по сингулярным значениям — это дополнительные алгоритмы, способные упростить данные за счет уменьшения количества строк и столбцов.
- Генеративное машинное обучение — это инновационный тип неконтролируемого обучения, заключающийся в генерации данных, аналогичных исходному набору данных. Генеративные модели способны рисовать реалистичные лица, сочинять музыку и писать стихи.
- Обучение с подкреплением — это тип машинного обучения, при котором агент вынужден ориентироваться в среде и достигать цели. Оно широко используется во многих передовых приложениях.

Упражнения

Упражнение 2.1

Примером контролируемого или неконтролируемого обучения является каждый из следующих сценариев? Объясните свои ответы. В случае неоднозначности ответа выберите один из вариантов и объясните, почему выбрали именно его.

- А. Система рекомендаций в социальной сети, которая предлагает пользователю потенциальных друзей.
- Б. Система на новостном сайте, которая разделяет новости на темы.
- В. Функция автоматического завершения предложений в Google.
- Г. Система рекомендаций в интернет-магазине, которая формирует рекомендации, основываясь на истории покупок.
- Д. Система в компании, выпускающей кредитные карты, которая фиксирует мошеннические транзакции.

Упражнение 2.2

Регрессию или классификацию необходимо использовать для любого из следующих применений машинного обучения? Объясните свои ответы. В случае неоднозначности ответа выберите один из вариантов и объясните, почему выбрали именно его.

- А. Интернет-магазин, предсказывающий, сколько денег пользователь потратит на сайте.
- Б. Голосовой помощник, декодирующий голос и превращающий его в текст.
- В. Продажа или покупка акций определенной компании.
- Г. Рекомендации видео от YouTube.

Упражнение 2.3

Ваша задача — построить беспилотный автомобиль. Приведите по крайней мере три примера задач машинного обучения, которые вам пришлось бы при этом решить. В каждом примере объясните, какое обучение вы используете, контролируемое или неконтролируемое. Если контролируемое, то применяете регрессию или классификацию? Если используете другие типы машинного обучения, объясните, какие и почему.



В этой главе

- ✓ Что такое линейная регрессия.
 - ✓ Прокладывание прямой через набор точек данных.
 - ✓ Кодирование алгоритма линейной регрессии на Python.
 - ✓ Использование Turi Create для построения модели линейной регрессии для прогнозирования цен на жилье на реальном наборе данных.
 - ✓ Что такое полиномиальная регрессия.
 - ✓ Подгонка более сложной кривой к нелинейным данным.
 - ✓ Обсуждение примеров линейной регрессии в реальных задачах, таких как медицинские приложения и рекомендательные системы.
-



В этой главе мы узнаем о линейной регрессии. Линейная регрессия — эффективный и широко применяемый метод оценки значений, таких как цена дома, стоимость определенных акций, ожидаемая продолжительность жизни отдельного человека или количество времени, которое пользователь проведет за просмотром видео или на веб-сайте. Возможно, вы уже встречались с линейной регрессией в виде множества сложных формул, включая производные, системы уравнений и определители. Однако мы можем представить линейную регрессию более наглядно и менее формально. В этой главе, чтобы понять линейную регрессию, нам понадобится лишь умение визуализировать точки и прямые.

Допустим, у нас есть несколько точек, которые более или менее образуют прямую (рис. 3.1).

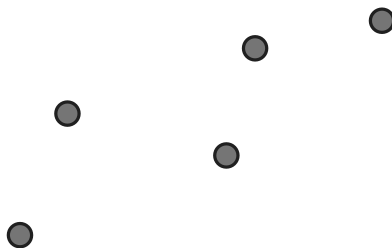


Рис. 3.1. Точки, которые в приближении образуют прямую

Цель линейной регрессии состоит в том, чтобы построить прямую, проходящую как можно ближе к этим точкам. Какой, по-вашему, была бы прямая, проходящая как можно ближе к ним? Как насчет той, что показана на рис. 3.2?

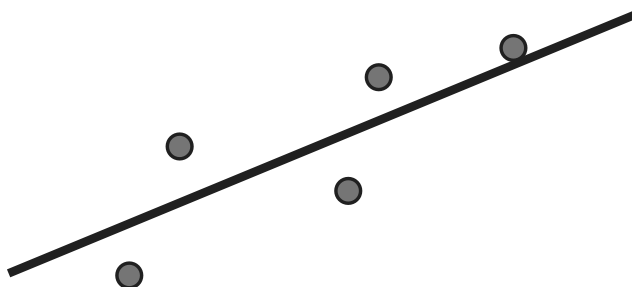


Рис. 3.2. Прямая, проходящая вблизи точек

Если думать о точках как о домах в городе, то наша цель — проложить дорогу, которая проходит через город. Мы хотим, чтобы прямая проходила как можно ближе к точкам, потому что все жители города хотят жить рядом с дорогой и наша цель — как можно более полно удовлетворить их желание.

Мы также можем изобразить точки в виде магнитов, прикрученных к полу и поэтому неподвижных. Теперь представьте, что вы кладете на них прямой металлический стержень. Он будет двигаться, но, поскольку магниты притягивают его, в конечном итоге окажется в положении равновесия — как можно ближе ко всем точкам.

Конечно, постановка задачи порождает некоторую неоднозначность. Хотим ли мы, чтобы дорога проходила более или менее близко ко всем домам или, может быть, очень близко к нескольким из них и немного дальше от других? Вот часть возникающих вопросов.

- Что мы подразумеваем под точками, которые в приближении образуют прямую?
- Что мы подразумеваем под прямой, которая проходит очень близко к точкам?
- Как нам найти такую прямую?
- Как это может пригодиться в реальном мире?
- Почему это относится к машинному обучению?

В этой главе мы ответим на все эти вопросы и построим модель линейной регрессии для прогнозирования цен на жилье на реальном наборе данных.

Вы можете найти весь код из этой главы в следующем репозитории GitHub: https://github.com/luiguiserrano/manning/tree/master/Chapter_3_Linear_Regression.

Задача: спрогнозировать цену дома

Допустим, мы агенты по недвижимости, ответственные за продажу нового дома. Мы не знаем цены и хотим определить ее, сравнив наш дом с другими. Мы принимаем во внимание признаки дома, которые способны повлиять на цену,

такие как размер, количество комнат, местоположение, уровень преступности в районе, качество образования в местных школах и расстояние до магазинов. В конце концов нам нужна формула для всех этих признаков, которая даст цену дома или, по крайней мере, хорошую его оценку.

Решение: построить регрессионную модель цен на жилье

Рассмотрим как можно более простой пример. Обратимся только к одному признаку — количеству комнат. В нашем доме четыре комнаты, а рядом расположены шесть домов с одной, двумя, тремя, пятью, шестью и семью комнатами. Их цены приведены в табл. 3.1.

Таблица 3.1. Дом 4 — тот самый, цену которого мы пытаемся определить

Количество комнат	Цена, в долларах
1	150
2	200
3	250
4	?
5	350
6	400
7	450

В какую сумму вы бы оценили дом 4, основываясь только на информации из таблицы? Если ваше предположение — 300 долларов, то мы мыслим одинаково. Вы, вероятно, увидели закономерность и использовали ее, чтобы определить цену дома. То, что вы проделали мысленно, и было линейной регрессией. Давайте подробнее изучим эту закономерность. Возможно, вы заметили, что каждый раз, когда вы добавляете комнату, цена дома возрастает на 50 долларов. Итак, мы можем рассматривать цену дома как комбинацию двух показателей: базовой цены 100 долларов и дополнительной платы в размере 50 долларов за каждую комнату. Это можно обобщить в простой формуле:

$$\text{цена} = 100 + 50 \times \text{количество комнат.}$$

Иными словами, мы разработали модель, представленную формулой, которая дает *прогноз* цены дома, основанный на *признаке*, представляющем собой количество комнат. Цена за комнату называется *весом* соответствующего признака, а базовая цена — *смещением* модели. Все это важные понятия в машинном обучении. Мы изучили некоторые из них в главах 1 и 2, но давайте освежим память, определив их с точки зрения данной задачи.

ПРИЗНАКИ точки данных — это признаки, которые мы используем для составления прогноза. В данном случае признаками выступают количество комнат в доме, уровень преступности в районе, возраст дома, размер и т. д. Здесь мы остановились на одном признаке — количестве комнат в доме.

МЕТКА — это цель, которую мы пытаемся предсказать по признакам. В данном случае меткой служит цена дома.

МОДЕЛЬ машинного обучения — это правило или формула, которая предсказывает метку на основе признаков. Здесь модель — это уравнение, которое мы вывели для определения цены.

ПРОГНОЗИРОВАНИЕ — это результат работы модели. Если модель сообщает: «Я думаю, что дом с четырьмя комнатами будет стоить 300 долларов», то прогноз равен 300.

ВЕСА В формуле, соответствующей модели, каждый признак умножается на некий коэффициент. Эти факторы являются весовыми коэффициентами. В предыдущей формуле единственным признаком выступает количество комнат, а его вес равен 50.

СМЕЩЕНИЕ Формула, соответствующая модели, содержит константу, которая не привязана ни к одному из признаков. Эта константа называется смещением. В этой модели смещение равно 100, и оно соответствует базовой цене дома.

Теперь вопрос: как же мы пришли к этой формуле? Или, более конкретно, как мы можем заставить компьютер выдать вес и смещение? В качестве иллюстрации рассмотрим чуть более сложный пример. И поскольку это проблема машинного обучения, мы подойдем к ней, используя структуру вспоминания — формулирования — прогнозирования, которую изучили в главе 2. Более конкретно, *вспомним* цены на другие дома, *сформулируем* модель цены и будем применять ее для *прогнозирования* цены нового дома.

Этап вспоминания: изучение цен существующих домов

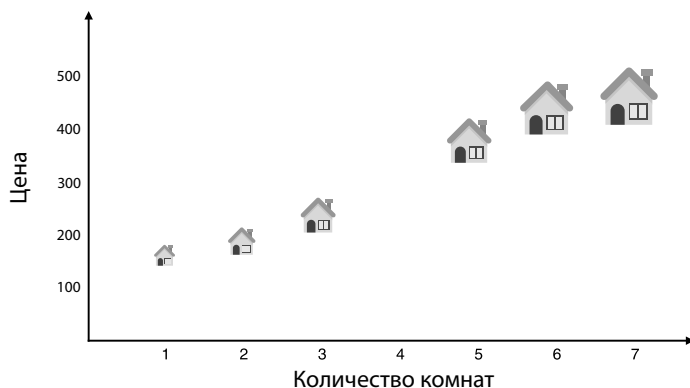
Чтобы более четко представить себе процесс, рассмотрим несколько более сложный набор данных, приведенный в табл. 3.2.

Этот набор данных похож на предыдущий, за исключением того, что теперь цены не укладываются в простой шаблон, где каждая следующая на 50 долларов больше предыдущей. Однако все это недалеко ушло от исходного набора данных, поэтому можно ожидать, что аналогичный шаблон должен хорошо приближать эти значения.

Обычно первое, что мы делаем, когда получаем новый набор данных, — строим его график. На рис. 3.3 изображен график точек в системе координат, в которой горизонтальная ось представляет количество комнат, а вертикальная — цену дома.

Таблица 3.2. Чуть более сложный набор данных о домах с указанием количества комнат в них и их цены

Количество комнат	Цена, в долларах
1	155
2	197
3	244
4	?
5	356
6	407
7	448

**Рис. 3.3.** График для набора данных, приведенного в табл. 3.2

Следующий шаг: формулирование правила, которое оценивает дом

Набор данных, приведенных в табл. 3.2, довольно близок к набору из табл. 3.1, поэтому мы можем без опасений использовать ту же формулу. Разница лишь в том, что сейчас цены не совсем такие, как указано в формуле, и есть небольшое рассогласование. Мы можем записать уравнение следующим образом:

цена = $100 + 50 \times \text{количество комнат}$ + небольшое рассогласование.

Если мы хотим спрогнозировать цены, можем применить это уравнение. Даже если не уверены, что получим фактическое значение, мы знаем, что, скорее всего, приблизимся к нему. Теперь вопрос в том, как мы нашли это уравнение? И, самое главное, как его найти компьютеру?

Вернемся к графику и посмотрим, что на нем означает наше уравнение. Что произойдет, если мы посмотрим на все точки, в которых вертикальная координата (y) в $100 + 50$ раз превышает горизонтальную координату (x)? Этот набор точек образует прямую с наклоном 50 и пересечение с осью Y (y -пересечение) в точке 100. Прежде чем развернуть предыдущее утверждение, дадим определения наклона, y -пересечения и уравнения прямой. Мы рассмотрим их подробнее в разделе «Ускоренный курс по наклону и y -пересечению».

НАКЛОН прямой — показатель того, насколько она отвесная. Он рассчитывается делением подъема на выброс (то есть мы делим то, на сколько единиц прямая поднимается, на то, на сколько единиц она проходит вправо). Это соотношение постоянно по всей прямой. В модели машинного обучения это вес соответствующего признака, который говорит нам, какое значение подъема метки мы ожидаем, когда увеличиваем значение признака на единицу. Если прямая горизонтальна, то наклон равен нулю, а если идет вниз, то он отрицательный.

Y-ПЕРЕСЕЧЕНИЕ — это высота, на которой прямая пересекает вертикальную ось (Y). В модели машинного обучения это смещение говорит нам, какой будет метка в точке данных, где все признаки точно равны нулю.

ЛИНЕЙНОЕ УРАВНЕНИЕ — это уравнение прямой. Оно задается двумя параметрами — наклоном и y -пересечением. Если наклон равен m , а y -пересечение равно b , то уравнение прямой выглядит так: $y = mx + b$, а линия образована всеми точками (x, y) , которые удовлетворяют уравнению. В модели машинного обучения x — это значение признака, а y — прогноз для метки. Вес и смещение модели равны m и b соответственно.

Теперь можем проанализировать уравнение. Когда мы говорим, что наклон прямой равен 50, это означает, что каждый раз, добавляя в дом одну комнату, мы вычисляем, что цена дома вырастет на 50 долларов. Когда говорим, что y -пересечение прямой равно 100, это означает, что стоимость гипотетического дома с нулевым количеством комнат базовая — 100 долларов. Эта прямая изображена на рис. 3.4.

Итак, почему из всех возможных прямых (каждая с собственным уравнением) мы выбрали именно эту? Потому что она проходит близко к точкам. Может быть, существует вариант и получше, но мы по крайней мере знаем, что этот вариант хорош, в отличие от того, который вообще не приближается к точкам. Теперь возвратимся к первоначальной задаче, где имеется набор домов и нужно построить дорогу как можно ближе к ним.

Как найти эту прямую? Мы рассмотрим это позже в данной главе. Пока же предположим, что у нас есть волшебный хрустальный шар, который для множества точек находит прямую, проходящую ближе всего к ним.

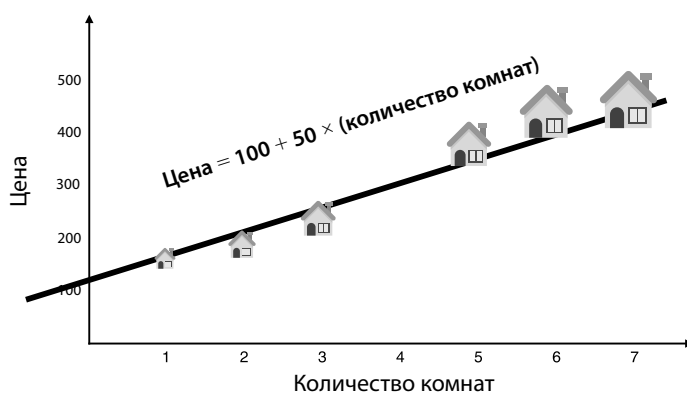


Рис. 3.4. Модель, которую мы формулируем, представляет собой прямую, проходящую как можно ближе ко всем домам

Шаг прогнозирования: что мы делаем, когда на продажу выставляется новый дом?

Теперь перейдем к использованию нашей модели для прогнозирования цены дома с четырьмя комнатами. Мы включаем число 4 в качестве признака в составленную ранее формулу, чтобы получить следующее:

$$\text{цена} = 100 + 50 \times 4 = 300.$$

Таким образом, модель предсказывает, что дом стоит 300 долларов. Это можно увидеть наглядно с помощью прямой, как показано на рис. 3.5.

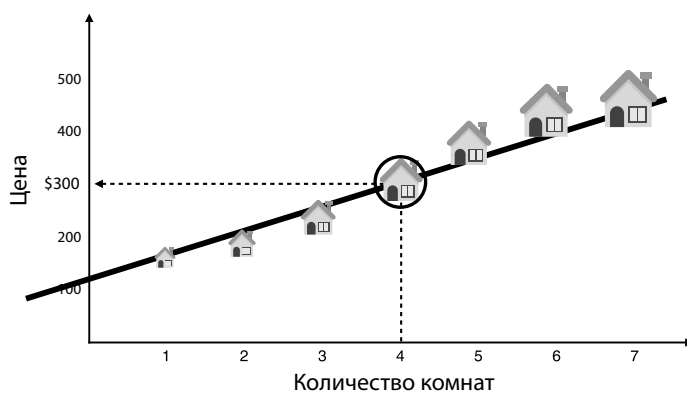


Рис. 3.5. Используя модель (прямую), делаем вывод, что прогнозируемая цена дома составляет 300 долларов

А если у нас будет больше переменных? Многомерная линейная регрессия

В предыдущих разделах вы узнали о модели, которая предсказывает цену дома на основе одного признака — количества комнат. Можно представить себе множество других признаков, которые могли бы помочь предсказать цену дома, таких как размер и уровень школы по соседству и возраст дома. Может ли наша модель линейной регрессии учитывать новые переменные? Несомненно. Когда единственным признаком служит количество комнат, модель прогнозирует цену как сумму характеристик, умноженных на их веса, плюс смещение. Если у нас больше функций, то нужно лишь умножить их на соответствующие веса и добавить результаты к прогнозируемой цене. Таким образом, модель цены дома может выглядеть следующим образом:

$$\begin{aligned} \text{цена} = & 30 \times \text{количество комнат} + 1,5 \times \text{размер школы} + \\ & + 10 \times \text{уровень школы} - 2 \times \text{возраст дома} + 50. \end{aligned}$$

Почему в этом уравнении все веса положительны, за исключением того, который соответствует возрасту дома? Причина кроется в том, что три другие характеристики (количество комнат, размер и качество школы) *положительно коррелируют* с ценой дома. Другими словами, поскольку удачно расположенные дома большего размера стоят дороже, то чем больше эти признаки, тем выше ожидаемая итоговая цена дома. Однако поскольку мы предполагаем, что старые дома, как правило, дешевле новых, то признак возраста *отрицательно коррелирует* с ценой дома.

Что делать, если вес признака равен нулю? Это происходит, когда он не имеет отношения к цене. Например, представьте признак, который задает количество соседей, фамилия которых начинается с буквы «А». Он по большому счету не имеет отношения к цене дома, поэтому мы ожидаем, что в разумной модели вес, соответствующий этой функции, будет либо нулевым, либо очень близким к нулю.

Аналогично, если признак имеет очень высокий вес (отрицательный или положительный), мы интерпретируем этот факт следующим образом: модель сообщает нам о том, что данный признак важен при определении цены дома. Похоже, что в предыдущей модели количество комнат — важный признак, потому что его вес самый большой (в абсолютном значении).

В разделе «Понижение размерности упрощает данные без потери слишком большого объема информации» в главе 2 мы связали количество столбцов в наборе данных с измерением, в котором расположен этот набор. Таким образом, набор данных с двумя столбцами может быть представлен как набор точек на плоскости, а набор данных с тремя столбцами — как набор точек в трехмерном пространстве. В таком наборе данных модель линейной регрессии соответствует уже не прямой, а плоскости, которая проходит как можно ближе к точкам. Представьте, что в комнате неподвижно замерло множество мух и наша задача состоит в том, чтобы попытаться расположить гигантский фанерный лист как

можно ближе ко всем ним. Это многомерная линейная регрессия с тремя переменными. Проблему становится трудно визуализировать для наборов данных с большим количеством столбцов, но мы всегда можем представить себе линейное уравнение со многими переменными.

В этой главе мы в основном имеем дело с обучением моделей линейной регрессии только с одним признаком, но процедура будет аналогичной и для большего количества признаков. Помните об этом во время чтения и попробуйте представить, как бы вы распространили каждое из следующих утверждений на случай с несколькими признаками.

Несколько быстрых ответов на возникшие вопросы

Допускаю, что у вас в голове уже роится множество вопросов. Рассмотрим некоторые (надеюсь, все) из них.

1. Что произойдет, если модель допустит ошибку?
2. Как мы пришли к формуле, которая предсказывает цену? И что бы мы делали, если бы вместо шести домов их были тысячи?
3. Допустим, мы построили модель прогнозирования, после чего на рынке начали появляться новые дома. Есть ли способ дополнить модель новой информацией?

Далее мы подробно ответим на все эти вопросы, а пока вот несколько кратких ответов.

1. Что произойдет, если модель допустит ошибку?

Модель оценивает стоимость дома, поэтому мы ожидаем, что она почти всегда будет допускать небольшие ошибки, так как очень трудно определить точную цену. Процесс обучения состоит в том, чтобы найти модель, которая допускает наименьшие ошибки в наших точках.

2. Как мы пришли к формуле, которая предсказывает цену? И что бы мы делали, если бы вместо шести домов были тысячи?

Да, это главный вопрос, который мы рассматриваем в этой главе! Когда у нас шесть домов, задача провести прямую, которая проходит рядом с ними, проста, но, если домов тысячи, задача усложняется. В этой главе мы разрабатываем алгоритм или процедуру, позволяющую компьютеру находить верную прямую.

3. Допустим, мы построили модель прогнозирования, после чего на рынке начали появляться новые дома. Есть ли способ дополнить модель новой информацией?

Конечно! Мы построим модель таким образом, чтобы ее можно было легко обновлять при появлении новых данных. В машинном обучении это одна из главных задач. Если мы построили модель таким образом, что всю ее нужно пересчитывать каждый раз при поступлении новых данных, то она окажется не очень полезной.

Как заставить компьютер провести прямую. Алгоритм линейной регрессии

Переходим к главному вопросу главы: как нам заставить компьютер провести прямую, которая проходит действительно близко к точкам? Сделаем это так же, как все остальное, что мы делаем в машинном обучении, — шаг за шагом. Начните со случайной прямой и придумайте способ *немного* ее улучшить, переместив ближе к точкам. Повторяйте этот процесс раз за разом, и — вуаля! — у вас получится желаемая прямая. Этот процесс называется алгоритмом линейной регрессии.

Процедура может показаться простенькой, но она действительно хорошо работает. Начните со случайной прямой. Выберите случайную точку в наборе данных и переместите прямую немного ближе к ней. Повторяйте этот процесс много раз, при этом всегда выбирая случайную точку в наборе данных. Далее приведен псевдокод для алгоритма линейной регрессии, реализованной подобным геометрическим способом. Процесс показан на рис. 3.6. Мы начинаем в левом верхнем углу со случайной прямой и заканчиваем в левом нижнем углу прямой, которая хорошо соответствует набору данных. На каждом этапе происходят два действия: мы выбираем случайную точку, а затем точка просит прямую приблизиться к ней. После множества итераций прямая примет удачное положение. На рисунке для простоты показаны только три итерации, но в реальной жизни их потребуется гораздо больше.

Псевдокод для алгоритма линейной регрессии (геометрический)

Входные данные — набор точек на плоскости.

Результат — прямая, проходящая близко к точкам.

Процедура

- Выберите случайную прямую.
- Повторяйте многократно:
 - выберите случайную точку данных;
 - переместите прямую немного ближе к ней.

Возврат — полученная прямая.

Это был своего рода взгляд с высоты птичьего полета. Чтобы изучить процесс подробнее, нужно углубиться в математику. Начнем с определения некоторых переменных:

- p — цена дома в наборе данных;
- $\hat{p}$ — прогнозируемая цена дома;
- r — количество комнат;
- m — цена за комнату;
- b — базовая цена за дом.

Почему мы видим шляпку над прогнозируемой ценой $\hat{p}$? На протяжении всей этой книги она указывает на то, что это переменная, которую предсказывает

наша модель. Таким образом, на основе набора данных мы можем определить фактическую цену дома по его прогнозируемой цене.

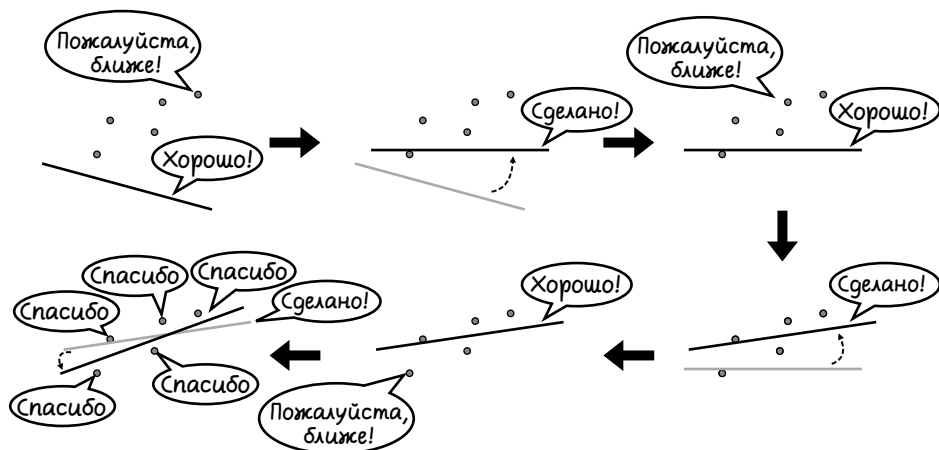


Рис. 3.6. Иллюстрация алгоритма линейной регрессии

Итак, уравнение модели линейной регрессии, которая предсказывает цену как базовую стоимость плюс цену за комнату, умноженную на количество комнат, выглядит так:

$$\hat{p} = mr + b.$$

Это формальный способ сказать:

прогнозируемая цена = цена за комнату × количество комнат +
+ базовая цена дома.

Чтобы получить представление об алгоритме линейной регрессии, представьте, что у нас есть модель, в которой цена за комнату составляет 40 долларов, а базовая цена дома — 50 долларов. Такая модель предсказывает цену дома, используя следующую формулу:

$$\hat{p} = 40r + 50.$$

Проиллюстрируем алгоритм линейной регрессии: представьте, что в нашем наборе данных есть дом с двумя комнатами, который стоит 150 долларов. Модель предсказывает, что цена дома составляет $50 + 40 \times 2 = 130$ долларов. Это неплохой прогноз, но он ниже, чем цена дома. Как можно улучшить модель? Похоже, ее ошибка в том, что она думает: дом слишком дешевый. Возможно, у модели низкая базовая цена, или низкая цена за комнату, или и то, и другое. Если мы немного увеличим и то и другое, то сможем получить более точную оценку. Давайте увеличим цену за комнату

на 0,5 доллара, а базовую цену — на 1 доллар (цифры взяты с потолка). Новое уравнение выглядит так:

$$\hat{p} = 40,5r + 51.$$

Новая прогнозируемая цена дома составляет $40,5r + 51 = 132$. Поскольку 132 доллара ближе к 150 долларам, новая модель дает лучший прогноз для этого дома. Следовательно, это лучшая модель для этой точки данных. Мы не знаем, является ли она лучшей моделью для других точек данных, но давайте пока не будем беспокоиться об этом. Идея алгоритма линейной регрессии состоит в том, чтобы многократно повторять описанный процесс. Псевдокод алгоритма линейной регрессии выглядит так.

Псевдокод для алгоритма линейной регрессии

Входные данные — набор данных точек.

Результат — модель линейной регрессии, соответствующая этому набору данных.

Процедура

- Выберите модель со случайными весами и случайным смещением.
- Повторяйте многократно:
 - выберите случайную точку данных;
 - слегка отрегулируйте веса и смещение, чтобы улучшить прогноз для конкретной точки данных.

Возврат — полученная модель.

У вас могут возникнуть несколько вопросов, например, такие:

- На сколько я должен отрегулировать вес?
- Сколько раз я должен повторить алгоритм? Другими словами, как мне узнать, когда заканчивать?
- Откуда я знаю, что алгоритм работает?

Далее мы ответим на все эти вопросы. В разделах «Квадратический подход...» и «Абсолютный подход» разберем несколько интересных приемов, помогающих найти хорошие значения для корректировки весов. В разделах «Абсолютная погрешность...» и «Квадратическая погрешность...» увидим функцию ошибки, которая поможет решить, когда остановить алгоритм. И наконец, в разделе «Градиентный спуск...» рассмотрим одноименный эффективный метод, который объясняет, почему этот алгоритм работает. Но начнем мы с перемещения прямой на плоскости.

Ускоренный курс по наклону и y-пересечению

В разделе «Следующий шаг: формулирование правила, которое оценивает дом» мы говорили об уравнении прямой. Здесь же узнаем, как управлять этим уравнением для перемещения прямой. Напомним, что уравнение прямой имеет две составляющие:

- наклон;
- y -пересечение.

Наклон показывает, насколько крута прямая, а y -пересечение — где она расположена. Наклон определяется как подъем, деленный на выброс, а y -пересечение показывает, где прямая пересекает ось Y (вертикальную ось). На примере на рис. 3.7 мы можем видеть и то и другое. Уравнение этой прямой:

$$y = 0,5x + 2.$$

Что же оно означает? То, что наклон равен 0,5, а y -пересечение равно 2.

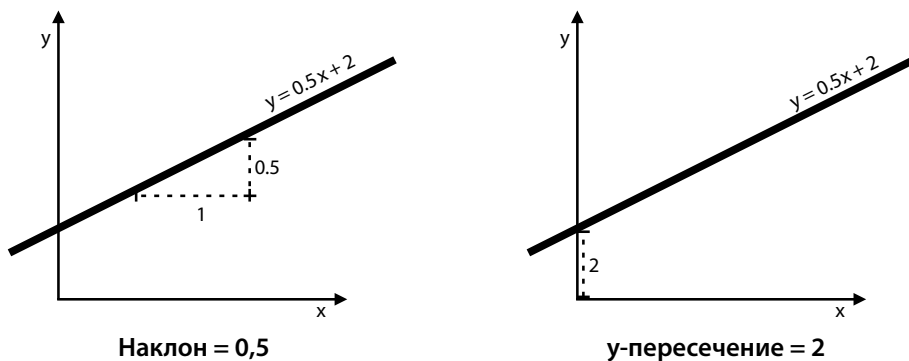


Рис. 3.7. Прямая $y = 0,5x + 2$: слева — наклон 0,5; справа — y -пересечение 2

Когда мы говорим, что наклон равен 0,5, это означает, что при движении вдоль этой прямой на каждую единицу, на которую перемещаем вправо, мы поднимаемся на 0,5 единицы. Наклон может быть нулевым, если мы не движемся вверх, или отрицательным, если опускаемся. Вертикальные прямые имеют неопределенный наклон, но, как правило, в линейной регрессии они не встречаются. Многие прямые могут иметь одинаковый наклон. Если я проведу любую прямую, параллельную линии, изображенной на рис. 3.7, она также будет подниматься на 0,5 единицы за каждую единицу, на которую мы перемещаемся вправо. Вот тут-то и вступает в дело y -пересечение: оно указывает, где линия пересекает ось Y . Прямая пересекает ось Y на высоте 2, это и есть y -пересечение.

Другими словами, наклон указывает направление, в котором построена прямая, а y -пересечение — ее местоположение. Обратите внимание на то, что при указании наклона и y -пересечения прямая задана полностью. На рис. 3.8 мы видим разные прямые с одинаковым y -пересечением и разные прямые с одинаковым наклоном. Слева изображены несколько прямых с одинаковым y -пересечением и разными наклонами. Чем больше наклон, тем круче прямая. Справа — несколько прямых с одинаковым наклоном и разными y -пересечениями. Чем больше y -пересечение, тем выше расположена прямая.

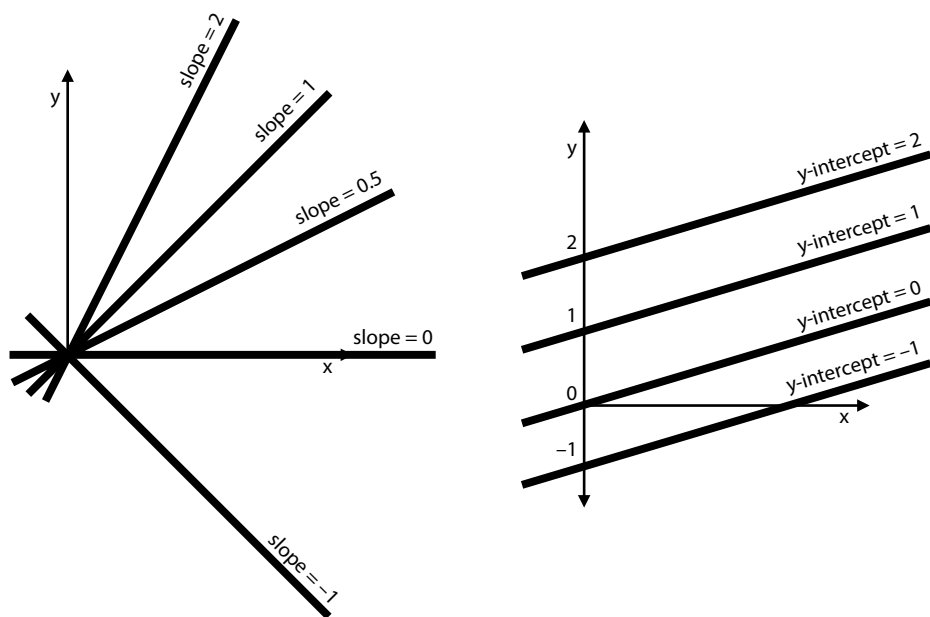


Рис. 3.8. Примеры наклона и у-пересечения

В рассматриваемом примере о жилье наклон представляет цену за комнату, а у-пересечение — базовую цену дома. Будем иметь это в виду при манипуляциях с прямыми и подумаем о том, как это влияет на нашу модель цен на жилье.

Из определений наклона и у-пересечения можно сделать следующие выводы.

Изменение наклона (рис. 3.9, *слева*)

- Если увеличить наклон прямой, она повернется против часовой стрелки.
- Если уменьшить наклон прямой, она повернется по часовой стрелке.

Повороты происходят на оси вращения, а именно в точке пересечения прямой и оси Y .

Изменение у-пересечения (см. рис. 3.9, *справа*)

- Если увеличить у-пересечение, прямая переместится вверх.
- Если уменьшить у-пересечение, прямая переместится вниз.

Рисунок 3.9 иллюстрирует эти повороты и перемещения, что пригодится, когда мы захотим скорректировать свои модели линейной регрессии.

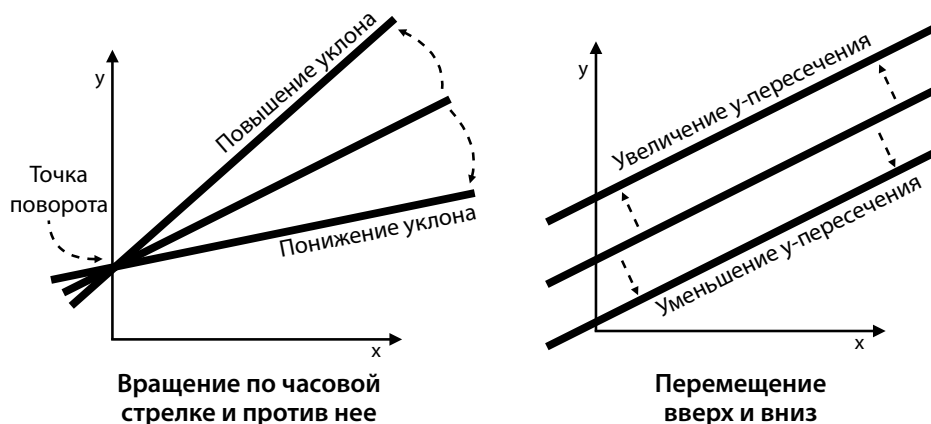


Рис. 3.9. Изменение наклона и y-пересечения

Как объяснялось ранее, в общем случае уравнение прямой записывается как $y = mx + b$, где x и y соответствуют координатам по горизонтали и вертикали, m — это наклон, а b — y-пересечение. На протяжении всей главы, придерживаясь этих обозначений, мы будем записывать уравнение в виде $\hat{p} = mr + b$, где $\hat{p}$ соответствует прогнозируемой цене, r — количеству комнат, m (наклон) — цене за комнату, а b (y-пересечение) — базовой цене дома.

Простой подход для перемещения прямой ближе к набору точек, по одной точке за раз

Напомним, что алгоритм линейной регрессии состоит из повторения шага, на котором мы перемещаем прямую ближе к точке. Можем сделать это с помощью вращений и перемещений. В этом разделе мы изучим подход, называемый *простым*, который состоит в том, чтобы слегка повернуть и переместить прямую в направлении точки и тем самым сблизить их (рис. 3.10).

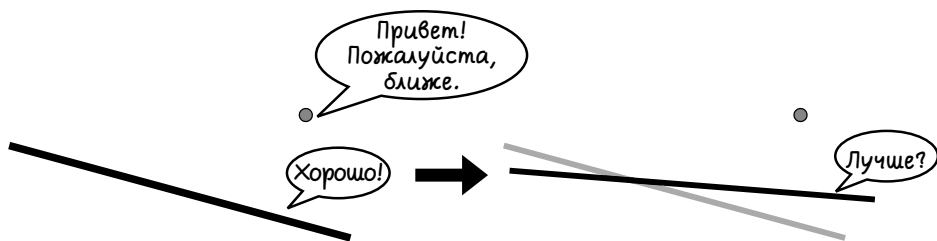


Рис. 3.10. Наша цель — немного повернуть и переместить прямую, чтобы приблизить ее к точке

Хитрость правильного перемещения прямой к точке состоит в том, чтобы определить, где находится точка относительно прямой. Если она расположена выше прямой, то нужно переместить прямую вверх, а если ниже, то вниз. Вращение немного сложнее, но поскольку ось вращения — это точка пересечения линии и оси Y , мы понимаем, что если точка выше прямой и справа от оси Y или ниже прямой и слева от оси Y , то нужно повернуть прямую против часовой стрелки. В двух других сценариях вращать ее нужно по часовой стрелке. Итак, вот четыре сценария, которые проиллюстрированы на рис. 3.11.

Сценарий 1. Если точка находится над прямой и справа от оси Y , вращаем прямую против часовой стрелки и перемещаем ее вверх.

Сценарий 2. Если точка находится над прямой и слева от оси Y , вращаем прямую по часовой стрелке и перемещаем ее вверх.

Сценарий 3. Если точка находится ниже прямой и справа от оси Y , вращаем прямую по часовой стрелке и перемещаем ее вниз.

Сценарий 4. Если точка находится ниже прямой и слева от оси Y , вращаем прямую против часовой стрелки и перемещаем ее вниз.

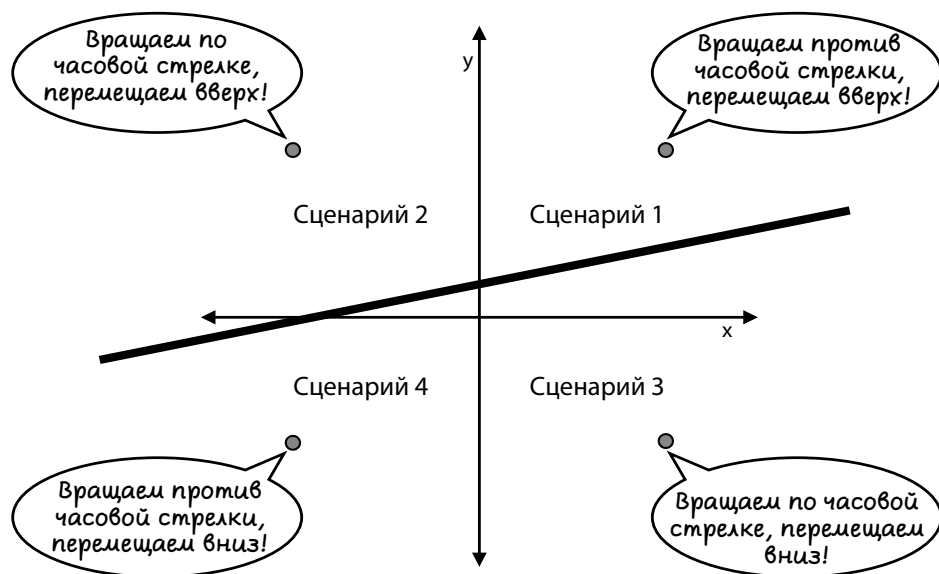


Рис. 3.11. Четыре сценария. В каждом из них мы должны повернуть и переместить прямую по-своему, чтобы приблизить ее к соответствующей точке

Теперь, когда у нас есть эти четыре сценария, можем написать псевдокод для простого подхода. Но сначала поясним некоторые обозначения. В этом разделе мы говорили о прямых, построенных в соответствии с уравнением $y = mx + b$,

где m — наклон, а b — y -пересечение. В примере с жильем использовали аналогичную нотацию:

- точка с координатами (r, p) соответствует дому с r комнатами и ценой p ;
- наклон m соответствует цене за комнату;
- y -пересечение b соответствует базовой цене дома;
- прогноз $\hat{p} = mr + b$ соответствует прогнозируемой цене дома.

Псевдокод для простого подхода

Входные данные:

- линия с наклоном m , y -пересечением b и уравнением $\hat{p} = mr + b$;
- точка с координатами (r, p) .

Результат — линия $\hat{p} = m'r + b$, которая ближе к точке.

Процедура: выберите два очень маленьких случайных числа и назовите их η_1 и η_2 (греческая буква «эта»).

Сценарий 1. Если точка находится над прямой и справа от оси Y , вращаем прямую против часовой стрелки и перемещаем ее вверх.

- Добавляем η_1 к наклону m , получаем $m' = m + \eta_1$.
- Добавляем η_2 к y -пересечению b , получаем $b' = b + \eta_2$.

Сценарий 2. Если точка находится над прямой и слева от оси y , вращаем прямую по часовой стрелке и перемещаем ее вверх.

- Вычитаем η_1 из наклона m , получаем $m' = m - \eta_1$.
- Добавляем η_2 к y -пересечению b , получаем $b' = b + \eta_2$.

Сценарий 3. Если точка находится ниже прямой и справа от оси Y , вращаем прямую по часовой стрелке и перемещаем ее вниз.

- Вычитаем η_1 из наклона m , получаем $m' = m - \eta_1$.
- Вычитаем η_2 из y -пересечения b , получаем $b' = b - \eta_2$.

Сценарий 4. Если точка находится ниже прямой и слева от оси Y , вращаем прямую против часовой стрелки и перемещаем ее вниз.

- Добавляем η_1 к наклону m , получаем $m' = m + \eta_1$.
- Вычитаем η_2 из y -пересечения b , получаем $b' = b - \eta_2$.

Возврат — прямая $\hat{p} = m'r + b'$.

Обратите внимание на то, что в нашем примере добавление небольшого числа к наклону или вычитание из него означает увеличение или уменьшение цены за комнату. Аналогично добавление небольшого числа к y -пересечению или вычитание из него означает увеличение или уменьшение базовой цены дома. Кроме того, поскольку координата x — количество комнат, это число не может быть отрицательным. Таким образом, здесь важны только сценарии 1 и 3, а это значит, что мы можем обобщить простой подход так.

Простой подход

- Если модель дала цену за дом, которая ниже фактической цены, добавьте небольшую случайную сумму к цене за комнату и к базовой цене дома.
- Если модель дала цену за дом, которая выше фактической цены, вычтите небольшую случайную сумму из цены за комнату и базовой цены дома.

Такой подход позволит привести к определенному успеху на практике, но это далеко не лучший способ перемещения прямых. К его работе могут возникнуть некоторые вопросы, например:

- Можем ли мы выбрать лучшие значения для η_1 и η_2 ?
- Можем ли мы превратить эти четыре случая в два или, возможно, один?

Ответ на оба вопроса — да, и в двух следующих разделах мы увидим, как это делается.

Квадратический подход — более умный способ приблизить прямую к одной из точек

В этом разделе рассмотрим эффективный способ приблизить прямую к точке. Я называю его *квадратическим подходом*. Напомним, что простой подход состоял из четырех случаев, которые основывались на положении точки относительно прямой. Квадратический подход сводит эти четыре случая к одному, найдя значения с правильными знаками (+ или −) для добавления к наклону и у-пересечению, чтобы прямая всегда приближалась к точке.

Начнем с у-пересечения. Обратите внимание на следующие два наблюдения.

- **Наблюдение 1.** При простом подходе, когда точка находится над прямой, мы добавляем небольшое количество к у-пересечению. Когда она находится ниже прямой, вычитаем небольшое количество.
- **Наблюдение 2.** Если точка находится выше прямой, значение $p - \hat{p}$ (разница между ценой и прогнозируемой ценой) является положительным, если ниже прямой — отрицательным. Это наблюдение проиллюстрировано на рис. 3.12.

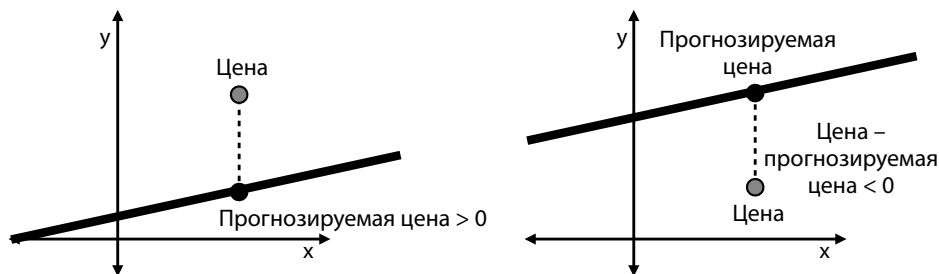


Рис. 3.12. Слева — когда точка находится выше прямой, цена больше прогнозируемой цены, поэтому разница положительна. Справа — когда точка находится ниже прямой, цена меньше прогнозируемой цены, поэтому разница отрицательна

Объединяя наблюдения 1 и 2, мы приходим к выводу: если добавить разность $p - \hat{p}$ к y -пересечению, линия всегда будет двигаться к точке, потому что это значение положительное, когда точка находится выше линии, и отрицательное, когда ниже. Однако в машинном обучении мы всегда стремимся делать небольшие шаги. В связи с этим вводим важную концепцию — скорость обучения.

СКОРОСТЬ ОБУЧЕНИЯ — очень маленькое число, которое мы выбираем перед обучением модели. Оно помогает убедиться, что модель при обучении меняется по чуть-чуть. В этой книге скорость обучения будет обозначаться η .

Поскольку скорость обучения невелика, таково же и значение $\eta(p - \hat{p})$. Это значение, которое мы добавляем к y -пересечению, чтобы переместить прямую в направлении точки.

Значение, которое нужно добавить к наклону, аналогично, но оно немного сложнее. Обратите внимание на следующие два наблюдения.

- **Наблюдение 3.** При простом подходе, когда расположение точки соответствует сценарию 1 или 4 (выше прямой и справа от вертикальной оси или ниже прямой и слева от вертикальной оси), вращаем прямую против часовой стрелки. В противном случае (сценарий 2 или 3) — по часовой стрелке.
- **Наблюдение 4.** Если точка (r, p) находится справа от вертикальной оси, то значение r положительное. Если точка находится слева от вертикальной оси, то r отрицательное. Это наблюдение проиллюстрировано на рис. 3.13. Обратите внимание на то, что в этом примере r никогда не будет отрицательным, потому что это количество комнат. Однако в другом случае признак может быть отрицательным.

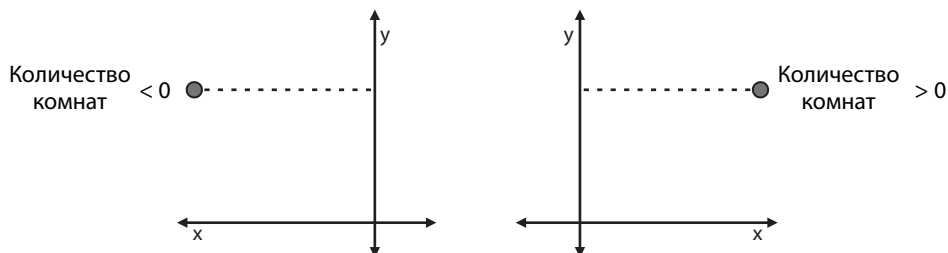


Рис. 3.13. Количество комнат: *слева* — отрицательное, когда точка находится слева от оси Y ; *справа* — положительное, когда точка находится справа от оси Y

Рассмотрим значение $r(p - \hat{p})$. Оно положительно, когда r и $p - \hat{p}$ или оба отрицательны, или оба положительны. Это именно то, что происходит в сценариях 1 и 4. Аналогично, $r(p - \hat{p})$ отрицательно в сценариях 2 и 3. Следовательно, в соответствии с наблюдением 4 это та величина, которую нужно добавить к наклону. Мы хотим, чтобы это значение было небольшим, поэтому снова умножаем его на скорость обучения и приходим к выводу, что добавление $\eta r(p - \hat{p})$ к наклону всегда будет перемещать прямую в направлении точки.

Теперь мы можем написать псевдокод для квадратического подхода.

Псевдокод для квадратического подхода

Входные данные:

- прямая с наклоном m , y -пересечением b и уравнением $\hat{p} = mr + b$;
- точка с координатами (r, p) ;
- небольшое положительное значение η (скорость обучения).

Результат — прямая $\hat{p} = m'r + b'$, которая находится ближе к точке.

Процедура

- Добавляем $\eta r(p - \hat{p})$ к наклону m , получаем $m' = m + \eta r(p - \hat{p})$ (при этом прямая вращается).
- Добавляем $\eta r(p - \hat{p})$ к y -пересечению b , получаем $b' = b + \eta(p - \hat{p})$ (это перемещает прямую).

Возврат — прямая $\hat{p} = m'r + b'$.

Теперь мы готовы закодировать этот алгоритм на Python! Код для этого раздела следующий.

- Блокнот `Coding_linear_regression.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_3_Linear_Regression/Coding_linear_regression.ipynb

А вот код для квадратического подхода:

```
def square_trick(base_price, price_per_room, num_rooms, price,
                 learning_rate):
    predicted_price = base_price + price_per_room*num_rooms
    base_price += learning_rate*(price-predicted_price)
    price_per_room += learning_rate*num_rooms*(price-predicted_price)
    return price_per_room, base_price
```

Вычисляет прогноз

Перемещает прямую

Вращает прямую

Абсолютный подход — еще один полезный способ приближения прямой к точкам

Квадратический подход эффективен, но другой полезный подход, который мы называем *абсолютным*, является промежуточным звеном между простым и квадратическим подходами. В квадратическом подходе мы использовали две величины, $p - \hat{p}$ (цена – прогнозируемая цена) и r (количество комнат), чтобы свести четыре сценария к одному. В абсолютном подходе применяем только r , чтобы свести четыре случая к двум. Другими словами, вот так выглядит абсолютный подход.

Псевдокод для абсолютного подхода**Входные данные:**

- прямая с наклоном m , y -пересечением b и уравнением $\hat{p} = mr + b$;
- точка с координатами (r, p) ;
- небольшое положительное значение η (скорость обучения).

Результат — прямая $\hat{p} = m'r + b'$, которая находится ближе к точке.

Процедура

Случай 1. Если точка находится над прямой, то есть если $p > \hat{p}$:

- добавляем ηr к наклону m , получаем $m' = m + \eta r$ (при этом прямая поворачивается против часовой стрелки, если точка находится справа от оси Y , и по часовой стрелке, если слева);
- добавляем η к y -пересечению b , получаем $b' = b + \eta$ (это перемещает прямую вверх).

Случай 2. Если точка находится ниже прямой, то есть если $p < \hat{p}$:

- вычитаем ηr из наклона m , получаем $m' = m - \eta r$ (при этом прямая поворачивается по часовой стрелке, если точка находится справа от оси Y , и против часовой стрелки, если слева);
- вычитаем η из y -пересечения b , получаем $b' = b - \eta$ (это перемещает прямую вниз).

Возврат — прямая $\hat{p} = m'r + b'$.

Вот код для абсолютного подхода:

```
def absolute_trick(base_price, price_per_room, num_rooms, price,
    learning_rate):
    predicted_price = base_price + price_per_room*num_rooms
    if price > predicted_price:
        price_per_room += learning_rate*num_rooms
        base_price += learning_rate
    else:
        price_per_room -= learning_rate*num_rooms
        base_price -= learning_rate
    return price_per_room, base_price
```

Удостоверьтесь самостоятельно, что сумма, добавляемая к каждому из весов, действительно имеет правильный знак, как мы сделали с квадратическим подходом.

Алгоритм линейной регрессии. Многократное повторение абсолютного или квадратического подхода для приближения прямой к точкам

Теперь, сделав всю тяжелую работу, мы готовы разработать алгоритм линейной регрессии! Этот алгоритм принимает в качестве входных данных набор точек и возвращает прямую, которая хорошо им соответствует. Он

состоит в том, чтобы начать со случайных значений для существующих наклона и у-пересечения, а затем многократно повторять процедуру их обновления с использованием абсолютного или квадратического подхода. Вот его псевдокод.

Псевдокод для алгоритма линейной регрессии

Входные данные — набор данных домов с указанием количества комнат и цен.

Результат — веса модели — цена за комнату и базовая цена.

Процедура

- Начинаем со случайных значений для наклона и у-пересечения.
- Повторяем многократно:
 - выбираем случайную точку данных;
 - обновляем наклон и у-пересечение с помощью абсолютного или квадратического подхода.

Каждая итерация цикла называется *периодом*, и мы устанавливаем это число в начале нашего алгоритма. Простой подход в основном использовался для иллюстрации — как упоминалось ранее, он работает не очень хорошо. В реальной жизни мы применяем абсолютный или квадратический подход, которые работают гораздо лучше. В реальности используется и то и другое, но квадратический подход более популярен. Поэтому в нашем алгоритме будем применять именно его, но, если хотите, можете взять и абсолютный подход.

Вот код для алгоритма линейной регрессии. Обратите внимание на то, что мы задействовали пакет Python `random` для генерации случайных чисел для начальных значений (наклон и у-пересечение) и выбора точек внутри цикла:

```
import random
def linear_regression(features, labels, learning_rate=0.01, epochs = 1000):
    price_per_room = random.random()
    base_price = random.random()
    for epoch in range(epochs):
        i = random.randint(0, len(features)-1)
        num_rooms = features[i]
        price = labels[i]
        price_per_room, base_price = square_trick(base_price,
                                                    price_per_room,
                                                    num_rooms,
                                                    price,
                                                    learning_rate=learning_rate)
    return price_per_room, base_price
```

Импортируем пакет `random` для генерации случайных (псевдослучайных) чисел

Генерируем случайные значения для наклона и у-пересечения

Выбираем случайную точку в наборе данных

Повторяем шаг обновления много раз

Применяем квадратический подход для перемещения линии ближе к точке

Следующий шаг — запустить этот алгоритм для построения модели, соответствующей нашему набору данных.

Загрузка данных и построение их графика

На протяжении всей этой главы мы загружаем и строим графики данных и моделей, используя Matplotlib и NumPy — два очень полезных пакета Python. NumPy применяем для хранения массивов и выполнения математических операций, а Matplotlib — для создания графиков.

В первую очередь мы кодируем признаки и метки набора данных, приведенных в табл. 3.2, в виде массивов NumPy следующим образом:

```
import numpy as np
features = np.array([1,2,3,5,6,7])
labels = np.array([155, 197, 244, 356, 407, 448])
```

Далее строим график набора данных. В репозитории в файле `utils.py` есть некоторые функции для построения кода, на который вам предлагается взглянуть. График набора данных показан на рис. 3.14. Обратите внимание на то, что точки практически образуют прямую.

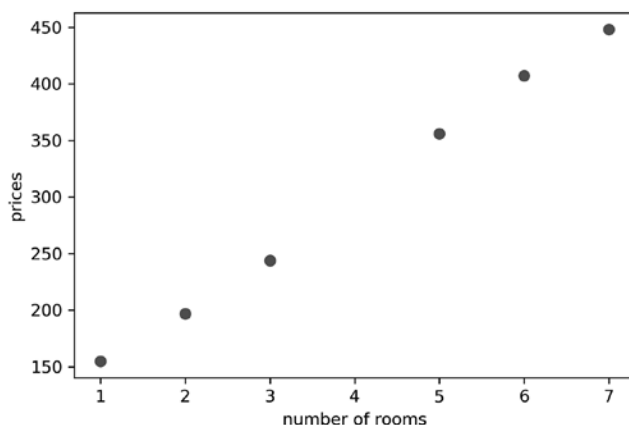


Рис. 3.14. График точек, приведенных в табл. 3.2

Используем алгоритм линейной регрессии в наборе данных

Теперь применим алгоритм, чтобы приблизить прямую к точкам. Следующая строка кода запускает алгоритм с признаками, метками, скоростью обучения, равной 0,01, и количеством периодов, равным 10 000:

```
linear_regression(features, labels, learning_rate = 0.01, epochs = 10000)
```

Результатом служит график, показанный на рис. 3.15. На нем изображена прямая, где цена за комнату (округленная) составляет 51,05 доллара, а базовая цена — 99,1 доллара. Это недалеко от 50 и 100 долларов, которые мы рассматривали ранее в этой главе.



Рис. 3.15. График точек, приведенных в табл. 3.2, и прямая, полученная с помощью алгоритма линейной регрессии

Чтобы представить себе процесс, давайте еще раз взглянем на прогрессию. На рис. 3.16 мы видим несколько промежуточных прямых. Обратите внимание на то, что прямая начинается далеко от точек. По мере работы алгоритма она медленно сдвигается, чтобы с каждым разом соответствовать точкам все лучше и лучше. Сначала (в первые десять периодов) прямая быстро движется к хорошему решению. После 50-го периода она уже хорошая, но все еще неидеально подходит к точкам. Если мы позволим алгоритму работать в течение целых 10 000 периодов, то получим отличное положение прямой.

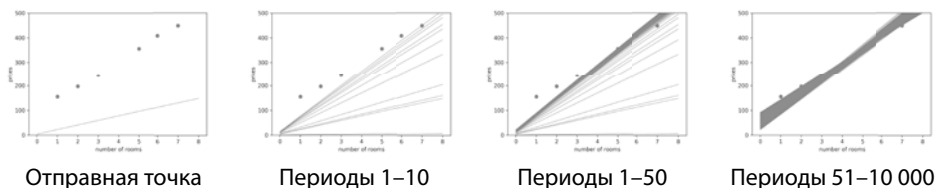


Рис. 3.16. Показаны некоторые прямые алгоритма линейной регрессии по мере приближения к наилучшему решению. На первом фрагменте показана отправная точка, на втором — первые десять периодов. Обратите внимание на то, как прямая приближается к точкам. На третьем фрагменте показаны первые 50 периодов, на четвертом — периоды от 51-го до 10 000-го (последний)

Использование модели для составления прогнозов

Теперь, когда у нас есть новенькая модель линейной регрессии, можем применять ее для составления прогнозов! Напомним из начала главы, что нашей целью было спрогнозировать цену дома с четырьмя комнатами. В предыдущем разделе мы запустили алгоритм и получили наклон (цена за комнату) 51,05 доллара и y -пересечение (базовая цена дома) 99,1 доллара. Таким образом, уравнение будет следующим:

$$\hat{p} = 51,05r + 99,1.$$

Прогноз, который модель делает для дома с $r = 4$ комнатами, равен:

$$\hat{p} = 51,05 \cdot 4 + 99,1 = 303,3.$$

Обратите внимание: 303,3 доллара — это недалеко от 300 долларов из начала главы!

Общий алгоритм линейной регрессии (опционально)

Это необязательный раздел, поскольку в нем основное внимание уделяется математическим деталям более абстрактного алгоритма, используемого для общего набора данных. Тем не менее рекомендую прочитать его, чтобы привыкнуть к обозначениям, которые встречаются в большей части литературы по машинному обучению.

В предыдущих разделах мы описали алгоритм линейной регрессии для набора данных с одним признаком. Но как можно предположить, в реальной жизни мы будем работать с наборами данных со многими признаками. Для этого нужен общий алгоритм. Хорошая новость: общий алгоритм несильно отличается от конкретного, с которым мы познакомились в этой главе. Единственное отличие заключается в том, что каждый из признаков обновляется таким же образом, как обновлялся наклон. В примере с жильем у нас был один наклон и одно y -пересечение. В общем случае подумайте о множестве наклонов и единственном y -пересечении.

Общий случай предусматривает набор данных, состоящий из m точек и n признаков. Таким образом, модель имеет m весов (думайте о них как об обобщении наклона) и одно смещение. Обозначения выглядят следующим образом:

- точки данных — это $x^{(1)}, x^{(2)} \dots x^{(m)}$. Каждая точка имеет вид $x^{(i)} = (x_1^{(i)}, x_2^{(i)} \dots x_n^{(i)})$;
- соответствующие метки — $y_1, y_2 \dots y_m$;
- веса модели — $w_1, w_2 \dots w_n$;
- смещение модели — b .

Псевдокод для общего квадратического подхода

Входные данные:

- модель $\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$;
- точка с координатами (x, y) ;
- небольшое положительное значение η (скорость обучения).

Результат — модель $\hat{y} = w'_1x_1 + w'_2x_2 + \dots + w'_nx_n + b'$, которая ближе к точке.

Процедура

- Добавляем $\eta(y - \hat{y})$ к у-пересечению b , получаем $b' = b + \eta(y - \hat{y})$.
- Для $i = 1, 2, \dots, n$ добавляем $\eta x_i (y - \hat{y})$ к весу w_i , получаем $w'_i = w_i + \eta x_i (y - \hat{y})$.

Возврат — модель $\hat{y} = w'_1x_1 + w'_2x_2 + \dots + w'_nx_n + b'$.

Псевдокод общего алгоритма линейной регрессии такой же, как в разделе «Алгоритм линейной регрессии», поскольку он состоит из повторения общего квадратического подхода, поэтому мы его опустим.

Как мы измеряем результаты? Функция ошибок

В предыдущих разделах мы разработали непосредственный подход к поиску наилучшего соответствия прямой. Однако во многих случаях проблемы в машинном обучении трудно решать с помощью прямого подхода. Косвенный, более механический способ сделать это — использовать *функцию ошибок*. Так называется показатель, который сообщает, как работает наша модель. Например, взгляните на две модели, изображенные на рис. 3.17. Та, что слева, — плохая, а та, что справа, — хорошая. Функция ошибок измеряет это, присваивая большое значение погрешности плохой модели и небольшое значение — хорошей. В литературе функции ошибок иногда называют *функциями потерь* или *функциями затрат*. В книге мы называем их функциями ошибок, за исключением некоторых особых случаев, когда иного требует чаще используемое название.

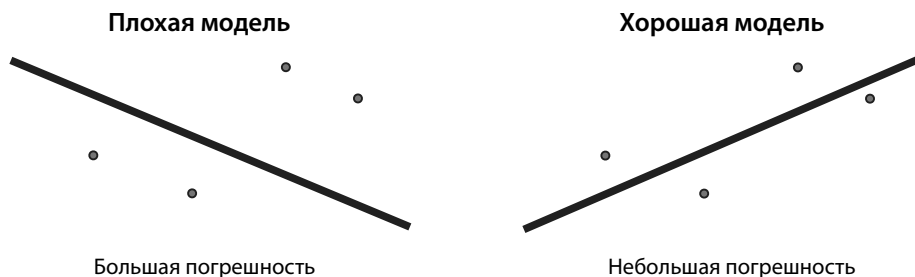


Рис. 3.17. Две модели: *слева* — плохая; *справа* — хорошая

Теперь вопрос в том, как мы определяем хорошую функцию ошибок для моделей линейной регрессии. Есть два распространенных способа сделать

это — вычисление *абсолютной погрешности* и *квадратической погрешности*. Если коротко, то абсолютная погрешность — это сумма вертикальных расстояний от прямой до точек в наборе данных, а квадратическая погрешность — это сумма квадратов этих расстояний.

Далее в нескольких разделах мы подробнее рассмотрим эти две функции ошибок. Затем увидим, как их уменьшить, используя метод, называемый градиентным спуском. Наконец, построим одну из функций ошибок в нашем примере и посмотрим, как быстро метод градиентного спуска помогает ее уменьшить.

Абсолютная погрешность. Сумма расстояний, которая говорит о качестве модели

В этом разделе мы рассмотрим абсолютную погрешность, которая служит показателем, сообщающим, насколько хороша модель. Абсолютная погрешность — это сумма расстояний между точками данных и прямой. Почему это называется абсолютной погрешностью? Чтобы вычислить каждое из расстояний, мы берем разницу между меткой и предсказанной меткой. Она может быть положительной или отрицательной в зависимости от того, находится точка выше или ниже прямой. Чтобы превратить эту разницу в число, всегда положительное, мы берем его абсолютное значение.

По определению, хорошая модель линейной регрессии — та, в которой прямая близка к точкам. Что означает *близка* в данном случае? Это субъективный вопрос, потому что прямая, которая близка к некоторым точкам, может быть далека от других. В таком случае не предпочтительнее ли выбрать прямую, которая находится очень близко к некоторым точкам и далеко от других? Или мы попытаемся выбрать ту, которая довольно близка ко всем точкам? Абсолютная погрешность помогает принять это решение. Прямая, которую мы выбираем, — это прямая, которая минимизирует абсолютную погрешность, то есть та, для которой сумма расстояний по вертикали от каждой из точек до прямой минимальна. На рис. 3.18 вы видите две прямые, их абсолютная погрешность показана как сумма вертикальных сегментов. Прямая слева имеет большую абсолютную погрешность, а та, что справа, — небольшую. Таким образом, между этими двумя мы бы выбрали ту, что справа.

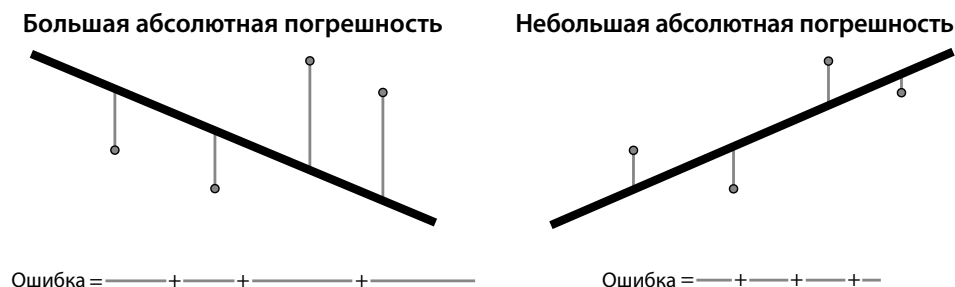


Рис. 3.18. Абсолютная погрешность равна сумме расстояний по вертикали от точек до прямой. Для плохой модели (слева) она велика, для хорошей (справа) — мала

Квадратическая погрешность. Показатель, который определяет, насколько хороша модель, суммируя квадраты расстояний

Квадратическая погрешность очень похожа на абсолютную, за исключением того, что вместо взятия абсолютного значения разницы между меткой и предсказанной меткой мы берем его квадрат. Это всегда превращает число в положительное — таков результат возведения числа в квадрат. Процесс проиллюстрирован на рис. 3.19, где квадратическая погрешность показана как сумма площадей квадратов расстояний от точек до прямой. Видно, что плохая модель (*слева*) имеет большую квадратическую погрешность, а хорошая модель (*справа*) — небольшую.

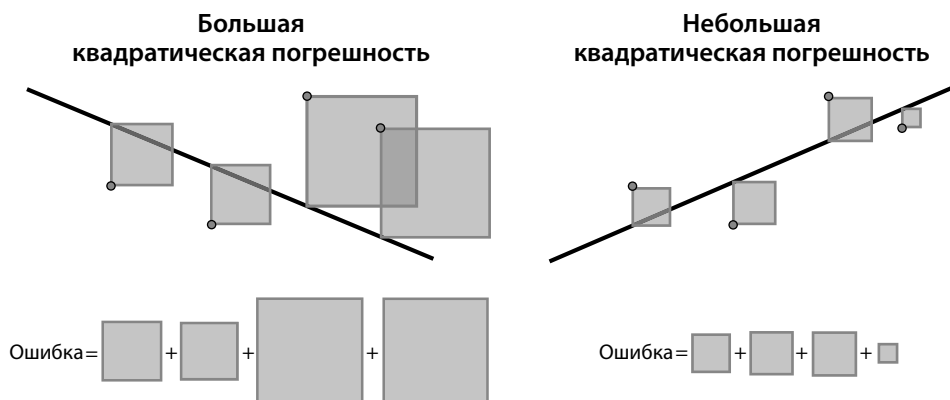


Рис. 3.19. Квадратическая погрешность представляет собой сумму квадратов расстояний по вертикали от точек до прямой

Как упоминалось ранее, квадратическая погрешность на практике используется чаще, чем абсолютная. Почему? Квадрат имеет гораздо более приятную производную, чем абсолютное значение, что полезно в процессе обучения.

Средние абсолютные и средние квадратические ошибки чаще встречаются в реальной жизни

На протяжении всей главы мы используем абсолютные и квадратические погрешности (или ошибки) в целях иллюстрации. Однако на практике гораздо чаще применяются *средняя абсолютная погрешность* и *средняя квадратическая погрешность*. Они определяются аналогичным образом, за исключением того, что вместо сумм мы вычисляем средние значения. Таким образом, средняя абсолютная погрешность — это среднее значение вертикальных расстояний от точек до прямой, а средняя квадратическая погрешность — это среднее значение квадратов этих же расстояний. Почему они встречаются

чаще? Представьте, что мы хотели бы сравнить погрешность или модель с помощью двух наборов данных: с 10 точками и с 1 млн точек. Если погрешность представляет собой сумму расстояний, по одному на каждую точку, то ошибка, вероятно, намного выше в наборе данных из 1 млн точек, потому что суммируется гораздо больше чисел, чем в наборе данных с 10 точками. Если мы хотим сравнить их должным образом, то вместо этого используем средние значения при расчете погрешности, чтобы определить, как далеко прямая находится *в среднем* от каждой точки.

Другая часто применяемая погрешность — это *RMSE* — *квадратный корень из средней квадратической ошибки*. Эта погрешность используется для сопоставления единиц измерения в задаче, а также для того, чтобы мы могли лучше понять, сильно ли модель ошибается при прогнозировании. Как это происходит? Представьте такой сценарий: мы пытаемся спрогнозировать цены на жилье, а единицами измерения цены и прогнозируемой цены являются, например, доллары. Единицами измерения квадратической погрешности и средней квадратической погрешности служат доллары в квадрате, что не является общепринятой единицей измерения. Если мы вычислим квадратный корень из этого значения, то получим не только правильную единицу измерения, но и более точное представление о том, на сколько долларов (примерно) модель ошибается на каждый дом. Скажем, если средняя квадратическая ошибка составляет 10 000 долларов, то мы можем ожидать, что модель допустит ошибку около 10 000 долларов для любого прогноза.

Градиентный спуск. Как уменьшить функцию ошибок, медленно спускаясь с горы

В этом разделе я покажу, как уменьшить любую из рассмотренных ранее погрешностей, с помощью метода, аналогичного тому, который мы использовали бы для медленного спуска с горы. В этом процессе задействуются производные, но, чтобы его понять, они вам не нужны. Ранее мы уже применяли их в тренировочном процессе в разделах «Квадратический подход...» и «Абсолютный подход...». Каждый раз, когда мы «перемещаем небольшую величину в данном направлении», мы вычисляем в фоновом режиме производную функции ошибок и используем ее, чтобы указать направление, в котором следует перемещать нашу прямую. Если вы любите математическое исчисление и хотите увидеть весь вывод этого алгоритма с применением производных и градиентов, прочитайте приложение Б. Давайте сделаем шаг назад и посмотрим на линейную регрессию со стороны. Что именно мы хотим сделать?

Мы хотим найти прямую, которая наилучшим образом соответствует нашим данным. У нас есть показатель, называемый функцией ошибок, который сообщает, как далеко прямая отстоит от данных. Таким образом, если бы мы могли просто уменьшить это значение настолько, насколько возможно, то нашли бы наилучшее положение прямой. Этот процесс, распространенный

во многих областях математики, называется *минимизирующей функцией*, то есть нахождением наименьшего возможного значения, которое может вернуть функция. Вот тут-то и пригодится градиентный спуск — это отличный способ минимизировать функцию.

Функция, которую мы пытаемся минимизировать, — это погрешность (абсолютная или квадратическая) нашей модели. Небольшое предостережение: градиентный спуск не всегда находит точное минимальное значение функции, но способен найти что-то очень близкое к нему. Хорошей новостью является то, что на практике градиентный спуск — это быстрый и эффективный метод при нахождении точек, где функция имеет минимальные значения.

Как работает градиентный спуск? Это эквивалент спуска с горы. Допустим, мы оказались на вершине высокой горы Ошибочной. Мы хотим спуститься, но вокруг туман и видно только на расстоянии примерно 1 м. Что нам делать? Хороший метод — оглядеться вокруг и выяснить, в каком направлении можно сделать один-единственный шаг, чтобы чуть-чуть спуститься. Этот процесс проиллюстрирован на рис. 3.20.

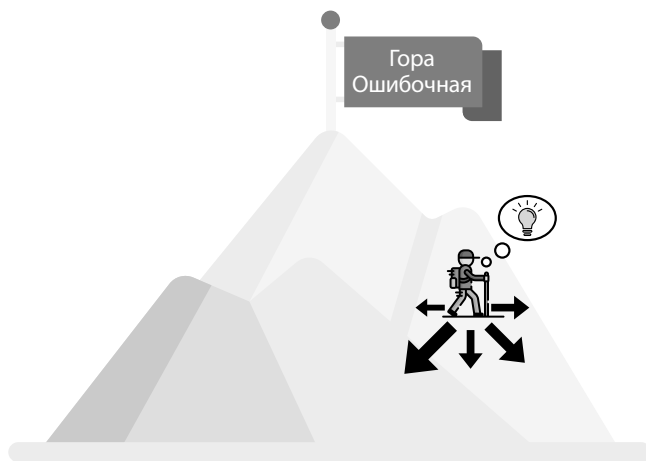


Рис. 3.20. Мы находимся на вершине горы Ошибочной и хотим добраться до ее подножия, но видимость ограничена

Способ спуска — изучить все направления, в которых можно сделать один шаг, и выяснить, какое из них помогает спуститься ниже всего. Тогда мы на один шаг ближе к подножию.

Найдя такое направление, мы делаем один маленький шаг, и поскольку он был сделан в направлении наибольшего спуска, то, скорее всего, мы спустились на небольшую величину. Теперь мы должны лишь повторить этот процесс множество раз, пока (надеюсь) не достигнем подножия. Этот процесс проиллюстрирован на рис. 3.21.



Рис. 3.21. Способ спуститься с горы состоит в том, чтобы сделать один маленький шаг в направлении, которое даст нам спуститься ниже всего, и продолжать делать это в течение длительного времени

Почему я сказал «надеюсь»? У этого процесса есть много подводных камней. Мы можем достичь как подножия, так и долины, и тогда нам больше некуда будет спускаться. Мы не станем сейчас на этом останавливаться, но есть несколько методов, позволяющих снизить вероятность того, что подобное произойдет. В приложении Б описаны некоторые из них.

Большая часть математики, которую мы здесь замечаем под ковер, более подробно объясняется в приложении Б. Но то, что мы сделали в этой главе, было именно градиентным спуском. Как это происходит? Градиентный спуск работает следующим образом.

1. Начинаем где-нибудь на горе.
2. Ищем наилучшее направление, чтобы сделать один маленький шаг.
3. Делаем этот шаг.
4. Повторяем шаги 2 и 3 множество раз.

Это может показаться знакомым, потому что в разделе «Алгоритм линейной регрессии» после определения абсолютных и квадратических подходов мы определили алгоритм линейной регрессии следующим образом.

1. Начинаем с любой прямой.
2. Ищем наилучшее направление, чтобы немного сдвинуть прямую, используя либо абсолютный, либо квадратический подход.
3. Немного перемещаем прямую в этом направлении.
4. Повторяем шаги 2 и 3 множество раз.

Представление об этом процессе показано на рис. 3.22. Единственное отличие состоит в том, что эта функция ошибок меньше похожа на гору,

больше — на долину, а наша цель — спуститься до самой низкой точки. Каждая точка в этой долине соответствует некоторой модели (прямой), которая пытается соответствовать нашим данным. Точки ниже — это хорошие модели с небольшой погрешностью, точки выше — плохие модели с большой погрешностью. Способ спуска — начать где-то и постоянно делать шаги, которые заставляют нас спускаться. Градиент позволяет определить направление шага, который поможет спуститься ниже всего.

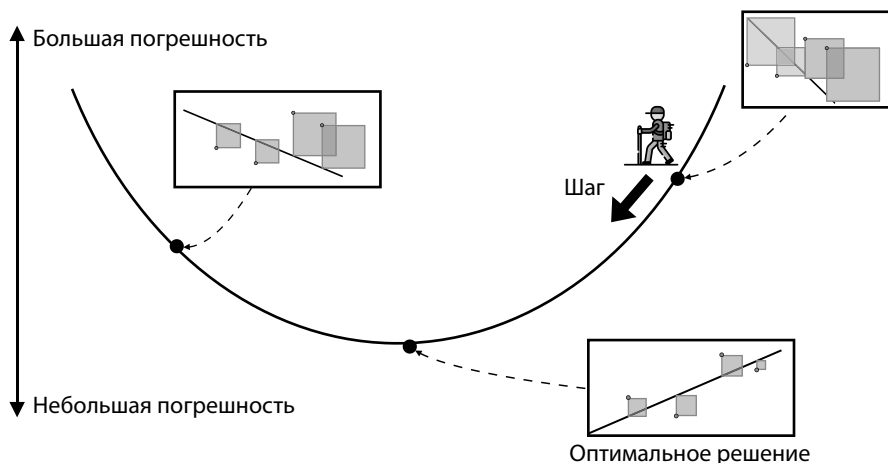


Рис. 3.22. Каждая точка на этой горе соответствует отдельной модели

Высота точки — это погрешность конкретной модели. Таким образом, плохие модели находятся наверху, а хорошие — внизу. Мы стараемся спуститься как можно ниже. Каждый шаг ведет нас от одной модели к немного лучшей модели. Если мы будем делать подобный шаг раз за разом, то в итоге придем к лучшей модели (или по крайней мере к довольно хорошей!).

Построение графика функции ошибок и умение вовремя остановиться

В этом разделе мы рассмотрим график функции ошибок для обучения, реализованного ранее в разделе «Используем алгоритм линейной регрессии в наборе данных». Он дает полезную информацию об обучении этой модели. В репозитории мы также построили график функции средней квадратической ошибки, определенной в разделе «Средние абсолютные и средние квадратические погрешности...». Код для вычисления RMSE следующий:

```
def rmse(labels, predictions):
    n = len(labels)
    differences = np.subtract(labels, predictions)
    return np.sqrt(1.0/n * (np.dot(differences, differences)))
```


СКАЛЯРНОЕ ПРОИЗВЕДЕНИЕ Для кодирования функции RMSE мы использовали скалярное произведение, которое представляет собой простой способ записать сумму произведений соответствующих членов в двух векторах. Например, скалярное произведение векторов (1, 2, 3) и (4, 5, 6) — это $1 \times 4 + 2 \times 5 + 3 \times 6 = 32$. Если мы вычислим скалярное произведение вектора и самого себя, то получим сумму квадратов элементов.

График погрешности показан на рис. 3.23. Обратите внимание на то, что он быстро снизился примерно после 1000 повторений и в дальнейшем изменялся не сильно. График дает нам полезную информацию: для этой модели мы можем выполнить обучающий алгоритм из 1000 или 2000 итераций вместо 10 000 и получить аналогичный результат.

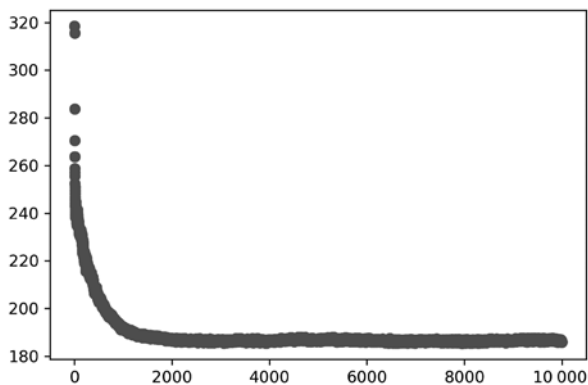


Рис. 3.23. График средней квадратической погрешности для рассматриваемого примера

В общем, функция ошибок дает достаточно информации для того, чтобы решить, когда прекратить выполнение алгоритма. Часто решение основывается на времени и доступных вычислительных мощностях. Однако на практике обычно используются и другие полезные критерии, например:

- когда функция потерь достигает значения, которое мы определили предварительно;
- когда функция потерь не уменьшается на значительную величину в течение нескольких периодов.

Одна точка за раз или несколько? Стохастический и пакетный градиентный спуск

В разделе «Как заставить компьютер провести прямую...» мы обучили модель линейной регрессии, много раз повторяя единственный шаг. Он состоял из выбора одной точки и перемещения прямой ближе к ней. В разделе «Как

мы измеряем результаты? Функция ошибок» мы обучили модель линейной регрессии, вычислив погрешность (абсолютную или квадратическую) и уменьшив ее с помощью градиентного спуска. Однако эта погрешность была вычислена для всего набора данных, а не для одной точки за раз. Почему именно так? Реальность такова, что мы можем обучать модели, выполняя итерации по одной точке за раз или по всему набору данных. Однако, когда наборы данных очень велики, оба варианта могут оказаться дорогостоящими. Можно использовать полезный метод, называемый *мини-пакетным обучением*, который состоит в разделении данных на множество мини-пакетов. На каждой итерации алгоритма линейной регрессии мы выбираем один из мини-пакетов и, чтобы уменьшить погрешность в нем, приступаем к корректировке весов модели. Решение задействовать одну точку, мини-пакет точек или весь набор данных на каждой итерации приводит к трем общим типам алгоритмов градиентного спуска. Использование одной точки за раз называется стохастическим градиентным спуском. Применение мини-пакета называется мини-пакетным градиентным спуском.

Использование всего набора данных называется *пакетным градиентным спуском*. Этот процесс более подробно проиллюстрирован в приложении Б.

Реальное применение. Использование Turi Create для прогнозирования цен на жилье в Индии

В этом разделе я покажу реальное приложение. Мы будем применять линейную регрессию для прогнозирования цен на жилье в Хайдарабаде (Индия). Набор данных взят с Kaggle — популярного сайта для соревнований по машинному обучению. Код для этого раздела следующий.

- **Блокнот** `House_price_predictions.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_3_Linear_Regression/House_price_predictions.ipynb.
- **Набор данных** `Hyderabad.csv`.

Набор данных содержит 6207 строк (по одной на дом) и 39 столбцов (признаков). Как можно предположить, мы не будем кодировать алгоритм вручную. Вместо этого используем Turi Create — популярный полезный пакет, в котором реализованы многие алгоритмы машинного обучения. Основной объект для хранения данных в Turi Create — это `SFrame`. Начинаем с загрузки данных в `SFrame` с помощью следующей команды:

```
data = tc.SFrame('Hyderabad.csv')
```

Таблица огромна, но вы можете взглянуть на первые несколько строк и столбцов (табл. 3.3).

Таблица 3.3. Первые пять строк и семь столбцов набора данных о ценах на жилье в Хайдарабаде

Цена	Район	Количество спален	Перепродажа	Обслуживающий персонал	Спортзал	Бассейн
30 000 000	3340	4	0	1	1	1
7 888 000	1045	2	0	0	1	1
4 866 000	1179	2	0	0	1	1
8 358 000	1675	3	0	0	0	0
6 845 000	1670	3	0	1	1	1

Обучение модели линейной регрессии в Turi Create требует всего одной строки кода. Мы используем функцию `create` из пакета `linear_regression`. В ней нужно только указать цель (метку), которая является ценой:

```
model = tc.linear_regression.create(data, target='Price')
```

Обучение может занять несколько минут, но после него мы получим определенную информацию. Одним из полей вывода будет средняя квадратическая погрешность. Для этой модели RMSE составляет около 3 000 000. Это большое значение RMSE, но оно не означает, что модель дает плохие прогнозы. Оно может означать, что в наборе данных много выбросов. Как можно представить, цена дома может зависеть от многих других характеристик, которых нет в наборе данных.

Мы можем использовать модель для прогнозирования цены дома площадью 1000 м² с тремя спальнями следующим образом:

```
house = tc.SFrame({'Area': [1000], 'No. of Bedrooms':[3]})
model.predict(house)
Результат: 2594841
```

Модель показывает, что цена дома площадью 1000 м² с тремя спальнями составит 2 594 841.

Мы можем обучить модель и с помощью меньшего количества признаков. Функция `create` позволяет вводить признаки, которые хотим использовать в виде массива. Следующая строка кода обучает модель под названием `simple_model`, которая задействует район для прогнозирования цены:

```
simple_model = tc.linear_regression.create(data, features=['Area'],
    target='Price')
```

Мы можем исследовать веса этой модели с помощью следующей строки кода:

```
simple_model.coefficients
```

Вывод дает следующие веса:

- наклон 9664,97;
- у-пересечение -6 105 981,01.

Пересечение — это смещение, а коэффициент для района — это наклон линии, когда мы отображаем район и цену. График точек с соответствующей моделью — прямой, которую мы получили, используя только признак района для прогнозирования цены, показан на рис. 3.24.

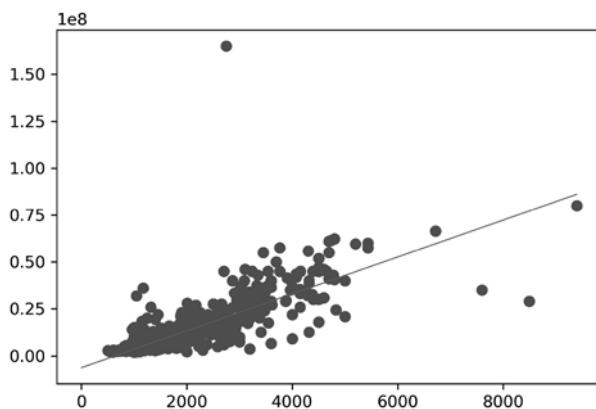


Рис. 3.24. Набор данных о ценах на жилье в Хайдарабаде, ограниченный районом и ценой

С этим набором данных можно проделать гораздо больше, так что приглашаю вас продолжить изучение. Например, попробуйте выяснить, какие признаки более важные, посмотрев на веса модели. Я советую просмотреть документацию Turi Create (<https://apple.github.io/turicreate/docs/api/>), чтобы узнать о других функциях и подходах, с помощью которых можно улучшить эту модель.

Что делать, если данные не расположены на прямой? Полиномиальная регрессия

В предыдущих разделах мы узнали, как построить прямую, наилучшим образом подходящую для наших данных. Но что произойдет, если они не будут похожи на прямую? В этом разделе мы изучим эффективное расширение линейной регрессии, называемое *полиномиальной регрессией*. Она-то и поможет нам со случаями, когда данные более сложные.

Особый вид криволинейных функций. Многочлены

Чтобы изучить полиномиальную регрессию, сначала придется выяснить, что такое многочлен. *Многочлены* — это класс функций, полезных при моделировании нелинейных данных.

Мы уже встречались с ними, так как любая прямая — это многочлен степени 1. Параболы — это примеры многочленов степени 2. Формально многочлен — это функция от одной переменной, которая может быть выражена как сумма степеней, кратных этой переменной. Степени переменной x равны 1, x , x^2 , x^3 ... Обратите внимание на то, что две первые — это $x^0 = 1$ и $x^1 = x$. Далее приведены примеры многочленов:

- $y = 4$;
- $y = 3x + 2$;
- $y = x^2 - 2x + 5$;
- $y = 2x^3 + 8x^2 - 40$.

Мы определяем *степень* многочлена как показатель наивысшей степени в выражении многочлена. Например, многочлен $y = 2x^3 + 8x^2 - 40$ имеет степень 3, потому что 3 — это наивысший показатель, до которого повышается переменная x . Обратите внимание: приведенные ранее многочлены имеют степень 0, 1, 2 и 3. Многочлен степени 0 всегда константа, а многочлен степени 1 представляет собой линейное уравнение, подобное тем, которые мы встречали ранее в этой главе.

График многочлена очень похож на кривую, которая колеблется несколько раз. Количество колебаний зависит от степени многочлена. Если он имеет степень d , то его график представляет собой кривую, которая колеблется не более $d - 1$ раза (для $d > 1$). На рис. 3.25 мы видим графики некоторых многочленов.

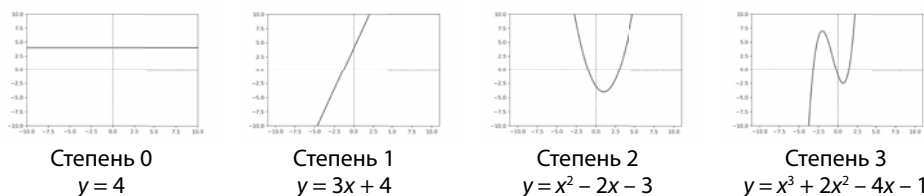


Рис. 3.25. Многочлены — это функции, которые помогают лучше моделировать данные

Обратите внимание на то, что многочлены степени 0 — это прямые. Многочлены степени 1 представляют собой прямые с наклоном, отличным от 0. Многочлены степени 2 являются квадратическими (параболами). Многочлены степени 3 выглядят как кривая, которая колеблется дважды (хотя потенциально они могут колебаться меньшее количество раз). Как бы выглядел график многочлена степени 100? Например, график $y = x^{100} - 8x^{62} + 73x^{27} - 4x + 38$? Чтобы это выяснить, нам пришлось бы его построить, но мы знаем наверняка, что это кривая, которая колеблется не более 99 раз.

Нелинейные данные? Нет проблем — попробуем подвинуть к ним полиномиальную кривую

В этом разделе мы увидим, что произойдет, если данные не являются линейными (то есть не выглядят так, как будто образуют линию) и мы хотим подогнать к ним полиномиальную кривую. Допустим, данные выглядят так, как показано в левой части рис. 3.26. Как бы мы ни старались, мы не сможем найти соответствующую им хорошую прямую. Нет проблем! Если мы решим подогнать многочлен степени 3 (также называемый кубическим), то получим кривую, показанную на рис. 3.26 справа, которая гораздо лучше соответствует данным.

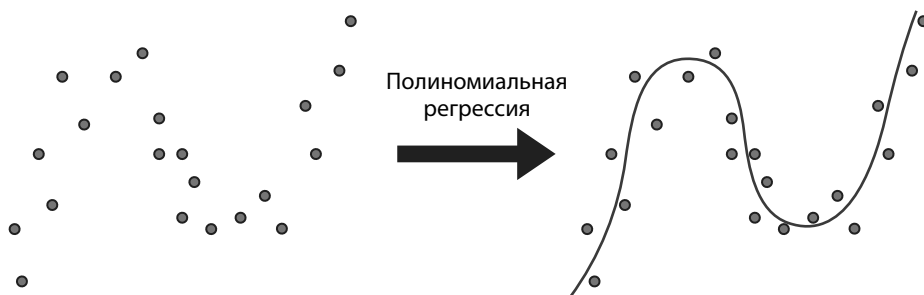


Рис. 3.26. Полиномиальная регрессия полезна, когда дело доходит до моделирования нелинейных данных

Процесс обучения модели полиномиальной регрессии аналогичен процессу обучения модели линейной регрессии. Единственное отличие заключается в том, что нужно добавить больше столбцов в набор данных, прежде чем применять линейную регрессию. Например, если мы решим подогнать многочлен степени 3 к данным, изображенным на рис. 3.26, нужно будет добавить два столбца, соответствующих квадрату и кубу признака. Если хотите изучить все это более подробно, пожалуйста, ознакомьтесь с разделом «Полиномиальная регрессия, тестирование и регуляризация с помощью Turi Create» в главе 4, где мы изучаем пример полиномиальной регрессии в параболическом наборе данных.

Небольшое условие при обучении модели полиномиальной регрессии заключается в том, что мы должны определить степень многочлена перед началом обучения. Как же мы выбираем эту степень? Нам нужны прямая (степень 1), парабола (степень 2), кривая третьего порядка (степень 3) или некая кривая степени 50? Этот вопрос важен, и мы рассмотрим его в главе 4, когда будем изучать переобучение, недообучение и регуляризацию!

Параметры и гиперпараметры

Параметры и гиперпараметры — одни из наиболее важных понятий в машинном обучении, и в этом разделе мы узнаем, что это такое и как их различать.

Как мы узнали из данной главы, регрессионные модели определяются их весами и смещением — *параметрами* модели. Однако перед обучением модели мы можем подкрутить множество других ручек, таких как скорость обучения, количество периодов, степень (если рассматривать модель полиномиальной регрессии) и многие другие. Они называются *гиперпараметрами*.

Все модели машинного обучения, которые мы изучаем в этой книге, имеют четко определенные параметры и гиперпараметры. Их легко спутать, поэтому эмпирическое правило, позволяющее отличать их друг от друга, звучит так:

- любая величина, которую вы устанавливаете *перед* процессом обучения, — это гиперпараметр;
- любая величина, которую модель создает или изменяет *в процессе* обучения, — это параметр.

Применение регрессии

Влияние машинного обучения измеряется не только мощностью его алгоритмов, но и шириной применения. В этом разделе рассмотрим некоторые способы применения линейной регрессии в реальной жизни. В каждом из примеров мы описываем проблему, изучаем некоторые особенности ее решения, а затем позволяем линейной регрессии творить свое волшебство.

Системы рекомендаций

Машинное обучение широко применяется для создания рекомендаций в некоторых наиболее известных приложениях, включая YouTube, Netflix, Facebook, Spotify и Amazon. Ключевую роль в большинстве этих рекомендательных систем играет регрессия. Поскольку регрессия предсказывает количество, то все, что нам нужно сделать для получения хороших рекомендаций, — выяснить, какое количество лучше всего отражает взаимодействие с пользователем или удовлетворенность пользователей. Далее приведены более конкретные примеры.

Рекомендации по видео и музыке

Одним из способов создания рекомендаций по видео и музыке выступает прогнозирование количества времени, в течение которого пользователь будет смотреть видео или слушать песню. Для этого можно создать модель линейной регрессии, где метки на данных — это количество минут, в течение которых каждая песня прослушивается каждым пользователем. Признаками могут служить демографические данные пользователя, такие как его возраст, местоположение и профессия, они могут быть также поведенческими — другие видео или песни, на которые пользователь нажимал или с которыми взаимодействовал.

Рекомендации по продукту

Магазины и веб-сайты электронной коммерции также применяют линейную регрессию для прогнозирования своих продаж. Один из способов — предсказать, сколько покупатель потратит в магазине. Мы можем сделать это с помощью линейной регрессии. Меткой для прогнозирования может быть сумма, потраченная пользователем, а признаками — демография и поведение, как и в рекомендациях по видео и музыке.

Здравоохранение

Регрессия имеет множество применений в здравоохранении. В зависимости от того, какую проблему мы хотим решить, ключевым моментом является предсказание правильной метки. Вот несколько примеров:

- прогнозирование продолжительности жизни пациента на основе состояния его здоровья в данный момент;
- прогнозирование продолжительности пребывания в больнице на основе симптомов заболевания.

Резюме

- Регрессия — важная часть машинного обучения. Она состоит из обучения алгоритма с помеченными данными и его использования для прогнозирования будущих (немаркированных) данных.
- Помеченные данные — это данные, поставляемые с метками, которые в случае регрессии являются числами. Например, числа могут быть ценами на дома.
- В наборе данных признаки — это свойства, которые мы используем для прогнозирования метки. Так, если мы хотим спрогнозировать цены на жилье, характеристики — это все, что описывает дом и что может определить цену, например размер, количество комнат, качество ближайшей школы, уровень преступности в районе, возраст дома и расстояние до шоссе.
- Метод линейной регрессии для прогнозирования состоит в присвоении веса каждому из признаков и суммировании весов, умноженных на соответствующие признаки, плюс смещение.
- Графически можно представить алгоритм линейной регрессии как попытку провести прямую как можно ближе к набору точек.
- Работа алгоритма линейной регрессии заключается в том, что он начинает со случайной прямой, а затем медленно приближает ее к каждой из неправильно классифицированных точек, пытаясь классифицировать их правильно.

- Полиномиальная регрессия — это общий случай линейной регрессии, в которой для моделирования данных мы используем кривые вместо прямых. Это особенно полезно, когда набор данных нелинеен.
- Регрессия имеет множество применений, включая системы рекомендаций, электронную коммерцию и здравоохранение.

Упражнения

Упражнение 3.1

Веб-сайт обучил модель линейной регрессии для прогнозирования времени, которое пользователь проведет на сайте. Полученная формула выглядит так:

$$\hat{t} = 0,8d + 0,5m + 0,5y + 0,2a + 1,5,$$

где $\hat{t}$ — прогнозируемое время в минутах, d , m , y и a — переменные индикатора (они принимают только значения 0 или 1), определенные следующим образом:

- d — это переменная, указывающая, применяет ли пользователь ПК;
- m — переменная, указывающая, задействует ли пользователь мобильное устройство;
- y — переменная, указывающая, молодой ли пользователь (моложе 21 года);
- a — переменная, указывающая, взрослый ли пользователь (21 год или старше).

Пример: если пользователю 30 лет и он работает на ПК, то $d = 1$, $m = 0$, $y = 0$ и $a = 1$.

Если 45-летний человек просматривает веб-сайт с телефона, каким будет ожидаемое время, которое он проведет на сайте?

Упражнение 3.2

Представьте, что мы обучили модель линейной регрессии на медицинском наборе данных. Она предсказывает ожидаемую продолжительность жизни пациента. Каждому из признаков в наборе данных модель будет присваивать определенный вес.

А. Вес, присвоенный следующим величинам, — это положительное число, отрицательное число или ноль? (*Примечание:* если считаете, что вес — очень маленькое положительное или отрицательное число, то можете сказать, что это ноль.)

1. Количество часов, которые пациент затрачивает на физические упражнения в неделю.
2. Количество сигарет, которые пациент выкуривает в неделю.
3. Количество у пациента членов семьи с проблемами с сердцем.

4. Количество у пациента братьев и сестер.
 5. Был пациент госпитализирован или нет.
- Б. У модели есть смещение. Как вы думаете, оно положительное, отрицательное или нулевое?

Упражнение 3.3

Далее приведен набор данных домов с размерами (в квадратных футах) и ценами (в долларах).

Дом	Размер (s)	Вознаграждение (p)
1	100	200
2	200	475
3	200	400
4	250	520
5	325	735

Предположим, мы обучили модель, в которой прогноз цены дома, основанный на размере, выглядит следующим образом:

$$\hat{p} = 2s + 50.$$

- А. Вычислите прогнозы, которые эта модель делает для набора данных.
- Б. Вычислите среднюю абсолютную погрешность модели.
- В. Вычислите среднюю квадратическую погрешность модели.

Упражнение 3.4

Наша цель состоит в том, чтобы переместить линию $\hat{y} = 2x + 3$ ближе к точке $(x, y) = (5, 15)$, используя приемы, которые освоили в этой главе. Для следующих двух задач возьмите скорость обучения $\eta = 0,01$.

- А. Примените абсолютный подход, чтобы передвинуть линию выше — и она оказалась ближе к точке.
- Б. Примените квадратический подход, чтобы передвинуть линию выше — и она оказалась ближе к точке.



В этой главе

- ✓ Что такое недообучение и переобучение.
- ✓ Некоторые решения, позволяющие избежать переобучения: тестирование, график сложности модели и регуляризация.
- ✓ Вычисление сложности модели с использованием норм $L1$ и $L2$.
- ✓ Выбор наилучшей модели с точки зрения производительности и сложности.



Эта глава отличается от большинства других тем, что в ней вы не найдете конкретного алгоритма машинного обучения. Вместо этого в ней описываются некоторые проблемы, с которыми могут столкнуться модели машинного обучения, а также эффективные способы их решения на практике.

Представьте, что вы изучили несколько отличных алгоритмов машинного обучения и готовы их применять. Вы приступаете к обработке данных, ваша первая задача — создать модель машинного обучения для набора данных клиентов. Вы создаете ее и запускаете. Однако все идет не по плану, и модель не слишком хорошо справляется с прогнозированием. Что же случилось?

Оказывается, это типичная история, потому что с нашими моделями не так может пойти многое. К счастью, мы располагаем целым арсеналом методов их улучшения. В этой главе я покажу вам две проблемы, которые часто возникают при обучении моделей: недообучение и переобучение. Затем продемонстрирую ряд решений, позволяющих избежать недообучения и переобучения моделей: тестирование и валидацию, график сложности модели и регуляризацию.

Попробуем понять, что такое недообучение и переобучение, с помощью следующей аналогии. Допустим, нам нужно подготовиться к экзамену. Во время учебного процесса сразу несколько вещей могут пойти не так. Возможно, мы недостаточно усердно занимались. Исправить это невозможно, и мы, скорее всего, плохо справимся с экзаменом. А если мы учились усиленно, но не так? Например, вместо того чтобы сосредоточиться на понимании, решили дословно заучить весь учебник. Поможет ли это на экзамене? Скорее всего, нет, потому что мы просто вызубрили все без должного понимания. Лучшим вариантом, конечно, было бы подготовиться к экзамену правильно, то есть так, чтобы мы могли ответить на новые вопросы по этой теме.

В машинном обучении *недообучение* очень похоже на недостаточную подготовку к экзамену. Такое происходит, когда мы пытаемся обучить модель, а она слишком проста и не в состоянии изучить данные. *Переобучение*, в свою очередь, напоминает заучивание всего учебника вместо подготовки к экзамену. Это происходит, когда мы пытаемся обучить слишком сложную модель и она запоминает данные вместо того, чтобы как следует в них разобраться. Хорошая

модель, не страдающая от недообучения или переобучения, — это такая, которая как будто хорошо подготовилась к экзамену. Это модель, которая должным образом изучает данные и способна давать хорошие прогнозы на основе новых данных, которые она еще не встречала.

Другой способ понять суть недообучения и переобучения — рассмотреть реальную задачу. Мы можем совершить две ошибки: чрезмерно упростить задачу и предложить излишне простое решение или чрезмерно усложнить задачу и предложить чрезмерно сложное решение.

Представьте, что наша задача состоит в том, чтобы убить Годзиллу, и мы вступаем в бой, не имея ничего, кроме мухобойки (рис. 4.1, *слева*). Это пример чрезмерного *упрощения*. Такой подход нам не подойдет, потому что мы недооценили задачу и оказались неподготовленными. Это недообучение: набор данных сложен, а мы подходим к его моделированию, не имея ничего, кроме простой модели. Модель не сможет охватить всю сложность набора данных.

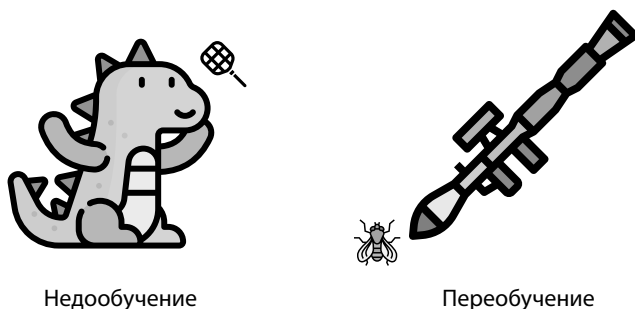


Рис. 4.1. Недообучение и переобучение — две проблемы, которые могут возникнуть при обучении модели

И напротив, если задача состоит в том, чтобы убить муху, и мы используем для этого гранатомет (см. рис. 4.1, *справа*), то это пример чрезмерного *усложнения*. Да, так можно убить муху, но также уничтожить все вокруг и подвергнуться риску. Мы переоценили проблему, и ее решение оказалось не очень хорошим. Это переобучение: данные просты, а мы пытаемся подогнать их под слишком сложную модель. Модель вместит данные, но она будет запоминать их вместо того, чтобы изучать. Когда я узнал о переобучении, моя первая реакция была такой: «Ну, это вообще не проблема. Если я использую слишком сложную модель, я же все равно могу моделировать свои данные, так?» Правильно, но настоящая проблема с переобучением заключается в том, чтобы заставить модель делать прогнозы на основе новых данных. Как мы убедимся позже в этой главе, прогнозы, скорее всего, будут ужасными.

Как мы знаем из раздела «Параметры и гиперпараметры» главы 3, каждая модель машинного обучения имеет гиперпараметры — «ручки настройки», которые мы подкручиваем перед обучением модели. Установка правильных гиперпараметров для нашей модели чрезвычайно важна. Если задать какие-то из

них неверно, легко столкнуться с недообучением или переобучением. Методы, которые мы рассмотрим в этой главе, пригодятся для правильной настройки гиперпараметров.

Чтобы прояснить эти понятия, обратимся к примеру с набором данных и несколькими моделями, которые создаются изменением одного конкретного гиперпараметра — степени многочлена.

Вы можете найти весь код для этой главы в следующем репозитории GitHub: https://github.com/luisguiserrano/manning/tree/master/Chapter_4_Testing_Overfitting_Underfitting.

Пример недообучения и переобучения с использованием полиномиальной регрессии

В этом разделе мы увидим недообучение и переобучение в случае с одним и тем же набором данных. Внимательно посмотрите на набор данных на рис. 4.2 и попробуйте подогнать к нему модель полиномиальной регрессии (см. раздел «Что делать, если данные не расположены на прямой? Полиномиальная регрессия» в главе 3). Давайте подумаем о том, какой многочлен соответствовал бы такому набору данных. Будет ли это прямая, парабола, кривая третьего порядка или, возможно, многочлен 100-й степени? Напомним, что степень многочлена — наивысший из имеющихся порядков. Например, многочлен $2x^{14} + 9x^6 - 3x + 2$ имеет степень 14.

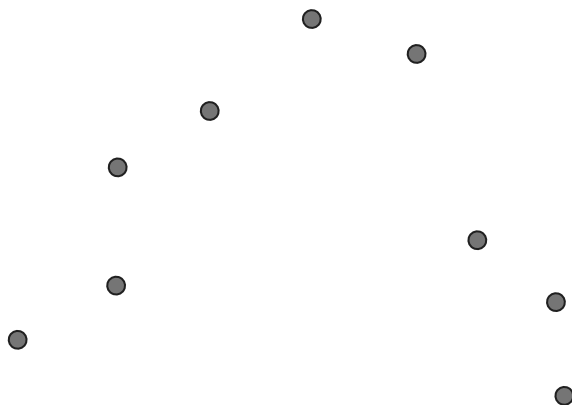


Рис. 4.2. На этом наборе данных мы обучаем некоторые модели и демонстрируем недообучение и переобучение

Я думаю, что этот набор данных напоминает параболу, которая открывается вниз (как грустный смайлик). Это многочлен 2-й степени. Однако мы люди и увидели это глазами. Компьютер так не умеет. Ему нужно перебрать множество значений степени многочлена и каким-то образом выбрать наилучшее. Допустим, компьютер попытается сопоставить его с многочленами

степеней 1, 2 и 10. Если сопоставить многочлены степени 1 (прямая), 2 (квадратическая) и 10 (кривая, которая колеблется не более девяти раз) с нашим набором данных, мы получим результаты, показанные на рис. 4.3.

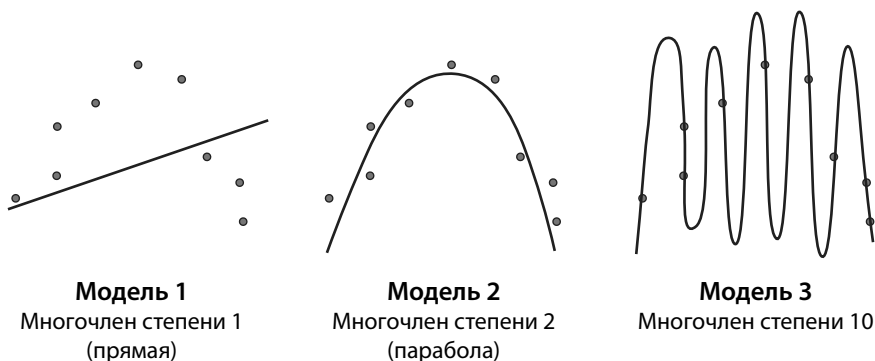


Рис. 4.3. Три модели для одного и того же набора данных. Какая из них выглядит наиболее подходящей?

Здесь мы видим три модели: модель 1 — многочлен степени 1, представляющий собой прямую, модель 2 — многочлен степени 2 (или квадратический) и модель 3 — многочлен степени 10. Обратите внимание на то, что модель 1 слишком проста, потому что это прямая, пытающаяся соответствовать квадратическому набору данных. Мы не сможем найти хорошую прямую, соответствующую такому набору данных, потому что он сам попросту не выглядит как прямая. Таким образом, модель 1 — это наглядный пример недообучения. Модель 2, напротив, довольно хорошо соответствует данным. Она и не недообучена, и не переобучена. Модель 3 очень хорошо соответствует данным, но она полностью упускает суть. Данные должны выглядеть как парабола с небольшим количеством шума, но модель рисует очень сложный многочлен степени 10, который проходит через каждую из точек, но не отражает суть данных. Это наглядный пример переобучения.

Подводя итог рассуждений, приведем наблюдение, которое мы используем как на протяжении всей этой главы, так и во многих других эпизодах книги: очень простые модели, как правило, недообучены. Очень сложные модели, как правило, переобучены. Цель состоит в том, чтобы найти модель, которая не была бы ни слишком простой, ни слишком сложной и при этом хорошо отражала суть данных.

Мы вот-вот перейдем к самой сложной части. Будучи людьми, мы понимаем, что наилучшим образом нам подходит модель 2. Но что видит компьютер? Он способен вычислять лишь функции ошибок. Как вы, возможно, помните из главы 3, мы определили две функции ошибок: абсолютную и квадратическую погрешность. Для наглядности в этом примере будем использовать абсолютную погрешность, которая представляет собой среднее значение суммы абсолютных расстояний от точек до кривой, хотя те же аргументы будут работать и с квадратической погрешностью. В модели 1 точки находятся далеко от прямой,

поэтому погрешность велика. В модели 2 эти расстояния невелики, поэтому и погрешность невелика. Однако в модели 3 расстояния равны нулю, потому что все точки попадают на кривую! Это означает, что компьютер посчитает, что идеальная модель — 3. Нас это не устраивает. Нам нужен способ сообщить компьютеру, что лучшая модель — 2, а модель 3 переобучена. Как мы можем это проделать? Отложите, пожалуйста, книгу на несколько минут и поразмышляйте о задаче самостоятельно, потому что у нее есть несколько решений.

Как заставить компьютер выбрать правильную модель? Тестированием!

Один из способов определить, подходит ли модель, — протестировать ее. Как раз это мы и сделаем. Тестирование модели состоит в выборе небольшого набора точек из набора данных и использовании их не для обучения, а для тестирования эффективности модели. Такой набор точек называется *тестовым набором*. Оставшийся набор точек (большинство), который задействуется для обучения модели, называется *обучающим набором*. Обучив модель, применим тестовый набор для ее оценки. Таким образом мы удостоверимся, что модель хорошо работает с новыми данными, а не просто запомнила обучающий набор. Возвращаясь к аналогии с экзаменом, представим себе обучение и тестирование следующим образом. Предположим, что в конце книги, по которой мы готовимся, имеются 100 вопросов. Выбираем 80 из них для обучения, что означает: мы тщательно изучаем их, ищем ответы и получаем знания. Затем используем оставшиеся 20 вопросов, чтобы проверить себя, — пытаемся ответить на них, не заглядывая в книгу, как будет на экзамене.

Теперь посмотрим, как этот метод работает с нашим набором данных и моделями. Обратите внимание на то, что реальная проблема с моделью 3 заключается не в том, что она не соответствует данным, а в том, что она не распространяется на новые данные. Другими словами, если бы вы обучили модель 3 на этом наборе данных, а потом появилось бы несколько новых точек, смогла бы модель делать надежные прогнозы с их помощью? Вероятно, нет, потому что она просто запомнила весь набор данных, не уловив его сути. В данном случае суть набора данных заключается в том, что он выглядит как парабола, которая открывается вниз.

Столбцы на рис. 4.4 представляют модели степени 1, 2 и 10. Строки — погрешности обучения и тестирования. Два белых треугольника — это тестовый набор данных. Обучающий набор состоит из кружков. Теперь подробно разберем этот рисунок и посмотрим, как модели работают с обучающим и тестовым наборами. Другими словами, изучим погрешность, которую создает модель в обоих наборах данных. Назовем эти две погрешности *погрешностью обучения* и *погрешностью тестирования*.

Верхний ряд на рис. 4.4 соответствует набору обучающих данных, а нижний — тестовых. Чтобы проиллюстрировать погрешность, мы провели вертикальные линии от точки к модели. Средняя абсолютная погрешность в точности равна среднему значению длин этих линий. Погрешность обучения уменьшается

по мере увеличения сложности модели. Глядя на верхний ряд, мы видим, что модель 1 имеет большую погрешность обучения, модель 2 — небольшую, а модель 3 — совсем крошечную, фактически нулевую. Таким образом, модель 3 лучше всего справляется с обучающим набором.

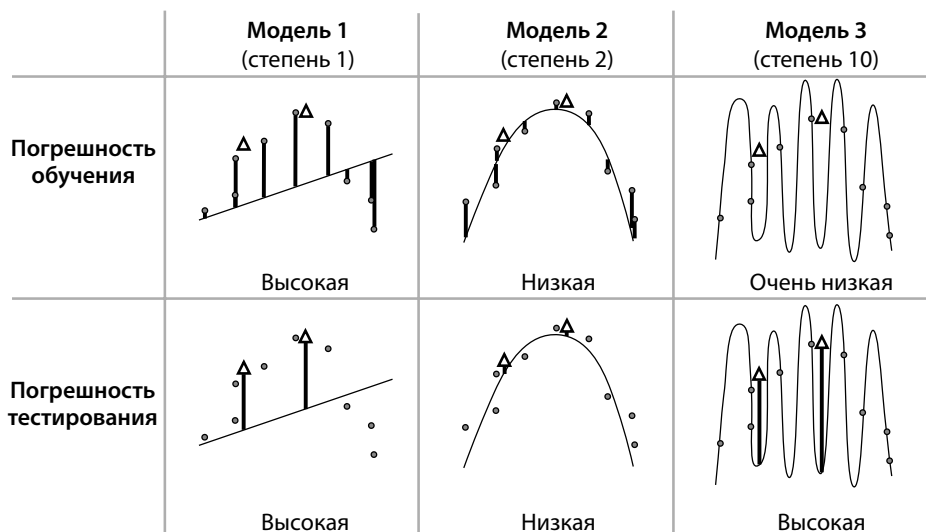


Рис. 4.4. Можно использовать эту таблицу, чтобы решить, насколько сложной мы хотим видеть свою модель

Однако в тестовом наборе все меняется. По мере увеличения сложности модели погрешность тестирования снижается, а затем восстанавливается. Модель 1 по-прежнему имеет большую погрешность тестирования, то есть это попросту плохая модель, не справляющаяся с обучением и набором тестов: она недообучена. Модель 2 имеет небольшую погрешность тестирования — это хорошая модель. Она подходит как для обучающего, так и для тестового набора. А модель 3 дает большую погрешность тестирования. Поскольку она ужасно сработала на тестовом наборе, но при этом так хорошо справилась с обучающим, мы делаем вывод: она переобучена. Таким образом, наилучшая из трех моделей — 2, поскольку она дает низкую погрешность тестирования.

Давайте подведем итог того, что уже узнали.

Модели могут быть:

- недообученными — используется модель, слишком простая для набора данных;
- подходящими — применяется модель со степенью сложности, подходящей для набора данных;
- переобученными — используется модель, слишком сложная для набора данных.

На обучающем наборе:

- недообученная модель будет работать плохо (большая погрешность обучения);
- хорошая модель будет работать хорошо (небольшая погрешность обучения);
- переобученная будет работать очень хорошо (очень небольшая погрешность обучения).

На тестовом наборе:

- недообученная модель будет работать плохо (большая погрешность тестирования);
- хорошая модель будет работать хорошо (небольшая погрешность тестирования);
- переобученная модель будет работать плохо (большая погрешность тестирования).

Таким образом, определить, подходит ли модель, недообучена она или переобучена, можно, взглянув на погрешности обучения и тестирования. Если обе они велики, то модель недообучена. Если обе невелики, то это хорошая модель. Если погрешность обучения невелика, а погрешность тестирования высока, то модель переобучена.

Как выбрать набор для тестирования и насколько большим он должен быть?

Непростой вопрос. Откуда я взял две новые точки? Если мы обучаем производственную модель, в которой данные поступают постоянно, то можем выбрать некоторые из новых точек в качестве тестовых данных.

Но что если у нас нет способа получить новые точки и все, чем мы можем пользоваться, — это набор данных из десяти точек? Когда подобное происходит, мы просто жертвуем частью данных и применяем их в качестве тестового набора. Сколькими именно? Это зависит от того, сколько у нас данных и насколько хорошей работы модели мы добиваемся. На практике любое значение от 10 до 20 % работает более или менее хорошо.

Можно ли использовать тестовые данные для обучения модели? Нет

В машинном обучении всегда нужно следовать важному правилу: разделив данные на обучающие и тестовые, мы должны применять для обучения модели обучающие данные и ни при каких обстоятельствах не касаться тестовых данных в ходе этого или в процессе принятия решений о гиперпараметрах модели. Нарушение этого правила, скорее всего, приведет к переобучению, даже если это неочевидно для человека. На многих соревнованиях по машинному обучению

команды представляли модели (по их мнению, замечательные) лишь для того, чтобы они с треском провалились при тестировании на секретном наборе данных. Это могло быть связано с тем, что специалисты по обработке данных, обучающие модели, каким-то образом (возможно, непреднамеренно) использовали тестовые данные для обучения. На самом деле данное правило настолько важно, что мы сделаем его золотым правилом этой книги.

ЗОЛОТОЕ ПРАВИЛО: никогда не применяйте для обучения тестовые данные.

Сейчас может показаться, что следовать этому правилу легко. Но, как мы увидим в дальнейшем, его очень легко случайно нарушить.

На самом деле в этой главе мы уже нарушали золотое правило. Можете ли вы сказать, где именно? Вернитесь и попытайтесь найти это место. Мы увидим, где именно это произошло, уже в следующем разделе.

Где нарушено золотое правило и как это исправить? Контрольный набор

В этом разделе мы увидим, где было нарушено золотое правило, и разберем технику валидации, которая нам пригодится.

Мы нарушили золотое правило в разделе «Как заставить компьютер выбрать правильную модель?». Напомним, что у нас было три модели полиномиальной регрессии: одна 1-й степени, одна 2-й степени и одна 10-й степени — и мы не знали, какую из них выбрать. Мы использовали обучающие данные для обучения трех моделей, а затем задействовали тестовые данные, чтобы определиться с выбором. Но мы не должны применять тестовые данные для обучения модели или принятия каких-либо решений по поводу модели или ее гиперпараметров. Сделав это, мы рискуем столкнуться с переобучением! Мы потенциально переобучаемся каждый раз, когда строим модель, которая слишком хорошо соответствует набору данных.

Что же делать? Решение здесь простое: мы в очередной раз разбиваем набор данных. Введем новый набор — *контрольную выборку*, с помощью которой в дальнейшем будем принимать решения по набору данных. Итак, мы разбиваем набор данных на три части:

- **обучающий набор** для обучения всех наших моделей;
- **контрольный набор** для принятия решений о том, какую модель использовать;
- **тестовый набор** для проверки того, насколько хорошо модель справилась.

Таким образом, в примере были бы еще две контрольные точки, и наблюдение ошибки валидации помогло бы нам решить, что наилучшей моделью была бы модель 2. Тестовый же набор мы должны применять в самом конце, чтобы понять, насколько хорошо сработала модель. Если она недостаточно хороша, нужно все перечеркнуть и начать с нуля.

Что касается размеров наборов для тестирования и валидации, обычно используется разделение 60–20–20 или 80–10–10 — другими словами, 60 % для обучения, 20 % для валидации, 20 % для тестирования или 80 % для обучения, 10 % для валидации и 10 % для тестирования. Числа произвольны, но они, как правило, хорошо работают, так как оставляют большую часть данных для обучения, однако все же позволяют проверить модель на достаточно большом наборе.

Численный способ определить нужную сложность модели. График сложности модели

В предыдущих разделах мы узнали, как использовать контрольный набор, чтобы решить, какая из трех моделей подходит лучше всего. В этом разделе поговорим о *графике сложности модели*, который помогает выбирать среди множества других моделей. Представьте, что мы получили другой, гораздо более сложный набор данных и теперь пытаемся построить соответствующую полиномиальную регрессионную модель. Мы хотим выбрать степень модели из чисел от 0 до 10 включительно. Как сказано в предыдущем разделе, для этого требуется выбрать ту, которая имеет наименьшую ошибку валидации.

Построение графиков ошибок обучения и тестирования может дать кое-какую ценную информацию и помочь выявить тенденции. На рис. 4.5 мы видим график сложности модели, на котором горизонтальная ось представляет степень нескольких моделей полиномиальной регрессии от 0 до 10 (то есть сложность модели), а вертикальная ось — значение ошибки, которая в данном случае определяется средней абсолютной ошибкой. Ромбы представляют ошибку обучения, а кружки — ошибку валидации. Обратите внимание на то, что вначале ошибка обучения велика и уменьшается по мере того, как мы двигаемся вправо. Это происходит потому, что чем сложнее модель, тем лучше она соответствует данным обучения. Ошибка валидации вначале тоже велика, затем уменьшается и снова увеличивается — очень простые модели не способны хорошо соответствовать данным (они недообучены), в то время как очень сложные модели соответствуют обучающим данным, но не контрольным данным, потому что они переобучены. Золотая середина — это то место, где модель и не недообучена, и не переобучена, и мы можем найти ее с помощью графика сложности модели.

Обратите внимание на то, что на графике сложности модели наименьшее значение ошибки валидации имеет степень 4, что означает: для этого набора данных наиболее подходящей моделью (среди тех, которые мы рассматриваем) служит полиномиальная регрессионная модель степени 4. Глядя на левую часть графика, мы можем заметить, что, когда степень полинома мала, ошибки обучения и валидации велики, а это означает: модели нам не совсем подходят. Глядя на правую часть, видим, что ошибка обучения становится все меньше и меньше, а ошибка валидации — все больше и больше, а это означает: модели переобучаются. Нужный момент наступает в районе 4, и это требующаяся нам модель.

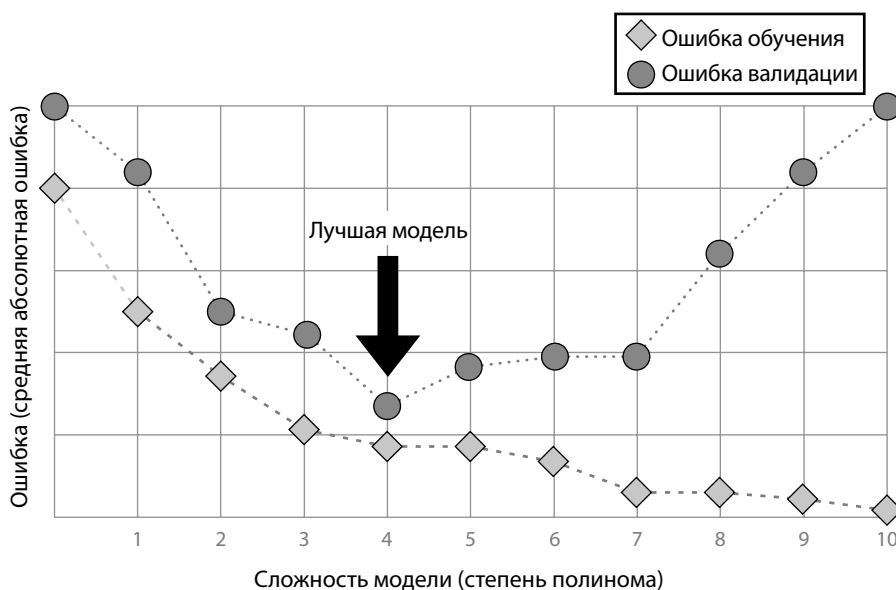


Рис. 4.5. График сложности модели — эффективный инструмент, помогающий определить идеальную сложность модели, чтобы избежать недообучения и переобучения

Одно из преимуществ графика сложности модели заключается в том, что независимо от того, насколько велик набор данных или сколько разных моделей мы опробуем, он всегда выглядит как две кривые: одна неизменно идет вниз (ошибка обучения), вторая идет вниз, а затем снова вверх (ошибка валидации). Конечно, в большом и сложном наборе данных эти кривые могут колебаться, и определить их поведение будет сложнее. Однако график сложности модели всегда служит полезным инструментом для специалистов по обработке данных, помогая найти подходящую точку и решить, насколько сложными должны быть модели, чтобы избежать как недостаточного, так и чрезмерного обучения.

Но зачем нужен подобный график, если нам всего лишь требуется выбрать модель с наименьшей ошибкой валидации? Этот метод верен в теории, но на практике вы, как специалист по обработке данных, можете иметь гораздо лучшее представление о решаемой проблеме, ограничениях и контрольных показателях. Если видите, например, что модель с наименьшей ошибкой валидации все еще довольно сложна и существует гораздо более простая модель, которая имеет чуть большую ошибку валидации, то можете склониться к выбору именно этой модели. Лучший специалист по обработке данных — тот, кто способен объединить теоретические инструменты с собственными реальными знаниями для построения наилучших и наиболее эффективных моделей.

Еще один способ избежать переобучения — регуляризация

В данном разделе мы обсудим еще один полезный метод, позволяющий избежать переобучения в моделях и не требующий тестового набора, — *регуляризацию*. Она основана на том же наблюдении, которое мы сделали в разделе «Пример недообучения и переобучения с использованием полиномиальной регрессии», где пришли к выводу, что простые модели имеют склонность к недообучению, а сложные — к переобучению. В рассмотренных ранее методах мы протестировали несколько моделей и выбрали ту, которая наилучшим образом сочетала производительность и сложность. Когда же мы применяем регуляризацию, нам не нужно обучать несколько моделей. Мы просто обучаем модель один раз, но во время этого стараемся не только улучшать ее производительность, но и уменьшать сложность. Ключом к этому является одновременное измерение производительности и сложности.

Прежде чем перейти к подробностям, рассмотрим аналогию, помогающую понять, что такое одновременное измерение производительности и сложности моделей. Представьте, что у нас есть три дома и у всех одна и та же проблема — протекает крыша (рис. 4.6). Приходят три кровельщика, и каждый чинит какой-то из домов. Первый кровельщик использовал тканевые материалы, второй — кровельную черепицу, а третий — титан. Интуитивно нам кажется, что лучшим вариантом является выбор второго кровельщика, потому что первый чрезмерно упростил задачу (недообучение), а третий чрезмерно усложнил (переобучение).



Рис. 4.6. Аналогия с недообучением и переобучением

Однако нам нужно принять решение на основе чисел, поэтому проведем некоторые измерения. Оценить работу кровельщиков можно по тому, сколько миллилитров воды натекло после того, как они починили крышу. Получились следующие оценки:

- первый кровельщик — 1000 мл;
- второй кровельщик — 1 мл;
- третий кровельщик — 0 мл.

Похоже, первый кровельщик справился ужасно, потому что крыша все еще течет. Но кого же выбрать из кровельщиков — второго или третьего? Возможно, третьего, у которого оказались наилучшие результаты? Показатель производительности недостаточно хорош: он правильно удаляет первого кровельщика из уравнения, но ошибочно указывает нам на третьего кровельщика вместо второго. Для принятия правильного решения необходимо оценить сложность их работы. Хорошим показателем послужит то, сколько они взяли с нас за ремонт крыши. Цены были следующими:

- первый кровельщик — 1 доллар;
- второй кровельщик — 100 долларов;
- третий кровельщик — 100 000 долларов.

Теперь мы можем сказать, что второй кровельщик лучше, чем третий, потому что у них оказалась схожая производительность, но второй взял меньшую плату. Тем не менее первый кровельщик был самым дешевым — почему тогда не выбрать его? Похоже, нам требуется объединить показатели производительности и сложности. Мы можем сложить количество воды, протекшей с крыши, и цену, чтобы получить следующее.

Производительность + сложность:

- первый кровельщик — 1001;
- второй кровельщик — 101;
- третий кровельщик — 100 000.

Теперь очевидно, что второй кровельщик является лучшим выбором, а это значит, что оптимизация производительности и сложности одновременно дает хорошие и к тому же максимально простые результаты. В этом и заключается регуляризация: измерение производительности и сложности с помощью двух разных функций ошибок и их сложение для получения более надежной функции ошибок. Эта новая функция ошибок гарантирует, что модель работает хорошо и не слишком сложна. В следующих разделах мы подробнее поговорим о том, как определить эти две функции ошибок. Но перед этим рассмотрим еще один пример переобучения.

Еще один пример переобучения — рекомендации фильмов

В этом разделе мы узнаем о менее заметном способе, в результате которого модель может быть переобучена, — на этот раз он связан не со степенью полинома, а с количеством признаков и размером коэффициентов. Представьте, что у нас имеется веб-сайт для потоковой передачи фильмов и мы пытаемся создать систему рекомендаций. Для простоты условимся, что у нас только десять

фильмов: M1, M2... M10. После выхода фильма M11 мы хотим построить модель линейной регрессии, чтобы рекомендовать его на основе предыдущих десяти. У нас есть набор данных из 100 пользователей. Для каждого из них имеем десять функций, представляющих собой время в секундах, в течение которого пользователь просматривал каждый из десяти фильмов. Если он не смотрел фильм, время равно 0. Метка для каждого пользователя — это количество времени, в течение которого он, предположительно, будет смотреть фильм M11. Нам нужно построить модель, которая соответствует этому набору данных. Учитывая, что она представляет собой модель линейной регрессии, уравнение для прогнозируемого времени, в течение которого пользователь будет смотреть фильм M11, является линейным и выглядит так:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6 + w_7x_7 + w_8x_8 + w_9x_9 + w_{10}x_{10} + b,$$

где $\hat{y}$ — количество времени, в течение которого, по прогнозам модели, пользователь будет смотреть фильм M11, x_i — количество времени, в течение которого пользователь смотрел фильм i , при $i = 1, 2 \dots 10$, w_i — вес, связанный с фильмом i , b — смещение.

Теперь проверим свою интуицию. Какая (или какие) из следующих двух моделей, заданных уравнениями, выглядит так, как будто она может быть переобучена?

Модель 1: $\hat{y} = 2x_3 + 1,4x_7 - 0,5x_9 + 4.$

Модель 2: $\hat{y} = 22x_1 - 103x_2 - 14x_3 + 109x_4 - 93x_5 + 203x_6 + 87x_7 - 55x_8 + 378x_9 - 25x_{10} + 8.$

Если вы думаете так же, как и я, модель 2 кажется несколько сложной, и, возможно, именно она переобучена. Интуиция говорит нам о том, что маловероятно, что время, в течение которого пользователь смотрел фильм 2, нужно умножить на -103 , а затем добавить к другим числам, чтобы получить прогноз. Это может хорошо соответствовать данным, но определенно выглядит так, как будто она запоминает данные, а не изучает их.

Модель 1 выглядит намного проще и дает нам кое-какую интересную информацию. Из того факта, что большинство коэффициентов равно нулю, за исключением коэффициентов для фильмов 3, 7 и 9, мы понимаем, что единственными тремя фильмами, связанными с фильмом 11, являются как раз эти три. Кроме того, из того факта, что коэффициенты фильмов 3 и 7 положительны, мы понимаем, что если пользователь смотрел фильм 3 или 7, то он, скорее всего, посмотрит и фильм 11. Поскольку коэффициент фильма 9 отрицательный, то, если пользователь смотрел фильм 9, он, скорее всего, не будет смотреть фильм 11.

Наша цель — построить модель, подобную модели 1, и избегать моделей, подобных модели 2. Но, к сожалению, если модель 2 выдает меньшую ошибку, нежели модель 1, то при запуске алгоритма линейной регрессии будет выбрана модель 2. Что же делать? Вот тут-то и приходит на помощь регуляризация. Первое, что нам нужно, — это показатель, который говорит о том, что модель 2 гораздо сложнее модели 1.

Измерение сложности модели. Норма L1 и L2

В этом разделе мы изучим два способа измерения сложности модели. Но перед этим посмотрим на модели 1 и 2 из предыдущего раздела и попробуем придумать какую-нибудь формулу, которая дает небольшой результат для модели 1 и большой — для модели 2.

Обратите внимание на то, что модель с большим количеством коэффициентов или более высокими коэффициентами, как правило, сложнее. Следовательно, любая формула, соответствующая этому, будет работать. Например, такая:

- сумма абсолютных значений коэффициентов;
- сумма квадратов коэффициентов.

Первая из них называется *нормой L1*, а вторая — *нормой L2*. Они вытекают из более общей теории L^p -пространств, названной в честь французского математика Анри Лебега. Мы используем абсолютные значения и квадраты, чтобы избавиться от отрицательных коэффициентов. В противном случае большие отрицательные числа будут компенсироваться большими положительными числами и мы можем получить небольшое значение для очень сложной модели.

Но прежде чем начать вычислять нормы, нужно упомянуть небольшую техническую деталь: смещение в моделях не включено в норму L1 и L2. Почему? Смещение в модели — это именно то количество секунд, в течение которого (как мы ожидаем) пользователь будет смотреть фильм 11, если он не смотрел ни одного из предыдущих 10 фильмов. Это число не связано со сложностью модели, поэтому мы его не трогаем. Далее приводится расчет нормы L1 для моделей 1 и 2.

Напомним, что уравнения для моделей следующие.

Модель 1: $\hat{y} = 2x_3 + 1,4x_7 - 0,5x_9 + 8$.

Модель 2: $\hat{y} = 22x_1 - 103x_2 - 14x_3 + 109x_4 - 93x_5 + 203x_6 + 87x_7 - 55x_8 + 378x_9 - 25x_{10} + 8$.

Норма L1:

- **модель 1:** $|2| + |1,4| + |-0,5| = 3,9$;
- **модель 2:** $|22| + |-103| + |-14| + |109| + |-93| + |203| + |87| + |-55| + |378| + |-25| = 1,089$.

Норма L2:

- **модель 1:** $2^2 + 1,4^2 + (-0,5)^2 = 6,21$;
- **модель 2:** $22^2 + (-103)^2 + (-14)^2 + 109^2 + (-93)^2 + 203^2 + 87^2 + (-55)^2 + 378^2 + (-25)^2 = 227,131$.

Как и ожидалось, нормы L1 и L2 модели 2 намного больше, чем соответствующие нормы модели 1.

Нормы L1 и L2 могут быть вычислены также по полиномам, если взять либо сумму абсолютных значений, либо сумму квадратов коэффициентов, за исключением постоянного. Вернемся к примеру из начала главы, где три модели представляли собой полином степени 1 (прямая), степени 2 (парабола) и степени 10 (кривая, которая колеблется 9 раз). Представьте, что их формулы следующие:

- **модель 1:** $\hat{y} = 2x + 3$;
- **модель 2:** $\hat{y} = -x^2 + 6x - 2$;
- **модель 3:** $\hat{y} = x^9 + 4x^8 - 9x^7 + 3x^6 - 14x^5 - 2x^4 - 9x^3 + x^2 + 6x + 10$.

Нормы L1 и L2 рассчитываются следующим образом.

Норма L1:

- **модель 1:** $|2| = 2$;
- **модель 2:** $|-1| + |6| = 7$;
- **модель 3:** $|1| + |4| + |-9| + |3| + |-14| + |-2| + |-9| + |1| + |6| = 49$.

Норма L2:

- **модель 1:** $2^2 = 4$;
- **модель 2:** $(-1)^2 + 6^2 = 37$;
- **модель 3:** $1^2 + 4^2 + (-9)^2 + 3^2 + (-14)^2 + (-2)^2 + (-9)^2 + 1^2 + 6^2 = 425$.

Теперь, когда у нас есть два способа измерения сложности моделей, приступим к процессу обучения.

Изменение функции ошибки для решения проблемы. Регрессия лассо и гребневая регрессия

Теперь, проделав большую часть тяжелой работы, подготовим модель линейной регрессии с использованием регуляризации. Для модели у нас есть две меры: мера производительности (функция ошибок) и мера сложности (норма L1 или L2).

Напомним, что в аналогии с кровельщиком нашей целью было найти кровельщика, который обеспечивал бы как хорошее качество, так и низкую сложность. Мы проделали это, сведя к минимуму сумму двух чисел: показатель качества и показатель сложности. Регуляризация состоит в применении того же принципа к модели машинного обучения. Для этого у нас есть две величины: ошибка регрессии и слагаемое регуляризации.

ОШИБКА РЕГРЕССИИ — показатель качества модели. В данном случае это могут быть абсолютные или квадратические погрешности, о которых мы узнали в главе 3.

СЛАГАЕМОЕ РЕГУЛЯРИЗАЦИИ — мера сложности модели. Это может быть норма L1 или L2 модели.

Величина, которую мы хотим свести к минимуму, чтобы найти хорошую и не слишком сложную модель, — это модифицированная ошибка, определяемая как сумма двух слагаемых:

$$\text{ошибка} = \text{ошибка регрессии} + \text{слагаемое регуляризации}.$$

Регуляризация настолько распространена, что сами модели имеют разные названия в зависимости от того, какая норма используется. Если мы обучаем регрессионную модель с помощью нормы L1, эта модель называется *регрессией лассо*. «Лассо» расшифровывается как «оператор наименьшей абсолютной усадки и отбора» (least absolute shrinkage and selection operator). Функция ошибки:

$$\text{ошибка регрессии лассо} = \text{ошибка регрессии} + \text{норма L1}.$$

Если же мы обучаем модель с помощью нормы L2, это называется *гребневой регрессией*. Название «гребневая» происходит от формы функции ошибки, потому что при построении добавление члена нормы L2 к функции ошибки регрессии превращает острый угол в пологую долину. Функция ошибки:

$$\text{ошибка гребневой регрессии} = \text{ошибка регрессии} + \text{норма L2}.$$

Как лассо, так и гребневая регрессия хорошо работают на практике. Решение о том, какую из них использовать, зависит от ряда особенностей, о которых мы узнаем в следующих разделах. Но прежде чем перейти к ним, нужно проработать некоторые детали и убедиться, что наши регуляризованные модели работают как следует.

Регулирование уровня производительности и сложности модели. Параметр регуляризации

Поскольку процесс обучения модели предполагает максимально возможное снижение функции затрат, модель, обученная с помощью регуляризации, в принципе должна быть высокопроизводительной и несложной. Мы наблюдаем своеобразное перетягивание каната — попытка улучшить производительность модели может сделать ее более сложной, а попытка упростить ее может снизить производительность. К счастью, большинство методов машинного обучения снабжены регуляторами (гиперпараметрами), которые специалист по обработке данных может использовать для построения наилучших моделей из возможных, и регуляризация не исключение. В этом разделе мы увидим, как применять гиперпараметр для настройки производительности и сложности.

Этот гиперпараметр называется *параметром регуляризации*, его цель в том, чтобы определить, должен ли процесс обучения модели опираться на производительность или простоту. Параметр регуляризации обозначается λ (греческая буква «лямбда»). Мы умножаем слагаемое регуляризации на λ , добавляем его к ошибке регрессии и используем результат для обучения модели. Новая ошибка выглядит так:

$$\text{ошибка} = \text{ошибка регрессии} + \lambda \text{ слагаемое регуляризации}.$$

Выбор значения 0 для λ отменяет слагаемое регуляризации, таким образом, мы получаем ту же регрессионную модель, что и в главе 3. Выбор большого значения λ дает простую модель, возможно, низкой степени, которая может не очень хорошо соответствовать набору данных. Очень важно выбрать хорошее значение λ , и отличным методом для этого является валидация. Обычно выбирают степени 10, такие как 10; 1; 0,1; 0,01, но в целом это произвольно. Среди них мы выбираем число, которое обеспечивает наилучшую производительность модели на контрольном наборе.

Эффекты регуляризации L1 и L2 в коэффициентах модели

В этом разделе мы увидим принципиальные различия между регуляризациями L1 и L2 и выработаем некоторые идеи относительно того, какую из них использовать в различных сценариях. На первый взгляд они похожи, но их влияние на коэффициенты весьма интересно. В зависимости от того, какой тип модели нам нужен, выбор между применением регуляризации L1 и L2 может оказаться решающим.

Вернемся к примеру с рекомендациями фильмов, где мы строим регрессионную модель для прогнозирования количества времени в секундах, в течение которого пользователь будет смотреть фильм. Наш прогноз базируется на количестве времени, в течение которого один и тот же человек смотрел десять разных фильмов. Представим, что мы обучили модель и полученное уравнение выглядит так:

$$\hat{y} = 22x_1 - 103x_2 - 14x_3 + 109x_4 - 93x_5 + 203x_6 + 87x_7 - 55x_8 + 378x_9 - 25x_{10} + 8.$$

Если добавим регуляризацию и снова обучим модель, то получим более простую модель. Следующие два признака могут быть показаны математически.

- Если мы используем регуляризацию L1 (регрессию лассо), то получим модель с меньшим количеством коэффициентов. Другими словами, регуляризация L1 обращает некоторые коэффициенты в ноль. Таким образом, мы можем получить уравнение типа $\hat{y} = 2x_3 + 1,4x_7 - 0,5x_9 + 8$.
- Если мы задействуем регуляризацию L2 (гребневую регрессию), то получим модель с меньшими коэффициентами. Другими словами, регуляризация L2 уменьшает все коэффициенты, но редко превращает их в ноль. Таким образом, мы можем получить уравнение типа

$$\begin{aligned} \hat{y} = & 0,2x_1 - 0,8x_2 - 1,1x_3 + 2,4x_4 - 0,03x_5 + 1,02x_6 + \\ & + 3,1x_7 - 2x_8 + 2,9x_9 - 0,04x_{10} + 8. \end{aligned}$$

Итак, в зависимости от того, какое уравнение нам требуется, мы можем выбрать между регуляризациями L1 и L2.

Краткое эмпирическое правило, которое стоит применять, решая, регуляризацию L1 или L2 использовать, гласит: если у нас слишком много признаков и мы хотели бы избавиться от большинства из них, регуляризация L1 подойдет идеально. Если функций всего несколько и мы считаем, что все они

релевантны, то регуляризация L2 — то, что нужно, потому что она не удалит полезные признаки.

Пример проблемы, в которой есть много признаков (и регуляризация L1 может нам помочь), — система рекомендаций фильмов, которую мы изучали в разделе «Еще один пример переобучения — рекомендации фильмов». В этой модели каждый признак соответствует одному из фильмов, и наша цель — найти несколько фильмов, связанных с тем, который нас интересует. Таким образом, нам нужна модель, для которой большинство коэффициентов равны нулю, за исключением нескольких.

Примером, в котором мы должны задействовать регуляризацию L2, может служить полиномиальный пример, приведенный в начале раздела «Пример недообучения и переобучения с использованием полиномиальной регрессии». Для этой модели у нас был только один признак, x . Регуляризация L2 даст хорошую полиномиальную модель с малыми коэффициентами, которая не будет сильно колебаться и, следовательно, реже рискует переобучиться. В разделе «Полиномиальная регрессия, тестирование и регуляризация с помощью Turi Create» мы увидим полиномиальный пример, для которого правильнее применять регуляризацию L2.

Ресурсы, соответствующие этой главе (см. приложение В), позволят вам углубиться в математические причины, по которым регуляризация L1 превращает коэффициенты в ноль, тогда как регуляризация L2 превращает их в малые числа. В следующем разделе мы узнаем, как задействовать для этого интуицию.

Интуитивный способ понять регуляризацию

В этом разделе мы узнаем о том, что нормы L1 и L2 отличаются тем, как они ухудшают сложность. Этот раздел в основном интуитивно понятен и разработан в качестве примера, но, если вы хотите углубиться в стоящую за ним формальную математику, обратитесь к приложению Б. Пытаясь понять, как работает модель машинного обучения, мы должны смотреть дальше функции ошибок. Функция ошибки говорит: «Это ошибка, и если вы уменьшите ее, то в итоге получите хорошую модель». Но это все равно что сказать: «Секрет успеха в жизни заключается в том, чтобы совершать как можно меньше ошибок». Разве не лучше позитивное послание, например: «Вот то, что вы можете сделать, чтобы улучшить свою жизнь», чем такое: «Это то, чего вам следует избегать»? Давайте взглянем на регуляризацию с этой точки зрения.

В главе 3 мы изучили абсолютный и квадратический подходы, которые позволили получить более четкое представление о регрессии. На каждом этапе процесса обучения мы просто выбираем точку или несколько точек и приближаем к ним прямую. Многократное повторение этого процесса в итоге обеспечит хорошую подгонку прямой. Можно повторить то, как мы определили алгоритм линейной регрессии в главе 3.

Псевдокод для алгоритма линейной регрессии

Входные данные — набор данных точек.

Результат — модель линейной регрессии, соответствующая этому набору данных.

Процедура

- Выберите модель со случайными весами и случайным смещением.
- Повторяйте многократно:
 - выберите случайную точку данных;
 - слегка отрегулируйте веса и смещение, чтобы улучшить прогнозирование для нее.
- Модель готова!

Можно ли использовать подобные рассуждения, чтобы понять регуляризацию? Да, можно.

Для простоты предположим, что мы находимся в середине обучения и хотим упростить модель. Это можно сделать, уменьшив коэффициенты. Для простоты предположим, что модель имеет три коэффициента: 3, 10 и 18. Можем ли мы сделать маленький шаг и уменьшить их на небольшую величину? Конечно, можем! Вот два способа сделать это. Для обоих требуется небольшое число λ , которое мы пока установим равным 0,01.

Способ 1. Вычтите λ из каждого из положительных параметров и добавьте λ к каждому из отрицательных. Если они равны нулю, не меняйте их.

Способ 2. Умножьте их все на $1 - \lambda$. Обратите внимание на то, что это число близко к 1, потому что λ мало.

Используя метод 1, мы получаем числа 2,99, 9,99 и 17,99.

Применяя метод 2, получаем числа 2,97, 9,9 и 17,82.

В этом случае λ ведет себя подобно скорости обучения. Фактически она тесно связана со скоростью регуляризации (подробности см. в разделе «Использование градиентного спуска для регуляризации» в приложении Б). Обратите внимание на то, что в обоих случаях мы уменьшаем размер коэффициентов. Теперь нужно лишь многократно уменьшать коэффициенты на каждом этапе алгоритма. Другими словами, вот как мы сейчас обучаем модель.

Входные данные — набор данных точек.

Результат — модель линейной регрессии, соответствующая этому набору данных.

Процедура

- Выберите модель со случайными весами и случайным смещением.
- Повторяйте многократно:
 - выберите случайную точку данных;
 - слегка отрегулируйте веса и смещение, чтобы улучшить прогноз для этой конкретной точки данных;
 - **слегка уменьшите коэффициенты, используя метод 1 или 2.**
- Модель готова!

Используя метод 1, мы обучаем модель с помощью регуляризации L1 или регрессии лассо. А если мы применяем метод 2, то обучаем с помощью регуляризации L2 или гребневой регрессии. Для этого имеется математическое обоснование, которое описано в приложении Б в разделе «Использование градиентного спуска для регуляризации».

В предыдущем разделе мы узнали, что регуляризация L1 имеет тенденцию превращать многие коэффициенты в 0, тогда как регуляризация L2 уменьшает их, но не превращает в ноль. Это явление теперь гораздо легче понять. Предположим, что коэффициент равен 2, а параметр регуляризации $\lambda = 0,01$. Обратите внимание на то, что произойдет, если мы используем метод 1 для уменьшения коэффициента и повторим этот процесс 200 раз. Мы получим такую последовательность значений:

$$2 \rightarrow 1,99 \rightarrow 1,98 \rightarrow \dots \rightarrow 0,02 \rightarrow 0,01 \rightarrow 0.$$

После 200 периодов обучения коэффициент становится равным 0 и больше не меняется. Теперь посмотрим, что произойдет, если мы 200 раз применим метод 2 с той же скоростью обучения $\eta = 0,01$. Получим следующую последовательность значений:

$$2 \rightarrow 1,98 \rightarrow 1,9602 \rightarrow \dots \rightarrow 0,2734 \rightarrow 0,2707 \rightarrow 0,268.$$

Обратите внимание на то, что коэффициент резко снизился, но не стал нулевым. Независимо от числа периодов он никогда не станет равным нулю. Когда мы многократно умножаем неотрицательное число на 0,99, оно никогда не становится равным нулю. Это показано на рис. 4.7.

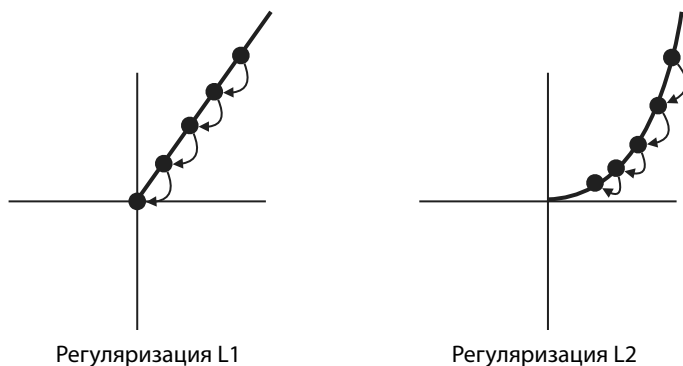


Рис. 4.7. Уменьшение размера коэффициента с помощью L1 и L2

Регуляризация L1 (*слева*) уменьшает размер коэффициента намного быстрее, поскольку вычитает фиксированную величину. Поэтому, скорее всего, в итоге он станет нулевым. Регуляризация L2 (*справа*) занимает гораздо больше времени, так как умножает коэффициент на небольшое число. Поэтому он никогда не будет равен нулю.

Полиномиальная регрессия, тестирование и регуляризация с помощью Turi Create

В этом разделе рассмотрим пример полиномиальной регрессии с регуляризацией в Turi Create. Вот код для этого раздела.

- **Блокнот:** https://github.com/luisguiserrano/manning/blob/master/Chapter_4_Testing_Overfitting_Underfitting/Polynomial_regression_regularization.ipynb.

Мы начнем с набора данных, который показан на рис. 4.8. Видим, что кривая, которая лучше всего соответствует этим данным, представляет собой параболу, которая открывается вниз (грустный смайлик). Так что это не та задача, которую можно решить с помощью линейной регрессии, — мы должны использовать полиномиальную регрессию. Кроме того, применим регуляризацию для настройки нашей модели. Набор данных хранится в SFrame под именем `data`, его первые несколько строк приведены в табл. 4.1.

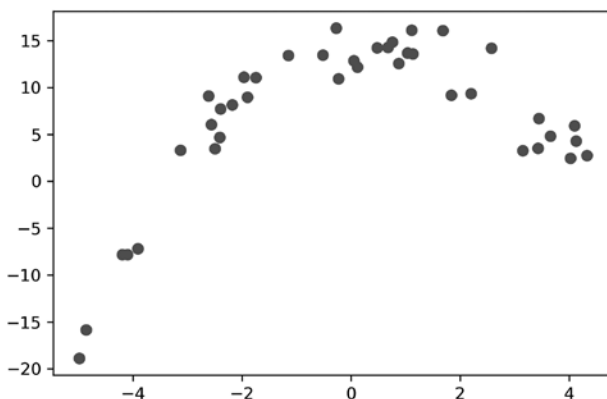


Рис. 4.8. Набор данных

Таблица 4.1. Первые четыре строки набора данных

<i>x</i>	<i>y</i>
3,4442185152504816	6,685961311021467
-2,4108324970703663	4,690236225597948
0,11274721368608542	12,205789026637378
-1,9668727392107255	11,133217991032268

Реализация полиномиальной регрессии в Turi Create состоит в том, чтобы добавить в исходный набор данных множество столбцов, соответствующих степеням основного признака, и применить линейную регрессию к расширенному

набору данных. Если основным признаком является, скажем, x , то добавляем столбцы со значениями x^2 , x^3 , x^4 и т. д. Таким образом, наша модель находит линейные комбинации степеней x , которые в точности являются полиномами в x . Если `SFrame`, содержащий данные, называется `data`, то для добавления столбцов для степеней до x^{199} мы применяем следующий код:

```
for i in range(2,200):
    string = 'x^'+str(i)
    data[string] = data['x'].apply(lambda x:x**i)
```

Первые несколько строк и столбцов результирующего набора данных приведены в табл. 4.2. Столбец x^k соответствует переменной x^k , где $k = 2, 3, 4$. Набор данных содержит 200 столбцов.

Таблица 4.2. Четыре верхние строки и пять крайних левых столбцов нашего набора данных

x	y	x^2	x^3	x^4
3,445	6,686	11,863	40,858	140,722
-2,411	4,690	5,812	-14,012	33,781
0,113	12,206	0,013	0,001	0,000
-1,967	11,133	3,869	-7,609	14,966

Теперь применим линейную регрессию к этому набору данных с 200 столбцами. Обратите внимание на то, что модель линейной регрессии здесь выглядит как линейная комбинация переменных в столбцах. Но поскольку каждый столбец соответствует одночлену, то полученная модель выглядит как многочлен по переменной x .

Прежде чем обучать какие-то модели, нужно разделить данные на обучающий и тестовый наборы данных, используя следующую строку кода:

```
train, test = data.random_split(.8)
```

Теперь набор данных разделен на обучающий набор `train` и тестовый набор `test`. В репозитории указано случайное начальное значение, поэтому мы всегда получаем одни и те же результаты, хотя на практике в этом нет необходимости.

Способ применения регуляризации в `Turi Create` прост: нам нужно лишь указать параметры `l1_penalty` и `l2_penalty` в методе `create` при обучении модели. Этот штраф является именно тем параметром регуляризации, который мы ввели в разделе «Регулирование уровня производительности и сложности модели...». Штраф 0 означает, что мы не используем регуляризацию. Таким образом, мы будем обучать три разные модели со следующими параметрами.

- Модель без регуляризации:
 - `l1_penalty=0`;
 - `l2_penalty=0`.

- Модель регуляризации L1:
 - `l1_penalty=0.1`;
 - `l2_penalty=0`.
- Модель регуляризации L2:
 - `l1_penalty=0`;
 - `l2_penalty=0.1`.

Обучим модели с помощью следующих трех строк кода:

```
model_no_reg = tc.linear_regression.create(train, target='y',
    l1_penalty=0.0, l2_penalty=0.0)
model_L1_reg = tc.linear_regression.create(train, target='y',
    l1_penalty=0.1, l2_penalty=0.0)
model_L2_reg = tc.linear_regression.create(train, target='y',
    l1_penalty=0.0, l2_penalty=0.1)
```

Первая модель не использует регуляризацию, вторая применяет регуляризацию L1 с параметром 0,1, а третья — регуляризацию L2 с параметром 0,1. Графики результирующих функций показаны на рис. 4.9. Обратите внимание на то, что на нем точки в обучающем наборе показаны кружками, а точки в тестовом наборе — треугольниками.

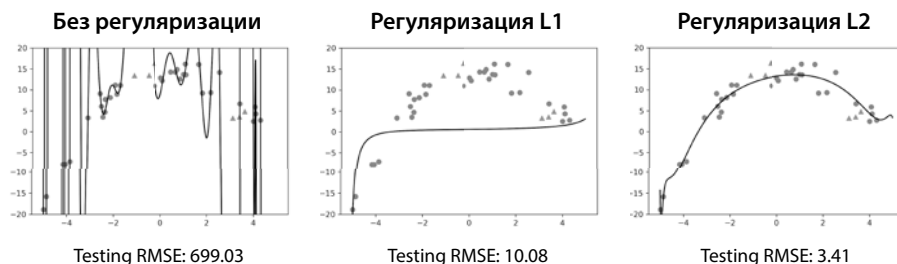


Рис. 4.9. Три модели полиномиальной регрессии для набора данных: слева — без регуляризации; по центру — регуляризация L1 с параметром 0,1; справа — регуляризация L2 с параметром 0,1

Обратите внимание на то, что модель без регуляризации довольно хорошо подходит для обучающих точек, но она хаотична и плохо работает для тестовых точек. Модель с регуляризацией L1 хорошо работает как с обучающими, так и с тестовыми наборами. Модель с регуляризацией L2 прекрасно справляется как с обучающими, так и с тестовыми наборами и, похоже, действительно отражает форму данных.

Также обратите внимание: во всех трех моделях граничная кривая несколько хаотична в конечных точках. Это вполне объяснимо, потому что конечные точки имеют меньше данных и для модели вполне естественно не знать, что делать, когда данных нет. Оценивать модели следует по тому, хорошо ли они работают внутри границ набора данных, и мы не должны ожидать, что модель будет хорошо работать за этими границами. Даже мы, люди, скорее всего, не

сможем строить хорошие прогнозы за пределами границ модели. Например, как, по-вашему, эта кривая будет выглядеть за пределами набора данных? Будет ли она продолжаться как парабола, которая открывается вниз? Будет ли колебаться вечно, как синусоидальная функция? Если мы этого не знаем, то не следует ожидать, что это знает модель. Таким образом, постарайтесь не обращать внимания на странное поведение кривой в конечных точках на рис. 4.9 и сосредоточьтесь на поведении модели внутри интервала, в котором расположены данные.

Чтобы найти ошибку тестирования, используем следующую строку кода с соответствующим именем модели. Эта строка возвращает максимальную ошибку и среднюю квадратическую погрешность:

```
model.predict(test)
```

Тестовые средние квадратические погрешности для моделей приведены далее:

- модель без регуляризации — 699,03;
- модель с регуляризацией L1 — 10,08;
- модель с регуляризацией L2 — 3,41.

Модель без регуляризации получила действительно большую среднюю квадратическую ошибку! Из двух других моделей намного лучше сработала модель с регуляризацией L2. Вот два вопроса, над которыми стоит задуматься.

1. Почему модель с регуляризацией L2 показала лучшие результаты, чем модель с регуляризацией L1?
2. Почему модель с регуляризацией L1 выглядит плоской, в то время как модели с регуляризацией L2 удалось отразить форму данных?

На эти два вопроса можно ответить одинаково, и, чтобы найти этот ответ, мы рассмотрим коэффициенты многочленов. Их можно получить с помощью следующей строки кода:

```
model.coefficients
```

Каждый многочлен имеет 200 коэффициентов, поэтому мы не будем отображать их все, но в табл. 4.3 вы видите первые пять коэффициентов для трех моделей. Что вы заметили? Надеюсь, то, что модель без регуляризации получила большие коэффициенты, модель с регуляризацией L1 — коэффициенты, очень близкие к 0, а модель с регуляризацией L2 — малые коэффициенты.

Таблица 4.3. Первые пять коэффициентов полиномов в трех моделях

Коэффициент	model_no_reg	model_L1_reg	model_L2_reg
$x^0 = 1$	8,41	0,5700	13,240
x^1	15,87	0,0700	0,870
x^2	108,87	-0,0040	-0,520
x^3	-212,89	0,0002	0,006
x^4	-97,13	-0,0002	-0,020

Взглянув на табл. 4.3, мы видим, что предсказания для трех моделей являются полиномами степени 200. Первые слагаемые выглядят следующим образом:

- модель без регуляризации — $\hat{y} = 8,41 + 15,87x + 108,87x^2 - 212,89x^3 - 97,13x^4 + \dots$
- модель с регуляризацией L1 — $\hat{y} = 0,57 + 0,07x - 0,004x^2 + 0,0002x^3 - 0,0002x^4 + \dots$
- модель с регуляризацией L2 — $\hat{y} = 13,24 + 0,87x - 0,52x^2 + 0,006x^3 - 0,02x^4 + \dots$

Из этих многочленов мы можем вывести следующее.

- Для модели без регуляризации все коэффициенты большие. Это означает, что многочлен хаотичен и не подходит для прогнозирования.
- Для модели с регуляризацией L1 все коэффициенты, за исключением постоянного (первого), малы — почти 0. Это означает, что при значениях, близких к нулю, многочлен очень похож на горизонтальную прямую $\hat{y} = 0,57$. Это лучше, чем предыдущая модель, но все же не очень подходит для составления прогнозов.
- Для модели с регуляризацией L2 коэффициенты уменьшаются с ростом степени, но все еще не так уж малы. Это дает вполне приличный многочлен для составления прогнозов.

Резюме

- Когда дело доходит до обучающих моделей, возникает множество проблем. Две проблемы, которые наблюдаются довольно часто, — это недообучение и переобучение.
- Недообучение возникает, когда мы используем для набора данных слишком простую модель. Переобучение происходит, когда применяемая модель слишком сложна.
- Эффективный способ выявить переобучение и недообучение — использование тестового набора данных.
- Чтобы протестировать модель, мы разделяем данные на два набора: обучающий и тестовый. Обучающий набор применяется для обучения модели, а тестовый — для ее оценки.
- Золотое правило машинного обучения заключается в том, чтобы никогда не задействовать тестовые данные для обучения или принятия решений в моделях.
- Контрольный набор — это еще одна часть набора данных, с помощью которой принимаем решения о гиперпараметрах модели.

- Недостаточно подходящая модель будет плохо работать на обучающем и контрольном наборах. Переобученная модель будет хорошо работать на обучающем наборе, но плохо на контрольном. Хорошая модель будет прекрасно работать как на обучающем, так и на контрольном наборе.
- График сложности модели используется для корректного определения ее сложности, чтобы она не переобучалась и не недообучалась.
- Регуляризация — крайне важный метод уменьшения переобучения в моделях машинного обучения. Он состоит в добавлении меры сложности (слагаемого регуляризации) к функции ошибки во время обучения.
- Нормы L1 и L2 — два наиболее распространенных показателя сложности, применяемых при регуляризации.
- Использование нормы L1 приводит к регуляризации L1, или регрессии лассо, нормы L2 — к регуляризации L2, или гребневой регрессии.
- Регуляризация L1 рекомендуется, когда набор данных содержит множество признаков и мы хотим свести многие из них к нулю. Регуляризацию L2 стоит применять, когда признаков в наборе данных мало и мы хотим сделать их маленькими, но не равными нулю.

Упражнения

Упражнение 4.1

Мы обучили четыре модели на одном и том же наборе данных с разными гиперпараметрами. В следующей таблице записали ошибки обучения и тестирования для каждой из моделей.

Модель	Ошибка обучения	Ошибка тестирования
1	0,1	1,8
2	0,4	1,2
3	0,6	0,8
4	1,9	2,3

- Какую модель вы бы выбрали для заданного набора данных?
- Какая модель выглядит так, будто она недостаточно хорошо соответствует данным?
- Какая модель выглядит так, будто она переобучена?

Упражнение 4.2

Имеется следующий набор данных.

x	y
1	2,0
2	2,5
3	6,0
4	14,5
5	34,0

Мы обучаем модель полиномиальной регрессии, которая предсказывает значение y как $\hat{y}$, где

$$\hat{y} = 2x^2 - 5x + 4.$$

Если параметр регуляризации $\lambda = 0,1$, а функция ошибки, которую мы использовали для обучения этого набора данных, представляет собой среднее абсолютное значение (MAE), определите следующее.

- А. Ошибку регрессии лассо модели (с помощью нормы L1).
- Б. Ошибку гребневой регрессии модели (с помощью нормы L2).

Использование прямых для разделения точек. Алгоритм персептрона

5



В этой главе

- ✓ Что такое классификация.
- ✓ Анализ настроений: как с помощью машинного обучения отделить радостные предложения от грустных.
- ✓ Как провести прямую, разделяющую точки двух цветов.
- ✓ Что такое персептрон и как мы его обучаем.
- ✓ Кодирование алгоритма персептрона на Python и Turi Create.



В этой главе мы изучим раздел машинного обучения, называемый *классификацией*. Модели классификации аналогичны регрессионным моделям в том смысле, что их целью является прогнозирование меток набора данных на основе признаков. Разница заключается в том, что регрессионные модели нацелены на предсказание числа, а классификационные — на предсказание состояния или категории. Классификационные модели часто называют *классификаторами*, и мы будем использовать данные термины как взаимозаменяемые. Многие классификаторы предсказывают одно из двух возможных состояний (чаще всего «да/нет»), хотя можно построить и такие, которые прогнозируют выбор из большего числа возможных состояний. Далее приведены наиболее известные примеры классификаторов.

- Модель рекомендаций, которая прогнозирует, заинтересует ли пользователя определенный фильм.
- Модель электронной почты, которая прогнозирует, является ли электронное письмо спамом.
- Медицинская модель, которая прогнозирует, болен пациент или здоров.
- Модель распознавания изображений, которая прогнозирует, содержит ли изображение автомобиль, птицу, кошку или собаку.
- Модель распознавания голоса, которая прогнозирует, произнес ли пользователь определенную команду.

Классификация — популярная область машинного обучения, и основная часть глав этой книги (главы 5, 6, 8–12) рассказывает о различных моделях классификации. Здесь мы изучим модель *персептрона*, также называемую *персептронным классификатором*, или просто *персептроном*. Персептрон похож на модель линейной регрессии в том смысле, что он применяет линейную комбинацию признаков для прогнозирования, а также служит строительным блоком нейронных сетей (о них мы узнаем в главе 10). Более того, процесс обучения персептрона аналогичен процессу обучения модели линейной регрессии. Так же, как алгоритм линейной регрессии в главе 3, мы разработаем алгоритм персептрона двумя способами: используя подход, который можно повторять

многократно, и определяя функцию ошибки, которую можно минимизировать с помощью градиентного спуска.

Основным примером моделей классификации, который станем изучать в этой главе, является *анализ настроений*. При анализе настроений цель модели в том, чтобы спрогнозировать настроение предложения. Другими словами, модель предсказывает, радостное предложение или грустное. Например, хорошая модель анализа настроений способна предсказать, что предложение «Я чувствую себя прекрасно!» — радостное, а «Какой ужасный день!» — грустное.

Анализ настроений используется во многих практических приложениях.

- Когда компания анализирует разговоры между клиентами и службой технической поддержки, чтобы оценить качество консультации.
- При анализе тона цифрового присутствия бренда, например комментариев в социальных сетях или отзывов, связанных с его продуктами.
- Когда социальная платформа, такая как Twitter, анализирует общее настроение определенной группы людей после какого-либо события.
- Когда инвестор использует общественное мнение о компании для прогнозирования цены акций.

Каким образом можно построить классификатор анализа настроений? Другими словами, как создать модель машинного обучения, которая принимает предложение в качестве входных данных, а в качестве выходных — сообщает, радостное предложение или грустное. Конечно, эта модель будет допускать ошибки, но постараемся построить ее так, чтобы их было как можно меньше. Давайте отложим книгу на пару минут и подумаем, как мы могли бы создать подобную модель.

Вот какова моя идея. Радостные предложения, как правило, содержат слова радости, такие как *замечательный*, *счастливый* или *радость*, в то время как грустные предложения, как правило, содержат грустные слова, такие как *ужасно*, *грустно* или *отчаяние*. Классификатор может оценивать «радость» каждого отдельного слова в словаре. Радостным словам можно присваивать положительные оценки, а грустным — отрицательные. Нейтральным словам, таким как *предлоги*, можно присваивать нулевой балл. Когда мы вводим предложение в классификатор, он просто складывает оценки всех слов. Если результат положительный, то классификатор делает вывод, что предложение радостное. Если результат отрицательный, он делает вывод, что предложение грустное. Теперь цель состоит в том, чтобы задать оценки для всех слов в словаре. Для этого мы и используем машинное обучение.

Только что построенная модель называется *моделью персептрона*. В этой главе мы узнаем формальное определение персептрона, а также как его обучать, находя идеальные оценки для всех слов, чтобы классификатор допускал как можно меньше ошибок.

Процесс обучения персептрона называется *алгоритмом персептрона*, и он не так уж сильно отличается от алгоритма линейной регрессии, который мы изучили в главе 3. Вот идея алгоритма персептрона: для обучения модели нам

прежде всего нужен набор данных, содержащий множество предложений вместе с метками (радостный/грустный). Мы начинаем строить классификатор, присваивая каждому слову случайную оценку. Затем несколько раз просматриваем все предложения в наборе данных. Для каждого предложения слегка корректируем оценки, чтобы классификатор улучшил прогноз для него. Как можно изменить результаты? Мы сделаем это с помощью *метода персептрона*, который изучим в разделе «Метод персептрона — способ немного улучшить персептрон». Эквивалентный способ обучения моделей персептрона — использование функции ошибок, как мы делали в главе 3. После чего применим градиентный спуск, чтобы минимизировать функцию.

Однако язык сложен сам по себе — в нем есть нюансы, двусмысленности и сарказм. Не потеряем ли мы слишком много информации, если сведем слова к простым баллам? Ответ — да. Мы действительно потеряем много информации и из-за этого не сможем создать идеальный классификатор. Хорошей же новостью является то, что с помощью этого метода все равно сможем создать классификатор, который в *большинстве* случаев сработает правильно. Вот доказательство того, что метод, который мы применяем, не может работать правильно все время. В предложениях «Мне не грустно, я счастлив» и «Я не счастлив, мне грустно» встречаются одни и те же слова, но значение предложений разное. Следовательно, независимо от того, какие оценки мы дадим конкретным словам, два предложения получат одинаковую оценку и в итоге классификатор вернет для них один и тот же прогноз. У них разные метки, следовательно, классификатор, по-видимому, ошибся с одним из них.

Решением проблемы станет создание классификатора, который учитывает порядок слов или другие элементы, такие как знаки препинания или идиомы. Некоторые модели, такие как *скрытые марковские модели*, *рекуррентные нейронные сети* или *сети долговременной кратковременной памяти (LSTM)*, успешно работают с последовательными данными, но мы не будем включать их в эту книгу. Тем не менее, если вы хотите изучить эти модели, то в приложении В найдете несколько очень полезных ссылок по этой теме.

Весь код для этой главы можно найти в следующем репозитории GitHub: https://github.com/luiguiserrano/manning/tree/master/Chapter_5_Perceptron_Algorithm.

Проблема: мы на чужой планете и не знаем местного языка!

Представьте себе такой сценарий: мы астронавты и только что приземлились на далекой планете, населенной инопланетянами. Нам бы очень хотелось пообщаться с ними, но они говорят на странном языке, которого мы не понимаем. Мы замечаем, что у инопланетян два настроения: радостное и грустное. Первый шаг в общении с ними — выяснить, радостные они или грустные, основываясь на том, что они говорят. Другими словами, мы хотим создать классификатор анализа настроений.

Нам удастся подружиться с четырьмя инопланетянами, и мы начинаем наблюдать за их настроением, изучая, что они говорят. Согласно наблюдениям, двое из них счастливы, а двое грустны. Кроме того, они повторяют одно и то же предложение снова и снова. В их языке, похоже, есть только два слова: «аак» и «бип». Мы формируем следующий набор данных с предложением, которое они произносят, и их настроением.

Набор данных

- Инопланетянин 1:
 - настроение — радостный;
 - предложение: «Аак аак аак!»
- Инопланетянин 2:
 - настроение — грустный;
 - предложение: «Бип бип!»
- Инопланетянин 3:
 - настроение — радостный;
 - предложение: «Аак бип аак!»
- Инопланетянин 4:
 - настроение — грустный;
 - предложение: «Аак бип бип бип!»

Внезапно появляется пятый инопланетянин и говорит: «Аак бип аак аак!» Мы не можем точно определить его настроение. Исходя из того, что мы знаем, как можно спрогнозировать настроение инопланетянина (рис. 5.1)?

Данные	Прогноз
 Аак аак аак!	Инопланетянин радостен или печален?  Аак бип аак аак!
 Бип бип!	
 Аак бип аак!	
 Аак бип бип бип!	

Рис. 5.1. Набор данных об инопланетянах. Пятый инопланетянин произносит другую фразу. Можем ли мы спрогнозировать, счастлив он или печален?

Мы предполагаем, что этот инопланетянин счастлив, потому что, хотя и не знаем языка, слово «аак», кажется, чаще встречается в радостных предложениях, в то время как слово «бип», похоже, чаще присутствует в грустных. Возможно, «аак» означает что-то позитивное, например «радость» или «счастье», тогда как «бип» — что-то грустное, например «отчаяние» или «печаль».

Наблюдение приводит к появлению первого классификатора анализа настроений. Он делает прогноз, подсчитывая количество слов «аак» и «бип». Если «аак» больше, чем «бип», то классификатор прогнозирует, что предложение радостное, а если меньше — грустное. Что будет, если оба слова встречаются одинаковое количество раз? У нас нет оснований для вывода, поэтому предположим, что по умолчанию прогноз таков: предложение радостное. На практике подобные крайние случаи происходят нечасто, поэтому они не создадут большой проблемы.

Классификатор, который мы только что построили, представляет собой персептрон (или линейный классификатор). Мы можем записать его в терминах баллов или весов следующим образом.

Классификатор анализа настроений

Присвоить словам, из которых состоит данное предложение, следующие баллы:

- «аак» — 1 балл;
- «бип» — -1 балл.

Правило. Рассчитать оценку предложения, сложив баллы всех слов в нем следующим образом:

- если оценка положительная или нулевая, выдать прогноз, что предложение радостное;
- если оценка отрицательная, выдать прогноз, что предложение грустное.

В большинстве ситуаций полезно построить график данных, потому что иногда становятся видны интересные закономерности. В табл. 5.1 представлены четыре инопланетянина, количество произнесенных каждым из них слов «аак» и «бип», а также их настроение.

Таблица 5.1. Набор данных об инопланетянах

Предложение	Аак	Бип	Настроение
Аак аак аак!	3	0	Радостный
Бип бип!	0	2	Грустный
Аак бип аак!	2	1	Радостный
Аак бип бип бип!	1	3	Грустный

График состоит из двух осей, горизонтальной X и вертикальной Y . По горизонтальной оси откладываем количество появлений «аак», а по вертикальной — количество появлений «бип» (рис. 5.2).

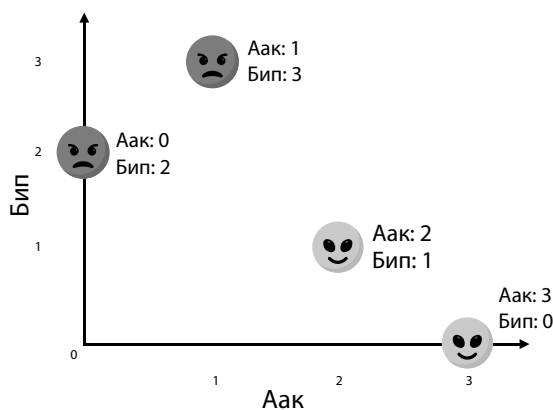


Рис. 5.2. График набора данных об инопланетянах

Обратите внимание на то, что здесь радостные инопланетяне расположены в правом нижнем углу, а грустные — в левом верхнем. Это связано с тем, что в нижней правой области предложения имеют больше «аак», чем «бип», а в верхней левой — наоборот. Фактически прямая, образованная всеми предложениями с одинаковым количеством появлений «аак» и «бип», разделяет эти две области (рис. 5.3). Уравнение этой прямой следующее:

$$\#аак = \#бип.$$

Или эквивалентное:

$$\#аак - \#бип = 0.$$

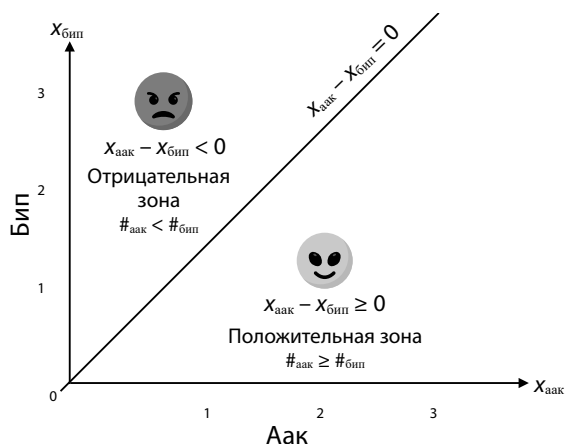


Рис. 5.3. Классификатор представляет собой диагональную прямую, разделяющую «радостные» и «грустные» точки

На протяжении всей этой главы мы будем использовать переменные x с разными индексами, чтобы указать количество появлений слова в предложении. В этом случае слово «аак» появляется $x_{\text{аак}}$ раз, а слово «бип» — $x_{\text{бип}}$ раз.

С помощью этого обозначения уравнение классификатора превращается в $x_{\text{аак}} - x_{\text{бип}} = 0$ или эквивалентное $x_{\text{аак}} = x_{\text{бип}}$. Это уравнение прямой на плоскости. Если для вас оно таковым не выглядит, подумайте об уравнении прямой $y = x$, за исключением того, что вместо x у нас $x_{\text{аак}}$, а вместо y — $x_{\text{бип}}$. Почему бы вместо этого не использовать x и y , как мы делали еще в средней школе? Я бы с радостью, но, к сожалению, у нас понадобится для кое-чего другого (прогноз) позже. Таким образом, представим ось $x_{\text{аак}}$ как горизонтальную, а ось $x_{\text{бип}}$ — как вертикальную. Вместе с этим уравнением мы получаем две важные области, которые называем *положительной зоной* и *отрицательной зоной*. Они определяются так.

- **Положительная зона** — область на плоскости, для которой $x_{\text{аак}} - x_{\text{бип}} \geq 0$. Это соответствует предложениям, в которых слово «аак» встречается по крайней мере столько же раз, сколько и «бип».
- **Отрицательная зона** — область на плоскости, для которой $x_{\text{аак}} - x_{\text{бип}} < 0$. Это соответствует предложениям, в которых слово «аак» встречается реже, чем «бип».

Созданный нами классификатор предсказывает, что каждое предложение в положительной зоне — радостное, а каждое предложение в отрицательной — грустное. Поэтому наша цель — отыскать классификатор, который способен поместить как можно больше радостных предложений в положительную область и как можно больше грустных — в отрицательную. В нашем небольшом примере классификатор выполняет эту работу идеально. Так происходит не всегда, но алгоритм персептрона поможет найти классификатор, который действительно хорошо работает.

На рис. 5.3 мы видим прямую, соответствующую классификатору, а также положительную и отрицательную зоны. Если вы сравните рис. 5.2 и 5.3, то увидите, что текущий классификатор хорош, потому что все радостные предложения расположены в положительной зоне, а все грустные — в отрицательной.

Теперь, когда мы создали простой персептронный классификатор для анализа настроений, рассмотрим чуть более сложный пример.

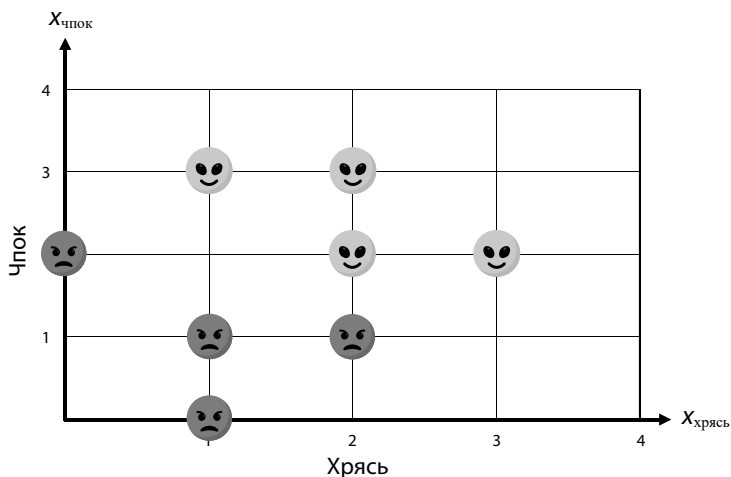
Немного более сложная планета

В этом разделе проанализируем более сложный пример, который вводит новый аспект восприятия — смещение. После того как мы научились общаться с обитателями первой планеты, нас отправляют с миссией на вторую планету, где у населения несколько более сложный язык. Цель остается прежней: создать классификатор анализа настроений в нем. В языке новой планеты есть два слова: «хрясь» и «чпок». Набор данных приведен в табл. 5.2. И снова мы записали каждое предложение, количество появлений каждого слова в нем и настроение инопланетянина.

Таблица 5.2. Новый набор данных — чужих слов

Предложение	Хрясь	Чпок	Настроение
Хрясь!	1	0	Грустный
Чпок чпок!	0	2	Грустный
Хрясь чпок!	1	1	Грустный
Хрясь чпок хрясь!	2	1	Грустный
Чпок хрясь чпок чпок!	1	3	Радостный
Хрясь чпок чпок хрясь!	2	2	Радостный
Чпок чпок хрясь хрясь хрясь!	3	2	Радостный
Хрясь чпок чпок хрясь чпок!	2	3	Радостный

Построение классификатора для этого набора данных немного сложнее, чем для предыдущего. Прежде всего должны ли мы присваивать положительные или отрицательные оценки словам «хрясь» и «чпок»? Давайте возьмем ручку и бумагу и попробуем придумать классификатор, который сможет правильно разделить радостные и грустные предложения в этом наборе данных. Не будет лишним взглянуть на график этого набора данных на рис. 5.4. Обратите внимание на то, что радостные, как правило, расположены вверху справа, а грустные — внизу слева.

**Рис. 5.4.** График нового набора инопланетных данных

Принцип работы данного классификатора состоит в том, чтобы подсчитать количество слов в предложении. Обратите внимание на то, что все предложения с одним, двумя или тремя словами грустные, а с четырьмя и пятью

словами — радостные. Это и есть классификатор! Он классифицирует предложения, состоящие из трех слов или менее, как грустные, а предложения, состоящие из четырех слов или более, — как радостные. Мы можем снова записать это более математическим способом.

Классификатор анализа настроений

В данном предложении присвоить словам следующие баллы:

- хрясь — 1 балл;
- чпок — 1 балл.

Правило. Рассчитать оценку предложения, сложив оценки всех его слов:

- если оценка составляет 4 балла или больше, выдать прогноз, что предложение радостное;
- если оценка равна 3 балла или меньше, выдать прогноз, что предложение грустное.

Чтобы упростить задачу, немного изменим правило, используя ограничение 3,5 балла.

Правило. Рассчитать оценку предложения, сложив оценки всех его слов:

- если оценка 3,5 балла или больше, выдать прогноз, что предложение радостное;
- если оценка меньше 3,5 балла, выдать прогноз, что предложение грустное.

Этот классификатор опять же соответствует прямой, показанной на рис. 5.5.

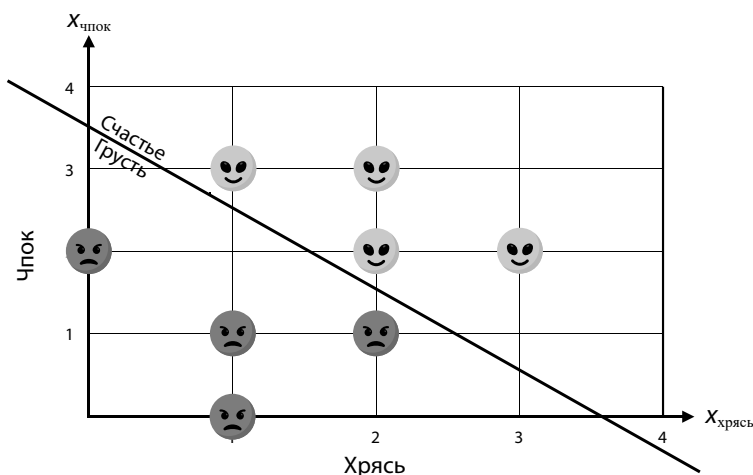


Рис. 5.5. Классификатор для нового набора инопланетных данных. Это снова прямая, которая разделяет радостных и грустных инопланетян

В предыдущем примере мы пришли к выводу, что слово «аак» было радостным, а «бип» — грустным. Что мы видим в этом примере? Кажется, что оба слова, «хрясь» и «чпок», радостные, потому что их оценки положительные. Почему же тогда предложение «Хрясь чпок» грустное? В нем не хватает слов. Обитатели этой планеты обладают индивидуальностью. Те, кто говорит мало, грустны, а те, кто говорит много, счастливы. Мы можем интерпретировать это так, что жители планеты по своей природе грустны, но могут избавиться от грусти, много разговаривая.

Другой важный элемент этого классификатора — пороговое значение 3,5 балла. Этот порог задействуется классификатором для прогнозирования, поскольку предложения с оценками выше порогового значения или равными ему классифицируются как радостные, а предложения с оценками ниже — как грустные. Однако пороговые значения не используют, вместо них мы обратимся к понятию *смещения*. Смещение — это отрицательное значение порога, мы добавляем его к баллам. Таким образом, классификатор может вычислить оценку и вернуть прогноз «радостное», если оценка неотрицательна, или «грустное», если отрицательна. В качестве последнего изменения в обозначениях назовем оценки слов весами. Наш классификатор может быть организован следующим образом.

Классификатор анализа настроений

Учитывая предложение, присвоить словам следующие веса и смещения.

Веса:

- хрясь — 1 балл;
- чпок — 1 балл.

Смещение: −3,5 балла.

Правило. Рассчитать оценку предложения, сложив веса всех его слов и смещение:

- если оценка больше или равна нулю, выдать прогноз, что предложение радостное;
- если оценка меньше нуля, выдать прогноз, что предложение грустное.

Уравнение оценки классификатора, а также прямой, изображенной на рис. 5.5, выглядит так:

$$\#хрясь + \#чпок - 3,5 = 0.$$

Обратите внимание на то, что определение персептрона с порогом 3,5 и со смещением −3,5 — это одно и то же, потому что следующие два уравнения эквивалентны:

$$\#хрясь + \#чпок \geq 3,5;$$

$$\#хрясь + \#чпок - 3,5 \geq 0.$$

Мы можем использовать обозначения, аналогичные приведенным в предыдущем разделе, где $x_{\text{хрясь}}$ — это количество появлений слова «хрясь», а $x_{\text{чпок}}$ — количество появлений слова «чпок». Таким образом, уравнение прямой, изображенной на рис. 3.5, можно записать в виде

$$x_{\text{хрясь}} + x_{\text{чпок}} - 3,5 = 0.$$

Эта прямая также делит плоскость на положительную и отрицательную зоны, определяемые следующим образом:

- **положительная зона** — область на плоскости, для которой $x_{\text{хрясь}} + x_{\text{чпок}} - 3,5 \geq 0$;
- **отрицательная зона** — область на плоскости, для которой $x_{\text{хрясь}} + x_{\text{чпок}} - 3,5 < 0$.

Обязан ли классификатор всегда работать правильно? Нет

В двух предыдущих примерах мы построили классификатор, который всегда выдавал правильный результат. Другими словами, он классифицировал два радостных предложения как радостные, а два грустных — как грустные. Это не так уж часто встречается на практике, особенно в наборах данных с большим количеством точек. Однако цель классификатора состоит в том, чтобы разделить точки как можно лучше. На рис. 5.6 мы видим набор данных с 17 точками (восемь радостных и девять грустных), которые невозможно идеально разделить на две части с помощью одной прямой. Тем не менее прямая делит их вполне сносно, неправильно классифицируя только три точки — две в радостной зоне и одну в грустной.

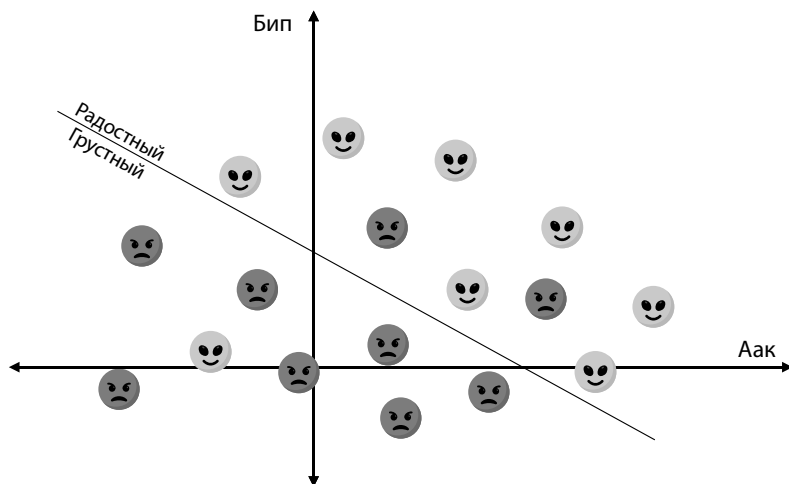


Рис. 5.6. Прямая хорошо разделяет набор данных

Более общий классификатор и несколько иной способ определения прямых

В этом разделе мы получим более общее представление о персептронном классификаторе. Ненадолго назовем наши слова 1 и 2, а переменные, отслеживающие их появление, — x_1 и x_2 . Уравнения двух предыдущих классификаторов будут выглядеть так:

$$x_1 - x_2 = 0;$$

$$x_1 + x_2 - 3,5 = 0.$$

Общая форма уравнения персептрона — $ax_1 + bx_2 + c = 0$, где a — оценка слова 1, b — оценка слова 2, а c — смещение. Это уравнение соответствует прямой, которая делит плоскость на две зоны следующим образом:

- **положительная зона** — зона на плоскости, для которой $ax_1 + bx_2 + c \geq 0$;
- **отрицательная зона** — зона на плоскости, для которой $ax_1 + bx_2 + c < 0$.

Например, если слово 1 имеет оценку 4, слово 2 — оценку $-2,5$, а смещение равно $1,8$, то уравнение этого классификатора будет иметь вид

$$4x_1 - 2,5x_2 + 1,8 = 0,$$

а положительная и отрицательная зоны — те, где $4x_1 - 2,5x_2 + 1,8 \geq 0$ и $4x_1 - 2,5x_2 + 1,8 < 0$ соответственно.

ОТВЛЕЧЕМСЯ. УРАВНЕНИЯ ПРЯМОЙ И ЗОН НА ПЛОСКОСТИ В главе 3 мы определили прямые, используя уравнение $y = mx + b$ на плоскости, где оси обозначены x и y . В этой главе мы определяем их с помощью уравнения $ax_1 + bx_2 + c = 0$ на плоскости, где оси — x_1 и x_2 . Чем же они различаются? Они оба — вполне допустимые способы задания прямой. Однако, в то время как первое уравнение полезно для моделей линейной регрессии, второе полезно для моделей персептрона (и в целом для других алгоритмов классификации, таких как логистическая регрессия, нейронные сети и машины опорных векторов, которые мы встретим в главах 6, 10 и 11 соответственно). Почему же это уравнение лучше подходит для моделей персептрона? Отсюда вытекают некоторые преимущества.

- Уравнение $ax_1 + bx_2 + c = 0$ не только определяет прямую, но и четко выделяет две зоны, положительную и отрицательную. Если бы мы хотели иметь ту же прямую, за исключением того, что положительные и отрицательные области поменялись бы местами, то рассмотрели бы уравнение $-ax_1 - bx_2 - c = 0$ (рис. 5.7).
- Используя уравнение $ax_1 + bx_2 + c = 0$, мы можем рисовать вертикальные прямые, потому что уравнение вертикальной прямой — $x = c$ или $1x_1 + 0x_2 - c = 0$. Вертикальные прямые нечасто появляются в моделях линейной регрессии, но встречаются в моделях классификации.

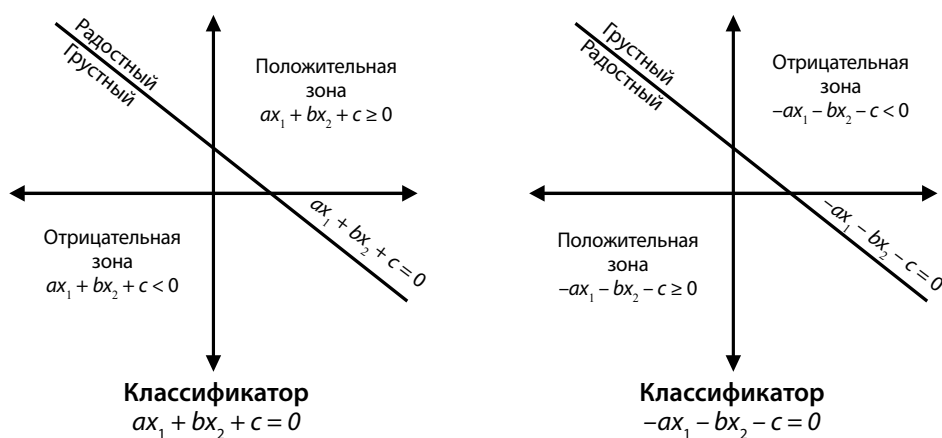


Рис. 5.7. Классификатор определяется прямой $ax_1 + bx_2 + c = 0$, положительной и отрицательной зоной

Ступенчатая функция и функции активации. Сжатый способ получения прогнозов

В этом разделе мы изучим математический способ получения прогнозов. Однако, прежде чем перейти к нему, нужно превратить все наши данные в цифры. Обратите внимание на метки в наборе данных — «радостный» и «грустный». Мы обозначим их 1 и 0 соответственно.

Оба перцептронных классификатора, которые мы построили в этой главе, были определены с использованием `if`. Так, классификатор прогнозировал «радостный» или «грустный» на основе общего балла предложения: если балл положительный или нулевой, классификатор выдает «радостный», а если отрицательный, то «грустный». Есть более прямой способ превратить оценку в прогноз — с помощью *ступенчатой функции*.

СТУПЕНЧАТАЯ ФУНКЦИЯ — функция, которая возвращает 1, если выходные данные неотрицательны, и 0, если отрицательны. Другими словами, если входные данные равны x , то:

- $\text{step}(x) = 1$, если $x \geq 0$;
- $\text{step}(x) = 0$, если $x < 0$.

На рис. 5.8 показан график ступенчатой функции.

С помощью ступенчатой функции мы можем легко выразить выходные данные перцептронного классификатора. В нашем наборе данных используем переменную y для ссылки на метки точно так же, как делали в главе 3. Прогноз, который модель делает для метки, обозначается $\hat{y}$. Выходные данные модели перцептрона записываются в сжатом виде как

$$\hat{y} = \text{step}(ax_1 + bx_2 + c).$$

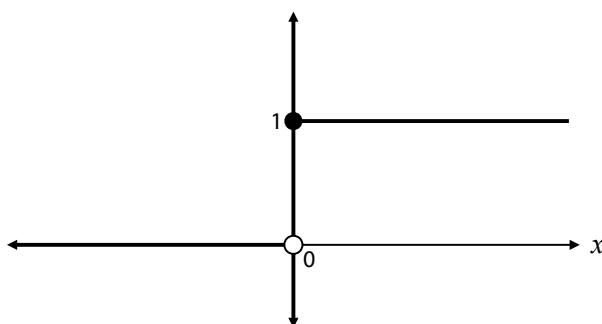


Рис. 5.8. Ступенчатая функция

Ступенчатая функция — это частный случай *функции активации*. Функция активации — важная концепция в машинном обучении, особенно в глубоком обучении, снова попадется нам на глаза она в главах 6 и 10. Формальное определение функции активации будет дано позже, поскольку вся ее мощь раскрывается при построении нейронных сетей. Пока же мы будем думать о ней как о функции, которую можно использовать для преобразования результатов в прогноз.

Что будет, если в языке больше двух слов? Общее определение персептронного классификатора

В двух примерах с инопланетянами в начале этого раздела мы построили персептронные классификаторы для языков с двумя словами. Но можно создавать классификаторы с любым требуемым количеством слов. Например, если бы у нас был язык с тремя словами, скажем, «аак», «бип» и «хрясь», классификатор делал бы предсказания в соответствии со следующей формулой:

$$\hat{y} = \text{step}(ax_{\text{аак}} + bx_{\text{бип}} + cx_{\text{хрясь}} + d),$$

где a , b и c — веса слов «аак», «бип» и «хрясь» соответственно, d — смещение. Как мы уже поняли, персептронные классификаторы анализа настроений для языков с двумя словами могут быть выражены в виде прямой на плоскости, разделяющей радостные и грустные точки. Классификаторы анализа настроений для языков с тремя словами также могут быть представлены геометрически. Точки можно изобразить как находящиеся в трехмерном пространстве. В этом случае каждая из осей соответствует одному из слов «аак», «бип» и «хрясь», а предложение — точке в пространстве, для которой три ее координаты — это количество появлений слов. Рис. 5.9 иллюстрирует пример, в котором предложение, содержащее пять «аак», восемь «бип» и три «хрясь», соответствует точке с координатами (5, 8, 3).

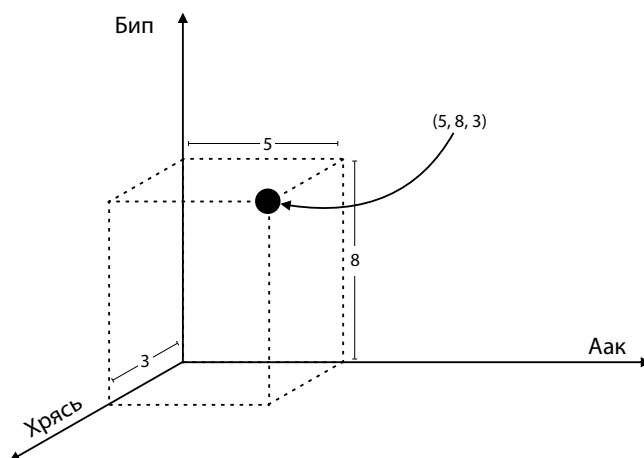


Рис. 5.9. Предложение из трех слов в виде точки в пространстве

Способ разделить эти точки — использовать плоскость. Плоскость в точности соответствует уравнению $ax_{\text{аак}} + bx_{\text{бип}} + cx_{\text{хрясь}} + d$ и показана на рис. 5.10.

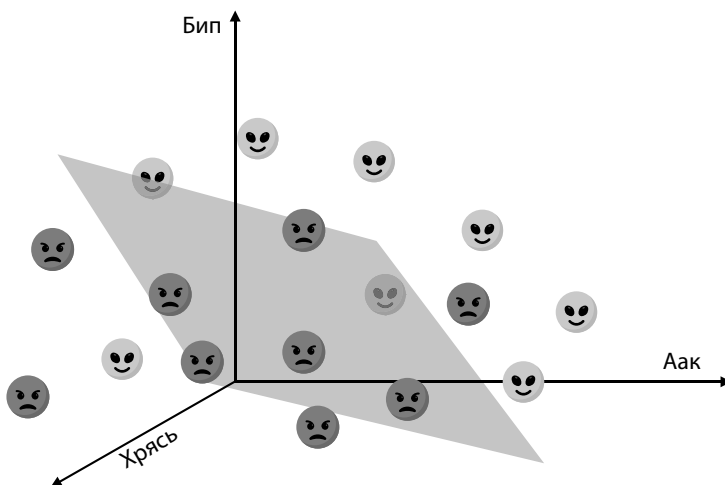


Рис. 5.10. Набор данных предложений из трех слов нанесен на график в трех измерениях. Классификатор представлен плоскостью, которая разбивает пространство на две области

Так мы можем создать персептронные классификаторы анализа настроений для языков с любым количеством слов. Допустим, в языке n слов, которые обозначим $1, 2 \dots n$. Набор данных состоит из m предложений: $x^{(1)}, x^{(2)} \dots x^{(m)}$. Каждое

предложение $x^{(1)}$ сопровождается меткой y_i , которая равна 1, если предложение радостное, и 0, если оно грустное. Способ, которым мы записываем каждое предложение, заключается в использовании количества появлений каждого из n слов. Следовательно, каждое предложение соответствует строке в наборе данных и может рассматриваться как вектор или n -кортеж чисел $x^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)})$, где $x_j^{(i)}$ — количество появлений слова j в i -м предложении.

Персептрон состоит из n весов (баллов), по одному для каждого из n слов в языке, и смещения. Веса обозначаются w_i , а смещение — b . Таким образом, прогноз, который классификатор делает для предложения $x^{(i)}$, — это

$$\hat{y}_i = \text{step}(w_1 x_1^{(i)} + w_2 x_2^{(i)} + \dots + w_n x_n^{(i)} + b).$$

Точно так же, как классификаторы с двумя словами могут быть представлены геометрически в виде прямой, которая разрезает плоскость на две области, а классификаторы с тремя словами — в виде плоскости, которая разрезает трехмерное пространство на две области, классификаторы с n словами также могут быть представлены геометрически. К сожалению, потребуется n измерений, чтобы это увидеть. Люди могут видеть только три измерения, поэтому нам, возможно, придется представить $(n - 1)$ -мерную плоскость (называемую *гиперплоскостью*), разрезающую n -мерное пространство на две области.

Однако тот факт, что мы не в силах представить их себе геометрически, не означает, что мы не можем получить хорошее представление о том, как они работают. Представьте, что наш классификатор построен на основе английского языка. Каждому слову присваивается определенный вес. Это равносильно просмотру словаря и присвоению каждому слову балла радости. Результат может выглядеть примерно так.

Веса (баллы):

- A — 0,1 балла;
- Aardvark — 0,2 балла;
- Aargh — -4 балла;
- ...
- Joy — 10 баллов;
- ...
- Suffering — -8,5 баллов;
- ...
- Zygote — 0,4 балла.

Смещение -2,3 балла.

Если это веса и смещения классификатора для прогноза о том, радостное предложение или грустное, то мы складываем оценки всех слов вместе с повторениями. Если результат больше или равен 2,3 (отрицательное смещение), предложение прогнозируется как радостное, в противном случае — как грустное.

Более того, подобная нотация работает для любого примера, а не только для анализа настроений. Если у нас есть другая задача с другими точками данных,

признаками и метками, мы можем закодировать ее, используя все те же переменные. Например, если у нас есть медицинское приложение, в котором мы пытаемся предсказать, болен пациент или здоров, применяя n весов и смещение, то все равно можем называть метки y , признаки x_i , веса w_i и смещение b .

Смещение, у-перехват и настроение инопланетянина-молчуна

Пока что мы хорошо представляем, что означают веса классификатора. Слова с положительным весом радостные, а с отрицательным весом — грустные. Слова с очень малым весом (положительным или отрицательным) — более нейтральные. Но что означает смещение?

В главе 3 мы уточняли, что смещением в регрессионной модели цен на жилье является базовая цена дома. Другими словами, это прогнозируемая цена гипотетического дома с нулем комнат (студия?). В модели персептрона смещение может быть интерпретировано как оценка пустого предложения. Другими словами, если инопланетянин абсолютно ничего не говорит, он радостен или печален? Если в предложении нет слов, то его оценка — это и есть смещение. Таким образом, если смещение положительное, то инопланетянин, который ничего не говорит, радостен, а если отрицательное — печален.

Геометрически разница между положительным и отрицательным смещением заключается в положении начала координат — точки с координатами $(0, 0)$ — относительно классификатора. Причина в том, что точка с координатами $(0, 0)$ соответствует предложению без слов. В классификаторах с положительным смещением начало координат находится в положительной зоне, тогда как в классификаторах с отрицательным смещением — в отрицательной (рис. 5.11).

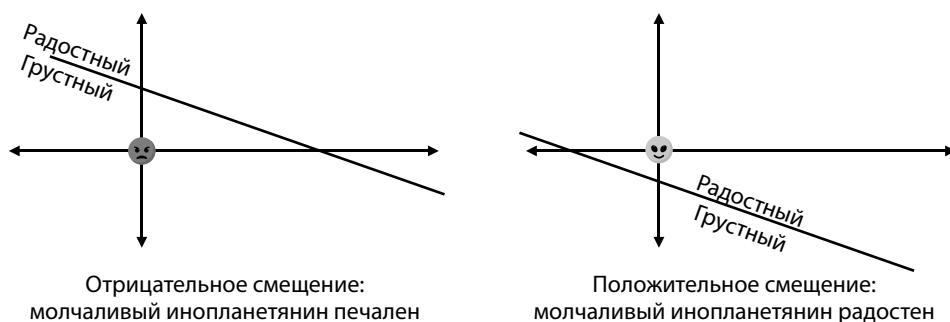


Рис. 5.11. Классификатор: *слева* — имеет отрицательное смещение, или положительный порог (у-перехват); *справа* — имеет положительное смещение, или отрицательный порог

В каких наборах данных для анализа настроений смещение может быть положительным или отрицательным? Вот пара примеров.

Пример 1 (положительное смещение). Набор данных онлайн-обзоров продукта. Представьте себе набор данных, в который мы записываем все отзывы о конкретном продукте на Amazon. Некоторые из них положительные, а некоторые отрицательные — это зависит от количества поставленных звезд. Как вы думаете, какой будет оценка пустого отзыва? По моему опыту, плохие отзывы, как правило, многословны, потому что клиент расстроен и описывает свой негативный опыт. А многие положительные отзывы пустые — клиент просто ставит хорошую оценку без объяснений, почему ему понравился продукт. Следовательно, такой классификатор, вероятно, имеет положительное смещение.

Пример 2 (отрицательное смещение). Набор данных разговоров с друзьями. Представьте, что мы записываем все разговоры с друзьями и классифицируем их как радостные или грустные. Если однажды мы встречаемся с другом, а он как воды в рот набрал, мы думаем, что он злится на нас или очень расстроен. Поэтому пустое предложение классифицируется как грустное. Это означает, что данный классификатор, по-видимому, имеет отрицательное смещение.

Как определить, хорошо ли работает классификатор? Функция ошибки

Теперь, когда мы определили, что такое перцептронный классификатор, попробуем понять, как его обучить. Другими словами, как найти такой классификатор, который наилучшим образом соответствует нашим данным. Но прежде чем приступить к обучению перцептронов, нужно усвоить важную концепцию, как именно их оценивать. Забегая вперед, в этом разделе мы изучаем полезную функцию ошибок, которая сообщит, хорошо ли перцептронный классификатор соответствует нашим данным. Точно так же, как в главе 3 в случае линейной регрессии работали абсолютные и квадратические погрешности, новая функция ошибок будет большой для классификаторов, которые плохо соответствуют данным, и маленькой — для соответствующих хорошо.

Как сравнить классификаторы? Функция ошибки

В этом разделе мы узнаем, как построить эффективную функцию ошибок, которая поможет определить, насколько хорош конкретный перцептрон. Прежде всего проверим свою интуицию. На рис. 5.12 показаны два разных перцептронных классификатора на одном и том же наборе данных. Классификаторы представлены в виде прямой с двумя четко определенными сторонами, «радостной» и «грустной». Очевидно, слева — плохой классификатор, а справа — хороший. Можно ли определить, насколько они хороши? Другими словами, можно ли присвоить каждому из них число таким образом, чтобы у того, что слева, оно было большим, а у правого — маленьким?

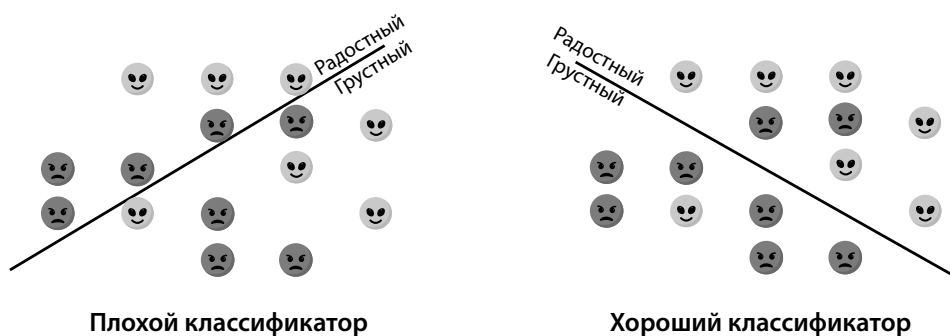


Рис. 5.12. Классификатор *слева* — плохой, он не очень хорошо разделяет точки, *справа* — хороший

Далее мы увидим различные ответы на этот вопрос, у каждого имеются плюсы и минусы. Один из них (спойлер: третий) мы используем для обучения перцептронов.

Функция ошибок 1: количество ошибок. Самый простой способ оценить классификатор — подсчитать количество ошибок, которые он допускает, другими словами, подсчитать количество точек, которые он классифицирует неправильно.

В нашем случае классификатор *слева* имеет ошибку 8, потому что ошибочно прогнозирует четыре «радостные» точки как «грустные», а четыре «грустные» — как «радостные». Хороший классификатор имеет ошибку 3, так как ошибочно прогнозирует одну «радостную» точку как «грустную», а две «грустные» — как «радостные». Все это показано на рис. 5.13.

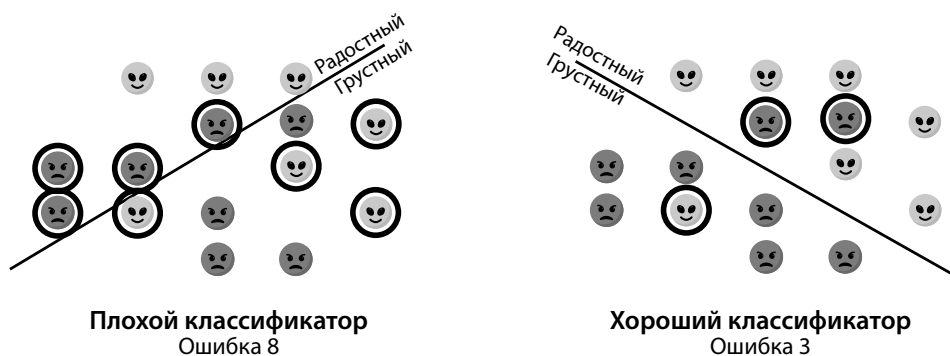


Рис. 5.13. Классификатор *слева* неправильно классифицирует восемь точек, а классификатор *справа* — три точки. Последний лучше подходит для нашего набора данных

Это неплохая функция ошибок, но хорошей ее назвать нельзя. Почему? Она сообщает нам, когда произошла ошибка, но не позволяет понять, насколько та серьезна. Например, если предложение грустное, а классификатор дал ему оценку 1, то он ошибся. Однако если другой классификатор дал ему оценку 100, то допустил гораздо более серьезную ошибку. Это можно увидеть на рис. 5.14. На нем оба классификатора неправильно определили грустную точку, спрогнозировав, что она радостная. Однако классификатор слева расположил прямую близко к точке, что означает, что грустная точка находится не слишком далеко от грустной зоны. Классификатор справа, напротив, разместил точку очень далеко от грустной зоны. В идеале нам хотелось бы, чтобы функция ошибок присваивала классификатору справа более высокую ошибку, чем классификатору слева.

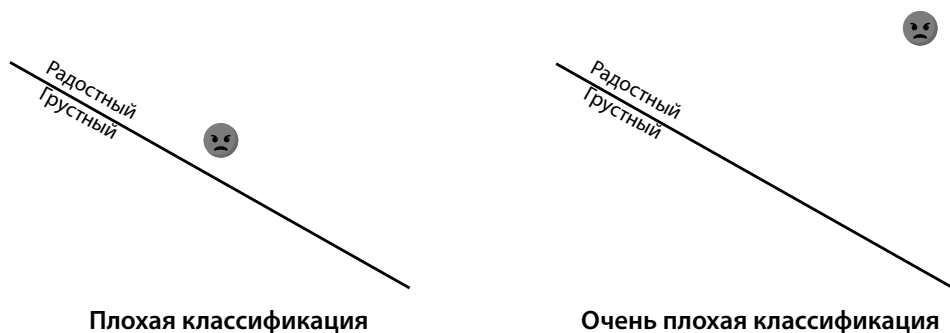


Рис. 5.14. Неправильная классификация

Но зачем вообще знать, насколько серьезны ошибки? Разве недостаточно просто их сосчитать? Вспомните, что мы проделали в главе 3 с алгоритмом линейной регрессии. А точнее, в разделе «Градиентный спуск: как уменьшить функцию ошибок...», где использовали градиентный спуск, чтобы уменьшить ошибку. Этот способ заключается в том, чтобы двигаться небольшими шагами, пока не достигнем точки, когда ошибка окажется незначительной. В алгоритме линейной регрессии мы немного меняли прямую и выбирали направление, в котором ошибка уменьшалась сильнее всего. Если ошибка вычисляется подсчетом количества неправильно классифицированных точек, то она будет принимать только целочисленные значения. Если мы немного покрутим прямую, ошибка может вообще не уменьшиться и мы не узнаем, в каком направлении двигаться. Цель градиентного спуска состоит в том, чтобы минимизировать функцию, делая небольшие шаги в направлении, в котором функция уменьшается сильнее всего (рис. 5.15, *слева*). Однако, чтобы мы могли это сделать, функция ошибки не должна быть плоской. Если функция принимает только целочисленные значения, это эквивалентно попытке спуститься с ацтекской лестницы. Когда мы на плоском уступе, то не знаем, куда двигаться дальше, потому что функция не уменьшается ни в одном направлении (рис. 5.15, *справа*). Хорошая функция ошибки похожа на ту, что слева, где можно легко увидеть нужное направление шага, который немного уменьшит функцию ошибки.

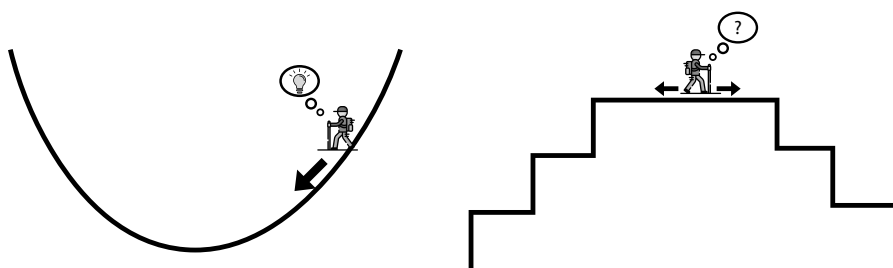


Рис. 5.15. Выполнение градиентного спуска для минимизации функции ошибок похоже на спуск с горы маленькими шагами

Нам нужна функция, которая измеряет величину ошибки и присваивает более высокую ошибку неправильно классифицированным точкам, которые находятся далеко от границы, а меньшую — находящимся близко от нее.

Функция ошибки 2: расстояние. Один из способов различить два классификатора на рис. 5.16 — это рассмотреть длину перпендикуляра от точки до прямой. Обратите внимание на то, что для классификатора слева это расстояние невелико, тогда как для классификатора справа — напротив.

Такая функция ошибки гораздо более эффективна. Вот что она делает.

- Точки, которые правильно классифицированы, дают ошибку 0.
- Неправильно классифицированные точки дают ошибку, равную расстоянию от этой точки до прямой.

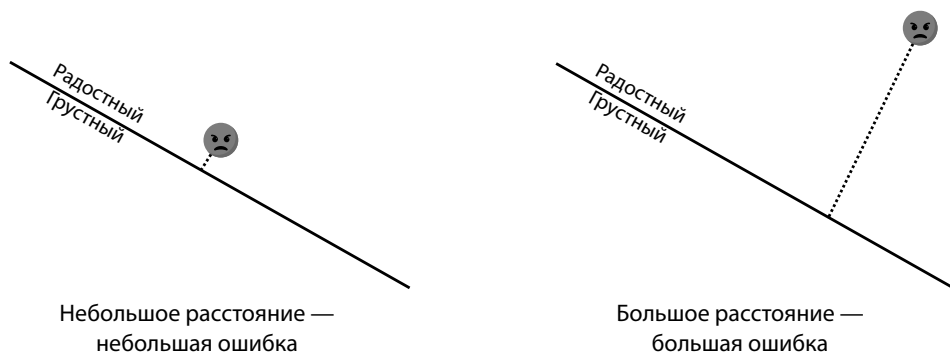


Рис. 5.16. Эффективный способ понять, насколько неправильно классификатор классифицирует точку, — измерить длину перпендикуляра от точки до прямой

Вернемся к двум классификаторам из начала раздела. Способ вычисления общей ошибки заключается в суммировании ошибок, соответствующих всем точкам данных (рис. 5.17). Это означает, что мы принимаем во внимание только неправильно классифицированные точки и складываем длины

перпендикуляров от них до прямой. Обратите внимание на то, что плохой классификатор (*слева*) имеет большую ошибку, а хороший (*справа*) — небольшую.

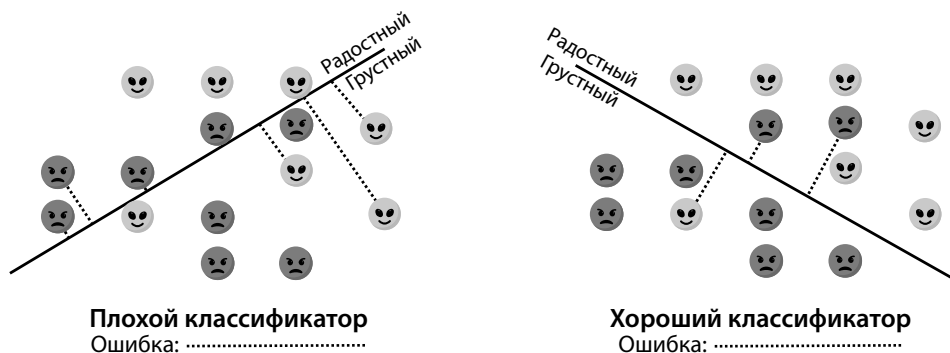


Рис. 5.17. Вычисление общей ошибки классификатора

Это *почти* та функция ошибки, которую мы и будем применять. Так почему бы ею не воспользоваться? Потому что расстояние от точки до прямой вычисляется по сложной формуле. Она содержит квадратный корень, так как вычисляется с помощью теоремы Пифагора. Квадратные корни имеют сложные производные, что усложняет момент, когда мы применяем алгоритм градиентного спуска. Не стоит недооценивать такое усложнение, потому что вместо него мы можем построить функцию ошибки, которую гораздо легче вычислять. Тем не менее ей все же удастся уловить суть функции ошибки: вернуть ошибку для неправильно классифицированных точек и изменить величину в зависимости от того, как далеко неправильно классифицированная точка отстоит от границы.

Функция ошибки 3: оценка. В этом разделе мы увидим, как построить стандартную *функцию ошибок перцептрона*. Во-первых, сведем вместе правила, которые хотим реализовать в функции ошибки, следующим образом.

- Функция ошибки правильно классифицированной точки равна 0.
- Функция ошибки неправильно классифицированной точки — положительное число:
 - для неправильно классифицированных точек, близких к границе, функция ошибки невелика;
 - для неправильно классифицированных точек, удаленных от границы, функция ошибки велика.
- Все это задается простой формулой.

Напомним, что классификатор прогнозирует метку 1 для точек в положительной зоне и метку 0 — для точек в отрицательной. Следовательно, неправильно классифицированная точка — это либо точка с меткой 0 в положительной зоне, либо с меткой 1 — в отрицательной.

Чтобы построить функцию ошибки персептрона, используем оценку, в частности следующие ее свойства.

1. Точки на границе имеют оценку 0.
2. Точки в положительной зоне имеют положительные оценки.
3. Точки в отрицательной зоне имеют отрицательные оценки.
4. Точки, близкие к границе, имеют невысокие оценки (то есть положительные или отрицательные оценки с малым абсолютным значением).
5. Точки, удаленные от границы, имеют высокие оценки (то есть положительные или отрицательные оценки с высоким абсолютным значением).

Неправильно классифицированной точке ошибка персептрона присвоит значение, пропорциональное расстоянию от нее до границы. Следовательно, ошибка для неправильно классифицированных точек, которые находятся далеко от границы, окажется высокой, а для находящихся близко к ней — низкой. Рассматривая свойства 4 и 5, мы видим, что абсолютное значение оценки всегда высокое для точек, удаленных от границы, и низкое — для близких к ней. Таким образом, определяем ошибку как сумму абсолютных значений оценки для неправильно классифицированных точек.

Рассмотрим классификатор, который присваивает веса a и b словам «аак» и «бип», а также имеет смещение c . Он делает прогноз $\hat{y} = \text{step}(ax_{\text{аак}} + bx_{\text{бип}} + c)$ для предложения с $x_{\text{аак}}$ появлениями слова «аак» и $x_{\text{бип}}$ появлениями слова «бип».

Ошибка для персептрона определяется следующим образом.

- Если предложение классифицировано правильно, ошибка равна 0.
- Если предложение классифицировано неправильно, ошибка равна $|x_{\text{аак}} + bx_{\text{бип}} + c|$.

Вот как выглядит определение **ошибки персептрона для общего сценария**, где нотация такая же как в разделе «Что будет, если в языке больше двух слов?..».

- Если точка классифицирована правильно, ошибка равна 0.
- Если точка классифицирована неправильно, ошибка равна $|w_1x_1 + w_2x_2 + \dots + w_nx_n + b|$.

Средняя ошибка персептрона. Способ вычисления ошибки для всего набора данных

Чтобы вычислить ошибку персептрона для всего набора данных, берем среднее значение всех ошибок, соответствующих всем точкам. Мы также можем использовать сумму, хотя в данной главе выбираем среднее значение и называем его *средней ошибкой персептрона*.

Чтобы проиллюстрировать среднюю ошибку персептрона, приведем пример. Рассмотрим набор данных, состоящий из четырех предложений, два из которых помечены как радостные, а два — как грустные (табл. 5.3).

Таблица 5.3. Новый набор данных об инопланетянах

Предложение	Аак	Бип	Метка (настроение)
Аак	1	0	Грустный
Бип	0	1	Радостный
Аак бип бип бип	1	3	Радостный
Аак бип бип аак аак	3	2	Грустный

Сравним следующие два классификатора в этом наборе данных.

Классификатор 1

Веса:

- аак — $a = 1$;
- бип — $b = 2$.

Смещение $c = -4$.

Оценка предложения $1x_{\text{аак}} + 2x_{\text{бип}} - 4$.

Прогноз: $\hat{y} = \text{step}(1x_{\text{аак}} + 2x_{\text{бип}} - 4)$.

Классификатор 2

Веса:

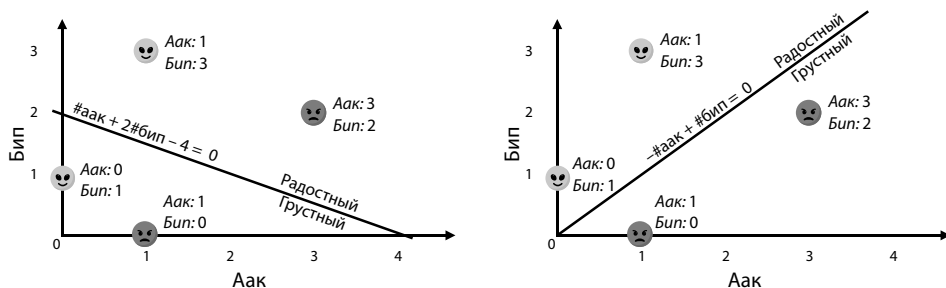
- аак — $a = -1$;
- бип — $b = 1$.

Смещение $c = 0$

Оценка предложения $-x_{\text{аак}} + x_{\text{бип}}$.

Прогноз: $\hat{y} = \text{step}(-x_{\text{аак}} + x_{\text{бип}})$.

Точки и классификаторы можно увидеть на рис. 5.18. На ваш взгляд, какой из них лучше? Похоже, что классификатор 2 получше, потому что он верно определяет каждую точку, в то время как классификатор 1 допускает две ошибки. Теперь подсчитаем ошибки и убедимся, что ошибка классификатора 1 больше, чем классификатора 2.

**Рис. 5.18.** Классификаторы: слева — 1; справа — 2

Прогнозы для обоих классификаторов рассчитаны в табл. 5.4.

Таблица 5.4. Набор данных из четырех предложений с метками

Положение ($x_{\text{аак}}, x_{\text{бип}}$)	Метка y	Классификатор 1			Классификатор 2		
		Оценка $1x_{\text{аак}} + 2x_{\text{бип}} - 4$	Прогноз $\text{step}(1x_{\text{аак}} + 2x_{\text{бип}} - 4)$	Ошибка	Оценка $-x_{\text{аак}} + 2x_{\text{бип}}$	Прогноз $\text{step}(-x_{\text{аак}} + 2x_{\text{бип}})$	Ошибка
(1, 0)	Грустный (0)	-3	0 (правильно)	0	-1	0 (правильно)	0
(0, 1)	Радостный (1)	-2	0 (неправильно)	2	1	1 (правильно)	0
(1, 3)	Радостный (1)	3	1 (правильно)	3	2	1 (правильно)	0
(3, 2)	Грустный (0)	3	1 (неправильно)	0	-1	0 (правильно)	0
Средняя ошибка персептрона				1,25	0		

Теперь перейдем к вычислению ошибок. Обратите внимание на то, что классификатор 1 неправильно определил только предложения 2 и 4. Предложение 2 радостное, но ошибочно классифицировано как грустное, а предложение 4 грустное, но ошибочно классифицировано как радостное. Ошибка предложения 2 — это абсолютное значение оценки, или $|-2| = 2$. Ошибка предложения 4 — это абсолютное значение оценки, или $|3| = 3$. В двух других предложениях ошибка равна 0, поскольку они классифицированы правильно. Таким образом, средняя ошибка персептрона классификатора 1 составляет:

$$\frac{1}{4}(0 + 2 + 0 + 3) = 1,25.$$

Классификатор 2 не допускает ошибок — он правильно определяет все точки. Следовательно, средняя ошибка персептрона классификатора 2 равна 0. Таким образом, мы приходим к выводу, что классификатор 2 лучше, чем классификатор 1. Кратко эти расчеты изложены в табл. 5.4 и на рис. 5.19.

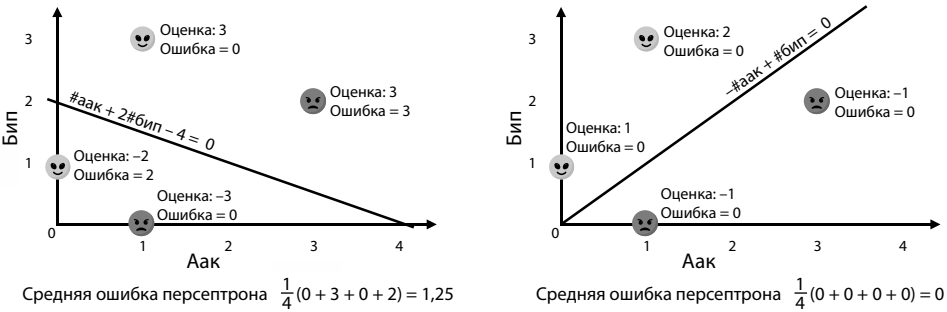


Рис. 5.19. Классификатор 2 лучше, чем классификатор 1

Теперь, когда мы знаем, как сравнивать классификаторы, перейдем к поиску наилучшего или по крайней мере довольно хорошего.

Как найти хороший классификатор? Алгоритм персептрона

Чтобы построить хороший персептронный классификатор, используем подход, аналогичный тому, что мы задействовали с линейной регрессией в главе 3. Этот процесс называется *алгоритмом персептрона* и состоит в том, чтобы начать со случайного персептронного классификатора и постепенно улучшать его, пока не получится хороший. Основные шаги алгоритма персептрона таковы.

1. Начать со случайного персептронного классификатора.
2. Немного улучшить классификатор (повторить много раз).
3. Измерить ошибку персептрона, чтобы решить, когда прекратить выполнение цикла.

Начнем с разработки шага внутри цикла, метода, используемого для небольшого улучшения персептронного классификатора и называемого *методом персептрона*. Это похоже на квадратический и абсолютный методы, которые мы изучили в разделах «Квадратический подход...» и «Абсолютный подход...» главы 3.

Метод персептрона — способ немного улучшить персептрон

Метод персептрона — это крошечный шаг, который помогает перейти от персептронного классификатора к чуть более хорошему классификатору персептрона. Начнем, однако, с описания несколько менее важного шага. Как и в главе 3, мы сначала сосредоточимся на одной точке и попытаемся улучшить классификатор для нее.

Есть два способа понять шаг персептрона, хотя они и эквивалентны. Первый способ — геометрический, когда мы думаем о классификаторе как о прямой.

Псевдокод для метода персептрона (геометрический)

Случай 1 — если точка классифицирована правильно, не трогать прямую.

Случай 2 — если точка классифицирована неправильно, переместить прямую немного ближе к ней.

Как это работает? Давайте поразмыслим. Если точка неправильно классифицирована, это означает, что она находится не на той стороне прямой. Перемещение прямой ближе к ней не обязательно приведет к тому, что она окажется на правой стороне, но по крайней мере приблизит ее к прямой и, таким образом, приблизит к правильной стороне прямой. Мы будем повторять процесс многократно, так что вполне можно предположить, что однажды переместим прямую за точку, тем самым правильно ее классифицировав. Процесс проиллюстрирован на рис. 5.20.

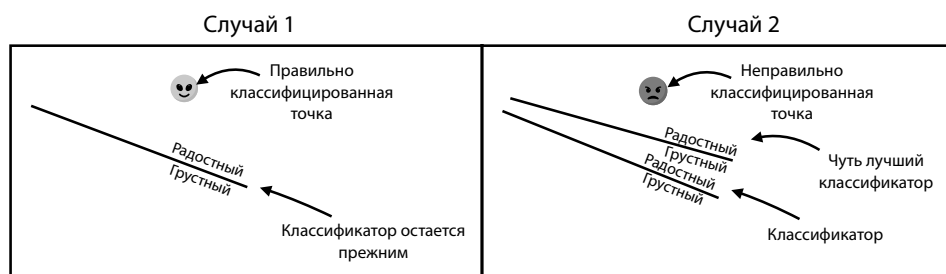


Рис. 5.20. Случай 1 (слева): правильно классифицированная точка говорит прямой оставаться на месте. Случай 2 (справа): неправильно классифицированная точка призывает прямую сдвинуться поближе к ней

Кроме того, мы можем рассмотреть метод персептрона с алгебраической точки зрения.

Псевдокод для метода персептрона (алгебраический)

Случай 1 — если точка классифицирована правильно, оставить классификатор таким, какой он есть.

Случай 2 — если точка классифицирована неправильно, значит, она производит положительную ошибку. Отрегулировать веса и смещение на небольшую величину, чтобы эта ошибка немного уменьшилась.

Геометрический способ — более простой путь визуализации этого метода, но алгебраический способ менее сложен в разработке, поэтому остановимся на нем. Подключим интуицию. Представьте, что у нас имеется классификатор для всего английского языка. Мы пробуем его на предложении «Мне грустно», и он прогнозирует, что оно радостное. Это явно неправильно. Где мы могли ошибиться? Если прогноз состоит в том, что предложение радостное, значит, оно получило положительную оценку. Но оно не должно было получать положительную оценку — для классификации в качестве грустного оценка должна быть отрицательной. Оценка рассчитывается как сумма оценок слов «мне» и «грустно» плюс смещение. Нам нужно уменьшить полученную оценку, чтобы сделать прогноз немного грустнее. Ничего страшного, если мы лишь немного уменьшим его и оценка по-прежнему окажется положительной. Надеемся, что многократное повторение этого процесса однажды превратит оценку в отрицательную и должным образом классифицирует предложение. Способ уменьшить оценку заключается в уменьшении всех ее частей, а именно весов слов «мне» и «грустно», а также смещения. Но на сколько их уменьшать? Мы уменьшаем их на величину, равную скорости обучения, которую мы рассматривали в разделе «Квадратический подход...» в главе 3.

Аналогично если наш классификатор ошибочно классифицирует предложение «Я счастлив» как грустное, то процедура заключается в том, чтобы немного увеличить веса слов «я» и «счастлив», а также смещение на величину, равную скорости обучения.

Проиллюстрируем это на числовом примере. В нем используем скорость обучения $\eta = 0,01$. Представьте, что в нашем распоряжении тот же классификатор, что и в предыдущем разделе, со следующими весами и смещением. Назовем его плохим, так как наша цель — его улучшение.

Плохой классификатор

Веса:

- аак — $a = 1$;
- бип — $b = 2$.

Смещение $c = -4$.

Прогноз $\hat{y} = \text{step}(x_{\text{аак}} + 2x_{\text{бип}} - 4)$.

Следующее предложение неправильно классифицировано моделью, и мы будем использовать его для улучшения весов.

Предложение 1 — «Бип аак аак бип бип бип бип».

Метка — грустный (0).

Для этого предложения количество появлений «аак» равно $x_{\text{аак}} = 2$, а количество появлений «бип» — $x_{\text{бип}} = 5$. Следовательно, оценка $1x_{\text{аак}} + 2x_{\text{бип}} - 4 = 1 \times 2 + 2 \times 5 - 4 = 8$, и прогноз $\hat{y} = \text{step}(8) = 1$.

Чтобы быть классифицированным как грустное, предложение должно иметь отрицательную оценку. Тем не менее классификатор дал ему оценку 8, что является положительным показателем. Нам требуется его уменьшить. Один из способов сделать это — вычесть скорость обучения из веса «аак», веса «бип» и смещения, получая таким образом новые веса, которые мы назовем $a' = 0,99$ и $b' = 1,99$, а также новое смещение $c' = 4,01$. Но заметьте: слово «бип» встречалось гораздо чаще, чем «аак». В некотором смысле первое более важно для оценки предложения, чем второе. Вероятно, нам следует уменьшить вес «бип» сильнее, чем оценку «аак». Уменьшим вес каждого слова на скорость запоминания, помноженную на количество появлений слова в предложении. Другими словами:

- слово «аак» появляется дважды, поэтому уменьшим его вес на две скорости обучения, или 0,02. В итоге получим новый вес $a' = 1 - 2 \times 0,01 = 0,98$;
- слово «бип» встречается пять раз, поэтому уменьшим его вес на пять скоростей обучения, или 0,05. Новый вес $b' = 2 - 5 \times 0,01 = 1,95$;
- смещение увеличивает оценку лишь единожды, поэтому уменьшаем его на скорость обучения (на 0,01). Новое смещение $c' = -4 - 0,01 = -4,01$.

ОТВЛЕЧЕМСЯ Вместо того чтобы вычитать скорость обучения из каждого веса, мы вычли скорость обучения, умноженную на количество появлений слова в предложении. Истинная причина этого — математическое исчисление. Другими словами, когда мы разрабатываем метод градиентного спуска, производная функции ошибки вынуждает нас так поступить. Этот процесс подробно описан в приложении Б, в разделе «Использование градиентного спуска для обучения моделей классификации».

Новый **улучшенный классификатор 1** выглядит так.

Веса:

- аак — $a' = 0,98$;
- бип — $b' = 1,95$.

Смещение $c' = -4,01$.

Прогноз $\hat{y} = \text{step}(0,98x_{\text{аак}} + 1,95x_{\text{бип}} - 4,01)$.

Давайте проверим ошибки обоих классификаторов. Напомним, что ошибка — это абсолютное значение оценки. Мы видим, что плохой классификатор выдает ошибку $|1x_{\text{аак}} + 2x_{\text{бип}} - 4| = |1 \times 2 + 2 \times 5 - 4| = 8$. Улучшенный классификатор выдает ошибку $|0,98x_{\text{аак}} + 1,95x_{\text{бип}} - 4,01| = |0,98 \times 2 + 1,95 \times 5 - 4,01| = 7,7$. Эта ошибка меньше, так что мы действительно улучшили классификатор для данной точки!

Случай, который мы только что разработали, включает неправильно классифицированную точку с отрицательной меткой. А если неправильно классифицированная точка имеет положительную метку? Процедура та же, за исключением того, что вместо вычитания суммы из весов мы ее прибавляем. Вернемся к плохому классификатору и рассмотрим следующее предложение.

Предложение 2 — «Аак аак».

Метка — радостный.

Прогноз для этого предложения такой:

$$\hat{y} = \text{step}(x_{\text{аак}} + 2x_{\text{бип}} - 4) = \text{step}(2 + 2 \times 0 - 4) = \text{step}(-2) = 0.$$

Поскольку прогноз грустный, предложение неправильно классифицировано. Оценка этого предложения равна -2 , и, чтобы классифицировать его как радостное, нужно, чтобы классификатор присвоил ему положительную оценку. Метод персептрона увеличит оценку на -2 за счет увеличения веса слов и смещения следующим образом.

- Слово «аак» встречается дважды, поэтому увеличим его вес на две скорости обучения, или на $0,02$. Получаем новый вес $a' = 1 + 2 \times 0,01 = 1,02$.
- Слово «бип» появляется ноль раз, поэтому не будем увеличивать его вес (это слово не имеет отношения к предложению).
- Смещение увеличивает оценку только один раз, поэтому увеличиваем смещение на скорость обучения (на $0,01$). Новое смещение $c' = -4 + 0,01 = -3,99$.

Новый **улучшенный классификатор 2** работает так.

Веса:

- аак — $a' = 1,02$;
- бип — $b' = 2$.

Смещение $c' = -3,99$.

Прогноз $\hat{y} = \text{step}(1,02x_{\text{аак}} + 2x_{\text{бип}} - 3,99)$.

Теперь сравним ошибки. Поскольку плохой классификатор дал предложению оценку -2 , то ошибка равна $|-2| = 2$. Второй классификатор дал

предложению оценку $1,02 x_{\text{аак}} + 2x_{\text{бип}} - 3,99 = 1,02 \times 2 + 2 \times 0 - 3,99 = -1,95$, и ошибка составляет 1,95. Таким образом, улучшенный классификатор имеет меньшую ошибку в этой точке, чем плохой, чего мы и ожидали.

Обобщим эти два случая и составим псевдокод для метода персептрона.

Псевдокод для метода персептрона

Входные данные:

- персептрон с весами a , b и смещением c ;
- точка с координатами (x_1, x_2) и меткой y ;
- небольшое положительное значение η (скорость обучения).

Результат — персептрон с новыми весами a' , b' и смещением c' .

Процедура

- Прогноз, который персептрон делает для этой точки, равен $\hat{y} = \text{step}(ax_1 + bx_2 + c)$.
- **Сценарий 1.** Если $\hat{y} = y$, **вернуть** исходный персептрон с весами a , b и смещением c .
- **Сценарий 2.** Если $\hat{y} = 1$ и $y = 0$, **вернуть** персептрон со следующими весами и смещением:
 - $a' = a - \eta x_1$;
 - $b' = b - \eta x_2$;
 - $c' = c - \eta x_1$.
- **Сценарий 3.** Если $\hat{y} = 0$ и $y = 1$, **вернуть** персептрон со следующими весами и смещением:
 - $a' = a + \eta x_1$;
 - $b' = b + \eta x_2$;
 - $c' = c + \eta x_1$.

Если персептрон верно классифицирует точку, выходной персептрон совпадает с входным и они оба выдают ошибку 0. Если персептрон неправильно классифицирует точку, выходной персептрон выдает меньшую ошибку, чем входной.

Далее приведен хитрый трюк для уплотнения псевдокода. Обратите внимание на то, что значение $y - \hat{y}$ равно 0, -1 и $+1$ для трех сценариев в методе персептрона. Следовательно, можно обобщить это так.

Псевдокод для метода персептрона

Входные данные:

- персептрон с весами a , b и смещением c ;
- точка с координатами (x_1, x_2) и меткой y ;
- небольшое значение η (скорость обучения).

Результат — персептрон с новыми весами a' , b' и смещением c' .

Процедура — прогноз, который персептрон делает для этой точки, равен $\hat{y} = \text{step}(ax_1 + bx_2 + c)$.

Возврат — персептрон со следующими весами и смещением:

- $a' = a + \eta(y - \hat{y})x_1$;
- $b' = b + \eta(y - \hat{y})x_2$;
- $c' = c + \eta(y - \hat{y})$.

Многократное повторение метода персептрона. Алгоритм персептрона

В этом разделе мы изучим *алгоритм персептрона*, который используется для обучения персептронного классификатора на наборе данных. Напомним, что метод персептрона позволяет немного улучшить персептрон и сделать лучший прогноз для одной точки. Алгоритм персептрона состоит в том, чтобы начать со случайного классификатора и постоянно совершенствовать его, многократно применяя метод персептрона.

Как говорилось в этой главе, мы можем подойти к проблеме с двух сторон: геометрически и алгебраически. Геометрически набор данных задается точками на плоскости, окрашенными в два цвета, а классификатор представляет собой прямую, которая пытается их разделить. На рис. 5.21 представлен набор радостных и грустных предложений, точно таких же, как в начале главы. Первый шаг алгоритма — провести случайную прямую. Ясно, что прямую на рис. 5.21 нельзя назвать хорошим персептронным классификатором, следовательно, она плохо справляется с разделением радостных и грустных предложений. Точки, которые правильно классифицированы, приказывают ей оставаться неподвижной. Точки, которые классифицированы неправильно, приказывают немного сдвинуться в их сторону.

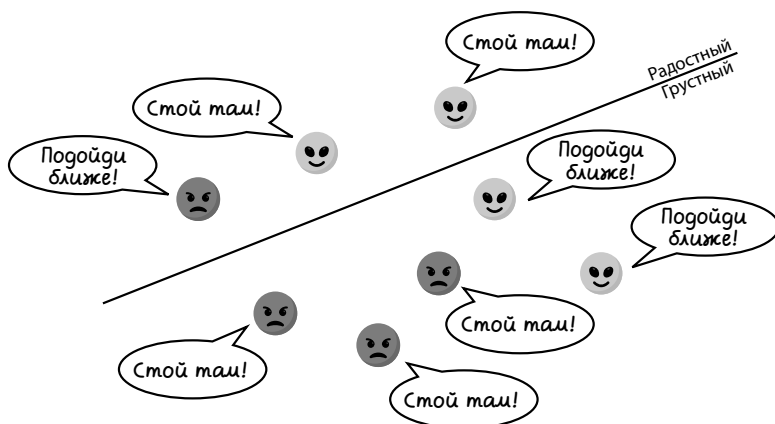


Рис. 5.21. Каждая точка указывает классификатору, что нужно сделать, чтобы улучшить свою жизнь

Следующий шаг в алгоритме персептрона состоит в случайном выборе одной точки (рис. 5.22). Если она классифицирована правильно, прямая остается неподвижной. Если точка классифицирована неправильно, то прямая перемещается немного ближе к ней и становится более подходящей для нее. Это может не подходить для других точек, но сейчас речь не об этом.

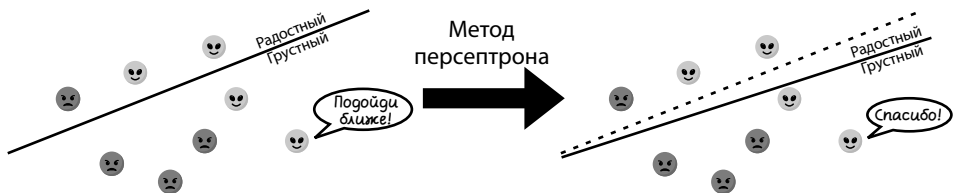


Рис. 5.22. Если мы применим метод персептрона к классификатору и неправильно классифицированной точке, классификатор слегка переместится в ее сторону

Можно предположить, что при многократном повторении процесса мы в конце концов можем подойти к хорошему решению. Эта процедура не всегда дает наилучшее решение. На практике же такой метод зачастую приводит к хорошему решению, как показано на рис. 5.23. Мы называем его алгоритмом персептрона.

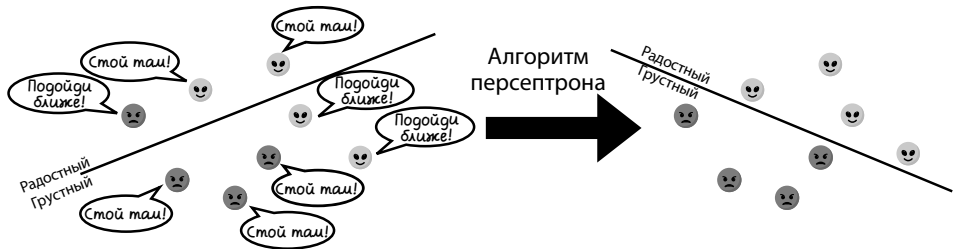


Рис. 5.23. Если применить метод персептрона много раз, каждый раз выбирая случайную точку, мы можем получить классификатор, который правильно определяет большинство точек

Количество запусков алгоритма равно количеству периодов. Таким образом, алгоритм имеет два гиперпараметра: количество периодов и скорость обучения. Псевдокод алгоритма персептрона выглядит следующим образом.

Псевдокод для алгоритма персептрона

Входные данные:

- набор данных точек, помеченных 1 и 0;
- количество периодов n ;
- скорость обучения η .

Результат — персептронный классификатор, состоящий из набора весов и смещения, соответствующих набору данных.

Процедура

- Начать со случайных значений весов и смещения персептронного классификатора.
- Повторять многократно:
 - выбрать случайную точку данных;
 - обновить веса и смещение с помощью метода персептрона.

Возврат — персептронный классификатор с обновленными весами и смещением.

Сколько раз мы должны запускать данный цикл? Другими словами, сколько периодов использовать? Далее приведены несколько критериев, которые помогут принять это решение.

- Запустите цикл фиксированное количество раз, которое может зависеть от доступной вычислительной мощности или времени.
- Выполняйте цикл до тех пор, пока ошибка не станет ниже определенного порога, который мы установили заранее.
- Выполняйте цикл до тех пор, пока ошибка не перестанет существенно изменяться в течение определенного количества периодов.

Обычно, если у нас достаточно вычислительных мощностей, можно запускать его гораздо дольше необходимого, потому что, как только у нас появится хорошо подобранный персептронный классификатор, он, как правило, сильно меняться уже не будет. В разделе «Кодирование алгоритма персептрона» мы закодируем алгоритм персептрона и проанализируем его, измеряя ошибку на каждом шаге, чтобы получить лучшее представление о том, когда следует прекратить его выполнение.

Обратите внимание на то, что в ряде случаев, таких как показанный на рис. 5.24, невозможно найти прямую, разделяющую два класса в наборе данных. Это нормально: цель состоит в том, чтобы найти прямую, которая разделяет набор данных с как можно меньшим количеством ошибок (как на рисунке), и алгоритм персептрона для этого достаточно хорош.

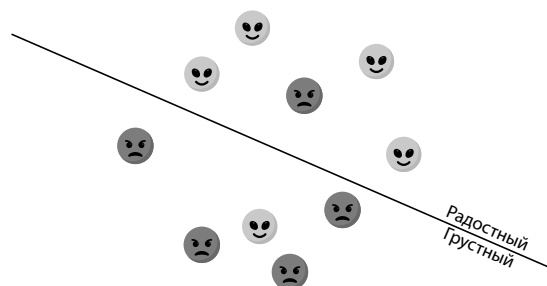


Рис. 5.24. Набор данных с двумя классами, которые невозможно разделить прямой

Градиентный спуск

Вы можете заметить, что процесс обучения этой модели выглядит очень знакомо. Фактически он похож на то, что мы делали в главе 3 с линейной регрессией. Напомним, что цель линейной регрессии — подогнать прямую как можно ближе к набору точек. В главе 3 мы обучали модель линейной регрессии, начав со случайной прямой, и делали небольшие шаги, подбираясь все ближе и ближе к точкам. Затем использовали аналогию со спуском с горы Ошибочной, делая маленькие шаги к подножию. Высота в каждой точке горы — это средняя функция ошибки персептрона, которую мы определили как абсолютную или квадратическую ошибку. Следовательно, спуск с горы эквивалентен минимизации ошибки, что эквивалентно нахождению наилучшего соответствия прямой. Мы назвали этот процесс градиентным спуском, потому что градиент — это именно тот вектор, который указывает в направлении наибольшего роста (поэтому его отрицательные значения указывают в направлении наибольшего уменьшения), и, сделав шаг в этом направлении, мы спустимся ниже всего.

В этой главе происходит то же самое. Проблема немного иная, так как не требуется подгонять прямую как можно ближе к набору точек. Вместо этого мы хотим провести прямую, которая наилучшим образом разделяет два набора точек. Алгоритм персептрона — это процесс, который начинается со случайной прямой и медленно перемещает ее шаг за шагом, чтобы получить наилучший разделитель. Здесь аналогия со спуском с горы работает точно так же. Единственное отличие в том, что для этой горы высота в каждой точке равна средней ошибке персептрона, о которой мы узнали в разделе «Как сравнить классификаторы? Функция ошибки».

Стохастический и пакетный градиентный спуск

Способ, которым мы разработали алгоритм персептрона в этом разделе, заключается в многократном взятии одной точки за раз и настройке персептрона (прямой), чтобы он немного лучше подходил для этой точки. Этот процесс называется периодом. Однако, как и в случае с линейной регрессией в разделе «Одна точка за раз или несколько?..» главы 3, более выигрышный подход состоит в том, чтобы брать за раз несколько точек и подстраивать персептрон для них всех. Крайний случай состоит в том, чтобы взять все точки в наборе за один раз и настраивать персептрон за один шаг так, чтобы он лучше соответствовал им всем. В упомянутом разделе мы называем эти подходы *стохастическим*, *мини-пакетным* и *пакетным градиентным спуском*. Здесь же применяем формальный алгоритм персептрона с использованием мини-пакетного градиентного спуска. Математические подробности приведены в приложении Б, в разделе «Использование градиентного спуска для обучения моделей классификации», где алгоритм персептрона описан в общем виде с применением градиентного спуска мини-пакета.

Кодирование алгоритма персептрона

Теперь, когда мы разработали алгоритм персептрона для своего приложения анализа настроений, создадим для него код. Сначала напомним код с нуля, чтобы он соответствовал исходному набору данных, а затем воспользуемся Tugl Create. В реальной жизни мы всегда задействуем библиотеку, и нам не нужно кодировать собственные алгоритмы. Тем не менее полезно хотя бы один раз закодировать часть алгоритмов — думайте об этом как о выполнении деления столбиком. Обычно мы делим столбиком без калькулятора — хорошо, что нам приходилось заниматься этим в школе. Теперь, проделывая это с помощью калькулятора, мы знаем, что творится «под капотом». Далее приведен код для этого раздела, а набор данных показан в табл. 5.5.

- **Блокнот** `Coding_perceptron_algorithm.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_5_Perceptron_Algorithm/Coding_perceptron_algorithm.ipynb

Таблица 5.5. Набор данных об инопланетянах, время, когда они произносили каждое из слов, и их настроение

Аак	Бип	Радостный/грустный
1	0	0
0	2	0
1	1	0
1	2	0
1	3	1
2	2	1
2	3	1
3	2	1

Начнем с определения набора данных как массива NumPy. Признаки соответствуют двум числам, соотносящимся с появлениями «аак» и «бип». Метки равны 1 для радостных предложений и 0 — для грустных:

```
import numpy as np
features = np.array([[1,0],[0,2],[1,1],[1,2],[1,3],[2,2],[2,3],[3,2]])
labels = np.array([0,0,0,0,1,1,1,1])
```

Это дает нам график, изображенный на рис. 5.25. На нем радостные предложения представлены треугольниками, а грустные — квадратами.

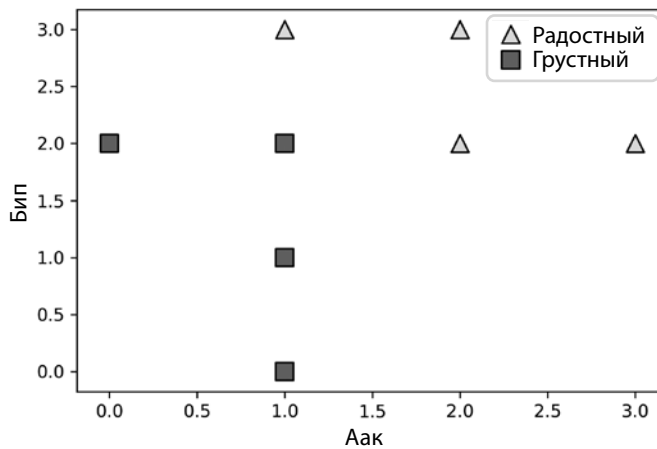


Рис. 5.25. График набора данных

Кодирование метода персептрона

В этом разделе мы закодируем метод персептрона. Сделаем это с помощью стохастического градиентного спуска (по одной точке за раз), но можем использовать также мини-пакетный или пакетный градиентный спуск. Начнем с кодирования функций оценки и прогноза. Обе они получают одни и те же входные данные, которые представляют собой веса модели, смещение и признаки одной точки данных. Функция оценки возвращает оценку, которую модель присваивает этой точке данных, а функция прогнозирования возвращает 1, если оценка больше или равна нулю, и 0, если оценка меньше нуля. Для этой функции мы используем точечное произведение, определенное в разделе «Построение графика функции ошибок и умение вовремя остановиться» главы 3:

```
def score(weights, bias, features):  
    return np.dot(features, weights) + bias
```

Вычисляет точечное произведение между весами и признаками, добавляет смещение и применяет ступенчатую функцию

Чтобы создать функцию прогнозирования, сначала пишем ступенчатую функцию. Прогнозирование — это пошаговая функция оценки:

```
def step(x):  
    if x >= 0:  
        return 1  
    else:  
        return 0  
  
def prediction(weights, bias, features):  
    return step(score(weights, bias, features))
```

Смотрит на результат и, если он положительный или нулевой, возвращает 1, если отрицательный — 0

Далее мы кодируем функцию ошибки для одной точки. Напомним, что ошибка равна нулю, если точка классифицирована правильно, и абсолютному значению оценки, если неправильно. Функция принимает в качестве входных данных веса и смещения модели, а также признаки и метку точки данных:

```
def error(weights, bias, features, label):
    pred = prediction(weights, bias, features)
    if pred == label:
        return 0
    else:
        return np.abs(score(weights, bias, features))
```

Если прогноз соответствует метке, то точка хорошо классифицирована, что означает нулевую ошибку

Если прогноз отличается от метки, то точка классифицируется неправильно, а это означает, что ошибка равна абсолютному значению оценки

Теперь напомним функцию для средней ошибки перцептрона. Она вычисляет среднее значение ошибок всех точек в нашем наборе данных:

```
def mean_perceptron_error(weights, bias, features, labels):
    total_error = 0
    for i in range(len(features)):
        total_error += error(weights, bias, features[i], labels[i])
    return total_error/len(features)
```

Перебирает данные и для каждой точки добавляет ошибку в эту точку, а затем возвращает эту ошибку

Сумма ошибок делится на количество точек, и получается средняя ошибка перцептрона

Теперь, когда у нас есть функция ошибки, можем сделать следующий шаг и закодировать метод перцептрона. Мы закодируем сокращенную версию алгоритма из конца раздела «Метод перцептрона — способ немного улучшить перцептрон». Однако в записной книжке он закодирован обоими способами, первый из которых использует оператор `if` для проверки того, хорошо ли классифицирована точка:

```
def perceptron_trick(weights, bias, features, label, learning_rate = 0.01):
    pred = prediction(weights, bias, features)
    for i in range(len(weights)):
        weights[i] += (label-pred)*features[i]*learning_rate
    bias += (label-pred)*learning_rate
    return weights, bias
```

Обновляет веса и смещения с использованием метода перцептрона

Кодирование алгоритма перцептрона

Теперь, когда нам доступен метод перцептрона, можем закодировать сам алгоритм перцептрона. Напомним, что он состоит в том, чтобы, начав со случайного перцептронного классификатора, повторять метод перцептрона много раз

(по количеству периодов). Чтобы отслеживать производительность алгоритма, мы будем отслеживать среднюю ошибку персептрона в каждый из периодов. В качестве входных данных у нас имеются данные (признаки и метки), скорость обучения, которая по умолчанию равна 0,01, и количество периодов — по умолчанию 200. Код для алгоритма персептрона выглядит следующим образом:

```
def perceptron_algorithm(features, labels, learning_rate = 0.01,
    epochs = 200):
    weights = [1.0 for i in range(len(features[0]))]
    bias = 0.0
    errors = []
    for epoch in range(epochs):
        error = mean_perceptron_error(weights, bias, features, labels)
        errors.append(error)
        i = random.randint(0, len(features)-1)
        weights, bias = perceptron_trick(weights, bias, features[i],
            labels[i])
    return weights, bias, errors
```

Повторяет процесс столько раз, сколько задано периодов

Инициализирует веса равными 1, а смещение — равным 0. Вы вполне можете инициализировать их небольшими случайными числами

Массив для хранения ошибок

Вычисляет среднюю ошибку персептрона и сохраняет ее

Выбирает случайную точку в наборе данных

Применяет алгоритм персептрона для обновления весов и смещения модели на основе данной точки

Давайте же запустим алгоритм на нашем наборе данных!

```
perceptron_algorithm(features, labels)
```

Вывод: ([0.6299999999999997, 0.17999999999999938], -1.0400000000000007)

Выходные данные показывают, что полученные нами веса и смещения следующие:

- вес «аак» — 0,63;
- вес «бип» — 0,18;
- смещение — -1,04.

Мы могли бы получить и другой ответ из-за случайности выбора точек внутри алгоритма. Чтобы код в репозитории всегда возвращал один и тот же ответ, случайное начальное значение устанавливается равным нулю.

На рис. 5.26 показаны два графика: слева — подгонка прямой, справа — функция ошибок. Линия, соответствующая результирующему персептрону, выглядит как жирная прямая, которая правильно классифицирует каждую точку. Более тонкие прямые соответствуют персептронам, полученным после каждого из 200 периодов. Обратите внимание на то, как в каждом периоде прямая становится все более подходящей для точек. Ошибка уменьшается в основном по мере увеличения количества периодов, пока не достигнет нуля примерно на 140-м периоде, что означает: все точки классифицированы правильно.

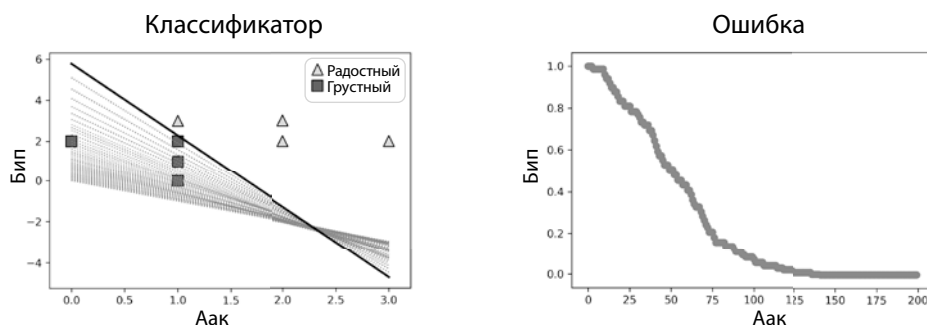


Рис. 5.26. Слева — график результирующего классификатора. Он правильно классифицирует каждую точку. Справа — график ошибок. Чем больше периодов проходит алгоритм персептрона, тем меньше становится ошибка

Это и есть код алгоритма персептрона! Как упоминалось ранее, на практике мы обычно не кодируем алгоритмы вручную, а используем библиотеки, например Turi Create или Scikit-Learn. Эту возможность и рассмотрим в следующем разделе.

Кодирование алгоритма персептрона с помощью Turi Create

В этом разделе мы научимся кодировать алгоритм персептрона в Turi Create. Код находится в том же блокноте, что и в предыдущем упражнении. Наша первая задача — импортировать Turi Create и создать SFrame со своими данными из словаря следующим образом:

```
import turicreate as tc

datadict = {'aack': features[:,0], 'beep': features[:,1], 'prediction':
labels}
data = tc.SFrame(datadict)
```

Далее мы создаем и обучаем персептронный классификатор, используя объект `logistic_classifier` и метод `create`, как показано в следующем коде. Входными данными служат набор данных и имя столбца, содержащего метки (`target`):

```
perceptron = tc.logistic_classifier.create(data, target='prediction')
```

Вывод:

Итерация	Проходы	Время	Точность обучения
1	2	1.003120	1.000000
2	3	1.004235	1.000000
3	4	1.004840	1.000000
4	5	1.005574	1.000000

УСПЕХ: Найдено оптимальное решение.

Обратите внимание на то, что алгоритм персептрона работал в течение четырех периодов и в последнем (а фактически во всех) имел точность обучения 1. Это означает, что каждая точка в наборе данных была правильно классифицирована.

Наконец, мы можем вывести веса и смещения модели, используя следующую команду:

```
perceptron.coefficients
```

Выходные данные этой функции показывают следующие веса и смещения для результирующего персептрона:

- вес «аак» — 2,7;
- вес «бип» — 2,46;
- смещение — -8,96.

Результаты отличаются от полученных вручную, но оба персептрона хорошо работают на нашем наборе данных.

Применение алгоритма персептрона

Алгоритм персептрона имеет множество применений в реальной жизни. Практически в любом случае, когда нам нужно ответить на вопрос «да» или «нет» и ответ предсказан на основе предыдущих данных, алгоритм персептрона способен помочь. Вот несколько примеров реального применения алгоритма персептрона.

Фильтры нежелательной почты

Точно так же, как мы предсказали, будет предложение радостным или грустным, основываясь на содержащихся в нем словах, можем предсказать, является электронное письмо спамом или нет, основываясь на употребленных в нем словах. Мы можем использовать и другие признаки, например:

- длину электронного письма;
- размер вложений;
- количество отправителей;
- является ли отправителем кто-либо из наших контактов.

В настоящее время алгоритм персептрона (и его более продвинутые аналоги: логистическая регрессия и нейронные сети) и другие модели классификации широко используются как часть конвейеров классификации спама большинством крупнейших поставщиков электронной почты.

Мы также можем классифицировать электронные письма, применяя алгоритмы классификации, такие как алгоритм персептрона. Разделение

электронной почты на личную, подписки и рекламные акции — это точно такая же задача. Даже составление потенциальных ответов на электронное письмо — тоже вопрос классификации, за исключением того факта, что теперь метки, которые мы используем, являются ответами на электронные письма.

Системы рекомендаций

Во многих системах рекомендация видео, фильма, песни или продукта пользователю сводится к ответу «да» или «нет». В этих случаях вопрос может быть любым из следующих.

- Нажмет ли пользователь на видео/фильм, который мы рекомендуем?
- Будет ли пользователь смотреть все видео/фильмы, которые мы рекомендуем?
- Будет ли пользователь слушать песню, которую мы рекомендуем?
- Купит ли пользователь продукт, который мы рекомендуем?

Признаки могут быть любыми — от демографических (возраст, пол, местоположение пользователя) до поведенческих (какие видео он смотрел, какие песни слушал, какие продукты покупал). Можно представить, каким длинным может быть пользовательский вектор! Для такой работы необходимы большие вычислительные мощности и очень продуманные реализации алгоритмов.

Такие компании, как Netflix, YouTube и Amazon, среди многих других, применяют в своих рекомендательных системах алгоритм персептрона или аналогичные, более продвинутые модели классификации.

Здравоохранение

Многие медицинские модели применяют алгоритмы классификации, такие как алгоритм персептрона, для ответа на следующие вопросы:

- Страдает ли пациент конкретным заболеванием?
- Будет ли определенное лечение действенным для пациента?

Признаками этих моделей обычно служат симптомы, наблюдаемые у пациента, а также его история болезни. Для таких типов алгоритмов требуется очень высокий уровень производительности.

Рекомендовать неправильное лечение — это гораздо более серьезное дело, чем предлагать видео, которое пользователь не станет смотреть. Для ознакомления с подобными типами анализа обратитесь к главе 7, где мы говорим о точности и других способах оценки моделей классификации.

Компьютерное зрение

Алгоритмы классификации, такие как алгоритм персептрона, широко используются в компьютерном зрении, а точнее, в распознавании изображений.

Представьте, что у нас есть картинка и мы хотим научить компьютер определять, изображена ли на ней собака. Это модель классификации, в которой признаками выступают пиксели изображения.

Алгоритм персептрона обладает приличной производительностью в организованных наборах данных изображений, таких как MNIST, который представляет собой набор данных рукописных цифр. Однако с более сложными изображениями он работает не очень хорошо. Для них используются модели, состоящие из комбинации множества персептронов. Эти модели удачно называются многослойными персептронами, или нейронными сетями, и о них мы подробно поговорим в главе 10.

Резюме

- Классификация — важная часть машинного обучения. Она похожа на регрессию в том смысле, что состоит из обучения алгоритма с помеченными данными и его использования для прогнозирования будущих (немаркированных) данных. Ее отличие от регрессии состоит в том, что при классификации прогнозами служат категории, такие как «да/нет», «спам/не спам» и т. д.
- Персептронные классификаторы работают по принципу присвоения веса каждому из признаков и смещения. Оценка точки данных рассчитывается как сумма произведений весов и признаков плюс смещение. Если оценка больше или равна нулю, классификатор прогнозирует «да», в противном случае — «нет».
- В задаче анализа настроений персептрон состоит из оценки каждого слова в словаре вместе со смещением. Радостные слова обычно дают положительные оценки, а грустные — отрицательные. Нейтральные слова, например *предлоги*, как правило, получают оценку, близкую к нулю.
- Смещение помогает определить, радостное пустое предложение или грустное. Если смещение положительное, то пустое предложение радостное, а если отрицательное, то грустное.
- Графически мы можем изобразить персептрон как прямую, разделяющую два класса точек, которые можно представить окрашенными в два разных цвета. В более высоких измерениях персептрон представляет собой разделяющую точки гиперплоскость.
- Алгоритм персептрона начинает со случайной прямой, а затем медленно перемещает ее, чтобы наилучшим образом разделить точки. На каждой итерации он выбирает случайную точку. Если точка классифицирована верно, прямая остается неподвижной. Если классификация неверная, то прямая сдвигается немного ближе к точке, чтобы «переступить» через нее и классифицировать ее правильно.
- Алгоритм персептрона применяется во множестве областей, включая спам-фильтры, рекомендательные системы и здравоохранение.

Упражнения

Упражнение 5.1

Возьмем данные о пациентах с положительным или отрицательным тестом на COVID-19. У них наблюдаются следующие симптомы: кашель (C), температура (F), затрудненное дыхание (B) и утомляемость (T).

Пациент	Кашель (C)	Температура (F)	Затрудненное дыхание (B)	Утомляемость (T)	Диагноз (D)
1		X	X	X	Болен
2	X	X		X	Болен
3	X		X	X	Болен
4	X	X	X		Болен
5	X			X	Здоров
6		X	X		Здоров
7		X			Здоров
8				X	Здоров

Постройте модель персептрона для классификации такого набора данных.

Подсказка: можно использовать алгоритм персептрона или прикинуть хорошую модель персептрона, которая сработает.

Упражнение 5.2

Подумайте над моделью персептрона, которая сопоставляет с точкой (x_1, x_2) прогноз $\hat{y} = \text{step}(2x_1 + 3x_2 - 4)$. Ее пограничная прямая описывается уравнением $2x_1 + 3x_2 - 4 = 0$. У нас имеется точка $p = (1, 1)$ с меткой 0.

- Убедитесь, что точка p классифицирована моделью неверно.
- Рассчитайте ошибку персептрона, которую дает модель в точке p .
- Используйте метод персептрона для получения новой модели, которая по-прежнему неверно классифицирует p , но уже с меньшей ошибкой. В качестве скорости обучения можно взять $\eta = 0,01$.
- Получите прогноз, данный новой моделью в точке p , после чего убедитесь, что полученная ошибка персептрона меньше предыдущей.

Упражнение 5.3

Персептроны особенно часто используются для реализации логических вентилей, таких как AND или OR.

- Постройте персептрон, моделирующий вентиль AND. Иными словами, создайте персептрон, удовлетворяющий приведенному в таблице набору данных (где x_1, x_2 — это признаки, а y — метка).

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

Б. Аналогичным образом постройте персептрон, моделирующий OR, используя следующий набор данных.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

В. Продемонстрируйте, что не существует персептрона, который моделирует вентиль XOR, заданный следующим набором данных.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

6

Непрерывный подход к разделению точек. Логистические классификаторы



В этой главе

- ✓ Разница между жесткими и мягкими назначениями в моделях классификации.
- ✓ Сигмоидная функция, функция непрерывной активации.
- ✓ Дискретные перцептроны против непрерывных перцептронов, называемых также логистическими классификаторами.
- ✓ Алгоритм логистической регрессии для классификации данных.
- ✓ Кодирование алгоритма логистической регрессии на Python.
- ✓ Использование логистического классификатора в Turi Create для анализа настроений в обзорах фильмов.
- ✓ Применение логистической функции со многими переменными для построения классификаторов для более чем двух классов.



В предыдущей главе мы построили классификатор, определяющий, является предложение радостным или грустным. Но предположим, что некоторые предложения могут быть более радостными, чем другие. Например, оба предложения: «У меня все хорошо» и «Сегодня был самый замечательный день в моей жизни!» — радостные, но второе намного радостнее первого. Хорошо бы иметь классификатор, который не только определяет радостные или грустные предложения, но и дает оценку того, насколько они радостные, — скажем, такой, который говорит нам, что первое предложение радостное на 60 %, а второе — на 95 %? В этой главе мы определим *логистический классификатор*, который делает именно это. Такой классификатор присваивает каждому предложению оценку от 0 до 1 таким образом, что чем радостнее предложение, тем она выше.

В двух словах, логистический классификатор — это тип модели, которая работает как перцептронный классификатор, за исключением того, что вместо ответа «да» или «нет» возвращает число от 0 до 1. Цель здесь состоит в том, чтобы присвоить оценки, близкие к 0, самым грустным предложениям, а близкие к 1 — самым радостным. Близкие к нейтральным предложения получают оценку 0,5. Смещение в 0,5 является общепринятым, хотя и выбрано произвольно. В главе 7 мы увидим, как настроить его для оптимизации нашей модели, а в этой главе возьмем 0,5.

Эта глава основана на главе 5, поскольку алгоритмы, которые мы здесь разрабатываем, похожи на создаваемые там, за исключением пары технических различий. Хорошее понимание главы 5 поможет лучше усвоить материал данного раздела. В главе 5 мы описали алгоритм перцептрона с использованием функции ошибок (она сообщает, насколько классификатор хорош) и итеративный шаг, который понемногу его улучшает. В этой главе изучаем алгоритм логистической регрессии, работающий аналогично. Основные различия вот в чем.

- Ступенчатая функция заменяется новой функцией активации, которая возвращает значения от 0 до 1.
- Функция ошибки перцептрона заменяется новой, основанной на вычислении вероятности.
- Метод обучения перцептрона заменен новым, который улучшает классификатор на основе новой функции ошибок.

ОТВЛЕЧЕМСЯ В этой главе мы проводим множество математических вычислений. Если вы будете самостоятельно использовать уравнения, то можете обнаружить, что ваши расчеты немного отличаются от приведенных в книге. Здесь числа округляются в самом конце уравнения, а не между шагами. Это не должно оказывать большого влияния на результат.

В конце главы применим полученные знания к реальному набору обзоров фильмов на популярном сайте IMDb (www.imdb.com). Мы используем логистический классификатор для прогноза о том, будут ли отзывы о фильмах положительными или отрицательными.

Код для этой главы доступен в следующем репозитории GitHub: https://github.com/luisguiserrano/manning/tree/master/Chapter_6_Logistic_Regression.

Логистические классификаторы — непрерывная версия персептронных классификаторов

В главе 5 мы рассмотрели персептрон, который представляет собой тип классификатора, использующего особенности данных для прогнозирования. Прогноз может быть равен 1 или 0. Это называется *дискретным персептроном*, потому что возвращается ответ из дискретного набора (набора, содержащего 0 и 1). В этой главе мы изучаем *непрерывные персептроны*, которые возвращают ответ, который может быть любым числом в интервале от 0 до 1. Более распространенное название непрерывных персептронов — *логистические классификаторы*. Выходные данные логистического классификатора можно интерпретировать как оценку, а его цель состоит в том, чтобы присвоить оценки, как можно более близкие к метке точек, — точки с меткой 0 должны получить оценки, близкие к 0, а точки с меткой 1 — близкие к 1.

Мы можем визуализировать непрерывные персептроны аналогично дискретным — с линией (или плоскостью высокой размерности), которая разделяет два класса данных. Единственное их отличие состоит в том, что прогноз дискретного персептрона выглядит так: все, что находится по одну сторону линии, имеет метку 1, по другую — метку 0, тогда как непрерывный персептрон присваивает значение от 0 до 1 всем точкам на основе их положения относительно прямой. Каждая точка на самой прямой получает оценку 0,5. Это означает, что модель не может решить, радостное предложение или грустное. Так, в примере с анализом настроения предложение «Сегодня вторник» не радостное, но и не грустное, поэтому модель присвоила бы ему оценку, близкую к 0,5. Точки в положительной зоне получают оценки больше 0,5, а те, что расположены дальше от прямой 0,5 в положительном направлении, — близкие к 1. Точки в отрицательной зоне получают оценки меньше 0,5, и опять же, чем дальше точка от прямой, тем ближе ее оценка к 0. Ни одна точка не получит значения 1 или 0 (если только мы не рассматриваем точки на бесконечности), что показано на рис. 6.1.

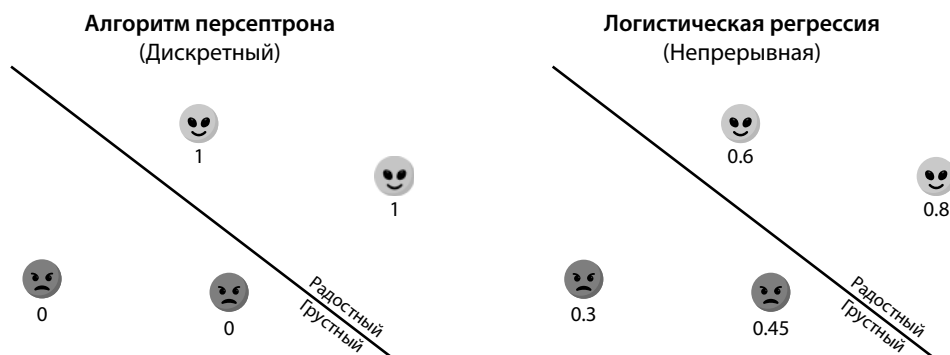


Рис. 6.1. Слева — алгоритм персептрона обучает дискретный персептрон, где предсказания равны 0 (радостный) и 1 (грустный).

Справа — алгоритм логистической регрессии обучает непрерывный персептрон, где предсказания представляют собой числа от 0 до 1, которые обозначают прогнозируемый уровень радости

Почему мы называем это *классификацией* вместо *регрессии*, учитывая, что логистический классификатор выдает не состояние как таковое, а число? Причина в том, что после подсчета оценок мы можем разделить их на два класса, а именно: с оценкой 0,5 или выше и оценкой ниже 0,5. Графически два класса разделены разграничивающей прямой, как и в случае с персептронным классификатором. Однако алгоритм, применяемый для обучения логистических классификаторов, называется *алгоритмом логистической регрессии*. Название немного странное, но мы оставим его как есть, чтобы оно соответствовало используемому в другой литературе.

Вероятностный подход к классификации. Сигмоидная функция

Можно ли немного изменить модели персептрона из предыдущего раздела, чтобы получать оценку каждого предложения вместо просто «радостный» или «грустный»? Вспомните, как именно делались прогнозы в моделях персептрона. Мы оценивали каждое предложение, рассматривая каждое слово, суммируя оценки и добавляя смещение. Если оценка была положительной, мы прогнозировали, что предложение радостное, а если отрицательной, что грустное. Другими словами, применяли ступенчатую функцию к оценке. Ступенчатая функция возвращает 1, если оценка оказалась неотрицательной, и 0 — если отрицательной.

Здесь мы сделаем нечто подобное. Возьмем функцию, которая получает оценку в качестве входных данных и выдает число от 0 до 1. Число близко к 1, если оценка положительная, и близко к 0, если отрицательная. Если оценка равна нулю, то результат равен 0,5. Представьте, что мы берем числовую строку целиком и сжимаем ее до интервала между 0 и 1. Как выглядит такая функция, показано на рис. 6.2.

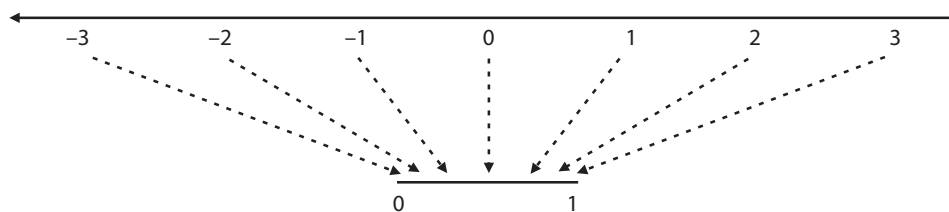


Рис. 6.2. Сигмоидная функция трансформирует всю числовую строку в интервал (0, 1)

В этом может помочь множество функций, в данном случае мы используем одну из них, называемую *сигмоидной* и обозначаемую греческой буквой «сигма» (σ). Формула сигмоидной кривой выглядит следующим образом:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Но по-настоящему важна здесь не формула, а то, что делает функция, сжимая строку действительных чисел в интервал (0, 1). На рис. 6.3 сравниваются графики ступенчатой и сигмоидной функций. Ступенчатая функция выдает значение 0 для любого отрицательного входного сигнала и 1 — для любого положительного или нулевого. На отметке 0 у нее наблюдается разрыв. Сигмоидная функция выдает значения меньше 0,5 для отрицательных входных данных и значения больше 0,5 — для положительных. При нулевом значении выдает значение 0,5. Она непрерывна и дифференцируема везде.

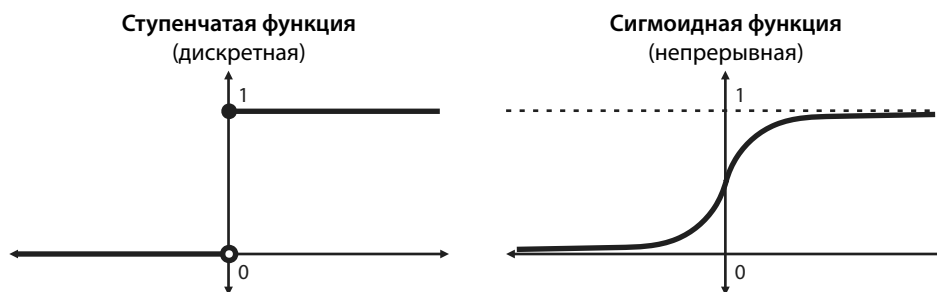


Рис. 6.3. Сравнение функций: *слева* — ступенчатая, используемая для построения дискретных перцептронов; *справа* — сигмоидная, применяемая для построения непрерывных перцептронов

Сигмоидная функция, как правило, лучше ступенчатой по целому ряду причин. Непрерывные прогнозы дают намного больше информации, чем дискретные. Кроме того, когда мы переходим к исчислению, сигмоидная функция имеет гораздо лучшую производную, чем ступенчатая. Ступенчатая функция имеет производную от нуля, за исключением начала координат, где

она не определена. В табл. 6.1 мы вычисляем некоторые значения сигмоидной функции и убеждаемся, что она работает так, как нам нужно. Обратите внимание на то, что для больших отрицательных входов вывод близок к 0, тогда как для больших положительных входов — близок к 1. Для входа 0 выход равен 0,5.

Таблица 6.1. Некоторые входные и соответствующие выходные данные в рамках сигмоидной функции

x	$\sigma(x)$
-5	0,007
-1	0,269
0	0,500
1	0,731
5	0,993

Прогноз логистического классификатора получается применением сигмоидной функции к оценке и возвращает число от 0 до 1. Как упоминалось ранее, в нашем примере это число можно интерпретировать как вероятность того, что предложение радостное.

В главе 5 мы определили функцию ошибки для персептрона (так называемую ошибку персептрона). Мы использовали ее для итеративного построения персептронного классификатора. В этой главе пойдем по тому же пути. Ошибка непрерывного персептрона слегка отличается от ошибки дискретного, но сходство все же есть.

Набор данных и прогнозы

В этой главе задействуем тот же пример, что и в предыдущей. Представим, что у нас есть набор предложений на иностранном языке с метками «радостный» и «грустный», обозначенными 1 и 0 соответственно. Набор данных для этой главы немного отличается от набора данных для главы 5. Он показан в табл. 6.2.

Таблица 6.2. Набор данных предложений с их радостными/грустными метками. Координаты — это количество появлений слов «аак» и «бип» в предложении

Предложение	Слова	Координаты (#аак, #бип)	Метка
1	Аак бип бип аак аак	(3, 2)	Грустный (0)
2	Бип аак бип	(1, 2)	Радостный (1)
3	Бип!	(0, 1)	Радостный (1)
4	Аак аак	(2, 0)	Грустный (0)

Используемая модель имеет следующие веса и смещение.

Логистический классификатор 1:

- вес «аак» $a = 1$;
- вес «бип» $b = 2$;
- смещение $c = -4$.

Мы применяем те же обозначения, что и в главе 5, где переменные $x_{\text{аак}}$ и $x_{\text{бип}}$ отражают появления «аак» и «бип» соответственно. Перцептронный классификатор будет делать прогноз в соответствии с формулой $\hat{y} = \text{step}(ax_{\text{аак}} + bx_{\text{бип}} + c)$, но поскольку это логистический классификатор, он использует сигмоидную функцию вместо ступенчатой. Таким образом, его прогноз равен $\hat{y} = \sigma(ax_{\text{аак}} + bx_{\text{бип}} + c)$. В этом случае прогноз выглядит так:

$$\hat{y} = \sigma(1x_{\text{аак}} + 2x_{\text{бип}} - 4).$$

Итак, классификатор делает следующие прогнозы для нашего набора данных:

- **Предложение 1:** $\hat{y} = \sigma(3 + 2 \times 2 - 4) = \sigma(3) = 0,953$.
- **Предложение 2:** $\hat{y} = \sigma(1 + 2 \times 2 - 4) = \sigma(1) = 0,731$.
- **Предложение 3:** $\hat{y} = \sigma(0 + 2 \times 1 - 4) = \sigma(-2) = 0,119$.
- **Предложение 4:** $\hat{y} = \sigma(2 + 2 \times 0 - 4) = \sigma(-2) = 0,119$.

Границей между классами «радостный» и «грустный» служит прямая $x_{\text{аак}} + 2x_{\text{бип}} - 4 = 0$, изображенная на рис. 6.4.

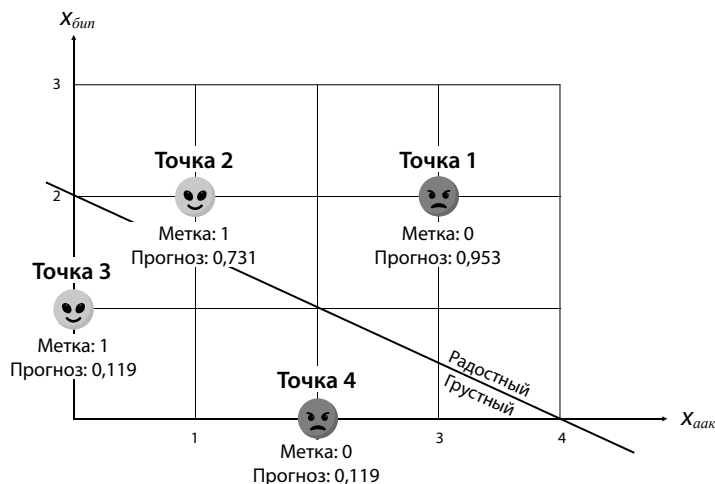


Рис. 6.4. График набора данных из табл. 6.2 с прогнозами. Обратите внимание на то, что точки 2 и 4 классифицированы правильно, а 1 и 3 — нет

Прямая делит плоскость на положительную (радостную) и отрицательную (грустную) зоны. Положительная зона формируется точками с прогнозом, бóльшим или равным 0,5, а отрицательная — с прогнозом меньше 0,5.

Функции ошибок. Абсолютная, квадратическая и логистическая функция потерь

В этом разделе мы построим три функции ошибок для логистического классификатора. Какими свойствами должна обладать хорошая функция ошибок? Далее приведены кое-какие примеры.

- Если точка классифицирована правильно, ошибка — небольшое число.
- Если точка классифицирована неправильно, ошибка — большое число.
- Ошибка классификатора для набора точек — это сумма (или среднее значение) ошибок во всех точках.

Многие функции удовлетворяют этим свойствам, и мы разберем три из них: абсолютную ошибку, квадратическую ошибку и логистическую функцию потерь. В табл. 6.3 содержатся метки и прогнозы для четырех точек, соответствующих предложениям в нашем наборе данных, со следующими характеристиками.

- Точкам на прямой дается прогноз 0,5.
- Точкам, находящимся в положительной зоне, даются прогнозы выше 0,5, и чем дальше они от прямой в этом направлении, тем ближе прогноз для них к 1.
- Точкам, находящимся в отрицательной зоне, даются прогнозы ниже 0,5, и чем дальше они от прямой в этом направлении, тем ближе прогноз для них к 0.

Таблица 6.3. Четыре точки — две счастливые и две грустные — со своими прогнозами (см. рис. 6.4).

Точка	Метка	Прогноз	Ошибка?
1	0 (Грустный)	0,953	Должно быть большим
2	1 (Радостный)	0,731	Должно быть небольшим
3	1 (Радостный)	0,119	Должно быть большим
4	0 (Грустный)	0,119	Должно быть небольшим

Обратите внимание на то, что точки 1 и 4 классифицированы правильно, а точки 2 и 3 — нет. Хорошая функция ошибок должна присваивать небольшие ошибки правильно классифицированным точкам, а большие — плохо классифицированным. Точки 2 и 4 дают прогноз, близкий к метке, поэтому у них должны быть небольшие ошибки. И наоборот, точки 1 и 3 получают прогноз, который далек от метки, так что они должны иметь большие ошибки. Далее рассмотрим три функции ошибок, обладающие этим конкретным свойством.

Функция ошибки 1. Абсолютная ошибка

Здесь *абсолютная ошибка* аналогична абсолютной ошибке, которую мы определяли для линейной регрессии в главе 3. Это абсолютное значение разницы между прогнозом и меткой. Как видите, она велика, когда предсказание находится далеко от метки, и мала, когда близко.

Функция ошибки 2. Квадратическая ошибка

Как и в случае линейной регрессии, у нас есть *квадратическая ошибка*. Это квадрат разницы между прогнозом и меткой, и он работает по той же причине, по которой работает абсолютная ошибка.

Прежде чем продолжить, рассчитаем абсолютную и квадратическую ошибки для точек, приведенных в табл. 6.4. Обратите внимание на то, что точки 2 и 4 (правильно классифицированные) имеют небольшие ошибки, а точки 1 и 3 (неправильно классифицированные) — большие, чего мы и добивались.

Таблица 6.4. К точкам из табл. 6.3 приложены абсолютная и квадратическая погрешности

Точка	Метка	Прогнозируемая метка	Абсолютная ошибка	Квадратическая ошибка
1	0 (Грустный)	0,953	0,953	0,908
2	1 (Радостный)	0,731	0,269	0,072
3	1 (Радостный)	0,119	0,881	0,776
4	0 (Грустный)	0,119	0,119	0,014

Абсолютные и квадратические ошибки могли напомнить вам о функциях ошибок, которые используются в регрессии. Однако в классификации они применяются не так широко. Самый популярный случай мы увидим дальше. Почему он широко распространен? Математика (производные) работает намного лучше со следующей функцией. Кроме того, все эти ошибки довольно малы — меньше 1 независимо от того, насколько плохо классифицирована точка. Причина в том, что разница (или квадрат разницы) между двумя числами, которые находятся в диапазоне от 0 до 1, тоже не более 1. Чтобы правильно обучать модели, нужны функции ошибок, которые принимают бóльшие значения, чем это. К счастью, для этого у нас есть третья функция ошибки.

Функция ошибки 3. Логистическая функция потерь

Логистическая функция потерь — это наиболее широко используемая функция ошибок для непрерывных персептронов. Большинство функций, описанных в этой книге, имеют в названии слово «*ошибка*», но в названии этой функции вместо него применяется слово «*потеря*». *Логистическая* часть в названии происходит от натурального логарифма, который мы используем в формуле. Однако истинной причиной логистической потери является вероятность.

Выходные сигналы непрерывного персептрона представляют собой числа от 0 до 1, поэтому их можно считать вероятностями. Модель присваивает вероятность каждой точке данных, и это вероятность того, что точка радостная. Исходя из этого, мы можем вывести вероятность того, что точка грустная. Равна она будет 1 минус вероятность того, что точка радостная. Например, если прогноз равен 0,75, это означает, что модель считает, что точка радостная с вероятностью 0,75 и грустная с вероятностью 0,25.

И вот мы приблизились к ключевому наблюдению. Цель модели состоит в том, чтобы присвоить высокие вероятности радостным точкам (с меткой 1) и низкие вероятности — грустным (с меткой 0). Обратите внимание: вероятность того, что точка грустная, равна 1 минус вероятность того, что точка радостная. Для каждой точки вычислим вероятность, которую модель дает ее метке. Для точек в нашем наборе данных соответствующие вероятности будут следующими.

- **Точка 1:**
 - метка — 0 (грустный);
 - прогноз (вероятность того, что точка радостная) — 0,953;
 - вероятность того, что это ее метка, — $1 - 0,953 = 0,047$.
- **Точка 2:**
 - метка — 1 (радостный);
 - прогноз (вероятность того, что точка радостная) — 0,731;
 - вероятность того, что это ее метка, — **0,731**.
- **Точка 3:**
 - метка — 1 (радостный);
 - прогноз (вероятность того, что точка радостная) — 0,119;
 - вероятность того, что это ее метка, — **0,119**.
- **Точка 4:**
 - метка — 0 (грустный);
 - прогноз (вероятность того, что точка радостная) — 0,119;
 - вероятность того, что это ее метка, — $1 - 0,119 = 0,881$.

Обратите внимание на то, что точки 2 и 4 хорошо классифицированы и модель присваивает высокую вероятность тому, что это их метка. Напротив, точки 1 и 3 плохо классифицированы, и модель присваивает низкую вероятность тому, что это их метка.

Логистический классификатор, в отличие от персептронного, не дает однозначных ответов. Персептрон сказал бы: «Я на 100 % уверен, что эта точка радостная», а логистический классификатор говорит: «Вероятность того, что ваша точка радостная, составляет 73 %, а того, что грустная, — 27 %». Цель персептронного классификатора состоит в том, чтобы давать верный прогноз как можно чаще, а цель логистического классификатора — в том, чтобы присвоить

каждой точке максимально возможную вероятность получения правильной метки. Классификатор присваивает четырем меткам вероятности 0,047, 0,731, 0,119 и 0,881. В идеале хотелось бы, чтобы цифры оказались выше. Как мы количественно оцениваем эти четыре числа? Одним из способов было бы сложить их или усреднить. Но поскольку это вероятности, естественным подходом будет умножение. Когда события независимы, вероятность того, что они произойдут одновременно, — это и есть произведение их вероятностей. Если мы предположим, что четыре прогноза независимы, то вероятность, которую эта модель присваивает меткам «грустный», «радостный», «радостный», «грустный», равна произведению четырех чисел, что составляет $0,047 \times 0,731 \times 0,119 \times 0,881 = 0,004$. Это очень маленькая вероятность. Будем надеяться, что модель, которая лучше соответствует набору данных, даст более высокую вероятность.

Вероятность, которую мы только что вычислили, выглядит хорошей числовой оценкой для нашей модели, но у нее есть проблемы. Например, это произведение множества малых чисел. А произведения множества небольших чисел как правило, совсем маленькие. Представьте, если бы в нашем наборе данных содержался миллион точек. Вероятность была бы результатом произведения миллиона чисел, каждое от 0 до 1. Это число окажется настолько маленьким, что компьютер даже не сможет его представить. Кроме того, манипулировать произведением миллиона чисел чрезвычайно сложно. Возможно, есть способ, который способен превратить результат в нечто, с чем гораздо легче что-то делать, например в сумму?

К счастью, удобный способ превращать произведения в суммы есть — логарифмы. В рамках этой книги все, что нужно знать о логарифме, — это то, что он превращает произведения в суммы. Если быть более точным, то логарифм произведения двух чисел представляет собой сумму логарифмов чисел:

$$\ln(ab) = \ln(a) + \ln(b).$$

Мы можем использовать логарифмы с основанием 2, 10 или e . В этой главе возьмем натуральный логарифм с основанием e . Однако те же результаты можно получить, если взять логарифм с любым другим основанием.

Если мы применим натуральный логарифм к нашему произведению вероятностей, то получим:

$$\begin{aligned} \ln(0,047 \times 0,731 \times 0,119 \times 0,881) &= \\ = \ln(0,047) + \ln(0,731) + \ln(0,119) + \ln(0,881) &= -5,616. \end{aligned}$$

Есть один нюанс. Обратите внимание на то, что результатом является отрицательное число. Это неизбежно, потому что логарифм числа от 0 до 1 всегда отрицателен. Следовательно, отрицательный логарифм произведения вероятностей — это всегда положительное число.

Логарифмическая потеря определяется как отрицательный логарифм произведения вероятностей, который также является суммой отрицательных логарифмов вероятностей. Кроме того, каждое из слагаемых представляет собой

логарифмическую потерю в конкретной точке. В табл. 6.5 приводится расчет логарифмических потерь для каждой из точек. Суммируя логарифмические потери всех точек, мы получаем общую логарифмическую потерю 5,616.

Таблица 6.5. Расчет логарифмических потерь для точек нашего набора данных

Точка	Метка	Прогнозируемая метка	Вероятность того, что метка правильная	Логарифмическая потеря
1	0 (Грустный)	0,953	0,047	$-\ln(0,047) = 3,049$
2	1 (Радостный)	0,731	0,731	$-\ln(0,731) = 0,313$
3	1 (Радостный)	0,119	0,119	$-\ln(0,119) = 2,127$
4	0 (Грустный)	0,119	0,881	$-\ln(0,881) = 0,127$

Обратите внимание на то, что хорошо классифицированные точки (2 и 4) имеют небольшую логарифмическую потерю, а плохо классифицированные (1 и 3) — большую. Причина в том, что если число x близко к 0, то $-\ln(x)$ — большое число, но если x близко к 1, то $-\ln(x)$ — малое число.

Подводя итог, можно сказать, что шаги по вычислению логарифмической потери таковы.

1. Для каждой точки вычисляем вероятность того, что классификатор даст ей ее метку:
 - для радостных точек эта вероятность и есть оценка;
 - для грустных точек вероятность равна 1 минус оценка.
2. Перемножив все эти вероятности, получаем общую вероятность, которую классификатор присвоил этим меткам.
3. К полученной общей вероятности применяем натуральный логарифм. Логарифм произведения — это сумма логарифмов множителей, поэтому мы получаем сумму логарифмов, по одному для каждой точки.
4. Мы замечаем, что все члены отрицательные, так как если число меньше 1, то логарифм будет отрицательным числом. Таким образом, мы умножаем все на -1 и получаем сумму положительных чисел. Это и есть логарифмическая потеря.

Логарифмические потери тесно связаны с концепцией *кросс-энтропии* — способом измерения сходства между двумя распределениями вероятностей. Более подробная информация о кросс-энтропии имеется в материалах, ссылки на которые приведены в приложении В.

Формула для логарифмической потери

Логарифмическую потерю для точки можно свести к красивой формуле. Напомним, что логарифмическая потеря — это отрицательный логарифм вероятности того, что точка получила правильную метку — радостную или грустную.

Прогноз, который модель дает каждой точке, равен $\hat{y}$, и это вероятность того, что точка радостная. Таким образом, вероятность того, что точка грустная (согласно модели), равна $1 - \hat{y}$. Следовательно, мы можем записать логарифмическую потерю так.

- Если метка равна 0, то логарифмическая потеря = $-\ln(1 - \hat{y})$.
- Если метка равна 1, то логарифмическая потеря = $-\ln(\hat{y})$.

Поскольку метка имеет значение y , предшествующее выражение «если» может быть сведено к следующей формуле:

$$\text{логарифмическая потеря} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y}).$$

Эта формула работает так: если метка равна 0, то первое слагаемое равно 0, а если метка равна 1, то второе слагаемое равно 0. Мы используем термин «логарифмическая потеря», когда говорим о логарифмической потере точки или всего набора данных. Логарифмические потери набора данных — это сумма логарифмических потерь в каждой точке.

Сравнение классификаторов с помощью логарифмических потерь

Теперь, когда мы разобрались с функцией ошибок для логистических классификаторов — логарифмической потерей, можем использовать ее для сравнения двух классификаторов. Напомним, что классификатор, который применялся в этой главе, определяется следующими весами и смещением.

Логистический классификатор 1:

- вес «аак» — $a = 1$;
- вес «бип» — $b = 2$;
- смещение — $c = -4$.

В этом разделе мы сравним его со следующим логистическим классификатором.

Логистический классификатор 2:

- вес «аак» — $a = -1$;
- вес «бип» — $b = 1$;
- смещение — $c = 0$.

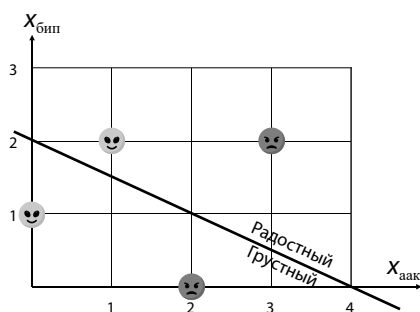
Прогнозы, которые делает каждый классификатор:

- **классификатор 1** — $\hat{y} = \sigma(x_{\text{аак}} + 2x_{\text{бип}} - 4)$;
- **классификатор 2** — $\hat{y} = \sigma(-x_{\text{аак}} + x_{\text{бип}})$.

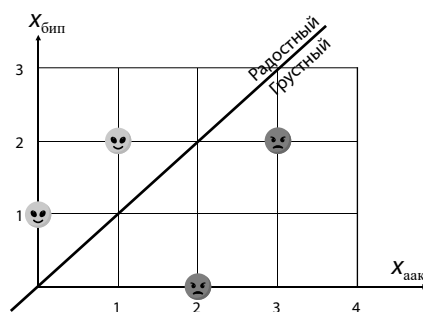
Прогнозы обоих классификаторов записаны в табл. 6.6, а график набора данных и две граничные прямые показаны на рис. 6.5.

Таблица 6.6. Расчет логарифмических потерь для точек в нашем наборе данных

Точка	Метка	Прогноз	
		классификатора 1	классификатора 2
1	0 (Грустный)	0,953	0,269
2	1 (Радостный)	0,731	0,731
3	1 (Радостный)	0,119	0,731
4	0 (Грустный)	0,881	0,119



Классификатор 1



Классификатор 2

Рис. 6.5. Слева — плохой классификатор, который допускает две ошибки. Справа — хороший классификатор, который правильно классифицирует все четыре точки

Обратите внимание на то, что прогнозы, сделанные классификатором 2, намного ближе к меткам точек, чем сделанные классификатором 1. Из результатов, приведенных в табл. 6.6 и на рис. 6.5, ясно, что классификатор 2 намного лучше, чем классификатор 1.

Например, на рис. 6.5 мы можем видеть, что классификатор 2 правильно расположил два радостных предложения в положительной зоне, а два грустных — в отрицательной. Далее сравниваем логарифмические потери. Напомним, что логарифмическая потеря для классификатора 1 составила 5,616. Мы должны получить меньшую логарифмическую потерю для классификатора 2, потому что он должен работать лучше.

Согласно формуле, логарифмическая потеря $= -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$, логарифмические потери для классификатора 2 в каждой из точек набора данных следующие:

- **точка 1** ($y = 0$, $\hat{y} = 0,269$):

$$\text{логарифмическая потеря} = \ln(1 - 0,269) = 0,313;$$

- **точка 2** ($y = 1, \hat{y} = 0,731$):

логарифмическая потеря = $\ln(0,731) = 0,313$;

- **точка 3** ($y = 1, \hat{y} = 0,731$):

логарифмическая потеря = $\ln(0,731) = 0,313$;

- **точка 4** ($y = 0, \hat{y} = 0,119$):

логарифмическая потеря = $\ln(1 - 0,119) = 0,127$.

Общая логарифмическая потеря для набора данных равна сумме четырех потерь, что составляет 1,067. Обращаю ваше внимание на то, что она намного меньше, чем 5,616. Это служит подтверждением того, что классификатор 2 действительно намного лучше.

Как найти хороший логистический классификатор?

Алгоритм логистической регрессии

В этом разделе мы узнаем, как обучить логистический классификатор. Этот процесс аналогичен процессу обучения модели линейной регрессии или персептронного классификатора и состоит из следующих шагов.

1. Начать со случайного логистического классификатора.
2. Многократно понемногу улучшать классификатор.
3. Измерить логистическую потерю и решить, когда прекратить выполнение цикла.

Ключом к алгоритму является шаг внутри цикла, который состоит из небольшого улучшения логистического классификатора. На этом шаге используется метод, называемый *логистическим методом*. Как вы увидите в следующем разделе, логистический метод похож на метод обучения персептрона.

Логистический метод — способ немного улучшить непрерывный персептрон

Напомним из главы 5, что метод обучения персептрона состоит в том, чтобы начать со случайного классификатора, последовательно выбрать случайную точку и применить метод обучения персептрона. В нем у нас были два сценария:

- **сценарий 1** — если точка классифицирована правильно, оставить прямую на месте;
- **сценарий 2** — если точка классифицирована неправильно, переместить прямую немного ближе к ней.

Логистический метод, проиллюстрированный на рис. 6.6, аналогичен методу обучения персептрона. Единственное отличие — когда точка хорошо классифицирована, мы отодвигаем прямую от нее. В методе тоже два сценария:

- **сценарий 1** — если точка классифицирована правильно, слегка отодвинуть прямую от нее;
- **сценарий 2** — если точка классифицирована неправильно, слегка переместить прямую ближе к ней.

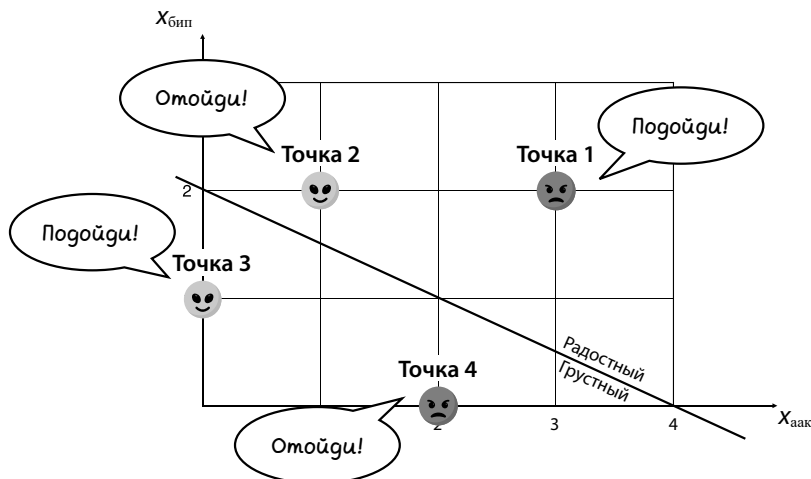


Рис. 6.6. Алгоритм логистической регрессии

Но зачем нам удалять прямую от правильно классифицированной точки? Если точка правильно классифицирована, значит, она находится в правильной зоне относительно прямой. Отодвинув линию, мы переместим точку еще глубже в правильную зону. Поскольку прогноз основан на том, как далеко точка находится от границы, для точек в положительной (радостной) зоне прогноз увеличивается, если они находятся дальше от линии. Аналогично для точек в отрицательной (грустной) зоне прогноз уменьшается, если они располагаются дальше от прямой. Таким образом, если метка точки равна 1, мы увеличиваем прогноз, смещая его еще ближе к 1, а если метка точки равна 0, уменьшаем прогноз, смещая его ближе к 0.

Например, посмотрите на классификатор 1 и первое предложение в нашем наборе данных. Напомним, что классификатор имеет веса $a = 1$, $b = 2$ и смещение $c = -4$. Предложение соответствует точке с координатами $(x_{\text{акк}}, x_{\text{бипп}}) = (3, 2)$, а метка $y = 0$. Прогноз, который мы получили для этой точки, был $\hat{y} = \sigma(3 + 2 \times 2 - 4) = \sigma(3) = 0,953$. Прогноз довольно далек от метки, поэтому ошибка значительная. По факту в табл. 6.5 мы рассчитали, что она составляет 3,049. Ошибка, допущенная этим классификатором, заключалась в том, что он посчитал, что

предложение радостнее, чем оно есть. Таким образом, чтобы настроить веса так, чтобы классификатор уменьшал прогноз для этого предложения, мы должны существенно уменьшить веса a , b и смещение c .

Используя ту же логику, мы можем проанализировать, как настроить веса для улучшения классификации других точек. Для второго предложения в наборе данных метка равна $y = 1$, а прогноз — $0,731$. Это хороший прогноз, но, если нужно его улучшить, следует немного увеличить веса и смещение. Для третьего предложения, поскольку метка равна $y = 1$, а прогноз $\hat{y} = 0,119$, мы должны существенно увеличить веса и смещение. Наконец, для четвертого предложения метка равна $y = 0$, а прогноз $\hat{y} = 0,119$, поэтому требуется немного уменьшить веса и смещение. Все это обобщено в табл. 6.7. Обратите внимание на то, что хорошо классифицированные точки (2 и 4) имеют небольшие логарифмические потери, а плохо классифицированные (1 и 3) — значительные.

Таблица 6.7. Расчет логарифмических потерь для точек в нашем наборе данных

Точка	Метка y	Прогноз $\hat{y}$ классификатора 1	Как настроить веса a , b и смещение c	$y - \hat{y}$
1	0	0,953	Уменьшить на большую величину	-0,953
2	1	0,731	Увеличить на небольшую величину	0,269
3	1	0,119	Увеличить на большую величину	0,881
4	0	0,119	Уменьшить на небольшую величину	-0,119

Следующие наблюдения помогут определить идеальную сумму, которую мы хотим добавить к весам и смещению, чтобы улучшить прогнозы.

- **Наблюдение 1.** Последний столбец табл. 6.7 содержит значение метки за вычетом прогноза. Обратите внимание на сходство между двумя крайними правыми столбцами в таблице. Это намек на то, что сумма, на которую мы должны обновить веса и смещение, должна быть кратна $y - \hat{y}$.
- **Наблюдение 2.** Представьте себе предложение, в котором слово «аак» встречается десять раз, а «бип» — один раз. Если мы хотим добавить значение к весам этих двух слов (или вычесть из них), разумнее обновить вес «аак» на большую величину, потому что это слово важнее для общей оценки предложения. Таким образом, сумма, на которую следует обновить вес «аак», должна быть умножена на $x_{\text{аак}}$, а сумма, на которую нужно обновить вес «бип», — умножена на $x_{\text{бип}}$.
- **Наблюдение 3.** Число, на которое мы обновляем веса и смещение, должно быть умножено также на скорость обучения η , потому что нам требуется, чтобы это число было небольшим.

Объединив три наблюдения, приходим к выводу, что следующий набор обновленных весов достаточно хорош:

$$a' = a + \eta(y - \hat{y})x_1;$$

$$b' = b + \eta(y - \hat{y})x_2;$$

$$c' = c + \eta(y - \hat{y}).$$

Далее следует псевдокод логистического метода. Обратите внимание на то, насколько он похож на псевдокод для метода обучения персептрона, который мы изучили в конце раздела «Метод персептрона — способ немного улучшить персептрон» в главе 5.

Псевдокод для логистического метода

Входные данные:

- логистический классификатор с весами a , b и смещением c ;
- точка с координатами (x_1, x_2) и меткой y ;
- небольшое значение η (скорость обучения).

Результат — персептрон с новыми весами a' , b' и смещением c' , который по крайней мере так же хорош, как и входной персептрон для этой точки

Процедура — прогноз, который персептрон делает в этой точке, равен $\hat{y} = \sigma(ax_1 + bx_2 + c)$.

Возврат — персептрон со следующими весами и смещением:

- $a' = a + \eta(y - \hat{y})x_1$;
- $b' = b + \eta(y - \hat{y})x_2$;
- $c' = c + \eta(y - \hat{y})$.

То, как мы обновляли веса и смещения в логистическом методе, — это не случайность, а результат применения алгоритма градиентного спуска для уменьшения логарифмической потери. Математические подробности описаны в приложении Б, в разделе «Использование градиентного спуска для обучения моделей классификации».

Чтобы убедиться в том, что в нашем случае логистический метод работает, применим его к текущему набору данных. Фактически мы применим его к каждой из четырех точек по отдельности, чтобы увидеть, насколько каждая из них изменит вес и смещение модели. Наконец, сравним логарифмическую потерю в момент до и после обновления и убедимся, что она действительно уменьшилась. Для следующих расчетов используем скорость обучения $\eta = 0,05$.

Обновление классификатора с помощью каждого из предложений

Первое предложение:

- начальные веса и смещение: $a = 1$, $b = 2$, $c = -4$;
- метка: $y = 0$;

- прогноз: 0,953;
- начальная логарифмическая потеря: $-0 \ln(0,953) - 1 \ln(1 - 0,953) = 3,049$;
- координаты точки: $x_{\text{аак}} = 3, x_{\text{бип}} = 2$;
- скорость обучения: $\eta = 0,01$;
- обновленные веса и смещения:
 - $a' = 1 + 0,05 \times (0 - 0,953) \times 3 = 0,857$;
 - $b' = 2 + 0,05 \times (0 - 0,953) \times 2 = 1,905$;
 - $c' = -4 + 0,05 \times (0 - 0,953) = -4,048$;
- обновленный прогноз: $\hat{y} = \sigma(0,857 \times 3 + 1,905 \times 2 - 4,048) = 0,912$
(обратите внимание на то, что прогноз уменьшился и теперь он ближе к метке 0);
- итоговая логарифмическая потеря: $-0 \ln(0,912) - 1 \ln(1 - 0,912) = 2,426$
(обратите внимание на то, что ошибка уменьшилась с 3,049 до 2,426).

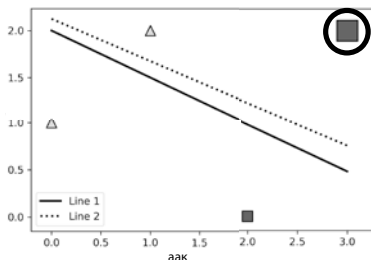
Расчеты для остальных трех точек приведены в табл. 6.8. Обратите внимание: здесь обновленный прогноз всегда ближе к метке, чем исходный, а конечная логарифмическая потеря всегда меньше, чем начальная. Это означает, что независимо от того, какая точка использована для логистического метода, мы будем улучшать модель для нее и уменьшать конечную логарифмическую потерю.

Таблица 6.8. Расчеты прогнозов, логарифмическая потеря, обновленные веса и обновленные прогнозы для всех точек

Точка	Координаты	Метка	Первичный прогноз	Первичная логарифмическая потеря	Обновленные веса	Обновленный прогноз	Окончательная логарифмическая потеря
1	(3, 2)	0	0,953	3,049	$a' = 0,857$ $b' = 1,905$ $c' = -4,048$	0,912	2,426
2	(1, 2)	1	0,731	0,313	$a' = 1,013$ $b' = 2,027$ $c' = -3,987$	0,747	0,292
3	(0, 1)	1	0,119	2,127	$a' = 1$ $b' = 2,044$ $c' = -3,956$	0,129	2,050
4	(2, 0)	0	0,119	0,127	$a' = 0,988$ $b' = 2$ $c' = -4,006$	0,127	0,123

В начале раздела мы увидели, что логистический метод может быть визуализирован геометрически так же, как перемещение пограничной прямой относительно точки. А если точнее, то прямая приближается к точке, если та классифицирована неправильно, и отдаляется от нее, если правильно. Мы можем убедиться в этом, построив график исходного и модифицированного

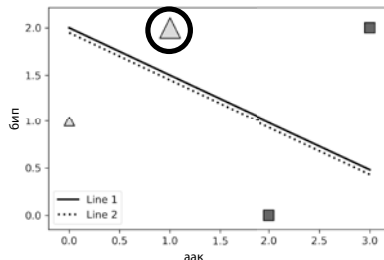
классификатора в четырех случаях, приведенных в табл. 6.8. На рис. 6.7 четыре графика. На каждом из них сплошная линия — это исходный классификатор, а пунктирная — классификатор, полученный применением логистического метода с использованием выделенной точки. Обратите внимание на то, что точки 2 и 4, классифицированные правильно, отодвигают прямую, а точки 1 и 3, классифицированные неправильно, приближают прямую к себе.



Точка: (3, 2) (классифицирована неправильно)

Прямая 1: $1x_{\text{аак}} + 2x_{\text{бип}} - 4 = 0$

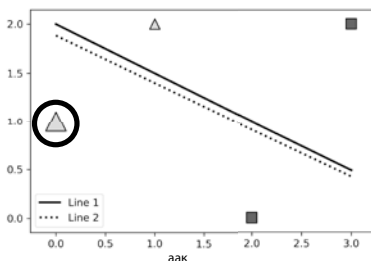
Прямая 2: $0,857x_{\text{аак}} + 1,905x_{\text{бип}} - 4,048 = 0$



Точка: (1, 2) (классифицирована правильно)

Прямая 1: $1x_{\text{аак}} + 2x_{\text{бип}} - 4 = 0$

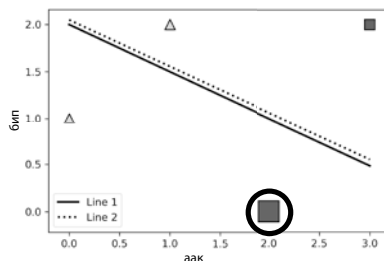
Прямая 2: $1,013x_{\text{аак}} + 2,027x_{\text{бип}} - 3,987 = 0$



Точка: (0, 1) (классифицирована неправильно)

Прямая 1: $1x_{\text{аак}} + 2x_{\text{бип}} - 4 = 0$

Прямая 2: $1x_{\text{аак}} + 2,044x_{\text{бип}} - 3,956 = 0$



Точка: (2, 0) (классифицирована правильно)

Прямая 1: $1x_{\text{аак}} + 2x_{\text{бип}} - 4 = 0$

Прямая 2: $0,988x_{\text{аак}} + 2x_{\text{бип}} - 4,006 = 0$

Рис. 6.7. Логистический метод, примененный к четырем точкам данных

Многократное повторение логистического метода — алгоритм логистической регрессии

Алгоритм логистической регрессии — это то, что мы используем для обучения логистического классификатора. Точно так же как алгоритм обучения персептрона состоит из многократного повторения метода обучения персептрона, алгоритм логистической регрессии состоит из многократного повторения логистического метода.

Псевдокод для алгоритма логистической регрессии

Входные данные:

- набор данных точек, помеченных как 1 и 0;
- количество периодов n ;
- скорость обучения η .

Результат — логистический классификатор, состоящий из набора весов и смещения, который соответствует набору данных.

Процедура

- Начать со случайных значений для весов и смещения логистического классификатора.
- Повторять многократно:
 - выбрать случайную точку данных;
 - обновить веса и смещение, используя логистический метод.

Возврат — персептронный классификатор с обновленными весами и смещением.

Как мы уже видели, каждая итерация логистического метода либо приближает прямую к неправильно классифицированной точке, либо отдаляет ее от правильно классифицированной.

Стохастический, мини-пакетный и пакетный градиентный спуск

Алгоритм логистической регрессии, как и линейная регрессия и персептрон, — это алгоритм, основанный на градиентном спуске. Если мы используем градиентный спуск для уменьшения логарифмических потерь, шаг градиентного спуска становится логистическим методом.

Общий алгоритм логистической регрессии работает для наборов данных не только с двумя признаками, но и с любым количеством признаков. В этом случае, как и в алгоритме персептрона, граница не будет прямой. Она будет выглядеть как гиперплоскость более высокой размерности, разделяющая точки в пространстве с большим числом измерений. Однако нам не требуется визуализировать многомерное пространство. Все, что мы хотим, — построить классификатор логистической регрессии с таким количеством весов, сколько признаков в наших данных. Логистический метод и логистический алгоритм обновляют веса аналогично тому, что мы делали в предыдущих разделах.

Как и в ранее изученных алгоритмах, на практике мы не обновляем модель, выбирая одну точку за раз. Вместо этого используем мини-пакетный градиентный спуск — берем пакет точек и обновляем модель, чтобы она лучше соответствовала этим точкам. Для получения наиболее общего алгоритма логистической регрессии и подробного математического вывода логистического метода с помощью градиентного спуска, пожалуйста, обратитесь к приложению Б (раздел «Использование градиентного спуска для обучения моделей классификации»).

Кодирование алгоритма логистической регрессии

В этом разделе мы узнаем, как вручную закодировать алгоритм логистической регрессии. Код для этого раздела приведен далее.

- **Блокнот** Coding_logistic_regression.ipynb:
https://github.com/luisguiserrano/manning/blob/master/Chapter_6_Logistic_Regression/Coding_logistic_regression.ipynb.

Мы протестируем код на том же наборе данных, который применяли в главе 5. Набор данных приведен в табл. 6.9.

Таблица 6.9. Набор данных, который будем использовать с логистическим классификатором

Аак x_1	Бип x_2	Метка y
1	0	0
0	2	0
1	1	0
1	2	0
1	3	1
2	2	1
2	3	1
3	2	1

Далее приведен код для загрузки нашего небольшого набора данных, а его график показан на рис. 6.8:

```
import numpy as np
features = np.array([[1,0],[0,2],[1,1],[1,2],[1,3],[2,2],[2,3],[3,2]])
labels = np.array([0,0,0,0,1,1,1,1])
```

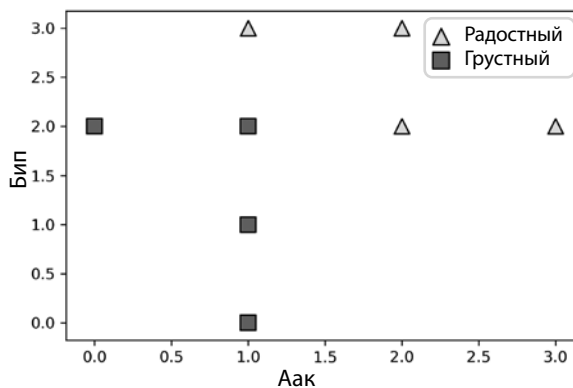


Рис. 6.8. График набора данных, где радостные предложения представлены треугольниками, а грустные — квадратами

Кодирование алгоритма логистической регрессии вручную

В этом разделе мы увидим, как вручную закодировать логистический метод и алгоритм логистической регрессии. Обобщая, будем кодировать алгоритм логистической регрессии для набора данных с n весами. Используемые обозначения:

- признаки — $x_1, x_2 \dots x_n$;
- метка — y ;
- веса — $w_1, w_2 \dots w_n$;
- смещение — b .

Оценка для конкретного предложения представляет собой сигмоидную функцию суммы весов каждого слова (w_i), умноженной на количество появлений (x_i), плюс смещение (b). Обратите внимание на то, что мы используем обозначение суммирования для

$$\sum_{i=1}^n a_i = a_1 + a_2 + \dots + a_n.$$

$$\text{Прогноз: } \hat{y} = \sigma(w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b) = \sigma\left(\sum_{i=1}^n w_i x_i + b\right).$$

Рассматривая проблему, мы будем ссылаться на $x_{\text{аак}}$ и $x_{\text{бип}}$ как на x_1 и x_2 соответственно. Их веса равны w_1 и w_2 , а смещение — b .

Начинаем с кодирования сигмоидной функции, оценки и прогноза. Напомним, что формула сигмоидной функции выглядит так:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

```
def sigmoid(x):
    return np.exp(x)/(1+np.exp(x))
```

Для функции оценки мы используем скалярное произведение между признаками и весами. Напомним, что скалярное произведение векторов $(x_1, x_2 \dots x_n)$ и $(w_1, w_2 \dots w_n)$ равно $w_1 x_1 + w_2 x_2 + \dots + w_n x_n$.

```
def score(weights, bias, features):
    return np.dot(weights, features) + bias
```

Наконец, вспомним, что прогноз — это сигмоидная функция активации, применяемая к оценке:

```
def prediction(weights, bias, features):
    return sigmoid(score(weights, bias, features))
```

Теперь, когда у нас есть прогноз, можем перейти к логарифмической потере. Вспомним формулу логарифмической потери:

$$\text{логарифмическая потеря} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y}).$$

Закодируем ее следующим образом:

```
def log_loss(weights, bias, features, label):
    pred = prediction(weights, bias, features)
    return -label*np.log(pred) - (1-label)*np.log(1-pred)
```

Нам нужна логарифмическая потеря по всему набору данных, поэтому сложим их для всех точек данных, как показано далее:

```
def total_log_loss(weights, bias, features, labels):
    total_error = 0
    for i in range(len(features)):
        total_error += log_loss(weights, bias, features[i], labels[i])
    return total_error
```

Теперь мы готовы к кодированию метода логистической регрессии и алгоритма логистической регрессии. Вспомним, что для более чем двух переменных шаг логистической регрессии для i -го веса представляет собой следующую формулу, где η — скорость обучения:

$$w_i \rightarrow w_i + \eta(y - \hat{y})x_i, \text{ где } i = 1, 2 \dots n;$$

$$b \rightarrow b + \eta(y - \hat{y}), \text{ где } i = 1, 2 \dots n.$$

```
def logistic_trick(weights, bias, features, label, learning_rate = 0.01):
    pred = prediction(weights, bias, features)
    for i in range(len(weights)):
        weights[i] += (label-pred)*features[i]*learning_rate
        bias += (label-pred)*learning_rate
    return weights, bias
```

```
def logistic_regression_algorithm(features, labels, learning_rate = 0.01,
    epochs = 1000):
    utils.plot_points(features, labels)
    weights = [1.0 for i in range(len(features[0]))]
    bias = 0.0
    errors = []
    for i in range(epochs):
        errors.append(total_log_loss(weights, bias, features, labels))
        j = random.randint(0, len(features)-1)
        weights, bias = logistic_trick(weights, bias, features[j],
            labels[j])
    return weights, bias
```

Теперь мы можем запустить алгоритм логистической регрессии для построения логистического классификатора, который соответствует нашему набору данных:

```
logistic_regression_algorithm(features, labels)
([0.46999999999999953, 0.09999999999999937], -0.6800000000000004)
```

Полученный нами классификатор имеет следующие веса и смещение:

- $w_1 = 0,47$;
- $w_2 = 0,1$;
- $b = -0,68$.

График классификатора (вместе с графиком предыдущих классификаторов в каждый из периодов) показан на рис. 6.9.

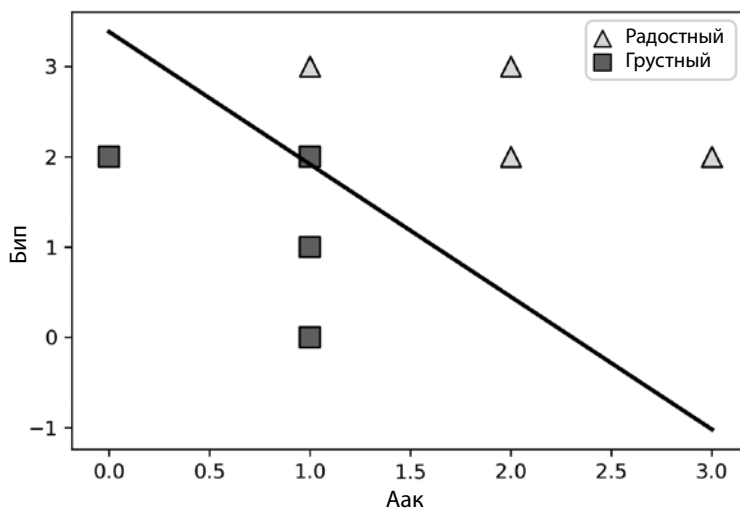


Рис. 6.9. Граница результирующего логистического классификатора

На рис. 6.10 мы видим график классификаторов, соответствующих всем периодам (*слева*), и график логарифмических потерь (*справа*). На графике промежуточных классификаторов (см. рис. 6.10, *слева*) конечный классификатор соответствует жирной линии. Обратите внимание на график логарифмических потерь. По мере запуска алгоритма для большего количества периодов логарифмическая потеря резко уменьшается, а это именно то, чего мы добиваемся. Кроме того, логарифмическая ошибка никогда не равна нулю, даже если все точки классифицированы правильно. Это связано с тем, что для любой точки независимо от того, насколько хорошо она классифицирована, логарифмическая ошибка никогда не равна нулю. Сравним это с рис. 5.26, где потери персептрона достигают нулевого значения, когда каждая точка классифицирована правильно.

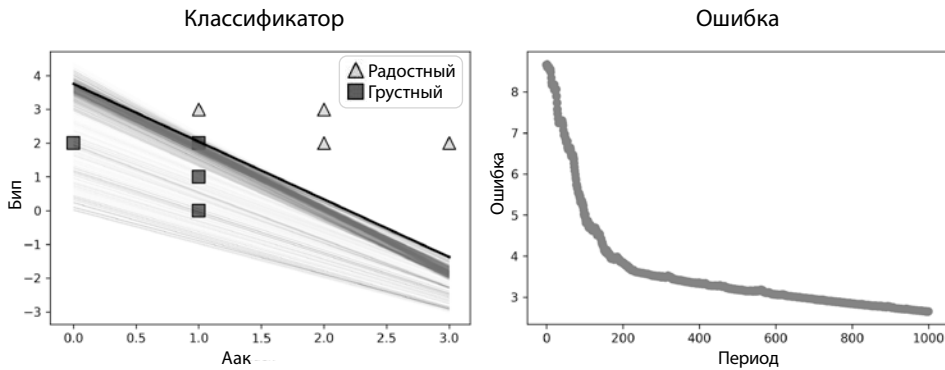


Рис. 6.10. Графики: *слева* — всех промежуточных шагов алгоритма логистической регрессии; *справа* — ошибок

Реальное применение. Классификация обзоров IMDB с помощью Turi Create

В данном разделе мы увидим реальное применение логистического классификатора при анализе настроений. Задействуем Turi Create для построения модели, которая анализирует обзоры фильмов на популярном сайте IMDB. Код для этого раздела приведен далее.

- **Блокнот** `Sentiment_analysis_IMDB.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_6_Logistic_Regression/Sentiment_analysis_IMDB.ipynb
- **Набор данных** `IMDB_Dataset.csv`.

Сначала импортируем Turi Create, загрузим набор данных и преобразуем его в SFrame, который назовем `movies`:

```
import turicreate as tc
movies = tc.SFrame('IMDB Dataset.csv')
```

Первые пять строк набора данных приведены в табл. 6.10. В графе **Отзыв** содержится текст отзыва, а в графе **Настроение** — настроение отзыва, которое может быть положительным или отрицательным.

Таблица 6.10. Первые пять строк набора данных IMDB

Отзыв	Настроение
One of the other reviewers has mentioned... (Один из других рецензентов отметил...)	Положительный
A wonderful little production... (Замечательная постановка...)	Положительный
I thought this was a wonderful day to spend... (Я подумал, сегодня прекрасный день для того, чтобы провести...)	Положительный
Basically, there's a family where a little... (В общем, есть семья, где маленький...)	Отрицательный
Petter Mattei's "Love in the time of money" is a... («Любовь во времена денег» Питера Маттеи — это...)	Положительный

Набор данных состоит из двух граф: одна с обзором в виде строки, вторая с настроением — положительным или отрицательным. В первую очередь обрабатываем строку, потому что каждое слово должно стать отдельным признаком. Для этой задачи пригодится встроенная функция `count_words` Turi Create в пакете `text_analytics`, поскольку она превращает предложение в словарь с подсчетом слов. Например, предложение «быть или не быть» превращается в словарь {'быть':2, 'или':1, 'не':1}. Затем добавим еще одну графу под названием `words`, содержащую этот словарь:

```
movies['words'] = tc.text_analytics.count_words(movies['review'])
```

В табл. 6.11 показаны первые несколько строк набора данных с новой графой. В ней размещены признаки для нашего логистического классификатора.

Таблица 6.11. Графа «Слова» представляет собой словарь, в который записывается каждое слово в обзоре и количество его появлений

Отзыв	Настроение	Слова
One of the other reviewers has mentioned...	Положительный	{'if': 1.0, 'viewing': 1.0, 'comfortable': 1.0, ...}
A wonderful little production...	Положительный	{'done': 1.0, 'every': 1.0, 'decorating': 1.0, ...}
I thought this was a wonderful day to spend...	Положительный	{'see': 1.0, 'go': 1.0, 'great': 1.0, 'superm ...}
Basically, there's a family where a little...	Отрицательный	{'them': 1.0, 'ignore': 1.0, 'dialogs': 1.0, ...}
Peter Mattei's "Love in the Time of Money" is a...	Положительный	{'work': 1.0, 'his': 1.0, 'for': 1.0, 'anxiously': ...}

Мы готовы обучать свою модель! Для этого используем функцию `create` в пакете `logistic_classifier`, в которой целью (меткой) будет графа `sentiment`, а признаками — графа `words`. Обратите внимание на то, что цель выражена в виде строки с именем графы, содержащей метку, а признаки имеют вид массива строк с именами граф, содержащих каждый из признаков (если нужно указать несколько граф):

```
model = tc.logistic_classifier.create(movies, features=['words'],
target='sentiment')
```

Теперь, когда мы обучили модель, можем посмотреть на веса слов с помощью команды `coefficients`. В полученной таблице несколько граф, но нас интересуют `index` и `value`, которые показывают слова и их веса. Первая пятерка:

- (intercept) — 0,065;
- if — -0,018;
- viewing — 0,089;
- comfortable — 0,517;
- become — 0,106.

Первая названная графа `intercept` — это смещение. Поскольку смещение модели положительное, то и пустой обзор будет положительным, как мы узнали в главе 5 в разделе «Смещение, у-перехват и настраивание инопланетянина-молчуна». Это имеет смысл, потому что пользователи, которые оценивают фильмы негативно, как правило, оставляют отзывы. А многие из тех, кто оценивает фильмы положительно, не оставляют никаких отзывов. Остальные слова нейтральны, поэтому их вес не имеет большого значения. Рассмотрим веса некоторых слов, таких как *wonderful*, *horrible* и *the*:

- wonderful — 1,043;
- horrible — -1,075;
- the — 0,0005.

Как видите, вес слова *wonderful* положительный, вес слова *horrible* отрицательный, а вес слова *the* невелик. Это разумно: *wonderful* — это позитивное слово, *horrible* — негативное, а *the* — нейтральное.

В качестве последнего шага найдем самые положительные и отрицательные отзывы. Для этого используем модель составления прогнозов для всех фильмов. Эти прогнозы будут сохранены в новой графе под названием `predictions` с помощью следующей команды:

```
movies['prediction'] = model.predict(movies, output_type='probability')
```

Давайте найдем самые позитивные и самые негативные фильмы согласно модели. Для этого отсортируем массив следующим образом.

- **Самый положительный отзыв:**

```
movies.sort('predictions')[-1]
```

Вывод: It seems to me that a lot of people don't know that *Blade* is actually a superhero movie on par with *X-Men*... (Мне кажется, многие не знают, что «Блэйд» — это фактически супергеройский фильм наравне с «Людьми Икс»...)

- **Самый негативный отзыв:**

```
movies.sort('predictions')[0]
```

Вывод: Even duller, if possible, than the original... (Гораздо более скучный, чем оригинал...)

Эту модель можно значительно улучшить. Например, некоторые методы обработки текста, такие как удаление знаков препинания, прописных букв или стоп-слов, таких как *the*, *and*, *of*, *it*, как правило, дают лучшие результаты. Но приятно знать, что с помощью пары строк кода мы можем создать собственный классификатор анализа настроений!

Классификация по нескольким классам. Логистическая функция со многими переменными

До сих пор мы видели, как непрерывные перцептроны классифицируют два класса: радостный и грустный. Но что если классов больше? В конце главы 5 мы упоминали, что дискретному перцептрону сложно классифицировать более чем два класса. Однако такая задача по плечу логистическому классификатору.

Представьте набор данных изображений с тремя метками: «собака», «кошка» и «птица». Способ построения классификатора, который прогнозирует одну из этих трех меток для каждого изображения, заключается в создании трех классификаторов, по одному для каждой метки. Когда поступает новое изображение, мы оцениваем его с помощью каждого из трех классификаторов. Классификатор, соответствующий определенному животному, возвращает вероятность того, что изображение является соответствующим животным. Затем мы классифицируем изображение как животное из классификатора, который вернул наибольшую вероятность.

Однако это не идеальный способ, потому что классификатор возвращает дискретный ответ, такой как «собака», «кошка» или «птица». А если нам нужен классификатор, который возвращает вероятности для трех животных? Скажем, ответ может иметь вид «10 % собака, 85 % кошка и 5 % птица». Здесь на помощь приходит логистическая функция со многими переменными.

Она работает следующим образом. Для начала вспомним, что логистический классификатор делает прогноз, используя двухэтапный процесс: сначала вычисляет оценку, а затем применяет к ней сигмоидную функцию. Давайте забудем о сигмоидной функции и вместо этого выведем результат. Предположим, что три классификатора вернули следующие оценки:

- классификатор собаки: 3;
- классификатор кошки: 2;
- классификатор птицы: -1 .

Как превратить эти оценки в вероятности? Вот вам хорошая идея: нормализовать. Это означает деление всех этих чисел на их сумму, равную 5, чтобы они в итоге сложились в единицу. Делая это, мы получаем вероятности $3/5$ для собаки, $2/5$ для кошки и $-1/5$ для птицы. Это работает, но не идеально, потому что вероятность того, что изображение окажется птицей, оказалась отрицательной. Но вероятности всегда должны быть положительными, поэтому нам требуется что-то другое.

Нам требуется функция, которая всегда положительна и увеличивается. Экспоненциальные функции отлично подойдут. Любая экспоненциальная функция, такая как 2^x , 3^x или 10^x , справится с подобной задачей. По умолчанию мы используем функцию e^x , которая обладает замечательными математическими свойствами (например, производной от e^x также является e^x). Применив ее к оценкам, получим следующие значения:

- классификатор собаки: $e^3 = 20,085$;
- классификатор кошки: $e^2 = 7,389$;
- классификатор птицы: $e^{-1} = 0,368$.

Теперь мы сделаем то, что делали раньше, — нормализуем или разделим каждое на сумму этих чисел, чтобы они сложились в единицу. Сумма составит $20,085 + 7,389 + 0,368 = 27,842$, таким образом, получаем:

- вероятность собаки: $20,085/27,842 = 0,721$;
- вероятность кошки: $7,389/27,842 = 0,265$;
- вероятность птицы: $0,368/27,842 = 0,013$.

Это три вероятности, выданные нашими тремя классификаторами. Функция, которую мы использовали, — это логистическая функция со многими переменными, и в общем виде она выглядит так: если у нас есть n классификаторов, которые возвращают n оценок $a_1, a_2, \dots, a_n$, то полученные вероятности равны $p_1, p_2, \dots, p_n$, где

$$p_i = \frac{e^{a_i}}{e^{a_1} + e^{a_2} + \dots + e^{a_n}}.$$

Эта формула известна как логистическая функция со многими переменными.

Что произойдет, если мы используем эту функцию только для двух классов? Получим сигмоидную функцию. Почему бы не убедиться в этом в качестве тренировки?

Резюме

- Непрерывные персептроны, или логистические классификаторы, похожи на персептронные классификаторы, за исключением того, что вместо дискретного прогноза (0 или 1) они прогнозируют любое число от 0 до 1.
- Логистические классификаторы более полезны, чем дискретные персептроны, потому что они дают больше информации. Помимо того что они сообщают, какой класс прогнозирует классификатор, они дают нам и вероятность. Хороший логистический классификатор присваивает низкие вероятности точкам с меткой 0 и высокие — точкам с меткой 1.
- Логарифмическая потеря — это функция ошибки для логистических классификаторов. Она рассчитывается отдельно для каждой точки как отрицательное значение натурального логарифма вероятности, которую классификатор присваивает ее метке.
- Общая логарифмическая потеря классификатора в наборе данных — это сумма логарифмических потерь в каждой точке.
- Логистический метод использует помеченную точку данных и пограничную прямую. Если точка классифицирована неправильно, прямая перемещается к точке, а если правильно, то от нее. Это практичнее, чем метод обучения персептрона, потому что последний не перемещает прямую, если точка классифицирована правильно.
- Алгоритм логистической регрессии используется для подгонки логистического классификатора к помеченному набору данных. Он состоит в том, чтобы начать с логистического классификатора со случайными весами и, непрерывно выбирая случайную точку, применять логистический метод с целью получить чуть лучший классификатор.
- Когда есть несколько классов для прогнозирования, можно построить несколько линейных классификаторов и объединить их с помощью логистической функции со многими переменными.

Упражнения

Упражнение 6.1

Стоматолог обучил логистический классификатор на наборе данных пациентов, чтобы спрогнозировать, есть ли у них больные зубы. Модель определила, что вероятность наличия больного зуба у пациента равна

$$\sigma(d + 0,5c - 0,8),$$

где d — переменная, показывающая, был ли у пациента в прошлом другой больной зуб, c — переменная, указывающая, любит ли он сладости.

Например, если пациент любит конфеты, то $c = 1$, а если нет, то $c = 0$. Какова вероятность того, что у пациента, который ест конфеты и лечил больной зуб в прошлом году, имеется больной зуб?

Упражнение 6.2

Рассмотрим логистический классификатор, который присваивает точке (x_1, x_2) прогноз $\hat{y} = \sigma(2x_1 + 3x_2 - 4)$, и точку $p = (1, 1)$ с меткой 0.

- А. Вычислите прогноз $\hat{y}$, который модель даст точке p .
- Б. Вычислите логарифмическую потерю, которую модель производит в точке p .
- В. Примените логистический метод, чтобы получить новую модель, которая приводит к меньшей логарифмической потере. Вы можете использовать значение скорости обучения $\eta = 0,1$.
- Г. Найдите прогноз, данный новой моделью в точке p , и убедитесь, что полученная логарифмическая потеря меньше исходной.

Упражнение 6.3

Используя модель, описанную в упражнении 6.2, найдите точку, для которой прогноз равен 0,8.

ПОДСКАЗКА Сначала найдите оценку, которая даст прогноз 0,8, и вспомните, что прогноз равен $\hat{y} = \sigma(\text{счет})$.

Как оценивать классификационные модели? Достоверность и ее друзья



В этой главе

- ✓ Типы ошибок, которые может допускать модель: ложноположительные и ложноотрицательные результаты.
 - ✓ Сведение этих ошибок в таблицу: матрица неточностей.
 - ✓ Что такое достоверность, полнота, точность, F-мера, чувствительность и избирательность и как они используются для оценки моделей.
 - ✓ Что такое кривая ROC и как с помощью нее отслеживать чувствительность и избирательность одновременно.
-



Эта глава немного отличается от двух предыдущих — основное внимание в ней уделяется не построению классификационных моделей, а их оценке. Для специалиста по машинному обучению возможность оценивать производительность различных моделей не менее важна, чем умение их обучать. Мы редко обучаем на наборе данных всего одну модель. Как правило, обучаем несколько разных и выбираем ту, которая работает лучше всего. Мы также должны убедиться в качестве моделей, прежде чем запускать их в производство. Качество модели не так-то легко измерить, так что в этой главе изучим ряд методов оценки классификационных моделей. В главе 4 мы узнали, как оценивать регрессионные модели, поэтому данную главу можно рассматривать как ее аналог, но уже с классификационными моделями.

Самый простой способ измерить производительность классификационной модели — вычислить ее достоверность. Однако мы увидим, что достоверность не дает полной картины. Некоторые модели демонстрируют большую достоверность, но при этом нехороши. Чтобы разобраться с этим, определим кое-какие полезные показатели, такие как точность и полнота. Затем объединим их в новый, более мощный показатель, называемый F-мерой. Эти показатели широко используют специалисты по обработке данных для оценки собственных моделей. Однако в других дисциплинах, таких как медицина, применяются другие аналогичные показатели, например чувствительность и избирательность. С помощью последних двух мы можем построить кривую, называемую кривой рабочей характеристики приемника (ROC). Кривая ROC — это простой график, который дает отличное представление о наших моделях.

Достоверность. Как часто модель оказывается права?

В этом разделе мы обсуждаем достоверность — простейшую и наиболее распространенную меру оценки моделей классификации. Достоверность модели — это процент случаев, когда она оказывается права. Другими словами, это отношение между количеством правильно спрогнозированных точек данных и их общим числом. Например, если мы оцениваем модель на тестовом наборе данных из 1000 элементов и модель дала правильный прогноз метки 875 раз, то она имеет достоверность $875/1000 = 0,875$, или 87,5 %.

Достоверность — это наиболее распространенный способ оценки классификационной модели, и мы всегда должны использовать его. Но, как вскоре увидим, иногда достоверность не дает полного представления о производительности модели. Начнем с рассмотрения двух примеров, которые будем изучать на протяжении всей этой главы.

Два примера моделей — коронавирус и спам

В этой главе мы используем свои показатели для оценки нескольких моделей на двух наборах данных. Первый — это набор медицинских данных пациентов, в котором у некоторых из них диагностирован коронавирус. Второй — набор электронных писем, часть из которых помечены как *спам*. Как мы узнали в главе 1, спам — это термин, которым называют нежелательную почту. В главе 8 изучим такой набор данных гораздо более подробно, когда доберемся до наивного байесовского алгоритма. В этой главе мы не строим модели. Вместо этого используем модели как черные ящики и оцениваем их на основе того, сколько точек данных они прогнозируют правильно. Оба набора данных полностью выдуманы.

Набор медицинских данных — пациенты с диагнозом «коронавирус»

Наш первый набор данных — это медицинские данные 1000 пациентов. У 10 из них диагностирован коронавирус, а у остальных 990 — диагноз «здоров». Таким образом, метками в этом наборе будут «болен» или «здоров» согласно диагнозу. Цель модели — прогнозирование диагноза на основе признаков каждого пациента.

Набор данных электронной почты — письма с пометкой «спам» или «полезное»

Второй набор данных — это 100 электронных писем. Из них 40 — спам, а остальные 60 — не спам. Метки в этом наборе данных — «спам» и «полезное». Цель модели — прогнозирование метки на основе признаков электронного письма.

Сверхэффективная, но сверхбесполезная модель

Достоверность — очень полезный показатель, но дает ли она полное представление о модели? Нет, и мы проиллюстрируем это на примере. Но пока сосредоточимся на наборе данных по коронавирусу. К набору данных электронной почты перейдем в следующем разделе.

Предположим, специалист по обработке данных сообщает нам: «Я разработал тест на коронавирус, который выполняется за 10 секунд, не требует никаких обследований, а его достоверность 99 %!» Радоваться нам или отнестись к этому утверждению скептически? Вероятнее всего, мы бы проявили здоровый скепсис. Почему? Скоро увидим, что вычисления достоверности модели не всегда достаточно. Модель может иметь достоверность 99 % и при этом быть совершенно бесполезной.

Можно ли построить совершенно бесполезную модель, которая предсказывает коронавирус на нашем наборе данных и права в 99 % случаев? Напомним, что набор данных содержит сведения о 1000 пациентов, 10 из которых заражены коронавирусом. Отложите книгу на минуту и подумайте о том, как можно построить модель обнаружения коронавируса, которая окажется права в 99 % случаев, для этого набора данных.

Модель может работать так: просто диагностировать каждого пациента как здорового. Хотя и простая, но это все равно модель, которая прогнозирует все как один класс.

Какова же достоверность такой модели? Из 1000 попыток она оказалась права 10 раз и права 990 раз. Это как раз дает нам достоверность 99 %. Однако эта модель, можно сказать, сообщает всем, что они здоровы, в разгар пандемии, что поистине ужасно!

Так в чем же проблема с моделью? Проблема в том, что ошибки не создаются равными и некоторые обходятся значительно дороже, чем другие, что мы увидим в следующем разделе.

Как устранить проблему с достоверностью? Определение различных типов ошибок и способы их измерения

В предыдущем разделе мы построили бесполезную модель, которая тем не менее обладала большой достоверностью. В этом разделе попробуем понять, что пошло не так. Мы изучим, в чем заключалась проблема с достоверностью вычислений у этой модели, и введем несколько слегка различающихся показателей, которые позволят лучше ее оценить.

Первое, что нам понадобится, — это типы ошибок. В следующем разделе мы увидим, что часть ошибок более критичны, чем другие. Затем в разделе от «Хранение правильно и неправильно классифицированных точек в таблице...» до «Полнота, точность или F-мера...» изучим различные показатели, которые лучше приспособлены для обнаружения критических ошибок, чем достоверность.

Ложноположительные и ложноотрицательные результаты — что хуже?

Во многих случаях общее количество ошибок не говорит всей правды о производительности модели. Нам придется углубляться и выявлять определенные типы ошибок другими способами. В этом разделе мы изучим два типа ошибок. Какими будут типы ошибок, которые способна допустить коронавирусная модель? Она может диагностировать здорового человека как больного или больного — как здорового. В нашей модели условно обозначим больных пациентов как положительных. Такие два типа ошибок называются ложноположительными и ложноотрицательными:

- **ложноположительный результат** — здоровому человеку ошибочно поставлен диагноз «болен»;
- **ложноотрицательный результат** — больному человеку ошибочно поставлен диагноз «здоров».

В общем случае ложноположительный результат — это точка данных с отрицательной меткой, но модель ошибочно классифицирует ее как положительную. Ложноотрицательный результат — это точка данных с положительной меткой, но модель ошибочно классифицирует ее как отрицательную. Нетрудно догадаться, что у случаев, которые верно диагностированы, также есть названия:

- **истинно положительный результат** — больному человеку поставлен диагноз «болен»;
- **истинно отрицательный результат** — здоровому человеку поставлен диагноз «здоров».

В общем случае истинно положительной называется точка данных, имеющая положительную метку, которая правильно классифицируется как положительная, а истинно отрицательной — точка с отрицательной меткой, которая правильно определяется как отрицательная.

Теперь обратимся к набору данных электронной почты. Допустим, у нас имеется модель, которая прогнозирует, является ли электронное письмо спамом. Будем считать, что положительные прогнозы — это спам. Таким образом, два типа ошибок выглядят так:

- **ложноположительный результат** — сообщение электронной почты ошибочно классифицировано как спам;
- **ложноотрицательный результат** — спам-письмо неправильно классифицировано как полезное.

И правильно классифицированные электронные письма:

- **истинно положительный результат** — спам-письмо правильно классифицировано как спам;
- **истинно отрицательный результат** — электронное письмо правильно классифицировано как полезное.

На рис. 7.1 показано графическое представление моделей, в которых вертикальная прямая служит границей, зона левее нее отрицательная, правее — положительная. Слева — коронавирусная модель, в которой люди диагностируются как здоровые или больные. Справа — модель обнаружения спама, в которой электронные письма классифицируются как спам или полезные. Для каждой модели мы выделили некоторые из ошибок, разделив их на ложноположительные и ложноотрицательные. Треугольники — это точки с положительными метками, кружки — с отрицательными. Четыре элемента, упомянутые ранее, таковы:

- треугольник справа от прямой — истинно положительный;
- треугольник слева от прямой — ложноотрицательный;
- кружок справа от прямой — ложноположительный;
- кружок слева от прямой — истинно отрицательный.

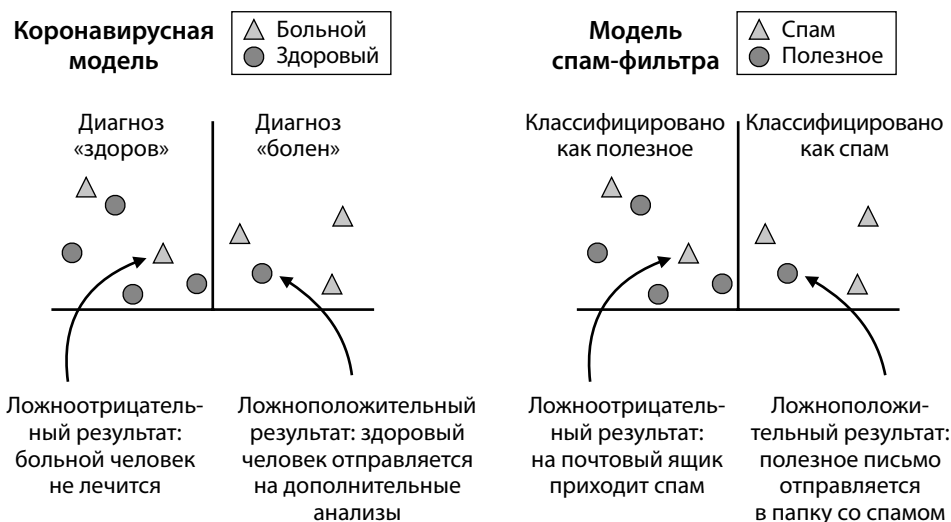


Рис. 7.1. Примеры двух моделей, которые широко используются в реальной жизни. Их мы будем рассматривать на протяжении всей главы

Обратите внимание на то, что обе модели дают следующие количественные результаты:

- три истинно положительных;
- четыре истинно отрицательных;
- один ложноположительный;
- два ложноотрицательных.

Чтобы увидеть разницу между коронавирусной моделью и моделью спам-фильтра, нам придется проанализировать, какая из них хуже в промежуток между ложноположительными и ложноотрицательными результатами. Прделаем это отдельно для каждой из них.

Анализ ложноположительных и ложноотрицательных результатов в коронавирусной модели

Давайте остановимся и подумаем. Что хуже в коронавирусной модели — ложноположительный или ложноотрицательный результат? Другими словами, что хуже — неправильно диагностировать здорового пациента или признать здоровым больного? Скажем, диагностировав пациента как здорового, мы отправляем его домой без лечения, а диагностировав пациента как больного, посылаем его на дополнительные анализы. Неправильный диагноз здоровому человеку может доставить неудобства, так как ему придется задержаться для дополнительных анализов. Однако неправильный диагноз больному человеку означает, что он не получит должного лечения и его состояние может ухудшиться. Кроме того, потенциально он может заразить многих других. Следовательно, **в коронавирусной модели ложноотрицательный результат намного хуже ложноположительного.**

Анализ ложноположительных и ложноотрицательных результатов в модели нежелательной почты

Теперь проведем тот же анализ для модели спам-фильтра. В этом случае предположим, что если классификатор спама классифицирует электронное письмо как спам, то оно автоматически удаляется. Если он классифицирует его как полезное, то письмо отправляется в почтовый ящик. Какая из ошибок выглядит хуже — ложноположительная или ложноотрицательная? Другими словами, что хуже — неправильно классифицировать электронное письмо как спам и удалить его или же неправильно классифицировать спам-письмо как полезное и отправить его в папку **Входящие**? Лично я думаю, что удаление хорошего письма намного хуже, чем отправка спама в папку **Входящие**. Случайное письмо со спамом в нашем почтовом ящике может раздражать, но удаленное полезное письмо может привести к катастрофе! Представьте, каким огорчением будет удаленное письмо от бабушки с сообщением о том, что она испекла печенье. Таким образом, **в модели спама ложноположительный результат намного хуже, чем ложноотрицательный.**

Вот в чем разница между этими двумя моделями. В коронавирусной модели хуже ложноотрицательный результат, тогда как в модели спам-фильтра хуже ложноположительный. Проблема с измерением достоверности в любой из этих моделей состоит в том, что достоверность расценивает оба типа ошибок как одинаково серьезные.

В разделе «Сверхэффективная, но сверхбесполезная модель» был приведен пример модели, которая диагностировала каждого пациента как здорового. Она допустила только 10 ошибок на выборке из 1000 пациентов. Однако все они были ложноотрицательными, что ужасно. Если бы эти 10 ошибок оказались ложноположительными, модель была бы намного лучше.

В следующих разделах мы разработаем два новых показателя, аналогичных достоверности. Первый поможет нам с моделями, в которых ложноотрицательные результаты хуже, а второй — с моделями, в которых хуже ложноположительные.

Хранение правильно и неправильно классифицированных точек в таблице. Матрица неточностей

В предыдущем разделе мы узнали о ложноположительных, ложноотрицательных, истинно положительных и истинно отрицательных результатах. Чтобы отслеживать эти четыре сущности, сведем их в таблицу, удачно названную матрицей неточностей. Для моделей бинарной классификации (они предсказывают два класса) *матрица неточностей* состоит из двух строк и двух столбцов. В строках мы отмечаем истинные метки (в медицинском примере это состояние человека, болен он или здоров), а в столбцах — прогнозируемые (диагноз человека «болен» или «здоров»). Элементы по диагонали классифицируются правильно, а элементы вне диагонали — нет. Общая матрица неточностей проиллюстрирована в табл. 7.1, а конкретные примеры моделей на этих двух наборах данных приведены в табл. 7.2–7.5. Ее так называют, потому что она позволяет легко увидеть, не путает ли модель два класса — положительный («болен») и отрицательный («здоров»). Матрица помогает подсчитать, сколько раз каждый класс был предсказан правильно, а сколько раз спутан с другим.

Таблица 7.1. Матрица неточностей

Состояние человека	Прогноз	
	положительный	отрицательный
Болен	Количество истинно положительных результатов	Количество ложноотрицательных результатов
Здоров	Количество ложноположительных результатов	Количество истинно отрицательных результатов

Для существующей модели (той, которая диагностирует каждого пациента как здорового), названной нами коронавирусной моделью 1, матрица неточностей показана в табл. 7.2. Она помогает разобраться в модели и различить два типа ошибок. Данная модель допускает 10 ложноотрицательных ошибок (больной человек диагностирован как здоровый) и ноль ложноположительных ошибок (здоровый человек диагностирован как больной). Обратите внимание на то, что модель дает слишком много ложноотрицательных результатов, которые в данном случае являются наихудшим типом ошибки, а это означает, что модель недостаточно хороша.

Таблица 7.2. Матрица неточностей коронавирусной модели 1

Коронавирусная модель 1	Диагноз	
	«болен» (прогноз положительный)	«здоров» (прогноз отрицательный)
Болен (положительный результат)	0 (количество истинно положительных)	10 (количество ложноотрицательных)
Здоров (отрицательный)	0 (количество ложноположительных)	990 (количество истинно отрицательных)

Для задач с большим количеством классов у нас будет большая матрица неточностей. Например, если модель разделяет изображения на трубокzubов, птиц, кошек и собак, то матрица неточностей представляет собой матрицу 4×4 , где в строках размещены истинные метки (тип животного), а в столбцах — предсказанные (тип животного, определенный моделью). В этой матрице неточностей правильно классифицированные точки также расположены по диагонали, а неправильно классифицированные — в стороне.

Полнота. Сколько положительных примеров классифицированы правильно?

Теперь, когда мы изучили два типа ошибок, рассмотрим показатель, который даст коронавирусной модели 1 гораздо более низкую оценку. Мы установили, что проблема заключается в том, что модель дает слишком много ложноотрицательных результатов. Иными словами, она диагностирует слишком много больных людей как здоровых.

Предположим на минуту, что мы вообще не возражаем против ложноположительных результатов. Скажем, если модель диагностирует здорового человека как больного, ему, возможно, потребуется пройти дополнительный тест или соблюдать карантин, но это не проблема. Само собой, ложные срабатывания тоже стоят дорого, но пока притворимся, что это не так. В этом случае нам понадобится показатель, который заменяет достоверность, придает большее значение поиску положительных случаев и уделяет меньше внимания ошибочной классификации отрицательных.

Чтобы найти его, нужно понять, какова наша цель. Если мы хотим вылечить коронавирус, то конечная цель — выявить всех больных в мире. Не имеет значения, что мы случайно отнесем к ним тех, кто не болен. Главное — найти всех больных. Это ключ к разгадке. Новый показатель, называемый *полнотой*, измеряет именно это — скольким больным наша модель поставила правильный диагноз.

Говоря в общем, полнота определяет долю правильных прогнозов среди точек данных с положительной меткой. Это количество истинно положительных результатов, поделенное на общее количество положительных результатов.

Коронавирусная модель 1 имеет в общей сложности 0 истинно положительных результатов из 10 положительных, поэтому ее полнота составляет $0/10 = 0$. Другой способ выражения — количество истинно положительных результатов, деленное на сумму истинно положительных и ложноотрицательных результатов:

$$\text{полнота} = \frac{\text{истинно положительные результаты}}{\text{истинно положительные} + \text{ложноотрицательные}}.$$

Для сравнения предположим, что у нас была также коронавирусная модель 2. Ее матрица неточностей приведена в табл. 7.3. Эта модель допустила больше ошибок, чем первая, — целых 50 вместо 10. Достоверность второй модели составляет $950/1000 = 0,95$, или 95 %. С точки зрения достоверности она не так хороша, как первая.

Однако вторая модель верно диагностировала восемь из десяти больных и 942 из 1000. Другими словами, у нее два ложноотрицательных результата и 48 ложноположительных.

Таблица 7.3. Матрица неточностей коронавирусной модели 2

Коронавирусная модель 2	Диагноз	
	«болен»	«здоров»
Больные	8 (истинно положительные)	2 (ложноотрицательные)
Здоровые	48 (ложноположительные)	942 (истинно отрицательные)

Полнота этой модели представляет собой количество истинно положительных результатов (восемь больных людей с верным диагнозом), деленное на общее количество положительных (10 больных), что составляет $8/10 = 0,8$, или 80 %. С точки зрения полноты вторая модель намного лучше первой. Давайте для ясности обобщим расчеты.

Коронавирусная модель 1:

- истинно положительные (больные пациенты с диагнозом «болен», отправленные на дополнительные анализы) — 0;
- ложноотрицательные (больные пациенты с диагнозом «здоров», отправленные домой) — 10;
- полнота — $0/10 = 0$ %.

Коронавирусная модель 2:

- истинно положительные (больные пациенты с диагнозом «болен», отправленные на дополнительные анализы) — 8;
- ложноотрицательные (больные пациенты с диагнозом «здоров», отправленные домой) — 2;
- полнота — $8/10 = 80$ %.

Такие модели, как коронавирусная, где ложноотрицательные результаты стоят намного дороже, чем ложноположительные, — это *модели с высокой полнотой*.

Теперь, когда у нас есть показатель полноты, можем ли мы ее перехитрить так же, как поступили с достоверностью? Другими словами, можем ли построить модель, обладающую 100 %-ной полнотой? Удивительно, но такую модель построить можно! Если мы создадим модель, которая диагностирует каждого пациента как больного, то ее полнота будет 100 %. Однако она тоже ужасна, потому что, несмотря на отсутствие ложноотрицательных результатов, у нее слишком много ложноположительных. Похоже, нам по-прежнему не хватает показателей для правильной оценки наших моделей.

Точность. Сколько примеров положительно классифицированы правильно?

В предыдущем разделе мы изучили полноту — показатель, который измеряет, насколько хорошо наша модель справилась с ложноотрицательными результатами. Он хорошо подходит для коронавирусной модели — мы знаем, что она не может позволить себе слишком много ложноотрицательных результатов. В этом разделе узнаем об аналогичном показателе — *точности*, который измеряет, насколько хорошо модель справляется с ложноположительными результатами. Мы будем использовать его для оценки модели спам-фильтра, потому что ей не нужно слишком много ложноположительных результатов.

Как и в случае с полнотой, чтобы разработать показатель, сначала нужно задать цель. Нам нужен спам-фильтр, который не удаляет полезные письма. Пусть вместо удаления он отправляет их в ящик со спамом. Затем мы покопаемся в нем, надеясь, что не найдем ни одного полезного письма. Таким образом, этот показатель призван измерять именно это — сколько писем из ящика со спамом на самом деле оказались спамом? Другими словами, сколько писем с прогнозом «спам» на самом деле являются таковыми. Этот показатель назовем *точностью*.

Если более формально, то точность принимает в расчет только те точки данных, которые были помечены как положительные, и определяет, сколько среди них истинно положительных. Поскольку точки данных с положительным прогнозом — это совокупность истинно положительных и ложноположительных результатов, формула выглядит следующим образом:

$$\text{точность} = \frac{\text{истинно положительные}}{\text{истинно положительные} + \text{ложноположительные}}.$$

Помните, что в нашем наборе данных из 100 электронных писем 40 являются спамом, а 60 — нет. Допустим, мы обучили две модели, называемые моделью

спам-фильтра 1 и моделью спам-фильтра 2. Их матрицы неточностей приведены в табл. 7.4 и 7.5 соответственно.

Таблица 7.4. Матрица неточностей модели спам-фильтра 1

Модель спам-фильтра 1	Прогноз	
	спам	полезное
Спам	30 (истинно положительные)	10 (ложноотрицательные)
Полезное	5 (ложноположительные)	55 (истинно отрицательные)

Таблица 7.5. Матрица неточностей модели спам-фильтра 2

Модель спам-фильтра 2	Прогноз	
	спам	полезное
Спам	35 (истинно положительные)	5 (ложноотрицательные)
Полезное	10 (ложноположительные)	50 (истинно отрицательные)

С точки зрения достоверности модели одинаково хороши — обе делают правильные прогнозы в 85 % случаев (85 правильных выводов относительно 100 электронных писем). Однако на первый взгляд кажется, что первая модель лучше, чем вторая, так как она удаляет только пять полезных писем, а вторая — 10. Давайте рассчитаем точность.

Модель спам-фильтра 1:

- истинно положительные (удаленные спам-письма) — 30;
- ложноположительные (удаленные полезные письма) — 5;
- точность — $30/35 = 85,7\%$.

Модель спам-фильтра 2:

- истинно положительные (удаленные спам-письма) — 35;
- ложноположительные (удаленные полезные письма) — 10;
- точность — $35/45 = 77,7\%$.

Как и предполагалось, оценка точности отдала победу первой модели. Мы пришли к выводу, что модели, подобные модели спам-фильтра, в которых ложноположительные результаты стоят намного дороже, чем ложноотрицательные, оказываются *высокоточными*. Так почему же первая модель лучше второй? Последняя удалила десять полезных электронных писем, тогда как первая — только пять. Вторая модель, возможно, очистила больше спама, чем первая, но это не компенсирует пять потерянных писем.

Теперь точно так же, как мы перехитрили достоверность и полноту, можем обмануть и точность. Рассмотрим пример спам-фильтра, который никогда не обнаруживает спама. Какова точность этой модели? Ответ будет непростым, потому что удалено ноль спам-писем (ноль истинно положительных) и ноль полезных писем (ноль ложноположительных). Мы не будем пытаться разделить ноль на ноль (тогда эта книга попросту сгорит). Вместо этого используем соглашение, согласно которому модель, не допускающая ложноположительных ошибок, имеет точность 100 %. Но, конечно, спам-фильтр, который ничего не делает, не может считаться хорошим.

Данный пример призван лишь показать, что независимо от того, насколько хороши показатели, их всегда можно обмануть. Это не значит, что они не работают. Достоверность, точность и полнота — полезные инструменты в арсенале специалиста по обработке данных. Мы сами должны решить, какие из них подходят для нашей модели, определив, какие ошибки обойдутся дороже других. Всегда избегайте ловушки и не предполагайте, что модель хорошая, до того как оцените ее с помощью различных показателей.

Сочетание полноты и точности как способ оптимизации обоих показателей. F -мера

В этом разделе обсудим F -меру — показатель, который сочетает в себе и полноту, и точность. В предыдущих разделах мы рассмотрели два примера: коронавирусную модель и модель спам-фильтра, в которых более значимыми были либо ложноотрицательные, либо ложноположительные результаты. Однако в реальной жизни важно и то и другое, хотя и в разной степени. Например, нам может понадобиться модель, которая не только не ставит неправильный диагноз больным, но и не ставит неправильный диагноз чрезмерно большому количеству здоровых. Неправильный диагноз здоровому человеку может повлечь за собой ненужное и болезненное обследование или даже операцию, что скажется на его самочувствии. Точно так же нам может понадобиться модель, которая не удаляет ни одного полезного электронного письма. Но хороший спам-фильтр все равно должен отлавливать большой объем спама, в противном случае он бесполезен. F -оценка сопровождается параметром β , поэтому более распространенный термин — F_β -мера. Когда $\beta = 1$, он называется F_1 -мерой.

Вычисление F -меры

Наша цель — отыскать показатель, который даст некоторое число между полнотой и точностью. Первое, что приходит на ум, — среднее значение полноты и точности. Но сработает ли это? Могло бы, но это не наш вариант по одной фундаментальной причине. Хорошая модель обладает хорошей полнотой и хорошей точностью. Если модель имеет, скажем, полноту 50 % и точность 100 %, то среднее значение составит 75 %. Это хороший показатель, но модель может и не

быть хорошей, потому что полноты 50 % недостаточно. Нам нужен показатель, который ведет себя как среднее значение, но при этом ближе к минимальному значению из двух.

Величина, подобная среднему значению двух чисел, называется *средним гармоническим*. Принимая во внимание, что среднее значение двух чисел a и b равно $(a + b) / 2$, их среднее гармоническое равно $2ab/(a + b)$. Среднее гармоническое значение обладает следующим свойством: оно всегда меньше или равно среднему значению. Если числа a и b равны, можно быстро проверить, что их среднее гармоническое равно им обоим, точно так же как и среднее значение. Но в других случаях среднее гармоническое окажется меньше. Рассмотрим пример: если $a = 1$ и $b = 9$, то среднее значение равно 5. Среднее гармоническое при этом равно $(2 \times 1 \times 9)/(1 + 9) = 1,8$.

F_1 -мера определяется как среднее гармоническое значение между точностью и полнотой:

$$F_1 = \frac{2PR}{P + R},$$

где P — точность, R — полнота.

Если оба числа большие, то F_1 -мера тоже высокая. Однако, если одно из них небольшое, F_1 -мера окажется низкой. Цель F_1 -меры состоит в том, чтобы определить, является ли высокой как полнота, так и точность, после чего поднять тревогу, если одна из этих оценок низкая.

Вычисление F_β -меры

В предыдущем разделе мы узнали об F_1 -мере — параметре, который для оценки модели сочетает в себе полноту и точность. Однако иногда нам нужно, чтобы полнота была больше, чем точность, или наоборот. Таким образом, нам может потребоваться при объединении двух оценок придать одной из них больший вес. Это означает, что иногда нам может понадобиться модель, которая принимает во внимание как ложноположительные, так и ложноотрицательные результаты, но придает больший вес одной категории. Например, коронавирусную модель гораздо сильнее беспокоят ложноотрицательные результаты, потому что от правильной идентификации вируса может зависеть жизнь человека. Тем не менее она по-прежнему не хочет давать слишком много ложноположительных прогнозов, потому что мы, скорее всего, не захотим тратить излишние ресурсы на повторное обследование здоровых людей. Модель спам-фильтра гораздо больше заботят ложноположительные результаты, потому что нам совсем не нужно удалять полезные электронные письма. Однако нет нужды генерировать слишком много ложноотрицательных результатов, что приведет к захламлению почтового ящика спам-сообщениями.

Тут-то и вступает в дело F_β -мера. Формула F_β -меры на первый взгляд кажется сложной, но, если внимательно посмотреть на нее, окажется, что она делает именно то, что нужно. В F_β -мере используется параметр, называемый β (греческая буква «бета»), который может принимать любое положительное

значение. Смысл β состоит в том, чтобы действовать как циферблат, который мы поворачиваем, чтобы придать большую значимость точности или полноте. Если мы передвинем шкалу β на ноль, то получим абсолютную точность, если на бесконечность — абсолютную полноту. В общем, чем ниже значение β , тем больший упор мы делаем на точность, а чем выше β , тем важнее для нас полнота. F_β -мера определяется следующим образом:

$$F_\beta = \frac{(1 + \beta^2)PR}{\beta^2 P + R}.$$

Давайте проанализируем формулу, обратив внимание на некоторые значения β .

Случай 1 — $\beta = 1$. Здесь F_β -мера становится такой:

$$F_1 = \frac{(1 + 1^2)PR}{1^2 P + R}.$$

Это то же самое, что и F_1 -мера, которая учитывает полноту и точность в равной степени.

Случай 2 — $\beta = 10$. Здесь F_β -мера становится такой:

$$F_{10} = \frac{(1 + 10^2)PR}{10^2 P + R}.$$

Это можно записать как

$$\frac{101PR}{100P + R}.$$

Это похоже на F_1 -меру, за исключением того факта, что параметру R придается гораздо большее значение, чем P . Чтобы понять это, обратите внимание на следующее: поскольку β стремится к ∞ , предел от F_β -меры равен R . Поэтому, когда требуется оценка, которая придает больший вес полноте, мы выбираем значение β , которое больше 1. Чем больше значение, тем больше внимания мы уделяем полноте и меньше — точности.

Случай 3 — $\beta = 0,1$. Здесь F_β -мера становится такой:

$$F_{0,1} = \frac{(1 + 0,1^2)PR}{0,1^2 P + R}.$$

Опять же можно записать это как

$$\frac{1,01PR}{0,01P + R}.$$

Это похоже на формулу из случая 2, за исключением того, что она придает P гораздо большее значение. Поэтому, когда нам нужна оценка между полнотой и точностью, которая придает больший вес точности, мы выбираем значение $\beta < 1$. Чем оно меньше, тем большее значение приобретает точность и меньшее — полнота. В определенном смысле можно сказать, что значение $\beta = 0$ дает нам точность, а значение $\beta = \infty$ — полноту.

Полнота, точность или F -мера — какую из оценок применять?

Итак, как мы применяем полноту и точность на практике? Как понять, что модель имеет высокую полноту или высокую точность? Используем ли мы для этого F -меру? Если да, то какое значение β выбрать? Ответы на эти вопросы зависят от нас, специалистов по обработке данных. Нам важно очень хорошо понимать решаемую проблему, что позволит определить, какая ошибка, ложноположительная или ложноотрицательная, стоит дороже.

В предыдущих двух примерах мы наблюдали следующее: поскольку коронавирусная модель должна фокусироваться на полноте, а не на точности, нужно выбрать большое значение β — скажем, 2. Напротив, модель спам-фильтра должна быть ориентирована на точность, поэтому мы должны выбрать небольшое значение β — скажем, 0,5. Получить дополнительную практику в анализе моделей и определении того, какие значения β следует использовать, можно, выполнив упражнение 7.4.

Полезный инструмент для оценки модели — кривая рабочих характеристик приемника

В разделе «Как устранить проблему с достоверностью?..» мы узнали, как оценивать модель, используя такие показатели, как точность, полнота и F -мера. Также поняли: одна из основных проблем оценки модели состоит в том, что существует более одного типа ошибок, причем разные типы имеют разную важность. Мы изучили два типа ошибок: ложноположительные и ложноотрицательные. В некоторых моделях ложноотрицательные результаты стоят намного дороже, чем ложноположительные, а в некоторых — наоборот.

В этом разделе я научу вас полезной технике оценки модели на основе ее производительности при ложноположительных и ложноотрицательных результатах одновременно. Кроме того, у этого метода есть важная особенность — шкала, которая позволяет переключаться между моделями, хорошо работающими при ложноположительных и ложноотрицательных результатах. Метод основан на *кривой рабочей характеристики приемника (ROC)*.

Прежде чем изучать кривую ROC, нам нужно ввести два новых показателя, называемых *избирательностью* и *чувствительностью*. На самом деле лишь один из них новый. Другой мы уже встречали.

Чувствительность и избирательность — два новых способа оценки модели

В разделе «Как устранить проблему с достоверностью?..» мы определили полноту и точность в качестве показателей и обнаружили, что при оценке

модели эти инструменты полезны как для ложноотрицательных, так и для ложноположительных результатов. В данном же разделе задействуем два разных, но очень похожих показателя: *чувствительность* и *избирательность*. Применяются они так же, как и предыдущие, но оказываются полезными, если нужно построить кривую ROC. Кроме того, если точность и полнота шире используются специалистами по обработке данных, то чувствительность и избирательность более распространены в медицинской области. Чувствительность и избирательность определяются так.

Чувствительность (истинно положительный показатель) — способность теста идентифицировать положительно помеченные точки. Это соотношение между количеством истинно положительных результатов и общим количеством положительных результатов (то же самое, что и полнота):

$$\text{чувствительность} = \frac{\text{истинно положительные}}{\text{истинно положительные} + \text{ложноотрицательные}}.$$

Избирательность (истинно отрицательный показатель) — способность теста идентифицировать точки с отрицательной меткой. Это соотношение между количеством истинно отрицательных и общим количеством отрицательных результатов:

$$\text{избирательность} = \frac{\text{истинно отрицательные}}{\text{истинно отрицательные} + \text{ложноположительные}}.$$

Как я уже упоминал, чувствительность — это то же, что и полнота. Однако избирательность — не то же самое, что точность (каждая номенклатура популярна в разных дисциплинах, и по этой причине используем обе). Мы подробнее говорим об этом в разделе «Полнота — это чувствительность, но точность и избирательность — разные вещи».

В коронавирусной модели чувствительность — это доля больных, которым модель правильно поставила диагноз. Избирательность — это доля здоровых людей, которых модель диагностировала правильно. Нас больше заботит правильная диагностика больных, поэтому нужно, чтобы коронавирусная модель обладала *высокой чувствительностью*.

В модели обнаружения нежелательной почты чувствительность — это доля спам-сообщений, которые мы правильно удалили. Избирательность заключается в доле полезных писем, которые правильно отправили в папку **Входящие**. Поскольку мы больше заботимся о правильном обнаружении нежелательных писем, нам нужно, чтобы модель спам-фильтра обладала *высокой избирательностью*.

Чтобы прояснить эти концепции, рассмотрим их на графическом примере, над которым работаем. А именно, рассчитаем избирательность и чувствительность для наших двух моделей на рис. 7.2 (он совпадает с рис. 7.1).

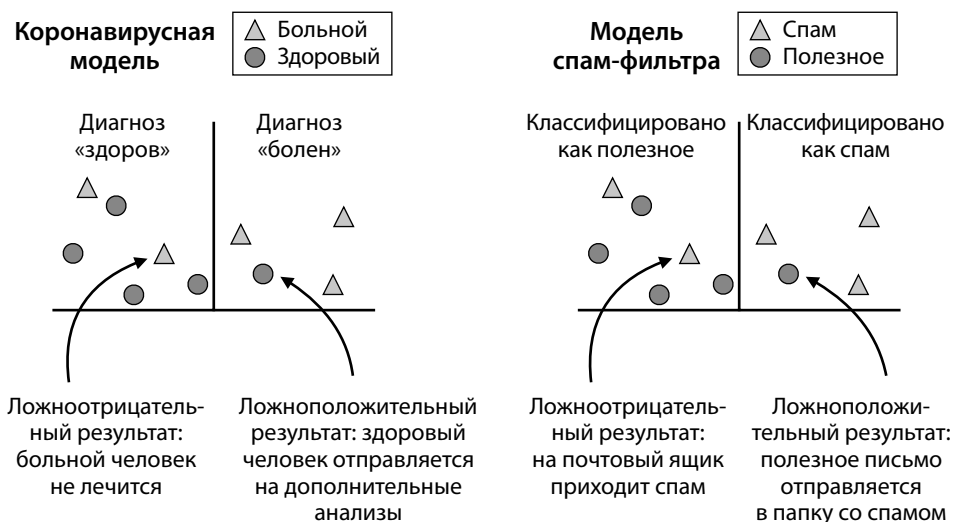


Рис. 7.2. Слева — коронавирусная модель, в которой люди диагностируются как здоровые или больные; справа — модель спам-фильтра, где электронные письма классифицируются как спам

Как мы видели ранее, эти две модели показывают следующие результаты:

- три истинно положительных;
- четыре истинно отрицательных;
- один ложноположительный;
- два ложноотрицательных.

Рассчитаем их избирательность и чувствительность.

Вычисление чувствительности

Расчет чувствительности: сколько положительных точек было правильно классифицировано моделью? Это равносильно вопросу: сколько треугольников находятся справа от прямой? У нас пять треугольников, и модель правильно классифицировала три из них, расположив справа от прямой. Поэтому чувствительность составляет $3/5$, что равно 0,6, или 60 %.

Вычисление избирательности

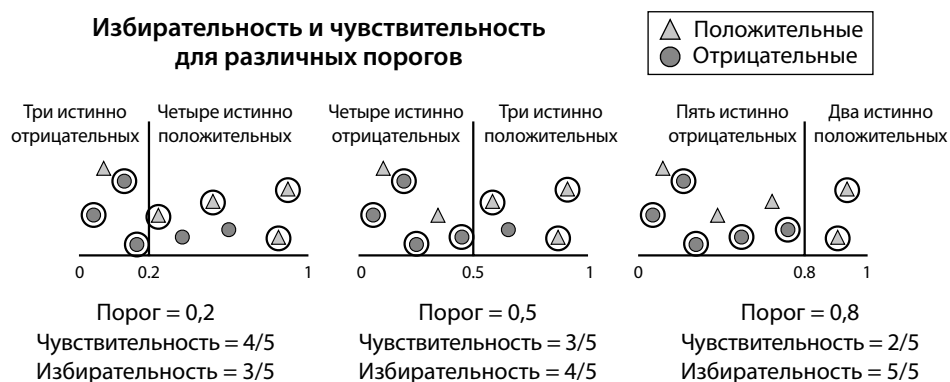
Расчет избирательности: сколько отрицательных точек было правильно классифицировано моделью? Это равносильно вопросу: сколько кружков размещены слева от прямой? У нас пять кружков, и модель правильно классифицировала четыре из них, расположив слева от прямой. Иными словами, избирательность составляет $4/5$, что равно 0,8, или 80 %.

Кривая рабочих характеристик приемника — способ оптимизации чувствительности и избирательности в модели

В этом разделе увидим, как построить кривую рабочей характеристики приемника, которая даст нам много информации о модели. Короче говоря, мы сделаем вот что: шаг за шагом будем модифицировать модель и записывать чувствительность и избирательность на каждом шаге.

Первое и единственное предположение, которое нужно сделать относительно модели, состоит в том, что она возвращает прогноз как непрерывное значение (вероятность). Это больше относится к таким моделям, как логистический классификатор, где прогнозом служит не класс, например положительный/отрицательный, а значение от 0 до 1, допустим 0,7. Что мы обычно делаем с этим значением, так это выбираем порог, например 0,5. После чего классифицируем каждую точку, обозначая прогноз, превышающий пороговое значение или равный ему, как положительный, а все остальные точки — как отрицательные. Такое пороговое значение может быть любым, не обязательно 0,5. Процедура заключается в изменении порога от 0 до 1 и регистрации чувствительности и избирательности модели при каждом пороговом значении.

Рассмотрим пример. Рассчитаем чувствительность и избирательность для трех различных пороговых значений: 0,2, 0,5 и 0,8. Каждая модель представлена вертикальной прямой. Она прогнозирует, что точки справа от нее положительные, а слева — отрицательные. Для каждой из моделей мы подсчитали количество истинно положительных и истинно отрицательных результатов, то есть количество положительных и отрицательных точек, которые были спрогнозированы верно. Мы использовали их для расчета чувствительности и избирательности. На рис. 7.3 видно, сколько точек находится слева и справа от прямой для каждого из пороговых значений. Давайте изучим их подробнее.



На рис. 7.3. Влияние изменения порога на чувствительность и избирательность. Слева — модель с низким порогом; в середине — со средним; справа — с высоким

Напомним, что чувствительность — это отношение истинно положительных результатов ко всем положительным, а избирательность — отношение истинно отрицательных ко всем отрицательным. Кроме того, для каждого из них имеется пять полных положительных и пять полных отрицательных результатов.

Пороговое значение 0,2:

- количество истинно положительных — 4;
- чувствительность — $4/5$;
- количество истинно отрицательных — 3;
- избирательность — $3/5$.

Пороговое значение 0,5:

- количество истинно положительных — 3;
- чувствительность — $3/5$;
- количество истинно отрицательных — 4;
- избирательность — $4/5$.

Пороговое значение 0,8:

- количество истинно положительных — 2;
- чувствительность — $2/5$;
- количество истинно отрицательных — 5;
- избирательность — $5/5 = 1$.

Обратите внимание на то, что низкий порог дает множество положительных прогнозов. Следовательно, у нас будет мало ложноотрицательных результатов, подразумевающих высокую оценку чувствительности, и много ложноположительных, подразумевающих низкую оценку избирательности. Аналогично, высокий порог подразумевает низкую оценку чувствительности и высокую оценку избирательности. По мере того как порог растет от низкого к высокому значению, чувствительность уменьшается, а избирательность растет. Это важный момент, которого мы коснемся позже, когда перейдем к определению наилучшего порога для модели.

Итак, мы готовы построить кривую ROC. Вначале возьмем пороговое значение 0 и будем медленно увеличивать его с небольшими интервалами, пока оно не достигнет 1. Для каждого увеличения порога мы проходим ровно через одну точку. Значения пороговых значений не важны — важно то, что на каждом шаге мы проходим ровно через одну точку (это возможно, потому что все точки дают разные оценки, но в целом это не обязательное требование). Таким образом, обозначим эти шаги как 0, 1, 2... 10. Мысленно проведите на рис. 7.3 вертикальную линию, начинающуюся с 0, медленно движущуюся слева направо и захватывающую по одной точке, пока не достигнет 1. Эти шаги отражены в табл. 7.6 вместе с количеством истинно положительных и истинно отрицательных результатов, чувствительностью и избирательностью на каждом этапе. Избирательность

модели вычисляем делением числа истинно положительных результатов на общее количество положительных прогнозов. В качестве последнего шага вычисляем избирательность, разделив количество истинно отрицательных результатов на общее количество отрицательных.

Следует отметить, что на первом шаге (шаг 0) линия находится на пороге 0. Это означает, что каждая точка классифицируется моделью как положительная. Все положительные точки также классифицируются как положительные, так что каждый положительный результат является истинно положительным. Это означает, что на отметке 0 чувствительность $5/5 = 1$. Но поскольку каждая отрицательная точка классифицируется как положительная, истинно отрицательных моментов не будет, поэтому избирательность равна $0/5 = 0$. Аналогично на последнем шаге (шаг 10) пороговое значение равно 1, и это можно проверить, так как каждая точка классифицируется как отрицательная. Чувствительность теперь равна 0, а избирательность 1. Для ясности три модели, изображенные на рис. 7.3, выделены в табл. 7.6 как отметки 4, 6 и 8 соответственно.

Таблица 7.6. Этапы повышения порога — важный шаг при построении кривой ROC

Шаг	Истинно положительные	Чувствительность	Истинно отрицательные	Избирательность
0	5	1,0	0	0,0
1	5	1,0	1	0,2
2	4	0,8	1	0,2
3	4	0,8	2	0,4
4	4	0,8	3	0,6
5	3	0,6	3	0,6
6	3	0,6	4	0,8
7	2	0,4	4	0,8
8	2	0,4	5	1,0
9	1	0,2	5	1,0
10	0	0,0	5	1,0

В качестве последнего шага построим значения чувствительности и избирательности. Это кривая ROC, которую мы видим на рис. 7.4. Здесь каждая из черных точек соответствует временному интервалу, обозначенному внутри точки, горизонтальная координата — чувствительности, а вертикальная — избирательности.

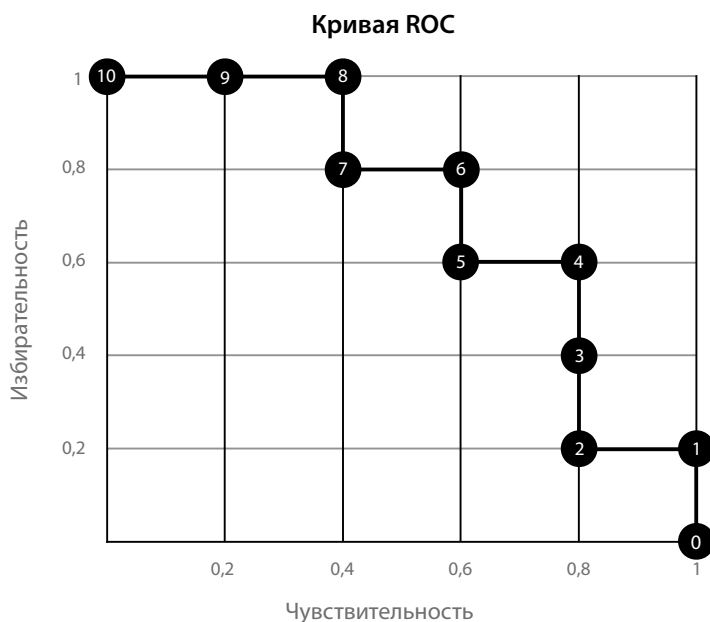


Рис. 7.4. Кривая ROC, дающая нам много информации о модели

Показатель того, насколько хороша модель, — AUC (площадь под кривой)

Как говорилось ранее, оценка модели машинного обучения — крайне важная задача, так что в данном разделе обсудим, как использовать кривую ROC для оценки модели. На самом деле мы уже проделали всю работу. Осталось лишь рассчитать площадь под кривой, или AUC. В верхней части рис. 7.5 показаны три модели, в которых прогноз задан горизонтальной осью (от 0 до 1). Внизу расположены три соответствующие кривые ROC. Размер каждого из квадратов $0,2 \times 0,2$. Количество квадратов под кривыми — 13, 18 и 25, что дает площади под кривой 0,52, 0,72 и 1.

Обратите внимание на то, что лучший результат, который способна дать модель, — $AUC = 1$ (соответствует модели справа). Худшее, что может показать модель, — $AUC = 0,5$. Это означает, что она ничем не отличается от простого угадывания. Такова модель слева. Модель в середине — это наша оригинальная модель с $AUC 0,72$.

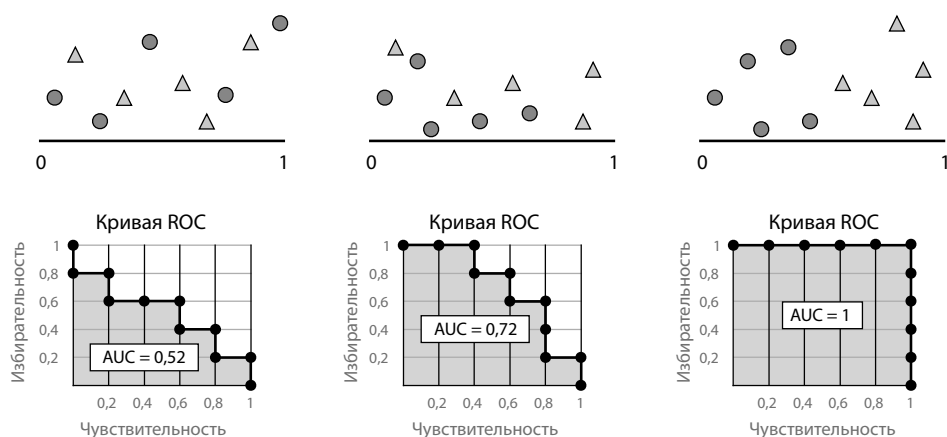


Рис. 7.5. AUC служит хорошим показателем того, насколько хороша модель

А что если у модели нулевая AUC? Вопрос неоднозначный. $AUC = 0$ означает, что модель неправильно классифицирует каждую точку. Но плохая ли это модель? На самом деле превосходная, потому что нам нужно только обратить все положительные и отрицательные прогнозы и получить идеальный результат. Это то же самое, что задать вопрос: «Правда или ложь?» — человеку, который всегда врет. Чтобы узнать у него правду, нужно лишь перевернуть все его ответы. Иными словами, худшее, что мы можем получить в модели бинарной классификации, — это $AUC = 0,5$, так как это соответствует человеку, который врет в 50 % случаев. От него не получить правдивой информации, потому что неизвестно, врет ли он в конкретном случае! Кстати, если имеется модель с $AUC < 0,5$, можно объединить положительные и отрицательные прогнозы, чтобы получить модель с $AUC > 0,5$.

Как принимать решения с помощью кривой ROC

ROC — это очень полезный график, который дает много информации о модели. В этом разделе мы узнаем, как использовать его для улучшения модели. Короче говоря, мы применим ROC для настройки порогового значения в модели и выбора наилучшей модели для нашего случая.

В начале главы мы представили две модели: коронавирусную модель и спам-фильтр. У них имелось важное различие: коронавирусная модель требовала высокой чувствительности, а модель спам-фильтра — высокой избирательности. Каждая модель требует определенной чувствительности и избирательности в зависимости от решаемой задачи. Допустим, мы обучаем модель, которая должна обладать высокой чувствительностью, и получаем модель с низкой чувствительностью и высокой избирательностью. Есть ли какой-нибудь способ обменять часть избирательности на чувствительность?

Ответ — да! Можно найти компромисс между избирательностью и чувствительностью, сдвигая порог. Напомним: впервые определив кривую ROC, мы отметили, что чем ниже порог, тем выше чувствительность и ниже избирательность в модели, а чем выше порог, тем ниже чувствительность и выше избирательность. Когда вертикальная линия, соответствующая порогу, находится в самом левом углу, каждая точка прогнозируется как положительная. Поэтому все положительные значения являются истинно положительными. Когда же вертикальная линия располагается в самом правом углу, каждая точка прогнозируется как отрицательная, поэтому все отрицательные значения являются истинно отрицательными. Когда мы перемещаем линию вправо, то теряем часть истинно положительных точек и получаем несколько истинно отрицательных. Таким образом, чувствительность уменьшается, а избирательность растет. Обратите внимание на то, что по мере изменения порога от 0 до 1 мы перемещаемся вверх и влево по кривой ROC, как показано на рис. 7.6.

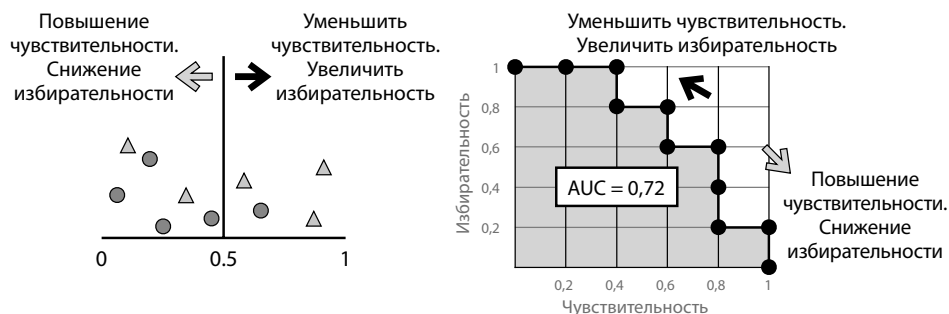


Рис. 7.6. Порог модели во многом зависит от чувствительности и избирательности, и эта взаимосвязь поможет выбрать идеальный порог для модели

Почему так происходит? Порог указывает, где мы проводим прямую при классификации точки. Например, в коронавирусной модели порог говорит, где провести черту, от которой зависит, куда отправляют человека — на дополнительные анализы или домой. Модель с низким порогом — это модель, которая отправляет людей на дополнительное обследование, если у них проявляются хотя бы легкие симптомы. Модель с высоким порогом — это модель, в которой людям нужны серьезные симптомы, чтобы попасть на дополнительные анализы. Поскольку мы хотим охватить всех больных людей, для этой модели нужен низкий порог, а это значит, что ей требуется высокая чувствительность. Для наглядности на рис. 7.7 показаны три порога, использованные ранее, а также точки, которым они соответствуют на кривой.

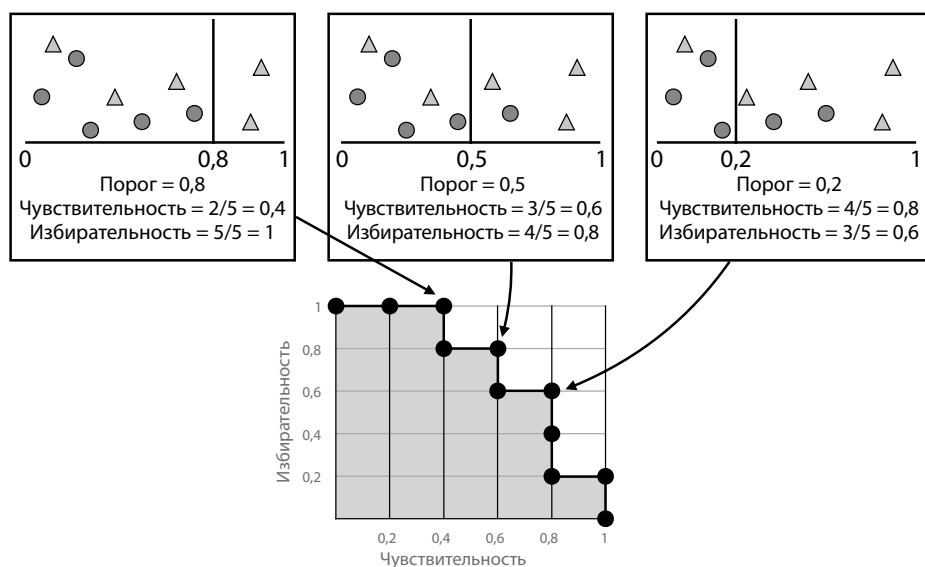


Рис. 7.7. Параллель между порогом модели и ее ROC:

слева — высокий порог, низкая чувствительность и высокая избирательность;
в середине — средние значения для всех трех параметров;
справа — низкий порог, высокая чувствительность и низкая избирательность

Если хочется, чтобы модель обладала высокой чувствительностью, мы просто сдвигаем порог влево, то есть уменьшаем его, пока не доберемся до точки на кривой, которая имеет нужную чувствительность. Обратите внимание на то, что модель может потерять некоторую избирательность, и это цена, которую мы платим. Напротив, если требуется более высокая избирательность, сдвигаем порог вправо, то есть увеличиваем, пока не дойдем до точки с нужной избирательностью. Опять же, в процессе мы потеряем в чувствительности. Кривая точно показывает, сколько мы получаем и теряем, поэтому для специалистов по обработке данных это отличный инструмент, помогающий определить наилучший порог для модели. На рис. 7.8 показан более общий пример с большим набором данных.

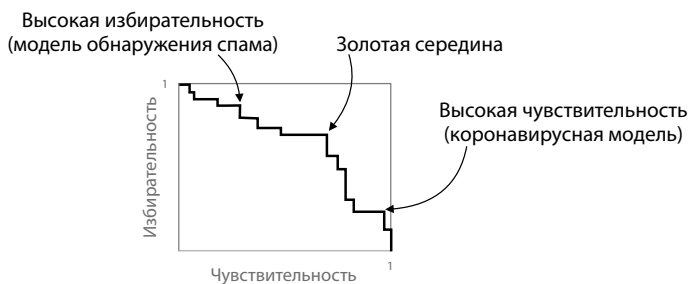


Рис. 7.8. Общий сценарий

Если нужна высокочувствительная модель, такая как коронавирусная, наша точка справа. Если требуется модель с высокой избирательностью, такая как спам-фильтр, можем выбрать точку слева. Однако, если мы хотим относительно высокой чувствительности и избирательности, наш выбор — середина. Мы, специалисты по обработке данных, несем ответственность за то, чтобы понять задачу достаточно хорошо и принять верное решение.

Полнота — это чувствительность, но точность и избирательность — разные вещи

Сейчас вы, возможно, задаетесь вопросом, как запомнить все эти термины. Дело в том, что их довольно трудно не путать. Большинству специалистов по обработке данных (включая автора) часто приходится быстро искать их в «Википедии», чтобы подстраховать свою память. Чтобы запомнить, что есть что, можно использовать мнемонику.

Например, думая о полноте, подумайте об автомобильной компании, которая выпустила автомобиль с фатальным недостатком. Им нужно найти и отозвать все неисправные автомобили в *полном* объеме. Если они случайно получают больше автомобилей, часть из которых окажутся исправными, то просто вернут их. Однако не найти одну из неисправных машин — большая проблема. Таким образом, *полнота* состоит в том, чтобы найти все примеры с положительной меткой. Это модель с высокой полнотой.

Однако если мы работаем в автомобильной компании и, перестаравшись, начинаем отзываться *все* машины, то босс может подойти и сказать: «Эй, вы отправляете на ремонт слишком много машин, а у нас заканчиваются запчасти. Не могли бы вы быть более избирательными и отправлять только те, которые *точно* неисправны». То есть нам нужно добавить модели точности и попытаться найти лишь неисправные машины, даже если мы случайно и пропустим некоторые из них (надеюсь, нет!). Это модель с высокой *точностью*.

Если говорить об избирательности и чувствительности, подумайте о датчике землетрясения, который подает звуковой сигнал каждый раз, когда происходит землетрясение. Этот датчик чрезвычайно *чувствителен*. Он сработает, даже если в соседнем доме чихнет моль. Датчик наверняка зафиксирует все землетрясения, но он отметит и многие другие события, которые не являются землетрясением. Это модель с высокой *чувствительностью*.

Теперь представим, что у этого датчика есть шкала и мы до упора уменьшаем его чувствительность. Теперь он будет подавать звуковой сигнал только при большом сотрясении. Когда датчик подаст звуковой сигнал, мы будем *знать*, что это землетрясение. Проблема в том, что он может пропустить несколько небольших или средних землетрясений. Другими словами, этот датчик очень *строго* относится к землетрясениям, поэтому он, скорее всего, не будет подавать звуковой сигнал, если происходит что-то другое. Это модель с высокой *избирательностью*.

Если мы вернемся и перечитаем предыдущие четыре абзаца, то заметим две вещи:

- полнота очень похожа на чувствительность;
- точность очень похожа на избирательность.

По крайней мере полнота и чувствительность преследуют одну и ту же цель — измерить количество ложноотрицательных результатов. Аналогично, точность и избирательность также имеют одинаковую цель, которая заключается в измерении количества ложноположительных результатов.

Оказывается, что полнота и чувствительность — это *совершенно* одинаковые явления. Однако точность и избирательность — далеко не одно и то же. Хотя обе они измеряют разные вещи, но отбраковывают модели с большим количеством ложноположительных результатов. Как же запомнить все эти показатели? Графическая эвристика вполне способна помочь нам запомнить полноту, точность, чувствительность и избирательность. На рис. 7.9 мы видим матрицу неточностей с четырьмя величинами: истинно положительными, истинно отрицательными, ложноположительными и ложноотрицательными результатами. Если мы сосредоточимся на верхней строке (примеры с положительной маркировкой), то можем рассчитать полноту, разделив число в правом столбце на сумму чисел в обоих столбцах. Если обратимся к левому столбцу (примерам, которые прогнозируются как положительные), сможем рассчитать точность, разделив число в верхней строке на сумму чисел в обеих строках. Если сосредоточимся на нижней строке (примеры, помеченные как отрицательные), то можем рассчитать избирательность, разделив число в левом столбце на сумму чисел в обоих столбцах. Другими словами:

- полнота и чувствительность соответствуют верхнему ряду;
- точность соответствует левому столбцу;
- избирательность соответствует нижнему ряду.

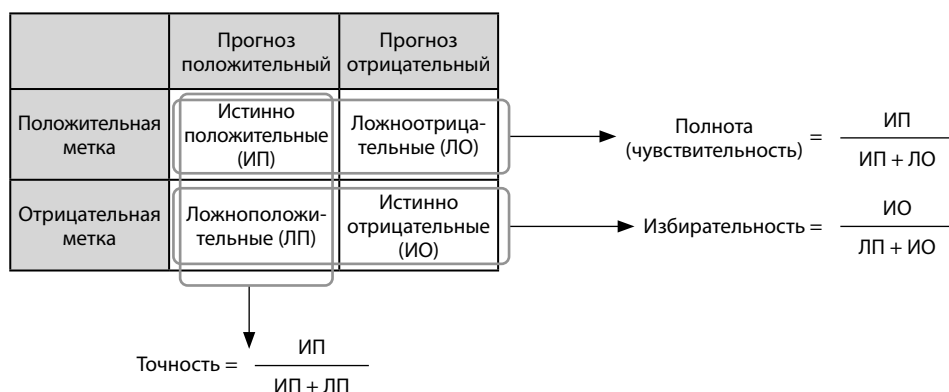


Рис. 7.9. Матрица неточностей: *верхняя строка* показывает полноту и чувствительность; *левый столбец* дает точность; *нижняя строка* дает избирательность

Подводя итог, отметим, что в обеих моделях эти величины следующие.

Медицинская модель

- **Полнота и чувствительность:** скольким из больных (положительных) диагноз «болен» поставлен правильно?
- **Точность:** сколько людей из тех, кому был поставлен диагноз «болен», на самом деле больны?
- **Избирательность:** скольким из здоровых людей (отрицательных) правильно поставлен диагноз «здоров»?

Модель электронной почты

- **Полнота и чувствительность:** сколько спам-писем (положительных) было удалено правильно?
- **Точность:** сколько из удаленных электронных писем на самом деле были спамом?
- **Избирательность:** сколько полезных писем (отрицательных) было верно отправлено в папку Входящие?

Резюме

- Умение оценивать модель так же важно, как и умение ее обучать.
- Для оценки модели есть несколько важных показателей. В этой главе мы узнали о некоторых из них: достоверности, полноте, точности, F -мере, избирательности и чувствительности.
- Достоверность вычисляется из соотношения между правильными прогнозами и общим числом прогнозов. Это полезно, но в ряде случаев может привести к сбою, особенно когда положительные и отрицательные метки не сбалансированы.
- Ошибки делятся на две категории — ложноположительные и ложноотрицательные:
 - ложноположительный результат — это точка с отрицательной меткой, которую модель неправильно прогнозирует как положительную;
 - ложноотрицательный результат — это точка, помеченная как положительная, которую модель неправильно прогнозирует как отрицательную.
- В ряде случаев ложноотрицательные и ложноположительные результаты имеют разную значимость.
- Полнота и точность — это полезные показатели оценки моделей, особенно когда те дают ложноотрицательные и ложноположительные результаты разного уровня значимости.
 - Полнота показывает, сколько положительных точек было правильно спрогнозировано моделью. Полнота небольшая, если она дает много ложноотрицательных результатов. По этой причине полнота — полезный

показатель в моделях, где не нужно много ложноотрицательных результатов, например в моделях для медицинской диагностики.

- Точность показывает, сколько точек, которые модель спрогнозировала как положительные, на самом деле являются положительными. Точность невелика, когда модель создает много ложноположительных результатов. По этой причине точность оказывается полезным показателем в моделях, где не нужно много ложноположительных результатов, таких как модели спам-фильтра.
- F_1 -мера — это полезный показатель, который сочетает в себе полноту и точность. Он возвращает значение между отзывом и точностью, ближе к меньшему.
- F_β -мера — это разновидность F_1 -меры, в которой можно настроить параметр β для придания значимости либо точности, либо полноте. Более высокие значения β делают более значимой полноту, а более низкие — точность. F_β -мера особенно полезна для оценки моделей, в которых точность или полнота имеют разную значимость, но нам по-прежнему нужно учитывать оба показателя.
- Чувствительность и избирательность — два полезных показателя, помогающих оценивать модели. Они широко используются в медицинской сфере.
 - Чувствительность, или истинно положительное отношение, измеряет, сколько положительных точек было правильно спрогнозировано моделью. Чувствительность низкая, когда модель дает много ложноотрицательных результатов. По этой причине чувствительность — полезный показатель в медицинских моделях, когда мы не хотим случайно оставить множество больных без лечения.
 - Избирательность, или истинно отрицательное отношение, измеряет, сколько отрицательных точек было правильно спрогнозировано моделью. Избирательность низкая, когда модель дает много ложноположительных результатов. По этой причине избирательность — полезный показатель в медицинских моделях, где мы не хотим случайно лечить или проводить лишние инвазивные обследования здоровых пациентов.
- Полнота и чувствительность — это одно и то же. Однако точность и избирательность — разные вещи. Точность гарантирует, что большинство прогнозируемых положительных результатов действительно положительные, а избирательность проверяет, что большинство истинно отрицательных результатов было определено.
- Увеличивая порог модели, мы уменьшаем ее чувствительность и повышаем избирательность.
- ROC, или кривая рабочих характеристик приемника, представляет собой полезный график, помогающий отслеживать чувствительность и избирательность модели для каждого отдельного значения порога.

- ROC также помогает определить, насколько хороша модель, посредством площади под кривой, или AUC. Чем ближе значение AUC к 1, тем лучше модель. Чем ближе значение AUC к 0,5, тем она хуже.
- Глядя на кривую ROC, мы принимаем решение о том, какой порог использовать, чтобы получить хорошие значения как для чувствительности, так и для избирательности (в зависимости от того, какое значение каждой из них требуется нашей модели). Это делает кривую ROC одним из самых распространенных и полезных способов оценки и улучшения модели.

Упражнения

Упражнение 7.1

Видеохостинг установил, что конкретному пользователю нравятся видео с животными и ничего более. На рисунке мы видим рекомендации, которые он получает, заходя на сайт.



Если это все данные, которыми мы располагаем согласно модели, ответьте на следующие вопросы.

- Какова достоверность модели?
- Какова полнота модели?
- Какова точность модели?
- Какова F_1 -мера модели?
- Можно ли сказать, что это хорошая рекомендательная модель?

Упражнение 7.2

Найдите чувствительность и избирательность медицинской модели со следующей матрицей неточностей.

	Прогноз	
	«Болен»	«Здоров»
Больные	120	22
Здоровые	63	795

Упражнение 7.3

Для следующих моделей установите, какая ошибка более серьезна — ложноположительная или ложноотрицательная. Исходя из этого, определите, на каком из двух показателей — точности или полноте — мы должны сделать акцент при оценке моделей.

1. Система рекомендаций фильмов, которая предсказывает, будет ли пользователь смотреть фильм.
2. Модель распознавания изображений, применяемая в беспилотных автомобилях и определяющая, есть ли на изображении пешеход.
3. Голосовой домашний помощник, который прогнозирует, отдал ли пользователь команду.

Упражнение 7.4

Имеются следующие модели.

1. Модель для беспилотного автомобиля, обнаруживающая пешехода на основе изображения с камеры машины.
2. Медицинская модель для диагностики смертельной болезни на основе симптомов.
3. Система рекомендаций, основанная на ранее просмотренных фильмах.
4. Голосовой помощник, который определяет, нужна ли пользователю помощь, на основе голосовой команды.
5. Модель обнаружения нежелательной почты, которая определяет, является ли электронное письмо спамом, на основе слов, из которых оно состоит.

Перед нами стоит задача — оценить данные модели с помощью F_β -меры. Вместе с тем значения β не даны. Какое значение β вы бы взяли для оценки каждой из моделей?

Как использовать вероятность по максимуму. Наивная байесовская модель

8



В этой главе

- ✓ Что такое теорема Байеса.
- ✓ Зависимые и независимые события.
- ✓ Предшествующие и последующие вероятности.
- ✓ Вычисление условных вероятностей на основе событий.
- ✓ Использование наивной модели Байеса для прогнозирования того, является ли электронное письмо спамом, на основе слов в теле письма.
- ✓ Кодирование наивного байесовского алгоритма на Python.



Наивная байесовская модель — важная модель машинного обучения, используемая для классификации. Это чисто вероятностная модель, которая означает, что прогноз представляет собой число от 0 до 1, указывающее на вероятность того, что метка положительна. Основным компонентом наивной байесовской модели выступает теорема Байеса.

Она играет фундаментальную роль в теории вероятностей и статистике, поскольку помогает вычислять вероятности. В ее основе предпосылка о том, что чем больше информации мы собираем о событии, тем более точно способны оценить вероятность. Предположим, нам требуется найти вероятность того, что сегодня пойдет снег. Если у нас нет информации о том, в какой местности мы находимся и какое сейчас время года, возможны лишь смутные предположения. Но если эту информацию предоставят, мы сможем лучше оценить вероятность. Например, я говорю вам, что думаю о каком-то виде животного и хочу, чтобы вы его угадали. Какова вероятность того, что это животное — собака? Учитывая, что у вас нет никакой информации, вероятность довольно мала. Однако если я добавлю, что животное, о котором я думаю, — домашний питомец, то вероятность значительно возрастает. Но если скажу, что у животного, о котором я думаю, есть крылья, вероятность сразу снизится до нуля. Каждый раз, когда я сообщаю новую информацию, ваша оценка вероятности того, что это собака, становится все более и более точной. Теорема Байеса — это способ формализовать подобный тип логики и выразить его в формулах.

Если говорить более конкретно, то теорема Байеса отвечает на вопрос: «Какова вероятность того, что при заданном Y произошло X ?» Это называется *условной вероятностью*. Вполне очевидно, что ответы на такого рода вопросы пригодятся в машинном обучении. Если мы способны ответить на вопрос: «Какова вероятность того, что метка положительна, учитывая признаки?», то у нас есть готовая модель классификации. Например, мы можем построить модель анализа настроений (как делали это в главе 6), отвечая на вопрос: «Какова вероятность того, что предложение *радостное*, учитывая содержащиеся в нем слова?» Однако, когда признаков (в данном случае слов) слишком много,

вычисление вероятности с использованием теоремы Байеса становится очень сложным. Вот тут-то и пригодится наивный алгоритм Байеса. Этот алгоритм ловко упрощает эти вычисления, помогая построить требуемую классификационную модель, называемую *наивной байесовской моделью*. Она называется так потому, что для упрощения вычислений мы делаем довольно наивное предположение, которое не обязательно верно. Однако оно помогает получить хорошую оценку вероятности.

В данной главе мы увидим, как используется теорема Байеса, рассматривая примеры из реальной жизни. Начнем с анализа интересного и в чем-то удивительного медицинского примера. Затем углубимся в наивную байесовскую модель, применив ее к распространенной проблеме машинного обучения — классификации спама. В итоге закодируем алгоритм на Python и применим его для прогнозирования на реальном наборе спам-писем.

Весь код для этой главы доступен в репозитории GitHub: https://github.com/luisguiserrano/manning/tree/master/Chapter_8_Naive_Bayes.

Болен или здоров? История с теоремой Байеса в качестве героя

Рассмотрим следующий сценарий. Вам позвонил мнительный друг, и у вас завязался следующий разговор.

Вы: «Привет!»

Друг: «Привет. У меня ужасные новости!»

Вы: «Что случилось?!»

Друг: «Я узнал об одной ужасной и редкой болезни и пошел к врачу, чтобы сдать анализ. Врач сказала, что проведет очень точный тест. А сегодня она позвонила мне и сообщила, что результат положительный! Похоже, я все-таки заразился!»

О, нет! Что вы должны сказать другу? Прежде всего успокоим его и попытаемся выяснить, насколько вероятно то, что у него именно эта болезнь.

Вы: «Во-первых, спокойно. Бывает, врачи ошибаются. Давай разберемся, насколько вероятно то, что у тебя действительно эта болезнь. Насколько точным, по словам врача, был тест?»

Друг: «Она сказала, что на 99 %. Это означает 99%-ную вероятность того, что я заболел!»

Вы: «Подожди, давайте посмотрим на *все* числа. Насколько вероятно, что у тебя может быть это заболевание, если не принимать во внимание тест? Сколько людей болеют им?»

Друг: «Я покопался в Интернете, и там говорят, что в среднем этим заболеванием болеет один из 10 000».

Вы: «Хорошо, сейчас найду листок бумаги» (*переводим друга в режим ожидания*).

Сделаем паузу для теста.

ТЕСТ. Каков, по вашему мнению, диапазон вероятности того, что у вашего друга данное заболевание (с учетом положительного результата теста)?

- 1) 0–20 %;
- 2) 20–40 %;
- 3) 40–60 %;
- 4) 60–80 %;
- 5) 80–100 %.

Давайте посчитаем. Если поразмыслить, то можно сказать, что у нас имеются два фрагмента информации.

- Тест верен в 99 % случаев. Если быть более точным (а мы проконсультировались с врачом), в среднем из каждых 100 здоровых людей тест правильно диагностирует 99 и из каждых 100 больных — тоже 99. Поэтому как на здоровых, так и на больных тест показывает достоверность 99 %.
- В среднем заболевает один из 10 000 человек.

Теперь проведем приблизительные расчеты, чтобы узнать, какова же искомая вероятность. Они показаны в матрице неточностей на рис. 8.1. Для справки: мы можем выбрать случайную группу в 1 млн человек. В среднем один из каждых 10 000 болен, поэтому мы ожидаем, что 100 человек заболеют данной болезнью, а 999 900 окажутся здоровы.

Прежде всего, проведем тест на 100 больных. Поскольку тест верен в 99 % случаев, мы ожидаем, что 99 из этих 100 человек будут правильно диагностированы как больные. Иными словами, получим 99 больных с положительным результатом теста.

Теперь проведем тест на 999 900 здоровых. Тест допускает ошибки в 1 % случаев, поэтому ожидается, что 1 % из этих 999 900 здоровых людей будут ошибочно диагностированы как больные, что составит 9999 здоровых людей с положительным результатом теста.

Это означает, что общее число людей с положительным результатом будет равно $99 + 9999 = 10\,098$. Из них лишь 99 действительно больны. Следовательно, вероятность того, что ваш друг болен (учитывая, что у него положительный результат теста), составляет $99/10\,098 = 0,0098$, или 0,98 %. Это менее 1 %! Вернемся к нашему другу.

Вы: «Не волнуйся, основываясь на цифрах, которые ты мне дал, вероятность того, что у тебя это заболевание (учитывая положительный результат теста), составляет менее 1 %!»

Друг: «О, боже мой, правда? А я уже испугался, спасибо!»

Вы: «Не благодари меня, благодари математику» (*здесь нужно подмигнуть*). Подведем итог наших расчетов. Мы располагаем следующими фактами.

- **Факт 1.** Из каждых 10 000 человек один страдает этим заболеванием.
- **Факт 2.** Из каждых 100 больных людей, которые сдают тест, 99 получают положительный результат, а один — отрицательный.
- **Факт 3.** Из каждых 100 здоровых людей, которые сдают тест, у 99 результат отрицательный, а у одного — положительный.

Выборку из 1 млн человек, представленную на рис. 8.1, можно разбить следующим образом.

- Согласно факту 1, мы ожидаем, что у 100 человек в нашей выборке будет означенное заболевание, а 999 900 человек окажутся здоровы.
- Согласно факту 2, из 100 больных у 99 был положительный результат теста, а у одного — отрицательный.
- Согласно факту 3, из 999 900 здоровых людей 9999 получили положительный результат, а 989 901 — отрицательный.

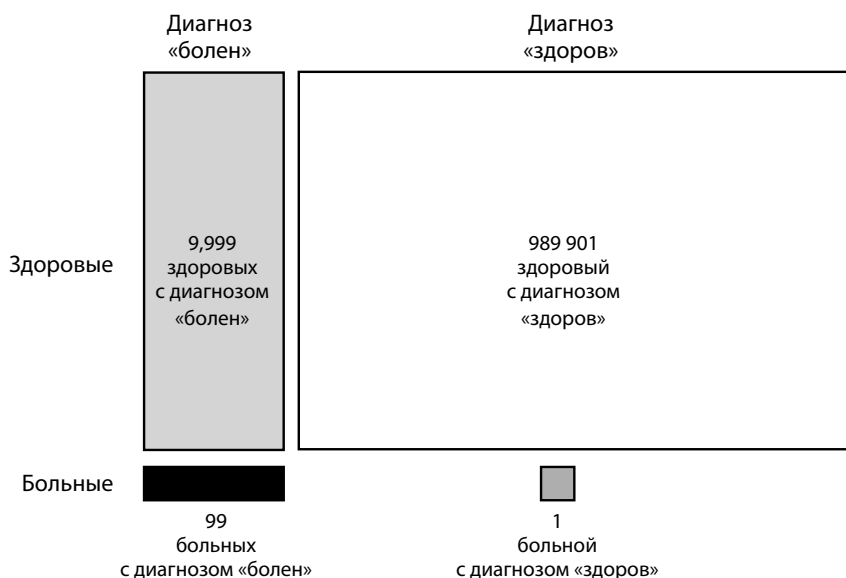


Рис. 8.1. Если вашему другу поставили диагноз «болен», у него гораздо больше шансов оказаться среди 9999 здоровых, чем среди 99 больных

Поскольку у вашего друга положительный результат теста, он должен быть в левой колонке на рис. 8.1. В ней 9999 здоровых людей с неверным диагнозом и 99 больных людей с верным. Вероятность того, что друг болен, составляет $99/9999 = 0,0098$, то есть менее 1 %.

Удивительно получается! Если тест верен в 99 % случаев, то почему же, черт возьми, он так ошибается? Если тест ошибается лишь в 1 % случаев, то сам по себе он не так уж плох. Но поскольку болеет всего один из 10 000, это означает, что человек заболевает лишь в 0,01 % случаев. Что более вероятно — быть среди 1 % тех, кому поставили неправильный диагноз, или среди 0,01 % больных? Хотя 1 % и небольшая группа, она все же намного больше 0,01 %. Проблема теста вот в чем: его частота ошибок намного больше, чем частота заболеваний. Аналогичная проблема рассматривалась в разделе «Два примера

моделей: коронавирус и спам» в главе 7. Так что мы не можем полагаться на достоверность измерений этой модели.

Чтобы лучше все это понять, можно использовать древовидные диаграммы. Мы начинаем с корня, расположенного слева, который дает две возможности: друг болен или здоров. Каждая из них разветвляется на еще две: другу поставят диагноз «здоров» или «болен». На рис. 8.2 показано дерево и количество пациентов для каждой ветви. Каждый пациент может оказаться больным или здоровым. Для каждой из вероятностей он может быть диагностирован как больной или здоровый, что дает четыре вероятности. Мы начинаем с 1 млн пациентов: 100 из них больны, а остальные 999 900 здоровы. Из 100 больных одному ошибочно ставят диагноз «здоров», а остальным 99 — правильный диагноз «болен». Из 999 900 здоровых пациентов 9999 диагностируются ошибочно как больные, а остальные 989 901 — как здоровые.

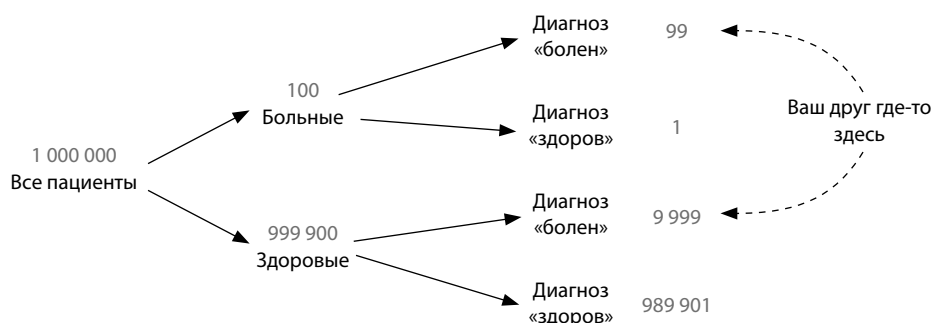


Рис. 8.2. Дерево вероятностей

На рис. 8.2 мы в очередной раз видим, что вероятность заболеть у вашего друга (учитывая, что тест дал положительный результат) составляет $99 / (99 + 9999) = 0,0098$, с учетом этого он может быть только в первой или третьей группах.

Прелюдия к теореме Байеса — априорная вероятность, событие и апостериорная вероятность

Теперь у нас есть все инструменты для формулирования теоремы Байеса. Основной целью этой теоремы является вычисление вероятности. Вначале, не имея на руках никакой информации, мы можем рассчитать только начальную вероятность, которую называем *априорной*. Затем происходит некое событие, которое дает информацию. После этого можно получить (рассчитать) гораздо лучшую оценку вероятности. Мы называем ее *апостериорной*. Априорная вероятность, событие и апостериорная вероятность проиллюстрированы на рис. 8.3.

АПРИОРНАЯ ВЕРОЯТНОСТЬ — начальная вероятность.

СОБЫТИЕ — происходит что-то, что дает нам информацию.

АПОСТЕРИОРНАЯ ВЕРОЯТНОСТЬ — окончательная (более точная) вероятность, которую мы вычисляем, используя априорную вероятность и событие.

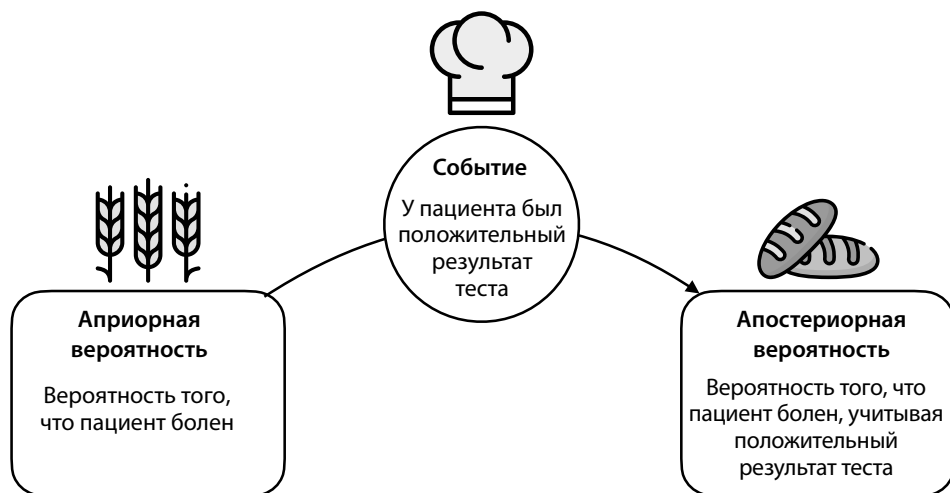


Рис. 8.3. Априорная вероятность, событие и апостериорная вероятность

Приведем пример. Представьте, что мы хотим выяснить вероятность дождя. Если мы ничего не знаем, то можем получить лишь приблизительную оценку — априорную. Если же посмотрим вокруг и обнаружим, что стоим в тропическом лесу Амазонии (событие), то сможем получить гораздо более точную оценку. На самом деле, если мы действительно в тропических лесах Амазонии, то сегодня, скорее всего, будет дождь. Эта новая оценка является апостериорной.

В медицинском примере нам требуется рассчитать вероятность того, что пациент болен.

- **Априорная вероятность.** Изначально она составляет $1/10\,000$, потому что у нас нет никакой другой информации, кроме факта, что один из каждых $10\,000$ пациентов болен. Эта вероятность, составляющая $1/10\,000$, или $0,00001$, является априорной.
- **Событие.** Внезапно появляется новая информация. В нашем случае пациент сдает тест и получает положительный результат.
- **Апостериорная вероятность.** После выдачи положительного результата мы пересчитываем вероятность болезни пациента и получаем $0,0098$. Это апостериорная вероятность.

Теорема Байеса — один из важнейших кирпичиков теории вероятностей и машинного обучения. Она настолько важна, что в честь нее названы несколько областей: *байесовское обучение*, *байесовская статистика* и *байесовский анализ*. В этой главе мы изучим теорему Байеса и вытекающую из нее важную классификационную модель — наивную байесовскую. Если в двух словах, то наивная байесовская модель делает то, что и большинство классификаторов, — предсказывает метку, исходя из набора признаков. Модель возвращает ответ в виде вероятности, которая вычисляется с помощью теоремы Байеса.

Пример использования — модель спам-фильтра

Пример, который мы изучаем в этой главе, — это модель фильтра для обнаружения нежелательной почты. Она помогает отделять спам от полезных писем. Как сказано в главах 1 и 7, спам — это название, данное нежелательной почте, а полезное письмо — это электронная почта, которая не является нежелательной.

Наивная байесовская модель выводит вероятность того, что электронное письмо является спамом или полезным письмом. Таким образом, мы сможем отправлять электронные письма — наиболее вероятный спам — в папку **Спам**, а остальные сохранять в почтовом ящике. Вероятность должна зависеть от ряда признаков электронного письма, таких как слова, составляющие его текст, отправитель, размер и т. д. В этой главе в качестве признаков будем рассматривать только текст. Данный пример не сильно отличается от анализа настроений, который мы изучали в главах 5 и 6. Ключевым моментом для этого классификатора спама является то, что каждое слово имеет определенную вероятность появления в спам-письме. Например, слово *«лотерея»* с большей вероятностью появится в спаме, чем слово *«встреча»*. Эта вероятность и лежит в основе наших расчетов.

Нахождение априорной вероятности. Вероятность того, что любое электронное письмо — спам

Какова вероятность того, что электронное письмо является спамом? Это сложный вопрос, но мы попробуем дать приблизительную оценку, которую называем априорной. Заглянем в свой почтовый ящик и подсчитаем, сколько писем в нем — спам. Представьте, что из 100 электронных писем 20 являются спамом, а 80 — полезными. Таким образом, 20 % электронных писем — это спам. Стремясь дать честную оценку, мы можем сказать следующее: *насколько нам известно*, вероятность того, что каждое новое электронное письмо будет спамом, составляет 0,2. Это априорная вероятность. Расчет показан на рис. 8.4, где спам-письма окрашены в темно-серый цвет, а полезные — в белый.

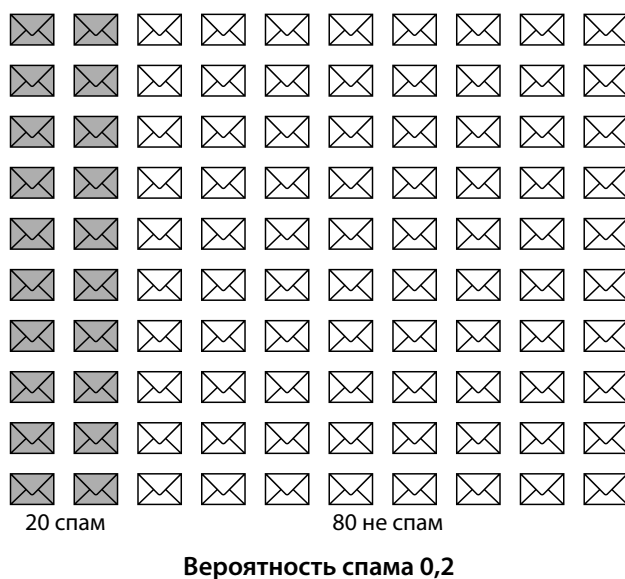


Рис. 8.4. Набор данных из 100 электронных писем, 20 из которых — спам

Нахождение апостериорной вероятности. Вероятность того, что электронное письмо — спам, поскольку содержит определенное слово

Конечно, не все электронные письма создаются одинаково. Нам требуется более обоснованное предположение о вероятности, сделать которое можно, исходя из свойства самого электронного письма. Мы можем учесть множество его свойств, таких как отправитель, размер или слова в теле письма. Для этого случая используем только слова, которые в нем встречаются. Тем не менее я рекомендую подумать, как в этом примере можно применить и другие свойства.

Допустим, мы находим определенное слово, скажем, «лотерея», которое, как правило, чаще встречается в спам-письмах. Оно и есть наше событие. Слово «лотерея» встречается в 15 спам-письмах и в пяти полезных. Таким образом, среди 20 электронных писем, содержащих слово «лотерея», 15 — это спам. То есть вероятность того, что электронное письмо, содержащее слово «лотерея», — это спам, составляет ровно $15/20 = 0,75$. Это апостериорная вероятность. Процесс ее вычисления показан на рис. 8.5. Серым изображены электронные письма, которые не содержат слова «лотерея».



Рис. 8.5. Вычисление апостериорной вероятности

Вот и все: мы вычислили вероятность того, что электронное письмо — спам, с учетом слова «лотерея». Подведем итог.

- **Априорная** вероятность составляет 0,2. Это вероятность того, что электронное письмо является спамом, если ничего не знать о нем самом.
- **Событие** заключается в том, что электронное письмо содержит слово «лотерея». Это помогает лучше оценить вероятность.
- **Апостериорная** вероятность равна 0,75. Это вероятность того, что электронное письмо является спамом, *с учетом* появления в нем слова «лотерея».

В этом примере мы рассчитали вероятность путем подсчета электронных писем и деления. В основном это сделано в целях обучения, так как в реальной жизни мы можем использовать короткую формулу для вычисления вероятности. Эта формула называется теоремой Байеса, и мы увидим ее чуть позже.

Что это еще за математика? Превращение соотношений в вероятности

Один из способов визуализации предыдущего примера — это дерево четырех возможностей, точно такое же, как для медицинского примера (см. рис. 8.2). Возможности того, что электронное письмо спам или полезное и что оно содержит или не содержит слово «лотерея». Нарисуем его (рис. 8.6), начав с корня,

который разделяется на две ветви. Верхняя ветвь соответствует спаму, а нижняя — полезным письмам. Каждая из ветвей разделяется еще на две — на электронные письма, содержащие и не содержащие слово «лотерея». Обратите внимание на то, что здесь мы также указали, сколько электронных писем из общего числа 100 принадлежит каждой конкретной группе.

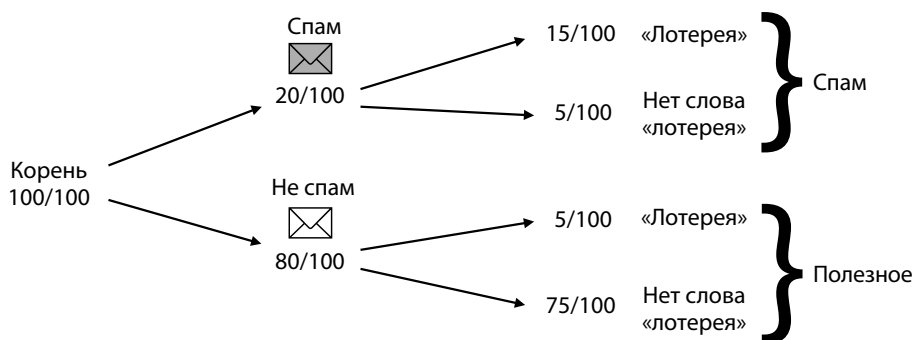


Рис. 8.6. Дерево вероятностей

Получив такое дерево и желая рассчитать вероятность того, что электронное письмо является спамом (учитывая, что оно содержит слово «лотерея»), мы просто удаляем ветви, в которых письма не содержат слова «лотерея». Все это показано на рис. 8.7.

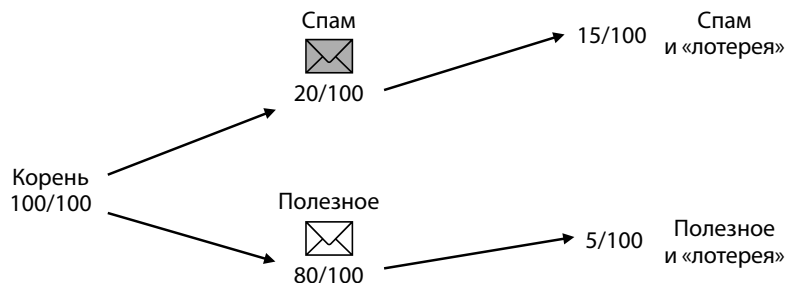


Рис. 8.7. Мы удалили у дерева две ветви, в которых электронные письма не содержат слова «лотерея»

Из первоначальных 100 электронных писем осталось 20, и из них 15 являются спамом, а пять — нет. Таким образом, вероятность того, что электронное письмо — это спам, учитывая, что оно содержит слово «лотерея», составляет $15/20 = 0,75$.

Но мы уже знаем об этом, так в чем же польза диаграммы? Помимо упрощения преимущество заключается в том, что обычно информация, которой мы располагаем, основана на вероятностях, а не на количестве электронных писем.

Часто неизвестно, сколько среди писем спама, а сколько полезных. Все, что мы знаем, может выглядеть так.

- Вероятность того, что электронное письмо является спамом, равна $1/5 = 0,2$.
- Вероятность того, что спам-письмо содержит слово «лотерея», составляет $3/4 = 0,75$.
- Вероятность того, что полезное электронное письмо содержит слово «лотерея», составляет $1/16 = 0,0625$.
- **Вопрос:** какова вероятность того, что электронное письмо, содержащее слово «лотерея», — это спам?

Прежде всего давайте проверим, достаточно ли нам информации. Знаем ли мы вероятность того, что электронное письмо не является спамом? Мы знаем: вероятность того, что это спам, равна $1/5 = 0,2$. *Единственная* другая возможность заключается в том, что письмо полезное, поэтому вероятность должна быть дополнением, или $4/5 = 0,8$. Это важное правило — правило дополнительных вероятностей.

ПРАВИЛО ДОПОЛНИТЕЛЬНЫХ ВЕРОЯТНОСТЕЙ Дополнение к событию E , обозначаемое E^c , является событием, противоположным E . Вероятность E^c равна 1 минус вероятность E , а именно:

$$P(E^c) = 1 - P(E).$$

Таким образом:

- $P(\text{спам}) = 1/5 = 0,2$ — вероятность того, что электронное письмо является спамом;
- $P(\text{полезное}) = 4/5 = 0,8$ — вероятность того, что электронное письмо полезное.

Теперь обратимся к другой информации. Вероятность того, что спам-письмо содержит слово «лотерея», составляет $3/4 = 0,75$. Это можно интерпретировать так: вероятность того, что электронное письмо содержит слово «лотерея», *являясь при этом* спамом, составляет 0,75. Это условная вероятность, где условием является то, что электронное письмо — спам. Мы обозначаем условие вертикальной чертой, так что это можно записать как $P(\text{«лотерея»} \mid \text{спам})$. Дополнением послужит $P(\text{нет «лотереи»} \mid \text{спам})$, а именно вероятность того, что спам-письмо *не содержит* слова «лотерея». Эта вероятность равна $1 - P(\text{«лотерея»} \mid \text{спам})$. Таким образом, можем вычислить и другие вероятности:

- $P(\text{«лотерея»} \mid \text{спам}) = 3/4 = 0,75$ — вероятность того, что спам-письмо содержит слово «лотерея»;
- $P(\text{нет «лотереи»} \mid \text{спам}) = 1/4 = 0,25$ — вероятность того, что спам-письмо не содержит слова «лотерея»;
- $P(\text{«лотерея»} \mid \text{полезное}) = 1/16 = 0,0625$ — вероятность того, что полезное письмо содержит слово «лотерея»;
- $P(\text{нет «лотереи»} \mid \text{полезное}) = 15/16 = 0,9375$ — вероятность того, что полезное электронное письмо не содержит слова «лотерея».

Следующее, что мы делаем, — находим вероятности двух событий, происходящих *одновременно*. То есть нам нужны следующие четыре вероятности:

- вероятность того, что электронное письмо является спамом *и* содержит слово «лотерея»;
- вероятность того, что электронное письмо является спамом *и* не содержит слова «лотерея»;
- вероятность того, что электронное письмо является полезным *и* содержит слово «лотерея»;
- вероятность того, что электронное письмо является полезным *и* не содержит слова «лотерея».

Они называются *пересечениями* событий и обозначаются символом $\cap$. Таким образом, нам нужно найти следующие вероятности:

- $P(\text{«лотерея»} \cap \text{спам})$;
- $P(\text{нет «лотереи»} \cap \text{спам})$;
- $P(\text{«лотерея»} \cap \text{полезное})$;
- $P(\text{нет «лотереи»} \cap \text{полезное})$.

Взглянем на некоторые числа. Мы знаем, что $1/5$, или 20 из 100 электронных писем являются спамом. Из этих 20 слово «лотерея» содержат $3/4$. В конце мы перемножаем числа $1/5$ и $3/4$, чтобы получить $3/20$, что то же самое, что $15/100$, — доля электронных писем, которые являются спамом и содержат слово «лотерея». Мы сделали следующее: умножили вероятность того, что письмо является спамом, на вероятность того, что спам-письмо содержит слово «лотерея», и получили вероятность того, что электронное письмо является спамом *и* содержит слово «лотерея». Вероятность того, что спам-письмо содержит слово «лотерея», — это именно условная вероятность, или вероятность того, что электронное письмо содержит слово «лотерея», *учитывая прямо* это спам-письмо. Это подводит нас к правилу умножения вероятностей.

ПРАВИЛО УМНОЖЕНИЯ ВЕРОЯТНОСТЕЙ Вероятность пересечения событий E и F равна произведению условной вероятности F с учетом E и вероятности F , а именно, $P(E \cap F) = P(E | F) \cap P(F)$.

Теперь мы можем рассчитать эти вероятности следующим образом:

- $P(\text{«лотерея»} \cap \text{спам}) = P(\text{«лотерея»} | \text{спам}) \cap P(\text{спам}) = (3/4) \times (1/5) = 3/20 = 0,15$;
- $P(\text{нет «лотереи»} \cap \text{спам}) = P(\text{нет «лотереи»} | \text{спам}) \cap P(\text{спам}) = (1/4) \times (1/5) = 1/20 = 0,05$;
- $P(\text{нет «лотереи»} \cap \text{полезное}) = P(\text{нет «лотереи»} | \text{полезное}) \cap P(\text{полезное}) = (1/16) \times (4/5) = 1/20 = 0,05$;
- $P(\text{нет «лотереи»} \cap \text{полезное}) = P(\text{нет «лотереи»} | \text{полезное}) \cap P(\text{полезное}) = (15/16) \times (4/5) = 15/20 = 0,75$.

Эти вероятности суммированы на рис. 8.8. Обратите внимание на то, что произведение вероятностей по краям — это вероятности, указанные справа. Кроме того, сумма всех четырех вероятностей равна 1, поскольку они охватывают все возможные сценарии.

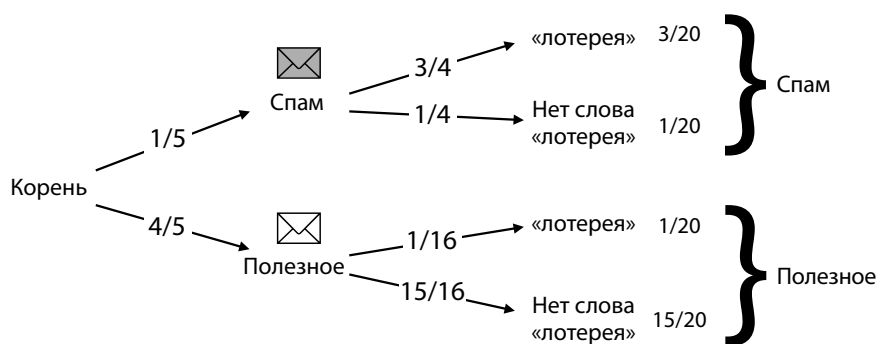


Рис. 8.8. То же дерево, что и на рис. 8.6, но уже с вероятностями

Почти закончили. Мы хотим найти $P(\text{спам} \mid \text{«лотерея»})$, представляющую собой вероятность того, что письмо является спамом, *учитывая*, что оно содержит слово «лотерея». Среди четырех рассмотренных событий оно встречается только в двух. Следовательно, нам нужны именно они:

- $P(\text{«лотерея»} \cap \text{спам}) = 3/20 = 0,15$;
- $P(\text{«лотерея»} \cap \text{полезное}) = 1/20 = 0,05$.

Другими словами, нас интересуют только две ветви, показанные на рис. 8.9, — первая и третья, то есть те, где электронное письмо содержит слово «лотерея».

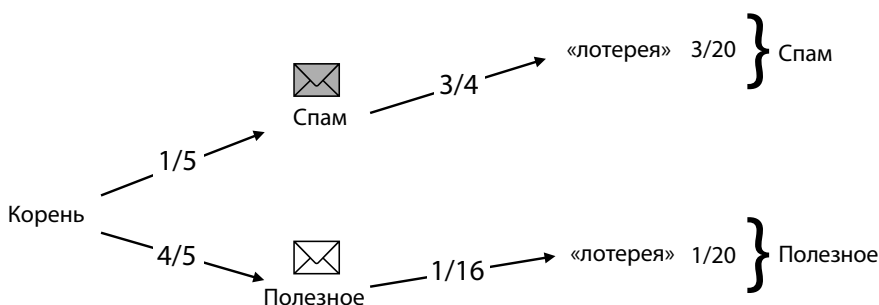


Рис. 8.9. Из дерева, приведенного на рис. 8.8, удалены две ветви, в которых письма не содержат слова «лотерея»

Первая — это вероятность того, что электронное письмо является спамом, а вторая — того, что оно полезное. Эти две вероятности не дают в сумме 1.

Однако, поскольку мы в данный момент находимся в мире, где электронные письма содержат слово «лотерея», то эти два варианта выступают единственно возможными сценариями. Таким образом, их вероятности *должны* в сумме давать 1. Кроме того, они по-прежнему должны обладать одинаковым соотношением по отношению друг к другу. Способ привести все в порядок — нормализация, то есть поиск двух чисел, которые находятся в том же соотношении друг к другу, что и $3/20$ и $1/20$, но в сумме дают 1.

Метод состоит в том, чтобы разделить оба числа на их сумму. В нашем случае это $\frac{3/20}{3/20+1/20}$ и $\frac{1/20}{3/20+1/20}$. Они упрощаются до $3/4$ и $1/4$, то есть до желаемых вероятностей. Итак, мы делаем вывод, что:

- $P(\text{спам} | \text{«лотерея»}) = 3/4 = 0,75$;
- $P(\text{полезное} | \text{«лотерея»}) = 1/4 = 0,25$.

Это именно то, что мы выяснили, когда считали электронные письма. Чтобы обобщить данную информацию, потребуется формула. У нас было две вероятности: вероятность того, что письмо является спамом *и* содержит слово «лотерея», и вероятность того, что письмо является спамом *и* не содержит слова «лотерея». Чтобы они давали в сумме 1, мы их нормализовали. Это то же самое, что разделить каждую из них на их сумму. На языке математики мы сделали следующее:

$$P(\text{спам} | \text{«лотерея»}) = \frac{P(\text{«лотерея»} \cap \text{спам})}{P(\text{«лотерея»} \cap \text{спам}) + P(\text{«лотерея»} \cap \text{полезное})}.$$

Если мы вспомним, какими были эти две вероятности, то, используя правило произведения, получим следующее:

$$\begin{aligned} P(\text{спам} | \text{«лотерея»}) &= \\ &= \frac{P(\text{«лотерея»} | \text{спам}) \times P(\text{спам})}{P(\text{«лотерея»} | \text{спам}) \cdot P(\text{спам}) + P(\text{«лотерея»} | \text{полезное}) \times P(\text{полезное})}. \end{aligned}$$

Для проверки подставляем числа и получаем:

$$P(\text{спам} | \text{«лотерея»}) = \frac{1/5 \cdot 3/4}{1/5 \cdot 3/4 + 4/5 \cdot 1/16} = \frac{3/20}{3/20 + 1/20} = \frac{3/20}{4/20} = \frac{3}{4} = 0,75.$$

Это и есть формула теоремы Байеса! Более формально она выглядит так.

ТЕОРЕМА БАЙЕСА Для событий E и F :

$$P(E | F) = \frac{P(F | E)P(E)}{P(F)}.$$

Поскольку событие F может быть разбито на два непересекающихся события: $F \cap E$ и $F \cap E^c$, то:

$$P(E | F) = \frac{P(F | E)P(E)}{P(F | E)P(E) + P(F | E^c)P(E^c)}.$$

А как быть, если слова два? Наивный байесовский алгоритм

В предыдущем разделе мы рассчитали вероятность того, что электронное письмо является спамом, с учетом содержания ключевого слова «лотерея». Однако слов в словаре гораздо больше, и нам может потребоваться рассчитать вероятность того, что электронное письмо является спамом, с учетом содержания нескольких слов. Понятно, что вычисления при этом усложнятся, но в данном разделе мы изучим способ, который поможет оценить эту вероятность.

Если коротко, то этот способ помогает вычислять апостериорную вероятность на основе двух событий вместо одного и легко обобщается на более чем два события. Он основан на той предпосылке, что, когда события независимы, вероятность того, что оба они произойдут одновременно, является произведением их вероятностей. События не всегда независимы, но предположение, что они независимы иногда, помогает делать хорошие приближения. Например, представьте себе остров с населением 1000 человек. Половина жителей (500) — женщины, а у 1/10 жителей (100) карие глаза. Как вы думаете, сколько среди жителей острова женщин с карими глазами? Если известно только это, нам не удастся узнать ответ, разве что пересчитать их. Однако если мы предположим, что пол и цвет глаз не зависят друг от друга, то сможем дать оценку, согласно которой половина из 1/10 населения состоит из женщин с карими глазами. Это составляет $1/2 \times 1/10 = 1/20$ населения. Поскольку общая численность равна 1000 человек, наша оценка числа женщин с карими глазами составит $1000 \times 1/20 = 50$ жителей. Может быть, когда мы отправимся на остров, то выясним, что это не совсем так, но, согласно имеющейся информации, 50 — это хорошая оценка. Кто-то скажет, что наше предположение о независимости пола и цвета глаз *наивно*, и возможно, так оно и есть, но это лучшая оценка, которую мы могли получить с учетом имеющейся информации.

Правило, использованное в предыдущем примере, — это правило произведения для независимых вероятностей.

ПРАВИЛО ПРОИЗВЕДЕНИЯ ДЛЯ НЕЗАВИСИМЫХ ВЕРОЯТНОСТЕЙ. Если события E и F независимы (возникновение одного никак не влияет на возникновение другого), то вероятностью того, что оба они произойдут (пересечение событий), будет произведение вероятностей каждого из событий. Другими словами:

$$P(E \cap F) = P(E) \times P(F).$$

Вернемся к примеру с электронной почтой. Определив вероятность того, что электронное письмо является спамом с учетом содержания слова «лотерея», мы заметили, что еще одно слово, «распродажа», также часто встречается в спам-письмах. Мы хотели бы найти вероятность того, что электронное письмо является спамом, с учетом того, что в нем есть как «лотерея», так и «распродажа». Начинаем с подсчета количества писем со спамом и полезных, содержащих слово «распродажа», и обнаруживаем, что оно встречается в шести из 20 писем со спамом и в четырех из 80 полезных. Таким образом, получаем следующие вероятности (показаны на рис. 8.10):

- $P(\text{«распродажа»} \mid \text{спам}) = 6/20 = 0,3$;
- $P(\text{«распродажа»} \mid \text{полезное}) = 4/80 = 0,05$.

Вероятность спама при наличии слова «распродажа» — 0,3



Рис. 8.10. В числе электронных писем, содержащих слово «распродажа» (не серые), шесть писем со спамом и четыре полезных

Можно снова применить теорему Байеса и прийти к такому выводу: вероятность того, что электронное письмо — спам, когда содержит слово «распродажа», равна 0,3. Я рекомендую вам провести расчеты самостоятельно. Но более важный вопрос заключается в следующем: какова вероятность того, что письмо содержит спам, если учесть, что в нем встречаются слова «лотерея» и «распродажа» одновременно? Прежде чем сделать это, давайте подсчитаем вероятность того, что письмо содержит слова «лотерея» и «распродажа», с учетом того, что это спам-письмо. Это легко: мы просматриваем все письма и подсчитываем, сколько спам-писем содержат слова «лотерея» и «распродажа».

Однако может возникнуть проблема, заключающаяся в том, что у нас вообще нет электронных писем со словами «лотерея» и «распродажа». Электронных писем всего 100, и, когда мы попытаемся найти в них эти два слова, у нас может оказаться недостаточно информации, чтобы правильно оценить вероятность. Что же делать? Одно из возможных решений — собирать данные до тех пор, пока у нас не окажется столько электронных писем, чтобы в некоторых из них появились эти два слова. Однако может случиться так, что мы будем не в силах собрать больше данных, так что придется работать с теми, что уже есть. Вот тут-то и поможет наивное предположение.

Попытаемся оценить эту вероятность таким же способом, как мы оценивали количество островитянок с карими глазами. Из предыдущего раздела мы знаем, что вероятность того, что слово «лотерея» появится в спам-письме, составляет 0,75. Как говорилось ранее, вероятность того, что в спаме появится слово «распродажа», равна 0,3. Таким образом, если мы наивно предположим, что появление этих двух слов независимо, вероятность появления обоих в спам-письме составит $0,75 \times 0,3 = 0,225$. Аналогично, поскольку мы вычислили, что вероятность появления в полезном письме слова «лотерея» равна 0,0625, а слова «распродажа» — 0,05, то вероятность полезного электронного письма, содержащего сразу оба, будет $0,0625 \times 0,05 = 0,003125$. Другими словами, мы провели следующие вычисления:

- $P(\text{«лотерея», «распродажа»} \mid \text{спам}) = P(\text{«лотерея»} \mid \text{спам}) P(\text{«распродажа»} \mid \text{спам}) = 0,75 \times 0,3 = 0,225$;
- $P(\text{«лотерея», «распродажа»} \mid \text{полезное}) = P(\text{«лотерея»} \mid \text{полезное}) P(\text{«распродажа»} \mid \text{полезное}) = 0,0625 \times 0,05 = 0,003125$.

Наивное предположение, которое мы сделали, звучит так.

НАИВНОЕ ПРЕДПОЛОЖЕНИЕ Слова появляются в электронном письме абсолютно независимо друг от друга. Другими словами, появление одного слова в письме никоим образом не влияет на появление другого.

Скорее всего, наивное предположение не соответствует действительности. Появление одного слова иногда способно очень сильно влиять на появление другого. Например, если электронное письмо содержит слово «соль», то в нем с большой вероятностью появится и слово «перец», потому что зачастую они встречаются вместе. Вот почему наше предположение наивно. Но оказывается, что оно не только хорошо работает на практике, но и значительно упрощает нашу математику. Это называется правилом произведения вероятностей, и проиллюстрировано оно на рис. 8.11. Предположим, что 20 % электронных писем содержат слово «лотерея», а 10 % — «распродажа». Мы делаем наивное предположение, что два слова независимы друг от друга. После этого процент электронных писем, содержащих оба слова, можно оценить как 2 %, а именно как произведение 20 и 10 %.

Теперь, когда у нас есть оценки вероятностей, переходим к поиску ожидаемого количества спама и полезных писем, содержащих слова «лотерея» и «распродажа».

- Поскольку в нашем распоряжении 20 спам-писем, а вероятность того, что спам-письмо содержит оба слова, равна 0,225, ожидаемое количество спам-писем, содержащих оба слова, составляет $20 \times 0,225 = 4,5$.
- Аналогично существует 80 полезных писем, и вероятность того, что такое письмо содержит оба слова, равна 0,003125, поэтому ожидаемое количество полезных писем, содержащих оба слова, составляет $80 \times 0,003125 = 0,25$.

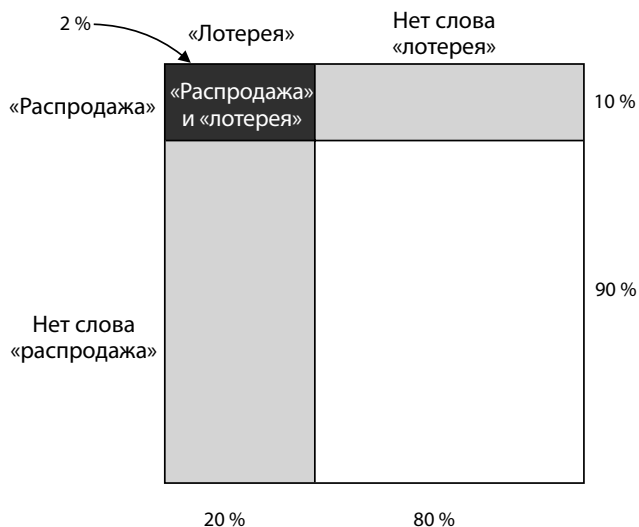


Рис. 8.11. Правило произведения вероятностей

Расчеты подразумевают, что, если мы ограничим набор данных только электронными письмами, содержащими слова «лотерея» и «распродажа», можно ожидать, что 4,5 из них окажутся спамом, а 0,25 — полезными. Таким образом, если нам требуется случайным образом выбрать одно из них, то какова вероятность того, что мы выберем письмо, содержащее спам? Может показаться, что с целыми числами все гораздо проще, но если взглянуть на рис. 8.12, то многое проясняется. У нас 4,5 спам-письма и 0,25 полезного (это ровно 1/4 электронного письма). Если мы бросим дротик и он попадет в одно из электронных писем, то какова вероятность того, что это будет спам-письмо? Итак, общее количество электронных писем (или общая площадь, если вам так проще) составляет $4,5 + 0,25 = 4,75$. Поскольку 4,5 — это спам, то вероятность того, что дротик попал в спам, равна $4,5 / 4,75 = 0,9474$. Это означает, что электронное письмо со словами «лотерея» и «распродажа» с вероятностью 94,74 % — спам. Это довольно высокая вероятность!

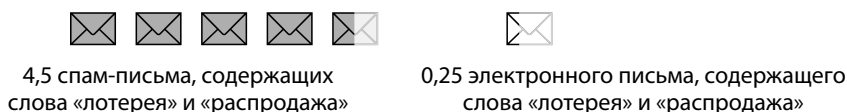


Рис. 8.12. У нас есть 4,5 спам-письма и 0,25 полезного. Мы бросаем дротик, и он попадает в одно из писем. Какова вероятность того, что он попал в спам-письмо? Ответ — 94,74 %

То, что мы только что проделали с вероятностью, — это и есть использование теоремы Байеса (за исключением событий $E = \text{«лотерея»} \cap \text{«распродажа»}$ и $F = \text{спам}$), чтобы получить формулу:

$$P(\text{спам} | \text{«лотерея»} \cap \text{«распродажа»}) = \frac{P(\text{«лотерея»} \cap \text{«распродажа»} | \text{спам}) P(\text{спам})}{P(\text{«лотерея»} \cap \text{«распродажа»} | \text{спам}) P(\text{спам}) + P(\text{«лотерея»} \cap \text{«распродажа»} | \text{полезное}) P(\text{полезное})}.$$

Затем мы (наивно) предположили, что появление слов «лотерея» и «распродажа» не зависит от спама (и полезности) в электронных письмах, чтобы получить следующие две формулы:

$$P(\text{«лотерея»} \cap \text{«распродажа»} | \text{спам}) = P(\text{«лотерея»} | \text{спам}) P(\text{«распродажа»} | \text{спам});$$

$$P(\text{«лотерея»} \cap \text{«распродажа»} | \text{полезное}) = P(\text{«лотерея»} | \text{полезное}) P(\text{«распродажа»} | \text{полезное}).$$

Подставив их в предыдущую формулу, получим:

$$P(\text{спам} | \text{«лотерея»} \cap \text{«распродажа»}) = \frac{P(\text{«лотерея»} | \text{спам}) P(\text{«распродажа»} | \text{спам}) P(\text{спам})}{P(\text{«лотерея»} | \text{спам}) P(\text{«лотерея»} | \text{спам}) P(\text{спам}) + P(\text{«лотерея»} | \text{полез.}) P(\text{«распродажа»} | \text{полез.}) P(\text{полез.})}.$$

И наконец, подставим следующие значения:

- $P(\text{«лотерея»} | \text{спам}) = \frac{3}{4};$
- $P(\text{«распродажа»} | \text{спам}) = \frac{3}{10};$
- $P(\text{спам}) = \frac{1}{5};$
- $P(\text{«лотерея»} | \text{полезное}) = \frac{1}{16};$
- $P(\text{«распродажа»} | \text{полезное}) = \frac{1}{20};$
- $P(\text{полезное}) = \frac{4}{5}.$

Получаем:

$$P(\text{спам} | \text{«лотерея»} \cap \text{«распродажа»}) = \frac{\frac{3}{4} \times \frac{3}{10} \times \frac{1}{5}}{\frac{3}{4} \times \frac{3}{10} \times \frac{1}{5} + \frac{1}{16} \times \frac{1}{20} \times \frac{4}{5}} = 0,9474.$$

Но что, если у нас больше двух слов?

В общем случае электронное письмо содержит n слов, $x_1, x_2, \dots, x_n$. Теорема Байеса утверждает, что вероятность того, что электронное письмо является спамом, учитывая, что оно содержит слова $x_1, x_2, \dots, x_n$, равна:

$$P(\text{спам} | x_1, x_2, \dots, x_n) = \frac{P(x_1, x_2, \dots, x_n | \text{спам})P(\text{спам})}{P(x_1, x_2, \dots, x_n | \text{спам})P(\text{спам}) + P(x_1, x_2, \dots, x_n | \text{полезное})P(\text{полезное})}.$$

В предыдущем уравнении мы удалили знак пересечения и заменили его запятой. Наивное предположение состоит в том, что появление всех этих слов независимо. Следовательно,

$$P(x_1, x_2, \dots, x_n | \text{спам}) = P(x_1 | \text{спам})P(x_2 | \text{спам}) \cdots P(x_n | \text{спам})$$

и

$$P(x_1, x_2, \dots, x_n | \text{полезное}) = P(x_1 | \text{полезное})P(x_2 | \text{полезное}) \cdots P(x_n | \text{полезное}).$$

Объединив последние три уравнения, получаем:

$$P(\text{спам} | x_1, x_2, \dots, x_n) = \frac{P(x_1 | \text{спам})P(x_2 | \text{спам}) \cdots P(x_n | \text{спам})P(\text{спам})}{P(x_1 | \text{спам})P(x_2 | \text{спам}) \cdots P(x_n | \text{спам})P(\text{спам}) + P(x_1 | \text{полез.})P(x_2 | \text{полез.}) \cdots P(x_n | \text{полез.})P(\text{полез.})}.$$

Каждое из значений в правой части можно расценивать как соотношение между количеством электронных писем. Например, $P(x_i | \text{спам})$ — это соотношение между количеством спам-писем, содержащих слово x_i , и общим их количеством.

В качестве небольшого примера предположим, что электронное письмо содержит слова «лотерея», «распродажа» и «мама». Мы берем слово «мама» и обращаем внимание на то, что оно встречается только в одном из 20 спам-писем и в 10 из 80 полезных. Следовательно, $P(\text{«мама»} | \text{спам}) = 1/20$ и $P(\text{«мама»} | \text{полезное}) = 1/8$. Используя те же вероятности для слов «лотерея» и «распродажа», что и в предыдущем разделе, мы получаем следующее:

$$\begin{aligned} P(\text{спам} | \text{«лотерея», «распродажа», «мама»}) &= \\ &= \frac{P(\text{«лотерея»} | \text{спам})P(\text{«распродажа»} | \text{спам})P(\text{«мама»} | \text{спам})P(\text{спам})}{P(\text{«лот.»} | \text{спам})P(\text{«распр.»} | \text{спам})P(\text{«мама»} | \text{спам})P(\text{спам}) + P(\text{«лот.»} | \text{полез.})P(\text{«распр.»} | \text{полез.})P(\text{«мама»} | \text{полез.})P(\text{полез.})} = \\ &= \frac{\frac{3}{4} \times \frac{3}{10} \times \frac{1}{20} \times \frac{1}{5}}{\frac{3}{4} \times \frac{3}{10} \times \frac{1}{20} \times \frac{1}{5} + \frac{1}{16} \times \frac{1}{20} \times \frac{1}{8} \times \frac{4}{5}} = 0,878. \end{aligned}$$

Обратите внимание на то, что добавление в уравнение слова «мама» уменьшило вероятность спама с 94,74 до 87,8 %, что имеет смысл, поскольку это слово с большей вероятностью встречается в полезных электронных письмах.

Построение модели спам-фильтра на основе реальных данных

Теперь, когда мы разработали наивный байесовский алгоритм, давайте засучим рукава и закодируем его. Ряд библиотек, таких как Scikit-Learn, имеет отличные реализации этого алгоритма, и я рекомендую вам с ними ознакомиться. Однако мы реализуем его вручную. Набор данных взят из Kaggle, и для получения ссылки на его загрузку, пожалуйста, ознакомьтесь с ресурсами для этой главы в приложении В. Вот код для этого раздела.

- **Блокнот** Coding_naive_Bayes.ipynb
https://github.com/luisguiserrano/manning/blob/master/Chapter_8_Naive_Bayes/Coding_naive_Bayes.ipynb
- **Набор данных** emails.csv.

В этом примере мы представим полезный пакет для обработки больших наборов данных под названием Pandas (чтобы узнать о нем больше, ознакомьтесь с разделом «Использование Pandas для загрузки набора данных» в главе 13). Основным объектом, применяемым для хранения наборов данных в Pandas, служит фрейм данных. Чтобы загрузить в него наши данные, применим следующую команду:

```
import pandas
emails = pandas.read_csv('emails.csv')
```

В табл. 8.1 можно увидеть первые пять строк набора данных.

Таблица 8.1. Первые строки набора данных электронной почты

Текст	Спам
Subject: naturally irresistible your corporate... (Тема: исключительность вашей корпорации)	1
Subject: the stock trading gunslinger fanny i... (Тема: биржевая торговля...)	1
Subject: unbelievable new homes made easy im... (Тема: потрясающие новые дома...)	1
Subject: 4 color printing special request add... (Тема: четырехцветная печать, специальный запрос...)	1
Subject: do not have money, get software cds... (Тема: нет денег, купите диски с ПО...)	1

Набор данных состоит из двух столбцов. Первый столбец — это текст электронного письма (вместе с темой) в строковом формате. Во втором столбце указывается, спам это (1) или полезное письмо (0). Для начала выполним кое-какую предварительную обработку данных.

Предварительная обработка данных

Начнем с преобразования текстовой строки в список слов. Прделаем это с помощью функции, которая использует `lower()` для перевода всех слов в нижний регистр и `split()` для преобразования слов в список. Мы лишь проверяем, появляется ли то или иное слово в электронном письме (без учета количества), поэтому переводим его в набор, а затем в список:

```
def process_email(text):
    text = text.lower()
    return list(set(text.split()))
```

Далее используем функцию `apply()` и применим это изменение ко всему столбцу. Назовем новую колонку `emails['words']`:

```
emails['words'] = emails['text'].apply(process_email)
```

Первые пять строк измененного набора данных почты показаны в табл. 8.2.

Таблица 8.2. Набор данных электронной почты с новым столбцом «Слова», который содержит список слов электронного письма (без повторов) и строку темы

Текст	Спам	Слова
Subject: naturally irresistible your corporate...	1	[letsyou, all, do, but, list, is, information,...]
Subject: the stock trading gunslinger fanny i...	1	[not, like, duane, trading, libretto, attainde...]
Subject: unbelievable new homes made easy im...	1	[im, have, \$, take, foward, all, limited, subj...]
Subject: 4 color printing special request add...	1	[color, azusa, pdf, printable, 8102, subject:,...]
Subject: do not have money, get software cds...	1	[get, not, have, all, do, subject:, be, by, me...]

Поиск априорных вероятностей

Сначала найдем вероятность того, что электронное письмо — спам (априорную). Для этого подсчитаем количество писем, которые являются спамом, и разделим его на общее их количество. Обратите внимание на то, что

количество спам-писем равно сумме записей в столбце **Спам**. Следующая строка выполняет эту задачу:

```
sum(emails['spam'])/len(emails)
0.2388268156424581
```

Мы пришли к выводу, что априорная вероятность того, что электронное письмо — это спам, составляет около 0,24. Это вероятность того, что электронное письмо является спамом, когда мы ничего не знаем о нем. Аналогично априорная вероятность того, что электронное письмо полезное, — около 0,76.

Нахождение апостериорной вероятности с помощью теоремы Байеса

Нам нужно найти вероятность того, что спам (и полезные) электронные письма содержат определенное слово. Прделаем это для всех слов одновременно. Следующая функция создает словарь `model`, который записывает каждое слово рядом с количеством его появлений в спам- и полезных письмах:

```
model = {}

for index, email in emails.iterrows():
    for word in email['words']:
        if word not in model:
            model[word] = {'spam': 1, 'ham': 1}
        if word in model:
            if email['spam']:
                model[word]['spam'] += 1
            else:
                model[word]['ham'] += 1
```

Обратите внимание на то, что количество инициализируется как 1, поэтому на самом деле мы регистрируем на одно письмо больше. Мы используем этот трюк, чтобы избежать нулевого количества и случайно не разделить на ноль. Теперь изучим часть строк словаря:

```
odel['lottery']
{'ham': 1, 'spam': 9}

model['sale']
{'ham': 42, 'spam': 39}
```

Это означает, что слово «лотерея» появляется в одном полезном электронном письме и девяти спам-письмах, в то время как слово «распродажа» — в 42 полезных электронных письмах и 39 спам-письмах. Хотя этот словарь не содержит никаких вероятностей, их можно вывести, разделив первую запись на сумму обеих записей. Таким образом, если электронное письмо содержит слово «лотерея», вероятность того, что оно является спамом, равна $9/(9 + 1) = 0,9$, а если в нем есть слово «распродажа», то $39/(39 + 42) = 0,48$.

Реализация наивного байесовского алгоритма

На вход алгоритма поступает электронная почта. Он просматривает все слова в электронном письме и для каждого определяет вероятность того, что оно содержится и в спаме, и в полезном письме. Вероятности вычисляются с использованием словаря, который задан в предыдущем разделе. Затем мы перемножаем эти вероятности (наивное предположение) и применяем теорему Байеса, чтобы найти вероятность того, что электронное письмо является спамом (с учетом того, что эти слова в конкретном письме содержатся). Код прогноза с применением данной модели выглядит следующим образом:

```
def predict_naive_bayes(email):
    total = len(emails)  # ← Вычисляем общее количество электронных писем,
    # ← нежелательных и полезных
    num_spam = sum(emails['spam'])
    num_ham = total - num_spam
    email = email.lower()  # ← Обрабатываем каждое электронное письмо,
    # ← превращая его в список слов в нижнем регистре
    words = set(email.split())
    spams = [1.0]
    hams = [1.0]
    for word in words:
        if word in model:
            spams.append(model[word]['spam']/num_spam*total)  # ←
            hams.append(model[word]['ham']/num_ham*total)
    prod_spams = np.long(np.prod(spams)*num_spam)  # ←
    prod_hams = np.long(np.prod(hams)*num_ham)
    # ← Для каждого слова вычисляем условную
    # ← вероятность того, что содержащее его
    # ← электронное письмо является спамом
    # ← (или полезным)
    return prod_spams/(prod_spams + prod_hams)  # ←
    # ← Перемножаем все предыдущие вероятности,
    # ← умноженные на априорную вероятность
    # ← того, что электронное письмо является
    # ← спамом, и называем все это prod_spams.
    # ← Выполняем аналогичный процесс для prod_hams
```

Нормализуем эти две вероятности, чтобы они составляли единицу (с помощью теоремы Байеса), и возвращаем результат

Вы можете заметить, что в коде мы проделали еще один небольшой фокус. Каждая вероятность умножается на общее количество электронных писем в наборе данных. Это влияет на расчеты, потому что этот коэффициент появляется и в числителе, и в знаменателе. Однако это гарантирует, что полученные произведения вероятностей не окажутся слишком малы для обработки их в Python.

Теперь, когда мы построили модель, протестируем ее, сделав прогнозы по некоторым электронным письмам:

```
predict_naive_bayes('Hi mom how are you')
0.12554358867163865
```

```
predict_naive_bayes('meet me at the lobby of the hotel at nine am')
0.00006964603508395
```

```
predict_naive_bayes('buy cheap lottery easy money now')
0.9999734722659664
```

```
predict_naive_bayes('asdfgh')
0.2388268156424581
```

Кажется, работает. Письма типа «Привет, мама, как дела?» с низкой вероятностью (около 0,12) являются спамом, а письма типа «Купите дешевые лотерейные билеты, легкие деньги прямо сейчас» с очень высокой вероятностью (более 0,99) представляют собой спам. Обратите внимание на то, что последнее электронное письмо, не содержащее ни одного слова из словаря, получает вероятность 0,2388, которая в точности соответствует априорной.

Что дальше

Мы увидели быструю реализацию наивного алгоритма Байеса. Но для крупных наборов данных и больших электронных писем следует использовать библиотеку. Такие пакеты, как Scikit-Learn, предлагают отличные реализации наивного байесовского алгоритма со множеством параметров тонкой настройки. Изучите этот и другие пакеты и примените наивный алгоритм Байеса для всех типов наборов данных!

Резюме

- Теорема Байеса — это метод, широко используемый в теории вероятностей, статистике и машинном обучении.
- Теорема Байеса состоит в вычислении апостериорной вероятности, основанной на априорной вероятности и событии.
- Априорная вероятность — это базовое вычисление вероятности с учетом очень небольшого количества информации.
- Теорема Байеса использует событие для гораздо лучшей оценки рассматриваемой вероятности.
- Наивный алгоритм Байеса применяется, когда требуется объединить априорную вероятность с несколькими событиями.
- Появление слова «наивный» вызвано тем, что мы делаем наивное предположение, а именно, что все рассматриваемые события независимы.

Упражнения

Упражнение 8.1

Для каждой пары событий А и В определите, зависимые они или независимые. Для п. А–Г приведите математическое обоснование. Для п. Д и Е дайте устное обоснование.

Честно три раза подбрасываем монету.

А: первый бросок — орел. В: третий бросок — решка.

А: первый бросок — орел. В: среди трех бросков нечетное количество — орел.

Выбрасывание двух кубиков.

А: на первом кубике 1. Б: на втором кубике 2.

А: на первом кубике 3. Б: на втором выпало более высокое значение, чем на первом.

Для следующего примера приведите устное обоснование. Предположим, мы живем там, где есть смена времен года.

А: на улице идет дождь. Б: сегодня понедельник.

А: на улице идет дождь. Б: сейчас июнь.

Упражнение 8.2

Представим офис, куда мы регулярно ходим за рабочими бумагами. В офисе работают два клерка, Аиша и Бето. Мы знаем, что Аиша работает там три дня в неделю, а Бето — два оставшихся. Однако расписание меняется каждую неделю, поэтому мы никогда не знаем, в какие три дня на месте Аиша, а в какие два — Бето.

А. Если мы придем в офис в случайный день, какова вероятность того, что клерком будет Аиша?

Мы смотрим в окно и замечаем, что клерк одет в красный свитер, хотя не можем точно рассмотреть, кто именно. Мы часто посещаем этот офис, поэтому знаем, что Бето чаще носит красное, чем Аиша. На самом деле Аиша носит красное в один день из трех (треть времени), а Бето — в один день из двух (половина времени).

Б. Какова вероятность того, что сегодня работает Аиша, при условии, что клерк одет в красное?

Упражнение 8.3

Возьмем данные о пациентах с положительным или отрицательным тестом на COVID-19. У них следующие симптомы: кашель (С), температура (F), затрудненное дыхание (В) и утомляемость (Т).

Пациент	Кашель (С)	Температура (F)	Затрудненное дыхание (В)	Утомляемость (Т)	Диагноз
1		X	X	X	Болен
2	X	X		X	Болен
3	X		X	X	Болен
4	X	X	X		Болен
5	X			X	Здоров
6		X	X		Здоров
7		X			Здоров
8				X	Здоров

Цель этого упражнения — построить наивную байесовскую модель, которая предсказывает диагноз по симптомам. Используйте наивный алгоритм Байеса, чтобы найти следующие вероятности.

ОБРАТИТЕ ВНИМАНИЕ Для последующих вопросов: симптомы, которые не упомянуты, нам совершенно неизвестны. Например, если мы знаем, что у пациента кашель, но ничего не говорится о жаре, это не значит, что у пациента нет жара.

- А. Вероятность того, что пациент болен, учитывая, что у него кашель.
- Б. Вероятность того, что пациент болен, учитывая, что он не утомлен.
- В. Вероятность того, что пациент болен, учитывая, что у него кашель и жар.
- Г. Вероятность того, что пациент болен, учитывая, что у него кашель и жар, но нет затрудненного дыхания.

Разбиение данных согласно ответам на вопросы. Деревья решений

9



В этой главе

- ✓ Что такое дерево решений.
 - ✓ Применение деревьев решений для классификации и регрессии.
 - ✓ Создание системы рекомендаций приложений с помощью информации пользователей.
 - ✓ Достоверность, индекс Джини и энтропия, а также их роль в построении деревьев решений.
 - ✓ Применение Scikit-Learn для обучения дерева решений на наборе данных о поступлении в университет.
-

USER FRIENDLY by Illiad



В этой главе мы изучим деревья принятия решений. Деревья решений — это мощные модели классификации и регрессии, которые дают много дополнительной информации о наборе данных. Как и предыдущие модели, которые мы изучали в этой книге, деревья решений обучаются с помощью размеченных данных, где прогнозируемые метки могут быть классами (для классификации) или значениями (для регрессии). Большую часть этой главы мы фокусируемся на деревьях решений для классификации, но ближе к концу главы обратимся к деревьям решений для регрессии. Однако структура и процесс обучения обоих типов деревьев схожи. В данной главе мы разработаем несколько вариантов их использования, включая систему рекомендаций приложений и модель прогнозирования поступления в университет.

Деревья принятия решений следуют интуитивному процессу составления прогнозов, который очень напоминает человеческие рассуждения. Рассмотрим следующий сценарий: мы хотим решить, стоит ли нам сегодня надевать куртку. Как выглядит процесс принятия решения? Можно выглянуть на улицу и проверить, не идет ли дождь. Если идет, то мы обязательно надеваем куртку. Если это не так, то можно проверить температуру. Если жарко, то мы не надеваем куртку, а если холодно — надеваем. На рис. 9.1 приведен график процесса принятия решений, где решения принимаются путем обхода дерева сверху вниз и выбора ветви, соответствующей правильному ответу.

Процесс принятия решений похож на дерево, за исключением того, что оно перевернуто. Дерево состоит из вершин, называемых *узлами*, и ребер. На самом верху расположен *корневой узел*, от которого отходят две ветви. Каждый из узлов имеет либо две, либо ни одной ветви (ребра), отходящих от него, и по этой причине такое дерево называется *двоичным*. Узлы, от которых исходят две ветви, называются *узлами принятия решений*, а от которых ни одной — *конечными узлами*, или *листьями*. Такое расположение узлов, листьев и ребер и есть то, что мы называем деревом решений. Деревья — это обычные объекты в информатике, потому что компьютеры разбивают каждый процесс на последовательность двоичных операций.

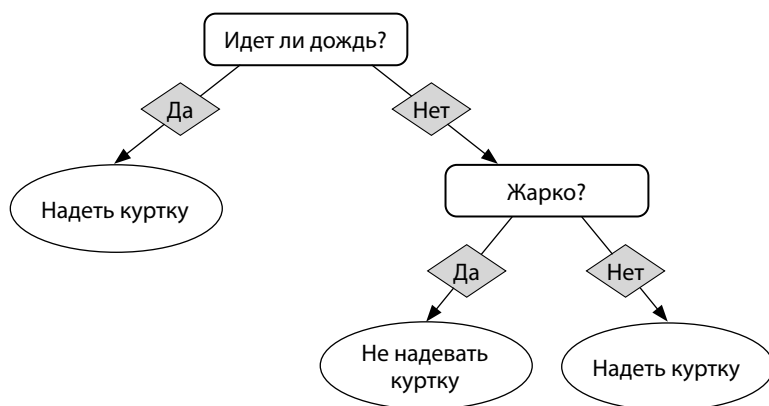


Рис. 9.1. Дерево решений для принятия решения о необходимости надеть куртку

Простейшее из возможных деревьев решений — *пенек решений* — формируется одним узлом принятия решений (корневым узлом) и двумя листьями. Оно представляет собой единственный вопрос, требующий ответа «да» или «нет», на основе которого мы немедленно принимаем решение.

Глубина дерева решений — это количество уровней под корневым узлом. Другой способ измерения — вычислить длину самого длинного пути от корневого узла до листа, где путь измеряется количеством содержащихся ребер. Дерево на рис. 9.1 имеет глубину 2. Пенек решений имеет глубину 1.

Вот краткое изложение всех определений, с которыми мы познакомились до этого момента.

ДЕРЕВО РЕШЕНИЙ — модель машинного обучения, основанная на вопросах, требующих ответа типа «да» или «нет», и представленная двоичным деревом. В дереве имеются корневой узел, узлы принятия решений, конечные узлы и ветви.

КОРНЕВОЙ УЗЕЛ — самый верхний узел дерева. Он содержит первый из вопросов, требующих ответа «да» или «нет». Для удобства называем его корнем.

УЗЕЛ ПРИНЯТИЯ РЕШЕНИЯ — каждый вопрос, на который нужно ответить «да» или «нет», в нашей модели представлен узлом принятия решения, от которого отходят две ветви — одна для «да» и одна для «нет».

КОНЕЧНЫЙ УЗЕЛ — узел, от которого не отходит ни одной ветви. Такие узлы представляют собой решения, которые мы принимаем после обхода дерева. Мы называем их листьями.

ВЕТВЬ — ребра, выходящие из каждого узла принятия решения и соответствующие ответам «да» и «нет» на вопрос, заданный в узле. В этой главе принято, что ответвление слева соответствует «да», справа — «нет».

ГЛУБИНА — количество уровней в дереве решений, или, иначе, количество ветвей на самом длинном пути от корневого узла к конечному.

На протяжении всей главы узлы изображаются в виде прямоугольников со скругленными краями, ответы в ветвях — в виде ромбов, а листья — в виде овалов. На рис. 9.2 показано, как в общем случае выглядит дерево решений. Каждый узел принятия решения содержит вопрос, требующий ответа «да» или «нет». Из каждого возможного ответа исходит одна ветвь, которая может привести к другому узлу принятия решения или же листу. Дерево имеет глубину 2, потому что самый длинный путь от листа до корня проходит через две ветви.

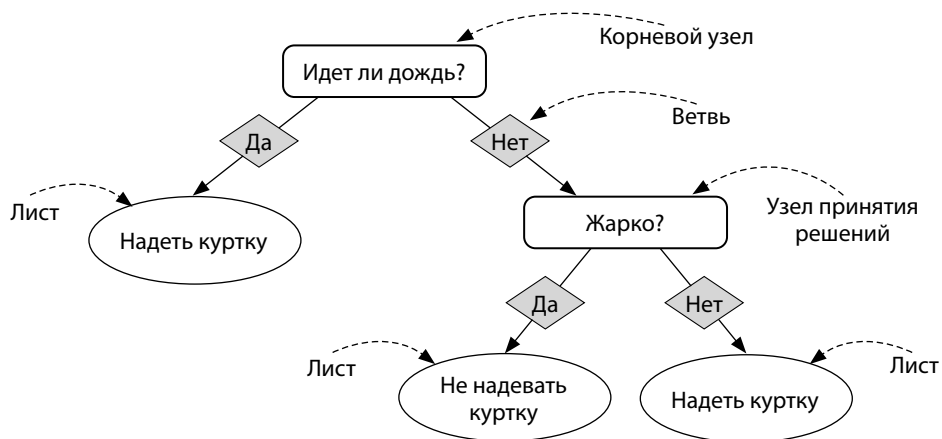


Рис. 9.2. Обычное дерево решений с корневым узлом, узлами принятия решений, ветвями и листьями

Как мы построили это дерево? Почему задавали именно эти вопросы? Можно также проверить, не понедельник ли сегодня, не видели ли мы красную машину на улице и не хочется ли нам есть, после чего построить следующее дерево решений (рис. 9.3).

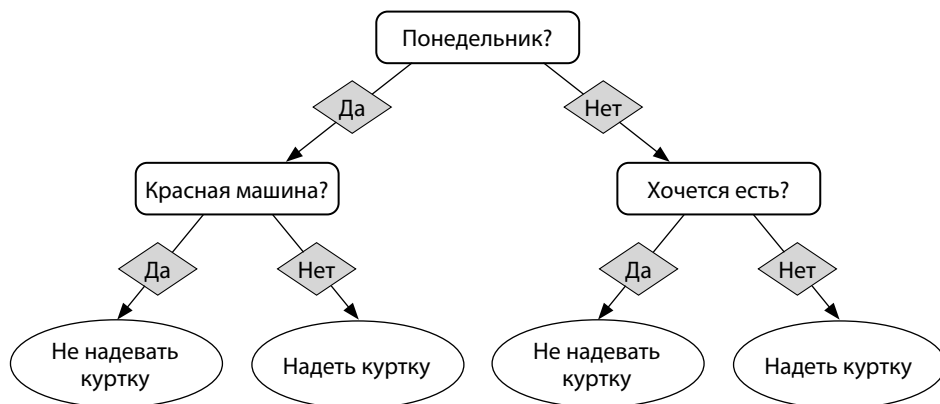


Рис. 9.3. Второе (возможно, не такое хорошее) дерево решений, позволяющее решить, нужно ли надевать куртку в определенный день

Какое же дерево более пригодно для принятия решения о том, надевать ли куртку: дерево 1 (см. рис. 9.1) или дерево 2 (см. рис. 9.3)? Нашего жизненного опыта хватает, чтобы понять, что для этой задачи дерево 1 намного лучше, чем дерево 2. Но откуда это знать компьютеру? Машины не обладают опытом как таковым, но у них есть кое-что похожее — данные. Если бы мы захотели мыслить как компьютер, то просто перебрали бы все возможные деревья, попробовали каждое из них в течение некоторого времени — скажем, одного года — и сравнили, подсчитав, сколько раз каждое дерево давало правильное решение. Предположим, что при использовании дерева 1 мы были бы правы большую часть дней, тогда как, применяя дерево 2, мы оказались бы вынуждены мерзнуть в холодный день без куртки или надевать ее в жару. Все, что нужно сделать компьютеру, — просмотреть все деревья, собрать данные и выяснить, какое из них лучше. Верно?

Почти! К сожалению, у компьютера поиск наиболее эффективного дерева занял бы очень много времени. Но, к счастью, имеются алгоритмы, которые существенно ускоряют его. Благодаря им мы можем использовать деревья решений во множестве областей, включая обнаружение спама, анализ настроений и медицинскую диагностику. В этой главе рассмотрим алгоритм быстрого построения хороших деревьев решений. Если коротко, то мы строим дерево по одному узлу за раз, начиная с вершины. Чтобы выбрать правильный вопрос, соответствующий каждому узлу, мы перебираем все возможные вопросы и выбираем тот, который оказывается правильным наибольшее количество раз. Процесс протекает следующим образом.

Выбор хорошего первого вопроса

Требуется выбрать хороший первый вопрос для корня дерева. Какой хороший вопрос поможет нам решить, надевать ли куртку в тот или иной день? Изначально он может быть любым. Допустим, у нас есть несколько вариантов первого вопроса.

1. Идет ли дождь?
2. На улице холодно?
3. Я голоден?
4. Стоит ли на улице красная машина?
5. Сегодня понедельник?

Какой из пяти вопросов, по-вашему мнению, наилучшим образом помогает решить, стоит ли надеть куртку? Интуиция подсказывает, что последние три вопроса в нашем случае бесполезны и никак не помогают принять решение. Допустим, исходя из опыта, мы заметили, что из первых двух более полезен первый. Его мы и используем, чтобы начать строить дерево. Пока что у нас есть простое дерево решений, или пенек решений, состоящий из этого единственного вопроса (рис. 9.4).

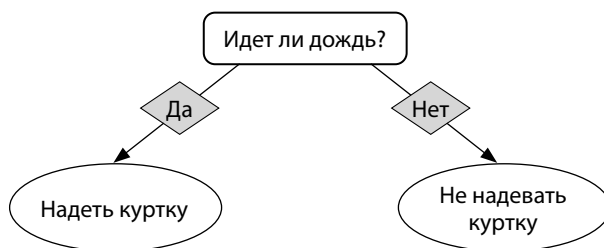


Рис. 9.4. Пенек решений, состоящий из вопроса: «Идет ли дождь?»
Если ответ «да», то мы решаем надеть куртку

Можно ли улучшить и его? Представьте, что мы начинаем замечать, что надетая во время дождя куртка — всегда верное решение. Однако бывают дни, когда дождя нет, а не надевать куртку — неправильное решение. Вот тут-то нам на помощь и приходит вопрос 2. Мы используем его следующим образом: убедившись, что дождя нет, проверяем температуру и, если холодно, решаем надеть куртку. Это превращает левый лист дерева в узел с двумя отходящими от него листьями (рис. 9.5).

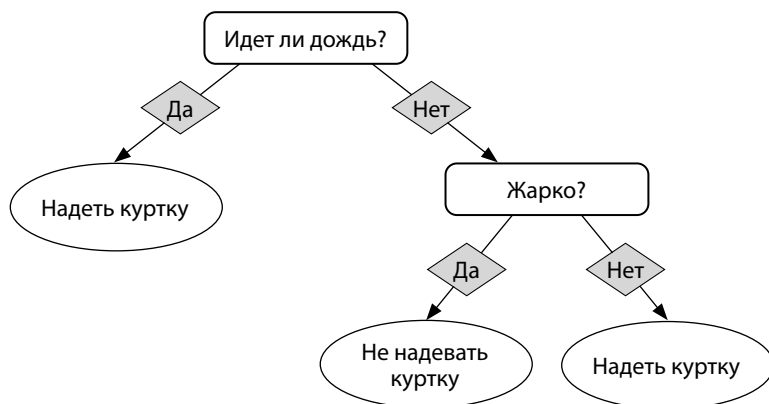


Рис. 9.5. Немного более сложное дерево решений, чем на рис. 9.4. Мы выбрали один лист и разделили его еще на два. Это то же самое дерево, что и на рис. 9.1

Теперь у нас есть дерево решений. Можно ли улучшить и его? Может быть, если добавить в него больше узлов и листьев. Но на данный момент этот вариант работает очень хорошо. В этом примере мы принимали решения, используя интуицию и опыт. В данной главе изучим алгоритм, который строит подобные деревья исключительно на основе данных.

У вас может возникнуть много вопросов, например:

1. Как именно решить, какой вопрос лучше всего задать?
2. Действительно ли процесс постоянного выбора наилучшего возможного вопроса позволяет нам строить *лучшее* дерево решений?

3. Почему бы вместо этого не построить все возможные деревья решений и не выбрать наилучшее?
4. Будем ли мы кодировать этот алгоритм?
5. Где можно встретить деревья решений в реальной жизни?
6. Вполне понятно, как деревья решений работают для классификации, но как они работают для регрессии?

В этой главе даются ответы на все эти вопросы, а здесь приведем несколько кратких ответов.

1. Как именно решить, какой вопрос лучше всего задать?

Есть несколько способов. Самый простой — использовать достоверность: какой вопрос помогает чаще оказываться правым? Но мы изучим и другие методы, такие как индекс Джини или энтропия.

2. Действительно ли процесс постоянного выбора наилучшего возможного вопроса поможет строить лучшее дерево решений?

На самом деле он не гарантирует получения наилучшего из возможных деревьев. Это называется *жадным алгоритмом*. Жадные алгоритмы работают следующим образом: в каждой точке алгоритм делает наилучший возможный шаг. Как правило, такие алгоритмы работают хорошо. Тем не менее наилучшие возможные ходы на конкретных этапах не всегда дают наилучший общий результат. Может случиться, что более слабый вопрос группирует данные таким образом, что в итоге получается лучшее дерево. Но в целом нужно отметить, что алгоритмы построения деревьев решений, как правило, работают быстро и эффективно. Изучите алгоритмы, которые рассматриваются в этой главе, и попытайтесь найти способы улучшить их, удалив жадность!

3. Почему бы вместо этого не построить все возможные деревья решений и не выбрать наилучшее?

Число возможных деревьев решений очень велико, особенно если набор данных содержит множество признаков. Просмотр всех — это очень долгий процесс. Однако для поиска каждого узла требуется только линейный поиск по признакам, а не по всем возможным деревьям, что делает его намного быстрее.

4. Будем ли мы кодировать этот алгоритм?

Его вполне можно закодировать вручную. Однако мы увидим, что из-за рекурсии кодирование может оказаться несколько трудоемким. Так что для построения деревьев решений с реальными данными будем использовать полезную библиотеку под названием Scikit-Learn.

5. Где можно встретить деревья решений в реальной жизни?

Во множестве областей! Они широко используются в машинном обучении не только потому, что очень хорошо работают, но и потому, что дают много информации о данных. Перечислим некоторые области, в которых задействуются деревья решений: системы рекомендаций

(для рекомендации видео, фильмов, приложений, продуктов для покупки и т. д.), спам-фильтры (чтобы понять, является ли электронное письмо спамом), анализ настроений (чтобы решить, является ли предложение радостным или грустным) и биология (чтобы понять, болен ли пациент, или даже выявить определенные иерархии в видах или типах геномов).

6. Вполне понятно, как деревья решений работают для классификации, но как они работают для регрессии?

Дерево решений регрессии выглядит точно так же, как дерево решений классификации, за исключением листьев. В дереве принятия решений о классификации листья принадлежат к классам, таким как «да» и «нет». В дереве решений регрессии листья содержат значения, такие как 4, 8,2 или −199. Прогноз, который делает наша модель, задается листом, к которому мы пришли, пересекая дерево по нисходящей.

Первый пример использования, который мы рассмотрим, — это популярное применение машинного обучения (и одно из моих любимых) — рекомендательные системы.

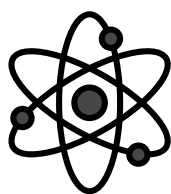
Код для этой главы доступен в репозитории GitHub: https://github.com/luisguiserrano/manning/tree/master/Chapter_9_Ddecision_Trees.

Задача: необходимо рекомендовать пользователям приложения, которые они, скорее всего, скачают

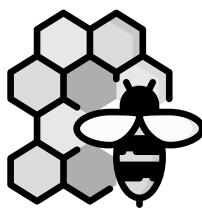
Рекомендательные системы — одно из наиболее распространенных и интересных приложений в области машинного обучения. Вы когда-нибудь задумывались, как Netflix рекомендует фильмы, как YouTube угадывает, какие видео вы захотите посмотреть, или как Amazon показывает вам товары, которые могли бы вас заинтересовать? Все это примеры рекомендательных систем. Один простой и интересный способ понять задачи рекомендации — это рассмотреть их как задачи классификации. Начнем с простого примера — нашей собственной системы рекомендаций приложений на основе деревьев решений.

Допустим, мы хотим создать систему, которая рекомендует пользователям, какое из нескольких приложений загрузить. В нашем магазине доступны три приложения (рис. 9.6):

- **«Счетчик атомов»**. Подсчитывает количество атомов в вашем теле;
- **«Карта улыев»**. Отображает ваше местоположение и находит ближайшие улыи;
- **«Австраломат»**. Приложение для поиска австралийских шахматистов в вашем регионе.



«Счетчик атомов»



«Карта ульев»



«Австраломат»

Рис. 9.6. Три рекомендуемых приложения

Обучающие данные представляют собой таблицу с платформой, применяемой пользователем (iPhone или Android), возрастом и загруженным приложением (в реальной жизни существует гораздо больше платформ, но для простоты предположим, что это единственные два варианта). Таблица содержит данные шести человек (табл. 9.1).

Таблица 9.1. Набор данных пользователей магазина приложений

Платформа	Возраст, лет	Приложение
iPhone	15	«Счетчик атомов»
iPhone	25	«Австраломат»
Android	32	«Карта ульев»
iPhone	35	«Австраломат»
Android	12	«Счетчик атомов»
Android	14	«Счетчик атомов»

Глядя на таблицу, какое приложение вы бы порекомендовали каждому из следующих трех клиентов?

- **Клиент 1** — 13-летний пользователь iPhone.
- **Клиент 2** — 28-летний пользователь iPhone.
- **Клиент 3** — 34-летний пользователь Android.

Вот что нужно сделать.

Клиент 1 — 13-летний пользователь iPhone. Ему стоит рекомендовать «Счетчик атомов». Похоже (глядя на трех клиентов-подростков), что молодые люди склонны загружать именно его.

Клиент 2 — 28-летний пользователь iPhone. Ему мы должны рекомендовать «Австраломат», потому что его загрузили оба пользователя iPhone в наборе данных (в возрасте 25 и 35 лет).

Клиент 3 — 34-летний пользователь Android. Этому клиенту рекомендуем «Карту ульев», потому что в наборе данных встречается один пользователь Android, которому 32 года и который загрузил это приложение.

Однако перебор клиента за клиентом выглядит весьма утомительным делом. Так что мы построим дерево решений, чтобы разобраться со всеми клиентами сразу.

Решение: создание системы рекомендации приложений

В этом разделе рассмотрим, как создать систему рекомендации приложений с использованием деревьев решений. Вкратце алгоритм построения дерева решений выглядит следующим образом.

1. Выяснить, какие данные наиболее полезны для решения о том, какое приложение рекомендовать.
2. Определить признак, по которому данные разбиваются на два меньших набора данных.
3. Повторить шаги 1 и 2 для каждого из двух меньших наборов данных.

Другими словами, мы решаем, какой из двух признаков, платформа или возраст, более эффективен при определении того, какое приложение будут загружать пользователи. После чего делаем его корнем дерева решений. Затем перебираем ветви, всегда выбирая наиболее определяющий признак для данных в этой ветви, и таким образом строим дерево решений.

Первый шаг построения модели — задать наилучший вопрос

Первым шагом при построении модели становится определение наиболее полезного признака. Другими словами, наиболее полезного вопроса, который можно задать. Прежде всего давайте немного упростим данные. Назовем всех в возрасте до 20 лет молодыми, а в возрасте 20 лет и старше — взрослыми (не волнуйтесь, мы скоро вернемся к исходному набору данных в разделе «Разделение данных с помощью непрерывных признаков, таких как возраст»). Измененный набор данных показан в табл. 9.2.

Таблица 9.2. Упрощенная версия набора данных из табл. 9.1, столбец возраста сведен к двум категориям: «молодой» и «взрослый»

Платформа	Возраст	Приложение
iPhone	Молодой	«Счетчик атомов»
iPhone	Взрослый	«Австраломат»
Android	Взрослый	«Карта ульеv»
iPhone	Взрослый	«Австраломат»
Android	Молодой	«Счетчик атомов»
Android	Молодой	«Счетчик атомов»

Строительными блоками деревьев решений служат вопросы вида «Применяет ли пользователь iPhone?» или «Молод ли пользователь?». Выберем один из них и сделаем его корнем. Но какой же из них выбрать? Выбрать необходимо тот, который лучше всего определяет загруженное ими приложение. Чтобы решить, какой вопрос подходит лучше, сравним их.

Первый вопрос: «iPhone или Android используется?»

Он разделяет пользователей на две группы: владельцев iPhone и Android. В каждой группе по три пользователя. Но нам нужно отслеживать, какое приложение скачал каждый из них. Быстрый взгляд на табл. 9.2 помогает заметить следующее.

- Из владельцев iPhone один загрузил «Счетчик атомов», двое — «Австраломат».
- Из обладателей Android двое скачали «Счетчик атомов», один — «Карту ульев».

Итоговый пенек решений показан на рис. 9.7.

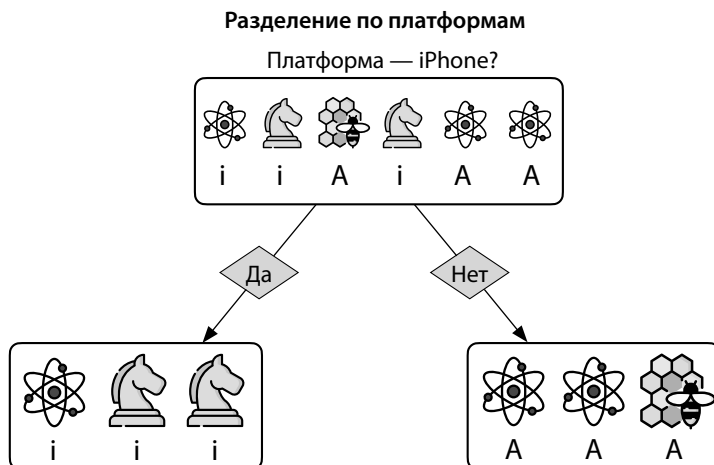


Рис. 9.7. Разделение пользователей по платформам: *слева* — владельцы iPhone; *справа* — Android

Теперь посмотрим, что произойдет, если разделить их по возрасту.

Второй вопрос: «Пользователь молодой или взрослый?»

Вопрос разделяет пользователей на две группы: «молодые» и «взрослые». Опять в каждой группе получаем по три человека. Взглянув на табл. 9.2, мы видим, что они загружали:

- все молодые скачали «Счетчик атомов»;
- из взрослых двое скачали «Австраломат» и один — «Карту ульев».

Итоговый пенек решений показан на рис. 9.8.

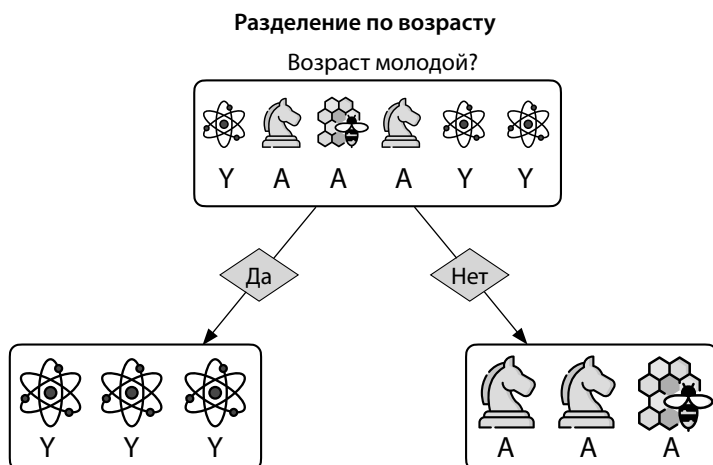


Рис. 9.8. Разделение пользователей по возрасту: *слева* — молодежь; *справа* — взрослые

Какой из рисунков, рис. 9.7 или рис. 9.8, демонстрирует лучшее разделение? Похоже, что второй вариант (основанный на возрасте) лучше, потому что он учитывает тот факт, что все три молодых человека скачали «Счетчик атомов». Но нам нужно, чтобы компьютер понял: возраст — лучший признак, поэтому дадим ему несколько цифр для сравнения. В этом разделе изучим три способа сравнения двух разделений: достоверность, расхождение Джини и энтропию. Начнем с первого — достоверности.

Достоверность. Как часто модель оказывается правой?

Мы узнали о достоверности в главе 7, но вот небольшое резюме. Достоверность — это доля правильно классифицированных точек данных от общего числа.

Предположим, что нам разрешено задать только один вопрос и с помощью него мы должны определить, какое приложение рекомендовать пользователю. Доступны два классификатора.

- **Классификатор 1** задает вопрос: «Какую платформу вы применяете?» — и, исходя из ответа, определяет, какое приложение рекомендовать.
- **Классификатор 2** задает вопрос: «Каков ваш возраст?» — и, исходя из ответа, определяет, какое приложение рекомендовать.

Рассмотрим их более подробно. Ключевое наблюдение заключается вот в чем: если требуется рекомендовать приложение, задав лишь один вопрос, то лучшим решением будет просмотреть всех людей, которые ответили так же, и рекомендовать наиболее популярное у них приложение.

Классификатор 1: «Какую платформу вы используете?»

- Если ответ: «iPhone», то замечаем, что из владельцев iPhone большинство загрузили «Австраломат». Поэтому рекомендуем «Австраломат» всем пользователям iPhone. Мы будем правы **в двух случаях из трех**.
- Если ответ: «Android», то замечаем, что из владельцев Android большинство загрузили «Счетчик атомов». Так что его и рекомендуем всем пользователям Android. Мы будем правы **в двух случаях из трех**.

Классификатор 2: «Ваш возраст?»

- Если ответ: «Молодой», то мы замечаем, что все молодые люди скачали «Счетчик атомов», значит, это наша рекомендация. Мы будем правы **в трех случаях из трех**.
- Если ответ: «Взрослый», то замечаем, что среди взрослых большинство скачали «Австраломат», поэтому рекомендуем этот вариант. Мы будем правы **в двух случаях из трех**.

Обратите внимание на то, что классификатор 1 верен **четыре раза из шести**, а классификатор 2 — **пять из шести**. Следовательно, для этого набора данных лучше использовать классификатор 2. На рис. 9.9 два классификатора и их достоверность — вопросы — сформулированы так, что на них есть ответы «да» или «нет», и это не меняет классификаторы или результат. Чтобы сделать прогноз для листа, каждый классификатор выбирает наиболее распространенную метку среди выборки этого листа.

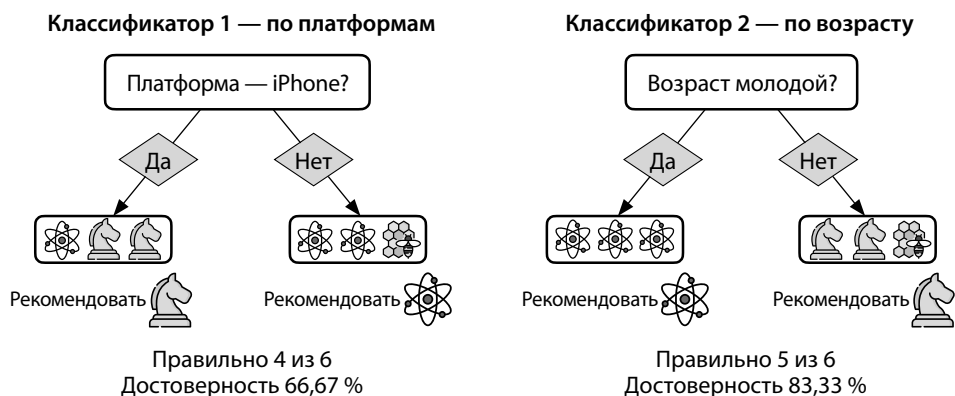


Рис. 9.9. Классификатор 1 использует платформу, а классификатор 2 — возраст

Индекс Джини. Насколько разнообразен мой набор данных?

Индекс примесей Джини, или *индекс Джини*, — это еще один способ сравнить различия в платформе и возрасте. Индекс Джини — это показатель разнообразия в наборе данных. Другими словами, если в имеющемся наборе все элементы похожи, то у него низкий индекс Джини, а если все они разные, то высокий. Для наглядности рассмотрим следующие два набора из 10 цветных шариков, где любые два шарика одного цвета неразличимы:

- **набор 1** — восемь красных шариков и два синих;
- **набор 2** — четыре красных шарика, три синих, два желтых и один зеленый.

Набор 1 выглядит более чистым, чем набор 2, потому что содержит в основном красные шарики и пару синих, в то время как в наборе 2 много разных цветов. Затем мы выводим меру примеси, которая присваивает низкое значение набору 1 и высокое — набору 2. Эта мера примеси зависит от вероятности. Зададим следующий вопрос: **«Если выбрать два случайных элемента из набора, какова вероятность того, что они будут разного цвета?»** Два элемента не обязательно должны быть разными, нам разрешено выбирать один и тот же элемент дважды.

Для набора 1 эта вероятность невелика, потому что шарики в нем имеют одинаковые цвета. Для набора 2 эта вероятность высока, потому что он разнообразен, и, если мы выберем два шарика, они, скорее всего, будут разного цвета. Теперь рассчитаем эти вероятности. Прежде всего, обратим внимание на то, что по закону дополнительных вероятностей (см. раздел «Что это еще за математика?..» в главе 8) вероятность того, что мы выберем два шарика разного цвета, равна 1 минус вероятность того, что выберем два шарика одного цвета:

$P(\text{выбор двух шариков разного цвета}) = 1 - P(\text{выбор двух шариков одного цвета}).$

Теперь рассчитаем вероятность того, что мы выберем два шарика одного цвета. Рассмотрим более широкий набор, где шарики окрашены в n цветов. Назовем их цвет 1, цвет 2 и так до цвета n . Поскольку два шарика должны быть одного из n цветов, вероятность выбора двух шариков одного цвета равна сумме вероятностей выбора двух шариков каждого из n цветов:

$$P(\text{выбор двух шариков одного цвета}) = P(\text{оба шарика имеют цвет 1}) + P(\text{оба шарика имеют цвет 2}) + \dots + P(\text{оба шарика имеют цвет } n).$$

В данном случае мы использовали правило суммы для непересекающихся вероятностей, которое гласит следующее.

ПРАВИЛО СУММЫ ДЛЯ НЕПЕРЕСЕКАЮЩИХСЯ ВЕРОЯТНОСТЕЙ Если два события, E и F , не пересекаются (то есть они никогда не происходят одновременно), то вероятность того, что произойдет одно из них (объединение событий), равна сумме вероятностей каждого из событий. Другими словами,

$$P(E \cup F) = P(E) + P(F).$$

Теперь для каждого из цветов вычислим вероятность того, что два шарика имеют одинаковый цвет. Обратите внимание на то, что мы выбираем каждый шарик совершенно независимо от других. Следовательно, по правилу произведения для независимых вероятностей (см. раздел «Что это еще за математика?..» в главе 8) вероятность того, что оба шарика имеют цвет 1, равна квадрату вероятности того, что мы выберем один шарик и он будет цвета 1. В общем случае, если p_i — это вероятность того, что мы выберем случайный шарик и он будет цвета i , то

$$P(\text{оба шарика имеют цвет } i) = p_i^2.$$

Сложив все эти формулы (рис. 9.10), мы получаем:

$$P(\text{выбор двух шариков разного цвета}) = 1 - p_1^2 - p_2^2 - \dots - p_n^2.$$

Эта формула и является индексом Джини множества.

$$\begin{aligned} \text{Индекс примеси Джини} &= P(\text{выбор двух шариков разного цвета}) \\ &= 1 - P(\text{выбор двух шариков одного цвета}) \\ &= 1 - p_1^2 - p_2^2 - \dots - p_n^2 \end{aligned}$$

Рис. 9.10. Краткое описание расчета индекса примесей Джини

Наконец, вероятность того, что мы выберем случайный шарик цвета i , равна количеству шариков цвета i , деленному на общее их количество. Это подводит нас к формальному определению индекса Джини.

ИНДЕКС ПРИМЕСЕЙ ДЖИНИ В наборе с m элементами, n классами и a_i элементами, принадлежащими к i -му классу,

$$\text{индекс примесей Джини} = 1 - p_1^2 - p_2^2 - \dots - p_n^2,$$

где $p_i = a_i/m$. Это можно интерпретировать как вероятность того, что если мы выберем два случайных элемента из набора, то они будут принадлежать к разным классам.

Теперь можно рассчитать индекс Джини для обоих наборов. Для наглядности расчет индекса Джини для набора 1 показан на рис. 9.11 (красный и синий цвета заменены черным и белым).

- **Набор 1** {красный, красный, красный, красный, красный, красный, красный, красный, синий, синий} (восемь красных шариков, два синих):

$$\text{индекс примесей Джини} = 1 - \left(\frac{8}{10}\right)^2 - \left(\frac{2}{10}\right)^2 = 1 - \frac{68}{100} = 0,32.$$

- **Набор 2** {красный, красный, красный, красный, синий, синий, синий, желтый, желтый, зеленый};

$$\text{индекс примесей Джини} = 1 - \left(\frac{4}{10}\right)^2 - \left(\frac{3}{10}\right)^2 - \left(\frac{2}{10}\right)^2 - \left(\frac{1}{10}\right)^2 = 1 - \frac{30}{100} = 0,7.$$

Если общая площадь квадрата равна 1, вероятность выбора двух черных шариков равна $0,8^2$, а вероятность выбора двух белых шариков равна $0,2^2$ (эти два представлены заштрихованными квадратами). Таким образом, вероятность выбора двух шариков разного цвета равна оставшейся площади: $1 - 0,8^2 - 0,2^2 = 0,32$. Это и есть индекс Джини. Действительно, индекс Джини набора 1 больше, чем набора 2.

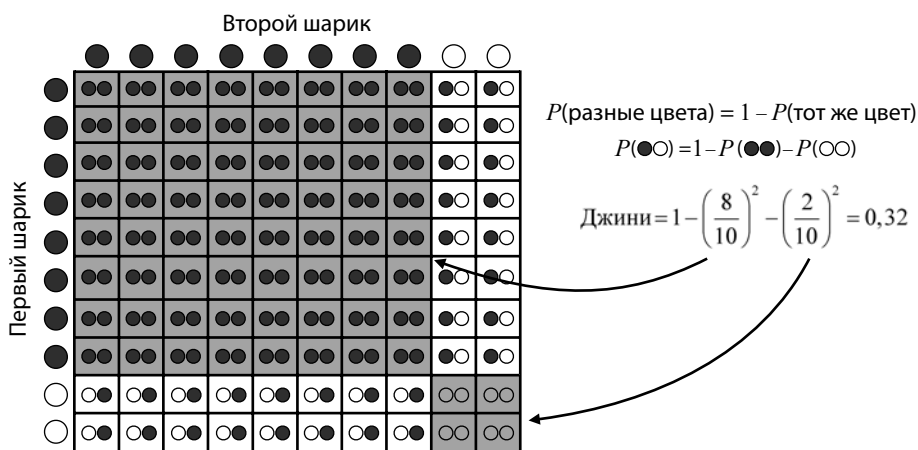


Рис. 9.11. Расчет индекса Джини для набора с восемью черными и двумя белыми шарами

Как можно использовать индекс Джини для принятия решения о том, какой из двух способов разделения данных (возраст или платформа) лучше? Очевидно, что если удастся разделить данные на два более чистых набора данных, мы получим лучшее разделение. Таким образом, вычислим индекс Джини для набора меток каждого из листьев. Если посмотреть на рис. 9.12, то вот какими будут метки листьев (мы сокращаем каждое приложение по первой букве в его названии).

Классификатор 1 (по платформам):

- левый лист (iPhone) — {A, C, C};
- правый лист (Android) — {A, A, B}.

Классификатор 2 (по возрасту):

- левый лист (молодой) — {A, A, A};
- правый лист (взрослый) — {B, C, C}.

Индексы Джини множеств $\{A, C, C\}$, $\{A, A, B\}$ и $\{B, C, C\}$ одинаковы —

$$1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 = 0,444.$$

Индекс Джини множества $\{A, A, A\}$ равен $1 - \left(\frac{3}{3}\right)^2 = 0$. В общем случае индекс Джини чистого множества всегда равен 0. Чтобы измерить чистоту разделения, мы усредняем показатели Джини двух листьев. Это приводит нас к следующим расчетам:

Классификатор 1 (по платформам):

$$\text{средний индекс Джини} = 1/2(0,444 + 0,444) = 0,444.$$

Классификатор 2 (по возрасту):

$$\text{средний индекс Джини} = 1/2(0,444 + 0) = 0,222.$$

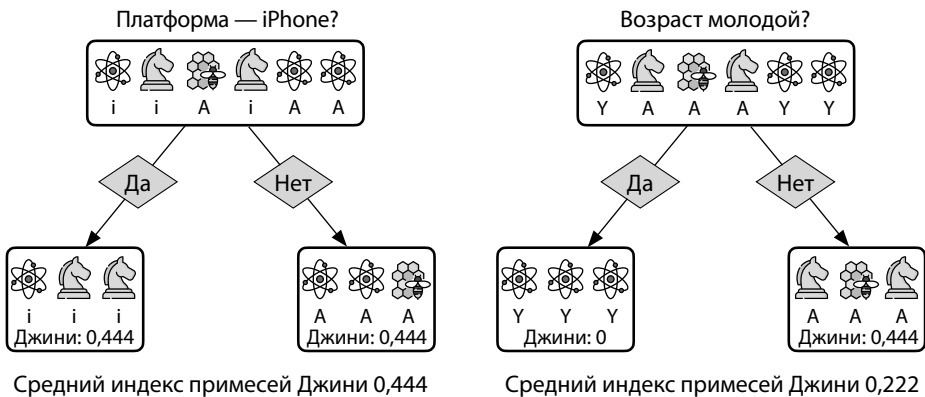


Рис. 9.12. Два способа разделения набора данных по платформе и возрасту и вычисление индекса Джини для каждого случая

Мы приходим к выводу, что второе разделение лучше, поскольку оно имеет более низкий средний индекс Джини. Поэтому решаем разделить набор данных по возрасту.

ОТВЛЕЧЕМСЯ Индекс примесей Джини не следует путать с коэффициентом Джини. Коэффициент Джини используется в статистике для расчета неравенства доходов или достатка в странах. В этой книге всякий раз, говоря об индексе Джини, мы имеем в виду индекс примесей Джини.

Энтропия — еще одна мера разнообразия, широко применяющаяся в теории информации

В этом разделе мы изучаем еще одну меру однородности множества — его энтропию, которая основана на физической концепции энтропии и очень важна

в теории вероятностей и информации. Чтобы понять, что такое энтропия, рассмотрим несколько странный вероятностный вопрос. Рассмотрим те же два набора цветных шаров, что и в предыдущем разделе, но теперь будем думать о цветах как об упорядоченном наборе.

- **Набор 1** {красный, красный, красный, красный, красный, красный, красный, красный, синий, синий} (восемь красных и два синих шарика).
- **Набор 2** {красный, красный, красный, красный, синий, синий, синий, желтый, желтый, зеленый} (четыре красных, три синих, два желтых и один зеленый шарик).

Теперь рассмотрим следующий сценарий: мы поместили набор 1 в мешок и начинаем доставать из него шарики, после чего немедленно возвращаем каждый из них обратно. Цвета выбранных нами шаров запишем. Представьте, что, проделав это 10 раз, мы получили такую последовательность: красный, красный, красный, синий, красный, синий, синий, красный, красный, красный.

Вот главный вопрос, который определяет энтропию: **«Какова вероятность того, что, следуя процедуре, описанной в предыдущем абзаце, мы получим точную последовательность, определяющую набор 1, который таков {красный, красный, красный, красный, красный, красный, красный, синий, синий}?»**

Эта вероятность не очень велика, потому что нам должно действительно повезти, чтобы получилась такая последовательность. Давайте ее подсчитаем. У нас есть восемь красных и два синих шарика, так что вероятность того, что мы выберем красный, равна $8/10$, а вероятность того, что мы выберем синий шарик, равна $2/10$. Поскольку все выборы независимы, вероятность того, что мы получим желаемую последовательность:

$$P(r, r, r, r, r, r, r, r, b, b) = \frac{8}{10} \times \frac{8}{10} \times \frac{8}{10} \times \frac{8}{10} \times \frac{8}{10} \times \frac{8}{10} \times \frac{8}{10} \times \frac{8}{10} \times \frac{2}{10} \times \frac{2}{10} =$$

$$= \left(\frac{8}{10}\right)^8 \left(\frac{2}{10}\right)^2 = 0,0067108864.$$

Это крошечное число, но сможете ли вы представить себе подобную вероятность для набора 2? Рассматривая его, мы выбираем шары из мешка с четырьмя красными, тремя синими, двумя желтыми и одним зеленым шариком, надеясь получить следующую последовательность: красный, красный, красный, красный, синий, синий, синий, желтый, желтый, зеленый.

Это почти невозможно, потому что у нас много цветов и не так много шаров каждого цвета. Вероятность вычисляется аналогично и составляет

$$P(r, r, r, r, b, b, b, y, y, g) = \frac{4}{10} \times \frac{4}{10} \times \frac{4}{10} \times \frac{4}{10} \times \frac{3}{10} \times \frac{3}{10} \times \frac{3}{10} \times \frac{2}{10} \times \frac{2}{10} \times \frac{1}{10} =$$

$$= \left(\frac{4}{10}\right)^4 \left(\frac{3}{10}\right)^3 \left(\frac{2}{10}\right)^2 \left(\frac{1}{10}\right)^1 = 0,0000027648.$$

Чем разнообразнее набор, тем маловероятнее, что мы сможем получить исходную последовательность, выбирая по одному шарiku за раз. И напротив, самый чистый набор, в котором все шары одного цвета, вытянуть таким образом очень легко. Например, если в исходном наборе 10 красных шаров, то каждый выбор случайного шарика дает нам красный. Таким образом, вероятность получения последовательности {красный, красный, красный, красный, красный, красный, красный, красный, красный, красный} равна 1.

Эти значения очень малы для большинства случаев, а ведь у нас всего 10 элементов. Представьте, если бы в наборе их был миллион, мы бы имели дело с ничтожными значениями. Когда приходится оперировать действительно маленькими числами, то лучший метод — использование логарифмов, потому что они обеспечивают удобный способ записи небольших чисел. Например, $0,000000000000001$ равно 10^{-15} , поэтому его логарифм по основанию 10 равен -15 . С таким числом гораздо удобнее работать.

Энтропия определяется следующим образом: мы начинаем с вероятности восстановления исходной последовательности при последовательном выборе элементов из нашего набора по одному за раз. Затем берем логарифм и делим на общее количество элементов в наборе. Поскольку деревья решений имеют дело с бинарными решениями, будем использовать логарифмы по основанию 2. Причина, по которой мы взяли отрицательное значение логарифма, заключается в том, что все логарифмы очень малых чисел отрицательны, поэтому умножаем на -1 , чтобы превратить его в положительное число. Поскольку мы взяли отрицательный результат, то чем разнообразнее набор, тем выше энтропия.

Теперь можем вычислить энтропии обоих множеств и расширить их, используя следующие два тождества:

$$\begin{aligned}\log(ab) &= \log(a) + \log(b); \\ \log(a^c) &= c \log(a).\end{aligned}$$

Набор 1 {красный, красный, красный, красный, красный, красный, красный, красный, синий, синий} (восемь красных шариков, два синих):

$$\text{энтропия} = -\frac{1}{10} \log_2 = \left[\left(\frac{8}{10} \right)^8 \left(\frac{2}{10} \right)^2 \right] - \frac{8}{10} \log_2 \left(\frac{8}{10} \right) - \frac{2}{10} \log_2 \left(\frac{2}{10} \right) = 0,722.$$

Набор 2 {красный, красный, красный, красный, синий, синий, синий, желтый, желтый, зеленый}:

$$\begin{aligned}\text{энтропия} &= -\frac{1}{10} \log_2 \left[\left(\frac{4}{10} \right)^4 \left(\frac{3}{10} \right)^3 \left(\frac{2}{10} \right)^2 \left(\frac{1}{10} \right)^1 \right] = \\ &= -\frac{4}{10} \log_2 \left(\frac{4}{10} \right) - \frac{3}{10} \log_2 \left(\frac{3}{10} \right) - \frac{2}{10} \log_2 \left(\frac{2}{10} \right) - \frac{1}{10} \log_2 \left(\frac{1}{10} \right) = 1,846.\end{aligned}$$

Обратите внимание на то, что энтропия множества 2 больше, чем энтропия множества 1, что означает: множество 2 более разнообразно. Далее приводится формальное определение энтропии.

ЭНТРОПИЯ В наборе с m элементами, n классами и a_i элементами, принадлежащими к i -му классу:

$$\text{энтропия} = -p_1 \log_2(p_1) - p_2 \log_2(p_2) - \dots - p_n \log_2(p_n),$$

где $p_i = a_i/m$.

Мы можем использовать энтропию, чтобы решить, какой из двух способов разделения данных (платформа или возраст) лучше, точно так же, как мы это сделали с индексом Джини. Эмпирическое правило заключается в том, что если мы можем разделить данные на два набора с меньшей суммарной энтропией, то получаем лучшее разделение. Таким образом, вычислим энтропию набора меток каждого из листьев. Вновь обратившись к рис. 9.12, мы видим следующие метки листьев (сокращаем каждое приложение до первой буквы в его названии):

Классификатор 1 (по платформе):

- левый лист: {A, C, C};
- правый лист: {A, A, B}.

Классификатор 2 (по возрасту):

- левый лист: {A, A, A};
- правый лист: {B, C, C}.

Энтропии множеств {A, C, C}, {A, A, B} и {B, C, C} все одинаковы и равны $-(2/3)\log_2(2/3) - (1/3)\log_2(1/3) = 0,918$. Энтропия множества {A, A, A} равна $-\log_2(3/3) = -\log_2(1) = 0$.

В общем случае энтропия множества, в котором все элементы одинаковы, всегда равна 0. Чтобы измерить чистоту разделения, усредняем энтропию наборов меток двух листьев следующим образом (показано на рис. 9.13).

Классификатор 1 (по платформам):

$$\text{средняя энтропия} = (1/2)(0,918 + 0,918) = 0,918.$$

Классификатор 2 (по возрасту):

$$\text{средняя энтропия} = (1/2)(0,918 + 0) = 0,459.$$

Обратите внимание на то, что разделение набора данных по возрасту дает два меньших набора данных с более низкой средней энтропией. Таким образом, мы снова приходим к выводу, что второе разделение лучше, потому что имеет более низкую среднюю энтропию.

Энтропия — чрезвычайно важное понятие в теории вероятностей и статистике, поскольку оно тесно связано с теорией информации, главным образом благодаря работам Клода Шеннона. Важная концепция, называемая *природой информации*, — это именно изменение энтропии. Для более глубокого погружения в тему, пожалуйста, загляните в приложение В — там есть ссылки на видеоролик и статью, в которых эта тема освещается гораздо подробнее.

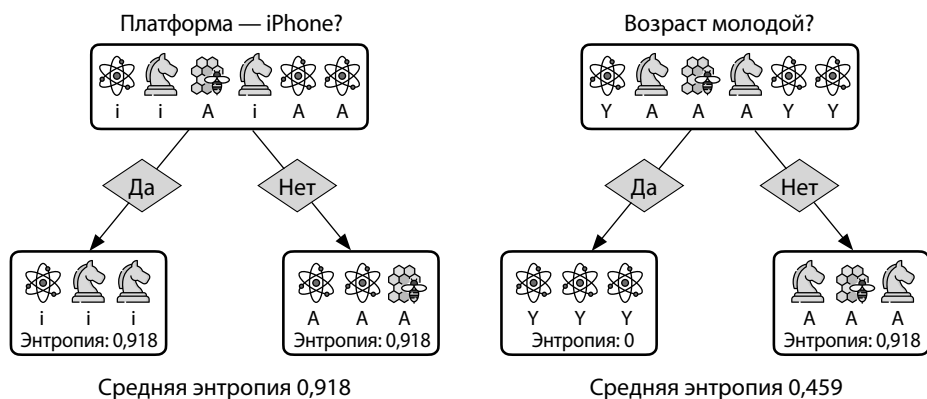
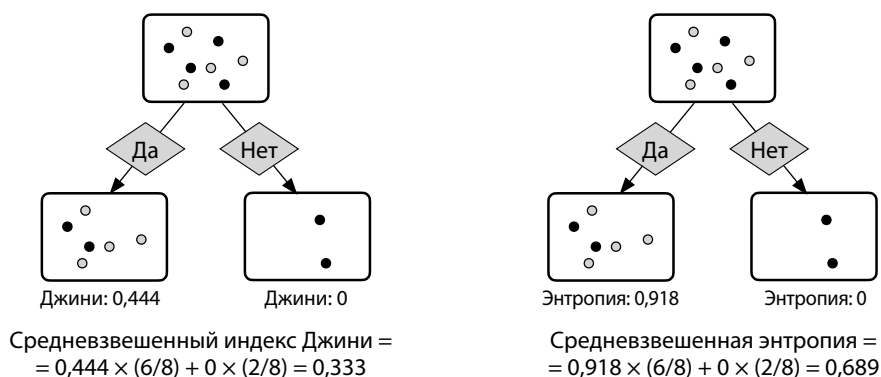


Рис. 9.13. Два способа разделения набора данных — по платформе и возрасту, а также вычисление их энтропии

Классы разных размеров? Нет проблем — берем средневзвешенные значения

Из предыдущих разделов мы узнали, как выполнить наилучшее возможное разделение, минимизируя средний индекс примеси Джини или энтропию. Однако представьте, что у вас есть набор данных с восемью точками данных (при обучении дерева решений мы называем их выборками) и вы разделяете его на два набора данных размером шесть и два. Можно предположить, что больший набор данных должен иметь большее значение при вычислении индекса примеси Джини или энтропии. Поэтому вместо того, чтобы рассматривать среднее значение, мы рассматриваем средневзвешенное значение, где каждому листу присваиваем долю точек, соответствующую этому листу. Следовательно, в этом случае мы бы взвесили первый индекс примеси Джини (или энтропию) на $6/8$, а второй — на $2/8$. На рис. 9.14 показан пример средневзвешенного индекса примесей Джини и средневзвешенной энтропии для разделения выборки.

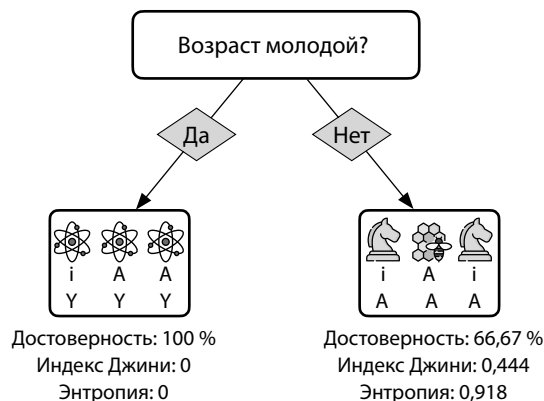


На рис. 9.14. Разделение набора данных из восьми элементов на два набора данных из шести и двух элементов

Теперь, когда мы изучили три метода выбора наилучшего разделения — достоверность, индекс Джини и энтропию, для построения дерева решений нам нужно лишь многократно повторить этот процесс! Подробно поговорим об этом в следующем разделе.

Второй шаг построения модели — повторение

В предыдущем разделе мы узнали, как наилучшим образом разделить данные с помощью одного из признаков. Это основная часть процесса обучения дерева решений. Для завершения построения дерева решений осталось лишь повторить данный шаг много раз. В этом разделе мы узнаем, как это проделать, используя три метода — достоверность, индекс Джини на основе признака «возраст» и энтропия. Как только мы это проделаем, набор данных будет разделен на два. Разделение данных на два набора, а также их достоверность, индекс Джини и энтропия показаны на рис. 9.15.



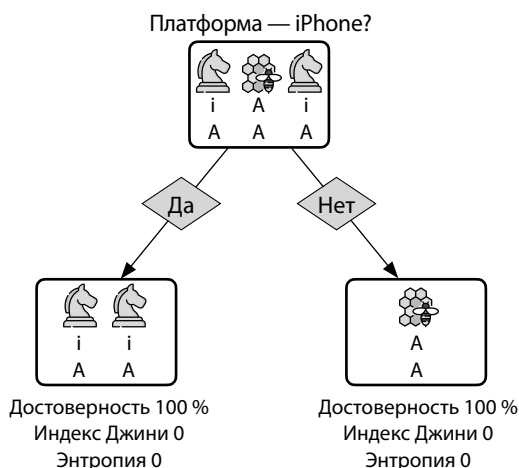


Рис. 9.16. Можно разделить правый лист дерева, изображенного на рис. 9.15, на основе платформы и получить два чистых набора данных. Каждый из них имеет достоверность 100 %, а индекс Джини и энтропия равны 0

На этом разделении мы заканчиваем, потому что больше не можем их улучшать. Таким образом, получаем дерево, изображенное на рис. 9.17.

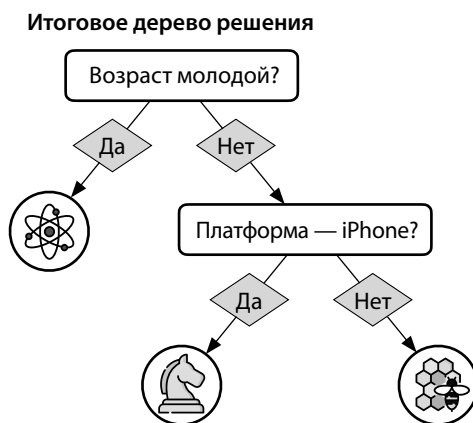


Рис. 9.17. Результирующее дерево решений имеет два узла и три листа. Оно правильно предсказывает каждую точку в исходном наборе данных

Это конец всего процесса, так как мы построили дерево решений, которое классифицирует весь наш набор данных. У нас есть почти весь псевдокод для алгоритма, за исключением некоторых заключительных подробностей, которые мы найдем в следующем разделе.

Последний шаг — когда прекратить построение дерева и других гиперпараметров

В предыдущем разделе мы построили дерево решений путем рекурсивного разделения набора данных. Каждое разделение выполнялось выбором наилучшего признака для разделения. Этот признак обнаруживался с использованием любого из следующих показателей: достоверности, индекса Джини или энтропии. Мы заканчиваем, когда часть набора данных, соответствующая каждому из конечных узлов, становится чистой — другими словами, когда все элементы в ней имеют одну и ту же метку.

В ходе этого процесса может возникнуть множество проблем. Например, если мы и дальше будем продолжать разделять данные, то можем оказаться в экстремальной ситуации, когда каждый лист содержит слишком мало элементов, что может привести к серьезному переобучению. Предотвратить подобный исход можно, введя условие остановки — любое из следующих.

1. Не разделять узел, если изменение достоверности, индекса Джини или энтропии ниже некоторого порога.
2. Не разделять узел, если в нем меньше определенного количества элементов.
3. Разделять узел только в случае, когда оба результирующих листа содержат по крайней мере определенное количество элементов.
4. Прекратить строить дерево после того, как достигнута определенная глубина.

Все перечисленные условия остановки требуют гиперпараметра. А если точнее, то это должны быть гиперпараметры, соответствующие предыдущим четырем условиям.

1. Минимальная величина изменения достоверности (или индекса Джини, или энтропии).
2. Минимальное количество элементов, которое должен иметь узел, чтобы его можно было разделить.
3. Минимальное количество элементов, разрешенных в конечном узле.
4. Максимальная глубина дерева.

Мы выбираем эти гиперпараметры либо на основе опыта, либо выполнив исчерпывающий поиск, в ходе которого ищем различные комбинации гиперпараметров и выбираем ту, которая работает лучше всего на нашем проверочном наборе. Этот процесс называется *поиском по сетке*, более подробно мы рассмотрим его в разделе «Настройка гиперпараметров для поиска наилучшей модели. Поиск по сетке» в главе 13.

Алгоритм дерева решений. Как построить дерево решений и делать с его помощью прогнозы

Теперь мы наконец готовы к разработке псевдокода алгоритма дерева решений, который позволит нам обучать дерево решений, чтобы оно соответствовало набору данных.

Псевдокод алгоритма дерева решений

Входные данные:

- обучающий набор данных с соответствующими метками;
- показатель для разделения данных (достоверность, индекс Джини или энтропия);
- одно условие остановки (или более).

Результат — дерево решений, соответствующее набору данных.

Процедура

- Добавить корневой узел и связать его со всем набором данных. Этот узел имеет уровень 0. Назовем его конечным узлом.
- Повторять до тех пор, пока условия остановки не будут выполнены на каждом конечном узле.
 - Выбрать один из конечных узлов на самом высоком уровне.
 - Просмотреть все признаки и выбрать тот, который оптимальным образом и в соответствии с определенным показателем разбивает элементы, соответствующие узлу. Связать этот признак с узлом.
 - Данный признак разбивает набор данных на две ветви. Создать два новых конечных узла, по одному для каждой ветви, и связать с каждым из них соответствующие элементы.
 - Если условия остановки допускают разделение, превратить узел в узел принятия решений и добавить под ним два новых конечных узла. Если уровень узла i , то два новых конечных узла находятся на уровне $i + 1$.
 - Если условия остановки не допускают разделения, узел становится конечным. Связать с ним наиболее распространенную среди его элементов метку. Эта метка — прогноз на данном листе.

Возврат — полученное дерево решений.

Чтобы делать прогнозы с помощью этого дерева, мы просто проходим по нему вниз, используя следующие правила.

- Пройти по дереву вниз. На каждом узле продолжать движение в направлении, отмеченном признаком.
- При переходе к листу прогнозом выступает метка, связанная с ним (наиболее распространенная среди элементов, связанных с этим листом в процессе обучения).

Вот так мы делаем прогнозы, применяя дерево решений для того, чтобы рекомендовать приложение, которое построили ранее. Когда приходит новый пользователь, мы проверяем его возраст и платформу, после чего выполняем следующие действия.

- Если пользователь молодой, рекомендуем ему «Счетчик атомов».
- Если он взрослый, проверяем его платформу:
 - если платформа — Android, рекомендуем «Карту ульев»;
 - если платформа — iPhone, рекомендуем «Австраломат».

ОТВЛЕЧЕМСЯ В литературе, посвященной обучению деревьев, встречаются термины *«прирост Джини»* и *«прирост информации»*. Прирост Джини — это разница между взвешенным индексом примеси Джини листьев и индексом примеси Джини (энтропией) узла принятия решений, который мы разделяем. Аналогично прирост информации представляет собой разницу между взвешенной энтропией листьев и энтропией корня. Более распространенный способ обучения деревьев решений — максимальное увеличение коэффициента Джини или прироста информации. Однако в этой главе мы обучаем деревья решений, минимизируя взвешенный индекс Джини или взвешенную энтропию. Процесс обучения точно такой же, потому что индекс примеси Джини (энтропия) узла принятия решения остается постоянным на протяжении всего хода разделения конкретного узла принятия решения.

Помимо вопросов типа «да/нет»

В разделе «Решение. Создание системы рекомендаций приложений» мы узнали, как построить дерево решений для конкретного случая, в котором каждый признак был категориальным и бинарным, что означает: у него есть только два класса, например платформа пользователя. Однако почти тот же алгоритм работает для построения дерева решений с категориальными признаками с большим количеством классов (например, собака/кошка/птица) и даже с числовыми признаками (такими как возраст или средний доход). Основные изменения происходят на шаге, где мы разделяем набор данных, и в этом разделе узнаем, как это делается.

Разделение данных с помощью не двоичных категориальных признаков, таких как собака/кошка/птица

Напомним, что когда мы хотим разделить набор данных на основе двоичного признака, то просто задаем один вопрос, требующий ответа типа «да» или «нет»: «Является ли признак X?» Например, когда признаком выступает платформа, следует задать вопрос: «Является ли пользователь пользователем iPhone?» Если признак с более чем двумя классами, мы просто задаем несколько вопросов. Например, если входные данные представляют собой животное, которое может быть собакой, кошкой или птицей, то задаем следующие вопросы:

- «Является ли животное собакой?»
- «Является ли животное кошкой?»
- «Является ли животное птицей?»

Независимо от того, сколько классов имеет признак, мы можем разделить его на несколько двоичных вопросов (рис. 9.18).

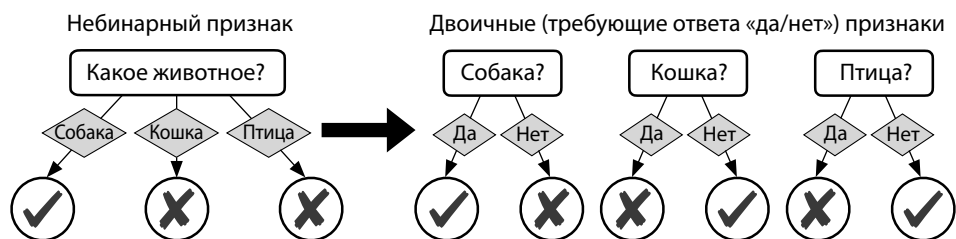


Рис. 9.18. Небинарный признак, например с тремя или более возможными категориями, превращаем в несколько бинарных (требующих ответа «да/нет»), по одному для каждой категории

Каждый из вопросов разделяет данные по-разному. Чтобы выяснить, какой из трех вопросов даст наилучшее разделение, мы используем те же методы, что и в разделе «Первый шаг к построению модели...»: достоверность, индекс Джини или энтропию. Процесс превращения небинарного категориального признака в несколько двоичных называется *прямым кодированием*. В разделе «Преобразование категориальных данных в числовые...» в главе 13 мы увидим, как они применяются в реальном наборе данных.

Разделение данных с помощью непрерывных признаков, таких как возраст

Напомним, что до того, как мы упростили набор данных, признак «возраст» содержал числа. Давайте вернемся к исходной таблице и построим дерево решений (табл. 9.3).

Таблица 9.3. Исходный набор данных рекомендаций по приложениям с указанием платформы и возраста пользователей (численного) (это табл. 9.1)

Платформа	Возраст, лет	Приложение
iPhone	15	«Счетчик атомов»
iPhone	25	«Австраломат»
Android	32	«Карта ульев»
iPhone	35	«Австраломат»
Android	12	«Счетчик атомов»
Android	14	«Счетчик атомов»

Идея состоит в том, чтобы превратить столбец возраста в несколько вопросов формы: «Пользователь моложе X?» или «Пользователь старше X?» Может показаться, что у нас получится бесконечно много вопросов, потому что существует бесконечно много чисел. Но обратите внимание на то, что многие

из вопросов разделяют данные одинаково. Например, вопросы «Пользователь младше 20 лет?» и «Пользователь младше 21 года?» дают одинаковое разделение. На самом деле возможны только семь разделений, показанных на рис. 9.19.

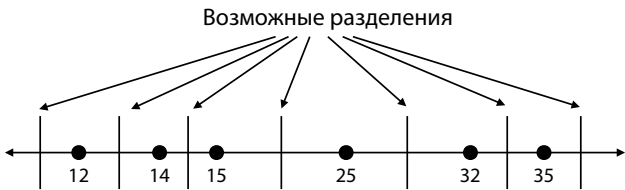


Рис. 9.19. Семь возможных способов разделения пользователей по возрасту

Как правило, в качестве возраста для разделения мы выбираем средние точки между последовательными возрастами (за исключением первой и последней). Для конечных точек мы можем выбрать любое случайное значение, которое находится вне интервала. Таким образом, у нас есть семь возможных вопросов, которые разделяют данные на два набора, как показано в табл. 9.4. В первый набор помещаем пользователей моложе порогового значения, а во второй — тех, кто старше. Здесь мы также рассчитали достоверность, индекс примеси Джини и энтропию каждого из разделений.

Таблица 9.4. Семь возможных вопросов с соответствующим разделением

Вопрос	Первый набор (да)	Второй набор (нет)	Метки	Взвешенная достоверность	Взвешенный индекс примесей Джини	Взвешенная энтропия
Пользователь младше 7 лет?	Пусто	12, 14, 15, 25, 32, 35	$\{\}$, {A, A, A, C, B, C}	3/6	0,611	1,459
Пользователь младше 13 лет?	12	14, 15, 25, 32, 35	{A}, {A, A, C, B, C}	3/6	0,533	1,268
Пользователь младше 14,5 лет?	12, 14	15, 25, 32, 35	{A, A}, {A, C, B, C}	4/6	0,417	1,000
Пользователь младше 20 лет?	12, 14, 15	25, 32, 35	{A, A, A}, {C, B, C}	5/6	0,222	0,459
Пользователь младше 28,5 лет?	12, 14, 15, 25	32, 35	{A, A, A, C}, {B, C}	4/6	0,416	0,874
Пользователь младше 33,5 лет?	12, 14, 15, 25, 32	35	{A, A, A, C, B}, {C}	4/6	0,467	1,145
Пользователь младше 100 лет?	12, 14, 15, 25, 32, 35	Пусто	{A, A, A, C, B, C}, {}	3/6	0,611	1/459

Обратите внимание: четвертый вопрос («Пользователь младше 20 лет?») дает наивысшую достоверность, наименьший взвешенный индекс Джини и наименьшую взвешенную энтропию и, следовательно, будет наилучшим разделением, которого можно достичь с помощью признака «возраст».

Выполните вычисления, приведенные в таблице, и убедитесь, что получаете такие же ответы. Полный расчет этих индексов Джини приведен здесь: https://github.com/luisguiserrano/manning/blob/master/Chapter_9_Decision_Trees/Gini_entropy_calculations.ipynb.

Для наглядности проведем расчеты достоверности, взвешенного индекса примеси Джини и взвешенной энтропии для третьего вопроса. Обратите внимание на то, что этот вопрос разбивает данные на следующие два набора.

- **Набор 1** (младше 14,5 лет):
 - возраст 12, 14 лет;
 - метки {A, A}.
- **Набор 2** (14,5 лет и старше):
 - возраст 15, 25, 32, 25 лет;
 - метки {A, C, B, C}.

Расчет достоверности

Наиболее распространенная метка в наборе 1 — «A», а в наборе 2 — «C», так что это прогнозы, которые мы сделаем для каждого из соответствующих листьев. В наборе 1 правильно спрогнозированы все элементы, а в наборе 2 — только два. Следовательно, это решение является правильным в четырех из шести точек данных с достоверностью $4/6 = 0,667$.

Для следующих двух вычислений обратите внимание на то, что:

- набор 1 чистый (все его метки одинаковы), поэтому его индекс примеси Джини и энтропия равны 0;
- в наборе 2 пропорции элементов с метками «A», «B» и «C» равны $1/4$, $1/4$ и $2/4 = 1/2$ соответственно.

Расчет взвешенного индекса примесей Джини

Индекс примеси Джини множества {A, A} равен 0.

Индекс примеси Джини множества {A, C, B, C} равен:

$$1 - (1/4)^2 - (1/4)^2 - (1/2)^2 = 0,625.$$

Средневзвешенное значение двух индексов примесей Джини составляет:

$$(2/6) \times 0 + (4/6) \times 0,625 = 0,417.$$

Расчет достоверности

Энтропия множества {A, A} равна 0. Энтропия множества {A, C, B, C} равна:

$$-(1/4)\log_2(1/4) - (1/4)\log_2(1/4) - (1/2)\log_2(1/2) = 1,5.$$

Средневзвешенное значение двух энтропий равно: $(2/6) \times 0 + (4/6) \times 1,5 = 1,0$.

Числовой признак становится серией вопросов, требующих ответа «да/нет», которые можно измерить и сравнить с другими такими же вопросами, поступающими от других признаков, после чего выбрать наилучший для этого узла принятия решений.

ОТВЛЕЧЕМСЯ Данная модель рекомендаций приложений крайне невелика, так что мы могли бы проделать все это вручную. Тем не менее, чтобы увидеть ее код, пожалуйста, загляните сюда: https://github.com/luisguiserrano/manning/blob/master/Chapter_9_Decision_Trees/App_recommendations.ipynb. В коде задействуется пакет Scikit-Learn, с которым мы более подробно познакомимся в разделе «Использование Scikit-Learn. Учимся строить дерево решений».

Графическая граница деревьев решений

В этом разделе я покажу вам две вещи: как построить дерево решений геометрически (в двух измерениях) и как закодировать дерево решений в популярном пакете машинного обучения Scikit-Learn.

Напомним, что в моделях классификации, таких как персептрон (см. главу 5) или логистический классификатор (см. главу 6), мы изображали границу модели, которая разделяла точки с метками 0 и 1, в виде прямой. Граница дерева решений также прекрасно смотрится, и, когда данные двумерны, она формируется комбинацией вертикальных и горизонтальных прямых. В данном разделе мы проиллюстрируем это на примере. На рис. 9.20 рассмотрим набор данных, где точки с меткой 1 изображены треугольниками, а точки с меткой 0 — квадратами. Горизонтальная и вертикальная оси обозначены x_0 и x_1 соответственно.

Если бы вам понадобилось разделить этот набор данных только одной горизонтальной или вертикальной прямой, как бы вы ее провели? Можно предположить несколько прямых в зависимости от критериев, которые используются для измерения эффективности решения. Продолжим и выберем вертикальную линию $x_0 = 5$. Это решение оставляет основную массу треугольников справа, а основную массу квадратов слева, за исключением двух неправильно классифицированных точек — одного квадрата и одного треугольника (рис. 9.21). Этот простой классификатор правильно определяет 8 из 10 точек с достоверностью 0,8. Проверьте все другие возможные вертикальные и горизонтальные линии, сравните их с помощью понравившегося показателя (достоверности, индекса Джини и энтропии) и убедитесь, что это прямая, которая наилучшим образом разделяет точки.

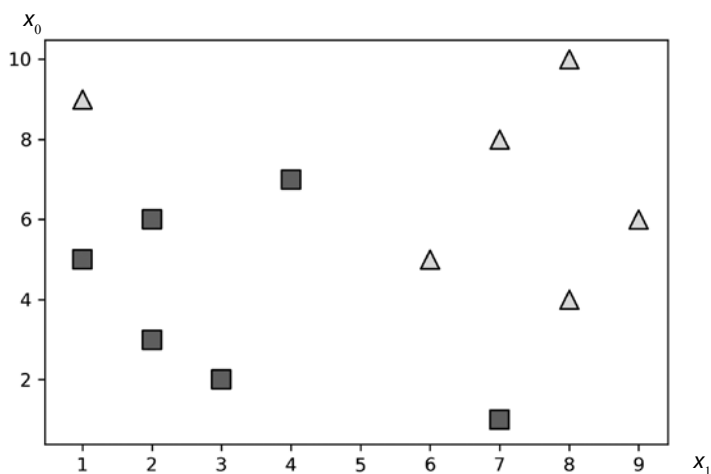


Рис. 9.20. Набор данных с двумя признаками (x_0 и x_1) и двумя метками (треугольник и квадрат), на котором мы будем обучать дерево решений

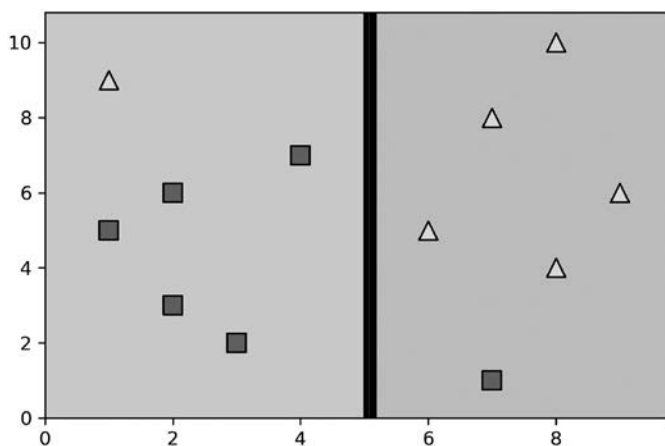


Рис. 9.21. Лучший классификатор — вертикальная прямая $x_1 = 5$, которая классифицирует все, что находится справа от нее, как треугольники, а все, что слева, — как квадраты

Теперь рассмотрим каждую половину отдельно. На этот раз легко увидеть, что две горизонтальные прямые $x_0 = 8$ и $x_0 = 2,5$ хорошо подойдут для левой и правой стороны соответственно. Они окончательно делят набор данных на квадраты и треугольники. Результат показан на рис. 9.22.

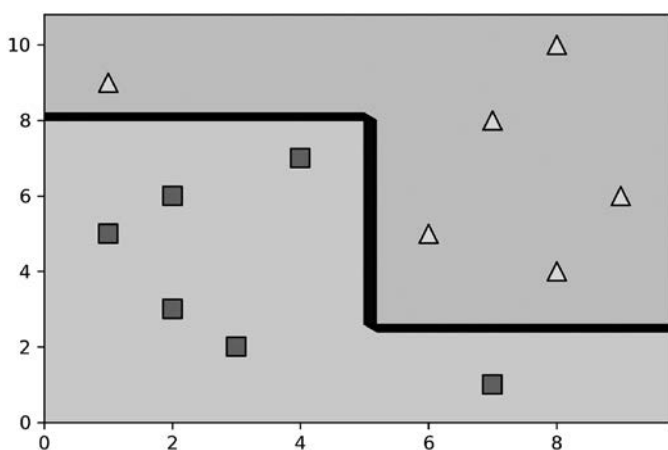


Рис. 9.22. Лучший выбор для классификации каждого из двух наборов данных (см. рис. 9.21) — горизонтальные прямые $x_1 = 8$ и $x_1 = 2,5$

То, что мы здесь проделали, — это и есть построение дерева решений. На каждом этапе мы брали каждое из двух свойств, x_0 и x_1 , и выбирали пороговое значение, которое наилучшим образом разделяет данные. В следующем разделе мы используем Scikit-Learn для построения того же дерева решений на данном наборе данных.

Использование Scikit-Learn. Учимся строить дерево решений

В этом разделе мы узнаем, как применять популярный пакет машинного обучения Scikit-Learn (сокращенно `sklearn`) для построения дерева решений. Код для этого раздела приведен далее.

- **Блокнот** `Graphical_example.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_9_Decision_Trees/Graphical_example.ipynb

Мы начнем с загрузки набора данных в виде фрейма данных Pandas под названием `dataset` (представлен в главе 8) с помощью следующего кода:

```
import pandas as pd
dataset = pd.DataFrame({
    'x_0': [7, 3, 2, 1, 2, 4, 1, 8, 6, 7, 8, 9],
    'x_1': [1, 2, 3, 5, 6, 7, 9, 10, 5, 8, 4, 6],
    'y': [0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1]})
```

Теперь разделяем признаки и метки, как показано далее:

```
features = dataset[['x_0', 'x_1']]
labels = dataset['y']
```

Чтобы построить дерево решений, создаем объект `DecisionTreeClassifier` и используем функцию `fit` следующим образом:

```
decision_tree = DecisionTreeClassifier()
decision_tree.fit(features, labels)
```

На рис. 9.23 изображено дерево, полученное с использованием функции `display_tree` файла `utils.py`.

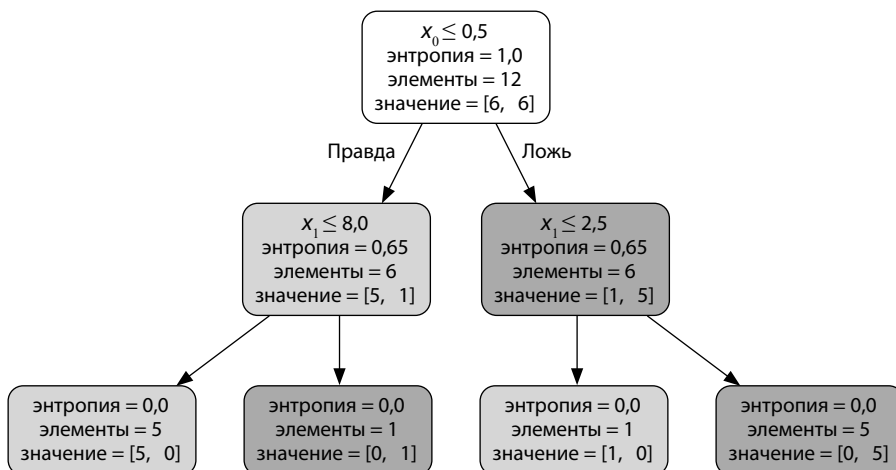


Рис. 9.23. Результирующее дерево решений глубины 2, которое соответствует границе, изображенной на рис. 9.22. Оно содержит три узла и четыре листа

Обратите внимание на то, что дерево на рис. 9.23 точно соответствует границе, полученной на рис. 9.22. Корневой узел соответствует первой вертикальной линии $x_0 = 5$, а точки на каждой стороне линии — двум ветвям. Две горизонтальные линии, $x_1 = 8,0$ и $x_1 = 2,5$, на левой и правой половинах рисунка соответствуют двум ветвям. Кроме того, каждый узел содержит следующую информацию:

- **Джини** — индекс примесей Джини меток в этом узле;
- **элементы** — количество точек данных (элементов), соответствующих этому узлу;
- **значение** — количество точек данных каждой из двух меток в этом узле.

Здесь видно, это дерево было обучено с помощью индекса Джини, который используется по умолчанию в Scikit-Learn. Чтобы обучить его с применением энтропии, мы можем указать это при построении `DecisionTree`:

```
decision_tree = DecisionTreeClassifier(criterion='entropy')
```

При обучении дерева можно указать и другие гиперпараметры, что мы увидим в следующем разделе, рассматривая более объемный пример.

Реальное применение. Моделирование приема студентов с помощью Scikit-Learn

В этом разделе мы используем деревья решений для построения модели, которая прогнозирует поступление в аспирантуру. Набор данных можно найти в Kaggle (ссылка приведена в приложении В). Как и в разделе «Графическая граница деревьев решений», мы будем задействовать Scikit-Learn для обучения дерева решений и Pandas для управления набором данных. Код для этого раздела приведен далее.

- **Блокнот** `University_admissions.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_9_Decision_Trees/University_Admissions.ipynb.

Набор данных `Admission_Predict.csv`.

Набор данных содержит следующие признаки:

- **оценка GRE** — число до 340;
- **оценка TOEFL** — число до 120;
- **рейтинг университета** — число от 1 до 5;
- **убедительность заявления о целях (SOP)** — число от 1 до 5;
- **средний балл бакалавриата (CGPA)** — число от 1 до 10;
- **убедительность рекомендательного письма (LOR)** — число от 1 до 5;
- **опыт исследований** — логическая переменная (0 или 1).

Метки в наборе данных — это вероятность поступления, которая представляет собой число от 0 до 1. Чтобы получить двоичные метки, мы будем считать каждого студента с шансом 0,75 или выше принятым, а любого другого — не принятым.

Код для загрузки набора данных в фрейм данных Pandas и выполнения этого шага предварительной обработки показан далее:

```
import pandas as pd
data = pd.read_csv('Admission_Predict.csv', index_col=0)
data['Admitted'] = data['Chance of Admit'] >= 0.75
data = data.drop(['Chance of Admit'], axis=1)
```

Первые несколько строк результирующего набора данных показаны в табл. 9.5. Здесь приведены баллы студентов в стандартизированных тестах, их оценки, рейтинги университетов, рекомендательные письма, заявления о целях и информация об их шансах на поступление в аспирантуру.

Таблица 9.5. Набор данных с 400 студентами

Оценка GRE	Оценка TOEFL	Рейтинг университета	SOP	LOR	CGPA	Исследования	Принят
337	118	4	4,5	4,5	9,65	1	True
324	107	4	4,0	4,5	8,87	1	True
316	104	3	3,0	3,5	8,00	1	False
322	110	3	3,5	2,5	8,67	1	True
314	103	2	2,0	3,0	8,21	0	False

Как мы уже видели в разделе «Графическая граница деревьев решений», Scikit-Learn требует, чтобы мы вводили признаки и метки по отдельности. Создадим фрейм данных Pandas под названием **features**, содержащий все столбцы, кроме «Принят», и серию Pandas под названием **labels**, содержащую только столбец «Принят». Код:

```
features = data.drop(['Admitted'], axis=1)
labels = data['Admitted']
```

Теперь создадим объект **DecisionTreeClassifier** (который назовем **dt**) и используем метод **fit**. Мы будем обучать его с помощью индекса Джини, как показано далее, поэтому нет необходимости указывать гиперпараметр **criterion**. Но попробуйте обучить его с помощью энтропии и сравните результаты с полученными здесь:

```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier()
dt.fit(features, labels)
```

Для прогнозирования можно использовать функцию **predict**. Например, вот как мы получаем прогнозы для первых пяти студентов:

```
dt.predict(features[0:5])
Output: array([ True,  True, False,  True, False])
```

Однако полученное дерево решений сильно переобучено. Один из способов определить это — применить функцию **score** и получить 100%-ное попадание на обучающем наборе. В этой главе мы не будем тестировать модель, но попробуем создать тестовый набор и проверить, не переобучена ли она. Другой способ определить переобучение — построить дерево и увидеть, что его глубина равна 10 (см. блокнот). В следующем разделе мы узнаем о некоторых гиперпараметрах, которые помогут предотвратить переобучение.

Настройка гиперпараметров в Scikit-Learn

Чтобы предотвратить переобучение, можно использовать гиперпараметры, которые изучили в разделе «Последний шаг. Когда прекратить построение дерева и другие гиперпараметры»:

- `max_depth` — максимально допустимая глубина;
- `max_features` — максимальное количество признаков, учитываемых при каждом разделении (полезно, когда признаков слишком много, а процесс обучения слишком долгий);
- `min_impurity_decrease` — для разделения узла уменьшение примеси должно быть выше этого порога;
- `min_impurity_split` — когда примесь в узле ниже этого порога, узел становится листом;
- `min_samples_leaf` — минимальное количество элементов, требующееся для конечного узла. Если в результате разделения остается лист с меньшим количеством элементов, разделение не выполняется;
- `min_samples_split` — минимальное количество элементов, необходимое для разделения узла. Для определения хорошей модели поиграйте с этими параметрами.

Мы же будем применять следующие:

- `max_depth = 3;`
- `min_samples_leaf = 10;`
- `min_samples_split = 10;`

```
dt_smaller = DecisionTreeClassifier(max_depth=3,
    min_samples_leaf=10, min_samples_split=10)
dt_smaller.fit(features, labels)
```

Результирующее дерево проиллюстрировано на рис. 9.24. Обратите внимание на то, что в нем все ребра справа соответствуют False, а слева — True.

Прогноз на каждом из листьев представляет собой метку, соответствующую большинству узлов в нем. В блокноте каждому узлу присвоен цвет, варьирующийся от оранжевого до синего. Узлы с большим количеством точек с меткой 0 — оранжевые, а с меткой 1 — синие. Обратите внимание на белый лист, на котором одинаковое количество точек с метками 0 и 1. Для него любой прогноз показывает одинаковую эффективность. В этом случае Scikit-Learn по умолчанию использует первый класс в списке, который в данном случае соответствует False.

Чтобы сделать прогноз, мы применим функцию `predict`. Например, спрогнозируем поступление для студента со следующими показателями:

- оценка GRE — 320;
- оценка TOEFL — 110;

- рейтинг университета — 3;
- SOP — 4,0;
- LOR — 3,5;
- CGPA — 8,9;
- исследования — 0 (нет исследований):

```
dt_smaller.predict([[320, 110, 3, 4.0, 3.5, 8.9, 0]])
```

Output: array([True])

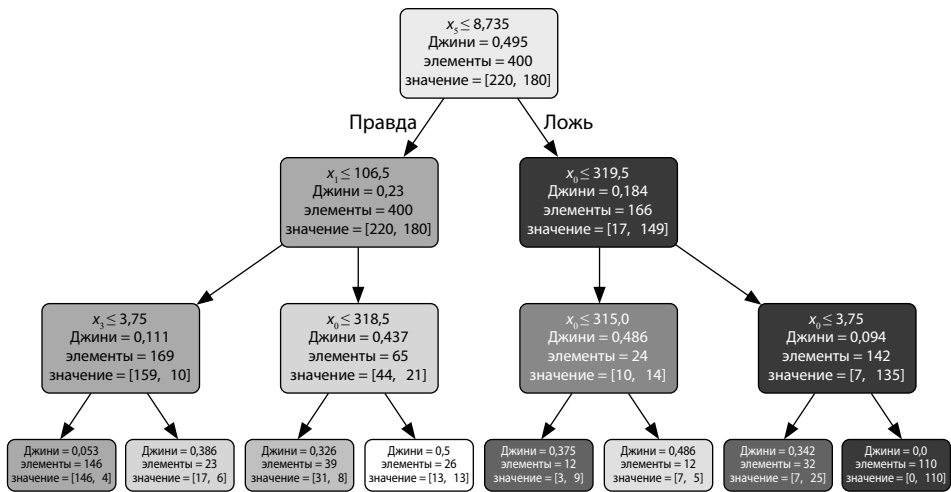


Рис. 9.24. Дерево решений глубины 3, обученное на наборе данных о приеме студентов

Дерево предсказывает, что студент будет принят.

Из этого дерева мы можем сделать следующие выводы о наборе данных.

- Наиболее важным признаком служит шестая колонка (X_5), соответствующая CGPA или оценкам. Оценка отсеечения составляет 8,735 балла из 10. На самом деле большинство прогнозов справа от корневого узла — «принят», а слева — «не принят», что подразумевает, что CGPA — очень значимый признак.
- Следующими после этого признака по важности идут оценка GRE (X_0) и оценка TOEFL (X_1), обе они относятся к стандартизированным тестам. На самом деле большинство студентов, получивших хорошие оценки, скорее всего, будут приняты, если только они не показали плохие результаты GRE, о чем свидетельствует шестой лист слева на дереве на рис. 9.24.
- Помимо оценок и стандартизированных тестов единственный признак, фигурирующий в дереве, — это SOP, или убедительность заявления о целях. Он находится внизу дерева и не сильно меняет прогноз.

Напомним, что построение дерева жадное по своей природе, следовательно, в каждой точке выбирается верхний признак. Однако это не гарантирует выбор наилучшего признака. Например, в процессе может встретиться очень сильная комбинация признаков, но ни один из них не будет сильным по отдельности, так что дерево окажется не в состоянии это уловить. Таким образом, даже получив некоторую информацию о наборе данных, мы не должны отбрасывать признаки, которых нет в дереве. При выборе признаков в этом наборе данных пригодился бы хороший алгоритм выбора свойств, такой как регуляризация L1.

Деревья решений для регрессии

В большей части этой главы мы использовали деревья решений для классификации, но, как упоминалось ранее, они выступают и хорошими регрессионными моделями. В этом разделе мы поговорим о том, как построить регрессионную модель дерева решений. Код для этого раздела приведен далее.

- **Блокнот** `Regression_decision_tree.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_9_Decision_Trees/Regression_decision_tree.ipynb.

Рассмотрим следующую проблему: у нас есть приложение, и мы хотим спрогнозировать уровень вовлеченности пользователей с точки зрения того, сколько дней в неделю они им пользовались. Единственный признак, которым мы располагаем, — это возраст пользователя. Набор данных показан в табл. 9.6, а его график — на рис. 9.25.

Таблица 9.6. Набор данных для восьми пользователей

Возраст, лет	Вовлеченность, дней в неделю
10	7
20	5
30	7
40	1
50	2
60	1
70	5
80	4

Судя по этому набору данных, у нас три группы пользователей. Молодые (10, 20, 30 лет) используют приложение часто, люди среднего возраста (40, 50, 60 лет) — периодически, а пожилые (70, 80 лет) — лишь иногда. Таким образом, разумно предположить следующее:

- если пользователю 34 года или младше, вовлеченность составит 6 дней в неделю;
- если ему от 35 до 64 лет, вовлеченность составит 1 день в неделю;
- если 65 лет и старше, вовлеченность составит 3,5 дня в неделю.

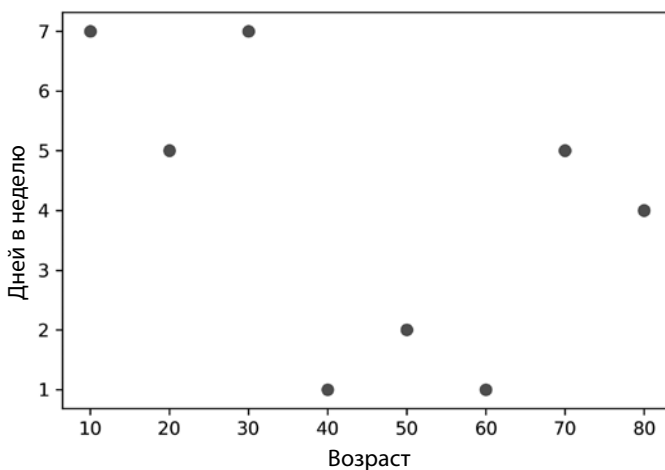


Рис. 9.25. График набора данных из табл. 9.6

Прогнозы дерева решений регрессии выглядят примерно так, потому что дерево решений разбивает пользователей на группы и прогнозирует фиксированное значение для каждой из них. Разделить пользователей можно с помощью признака точно так же, как мы это делали для задач классификации.

К счастью, алгоритм, с помощью которого обучают дерево решений регрессии, очень похож на тот, который мы применяли для обучения дерева решений классификации. Единственное отличие состоит в том, что для деревьев классификации мы использовали достоверность, индекс Джини или энтропию, а для деревьев регрессии — среднюю квадратическую ошибку (MSE). Этот показатель может показаться знакомым — мы применяли его для обучения моделей линейной регрессии в разделе «Как мы измеряем результаты? Функция ошибок» в главе 3.

Прежде чем перейти непосредственно к алгоритму, давайте подумаем о нем как о концепции. Представьте, что вам нужно подогнать прямую как можно ближе к набору данных, приведенному на рис. 9.25. Но есть загвоздка — прямая должна быть горизонтальной. Где ее следует расположить? Имеет смысл поместить ее в середину набора данных — другими словами, на высоте 4, равной среднему значению меток. Это очень простая классификационная модель, которая присваивает каждой точке один и тот же прогноз, равный 4.

Давайте сделаем следующий шаг. Если бы разрешалось использовать только два горизонтальных сегмента, как подогнать их как можно ближе к данным?

Здесь возможно несколько вариантов, один из которых — установить высокую планку для точек слева от 35 и низкую планку — справа. Получается пенек решений, который содержит вопрос: «Вы моложе 35 лет?» — и присваивает прогнозы на основе того, как пользователь на него ответил.

А если бы мы могли разделить каждый из этих двух горизонтальных сегментов еще на два — где следовало бы разместить линии? Мы можем продолжать до тех пор, пока не разделим пользователей на несколько групп с очень схожими метками. Затем спрогнозируем среднюю метку для всех пользователей в группе.

Процедура, которую мы только что выполнили, — это и есть процесс обучения дерева решений регрессии. Теперь подойдем к нему более формально. Напомним, что, когда признак является числовым, мы рассматриваем все возможные способы его разделения. Таким образом, возможными способами разделения возрастного признака являются, например, следующие отсечки: 15, 25, 35, 45, 55, 65 и 75. Каждая из них дает два меньших набора данных, которые мы называем левым и правым наборами данных. Далее выполняем следующие шаги.

1. Для каждого из меньших наборов данных прогнозируем среднее значение меток.
2. Вычисляем среднюю квадратическую ошибку прогноза.
3. Выбираем разграничение, которое дает наименьшую квадратическую ошибку.

Например, если разграничение равно 65, то два набора данных будут следующими:

- **левый** — пользователи моложе 65 лет. Метки имеют вид {7, 5, 7, 1, 2, 1};
- **правый** — пользователи старше 65 лет. Метки имеют вид {5, 4}.

Для каждого набора данных мы прогнозируем среднее значение меток, которое составит 3,833 для левого и 4,5 для правого. Таким образом, прогноз для первых шести пользователей составляет 3,833, а для последних двух — 4,5. Далее вычисляем MSE следующим образом:

$$\begin{aligned} \text{MSE} = (1/8)[(7 - 3,833)^2 + (5 - 3,833)^2 + (7 - 3,833)^2 + (1 - 3,833)^2 + \\ + (2 - 3,833)^2 + (1 - 3,833)^2 + (5 - 4,5)^2 + (4 - 4,5)^2] = 5,167. \end{aligned}$$

В табл. 9.7 мы видим значения, полученные для каждой из возможных отсечек. Каждая отсечка разбивает набор данных на два меньших набора, и для каждого из них прогнозом служит среднее значение меток. Средняя квадратическая ошибка вычисляется как среднее значение квадратов различий между метками и прогнозом. Обратите внимание на то, что разделение с наименьшей MSE получается на отсечке 35. Это дает нам корневой узел в дереве решений. Полные расчеты приведены в конце блокнота для этого раздела.

Таблица 9.7. Девять возможных способов разделения набора данных по возрасту с помощью отсечки

Отсечка	Набор меток		Прогноз для набора		MSE
	Левый	Правый	левого	правого	
0	{}	{7, 5, 7, 1, 2, 1, 5, 4}	None	4,0	5,250
15	{7}	{5, 7, 1, 2, 1, 5, 4}	7,0	3,571	3,964
25	{7, 5}	{7, 1, 2, 1, 5, 4}	6,0	3,333	3,917
35	{7, 5, 7}	{1, 2, 1, 5, 4}	6,333	2,6	1,983
45	{7, 5, 7, 1}	{2, 1, 5, 4}	5,0	3,0	4,250
55	{7, 5, 7, 1, 2}	{1, 5, 4}	4,4	3,333	4,983
65	{7, 5, 7, 1, 2, 1}	{5, 4}	3,833	4,5	5,167
75	{7, 5, 7, 1, 2, 1, 5}	{4}	4,0	4,0	5,250
100	{7, 5, 7, 1, 2, 1, 5, 4}	{}	4,0	None	5,250

Лучшая отсечка — 35 лет, потому что она дала прогноз с наименьшей средней квадратической ошибкой. Таким образом мы построили первый узел принятия решений в дереве решений регрессии. Следующие шаги заключаются в том, чтобы продолжить рекурсивное разделение левого и правого наборов данных подобным образом. Вместо того чтобы делать это вручную, мы, как и раньше, используем Scikit-Learn.

Прежде всего определим признаки и метки. Для этого можем применять массивы, как показано далее:

```
features = [[10],[20],[30],[40],[50],[60],[70],[80]]
labels = [7,5,7,1,2,1,5,4]
```

Теперь построим дерево решений регрессии максимальной глубины 2, используя объект `DecisionTreeRegressor`:

```
from sklearn.tree import DecisionTreeRegressor
dt_regressor = DecisionTreeRegressor(max_depth=2)
dt_regressor.fit(features, labels)
```

Результирующее дерево решений показано на рис. 9.26. Оно имеет три узла принятия решений и четыре листа. Как мы уже выяснили, первая отсечка находится на уровне 35 лет. Следующие две расположены на отметках 15 и 65 лет. Справа на рис. 9.26 видим прогнозы для каждого из этих четырех результирующих подмножеств. Они равны 7, 6, 1,33 и 4,5, что соответствует листьям на дереве.

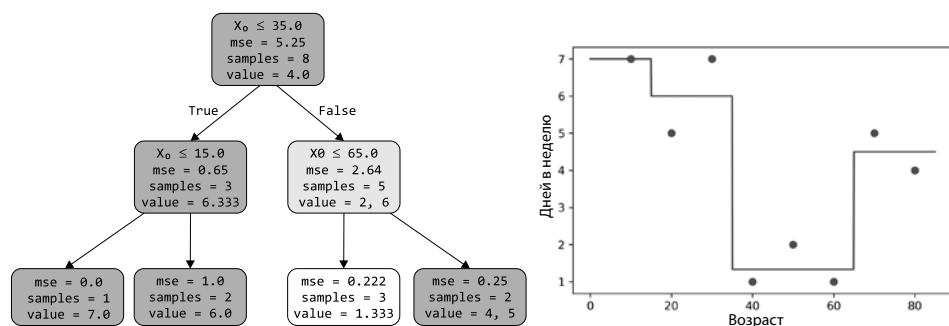


Рис. 9.26. Слева — результирующее дерево решений, полученное в Scikit-Learn; справа — график сделанных им прогнозов

Сфера применения

Деревья решений используются во множестве областей в реальной жизни. Одной из особенностей деревьев решений является то, что, помимо прогнозирования, они дают много информации о данных, поскольку организуют их иерархически. Во многих случаях эта информация имеет такую же или даже бóльшую ценность, как и возможность делать прогнозы. В этом разделе мы рассмотрим примеры деревьев решений, задействованных в реальной жизни в следующих областях:

- здравоохранение;
- системы рекомендаций.

Деревья принятия решений широко используются в здравоохранении

В медицине деревья решений широко применяются не только для составления прогнозов, но и для выявления определяющих признаков в прогнозировании. Разумно предположить, что в медицине черный ящик, выдающий: «Пациент болен» или «Пациент здоров», — это не самое лучшее решение. Однако дерево решений содержит много информации и о том, почему был сделан тот или иной прогноз. Решение о болезни пациента может приниматься исходя из его симптомов, семейной истории болезни, привычек и многих других факторов.

Деревья решений полезны в системах рекомендаций

В рекомендательных системах деревья решений тоже полезны. Одна из самых известных проблем рекомендательных систем — проблема премии Netflix — была решена с помощью деревьев решений. В 2006 году Netflix провела конкурс,

который включал в себя создание наилучшей из возможных систем рекомендаций для прогнозирования пользовательских рейтингов своих фильмов. В 2009 году компания присудила 1 000 000 долларов победителю, который улучшил алгоритм Netflix более чем на 10 %. Способ, которым этого достигли, заключался в использовании деревьев решений с градиентным усилением для объединения более 500 различных моделей. Другие системы рекомендаций применяют деревья решений для изучения вовлеченности пользователей и выявления демографических признаков, чтобы наилучшим образом определить ее.

В главе 12 мы узнаем больше о деревьях решений с градиентным усилением и случайных лесах. А пока определим их как совокупность множества деревьев решений, работающих совместно для получения наилучших прогнозов.

Резюме

- Деревья решений — это важные модели машинного обучения, используемые для классификации и регрессии.
- Принцип работы деревьев решений заключается в том, что мы задаем двоичные вопросы о данных и на основе ответов на них делаем прогноз.
- Алгоритм построения деревьев решений для классификации состоит в нахождении в данных признака, который наилучшим образом определяет метку, и повторении этого шага.
- Есть несколько способов определить, насколько хорошо признак определяет метку. В этой главе мы изучили три из них: достоверность, индекс примеси Джини и энтропию.
- Индекс примесей Джини измеряет чистоту набора. Так, набор, в котором у каждого элемента одна и та же метка, имеет индекс примеси Джини 0. Набор, в котором у каждого элемента своя метка, имеет индекс примеси Джини, близкий к 1.
- Энтропия — еще одна мера чистоты множества. Набор, в котором у каждого элемента одна и та же метка, имеет энтропию 0. Набор, в котором у половины элементов одна метка, а у второй половины — другая, имеет энтропию 1. При построении дерева решений разница в энтропии до и после разделения называется приростом информации.
- Алгоритм построения дерева решений для регрессии очень похож на алгоритм, используемый для классификации. Единственное отличие заключается в том, что для выбора наилучшего признака для разделения данных применяется средняя квадратическая ошибка.
- В двух измерениях древовидные графики регрессии выглядят как объединение нескольких горизонтальных линий, каждая из которых является прогнозом для элементов в конкретном листе.
- Область применения деревьев решений очень широка — от алгоритмов рекомендаций до медицины и биологии.

Упражнения

Упражнение 9.1

В следующей модели дерева решений по обнаружению спама определите, будет ли электронное письмо от вашей мамы с темой «Пожалуйста, зайдите в магазин, там распродажа» классифицировано как спам (рис. 9.27).

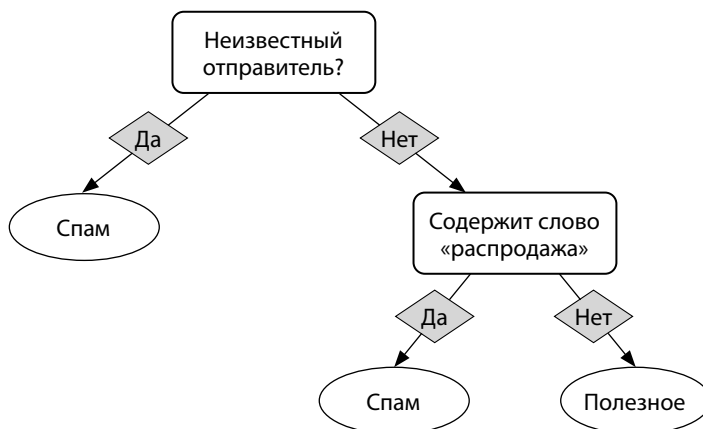


Рис. 9.27

Упражнение 9.2

Наша цель — построить модель дерева решений, определяющую, являются ли транзакции по кредитным картам мошенническими. Мы используем приведенный далее набор данных транзакций по кредитным картам со следующими признаками:

- **сумма** — сумма транзакции;
- **надежный поставщик** — у компании, выпускающей кредитные карты, есть список надежных поставщиков. Эта переменная указывает, входит ли поставщик в данный список.

Транзакция	Сумма, в долларах	Поставщик	Мошенничество
1	100	Ненадежный	Да
2	100	Надежный	Нет
3	10 000	Надежный	Нет
4	10 000	Ненадежный	Да
5	5000	Надежный	Да
6	100	Надежный	Нет

Создайте первый узел дерева решений со следующими спецификациями.

- А. Использование индекса примесей Джини.
- Б. Применение энтропии.

Упражнение 9.3

Далее приводится набор данных о пациентах, у которых был положительный или отрицательный результат теста на COVID-19. У них следующие симптомы: кашель (C), температура (F), затрудненное дыхание (B) и утомляемость (T).

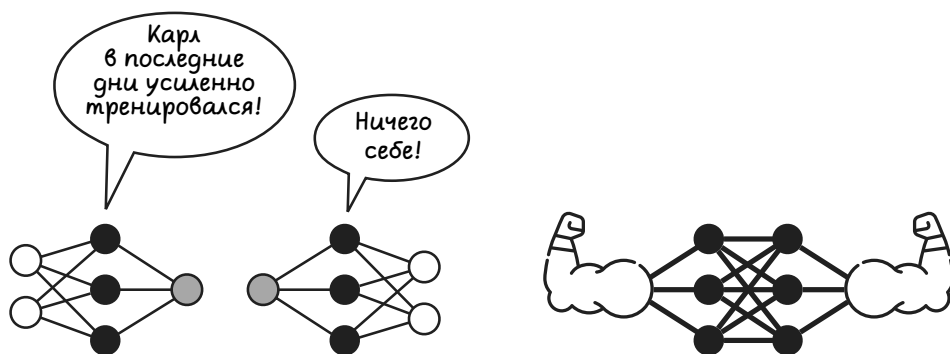
Пациент	Кашель (C)	Температура (F)	Затрудненное дыхание (B)	Утомляемость (T)	Диагноз
1		X	X	X	Болен
2	X	X		X	Болен
3	X		X	X	Болен
4	X	X	X		Болен
5	X			X	Здоров
6		X	X		Здоров
7		X			Здоров
8				X	Здоров

Используя достоверность, постройте дерево решений высотой 1 (пенек решений), которое классифицирует эти данные. Какова достоверность этого классификатора на наборе данных?



В этой главе

- ✓ Что такое нейронная сеть.
 - ✓ Архитектура нейронной сети: узлы, слои, глубина и функции активации.
 - ✓ Обучение нейронных сетей с использованием обратного распространения.
 - ✓ Потенциальные проблемы при обучении нейронных сетей: проблема исчезающего градиента и переобучение.
 - ✓ Методы улучшения обучения нейронных сетей: регуляризация и прореживание.
 - ✓ Использование Keras для обучения нейронных сетей анализу настроений и классификации изображений.
 - ✓ Применение нейронных сетей в качестве регрессионных моделей.
-



В этой главе мы изучаем *нейронные сети*, также называемые *многослойными перцептронами*. Нейронные сети — одна из самых популярных моделей машинного обучения, если не самая популярная. Они оказались настолько полезны, что данная область получила собственное название — *глубокое обучение*. Глубокое обучение нашло множество применений в самых передовых областях машинного обучения, включая распознавание изображений, обработку естественного языка, медицину и беспилотные автомобили. Нейронные сети предназначены для того, чтобы в широком смысле этого слова имитировать работу человеческого мозга. Они могут быть комплексными, как показано на рис. 10.1.

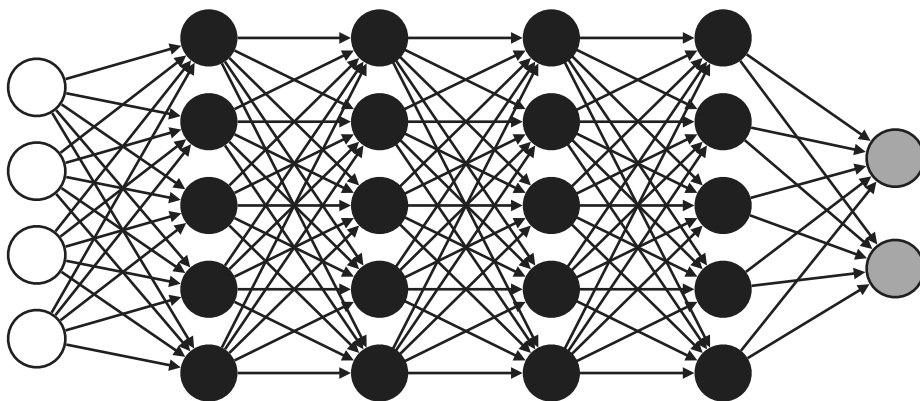


Рис. 10.1. Нейронная сеть. Выглядит чересчур сложно, но на следующих нескольких страницах мы все проясним

Нейронная сеть на рис. 10.1 выглядит устрашающе: множество узлов, ребер и т. д. Однако мы можем понять нейронные сети, используя гораздо более простые концепции. Один из способов разобраться в них — принять их как набор

персептронов (о них мы узнали из глав 5 и 6). Мне нравится рассматривать нейронные сети как собрания линейных классификаторов, которые порождают нелинейные классификаторы. При малых размерах линейные классификаторы будут выглядеть как линии или плоскости, а нелинейные классификаторы — как сложные кривые или поверхности. В этой главе мы обсудим знания, лежащие в основе нейронных сетей, а также подробности того, как они работают. После чего закодируем нейронные сети и применим для ряда задач, таких как распознавание изображений.

Нейронные сети используются для классификации и регрессии. В этой главе мы сосредоточимся в основном на классификационных нейронных сетях, но узнаем также о небольших изменениях, необходимых для того, чтобы они сработали, и для регрессии. Во-первых, добавим терминологию. Напомним, что в главе 5 мы изучили персептрон, а в главе 6 — логистический классификатор. Мы также узнали, что персептроны могут быть дискретными и непрерывными. Напомним: выход дискретного персептрона равен либо 0, либо 1, а выход непрерывного — это любое число в интервале $(0, 1)$. Для вычисления этого выходного сигнала дискретный персептрон использует ступенчатую функцию (см. раздел «Ступенчатая функция и функции активации...» главы 5), а непрерывный персептрон — сигмоидную функцию (см. раздел «Вероятностный подход к классификации. Сигмоидная функция» главы 6). В данной главе мы называем оба классификатора персептронами и при необходимости уточняем, о каком из них идет речь.

Код для этой главы доступен в репозитории GitHub: https://github.com/luiguiserrano/manning/tree/master/Chapter_10_Neural_Networks.

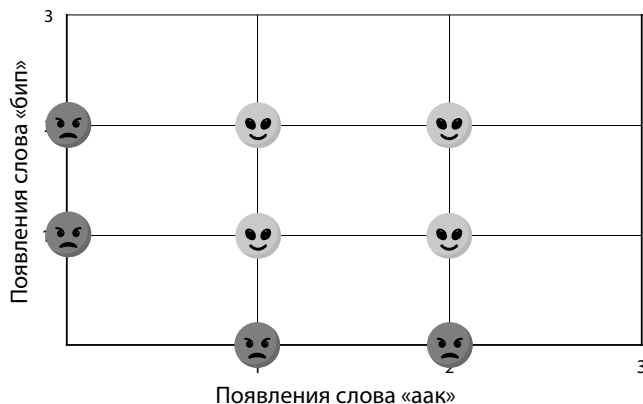
Нейронные сети на примере. Более сложная чужая планета

В этом разделе мы узнаем, что такое нейронная сеть, используя знакомый пример анализа настроений из глав 5 и 6. Сценарий прежний: мы оказываемся на далекой населенной планете. Кажется, что инопланетяне говорят на языке, образованном двумя словами, «аак» и «бип», и мы хотим построить модель машинного обучения, которая поможет определить, радуется инопланетянин или грустит, на основе произносимых им слов. Это называется анализом настроений, так что нам нужно построить модель для анализа настроений инопланетян. Мы записываем разговоры некоторых инопланетян и другими способами определяем, радостные они или грустные, получая таким образом набор данных, показанный в табл. 10.1. Первая графа представляет собой произнесенное предложение. Вторая и третья — количество появлений каждого из слов в предложении. Четвертая отражает настроение инопланетянина.

Таблица 10.1. Набор данных, где каждая строка представляет инопланетянина

Предложение	Аак	Бип	Настроение
Аак	1	0	Грустный
Аак аак	2	0	Грустный
Бип	0	1	Грустный
Бип бип	0	2	Грустный
Аак бип	1	1	Радостный
Аак аак бип	2	1	Радостный
Бип аак бип	1	2	Радостный
Бип аак бип аак	2	2	Радостный

Набор данных выглядит неплохо, и как будто нам вполне по силам подогнать классификатор к этим данным. Давайте сначала построим его график, как показано на рис. 10.2. Горизонтальная ось соответствует количеству появлений слова «аак», а вертикальная ось — слова «бип». Улыбающиеся лица соответствуют радостным инопланетянам, а печальные — грустным.

**Рис. 10.2.** График набора данных из табл. 10.1

Из рис. 10.2 очевидно, что мы не сможем подогнать к этим данным линейный классификатор. Другими словами, провести прямую, разделяющую радостные и грустные лица, невозможно. Что же делать? Мы уже изучили другие классификаторы, которые могут выполнять подобную работу, такие как наивный байесовский классификатор (см. главу 8) или деревья решений (см. главу 9). Но здесь остановимся на персептронах. Если наша цель в том, чтобы разделить точки на рис. 10.2, и одной прямой не обойтись, то что может сработать лучше, чем одна прямая? Как насчет таких вариантов:

1. Две прямые.
2. Кривая.

Это примеры нейронных сетей. Давайте вначале разберемся, почему первый из них (классификатор, задействующий две прямые) представляет собой нейронную сеть.

Решение. Если одной прямой для классификации набора данных недостаточно, используйте две

В этом разделе рассмотрим классификатор, который для разделения набора данных применяет две прямые. Есть множество способов проложить две прямые, разделяющие этот набор данных, и один из них показан на рис. 10.3. Давайте назовем их прямая 1 и прямая 2. Точки над обеими прямыми можно классифицировать как радостные, а остальные — как грустные. Объединение линейных классификаторов таким образом — основа нейронных сетей.

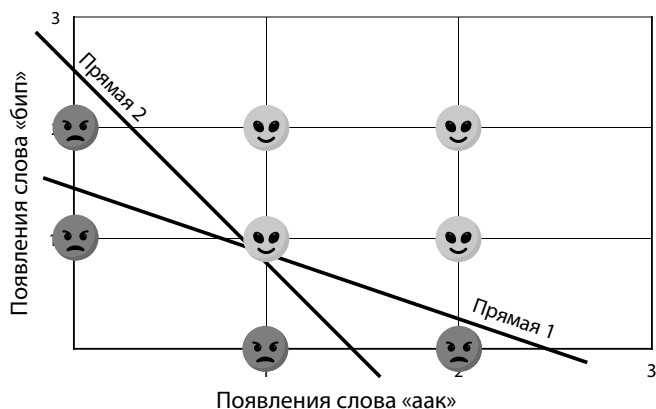


Рис. 10.3. Радостные и грустные точки в наборе данных не получится разделить одной прямой. Однако две прямые хорошо делают это

Мы можем определить наш классификатор следующим образом.

Классификатор анализа настроений

Предложение классифицируется как радостное, если соответствующая ему точка находится над двумя прямыми, показанными на рис. 10.3. Если точка находится ниже хотя бы одной из них, то предложение классифицируется как грустное.

А теперь немного посчитаем. Можем ли мы найти два уравнения для этих прямых? Множество уравнений подойдет, но мы воспользуемся следующими двумя (где x_a — это количество появлений слова «аак», а x_b — количество появлений слова «бип»):

- **прямая 1** — $6x_a + 10x_b - 15 = 0$;
- **прямая 2** — $10x_a + 6x_b - 15 = 0$.

ОТВЛЕЧЕМСЯ. КАК МЫ НАШЛИ ЭТИ УРАВНЕНИЯ? Обратите внимание на то, что прямая 1 проходит через точки (0; 1,5) и (2,5; 0). Следовательно, наклон определяется как изменение горизонтальной оси, деленное на изменение вертикальной оси, $-1,5/2,5 = -3/5$. Это Y-перехват, а именно — высота, на которой линия пересекает вертикальную ось, — 1,5. Таким образом, уравнение этой прямой

$$x_b = -\frac{3}{5}x_a + 1,5.$$

Манипулируя этим уравнением, получаем $6x_a + 10x_b - 15 = 0$.

Мы можем использовать аналогичный подход для нахождения уравнения для прямой 2.

Таким образом, наш классификатор выглядит следующим образом. Предложение классифицируется как радостное, если выполняются оба следующих неравенства:

- $6x_a + 10x_b - 15 \geq 0$;
- $10x_a + 6x_b - 15 \geq 0$.

Если хотя бы одно из них не выполняется, то предложение классифицируется как грустное.

В качестве проверки согласованности табл. 10.2 содержит значения каждого из двух уравнений. Справа от каждого из них мы сверяем значение уравнения с 0. В крайней правой графе сверяем с 0 оба значения. Четвертая и шестая графы соответствуют двум приведенным ранее неравенствам. В пятой и седьмой проверяется, дает ли уравнение каждой из прямых в каждой из точек данных неотрицательное значение. В последней графе проверяется, являются ли оба полученных значения неотрицательными.

Таблица 10.2. Тот же набор данных, что и в табл. 10.1, но с новыми графами

Предложение	Аак	Бип	Уравнение 1	Уравнение 1 ≥ 0 ?	Уравнение 2	Уравнение 2 ≥ 0 ?	Оба уравнения ≥ 0
Аак	1	0	-9	Нет	-5	Нет	Нет
Аак аак	2	0	-3	Нет	5	Да	Нет
Бип	0	1	-5	Нет	-9	Нет	Нет
Бип бип	0	2	5	Да	3	Нет	Нет
Аак бип	1	1	1	Да	1	Да	Да
Аак аак бип	1	2	11	Да	7	Да	Да
Бип аак бип	2	1	7	Да	11	Да	Да
Бип аак бип аак	2	2	17	Да	17	Да	Да

Обратите внимание на то, что крайняя правая графа в табл. 10.2 (да/нет) совпадает с крайней правой графой в табл. 10.1 (радостный/грустный). Это означает, что классификатору удалось правильно классифицировать все данные.

Почему именно две прямые? Разве счастье не линейно?

В главах 5 и 6 нам удалось сделать выводы о языке на основе уравнений классификаторов. Например, если вес слова «аак» был положительным, мы приходили к выводу, что это, скорее всего, радостное слово. А что же сейчас? Можем ли мы сделать какие-либо выводы о языке в классификаторе, который содержит два уравнения?

Обоснование наличия двух уравнений заключается в том, что, возможно, на чужой планете счастье — это не простая линейная вещь и основано оно на двух факторах. В реальной жизни счастье может базироваться на многом: успешной карьере в сочетании со счастливой семейной жизнью и едой на столе. Оно может быть основано на возможности выпить кофе с пончиком. В нашем случае предположим, что два аспекта счастья — это карьера и семья. Чтобы инопланетянин был радостным, у него должно быть и то и другое.

Похоже, что в данном случае и карьера, и семейное счастье являются простыми линейными классификаторами и каждый из них описывается одной из двух прямых. Предположим, что прямая 1 соответствует карьере, а прямая 2 — семейному счастью. Таким образом, мы можем представить себе инопланетное счастье в виде диаграммы, изображенной на рис. 10.4. На ней успешная карьера и семейное счастье объединены оператором AND, который проверяет, положительны ли оба значения. Если да, то инопланетянин радостный. Если какой-либо из них имеет отрицательное значение, он недоволен.

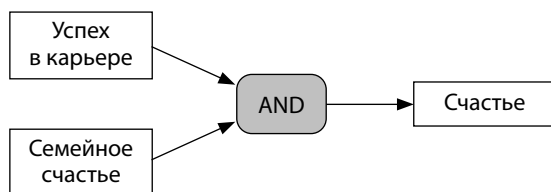


Рис. 10.4. Классификатор счастья формируется классификатором успеха в карьере, классификатором семейного счастья и оператором AND

Классификаторы карьеры и семейного счастья являются персептронами, поскольку задаются уравнением прямой. Можем ли мы превратить оператор AND в еще один персептрон? Ответ — да, и мы увидим, как это сделать, в следующем разделе.

Рисунок 10.4 начинает выглядеть как нейронная сеть. Еще несколько шагов и не-много математики, и мы доберемся до чего-то гораздо больше похожего на рис. 10.1.

Объединение выходных сигналов персептронов в другой персептрон

Рисунок 10.4 намекает на комбинацию персептронов, в которой мы подключаем выходы двух персептронов в качестве входных сигналов к третьему персептрон. Именно так строятся нейронные сети, и в этом разделе рассмотрим стоящую за этим математику.

В разделе «Ступенчатая функция и функции активации...» главы 5 мы определили ступенчатую функцию, которая возвращает 0, если входные данные отрицательны, и 1, если они положительны или равны нулю. Обратите внимание: поскольку мы используем ступенчатую функцию, это дискретные персептроны. Применяя эту функцию, мы можем определить классификаторы семейного счастья и карьеры следующим образом.

Классификатор успеха в карьере

Веса:

- «аак» — 6;
- «бип» — 10.

Смещение — 15.

Оценка предложения $6x_a + 10x_b - 15$.

Прогноз $C = \text{step}(6x_a + 10x_b - 15)$.

Классификатор семейного счастья

Веса:

- «аак» — 10;
- «бип» — 6.

Смещение — 15.

Оценка предложения $10x_a + 6x_b - 15$.

Прогноз $F = \text{step}(10x_a + 6x_b - 15)$.

Следующий шаг — включить выходные данные классификаторов семейного счастья и карьеры в новый классификатор счастья. Попробуйте проверить, работает ли такой классификатор. Рисунок 10.5 содержит две таблицы с результатами классификаторов карьеры и семейного счастья, а также таблицу, в которой первые две графы представляют собой выходные данные классификатора карьеры и семьи, а последняя — выходные данные классификатора счастья. Каждая из таблиц на рис. 10.5 соответствует персептрон.

Классификатор карьеры				Классификатор семьи				Классификатор радости			
x_a	x_b	$6x_a + 10x_b - 15$	$C = \text{step}(6x_a + 10x_b - 15)$	x_a	x_b	$10x_a + 6x_b - 15$	$F = \text{step}(10x_a + 6x_b - 15)$	C	F	$1 \cdot C + 1 \cdot F - 1.5$	$\hat{y} = \text{step}(C + F - 1.5)$
1	0	-9	0	1	0	-5	0	0	0	-1.5	0
2	0	-3	0	2	0	5	1	0	1	-0.5	0
0	1	-5	0	0	1	-9	0	0	0	-1.5	0
0	2	5	1	0	2	3	0	1	0	-0.5	0
1	1	1	1	1	1	1	1	1	1	0.5	1
1	2	11	1	1	2	7	1	1	1	0.5	1
2	1	7	1	2	1	11	1	1	1	0.5	1
2	2	17	1	2	2	17	1	1	1	0.5	1

Рис. 10.5. Три классификатора персептрона: для успеха в карьере, для семейного счастья и для радости, объединяющий два предыдущих. Выходные данные персептронов карьеры и семьи являются входными данными персептрона радости

Классификатор радости

Веса:

- карьера — 1;
- семья — 1;

Смещение -1,5.

Оценка предложения $1C + 1F - 1,5$.

Прогноз $\hat{y} = \text{step}(1C + 1F - 1,5)$.

Эта комбинация классификаторов представляет собой нейронную сеть. Далее мы увидим, как сделать ее похожей на изображение на рис. 10.1.

Графическое представление персептронов

В этом разделе я покажу, как графически отображать персептроны, что подведет нас к графическому представлению нейронных сетей. Мы называем их нейронными сетями, потому что их базовая единица — персептрон — отдаленно напоминает нейрон.

Нейрон состоит из трех основных частей: сомы, дендритов и аксона. В общих чертах: нейрон получает сигналы, поступающие от других нейронов через дендриты, обрабатывает их в соме и посылает через аксон сигнал,

который будет принят другими нейронами. Сравните это с персептроном, который принимает числа в качестве входных данных, применяет к ним математическую операцию, обычно состоящую из суммы, составленной с помощью функции активации, и выдает новое число. Процесс проиллюстрирован на рис. 10.6.

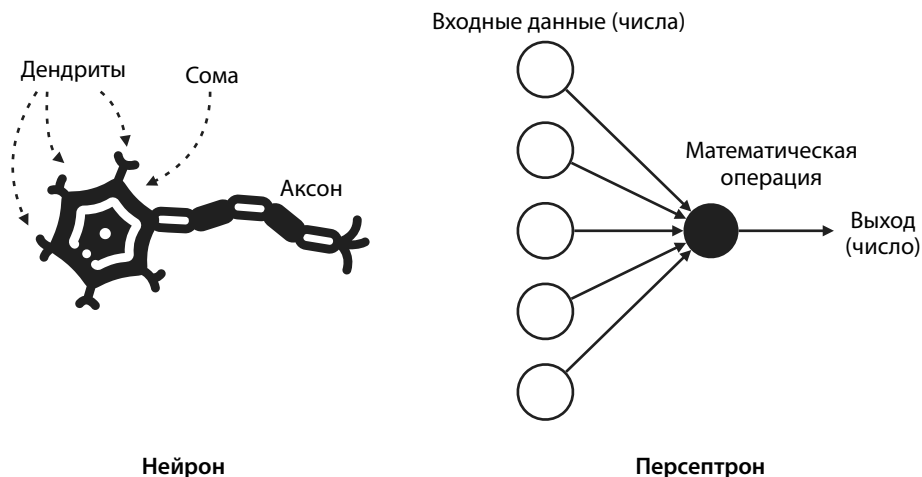


Рис. 10.6. Персептрон отдаленно напоминает нейрон

Подойдем более формально и вспомним определение персептрона из глав 5 и 6, в котором у нас имелись следующие сущности:

- **входные данные** $x_1, x_2, \dots, x_n$;
- **веса** $w_1, w_2, \dots, w_n$;
- **смещение** b ;
- **функция активации** — либо ступенчатая функция (для дискретных персептронов), либо сигмоидная функция (для непрерывных персептронов). (Далее в этой главе мы познакомимся с новыми функциями активации.);
- **прогноз**, определяемый по формуле: $\hat{y} = f(w_1x_1 + w_2x_2 + \dots + w_nx_n + b)$, где f — соответствующая функция активации.

То, как они расположены на схеме, показано на рис. 10.7: слева входные узлы, а справа — выходной узел. Входные переменные поступают на входные узлы. Конечный входной узел содержит не переменную, а значение 1. Веса расположены на ребрах, соединяющих входные узлы с выходным. Вес, соответствующий конечному входному узлу, — это смещение. Математические операции для вычисления прогноза выполняются внутри выходного узла, и он же выводит прогноз, заданный формулой: $\hat{y} = \sigma(3x_1 - 2x_2 + 4x_3 + 2)$.

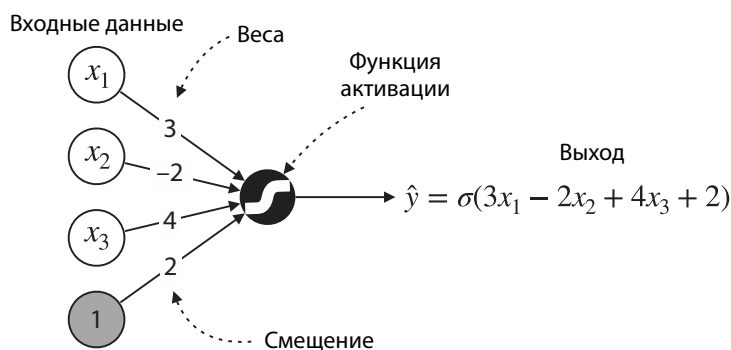


Рис. 10.7. Визуальное представление персептрона

Персептрон, определяемый уравнением $\hat{y} = \sigma(3x_1 - 2x_2 + 4x_3 + 2)$, показан на рис. 10.7. Обратите внимание на то, что в этом персептроне выполняются следующие шаги.

- Входные данные умножаются на их соответствующие веса и складываются, чтобы получить $3x_1 - 2x_2 + 4x_3$.
- Смещение добавляется к предыдущему уравнению, чтобы получить $3x_1 - 2x_2 + 4x_3 + 2$.
- Сигмоидная функция активации применяется для получения выходного значения $\hat{y} = \sigma(3x_1 - 2x_2 + 4x_3 + 2)$.

Например, если входным сигналом для этого персептрона является точка $(x_1, x_2, x_3) = (1, 3, 1)$, тогда на выходе получим:

$$\sigma(3 \times 1 - 2 \times 3 + 4 \times 1 + 2) = \sigma(3) = 0,953.$$

Если бы персептрон был определен с использованием ступенчатой функции вместо сигмоидной, результат был бы таким: $\text{step}(3 \times 1 - 2 \times 3 + 4 \times 1 + 2) = \text{step}(3) = 1$.

Это графическое представление упрощает объединение персептронов, как мы убедимся в дальнейшем.

Графическое представление нейронных сетей

Из предыдущего раздела мы узнали, что нейронная сеть — это объединение персептронов. Эта структура предназначена для приблизительной имитации человеческого мозга, в котором выходные данные нескольких нейронов становятся входными данными для другого нейрона. Точно так же в нейронной сети выходные данные нескольких персептронов становятся входными данными другого персептрона, как показано на рис. 10.8.

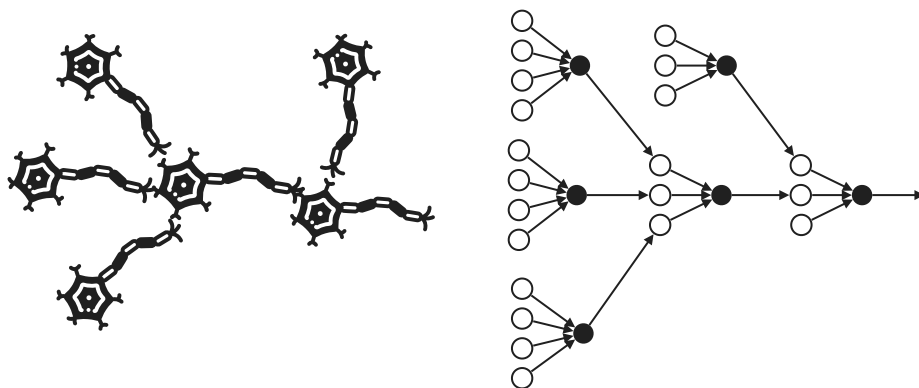


Рис. 10.8. Нейронные сети предназначены для имитации структуры мозга (в общих чертах)

Нейронная сеть, которую мы построили в предыдущем разделе, объединив персептрон успеха в карьере с персептроном семейного счастья, показана на рис. 10.9. Она использует ступенчатую функцию в качестве функции активации.

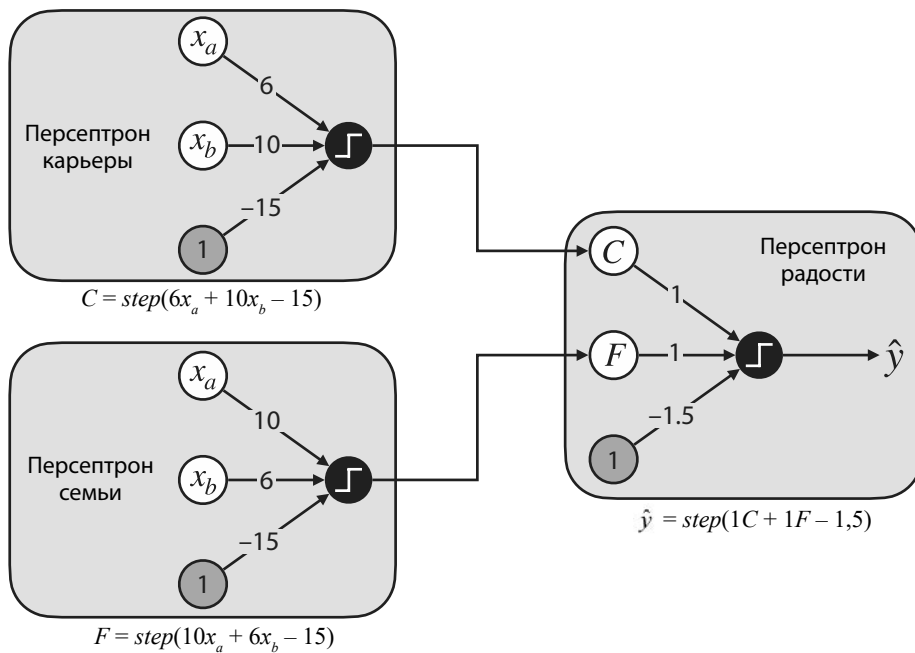


Рис. 10.9. Подключая выходные данные персептронов карьеры и семьи к персептрону радости, получаем нейронную сеть

Обратите внимание на то, что на диаграмме на рис. 10.9 входные данные для персептронов карьеры и семьи повторяются. Более чистый способ записи, при котором входные данные не повторяются, показан на рис. 10.10. На этой диаграмме ни свойства x_a и x_b , ни смещение не повторяются. Вместо этого все они подключаются к обоим узлам справа, красиво объединяя три персептрона в одну диаграмму.

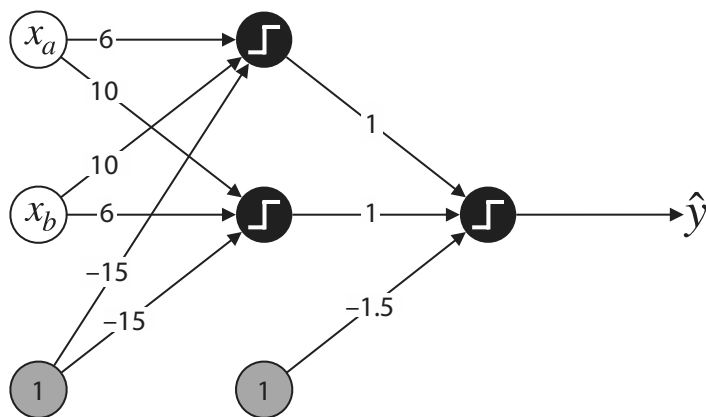


Рис. 10.10. Подчищенная версия диаграммы, приведенной на рис. 10.9

Эти три персептрона используют ступенчатую функцию. Мы сделали так исключительно в образовательных целях, потому что в реальной жизни нейронные сети никогда не задействуют ступенчатую функцию в качестве функции активации, поскольку она делает невозможным применение градиентного спуска (подробнее об этом рассказывается в разделе «Обучение нейронных сетей»). Сигмоидная же функция очень широко используется в нейронных сетях, а в разделе «Различные функции активации...» мы узнаем о некоторых других полезных функциях активации, применяемых на практике.

Граница нейронной сети

В главах 5 и 6 мы изучали границы персептронов, которые задаются прямыми. В этом разделе увидим, как выглядят границы нейронных сетей.

Напомним, что как дискретный, так и непрерывный персептрон (логистический классификатор) имеют линейную границу, заданную определяющим их линейным уравнением. Дискретный персептрон присваивает точкам прогнозы 0 и 1 в зависимости от того, по какую сторону прямой они находятся. Непрерывный персептрон присваивает каждой точке плоскости прогноз в диапазоне от 0 до 1. Точки над прямой получают прогноз 0,5, точки на одной стороне прямой — прогнозы выше 0,5, а точки на другой стороне — прогнозы ниже 0,5. Рисунок 10.11 иллюстрирует дискретные (*слева*) и непрерывные (*справа*) персептроны, соответствующие уравнению: $10x_a + 6x_b - 15 = 0$.

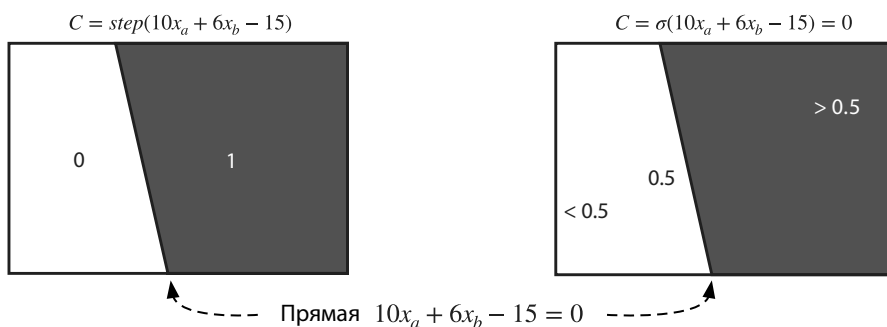


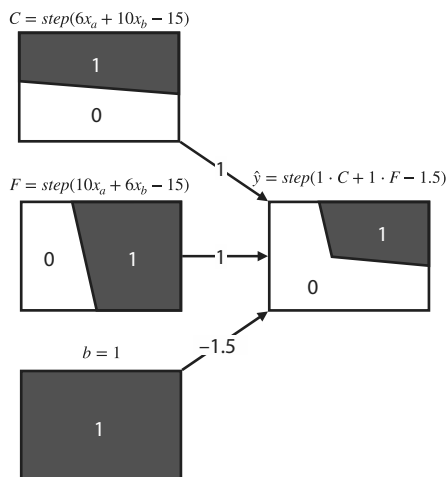
Рис. 10.11. Персептроны, соответствующие уравнению: $10x_a + 6x_b - 15 = 0$. Граница персептрона представляет собой прямую

Аналогичным образом мы можем визуализировать и выходные данные нейронной сети. Напомним, что вывод нейронной сети со ступенчатой функцией активации выглядит следующим образом:

- если $6x_a + 10x_b - 15 \geq 0$ и $10x_a + 6x_b - 15 \geq 0$, то результат равен 1;
- в противном случае вывод равен 0.

Эта граница проиллюстрирована в левой части рис. 10.12 двумя прямыми. Обратите внимание на то, что она задается как комбинация границ двух входных персептронов и узла смещения. Граница, полученная с помощью ступенчатой функции активации, выполнена ломаными линиями, тогда как граница, полученная с помощью сигмоидной функции активации, представляет собой кривую.

Со ступенчатой функцией активации



С сигмоидной функцией активации

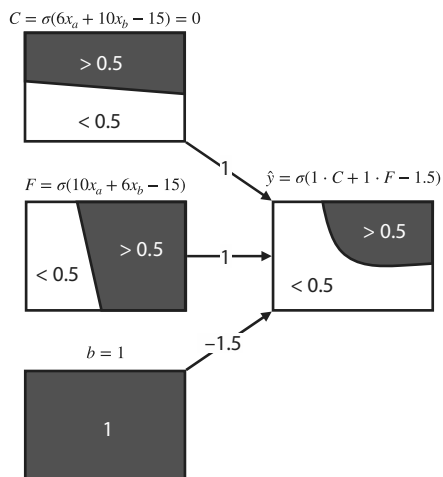


Рис. 10.12. Для построения нейронной сети используются выходные данные двух персептронов и узла смещения (он представлен классификатором, который всегда выдает значение 1) для третьего персептрона

Чтобы более тщательно изучить вопрос границ, загляните в блокнот: https://github.com/luisguiserrano/manning/blob/master/chapter_10_news_networks/Plotting_Boundaries.ipynb. В нем границы двух прямых и двух нейронных сетей нанесены на график с помощью функций ступенчатой и сигмоидной активации (рис. 10.13).

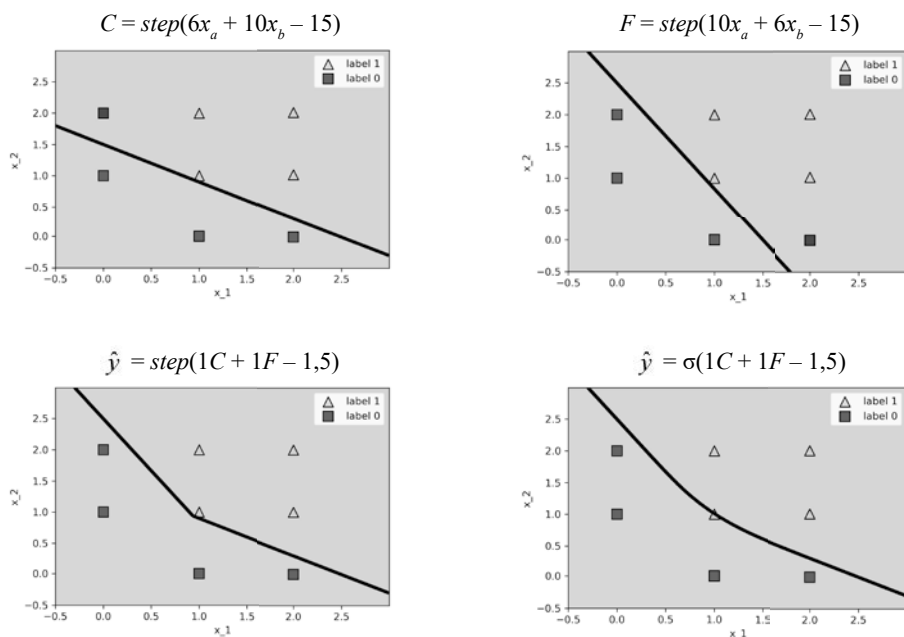


Рис. 10.13. Графики границ классификаторов: *вверху* — линейные классификаторы карьеры (*слева*) и семьи (*справа*); *внизу* — нейронные сети, использующие ступенчатую (*слева*) и сигмоидную (*справа*) функции

Обратите внимание на то, что нейронная сеть с сигмоидной функцией активации на самом деле плохо подходит для всего набора данных, потому что неправильно классифицирует точку (1, 1), что показано в правом нижнем углу рис. 10.13. Попробуйте изменить вес таким образом, чтобы он хорошо соответствовал данной точке (см. упражнение 10.3).

Общая архитектура полносвязной нейронной сети

В предыдущих разделах мы видели пример небольшой нейронной сети, но в реальной жизни сети намного больше. Узлы расположены слоями, как показано на рис. 10.14. Первый слой — входной, последний слой — выходной, а все промежуточные слои называются скрытыми. Все узлы слоя соединены со всеми узлами (не являющимися смещением) следующего слоя. Расположение узлов и слоев называется *архитектурой* нейронной сети. Количество слоев, исключая

входной, называется *глубиной* нейронной сети. Нейронная сеть на рис. 10.14 имеет глубину 3 и следующую архитектуру:

- входной слой размером 4;
- скрытый слой размером 5;
- скрытый слой размером 3;
- выходной слой размером 1.

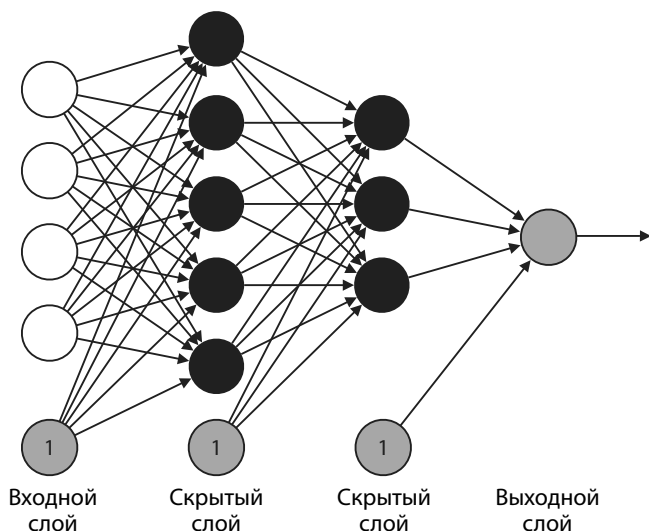


Рис. 10.14. Общая архитектура нейронной сети

Нейронные сети часто рисуют без узлов смещения, но предполагается, что они являются частью архитектуры. Однако мы не учитываем узлы смещения в архитектуре. Другими словами, размер слоя — это количество узлов в нем без узлов смещения.

Обратите внимание на то, что в нейронной сети, изображенной на рис. 10.14, каждый узел слоя связан с каждым узлом (не узлом смещения) следующего слоя. Кроме того, между непоследовательными слоями не происходит никаких соединений. Эта архитектура называется *полносвязной*. Для каких-то приложений мы используем различающиеся архитектуры, где присутствуют не все соединения или некоторые узлы соединены между непоследовательными слоями, — см. раздел «Другие архитектуры для более сложных наборов данных», чтобы узнать о некоторых из них. Однако в этой главе все нейронные сети, которые мы строим, полносвязные.

Изобразите границу нейронной сети, подобной показанной на рис. 10.15. На этой диаграмме вы можете увидеть классификатор, соответствующий каждому узлу. Этот классификатор имеет четко заданную границу. Границы узлов в каждом слое формируются путем объединения границ из предыдущего слоя.

Обратите внимание на то, что первый скрытый слой образован линейными классификаторами. Все узлы в нем соответствуют линейным классификаторам (персептронам), поэтому имеют вид прямых. Классификаторы в каждом последующем слое немного сложнее, чем в предыдущих. На этой диаграмме мы удалили узлы смещения.

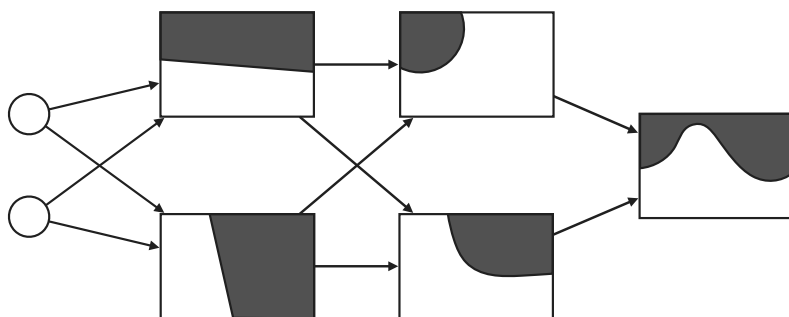


Рис. 10.15. То, как мне нравится визуализировать нейронные сети. Каждый из узлов соответствует классификатору

Отличным инструментом для понимания нейронных сетей служит площадка Playground TensorFlow, которая находится по адресу: <https://playground.tensorflow.org>. Там имеется несколько графических наборов данных и можно обучать нейронные сети с различными архитектурами и гиперпараметрами.

Обучение нейронных сетей

Из этой главы мы узнали, как выглядят нейронные сети в целом и что они совсем не так загадочны, как кажутся. Как же нам обучить одного из этих монстров? Теоретически процесс несложен, хотя и может оказаться дорогостоящим с точки зрения вычислений. Есть несколько приемов и эвристик, которые мы можем использовать, чтобы его ускорить. В данном разделе изучим процесс обучения. Обучение нейронной сети не очень отличается от обучения других моделей, таких как персептрон или логистический классификатор. Мы начинаем с инициализации всех весов и смещений случайным образом. Далее определяем функцию ошибки для измерения производительности нейронной сети. Наконец, многократно применим функцию ошибок для настройки весов и смещений модели, чтобы уменьшить функцию ошибок.

Функция ошибок. Способ измерения производительности нейронной сети

В этом разделе мы узнаем о функции ошибок, используемой для обучения нейронных сетей. К счастью, мы уже видели эту функцию раньше — функцию логарифмических потерь из раздела «Логистические классификаторы...» главы 6.

Вспомним формулу логарифмической потери:

$$\text{логарифмическая потеря} = -y \ln(\hat{y}) - (1-y) \ln(1-\hat{y}),$$

где y — метка, $\hat{y}$ — прогноз.

В качестве дополнения: хорошей причиной для использования логарифмической потери в задачах классификации является тот факт, что она возвращает небольшое значение, когда прогноз и метка находятся близко, и большое — когда далеко.

Обратное распространение — ключевой шаг в обучении нейронной сети

В этом разделе мы изучим ключевой шаг в процессе обучения нейронной сети. Напомним, что в главах 3, 5 и 6 (линейная регрессия, алгоритм персептрона и логистическая регрессия) для обучения моделей использовался градиентный спуск. Это относится и к нейронным сетям. Алгоритм обучения называется *алгоритмом обратного распространения*, и его псевдокод выглядит так.

Псевдокод алгоритма обратного распространения

- Инициализировать нейронную сеть со случайными весами и смещениями.
- Повторять многократно:
 - вычислить функцию потерь и ее градиент, а именно — производные по каждому из весов и смещений;
 - сделать небольшой шаг в направлении, противоположном градиенту, чтобы уменьшить функцию потерь на небольшую величину.
- Полученные веса соответствуют нейронной сети, которая, вероятно, хорошо соответствует данным.

Функция потерь нейронной сети сложна, потому что включает в себя логарифм прогноза, а прогноз сам является сложной функцией. Кроме того, нам нужно вычислить производную по многим переменным, соответствующим каждому из весов и смещений нейронной сети. В приложении Б, в разделе «Использование градиентного спуска для обучения нейронных сетей», мы рассмотрим математические детали алгоритма обратного распространения для нейронной сети с одним скрытым слоем или произвольного размера. Обратитесь к некоторым ресурсам, рекомендуемым в приложении Б, чтобы углубиться в математику обратного распространения для более глубоких нейронных сетей. На практике отличные пакеты, такие как Keras, TensorFlow и PyTorch, реализуют этот алгоритм с большой скоростью и производительностью. Вспомните, что, когда мы изучали модели линейной регрессии (см. главу 3), дискретные персептроны (см. главу 5) и непрерывные персептроны (см. главу 6), в процессе всегда был шаг, на котором мы перемещали прямую так, как было нужно для хорошего моделирования данных. Такую геометрию сложнее визуализировать в случае нейронных сетей, потому что все происходит в гораздо более высоких

измерениях. Однако мы все же способны сформировать мысленную картину обратного распространения, и для этого нам нужно сосредоточиться только на одном из узлов нейронной сети и одной точке данных. Представьте себе классификатор, подобный тому, что показан справа на рис. 10.16. На каждом этапе процесса обучения веса ребер обновляются. Если классификатор хороший, то его вес увеличивается на небольшую величину, а если плохой, то уменьшается.

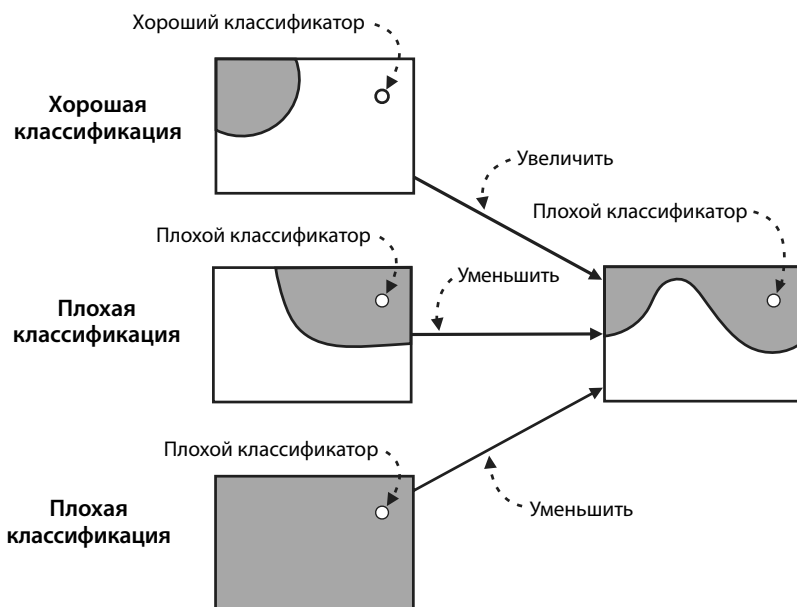


Рис. 10.16. Мысленная картина обратного распространения

Этот классификатор получен из трех классификаторов, размещенных слева (нижний соответствует смещению, которое мы представляем классификатором, который всегда возвращает прогноз 1). Результирующий классификатор неправильно классифицирует точку. Из трех входных классификаторов первый хорошо классифицирует точку, а два других — нет. Таким образом, шаг обратного распространения увеличит вес на ребре, соответствующем верхнему классификатору, и уменьшит вес, соответствующий двум классификаторам внизу. Это гарантирует, что результирующий классификатор будет больше похож на верхний и, таким образом, его классификация для точки улучшится.

Возможные проблемы. От переобучения до исчезающих градиентов

На практике нейронные сети работают очень хорошо. Но они сложны, и из-за этого возникает много проблем с их обучением. К счастью, для самых насущных у нас есть решение. Одна из проблем, с которой сталкиваются нейронные сети,

заключается в переобучении — поистине большие архитектуры потенциально могут запоминать данные, не обобщая их должным образом. В следующем разделе мы рассмотрим некоторые методы уменьшения переобучения при обучении нейронных сетей.

Еще одна серьезная проблема, с которой могут столкнуться нейронные сети, — это исчезающие градиенты. Обратите внимание на то, что сигмоидная функция (рис. 10.17) очень плоская на концах. Это означает: производные (касательные к кривой) слишком плоские, то есть их наклон очень близок к нулю. А это затрудняет обучение.

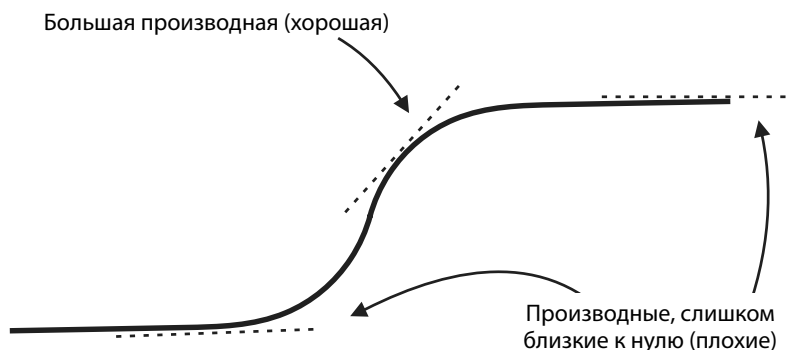


Рис. 10.17. Сигмоидная функция

В процессе обратного распространения мы складываем многие из этих сигмоидных функций, то есть повторно делаем выходные данные сигмоидной функции входными данными другой сигмоидной функции. Такая композиция ожидаемо дает производные, очень близкие к нулю, что означает: шаги, принятые во время обратного распространения, будут крошечными. Если это так, то может потребоваться очень много времени, чтобы найти хороший классификатор, а это проблема.

Есть несколько решений проблемы исчезающего градиента, на данный момент одно из наиболее эффективных — изменение функции активации. В разделе «Различные функции активации...» мы изучим кое-какие новые функции активации, которые помогут справиться с проблемой исчезающего градиента.

Методы обучения нейронных сетей — регуляризация и прореживание

Как упоминалось в предыдущем разделе, нейронные сети склонны к переобучению. Здесь мы обсудим ряд методов уменьшения переобучения во время обучения нейронных сетей.

Как выбрать правильную архитектуру? Это сложный вопрос, на который нет конкретного ответа. Эмпирическое правило состоит в том, чтобы сделать ошибку, выбрав архитектуру избыточного размера, а затем применить методы,

позволяющие уменьшить переобучение, которое может возникнуть в сети. В некотором смысле это похоже на выбор пары штанов, где есть только одна альтернатива — слишком маленькие или слишком большие. Выбрав слишком маленькие брюки, мы мало что сможем сделать. А если выберем слишком большие, то можем надеть ремень, чтобы они не спадали. Решение не идеальное, но единственное на данный момент. Выбор правильной архитектуры на основе набора данных — сложная проблема, и сейчас проводится множество исследований в этом направлении. Чтобы узнать о них больше, ознакомьтесь с ресурсами, приведенными в приложении В.

Регуляризация — способ уменьшить переобучение путем наказания за более высокие веса

Как известно из главы 4, можно использовать регуляризацию L1 и L2 для уменьшения переобучения в регрессионных и классификационных моделях, и нейронные сети не исключение. Способ применения регуляризации в нейронных сетях такой же, как и при линейной регрессии, — добавлением слагаемого регуляризации к функции ошибки. Если мы выполняем регуляризацию L1, слагаемое регуляризации равно параметру регуляризации (λ), умноженному на сумму абсолютных значений всех весов модели, за исключением смещения. Если выполняем регуляризацию L2, то вместо абсолютных значений берем сумму квадратов. Так, ошибка регуляризации L2 нейронной сети в примере, приведенном в разделе «Нейронные сети на примере», составляет:

$$\begin{aligned} \text{логарифмическая потеря} + \lambda(6^2 + 10^2 + 10^2 + 6^2 + 1^2 + 1^2) = \\ = \text{логарифмическая потеря} + 274\lambda. \end{aligned}$$

Прореживание — исключение того, что несколько сильных узлов не доминируют в обучении

Прореживание — это интересная техника, используемая для уменьшения переобучения в нейронных сетях. Чтобы понять ее, рассмотрим следующую аналогию: представьте, что мы правши и нам нравится ходить в спортзал. Через некоторое время мы замечаем, что наш правый бицепс растет быстрее левого. Затем начинаем тренироваться больше и понимаем: поскольку мы правши, то склонны поднимать тяжести правой рукой и не даем достаточную нагрузку на левую. Мы решаем, что так не пойдет, и принимаем решительные меры. Иногда мы привязываем правую руку за спиной и выполняем упражнение, вообще не используя ее. После этого замечаем, что левая рука начинает расти, что и требовалось. Теперь, чтобы заставить работать обе руки, мы делаем следующее: каждый день перед спортзалом подбрасываем две монеты, по одной на каждую руку. Если на левой монете выпадает орел, привязываем за спиной левую руку, а если на правой — правую. В какие-то дни мы будем работать обеими руками, в какие-то — только

одной, а какие-то дни будут вообще без рук (возможно, это дни тренировки ног). Случайность результата подбрасывания монет гарантирует, что в среднем мы работаем обеими руками почти одинаково.

Прореживание использует подобную логику, за исключением того, что вместо рук мы тренируем веса в нейронной сети. Когда в ней слишком много узлов, некоторые из них улавливают закономерности в данных, полезные для составления хороших прогнозов, а другие — закономерности, которые являются шумом или не относятся к делу. Процесс прореживания случайным образом в каждый период удаляет некоторые узлы и выполняет один шаг градиентного спуска для оставшихся. Отбрасывая ряд узлов, вполне вероятно иногда удалить и те, которые распознали полезные шаблоны, тем самым заставляя другие узлы заполнить пробелы.

Если конкретнее, то процесс прореживания придает каждому из нейронов небольшую вероятность p . В каждый период процесса обучения каждый нейрон удаляется с вероятностью p и нейронная сеть обучается только с оставшимися. Прореживание используется только для скрытых слоев, но не для входных или выходных. Процесс прореживания проиллюстрирован на рис. 10.18, где некоторые нейроны удаляются на каждом из четырех периодов обучения. В разные периоды мы выбираем случайные узлы для удаления из обучения, чтобы дать всем узлам возможность обновить веса и при этом несколько отдельных узлов не превратились в доминирующие.

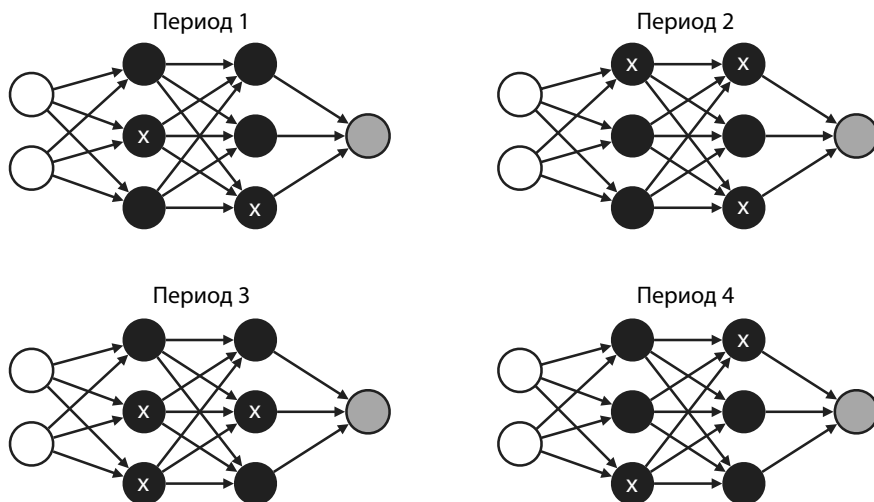


Рис. 10.18. Процесс прореживания

Прореживание очень хорошо работает на практике, и я призываю вас всегда использовать его при обучении нейронной сети. Как мы увидим позже в этой главе, задействуемые для обучения нейронных сетей библиотеки упрощают это.

Различные функции активации. Гиперболический тангенс и блок линейной ректификации

Как мы видели в разделе «Возможные проблемы...», сигмоидная функция слишком плоская, что порождает проблемы с исчезающими градиентами. Решение заключается в использовании других функций активации. В этом разделе мы рассмотрим две другие функции активации, которые имеют решающее значение для улучшения процесса обучения: гиперболический тангенс ($\tanh$) и блок линейной ректификации (ReLU).

Гиперболический тангенс

На практике функция *гиперболического тангенса*, как правило, работает лучше сигмоидной из-за своей формы. Задаётся она следующей формулой:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Тангенс чуть менее плоский, чем сигмоидная функция, но все равно имеет похожую форму (рис. 10.19). Это обеспечивает заметное улучшение по сравнению с сигмоидной функцией, но проблема исчезающего градиента по-прежнему существует.

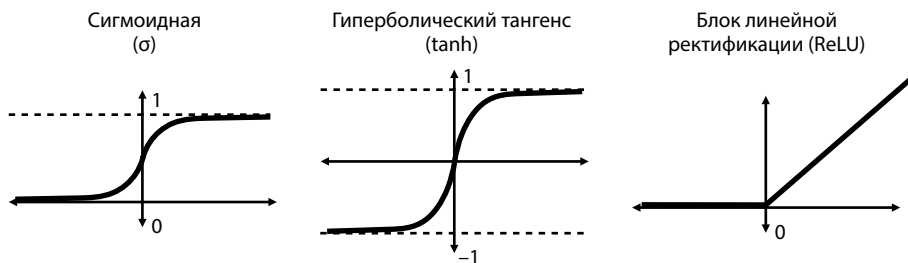


Рис. 10.19. Функции активации, используемые в нейронных сетях

Блок линейной ректификации

Гораздо более распространенной активацией, которая обычно применяется в нейронных сетях, является *блок линейной ректификации*. Она работает просто: если входные данные отрицательны, то выходные данные равны нулю, в противном случае выходные данные равны входным. Другими словами, она оставляет неотрицательные числа неизменными, а все отрицательные превращает в ноль. Для $x \geq 0$ $\text{ReLU}(x) = x$, а для $x < 0$ $\text{ReLU}(x) = 0$. ReLU служит хорошим решением проблемы исчезающего градиента, поскольку ее производная равна 1, когда входные данные положительны. Благодаря этому функция широко используется в масштабных нейронных сетях.

Самое замечательное в функциях активации то, что можно комбинировать разные функции в одной и той же нейронной сети. В одной из наиболее распространенных архитектур каждый узел использует функцию активации ReLU, за исключением последнего, который применяет сигмоидную. Причина ее появления в конце заключается в том, что, если задача является проблемой классификации, выходные данные нейронной сети должны быть между 0 и 1.

Нейронные сети с более чем одним выходом. Функция softmax

До сих пор нейронные сети, с которыми мы работали, имели только один выход. С помощью функции softmax нетрудно построить нейронную сеть, имеющую несколько выходов. Функция softmax — это многомерное расширение сигмоидной функции, и ее можно использовать для преобразования оценок в вероятности.

Лучше всего проиллюстрировать функцию softmax на примере. Представьте, что у нас есть нейронная сеть, задача которой состоит в том, чтобы определить, содержит ли изображение трубказуба, птицу, кошку или собаку. В последнем слое имеются четыре узла, по одному на каждое животное. Вместо применения к полученным из предыдущего слоя наборам сигмоидной функции применяем к каждому функцию softmax. Например, если оценки равны 0, 3, 1 и 1, softmax возвращает следующее:

$$\text{вероятность (трубказуб)} = \frac{e^0}{e^0 + e^3 + e^1 + e^1} = 0,0377;$$

$$\text{вероятность (птица)} = \frac{e^3}{e^0 + e^3 + e^1 + e^1} = 0,7573;$$

$$\text{вероятность (кошка)} = \frac{e^1}{e^0 + e^3 + e^1 + e^1} = 0,1025;$$

$$\text{вероятность (собака)} = \frac{e^1}{e^0 + e^3 + e^1 + e^1} = 0,1025.$$

Результаты указывают: нейронная сеть убеждена в том, что изображение соответствует птице.

Гиперпараметры

Как и большинство алгоритмов машинного обучения, нейронные сети используют множество гиперпараметров, которые мы можем точно настроить, чтобы заставить их работать еще лучше. Эти гиперпараметры определяют то, как проходит обучение, а именно: насколько долгим должен быть процесс, с какой скоростью и как мы будем вводить наши данные в модель. Далее приведены некоторые из наиболее важных гиперпараметров в нейронных сетях:

- **скорость обучения η** — размер шага, который используется во время обучения;
- **количество периодов** — количество шагов, сделанных в ходе обучения;
- **пакетный, или мини-пакетный, или стохастический градиентный спуск** — сколько точек одновременно вводится в процесс обучения, то есть вводятся точки по одной, партиями или все сразу;
- **архитектура:**
 - количество слоев в нейронной сети;
 - количество узлов на слой;
 - функции активации, используемые в каждом узле;
- **параметры регуляризации:**
 - регуляризация L1 или L2;
 - слагаемое регуляризации λ ;
- **вероятность прореживания p .**

Мы настраиваем эти гиперпараметры так же, как и для других алгоритмов, используя такие методы, как поиск по сетке. В главе 13 подробнее разберем эти методы на примере из реальной жизни.

Кодирование нейронных сетей в Keras

Теперь, когда мы изучили теорию, лежащую в основе нейронных сетей, пришло время применить ее на практике! Для разработки нейронных сетей создано множество замечательных пакетов, таких как Keras, TensorFlow и PyTorch. Все три средства очень эффективны, но в этой главе будем использовать Keras из-за его простоты. Мы построим две нейронные сети для двух наборов данных. Первый набор содержит точки с двумя признаками, а также метки 0 и 1. Набор данных двумерный, поэтому мы сможем наблюдать нелинейную границу, созданную моделью. Второй набор — это обычный набор данных, задействуемый в распознавании изображений, называемый набором данных MNIST (модифицированный NIST). Набор данных MNIST содержит рукописные цифры, которые можно классифицировать с помощью нейронной сети.

Графический пример в двух измерениях

В этом разделе мы обучим нейронную сеть в Keras на наборе данных, показанном на рис. 10.20. Набор данных содержит две метки, 0 и 1. Точки с меткой 0 обозначены квадратами, а точки с меткой 1 — треугольниками. Обратите внимание на то, что точки с меткой 1 расположены в основном в центре, тогда как точки с меткой 0 — по сторонам. Для такого набора данных нам нужен классификатор с нелинейной границей, что делает его хорошей задачей для нейронной сети. Код для этого раздела приведен далее.

- **Блокнот** Graphical_example.ipynb:
https://github.com/luisguiserrano/manning/blob/master/Chapter_10_Neural_Networks/Graphical_example.ipynb
- **Набор данных** one_circle.csv.

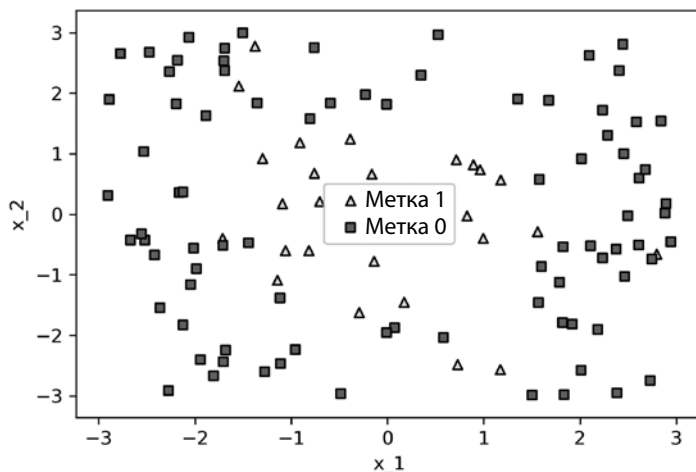


Рис. 10.20. Нейронные сети отлично подходят для нелинейно разделяемых множеств

Но прежде взглянем на некоторые случайные строки в данных. Входные данные будут называться x с признаками x_1 и x_2 , а выходные данные будут называться y . В табл. 10.3 приведены некоторые примеры данных. Набор данных состоит из 110 строк.

Таблица 10.3. Набор данных из 110 строк с двумя признаками и метками 0 и 1

x_1	x_2	y
-0,759416	2,753240	0
-1,885278	1,629527	0
...	...	...
0,729767	-2,479655	1
-1,715920	-0,393404	1

Прежде чем мы построим и обучим нейронные сети, понадобится кое-какая предварительная обработка данных.

Категоризация данных. Превращение небинарных объектов в двоичные

В этом наборе данных выходные данные — это числа 0 или 1, но они представляют собой два класса. В Keras рекомендуется классифицировать такой тип выходных данных. Это просто означает, что точки с меткой 0 теперь будут иметь метку [1, 0], а точки с меткой 1 — метку [0, 1]. Мы делаем это с помощью функции `to_categorical` следующим образом:

```
from tensorflow.keras.utils import to_categorical
categorized_y = np.array(to_categorical(y, 2))
```

Новые метки называются `categorized_y`.

Архитектура нейронной сети

Построим архитектуру нейронной сети для этого набора данных. Решение о том, какую архитектуру применять, — это не точная наука, но обычно рекомендуется делать немного больше, а не немного меньше. Для этого набора данных используем следующую архитектуру с двумя скрытыми слоями (рис. 10.21):

- входной слой, размер 2;
- первый скрытый слой:
 - размер 128;
 - функция активации ReLU;
- второй скрытый слой:
 - размер 64;
 - функция активации ReLU;
- выходной слой:
 - размер 2;
 - функция активации softmax.

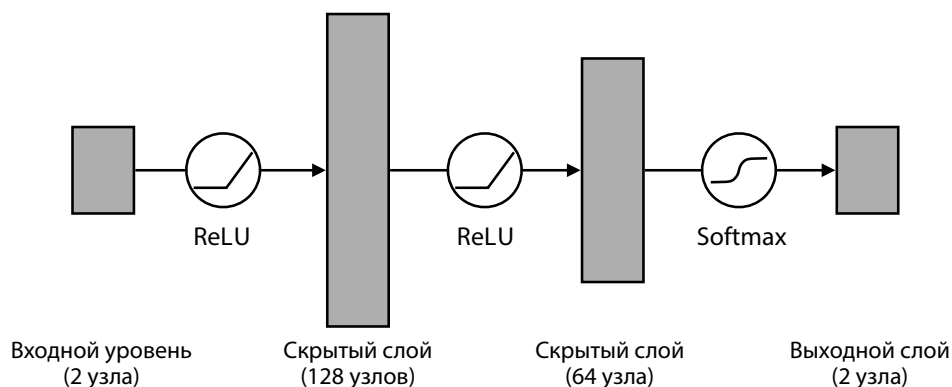


Рис. 10.21. Архитектура для классификации набора данных

Кроме того, добавим к скрытым слоям прореживание с вероятностью 0,2, чтобы предотвратить переобучение.

Построение модели в Keras

Построение нейронной сети в Keras требует всего нескольких строк кода. Прежде всего импортируем необходимые пакеты и функции:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Activation
```

Теперь перейдем к заданию модели с архитектурой, которую мы определили в предыдущем разделе. Сначала определяем саму модель:

```
model = Sequential()
model.add(Dense(128, activation='relu', input_shape=(2,)))
model.add(Dropout(.2))
model.add(Dense(64, activation='relu'))
model.add(Dropout(.2))
model.add(Dense(2, activation='softmax'))
```

Определяет модель

Добавляет первый скрытый слой с функцией активации ReLU

Добавляет прореживание с вероятностью 0,2

Добавляет второй скрытый слой с функцией активации ReLU

Добавляет выходной слой с функцией активации softmax

Как только модель определена, можем скомпилировать ее:

```
model.compile(loss = 'categorical_crossentropy', optimizer='adam',
              metrics=['accuracy'])
```

Параметры функции `compile` такие:

- `loss='categorical_crossentropy'`. Это функция потерь, которую мы определяем как логарифмическую потерю. Поскольку метки содержат более одного столбца, нужно использовать многомерную версию функции логарифмических потерь, называемую *категориальной кросс-энтропией*;
- `optimizer='adam'`. Пакеты, подобные Keras, имеют множество встроенных методов, которые помогают оптимизировать обучение модели. Всегда полезно использовать такой оптимизатор в процессе обучения. Одни из лучших — Adam, SGD, RMSProp и AdaGrad. Попробуйте провести то же обучение с другими оптимизаторами и посмотрите, как работают они;
- `metrics=['accuracy']`. По ходу обучения мы получаем отчеты о работе модели в каждом периоде. Этот флаг позволяет указать показатели, которые мы хотим видеть во время обучения, и для этого примера мы выбрали достоверность.

Запустив код, получим сводную информацию об архитектуре и ряде параметров:

Model: "sequential"

Layer (type)	Output Shape	Param #
=====	=====	=====
dense (Dense)	(None, 128)	384
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 2)	130
=====	=====	=====
Total params:	8770	
Trainable params:	8770	
Non-trainable params:	0	

Каждая строка в этом выводе представляет собой слой (в описательных целях прореженные слои рассматриваются как отдельные). Столбцы соответствуют типу слоя, форме (количеству узлов) и количеству параметров, что в точности соответствует количеству весов и смещений. Эта модель имеет в общей сложности 8770 обучаемых параметров.

Обучение модели

Для обучения достаточно одной простой строки кода:

```
model.fit(x, categorized_y, epochs=100, batch_size=10)
```

Рассмотрим каждый из входных данных этой функции подгонки:

- **x** и **categorized_y** — признаки и метки соответственно;
- **epochs** — количество запусков обратного распространения по всему набору данных, в данном случае 100 раз;
- **batch_size** — длина пакетов, использованных для обучения модели. Мы вводим данные в модель партиями по 10 штук. Для такого небольшого набора данных не требуется вводить их пакетно, но в этом примере делаем это для наглядности.

По мере обучения модель выводит некоторую информацию для каждого периода, а именно потери (функция ошибок) и достоверность. Обратите внимание на то, что первый период имеет высокие потери и низкую достоверность, тогда как последний — гораздо лучшие оба показателя:


```

Epoch 1/100
11/11 [=====] - 0s 2ms/step - loss: 0.5473 -
      accuracy: 0.7182
...
Epoch 100/100
11/11 [=====] - 0s 2ms/step - loss: 0.2110 -
      accuracy: 0.9000

```

Конечная достоверность модели после обучения составляет 0,9. Это хорошо, хотя нужно помнить, что достоверность должна быть рассчитана на тестовом наборе. Я этого делать не буду, но вы попробуйте самостоятельно разделить набор данных на обучающий и тестовый наборы, заново обучить нейронную сеть и посмотреть, какая точность получится для тестового набора. На рис. 10.22 показан график границы нейронной сети.

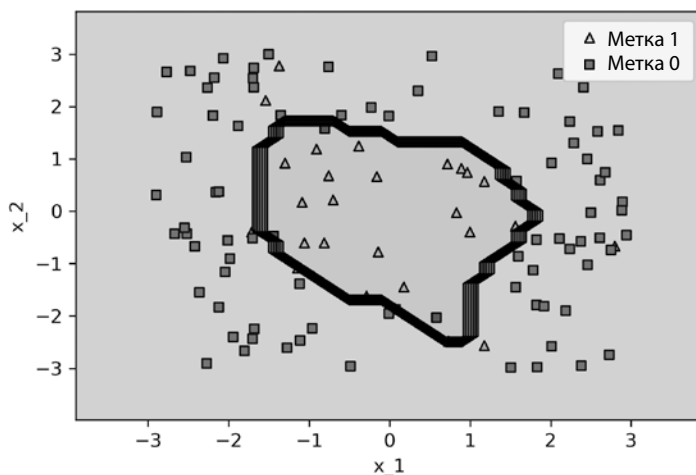


Рис. 10.22. Граница обученного классификатора нейронной сети

Обратите внимание на то, что модели удалось довольно хорошо классифицировать данные, обведя треугольники и оставив квадраты снаружи. Классификатор допустил кое-какие ошибки из-за зашумленных данных, но это норма. Граница указывает на небольшой уровень переобучения, но в целом все выглядит как хорошая модель.

Обучение нейронной сети для распознавания изображений

В этом разделе узнаем, как обучить нейронную сеть распознавать изображения. Набор данных, который мы используем, — это MNIST, популярный набор данных для распознавания изображений, который содержит 70 000 рукописных цифр от 0 до 9. Метка каждого изображения — это соответствующая

цифра. Каждое изображение (в оттенках серого) представляет собой матрицу 28×28 чисел от 0 до 255, где 0 представляет белый цвет, 255 — черный, а любое промежуточное число — оттенок серого. Код для этого раздела приведен далее.

- **Блокнот** `Image_recognition.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_10_Neural_Networks/Image_recognition.ipynb.
- **Набор данных** MNIST (поставляется с Keras).

Загрузка данных

Этот набор данных уже предварительно загружен в Keras, поэтому его легко загрузить в массивы NumPy. Фактически он уже разделен на обучающий и тестовый наборы размером 60 000 и 10 000 соответственно. Следующие строки кода загрузят их в массивы NumPy:

```
from tensorflow import keras
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
```

На рис. 10.23 показаны первые пять изображений набора данных и их метки.



Рис. 10.23. Примеры рукописных цифр в MNIST

Предварительная обработка данных

Нейронные сети в качестве входных данных получают векторы вместо матриц, поэтому мы должны превратить каждое изображение размером 28×28 в вектор длиной $28^2 = 784$. Для этого можно использовать функцию `reshape`:

```
x_train_reshaped = x_train.reshape(-1, 28*28)
x_test_reshaped = x_test.reshape(-1, 28*28)
```

Как и в предыдущем примере, мы должны классифицировать метки. Поскольку метка представляет собой число от 0 до 9, ее нужно превратить в вектор длиной 10, в котором позиция, соответствующая метке, равна 1, а остальные — 0. Мы можем проделать это с помощью следующих строк кода:

```
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

Процесс проиллюстрирован на рис. 10.24.

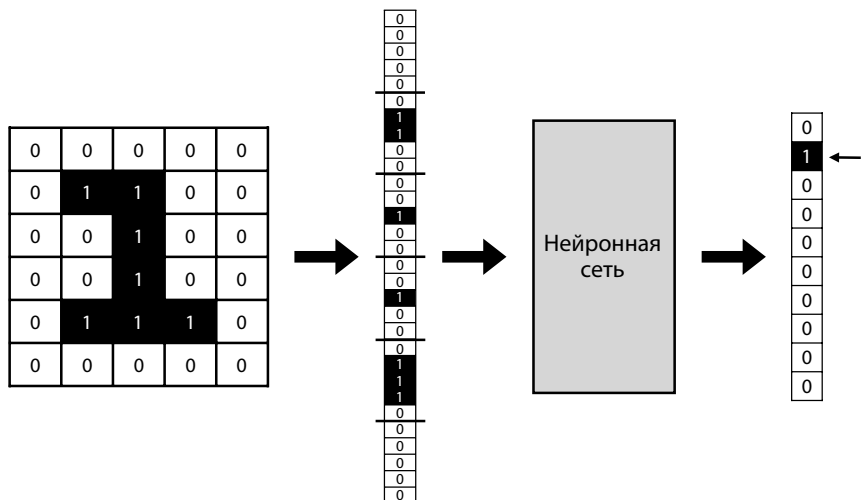


Рис. 10.24. Классификация меток

Построение и обучение модели

Мы можем использовать ту же архитектуру, что и в предыдущей модели, с небольшими изменениями, поскольку входные данные теперь имеют размер 784. В следующих строках кода определяем модель и ее архитектуру:

```
model = Sequential()
model.add(Dense(128, activation='relu', input_shape=(28*28,)))
model.add(Dropout(.2))
model.add(Dense(64, activation='relu'))
model.add(Dropout(.2))
model.add(Dense(10, activation='softmax'))
```

Теперь скомпилируем и обучим модель за десять периодов с размером пакета 10, как показано далее. Модель имеет 109 386 обучаемых параметров, поэтому обучение в течение десяти периодов может занять несколько минут:

```
model.compile(loss = 'categorical_crossentropy', optimizer='adam',
              metrics=['accuracy'])
model.fit(x_train_reshaped, y_train_cat, epochs=10, batch_size=10)
```

Глядя на выходные данные, мы видим, что модель имеет достоверность обучения 0,9164, что хорошо, но давайте оценим тестовую достоверность, чтобы убедиться, что модель не переобучена.

Оценка модели

Можно оценить достоверность на тестовом наборе, сделав прогнозы для него и сравнив их с метками. Нейронная сеть выводит векторы длиной 10 с вероятностями, которые присваивает каждой из меток, поэтому мы можем получить прогнозы, взглянув на запись максимального значения в этом векторе:

```
predictions_vector = model.predict(x_test_reshaped)
predictions = [np.argmax(pred) for pred in predictions_vector]
```

Сравнивая их с метками, получаем тестовую достоверность 0,942, что довольно хорошо. Мы можем добиться большего успеха с более сложными архитектурами, такими как сверточные нейронные сети (подробнее об этом в следующем разделе), но приятно знать, что с помощью небольшой полносвязной нейронной сети можно довольно неплохо справиться с задачей распознавания изображений.

Давайте рассмотрим некоторые прогнозы. На рис. 10.25 мы видим правильный (*слева*) и неправильный (*справа*). Обратите внимание на то, что неправильный прогноз — это плохо написанная цифра 3, которая слегка напоминает цифру 8.

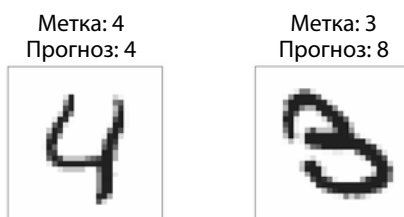


Рис. 10.25. Слева — изображение цифры 4, которое было правильно классифицировано нейронной сетью; справа — изображение цифры 3, которая была неправильно классифицирована как 8

Данное упражнение демонстрирует, насколько прост процесс обучения такой большой нейронной сети с помощью всего нескольких строк кода в Keras! Конечно, здесь огромный простор для творчества. Поиграйте с блокнотом, добавьте больше слоев в нейронную сеть, измените гиперпараметры и посмотрите, насколько вы сможете повысить тестовую достоверность этой модели!

Нейронные сети для регрессии

На протяжении всей этой главы мы изучали, как использовать нейронные сети в качестве модели классификации, но эти сети полезны и как регрессионные модели. К счастью, для превращения классификационной нейронной сети в регрессионную необходимо выполнить всего две небольшие настройки. Первая заключается в удалении конечной сигмоидной функции из нейронной сети.

Роль этой функции состоит в том, чтобы превратить входные данные в число 0 или 1, поэтому, если мы удалим ее, нейронная сеть сможет возвращать любое число. Вторая настройка заключается в изменении функции ошибки на абсолютную или среднюю квадратическую ошибку, поскольку эти функции ошибок относятся к регрессии. Все остальное останется прежним, включая обучение.

В качестве примера давайте посмотрим на персептрон на рис. 10.7. Он делает прогноз: $\hat{y} = \sigma(3x_1 - 2x_2 + 4x_3 + 2)$. Если мы уберем сигмоидную функцию активации, новый персептрон (рис. 10.26) сделает прогноз: $\hat{y} = 3x_1 - 2x_2 + 4x_3 + 2$. Обратите внимание на то, что он представляет собой модель линейной регрессии.

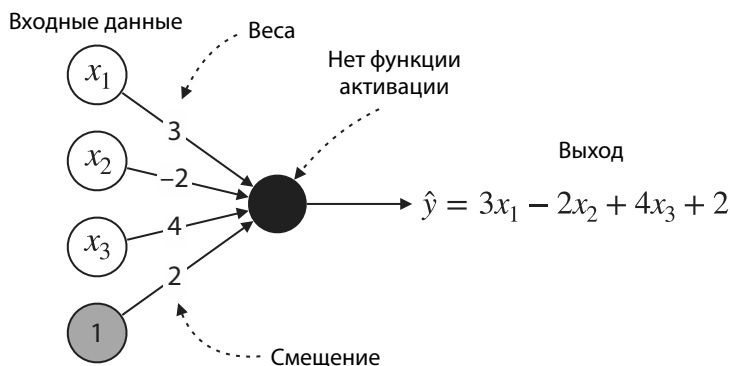


Рис. 10.26. Если мы удалим функцию активации из персептрона, то превратим классификационную модель в модель линейной регрессии

Чтобы проиллюстрировать процесс, обучим нейронную сеть в Keras на знакомом наборе данных — ценах на жилье в Хайдарабаде. Напомним, что в разделе «Реальное применение: использование Turi Create для прогнозирования цен на жилье в Индии» в главе 3 мы обучали линейную регрессионную модель, чтобы она соответствовала этому набору данных. Код для этого раздела приведен далее:

- **Блокнот** `House_price_predictions_neural_network.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_10_Neural_Networks/House_price_predictions_neural_network.ipynb.
- **Набор данных** `Hyderabad.csv`.

Подробные сведения о загрузке набора данных и его разделении на признаки и метки можно найти в блокноте. Архитектура нейронной сети, которую мы будем использовать, выглядит следующим образом:

- входной слой размером 38 (количество столбцов в наборе данных);
- скрытый слой размером 128 с функцией активации ReLU и параметром прореживания 0,2;

- скрытый слой размером 64 с функцией активации ReLU и параметром прореживания 0,2;
- выходной слой размером 1 без функции активации:

```
model = Sequential()
model.add(Dense(38, activation='relu', input_shape=(38,)))
model.add(Dropout(.2))
model.add(Dense(128, activation='relu'))
model.add(Dropout(.2))
model.add(Dense(64, activation='relu'))
model.add(Dropout(.2))
model.add(Dense(1))
```

Для обучения нейронной сети задействуем функцию средней квадратической ошибки и оптимизатор Adam. Мы будем обучаться в течение 10 периодов, используя размер пакета 10:

```
model.compile(loss = 'mean_squared_error', optimizer='adam')
model.fit(features, labels, epochs=10, batch_size=10)
```

Нейронная сеть сообщает о средней квадратической ошибке 5 535 425 в обучающем наборе данных. Изучите эту модель поглубже, добавив тестовый набор, поиграйте с архитектурой и посмотрите, насколько вы сможете ее улучшить.

Другие архитектуры для более сложных наборов данных

Нейронные сети полезны во многих областях, возможно, даже больше, чем любой другой существующий алгоритм машинного обучения. Одно из самых важных качеств нейронных сетей — их универсальность. Мы можем интересными способами изменять архитектуру, чтобы она лучше соответствовала нашим данным и решала поставленную задачу. Чтобы больше узнать об этих архитектурах, ознакомьтесь с книгой *Grokking Deep Learning* Эндрю Траска (Manning, 2019) и набором видеороликов, доступных в приложении В или на сайте <https://serrano.academy/neural-networks/>.

Как видят нейронные сети. Сверточные нейронные сети

Из этой главы мы узнали, что нейронные сети отлично справляются с изображениями и могут использоваться во многих областях, например:

- **распознавание изображений.** Входным сигналом служит изображение, а выходным — метка на нем. Далее приведены некоторые популярные наборы данных, применяемые для распознавания изображений:
 - MNIST — рукописные цифры на изображениях в масштабе 28×28 в сером цвете;

- CIFAR-10 — цветные изображения с десятью метками, такими как «самолет», «автомобиль» и т. д., размером 32×32 ;
- CIFAR-100 — аналогичен CIFAR-10, но уже со 100 метками, такими как «водные млекопитающие», «цветы» и т. д.;
- **семантическая сегментация.** Входные данные — это изображение, а выходные — не только метки предметов, найденных на изображении, но и их расположение на нем. Обычно нейронная сеть выводит местоположение в виде ограничительного прямоугольника на изображении.

В разделе «Обучение нейронной сети для распознавания изображений» мы построили небольшую полносвязную нейронную сеть, которая довольно хорошо классифицировала набор данных MNIST. Однако для более сложных изображений, таких как картинки и лица, она не подойдет, потому что при превращении изображения в длинный вектор теряется много информации. Для подобных сложных изображений нужны другие архитектуры, и здесь на помощь приходят сверточные нейронные сети (CNN).

Для получения подробной информации об этих нейронных сетях ознакомьтесь с ресурсами, приведенными в приложении В, а здесь дадим приблизительное описание того, как они работают. Представьте, что есть большое изображение, которое мы хотим обработать. Мы берем окно меньшего размера, скажем, 5×5 или 7×7 пикселей, и проводим его по большому изображению. Каждый раз, проходя по нему, применяем формулу, называемую *сверткой*. Заканчиваем мы немного меньшим отфильтрованным изображением, которое в некотором роде суммирует предыдущее, — сверточным слоем. Сверточная нейронная сеть состоит из нескольких сверточных слоев, за которыми следуют несколько полносвязных слоев.

Когда дело касается сложных изображений, мы обычно не занимаемся обучением нейронной сети с нуля. Полезный метод, называемый *обучением с переносом опыта*, состоит в том, чтобы начать с предварительно обученной сети и использовать свои данные для настройки некоторых ее параметров (обычно последнего слоя). Этот метод, как правило, не только хорошо работает, но и не требует больших вычислительных затрат. Такие сети, как InceptionV3, ImageNet, ResNet и VGG, были обучены компаниями и исследовательскими группами, обладающими значительными вычислительными мощностями, поэтому настоятельно рекомендуется их использовать.

Как нейронные сети разговаривают. Рекуррентные нейронные сети, управляемые рекуррентные блоки и длинная цепь элементов краткосрочной памяти

Одно из самых интересных применений нейронных сетей — то, что мы можем заставить их разговаривать с нами или понимать сказанное. Процесс включает в себя прослушивание того, что мы говорим, или чтение того, что пишем, анализ и способность реагировать или действовать. Способность компьютеров

понимать и обрабатывать язык называется *обработкой естественного языка*. Нейронные сети добились больших успехов на этом поприще. Пример анализа настроений, приведенный в начале этой главы, — это часть обработки естественного языка, поскольку включает в себя понимание предложений и определение их позитивной или негативной окраски. Существует и еще множество передовых областей, таких как:

- **машинный перевод** — перевод предложений с одних языков на другие;
- **распознавание речи** — декодирование человеческого голоса и превращение его в текст;
- **обобщение текста** — превращение больших текстов в несколько абзацев;
- **чат-боты** — системы, способные разговаривать с людьми и отвечать на вопросы. Они еще далеки от совершенства, но полезные чат-боты уже работают в определенных областях, таких как поддержка клиентов.

Наиболее полезными архитектурами, которые хорошо справляются с обработкой текстов, являются *рекуррентные нейронные сети* и некоторые более продвинутые их версии, называемые *длинной цепью элементов краткосрочной памяти* и *управляемыми рекуррентными блоками* (GRU). Чтобы получить представление о том, что это такое, представьте себе нейронную сеть, в которой выходные данные заводятся обратно в нее как часть входных данных. То есть нейронные сети обладают памятью, и при правильном обучении она способна помочь им разобраться в теме текста.

Как нейронные сети рисуют картины. Генеративные состязательные сети

Одно из самых интересных применений нейронных сетей — это генерация. До сих пор нейронные сети (и большинство других моделей машинного обучения, описанных в книге) хорошо работали в прогностическом машинном обучении, то есть были способны отвечать на вопросы: «Сколько это стоит?» или «Это кошка или собака?» Однако последние годы ознаменовались множеством достижений в увлекательной области, называемой *генеративным машинным обучением*. Это область машинного обучения, которая учит компьютер создавать что-то, а не просто отвечать на вопросы. Такие действия, как написание картины, рассказа или сочинение песни, представляют собой гораздо более высокий уровень понимания мира.

Без сомнения, одним из наиболее важных достижений за последние несколько лет стало развитие *генеративных состязательных сетей* (GAN). Они показывают потрясающие результаты, когда дело доходит до генерации изображений. GAN состоят из двух конкурирующих сетей — генератора и дискриминатора. Генератор пытается формировать изображения, выглядящие реально, а дискриминатор — отличить реальные изображения от поддельных. В процессе обучения мы передаем в дискриминатор изображения реальные

и созданные генератором. Будучи примененным к набору данных, состоящему из человеческих лиц, такой процесс приводит к созданию генератора, который способен творить изображения очень реалистично выглядящих лиц. На самом деле они кажутся настолько реальными, что людям часто бывает трудно отличить их от настоящих. Испытайте себя в противоборстве с GAN — www.whichfaceisreal.com.

Резюме

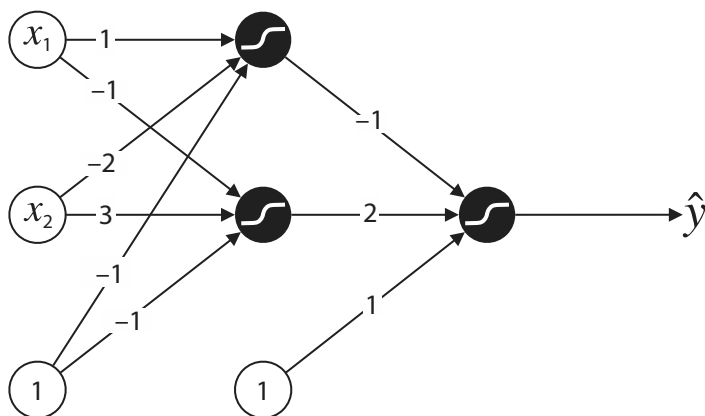
- Нейронные сети — это эффективная модель, используемая для классификации и регрессии. Сеть состоит из набора перцептронов, организованных в слои, где выходные данные одного слоя служат входными данными для следующего. Их сложность позволяет добиваться больших успехов в областях, которые трудны для других моделей машинного обучения.
- Нейронные сети применяются во многих передовых областях, включая распознавание изображений и обработку текста.
- Основным строительным блоком нейронной сети служит перцептрон. Он получает несколько значений в качестве входных данных и выдает одно значение, вычисляемое умножением входных данных на веса, добавлением смещения и применением функции активации.
- Популярные функции активации: сигмоидная, гиперболический тангенс, softmax и блок линейной ректификации. Они используются между слоями в нейронной сети для нарушения линейности, чтобы помочь строить более сложные границы.
- Сигмоидная функция — это простая функция, которая обращает любое действительное число в 0 или 1. Гиперболический тангенс работает аналогично, за исключением того, что на выходе мы получаем интервал между -1 и 1 . Их задача заключается в сведении входных данных к небольшому интервалу, чтобы ответы можно было интерпретировать как категорию. В основном они используются для конечного (выходного) слоя в нейронной сети. Из-за плоскостности их производных они могут порождать проблемы с исчезающими градиентами.
- Функция ReLU — это функция, которая обращает отрицательные числа в 0, а неотрицательные числа оставляет неизменными. Она заметно уменьшает проблему исчезающего градиента, поэтому используется при обучении нейронных сетей чаще, чем сигмоидная функция или гиперболический тангенс.
- Нейронные сети имеют очень сложную структуру, что затрудняет их обучение. Эффективный процесс, который мы применяем для обучения, называется обратным распространением. Оно состоит во взятии производной функции потерь и поиске всех частных производных по всем весам модели. Затем мы применяем эти производные для итеративного обновления весов модели, улучшая ее производительность.

- Нейронные сети склонны к переобучению и возникновению других проблем, таких как исчезающие градиенты. Для их уменьшения или устранения мы можем использовать такие методы, как регуляризация и прореживание.
- В нашем распоряжении имеется несколько полезных библиотек для обучения нейронных сетей, таких как Keras, TensorFlow и PyTorch. Эти пакеты очень упрощают обучение нейронных сетей, потому что нам требуется только определить архитектуру модели и функции ошибок, а они берут на себя обучение. Кроме того, в них добавлено множество встроенных передовых оптимизаторов, которыми можно воспользоваться.

Упражнения

Упражнение 10.1

На рисунке показана нейронная сеть, в которой все активации являются сигмоидными функциями.



Каким будет прогноз этой нейронной сети для входных данных (1, 1)?

Упражнение 10.2

Как мы узнали из упражнения 5.3, невозможно построить персептрон, имитирующий элемент XOR. Другими словами, невозможно сопоставить следующий набор данных с персептроном и получить 100%-ную достоверность.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Это связано с тем, что набор данных не является линейно разделяемым. Используя нейронную сеть глубиной 2, постройте персептрон, который имитирует элемент XOR, показанный ранее. Для получения дискретных выходных данных в качестве функций активации примените ступенчатую функцию вместо сигмоидной.

ПОДСКАЗКА Это трудно сделать с помощью метода обучения, вместо этого попробуйте внимательно следить за весами. Попробуйте сами (или найдите способ в Интернете) создать элемент XOR с помощью элементов AND, OR и NOT и используйте результаты упражнения 5.3 для облегчения задачи.

Упражнение 10.3

В конце раздела «Графическое представление нейронных сетей» мы увидели, что нейронная сеть, изображенная на рис. 10.13, с функцией активации не соответствует набору данных, приведенному в табл. 10.1, поскольку точка (1, 1) неправильно классифицирована.

- А. Убедитесь, что это так и есть.
- Б. Измените веса таким образом, чтобы нейронная сеть правильно классифицировала каждую точку.



В этой главе

- ✓ Что такое машина опорных векторов.
 - ✓ Какой из линейных классификаторов для набора данных имеет наилучшую границу.
 - ✓ Использование метода ядра для построения нелинейных классификаторов.
 - ✓ Кодирование машин с опорными векторами и метода ядра в Scikit-Learn.
-



Для разделения куриных наборов данных
эксперты рекомендуют метод ядра

В этой главе мы обсудим сильную классификационную модель — *машину опорных векторов* (SVM). Она похожа на персептрон в том смысле, что разделяет набор данных на два класса с помощью линейной границы. Однако SVM стремится найти линейную границу, расположенную как можно дальше от точек, из которых состоит набор данных. Мы также рассмотрим метод ядра, который задействуется в сочетании с SVM и помогает классифицировать наборы данных с использованием высоконелинейных границ.

Из главы 5 мы узнали о линейных классификаторах, или персептронах. Для двумерных данных они определяются прямой, которая разделяет набор данных, состоящий из точек с двумя метками. Однако мы отмечали, что разделять набор данных могут множество различных прямых, и это порождает вопрос: как узнать, какая из них наилучшая? На рис. 11.1 мы видим три различных линейных классификатора, которые разделяют набор данных. Какой из них предпочтительнее?

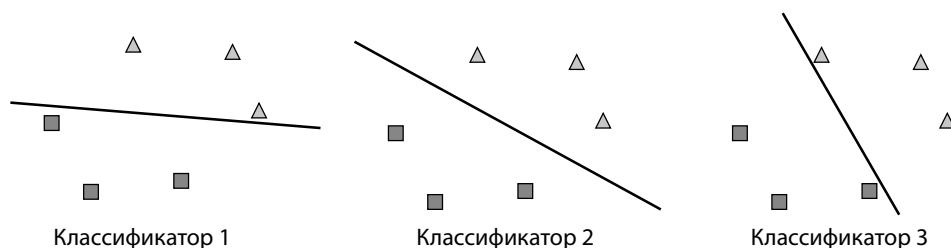


Рис. 11.1. Три классификатора, которые правильно классифицируют набор данных

Если ваш выбор — это классификатор 2, то я с вами соглашусь. Все три прямые хорошо разделяют набор данных, но вторая расположена более удачно. Первая и третья прямые находятся очень близко к некоторым точкам, в то время как вторая далека от всех точек. Если немного пошевелить эти три прямые, первая и третья могут пройти через некоторые точки, неправильно их классифицируя, а вторая все равно классифицирует их все правильно. Таким образом, классификатор 2 выглядит более надежным, чем классификаторы 1 и 3.

Вот тут-то и вступают в игру машины опорных векторов. Классификатор SVM использует две параллельные прямые вместо одной. Цель SVM двоякая — она пытается правильно классифицировать данные, а также как можно сильнее отдалить прямые друг от друга. На рис. 11.2 мы видим две параллельные прямые для трех классификаторов и — для сравнения — прямую, расположенную посередине между ними. Две внешние (пунктирные) прямые в классификаторе 2 отстоят дальше всего друг от друга, что и делает его лучшим.

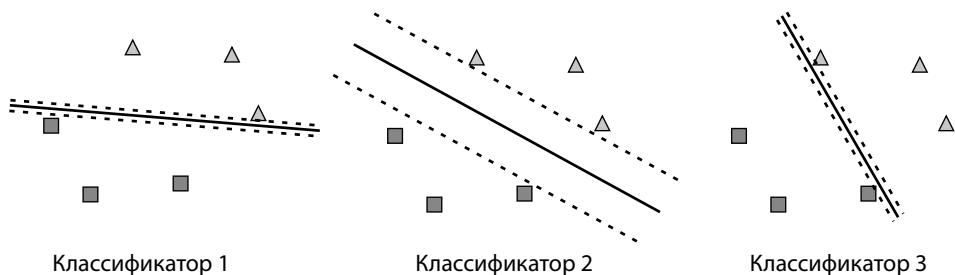


Рис. 11.2. Классификатор отображается в виде двух параллельных прямых, расположенных как можно дальше друг от друга

Мы можем визуализировать SVM как проходящую между точек прямую, которая старается оставаться как можно дальше от них. Мы также можем представить ее как две внешние параллельные прямые, пытающиеся держаться как можно дальше друг от друга. В этой главе в различных ситуациях будем использовать обе визуализации, потому что каждая из них полезна в своем случае.

Как же построить такой классификатор? Так же, как и раньше, но с немного иной функцией ошибок и немного иным итерационным шагом.

ОБРАТИТЕ ВНИМАНИЕ В этой главе все классификаторы дискретные, то есть их выходные данные равны 0 или 1. Иногда они описываются прогнозом $\hat{y} = \text{step}(f(x))$, а в других случаях — граничным уравнением $f(x) = 0$, то есть графиком функции, которая пытается разделить точки данных на два класса. Например, персептрон, который делает прогноз $\hat{y} = \text{step}(3x_1 + 4x_2 - 1)$, иногда описывается только линейным уравнением $3x_1 + 4x_2 - 1 = 0$. Для некоторых классификаторов в этой главе (особенно в разделе «Обучение SVM с нелинейными границами. Метод ядра») граничное уравнение не обязательно будет линейной функцией.

Здесь мы рассматриваем эту теорию в основном на одномерных и двумерных наборах данных (точки на прямой или плоскости соответственно). Однако машины опорных векторов одинаково хорошо работают с наборами данных и более высоких измерений. Линейные границы в одном измерении представляют собой точки, а в двух измерениях — прямые. Аналогично линейными границами в трех измерениях служат плоскости, а в более высоких измерениях — гиперплоскости, имеющие на одно измерение меньше, чем пространство,

в котором расположены точки. В каждом из этих случаев мы попытаемся найти границу, которая находится дальше всего от точек. На рис. 11.3 можно увидеть примеры границ для одного, двух и трех измерений. В одном измерении граница образована двумя точками, в двух измерениях — двумя прямыми, а в трех измерениях — двумя плоскостями. Во всех случаях мы стараемся как можно сильнее разделить эти пары. Средняя граница (точка, прямая или плоскость) изображена для наглядности.

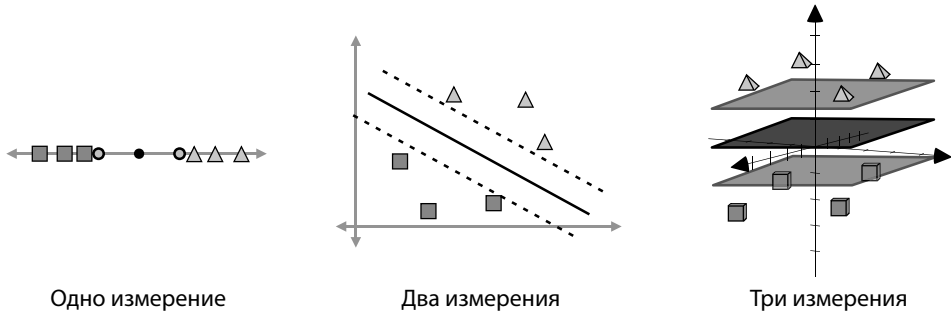


Рис. 11.3. Линейные границы для наборов данных в одном, двух и трех измерениях

Весь код для этой главы находится в репозитории GitHub: https://github.com/luisguiserrano/manning/tree/master/Chapter_11_Support_Vector_Machines.

Использование новой функции ошибок для создания более совершенных классификаторов

Как обычно бывает в моделях машинного обучения, SVM определяется с помощью функции ошибок. В этом разделе мы увидим особенную функцию ошибок для SVM, которая пытается максимизировать одновременно две вещи: классификацию точек и расстояние между прямыми.

Чтобы обучить SVM, необходимо построить функцию ошибок для классификатора, состоящего из двух прямых, расположенных как можно дальше друг от друга. Думая о построении функции ошибок, всегда стоит задавать вопрос: «Чего мы хотим достичь с помощью модели?» Далее приведены два результата, к которым мы можем прийти.

- Каждая из двух прямых должна классифицировать точки как можно лучше.
- Две прямые должны пролегать как можно дальше друг от друга.

Функция ошибки должна наказывать любую модель, которая не достигает этих целей. Поскольку нам нужны два результата, функция ошибок SVM должна быть суммой двух функций ошибок: первая наказывает неправильно

классифицированные точки, а вторая — прямые, находящиеся слишком близко друг к другу. Таким образом, функция ошибки может выглядеть так:

ошибка = ошибка классификации + ошибка расстояния.

В следующих двух разделах мы отдельно рассмотрим каждое из этих двух слагаемых.

Функция ошибки классификации. Попытка правильно классифицировать точки

В этом разделе мы изучаем функцию ошибки классификации. Это та часть функции ошибки, которая подталкивает классификатор к правильной классификации точек. Если коротко, то она вычисляется следующим образом. Поскольку классификатор состоит из двух прямых, мы рассматриваем их как два отдельных дискретных персептрона (см. главу 5). Затем вычисляем общую ошибку классификатора как сумму двух ошибок персептрона (раздел «Как сравнить классификаторы? Функция ошибки» главы 5). Давайте взглянем на пример.

SVM использует две параллельные прямые, и, к счастью, у них схожие уравнения: они имеют одинаковые веса, но разное смещение. Таким образом, в SVM мы берем центральную прямую в качестве системы отсчета $L: w_1x_1 + w_2x_2 + b = 0$ и строим две другие прямые, одну над ней и одну под ней:

- $L_+ - w_1x_1 + w_2x_2 + b = 1$;
- $L_- - w_1x_1 + w_2x_2 + b = -1$.

В качестве примера на рис. 11.4 показаны три параллельные прямые, L , L_+ и L_- , со следующими уравнениями:

- $L - 2x_1 + 3x_2 - 6 = 0$;
- $L_+ - 2x_1 + 3x_2 - 6 = 1$;
- $L_- - 2x_1 + 3x_2 - 6 = -1$.

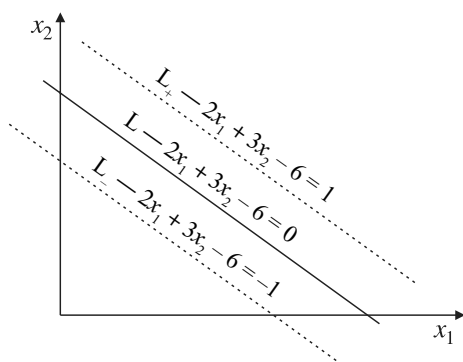


Рис. 11.4. Основная прямая L — посередине. Строим две параллельные равноудаленные прямые L_+ и L_- , слегка изменив уравнение L

Наш классификатор теперь состоит из прямых L_+ и L_- . Мы можем думать о L_+ и L_- как о двух независимых классификаторах персептронов, каждый из которых преследует одну и ту же цель — правильно классифицировать точки. Каждый классификатор поставляется со своей функцией ошибок персептрона, поэтому функция классификации определяется как сумма этих двух функций ошибок, что показано на рис. 11.5. Обратите внимание на то, что ошибка не равна длине перпендикулярного отрезка к границе, а пропорциональна ей.

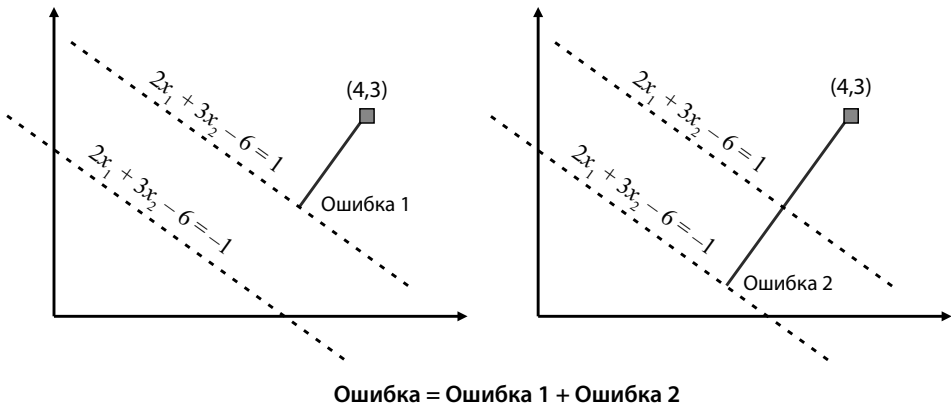


Рис. 11.5. Классификатор состоит из двух прямых, ошибка неправильно классифицированной точки измеряется по отношению к обеим прямым

В SVM обе прямые должны хорошо классифицировать точки. Следовательно, точка, находящаяся между двумя прямыми, всегда неправильно классифицируется одной из них, поэтому SVM не считает ее правильно классифицированной.

Из раздела «Как сравнить классификаторы? Функция ошибки» главы 5 мы знаем, что функция ошибки для дискретного персептрона с прогнозом $\hat{y} = \text{step}(w_1x_1 + w_2x_2 + b)$ в точке (p, q) задается следующим образом:

- 0, если точка правильно классифицирована;
- $|w_1x_1 + w_2x_2 + b|$, если неправильно.

В качестве примера рассмотрим точку $(4, 3)$ с меткой 0. Эта точка неправильно классифицируется обоими персептронами на рис. 11.5. Обратите внимание на то, что два персептрона дают следующие прогнозы:

- $L_+ - \hat{y} = \text{step}(2x_1 + 3x_2 - 7)$;
- $L_- - \hat{y} = \text{step}(2x_1 + 3x_2 - 5)$.

Следовательно, ее ошибка классификации по отношению к этому SVM составляет:

$$|2 \times 4 + 3 \times 3 - 7| + |2 \times 4 + 3 \times 3 - 5| = 10 + 12 = 22.$$

Функция ошибки расстояния. Попытка разместить две прямые как можно дальше друг от друга

Теперь, когда мы получили функцию ошибок, измеряющую ошибки классификации, требуется создать функцию, которая оценивает расстояние между двумя прямыми и подает сигнал тревоги, если оно мало. В этом разделе мы обсуждаем удивительно простую функцию ошибок, которая дает большое значение, если две прямые расположены близко, и маленькое, когда они далеко друг от друга.

Эта функция ошибки называется *функцией ошибки расстояния*, и мы уже встречали ее ранее — это слагаемое регуляризации, изученное в разделе «Изменение функции ошибки для решения проблемы...» главы 4. В частности, если прямые имеют уравнения $w_1x_1 + w_2x_2 + b = 1$ и $w_1x_1 + w_2x_2 + b = -1$, тогда функция ошибки равна $w_1^2 + w_2^2$. Почему так? Мы воспользуемся следующим фактом: перпендикулярное расстояние между двумя прямыми составляет точно

$$\frac{2}{\sqrt{w_1^2 + w_2^2}},$$

как показано на рис. 11.6. Если вы хотите разобраться в тонкостях расчета этого расстояния, пожалуйста, ознакомьтесь с упражнением 11.1 в конце главы.

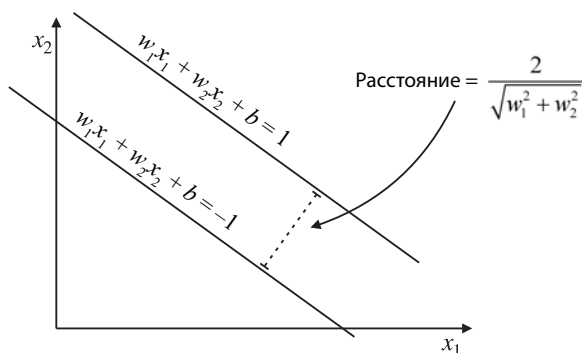


Рис. 11.6. Расстояние между двумя параллельными прямыми может быть рассчитано на основе уравнений этих прямых

Зная это, обратите внимание на следующее:

- когда $w_1^2 + w_2^2$ велико, $\frac{2}{\sqrt{w_1^2 + w_2^2}}$ маленькое;
- когда $w_1^2 + w_2^2$ мало, $\frac{2}{\sqrt{w_1^2 + w_2^2}}$ большое.

Поскольку мы хотим, чтобы прямые оказались как можно дальше друг от друга, то сумма $w_1^2 + w_2^2$ послужит хорошей функцией ошибки, поскольку дает большие значения для плохих классификаторов (тех, где прямые близки друг к другу) и малые — для хороших (тех, где линии удалены друг от друга).

На рис. 11.7 мы видим два примера классификаторов SVM. Их уравнения:

- SVM 1:
 - $L_+ - 3x_1 + 4x_2 + 5 = 1;$
 - $L_- - 3x_1 + 4x_2 - 5 = -1;$
- SVM 2:
 - $L_+ - 30x_1 + 40x_2 + 50 = 1;$
 - $L_- - 30x_1 + 40x_2 - 50 = -1.$

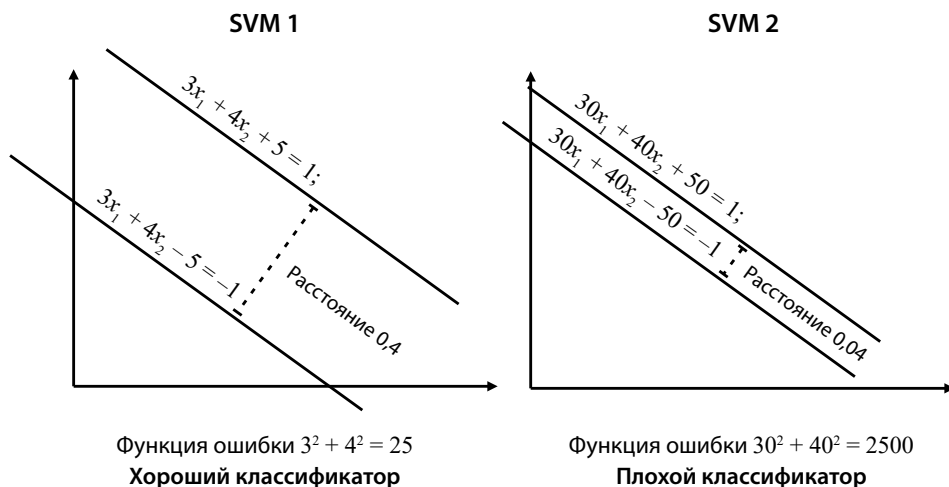


Рис. 11.7. SVM: *слева* — прямые находятся на расстоянии 0,4 друг от друга с ошибкой 25; *справа* — прямые находятся на расстоянии 0,04 друг от друга с ошибкой 2500

Их функции ошибки расстояния показаны далее:

- SVM 1 — функция ошибки расстояния $3^2 + 4^2 = 25;$
- SVM 2 — функция ошибки расстояния $30^2 + 40^2 = 2500.$

Обратите также внимание на то, что здесь прямые в SVM 2 расположены намного ближе, чем в SVM 1, что делает SVM 1 гораздо лучшим классификатором с точки зрения расстояния. Расстояние между прямыми в SVM 1:

$$\frac{2}{\sqrt{3^2 + 4^2}} = 0,4, \text{ тогда как в SVM 2: } \frac{2}{\sqrt{30^2 + 40^2}} = 0,04.$$

Сложение двух функций ошибок для получения функции ошибок

Теперь, когда мы построили функцию ошибки классификации и функцию ошибки расстояния, давайте посмотрим, как объединить их, чтобы создать функцию ошибки, которая поможет убедиться, что мы достигли обеих целей:

хорошей классификации точек и получения двух прямых, расположенных далеко друг от друга.

Чтобы получить ее, складываем функцию ошибки классификации с функцией ошибки расстояния:

$$\text{ошибка} = \text{ошибка классификации} + \text{ошибка расстояния}.$$

Хорошая SVM, которая минимизирует эту функцию ошибок, должна стремиться сделать как можно меньше ошибок классификации, одновременно пытаясь удерживать прямые как можно дальше друг от друга.

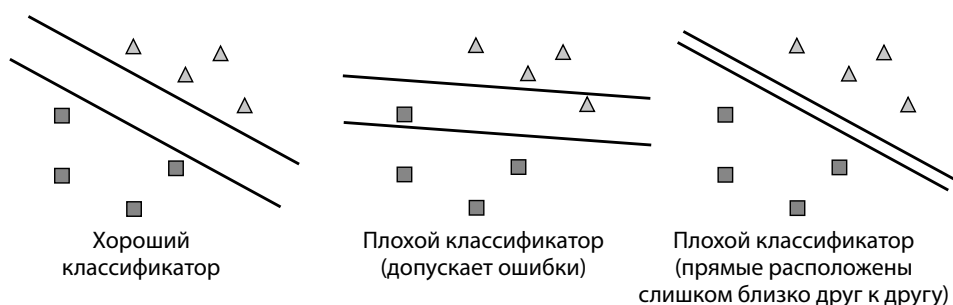


Рис. 11.8. SVM: *слева* — хорошая, состоящая из двух удачно расположенных прямых и правильно классифицирующая все точки; *в середине* — плохая, неправильно классифицирует две точки; *справа* — плохая, состоит из двух прямых, расположенных слишком близко друг к другу

На рис. 11.8 мы видим три SVM-классификатора для одного и того же набора данных. Тот, что слева, хорошо классифицирует данные, а прямые расположены далеко друг от друга, что снижает вероятность ошибок. Расположенный в середине допускает несколько ошибок (под верхней прямой есть треугольник, а над нижней — квадрат), поэтому он не является хорошим классификатором. Тот, что справа, верно классифицирует точки, но прямые расположены слишком близко друг к другу, поэтому его также нельзя считать хорошим.

На чем SVM должна сосредоточиться больше — на классификации или на расстоянии? Параметр C в этом поможет

В этом разделе изучим полезную технику настройки и улучшения модели, которая включает в себя введение параметра C. Он используется в тех случаях, когда необходимо обучить SVM, которая уделяет больше внимания классификации, чем расстоянию, или наоборот.

До этого момента казалось, что для создания хорошего классификатора SVM нужно всего лишь отслеживать две вещи. Необходимо, что классификатор допускал как можно меньше ошибок, при этом сохраняя прямые как можно дальше друг от друга. Но что, если потребуется пожертвовать одним ради другого?

На рис. 11.9 изображены два классификатора для одного и того же набора данных. Тот, что слева, допускает несколько ошибок, но прямые далеко друг от друга. Расположенный справа не делает ошибок, но прямые слишком близко друг к другу. Какой из них предпочесть?

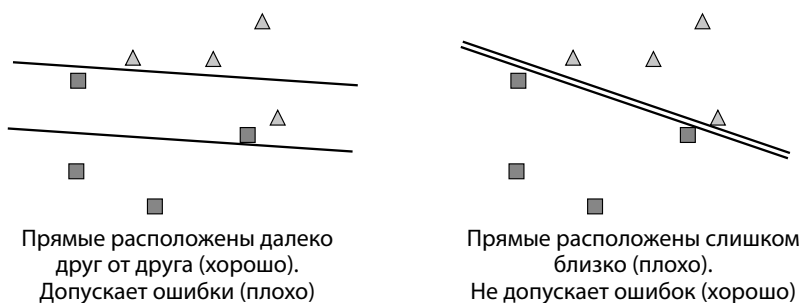


Рис. 11.9. У обоих классификаторов есть одно «за» и одно «против»

Оказывается, ответ на этот вопрос зависит от решаемой задачи. Иногда нужен классификатор, который допускает как можно меньше ошибок, даже если прямые расположены слишком близко, а иногда — классификатор, который хорошо разделяет прямые, пусть даже ценой нескольких ошибок. Как можно этим управлять? Используем параметр, который назовем параметром C . Мы немного модифицируем формулу ошибки, умножив ошибку классификации на C :

формула ошибки = $C \times (\text{ошибка классификации}) + (\text{ошибка расстояния})$.

Если параметр C велик, то в формуле ошибки преобладает ошибка классификации, поэтому классификатор фокусируется на правильной классификации точек. Если C мал, то в формуле преобладает ошибка расстояния, поэтому классификатор фокусируется на том, чтобы прямые находились далеко друг от друга.

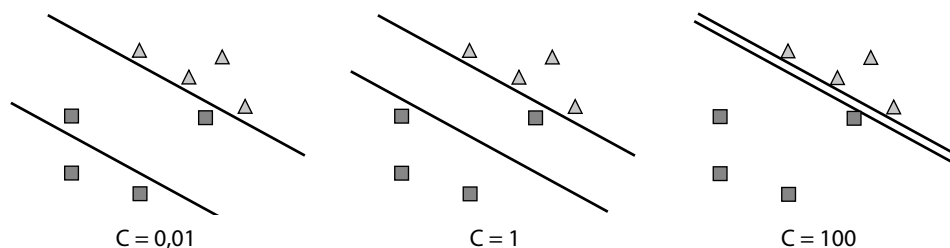


Рис. 11.10. Разные значения C переключают классификатор между режимом работы с удачно расположенными прямыми и режимом, который хорошо классифицирует точки

На рис. 11.10 мы видим три классификатора: один с большим значением C , который правильно классифицирует все точки, один с небольшим значением C ,

который удерживает прямые далеко друг от друга, и один с $C = 1$, который пытается сделать и то и другое. В реальной жизни C — это гиперпараметр, который можно настроить с помощью таких методов, как график сложности модели (см. раздел «Численный способ определить нужную сложность модели...» главы 4), или наших собственных знаний о решаемой задаче, данных и модели.

Кодирование машин с опорными векторами в Scikit-Learn

Теперь, узнав, что такое SVM, мы готовы закодировать ее и использовать для моделирования каких-нибудь данных. В Scikit-Learn кодировать SVM очень просто, и мы изучим это в данном разделе. А также узнаем, как задействовать параметр C в коде.

Кодирование простой SVM

Начнем с кодирования простой SVM на примере набора данных, а затем добавим дополнительные параметры. Набор данных называется `linear.csv`, а его график показан на рис. 11.11. Код для этого раздела приведен далее:

- **Блокнот** `SVM_graphical_example.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_11_Support_Vector_Machines/SVM_graphical_example.ipynb.
- **Набор данных** `linear.csv`.

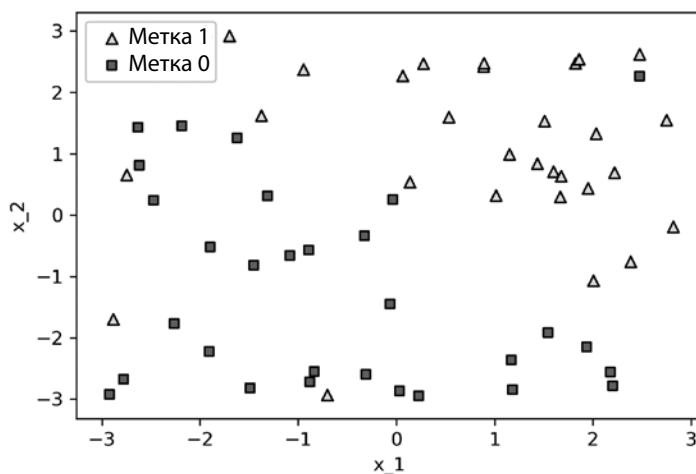


Рис. 11.11. Почти линейно разделяемый набор данных с некоторыми выбросами

Сначала мы импортируем данные из пакета SVM в Scikit-Learn и загрузим свои данные:

```
from sklearn.svm import SVC
```

Затем, как показано в следующем фрагменте кода, загружаем данные в два фрейма данных Pandas, `features` и `labels`:

```
svm_linear = SVC(kernel='linear')
svm_linear.fit(features, labels)
```

после чего определяем модель `svm_linear` и обучаем ее. Получаемая достоверность составляет 0,933, а график показан на рис. 11.12.

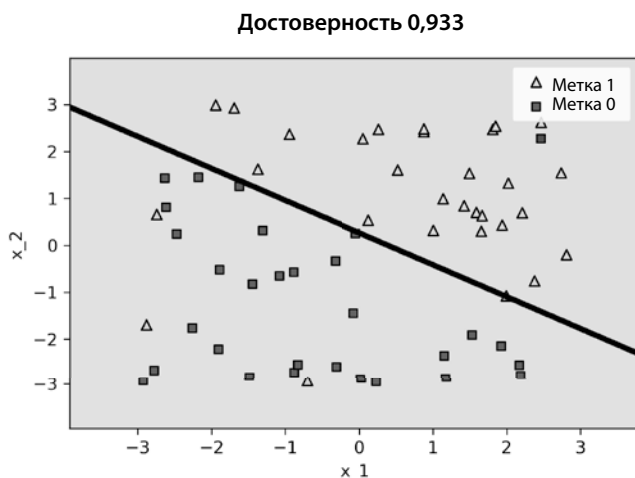


Рис. 11.12. График классификатора SVM, который мы построили в Scikit-Learn, состоит из прямой

Параметр C

В Scikit-Learn можно легко ввести в модель параметр `C`. Здесь мы обучаем и строим две модели: одну с очень маленьким значением 0,01, а другую с большим значением 100, как показано в следующем коде и на рис. 11.13:

```
svm_c_001 = SVC(kernel='linear', C=0.01)
svm_c_001.fit(features, labels)
```

```
svm_c_100 = SVC(kernel='linear', C=100)
svm_c_100.fit(features, labels)
```

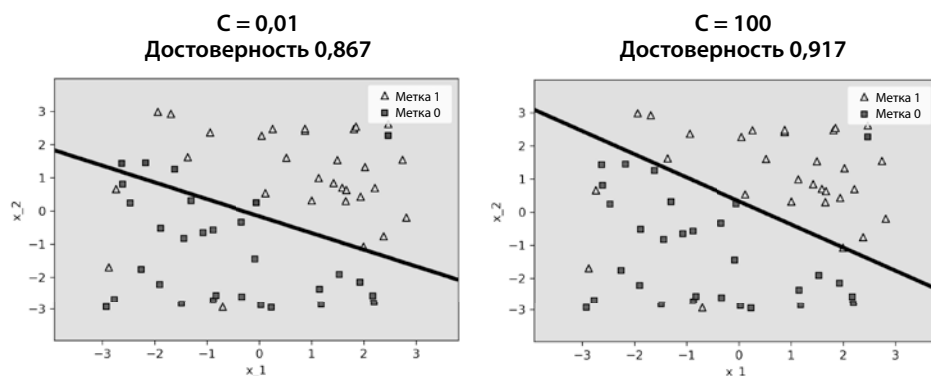


Рис. 11.13. Классификаторы: *слева* — имеет небольшое значение C и хорошо отделяет прямую от точек, допуская несколько ошибок; *справа* — имеет большое значение C и не допускает ошибок, хотя прямая проходит слишком близко к некоторым точкам

Мы видим, что модель с небольшим значением C не уделяет особого внимания правильной классификации точек и допускает ошибки, о чем свидетельствует ее низкая достоверность (0,867). В данном примере это не так просто увидеть, но классификатор уделяет большое внимание тому, чтобы прямая находилась как можно дальше от точек. Напротив, классификатор с большим значением C пытается правильно классифицировать все точки, что отражается на его более высокой достоверности.

Обучение SVM с нелинейными границами. Метод ядра

Как мы видели в предыдущих главах, не каждый набор данных является линейно разделяемым и часто приходится создавать нелинейные классификаторы, чтобы учесть сложность данных. В этом разделе мы изучим мощный метод, связанный с SVM, который называется *методом ядра*. Он призван помочь в создании сильно нелинейных классификаторов.

Если у нас имеется набор данных и мы понимаем, что не в силах разделить его с помощью линейного классификатора, то что мы можем сделать? Одна из идей состоит в том, чтобы добавить больше столбцов в набор данных и надеяться на то, что теперь он окажется линейно разделяемым. Метод ядра состоит в том, чтобы разумно добавить столбцы, построить линейный классификатор на новом наборе данных, а затем удалить добавленные столбцы, сохранив при этом теперь уже нелинейный классификатор.

Определение весьма многословно, но есть хороший геометрический способ увидеть метод в действии. Представьте себе двумерный набор данных, что означает: входные данные состоят из двух столбцов. Если добавить третий столбец, то набор данных станет трехмерным, как если бы точки, поставленные на бумаге, внезапно начали взлетать в небо на разную высоту. Может оказаться, что если мы каким-то разумным способом поднимем точки на разную высоту, то сможем разделить их плоскостью. Это и есть метод ядра, он показан на рис. 11.14.

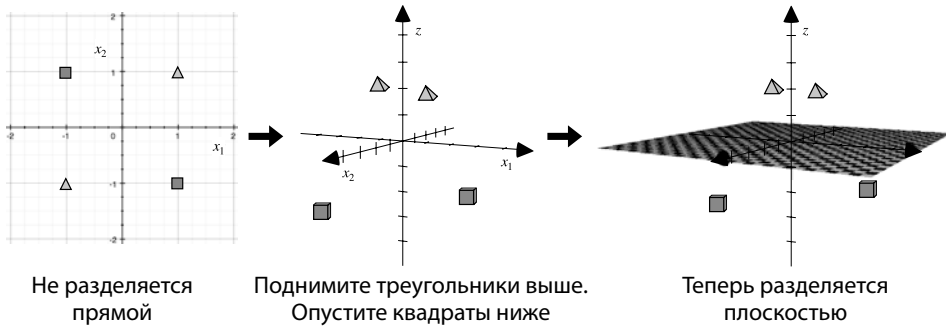


Рис. 11.14. Слева — набор не разделяется прямой; в середине — мы смотрим на картину в трех измерениях и продолжаем поднимать два треугольника и опускать два квадрата; справа — новый набор данных теперь разделяется плоскостью (источник — изображение, созданное с помощью Grapher™ от Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>)

Ядра, карты признаков и теория операторов

Теория, лежащая в основе метода ядра, берет начало в области математики, называемой *теорией операторов*. Ядро — это функция подобия, которая, если не углубляться, представляет собой функцию, сообщающую, являются ли две точки одинаковыми или разными (например, близкими или далекими). Ядро может привести к созданию *карты признаков*, которая представляет собой карту пространства между областью, в которой находится наш набор данных, и областью более высокой размерности.

Полная теория ядер и карт признаков не требуется для понимания классификаторов. Если вы хотите углубиться в данную тему, ознакомьтесь с ресурсами, приведенными в приложении В. Для целей этой главы мы рассмотрим метод ядра как способ добавления столбцов в набор данных, чтобы точки были разделяемыми. Например, набор данных на рис. 11.14 содержит два столбца, x_1 и x_2 , и мы добавили третий столбец со значением $x_1 x_2$. Его можно рассматривать также как функцию, которая отправляет точку (x_1, x_2) на плоскости в точку $(x_1, x_2, x_1 x_2)$ в пространстве. Как только точки окажутся

в трехмерном пространстве, мы сможем разделить их, используя плоскость, показанную справа на рис. 11.14. Чтобы изучить этот пример более подробно, см. упражнение 11.2.

Два ядра и соответствующие им карты признаков, которые мы видим в этой главе, — это *полиномиальное ядро* и *ядро радиальной базисной функции* (RBF). Оба они состоят в добавлении столбцов в набор данных различными, но очень эффективными способами.

Использование полиномиальных уравнений в своих интересах. Полиномиальное ядро

В этом разделе обсуждаем полиномиальное ядро, помогающее моделировать нелинейные наборы данных. Если говорить более конкретно, то метод ядра помогает моделировать данные с помощью полиномиальных уравнений, таких как окружности, параболы и гиперболы. Мы проиллюстрируем полиномиальное ядро двумя примерами.

Пример 1. Круговой набор данных

В качестве первого примера попробуем классифицировать набор данных, приведенный в табл. 11.1.

Таблица 11.1. Небольшой набор данных, изображенный на рис. 11.15

x_1	x_2	y
0,3	0,3	0
0,2	0,8	0
-0,6	0,4	0
0,6	-0,4	0
-0,4	-0,3	0
0,0	-0,8	0
-0,4	1,2	1
0,9	-0,7	1
-1,1	-0,8	1
0,7	0,9	1
-0,9	0,8	1
0,6	-1,0	1

График показан на рис. 11.15, где точки с меткой 0 изображены в виде квадратов, а точки с меткой 1 — треугольников.

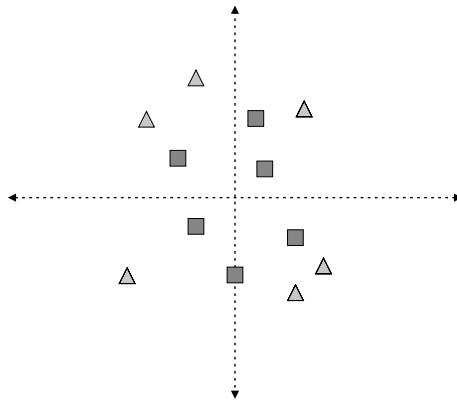


Рис. 11.15. График набора данных из табл. 11.1

Когда мы смотрим на график на рис. 11.15, становится ясно, что такой набор данных — хороший кандидат на применение метода ядра, так как прямая не сможет отделить квадраты от треугольников. Однако с помощью окружности это сделать вполне возможно (рис. 11.16). Теперь вопрос: если машина опорных векторов умеет рисовать только линейные границы, как нарисовать окружность?

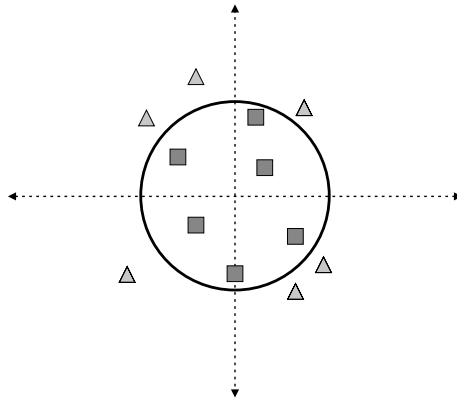


Рис. 11.16. Метод ядра дает классификатор с круговой границей, которая хорошо разделяет точки

Давайте поразмышляем над тем, как провести такую границу. Какая характеристика отделяет квадраты от треугольников? При взгляде на график кажется, что треугольники отстоят дальше от начала координат, чем квадраты. Формула, которая задает расстояние до начала координат, представляет собой квадратный корень из суммы квадратов двух координат. Если эти координаты равны x_1 и x_2 , то расстояние равно $\sqrt{x_1^2 + x_2^2}$. Давайте забудем про квадратный корень

и сосредоточимся на $x_1^2 + x_2^2$. Теперь добавим столбец с этим значением в табл. 11.1 и посмотрим, что произойдет. Результирующий набор данных показан в табл. 11.2.

Таблица 11.2. В табл. 11.1 добавлен столбец. Он состоит из суммы квадратов значений первых двух столбцов

x_1	x_2	$x_1^2 + x_2^2$	y
0,3	0,3	0,18	0
0,2	0,8	0,68	0
-0,6	0,4	0,52	0
0,6	-0,4	0,52	0
-0,4	-0,3	0,25	0
0,0	-0,8	0,64	0
-0,4	1,2	1,60	1
0,9	-0,7	1,30	1
-1,1	-0,8	1,85	1
0,7	0,9	1,30	1
-0,9	0,8	1,45	1
0,6	-1,0	1,36	1

Взглянув на табл. 11.2, можно увидеть тенденцию. Все точки, помеченные как 0, удовлетворяют условию, согласно которому сумма квадратов координат меньше 1, а точки, помеченные как 1, — условию, что эта сумма больше 1. Следовательно, уравнение по координатам, разделяющее точки, это $x_1^2 + x_2^2 = 1$. Обратите внимание на то, что это не линейное уравнение, поскольку переменные возведены в степень большую, чем единица. По сути, это уравнение окружности.

Геометрически это представлено на рис. 11.17. Шаг 1 — исходный набор лежит на плоскости, и его невозможно разделить на два класса прямой. Но если мы поднимем каждую точку (x_1, x_2) на высоту $x_1^2 + x_2^2$ (шаг 2), то тем самым поместим их в параболоид $z = x_1^2 + x_2^2$ (показано на рисунке). Расстояние, на которое мы подняли каждую точку, точно равно квадрату расстояния от этой точки до начала координат. Поэтому квадраты поднимаются невысоко, так как они расположены близко к началу координат, а треугольники — значительно (шаг 3), потому что находятся далеко от начала координат. Теперь квадраты и треугольники расположены далеко друг от друга, и мы можем разделить их горизонтальной плоскостью на высоте 1 — другими словами, плоскостью $z = 1$. В качестве последнего шага спроецируем все на плоскость (шаг 4). Пересечение параболоида и плоскости становится окружностью $x_1^2 + x_2^2 = 1$. Обратите внимание на то, что это уравнение нелинейно, так как содержит квадратические члены. Наконец, прогноз, который делает данный классификатор, задается как $\hat{y} = \text{step}(x_1^2 + x_2^2 - 1)$.

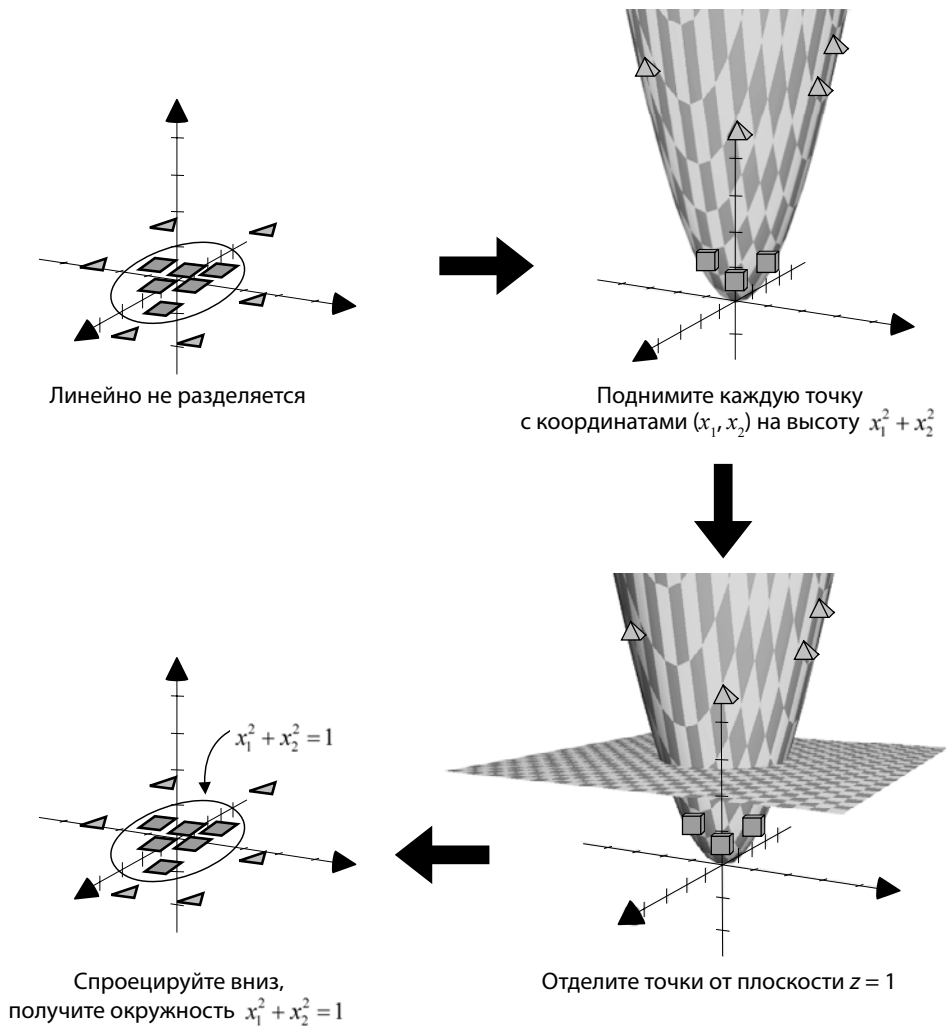


Рис. 11.17. Иллюстрация к методу ядра (источник — изображение, созданное с помощью Grapher™ от Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>)

Пример 2. Измененный набор данных XOR

Окружность — не единственная фигура, которую можно нарисовать. Рассмотрим очень простой набор данных, проиллюстрированный в табл. 11.3 и изображенный на рис. 11.18. Он аналогичен тому, который соответствует оператору XOR из упражнений 5.3 и 10.2. Если вы хотите решить ту же проблему с исходным набором данных XOR, то можете проделать это в упражнении 11.2.

Таблица 11.3. Измененный набор данных XOR

x_1	x_2	y
-1	-1	1
-1	1	0
1	-1	0
1	1	1

**Рис. 11.18.** График набора данных из табл. 11.3. Граничное уравнение классификатора, который отделяет квадраты от треугольников, — $x_1 x_2 = 0$, что соответствует объединению горизонтальной и вертикальной осей

Чтобы увидеть, что этот набор данных не является линейно разделяемым, взгляните на рисунок. Два треугольника расположились в противоположных углах большого квадрата, а два квадрата — в двух других углах. Невозможно провести прямую, отделяющую треугольники от квадратов. Однако мы можем применить для этого полиномиальное уравнение, и в данном случае это будет произведение двух функций. Добавим столбец, соответствующий произведению $x_1 x_2$, в исходный набор данных. Результат показан в табл. 11.4.

Таблица 11.4. Мы добавили в табл. 11.3 столбец, состоящий из произведения первых двух столбцов

x_1	x_2	$x_1 x_2$	y
-1	-1	1	1
-1	1	-1	0
1	-1	-1	0
1	1	1	1

Обратите внимание на то, что прослеживается четкая связь между двумя крайними правыми столбцами таблицы: столбец, соответствующий

произведению x_1x_2 , очень похож на столбец меток. Теперь мы можем увидеть, что хорошим классификатором для данных будет классификатор, имеющий граничное уравнение $x_1x_2 = 1$. Его график представляет собой объединение горизонтальной и вертикальной осей, так как для того, чтобы произведение x_1x_2 оказалось равным 0, нужно, чтобы $x_1 = 0$ или $x_2 = 0$. Прогноз, который делает этот классификатор, задается так: $\hat{y} = \text{step}(x_1x_2)$. Он равен 1 для точек, находящихся в северо-восточном и юго-западном квадрантах плоскости, и 0 — в двух других.

Выход за рамки квадратных уравнений. Полиномиальное ядро

В обоих предыдущих примерах для классификации не разделяемых линейно данных мы использовали полиномиальное выражение. В первом примере это было выражение $x_1^2 + x_2^2$, потому что это значение мало для точек, расположенных вблизи начала координат, и велико для точек, удаленных от него. Во втором примере выражение было x_1x_2 , что помогло разделить точки, находящиеся в разных квадрантах плоскости.

Как мы нашли эти выражения? В более сложном наборе данных у нас может не быть возможности взглянуть на график и найти нужное выражение. Поэтому нужен метод, или, другими словами, алгоритм. Так что мы рассмотрим все возможные одночлены степени 2 (квадратические), содержащие x_1 и x_2 . Это следующие три одночлена: x_1^2 , x_1x_2 и x_2^2 . Назовем эти новые переменные x_3 , x_4 и x_5 и будем относиться к ним так, как если бы они не имели абсолютно никакого отношения к x_1 и x_2 . Применим это к первому примеру с кругом. Набор данных из табл. 11.1 с добавленными столбцами показан в табл. 11.5.

Таблица 11.5. Мы добавили в табл. 11.1 еще три столбца, по одному на каждый из одночленов степени 2 по переменным x_1 и x_2

x_1	x_2	$x_3 = x_1^2$	$x_4 = x_1x_2$	$x_5 = x_2^2$	y
0,3	0,3	0,09	0,09	0,09	0
0,2	0,8	0,04	0,16	0,64	0
-0,6	0,4	0,36	-0,24	0,16	0
0,6	-0,4	0,36	-0,24	0,16	0
-0,4	-0,3	0,16	0,12	0,09	0
0,0	-0,8	0,00	0,00	0,64	0
-0,4	1,2	0,16	-0,48	1,44	1
0,9	-0,7	0,81	-0,63	0,49	1
-1,1	-0,8	1,21	0,88	0,64	1
0,7	0,9	0,49	0,63	0,81	1
-0,9	0,8	0,81	-0,72	0,64	1
0,6	-1,0	0,36	-0,60	1,00	1

Теперь мы можем создать SVM, которая классифицирует такой расширенный набор данных. Способ обучения SVM заключается в использовании методов из предыдущего раздела. Я рекомендую создать такой классификатор с помощью Scikit-Learn, Turi Create или пакета по вашему выбору. Вот одно тривиальное рабочее уравнение классификатора:

$$0x_1 + 0x_2 + 1x_3 + 0x_4 + 1x_5 - 1 = 0.$$

Помня, что $x_3 = x_1^2$ и $x_5 = x_2^2$, мы получаем искомое уравнение окружности:

$$x_1^2 + x_2^2 = 1.$$

Если хочется визуализировать этот процесс геометрически, как делалось в предыдущих случаях, все несколько усложняется. Наш прекрасный двумерный набор данных превратился в пятимерный. В нем точки, помеченные 0 и 1, теперь находятся далеко друг от друга и могут быть разделены четырехмерной гиперплоскостью. Когда мы проецируем все на два измерения, то получаем желаемую окружность.

Полиномиальное ядро дает подъем, который отправляет двумерную плоскость в пятимерное пространство. Отображение отправляет точку (x_1, x_2) в точку $(x_1, x_2, x_1^2, x_1x_2, x_2^2)$. Поскольку максимальная степень каждого одночлена равна 2, мы говорим, что это полиномиальное ядро степени 2. Для полиномиального ядра всегда нужно указывать степень.

Какие столбцы мы добавляем в набор данных, если используем полиномиальное ядро более высокой степени, скажем, k ? Добавляем по одному столбцу для каждого одночлена в заданном наборе переменных, степень которых меньше или равна k . Например, если применяем полиномиальное ядро степени 3 для переменных x_1 и x_2 , то добавляем столбцы, соответствующие одночленам $\{x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3\}$. Подобным образом мы можем проделать это и для большего числа переменных. Например, если мы используем полиномиальное ядро степени 2 для переменных x_1, x_2 и x_3 , то добавляем столбцы с одночленами $\{x_1, x_2, x_3, x_1^2, x_1x_2, x_1x_3, x_2^2, x_2x_3, x_3^2\}$.

Использование выпуклостей в более высоких измерениях в своих интересах. Ядро радиальной базисной функции

Следующее ядро, которое мы рассмотрим, — это *ядро радиальной базисной функции* (RBF). Оно чрезвычайно полезно на практике, потому что может помочь в построении нелинейных границ, используя определенные специальные функции, центрированные в каждой из точек данных. Чтобы дать определение ядру RBF, сначала рассмотрим одномерный пример, показанный на рис. 11.19. Этот набор данных не является линейно разделяемым — квадрат лежит точно между двумя треугольниками. Не существует точки на прямой, которая оставила бы все треугольники на одной стороне и квадрат на другой.

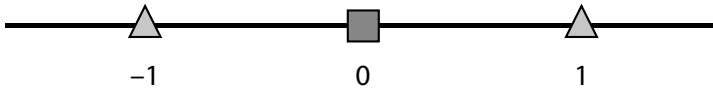


Рис. 11.19. Набор данных в одном измерении, который не может быть классифицирован линейным классификатором

Способ построения классификатора для этого набора данных заключается в том, что мы представим создание горы или долины в каждой из точек. Для точек, помеченных 1 (треугольники), это будет гора, а для точек, помеченных 0 (квадрат), — долина. Называются горы и долины *радиальными базисными функциями*. Результирующая фигура показана в верхней части рис. 11.20. Теперь изобразим горный хребет таким образом, чтобы в каждой точке высота была суммой всех высот гор и долин в этой точке. Результирующий горный хребет изображен в нижней части рис. 11.20. Граница классификатора соответствует точкам, в которых горный хребет находится на нулевой высоте, а именно двум выделенным точкам внизу. Этот классификатор определяет все, что находится в интервале между данными точками, как квадрат (метка 0), а все, что находится за пределами интервала, — как треугольник (метка 1).

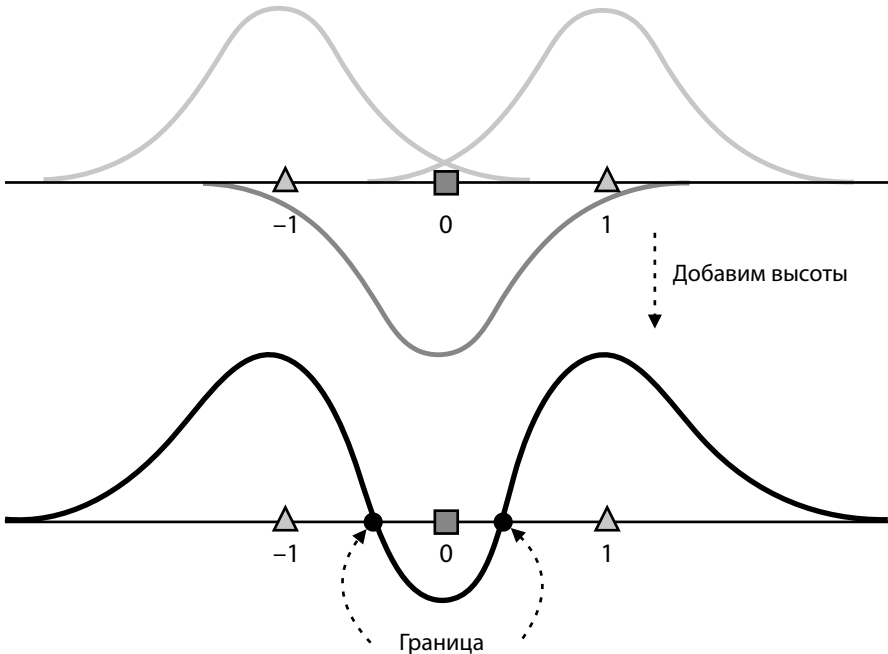


Рис. 11.20. Использование SVM с ядром RBF для разделения нелинейного набора данных в одном измерении

Это плюс кое-какая математика из следующего раздела и есть суть ядра RBF. Теперь применяем его для построения аналогичного классификатора на двумерном наборе данных.

Чтобы получить горы и долины на плоскости, представьте, что плоскость — это одеяло (рис. 11.21). Если ухватить его в одном месте и приподнять, то получится гора, если прижать, возникнет долина. Такие горы и долины являются радиальными базисными функциями. Это семейство функций, которые мы будем использовать для построения нелинейных классификаторов. Они называются так потому, что значение функции в точке зависит только от расстояния между точкой и центром. Мы можем поднять одеяло в любой понравившейся точке, и это даст другую радиальную базисную функцию для каждой точки. Ядро радиальной базисной функции создает карту, которая использует радиальные функции для добавления нескольких столбцов в набор данных, чтобы помочь нам разделить его.

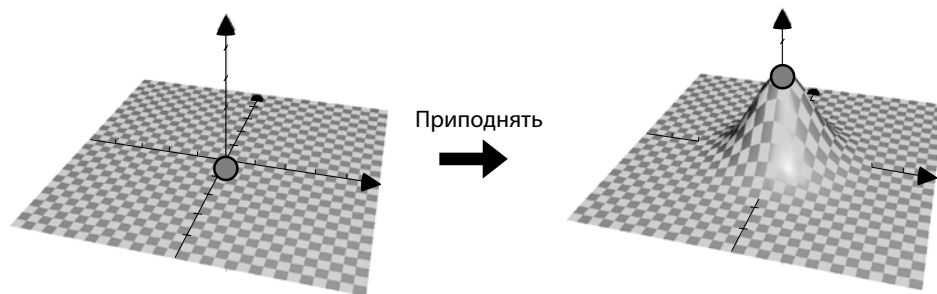


Рис. 11.21. Радиальная базисная функция состоит в поднятии плоскости в определенной точке (источник — изображение, созданное с помощью Grapher™ от Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>)

Как можно использовать это в качестве классификатора? Представьте: у нас есть набор данных (рис. 11.22, *слева*), где, как обычно, треугольники представляют точки с меткой 1, а квадраты — точки с меткой 0. Теперь поднимем плоскость в каждом треугольнике и опустим в каждом квадрате. Получим трехмерный график, показанный на рис. 11.22 *справа*. Обратите внимание на то, что теперь мы можем разделить набор данных плоскостью, что означает, что измененный набор данных линейно разделим.

Чтобы создать классификатор, построим плоскость на высоте 0 и пересечем ее с нашей поверхностью. Это то же самое, что смотреть на кривую, образованную точками на высоте 0. Представьте себе пейзаж с горами и морем. Кривая будет соответствовать береговой линии — месту, где встречаются вода и суша. Эта береговая линия представляет собой кривую, показанную на рис. 11.23, *слева*. Затем мы проецируем все обратно на плоскость и получаем желаемый классификатор, показанный на рис. 11.23, *справа*. Это и есть основная идея, лежащая в основе ядра RBF. Конечно, нам еще предстоит разобрать математику,

что мы и сделаем в следующих нескольких разделах. Но в принципе если мы можем представить, как поднимаем и опускаем одеяло, после чего строим классификатор на основе границы из точек, которые лежат на определенной высоте, то мы понимаем, что такое ядро RBF.

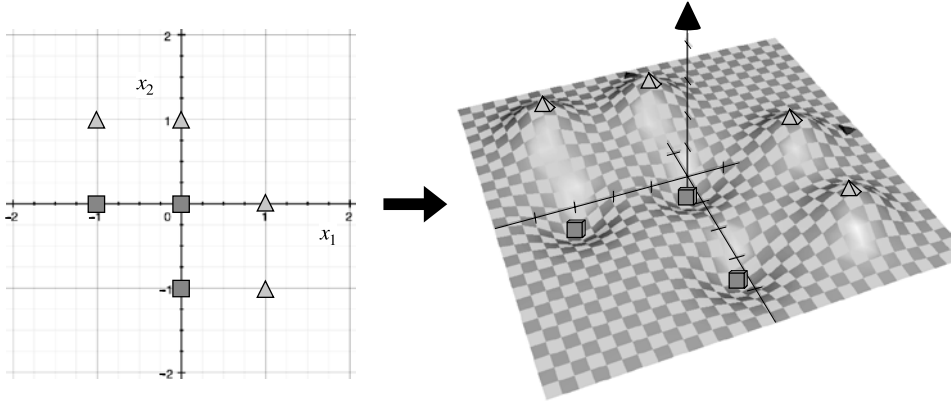


Рис. 11.22. Слева — набор данных в плоскости, который не является линейно разделимым; справа — с помощью радиальных базисных функций каждый из треугольников поднят, а каждый из квадратов опущен (источник — изображение, созданное с помощью Grapher™ от Golden Software, LLC; [https:// www.goldensoftware.com/products/grapher](https://www.goldensoftware.com/products/grapher))

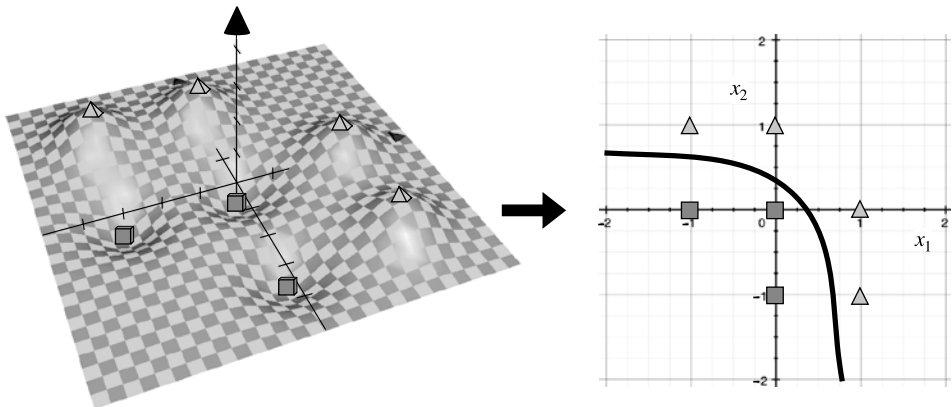


Рис. 11.23. Слева — на высоте 0 точки образуют кривую; справа — проекция точек на плоскость — это классификатор, отделяющий треугольники от квадратов (источник — изображение, созданное с помощью Grapher™ от Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>)

Более детальное рассмотрение радиальных базисных функций

Могут существовать радиальные базисные функции с любым количеством переменных. В начале раздела мы видели их с одной и двумя переменными. Простейшая радиальная базисная функция с одной переменной имеет формулу $y = e^{-x^2}$. Это выглядит как выпуклость над прямой (рис. 11.24). Она очень похожа на стандартное нормальное (гауссово) распределение. Стандартное нормальное распределение аналогично, но имеет несколько иную формулу, так что площадь под ним равна 1.

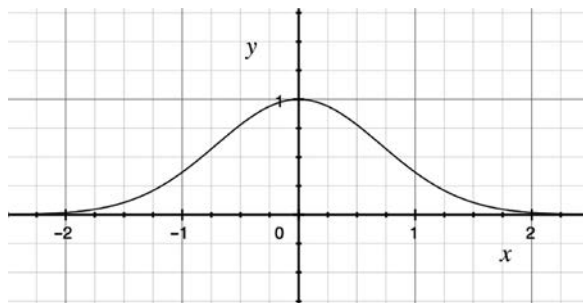


Рис. 11.24. Пример радиальной базисной функции

Обратите внимание на то, что выпуклость появляется при 0. Если бы мы хотели, чтобы она появилась в любой другой точке (скажем, p), то могли бы переписать формулу и получить $y = e^{-(x-p)^2}$. Например, радиальная базисная функция с центром в точке 5 выглядит как $y = e^{-(x-5)^2}$.

Формула основной радиальной базисной функции с двумя переменными выглядит как $z = e^{-(x^2+y^2)}$, ее график показан на рис. 11.25. Вы вновь можете заметить, что это очень похоже на многомерное нормальное распределение. Это также модифицированная версия многомерного нормального распределения.

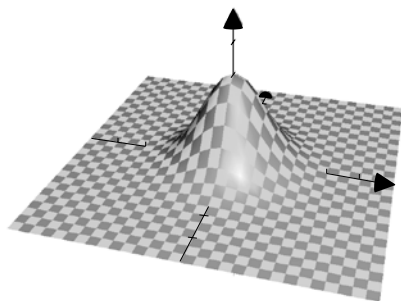


Рис. 11.25. Радиальная базисная функция с двумя переменными
(источник — изображение, созданное с помощью Grapher™ от Golden Software, LLC; <https://www.goldensoftware.com/продукты/grapher>)

Данная выпуклость расположена точно в точке $(0, 0)$. Если бы мы хотели, чтобы она появилась в любой другой точке, скажем, (p, q) , то могли бы перенести формулу и получить $y = e^{-[(x-p)^2 + (y-q)^2]}$. Например, радиальная базисная функция с центром в точке $(2, -3)$ — это $y = e^{-[(x-2)^2 + (y+3)^2]}$.

Формула базовой радиальной базисной функции с n переменными выглядит так: $y = e^{-(x_1^2 + \dots + x_n^2)}$. Мы не в силах изобразить график в $n + 1$ измерениях, но если представим, что поднимаем пальцами n -мерное одеяло, то так это и выглядит. Однако, поскольку алгоритм, который мы используем, является чисто математическим, компьютер без проблем запускает его с необходимым количеством переменных. Как обычно, центр этой n -мерной выпуклости расположен в точке 0 , но если бы мы хотели, чтобы он был в точке $(p_1, \dots, p_n)$, то формула превратилась бы в $y = e^{-[(x_1 - p_1)^2 + \dots + (x_n - p_n)^2]}$.

Показатель близости точек. Подобие

Чтобы построить SVM с использованием ядра RBF, потребуется понятие *подобия*. Мы говорим, что две точки подобны, если они находятся близко друг к другу, и не подобны, если далеко (рис. 11.26). Другими словами, подобие между двумя точками высокое, если они находятся близко друг к другу, и низкое, если далеко. Если пара точек является одной и той же точкой, то подобие равно 1. Теоретически подобие двух точек, находящихся на бесконечном расстоянии друг от друга, равно 0.

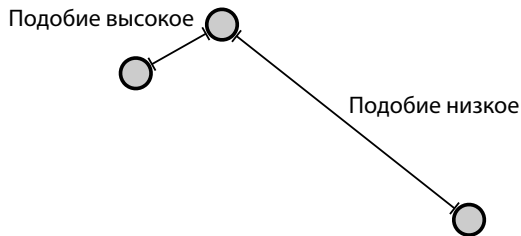


Рис. 11.26. Две точки, расположенные близко друг к другу, определяются как имеющие высокое подобие, две удаленные точки — как имеющие низкое подобие

Теперь нам нужно найти формулу подобия. Как мы видим, подобие между двумя точками уменьшается по мере увеличения расстояния между ними. Таким образом, для подобия будут работать многие формулы, если они удовлетворяют этому условию. Поскольку здесь мы используем экспоненциальные функции, определим подобие между точками p и q как

$$\text{подобие}(p, q) = e^{-\text{расстояние}(p, q)^2}.$$

Это выглядит как сложная формула подобия, но есть очень хороший способ разобраться в ней. Подобие между двумя точками, например p и q , которое мы хотим найти, точно соответствует высоте радиальной базисной функции, центрированной в точке p и применяемой в точке q . Это означает, что если мы ухватим одеяло в точке p и поднимем его, то его высота в точке q будет большой, если q близко к p , и маленькой, если q далеко от p . На рис. 11.27 это показано для одной переменной, но можно распространить данный процесс на любое число переменных, используя общую аналогию.

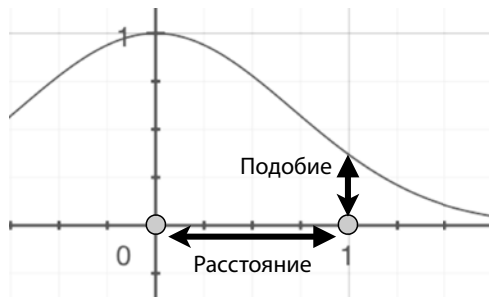


Рис. 11.27. Подобие определяется как высота точки в радиальной базисной функции, где входным значением служит расстояние

Обучение SVM с помощью ядра RBF

Теперь, когда у нас есть все инструменты для обучения SVM с использованием ядра RBF, поговорим о том, как можно все это объединить. Прежде всего рассмотрим простой набор данных, показанный на рис. 11.19. Он приведен в табл. 11.6.

Таблица 11.6. Одномерный набор данных, изображенный на рис. 11.19

Точка	x	y (метка)
1	-1	1
2	0	0
3	1	1

Как видите, этот набор данных не является линейно разделяемым, потому что точка с меткой 0 находится между двумя точками с меткой 1. Чтобы сделать его линейно разделяемым, добавим несколько столбцов. Три добавляемых столбца являются столбцами подобия, в них заносится подобие точек. Подобие двух точек с координатами x_1 и x_2 задается формулой $e^{-(x_1-x_2)^2}$, как указано в разделе «Использование выпуклостей в более высоких измерениях в своих интересах». Например, подобие точек 1 и 2 — это $e^{(-1-0)^2} = 0,368$. В столбце Sim1 запишем подобие точки 1 и трех других точек и т. д. Расширенный набор данных показан в табл. 11.7.

Таблица 11.7. Расширяем набор данных из табл. 11.6, добавив три новых столбца

Точка	x	Sim1	Sim2	Sim3	y
1	-1	1,000	0,368	0,018	1
2	0	0,368	1,000	0,368	0
3	1	0,018	0,368	1,000	1

Каждый столбец здесь соответствует подобию всех точек относительно каждой точки. Этот расширенный набор данных находится в четырехмерном пространстве и линейно разделим! Многие классификаторы смогут разделить такой набор, в частности тот, который имеет следующее граничное уравнение:

$$\hat{y} = \text{step}(\text{Sim1} - \text{Sim2} + \text{Sim3}).$$

Давайте проверим, спрогнозировав метку в каждой точке, как показано далее:

- **точка 1:** $\hat{y} = \text{step}(1 - 0,368 + 0,018) = \text{step}(0,65) = 1$;
- **точка 2:** $\hat{y} = \text{step}(0,368 - 1 + 0,368) = \text{step}(-0,264) = 0$;
- **точка 3:** $\hat{y} = \text{step}(0,018 - 0,368 + 1) = \text{step}(0,65) = 1$.

Более того, поскольку $\text{Sim1} = e^{(x+1)^2}$, $\text{Sim2} = e^{(x-0)^2}$ и $\text{Sim3} = e^{(x-1)^2}$, наш итоговый классификатор делает следующие прогнозы:

$$\hat{y} = \text{step}\left(e^{(x+1)^2} - e^{x^2} + e^{(x-1)^2}\right).$$

Теперь сделаем ту же процедуру, но в двух измерениях. Этот раздел не требует кода, но вычисления большие, поэтому, если хотите взглянуть на них, ищите в следующем блокноте: https://github.com/luisguiserrano/manning/blob/master/Chapter_11_Support_Vector_Machines/Calculating_similarities.ipynb.

Рассмотрим набор данных (табл. 11.8), который мы уже классифицировали, графически (см. рис. 11.22 и 11.23). Для удобства он еще раз показан на рис. 11.28. На этом графике точки с меткой 0 обозначены квадратами, а точки с меткой 1 — треугольниками, причем их невозможно разделить прямой. Для его классификации используем SVM с ядром RBF.

Таблица 11.8. Простой набор данных в двух измерениях, представленный на рис. 11.28

Точка	x_1	x_2	y
1	0	0	0
2	-1	0	0
3	0	-1	0
4	0	1	1
5	1	0	1
6	-1	1	1
7	1	-1	1

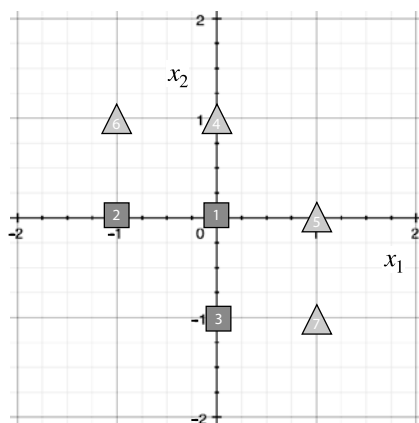


Рис. 11.28. График набора данных из табл. 11.8

Обратите внимание на то, что в первом столбце таблицы и на рис. 11.28 мы пронумеровали каждую точку. Это не часть данных, так сделано только для удобства. Теперь добавим в эту таблицу семь столбцов. Столбцы — это подобию по отношению к каждой точке. Например, для точки 1 мы добавляем столбец подобию с именем **Sim1**. Запись для каждой точки в этом столбце представляет собой величину подобию между этой точкой и точкой 1. Давайте вычислим одно из них, например с точкой 6. Расстояние между точкой 1 и точкой 6, согласно теореме Пифагора, — это

$$\text{расстояние}(\text{точка 1, точка 6}) = \sqrt{(0+1)^2 + (0-1)^2} = \sqrt{2}.$$

Следовательно, подобие вычисляется так:

$$\text{подобие}(\text{точка 1, точка 6}) = e^{-\text{расстояние}(q, p)^2} = e^{-2} = 0,135.$$

Это число расположено в строке 1 и столбце **Sim6** (и симметрично — в строке 6 и столбце **Sim1**). Заполните еще несколько значений в этой таблице, чтобы убедиться, что это так, или загляните в блокнот, где вычисляется вся таблица. Результат показан в табл. 11.9.

Обратите внимание на следующее.

1. Подобие каждой точки самой себе всегда равно 1.
2. Для каждой пары точек подобие является высоким, когда они расположены близко на графике, и низким, когда далеко.
3. Таблица, состоящая из столбцов **Sim1–Sim7**, симметрична, поскольку подобие p и q такое же, как и подобие q и p (поскольку зависит только от расстояния между p и q).
4. Подобие точек 6 и 7 выглядит как 0, но на самом деле это не так. Расстояние между точками 6 и 7 — это $\sqrt{2^2 + 2^2} = \sqrt{8}$, так что их подобие равно $e^{-8} = 0,00033546262$, что округляется до нуля, потому что мы используем только три значащие цифры.

Таблица 11.9. Мы добавили семь столбцов подобий к набору данных из табл. 11.8. Каждый из них фиксирует подобие со всеми остальными шестью точками

Точка	x_1	x_2	Sim1	Sim2	Sim3	Sim4	Sim5	Sim6	Sim7	y
1	0	0	1,000	0,368	0,368	0,368	0,368	0,135	0,135	0
2	-1	0	0,368	1,000	0,135	0,135	0,018	0,368	0,007	0
3	0	-1	0,368	0,135	1,000	0,018	0,135	0,007	0,368	0
4	0	1	0,368	0,135	0,018	1,000	0,135	0,368	0,007	1
5	1	0	0,368	0,018	0,135	0,135	1,000	0,007	0,368	1
6	-1	1	0,135	0,368	0,007	0,367	0,007	1,000	0,000	1
7	1	-1	0,135	0,007	0,368	0,007	0,368	0,000	1,000	1

Теперь перейдем к созданию классификатора! Обратите внимание на то, что для данных в маленькой табл. 11.8 линейный классификатор не работает, потому что точки не могут быть разделены прямой, но для гораздо большей табл. 11.9, в которой намного больше признаков (столбцов), такой классификатор уже можно применять. Мы приступаем к обучению SVM на этих данных. Многие SVM могут правильно классифицировать этот набор данных, и в блокноте я использовал Turi Create для его создания. Тем не менее более простой вариант работает не хуже. Этот классификатор имеет следующие веса:

- веса x_1 и x_2 равны 0;
- вес Sim p равен 1 для $p = 1, 2$ и 3;
- вес Sim p равен -1 для $p = 4, 5, 6$ и 7;
- смещение $b = 0$.

Мы обнаруживаем, что классификатор добавлял метку -1 к столбцам, соответствующим точкам, помеченным как 0, и +1 к столбцам, соответствующим точкам, помеченным как 1. Это эквивалентно процессу добавления горы в любой точке с меткой 1 и долины — в каждой точке с меткой 0, как показано на рис. 11.29. Чтобы математически проверить, что все работает, возьмите табл. 11.7, добавьте значения столбцов Sim4, Sim5, Sim6 и Sim7, затем вычтите значения столбцов Sim1, Sim2 и Sim3. Вы заметите, что в первых трех строках получается отрицательное число, а в последних четырех — положительное. Следовательно, мы можем использовать пороговое значение 0 для получения классификатора, который правильно классифицирует этот набор данных, потому что точки, помеченные 1, получают положительную оценку, а точки, помеченные 0, — отрицательную. Применение порогового значения 0 эквивалентно использованию береговой линии для разделения точек на графике на рис. 11.29. Плоскость пересекает поверхность по изогнутой границе. После проецирования обратно на плоскость эта изогнутая граница (изображена справа) и оказывается кривой, разделяющей треугольники и квадраты.

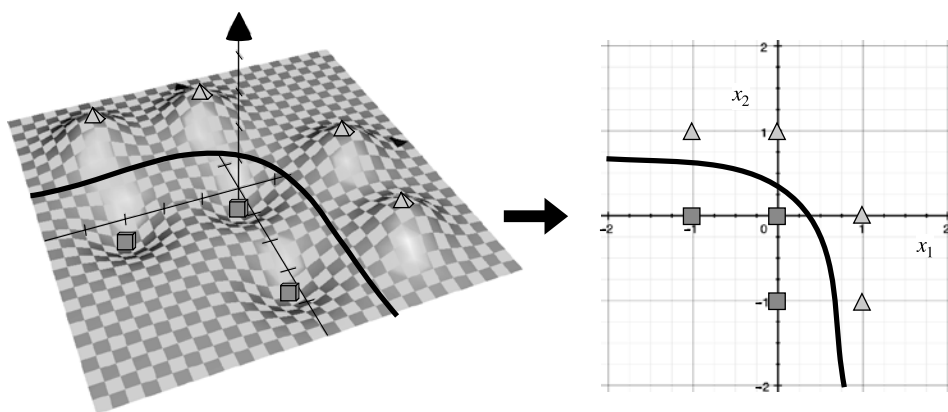


Рис. 11.29. В этом наборе данных каждый треугольник поднят, а каждый квадрат опущен. На высоте 0 построена плоскость, разделяющая квадраты и треугольники (источник — изображение, созданное с помощью Grapher™ от Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>)

Если мы подключим функцию подобия, то полученный классификатор будет следующим:

$$\hat{y} = \text{step} \left(-e^{x_1^2 + x_2^2} - e^{(x_1+1)^2 + x_2^2} - e^{x_1^2 + (x_2+1)^2} + e^{x_1^2 + (x_2-1)^2} + \right. \\ \left. + e^{(x_1-1)^2 + x_2^2} + e^{(x_1+1)^2 + (x_2-1)^2} + e^{(x_1-1)^2 + (x_2+1)^2} \right).$$

Подводя итог, мы нашли набор данных, который не был линейно разделяемым. Мы использовали радиальные базисные функции и подобие точек, чтобы добавить в него несколько столбцов. Это помогло построить линейный классификатор в гораздо более многомерном пространстве. Затем мы спроецировали многомерный линейный классификатор на плоскость, чтобы получить требуемый классификатор. Полученный искривленный классификатор показан на рис. 11.29.

Переобучение и недообучение в случае ядра RBF. Параметр γ

В начале этого раздела мы упоминали, что существует множество различных радиальных базисных функций, а точнее, по одной на каждую точку плоскости. На самом деле их гораздо больше. Некоторые из них поднимают плоскость в точке и образуют узкую поверхность, другие же образуют широкую. Некоторые примеры можно увидеть на рис. 11.30. На практике нам нужно уметь настраивать широту радиальных базисных функций. Для этого мы используем *параметр γ* («гамма»). Когда параметр γ мал, образуемая поверхность очень широкая, а когда велик, то очень узкая.

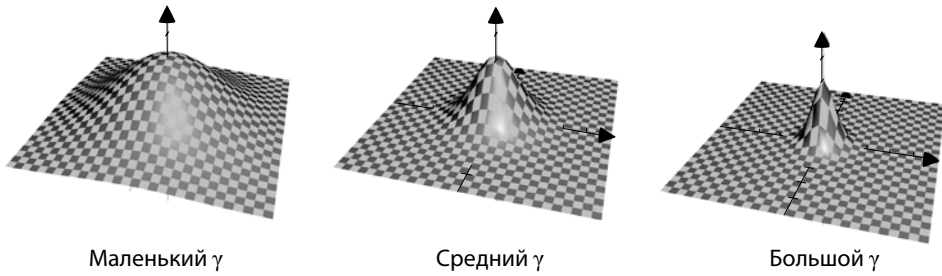


Рис. 11.30. Параметр γ определяет ширину поверхности (источник — изображение, созданное с помощью Grapher™ от Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>)

γ — это гиперпараметр. Напомним, что гиперпараметры — это настройки, используемые для обучения модели. Способ настройки этого гиперпараметра заключается в применении методов, которые мы рассматривали ранее, таких как график сложности модели (см. раздел «Численный способ определить нужную сложность модели...» главы 4). Различные значения γ ведут к переобучению и недообучению. Давайте вернемся к примеру в начале этого раздела с тремя различными значениями γ . Три модели показаны на рис. 11.31.

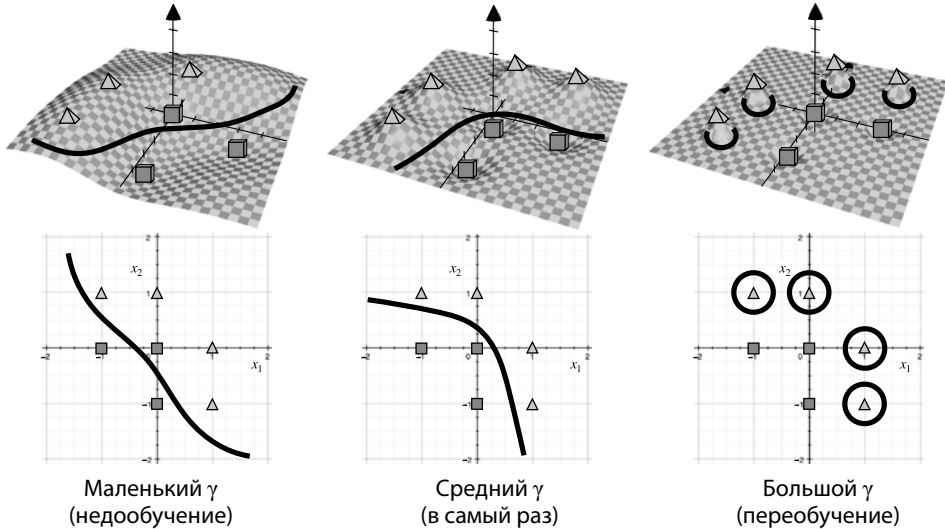


Рис. 11.31. Три SVM-классификатора с ядром RBF и различными значениями γ (источник — изображение, созданное с помощью Grapher™ от Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>)

Обратите внимание на то, что при очень малом значении γ модель переобучается, потому что кривая слишком проста и плохо классифицирует данные. При большом значении γ модель значительно переобучается, потому что строит крошечную гору для каждого треугольника и крошечную долину для каждого квадрата. Поэтому она классифицирует почти все как квадрат, за исключением областей непосредственно вокруг треугольников. Среднее значение γ срабатывает неплохо, потому что строит довольно простую границу, но при этом правильно классифицирует точки.

Уравнение для радиальной базисной функции не сильно меняется, когда мы добавляем параметр γ , — нам нужно лишь умножить на него показатель степени. В общем случае уравнение радиальной базисной функции выглядит так:

$$y = e^{-\gamma[(x_1 - p_1)^2 + \dots + (x_n - p_n)^2]}.$$

Не старайтесь заучивать эту формулу — просто помните, что даже при большом числе измерений создаваемые выпуклости могут быть широкими или узкими. Как обычно, все это можно закодировать и заставить работать, что мы и сделаем в следующем разделе.

Кодирование метода ядра

Теперь, когда мы изучили метод ядра для SVM, рассмотрим их кодирование в Scikit-Learn и обучим модель на более сложном наборе данных, используя полиномиальное ядро и ядро RBF. Чтобы обучить SVM в Scikit-Learn с определенным ядром, добавим это ядро в качестве параметра при определении SVM. Код для этого раздела приведен далее.

- **Блокнот** SVM_graphical_example.ipynb:
https://github.com/luisguiserrano/manning/blob/master/Chapter_11_Support_Vector_Machines/SVM_graphical_example.ipynb.
- **Набор данных:**
 - one_circle.csv;
 - two_circles.csv.

Кодирование полиномиального ядра для классификации кругового набора данных

В этом подразделе мы узнаем, как закодировать полиномиальное ядро в Scikit-Learn. Для этого используем набор данных `one_circle.csv`, показанный на рис. 11.32.

Обратите внимание на то, что, за исключением некоторых выбросов, набор данных в основном круговой. Мы обучим классификатор SVM, в котором укажем, что параметр `kernel` равен `poly`, а параметр `degree` равен 2, как показано в следующем фрагменте кода:

```
svm_degree_2 = SVC(kernel='poly', degree=2)
svm_degree_2.fit(features, labels)
```

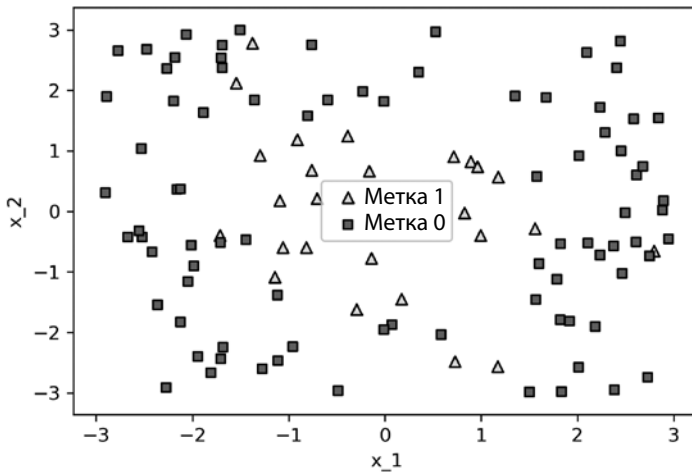


Рис. 11.32. Круговой набор данных с некоторым шумом. Для его классификации используем SVM с полиномиальным ядром

Причина, по которой мы задаем степень 2, заключается в том, что уравнение окружности является многочленом степени 2. Результат показан на рис. 11.33.

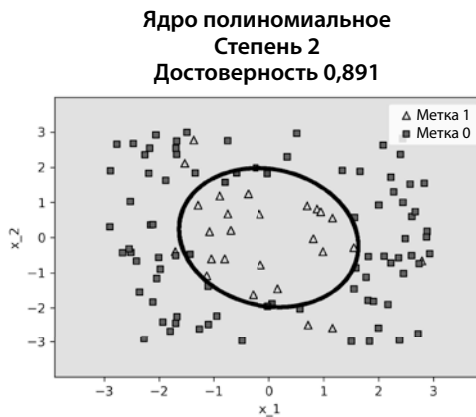


Рис. 11.33. Классификатор SVM с полиномиальным ядром степени 2

Обратите внимание на то, что этой SVM с полиномиальным ядром степени 2 удастся построить в целом круговую граничную область, что и требовалось.

Кодирование ядра RBF для классификации набора данных, образованного двумя пересекающимися окружностями, и работа с параметром γ

Мы построили окружность, а теперь усложним задачу. В этом подразделе узнаем, как закодировать несколько SVM с помощью ядра RBF для классификации набора данных, имеющего форму двух пересекающихся окружностей. Этот набор данных `two_circles.csv` показан на рис. 11.34.

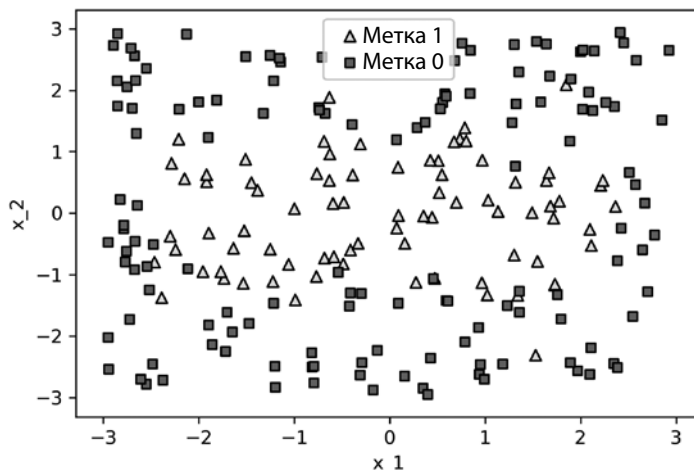


Рис. 11.34. Набор данных, состоящий из двух пересекающихся окружностей с некоторыми выбросами

Чтобы использовать ядро RBF, укажем `kernel = 'rbf'`. Мы также можем указать значение γ . Обучим четыре различных классификатора SVM для следующих значений γ — 0,1, 1, 10 и 100:

```
svm_gamma_01 = SVC(kernel='rbf', gamma=0.1)  ←  $\gamma = 0,1$ 
svm_gamma_01.fit(features, labels)

svm_gamma_1 = SVC(kernel='rbf', gamma=1)    ←  $\gamma = 1$ 
svm_gamma_1.fit(features, labels)

svm_gamma_10 = SVC(kernel='rbf', gamma=10)   ←  $\gamma = 10$ 
svm_gamma_10.fit(features, labels)

svm_gamma_100 = SVC(kernel='rbf', gamma=100) ←  $\gamma = 100$ 
svm_gamma_100.fit(features, labels)
```

Четыре классификатора показаны на рис. 11.35. Обратите внимание на то, что при $\gamma = 0,1$ модель немного недообучена, потому что считает, что граница представляет собой единственный овал, и допускает некоторые ошибки. Удачную модель, которая хорошо ухватывает суть данных, дает $\gamma = 1$. Добавившись

до $\gamma = 10$, мы видим, что модель начинает переобучаться. Она пытается правильно классифицировать каждую точку, включая выбросы, которые обводит отдельно. А при $\gamma = 100$ видим уже весьма серьезный уровень переобучения. Классификатор окружает каждый треугольник небольшой круглой областью, а все остальное классифицирует как квадрат.

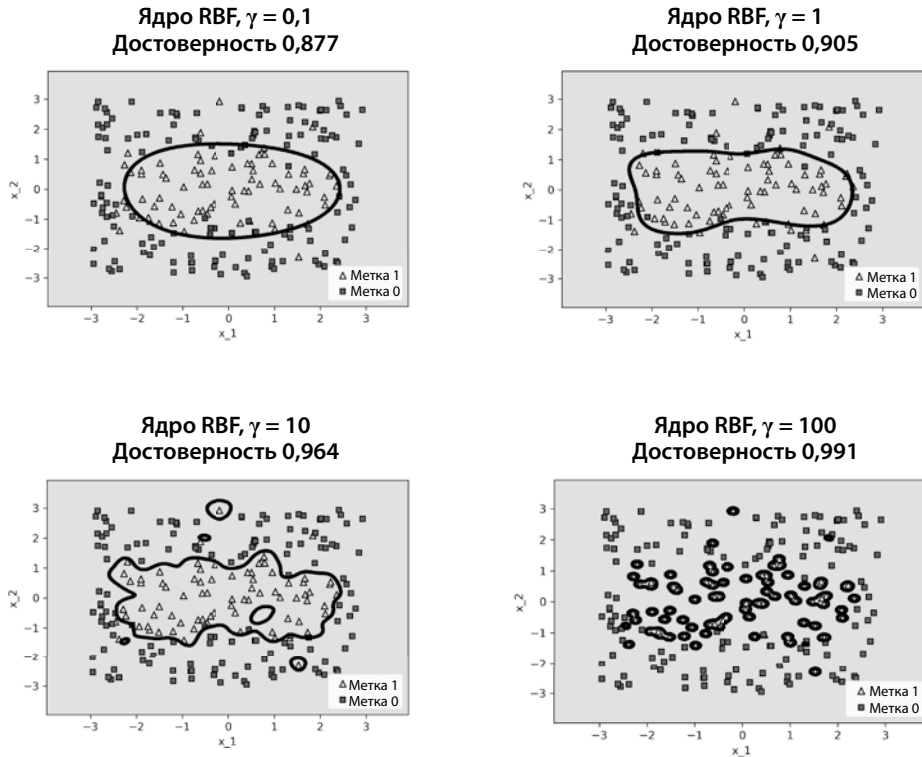


Рис. 11.35. Четыре классификатора SVM с ядром RBF и различными значениями γ

Таким образом, для этой модели $\gamma = 1$, по-видимому, — лучшее значение среди опробованных.

Резюме

- Машина опорных векторов (SVM) — это классификатор, который строит две параллельные прямые или гиперплоскости и пытается разместить их как можно дальше друг от друга, одновременно стараясь правильно классифицировать данные.
- Способ построения машин с опорными векторами заключается в использовании функции ошибок, которая состоит из двух слагаемых: суммы двух

ошибок персептрона, по одной на каждую параллельную прямую, и ошибки расстояния (высокой, если две параллельные прямые находятся далеко друг от друга, и низкой, когда они расположены близко друг к другу).

- Мы используем параметр C для задания приоритета между попытками правильно классифицировать точки и отодвинуть от них прямые. Это полезно во время обучения, потому что позволяет контролировать предпочтения классификатора в зависимости от того, нужен нам классификатор, который очень хорошо разделяет данные, или же классификатор с хорошо разнесенными границами.
- Метод ядра — это полезный и очень мощный инструмент для построения нелинейных классификаторов.
- Метод ядра состоит в использовании функций, которые помогают встраивать набор данных в многомерное пространство, где точки легче определять с помощью линейного классификатора. Это эквивалентно разумному добавлению столбцов в набор данных для получения линейно разделяемого расширенного набора данных.
- Нам доступны несколько различных ядер, таких как полиномиальное ядро и ядро RBF. Полиномиальное ядро позволяет строить полиномиальные области, такие как окружности, параболы и гиперболы. Ядро RBF позволяет создавать более сложные изогнутые области.

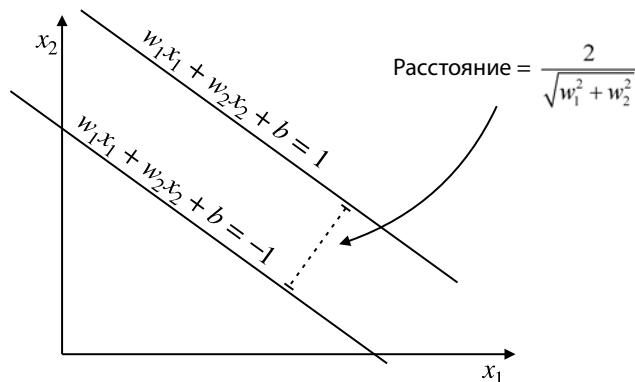
Упражнения

Упражнение 11.1

(Завершает вычисления, необходимые в разделе «Функция ошибки расстояния...»)

Покажите, что расстояние между прямыми:

$$w_1x_1 + w_2x_2 + b = 1 \text{ и } w_1x_1 + w_2x_2 + b = -1 \text{ — равно } \frac{2}{\sqrt{w_1^2 + w_2^2}}.$$



Упражнение 11.2

Как мы узнали из упражнения 5.3, невозможно построить модель персептрона, имитирующую элемент XOR. Другими словами, невозможно сопоставить следующий набор данных (со 100%-ной точностью) с моделью персептрона.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Это связано с тем, что набор данных не является линейно разделяемым. У SVM та же проблема, так как она является линейной моделью. Однако мы можем использовать ядро, чтобы обойти это ограничение. Какое ядро следует задействовать, чтобы превратить этот набор данных в линейно разделяемый? Как будет выглядеть результирующая SVM?

ПОДСКАЗКА Обратитесь к примеру 2, приведенному в разделе «Использование полиномиальных уравнений в своих интересах...», который решает очень похожую задачу.

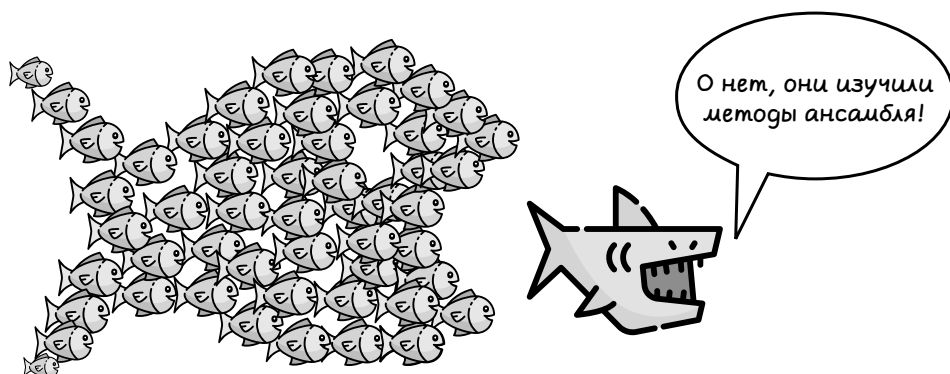
12

Комбинирование моделей для достижения максимальных результатов. Ансамблевое обучение



В этой главе

- ✓ Что такое ансамблевое обучение и как его использовать для объединения слабых классификаторов в более сильные.
 - ✓ Применение бэггинга для случайного объединения классификаторов.
 - ✓ Использование бустинга для более разумного объединения классификаторов.
 - ✓ Некоторые из наиболее популярных методов ансамбля: случайные леса, AdaBoost, градиентное повышение и XGBoost.
-



После изучения многих интересных и полезных моделей машинного обучения вполне естественно задать вопрос: можно ли объединять эти классификаторы? К счастью, это возможно, и в данной главе рассматриваются несколько способов построения более мощных моделей путем объединения более слабых. Два основных метода, которые мы изучим в этой главе, — это бэггинг и бустинг. Если в двух словах, то бэггинг состоит в том, чтобы случайным образом собрать несколько моделей и соединить их. Бустинг же заключается в том, чтобы строить эти модели более разумно, выбирая каждую из них стратегически, сосредотачиваясь на ошибках предыдущих моделей. Названные методы ансамбля показали потрясающие результаты при решении некоторых важных задач машинного обучения. Например, премию Netflix, которая присуждалась лучшей модели, соответствующей большому набору данных о зрительской аудитории Netflix, выиграла группа, использовавшая ансамбль различных моделей.

В этой главе мы познакомимся с некоторыми из самых мощных и популярных моделей бэггинга и бустинга, включая случайные леса, AdaBoost, градиентный бустинг и XGBoost. Большинство из них описаны для случая классификации и лишь некоторые — для регрессии. Однако большинство методов ансамбля работают в обоих случаях.

Немного терминологии: на протяжении всей книги мы называли модели машинного обучения моделями, а иногда регрессорами или классификаторами в зависимости от их задач. Здесь же вводим термин «обучающий алгоритм», который также относится к модели машинного обучения. В литературе, когда речь идет о методах ансамбля, принято использовать термины «слабый обучающий алгоритм» и «сильный обучающий алгоритм». Однако нет никакой разницы между моделью машинного обучения и обучающим алгоритмом.

Весь код для этой главы доступен в репозитории GitHub: https://github.com/luisguiserrano/manning/tree/master/Chapter_12_Ensemble_Methods.

С небольшой помощью наших друзей

Давайте визуализируем методы ансамбля, используя следующую аналогию: представьте, что нам нужно сдать экзамен, состоящий из 100 вопросов с ответами «правда/ложь» по различным темам, включая математику, географию, естественные науки, историю и музыку. К счастью, нам разрешили позвать на помощь пятерых друзей — Адриану, Боба, Карлоса, Дану и Эмили. Существует небольшое ограничение: все они работают полный рабочий день, и у них нет времени ответить на все 100 вопросов. Так что они будут рады помочь с некоторыми из них. Какие методы мы можем использовать, чтобы получить их помощь? Вот пара возможных техник.

Техника 1. Для каждого из друзей выберите несколько случайных вопросов и попросите ответить на них (убедитесь, что на каждый вопрос отвечает хотя бы один из друзей). Получив ответы, пройдите тест, выбрав вариант, который встречался в ответах наиболее часто. Например, если на 1-й вопрос два друга ответили «правда», а один — «ложь», то нужно ответить «правда» (если есть совпадения, можно выбрать один из выигрышных ответов случайным образом).

Техника 2. Мы даем тест Адриане и просим ее ответить только на те вопросы, в которых она наиболее уверена. Предполагая, что ее ответы достаточно хороши, удаляем их из теста. Далее задаем оставшиеся вопросы Бобу с теми же инструкциями. Продолжаем в том же духе, пока тест не пройдет через руки всех пятерых.

Метод 1 напоминает алгоритм бэггинга, а метод 2 — алгоритм бустинга. Чтобы быть более конкретным, бэггинг и бустинг используют набор моделей, называемых *слабыми алгоритмами*, и объединяют их в *сильные алгоритмы* (рис. 12.1).

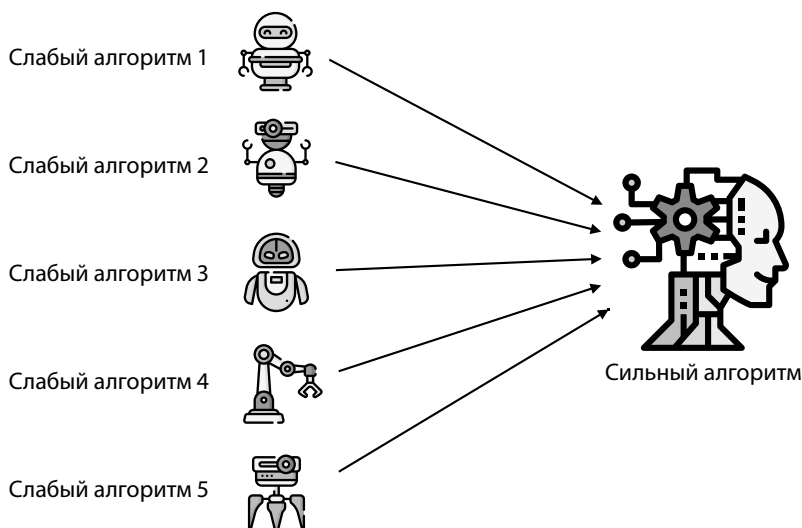


Рис. 12.1. Методы ансамбля состоят в объединении нескольких слабых обучающих алгоритмов в сильный

БЭГГИНГ. Создание случайных наборов путем выборки случайных точек из набора данных (с заменой). Обучайте разные модели на каждом из наборов. Эти модели — слабые обучающие алгоритмы. Затем из слабых моделей комбинируется сильный обучающий алгоритм, и прогнозирование выполняется голосованием (если это классификационная модель) или усреднением прогнозов (если регрессионная).

БУСТИНГ. Начните с обучения случайной модели, которая является первым слабым обучающим алгоритмом. Оцените ее по всему набору данных. Уменьшите точки с хорошими прогнозами и увеличьте точки с плохими. Обучите второй слабый алгоритм на измененном наборе данных. Продолжайте в том же духе до тех пор, пока не будут построены несколько моделей. Способ объединить их в сильный алгоритм такой же, как и для бэггинга, а именно путем голосования или усреднения прогнозов слабой модели. Более конкретно: если обучающие алгоритмы являются классификаторами, сильный алгоритм прогнозирует наиболее распространенный класс, предсказанный слабыми алгоритмами (отсюда термин «голосование»). Если модели являются регрессорами, сильный обучающий алгоритм предсказывает среднее значение прогнозов, данных слабыми.

Большинство моделей в этой главе в качестве слабых обучающих алгоритмов используют деревья решений (как для регрессии, так и для классификации). Они очень хорошо применимы для такого подхода. Однако, читая эту главу, подумайте о том, как бы вы могли комбинировать другие типы моделей, такие как перцептроны и SVM.

Мы на протяжении всей книги учились создавать очень хорошие модели. Почему же сейчас пытаемся объединить несколько слабых моделей вместо того, чтобы сразу построить сильную? Одна из причин в том, что методы ансамбля переобучаются гораздо меньше, чем другие модели. В двух словах: одной модели легко переобучиться, но, если у вас их несколько для одного и того же набора данных, их комбинация переобучается меньше. В каком-то смысле получается, что, если один обучающий алгоритм совершает ошибку, другие, как правило, исправляют ее и в среднем работают лучше.

В этой главе мы изучаем следующие модели:

- случайные леса;
- AdaBoost;
- бустинг градиента;
- XGBoost.

Первая из них — это алгоритм бэггинга, следующие три — бустинг. Все эти модели работают как для регрессии, так и для классификации. В образовательных целях мы изучим первые две как классификационные модели, а последние две — как регрессионные. Процесс аналогичен как для классификации, так и для регрессии. Тем не менее попробуйте представить, как это будет работать в обоих случаях. Чтобы узнать, как все эти алгоритмы работают в случае классификации и регрессии, смотрите ссылки на видео и материалы для чтения в приложении В, где подробно разобраны оба случая.

Бэггинг. Случайное объединение нескольких слабых моделей для создания сильной

В этом разделе рассмотрим одну из самых известных моделей бэггинга — случайный лес. В алгоритме случайного леса слабые обучающие алгоритмы представляют собой небольшие деревья решений, обученные на случайных подмножествах набора данных. Случайный лес хорошо подходит для задач классификации и регрессии, причем процесс в обоих случаях аналогичен. Мы рассмотрим случайные леса на примере классификации. Код для этого раздела приведен далее.

- **Блокнот** `Random_forests_and_AdaBoost.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_12_Ensemble_Methods/Random_forests_and_AdaBoost.ipynb.

Возьмем небольшой набор данных спама и полезных писем, аналогичный тому, который использовали в главе 8, рассматривая наивную байесовскую модель. Набор данных показан в табл. 12.1 и на рис. 12.2. Признаками набора данных выступают счетчики появления в электронном письме слов «лотерея» и «распродажа», а метка «да/нет» указывает на то, является ли электронное письмо спамом (да) или полезным (нет).

Таблица 12.1. Таблица спама и полезных электронных писем, а также количество появлений слов «лотерея» и «распродажа» в каждом из них

Лотерея	Распродажа	Спам
7	8	1
3	2	0
8	4	1
2	6	0
6	5	1
9	6	1
8	5	0
7	1	0
1	9	1
4	7	0
1	3	0
3	10	1
2	2	1
9	3	0
5	3	0
10	1	0
5	9	1
10	8	1

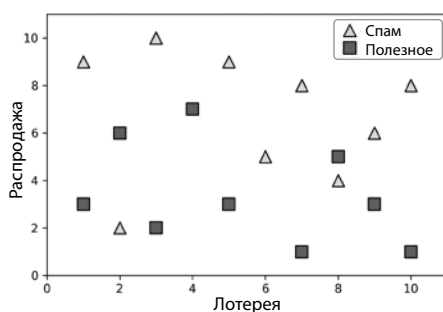


Рис. 12.2. График набора данных из табл. 12.1. Спам-письма представлены треугольниками, а полезные — квадратами

Обучение и переобучение дерева решений

Прежде чем перейти к случайным лесам, давайте применим классификатор дерева решений к этим данным и посмотрим, насколько хорошо он работает. Поскольку это то, что мы изучали в главе 9, на рис. 12.3 показан только конечный результат, но код можно найти в блокноте. Слева на рис. 12.3 фактическое дерево (довольно глубокое!), а справа — график границы. Обратите внимание на то, что он очень хорошо соответствует набору данных со 100%-ной достоверностью обучения, хотя явно переобучен. Это можно определить по двум выбросам, которые модель пытается правильно классифицировать, не замечая, что это выбросы.

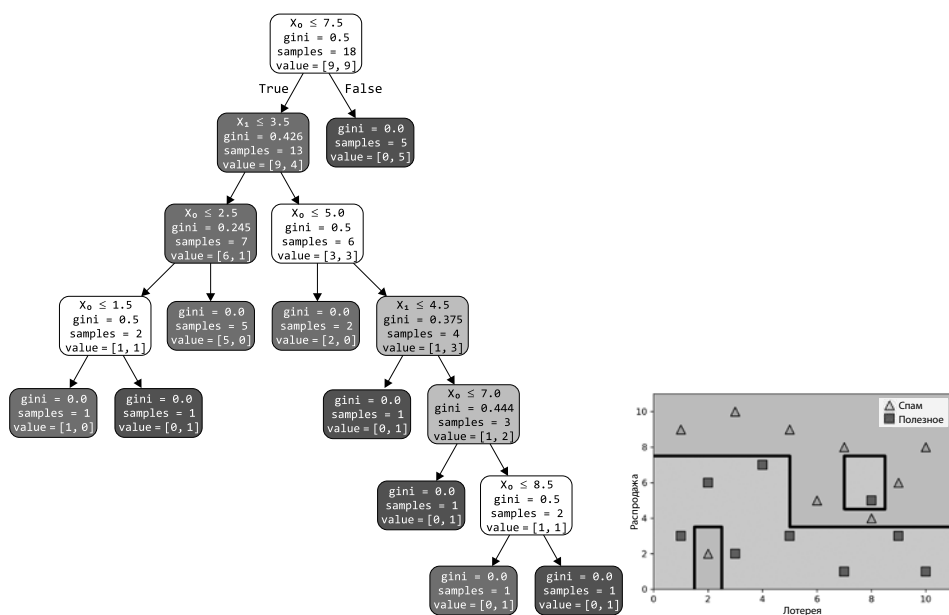


Рис. 12.3. Слева — дерево решений, которое классифицирует набор данных; справа — граница, определенная деревом решений

В следующих разделах поговорим о том, как устранить проблему переобучения с помощью алгоритма случайного леса.

Обучение случайного леса вручную

Здесь мы узнаем, как обучать случайный лес вручную. Прделаем это в образовательных целях, на практике все происходит не так. Если кратко, то мы выбираем случайные подмножества из набора данных и обучаем слабый алгоритм (дерево решений) на каждом из них. Некоторые точки данных могут принадлежать нескольким подмножествам, некоторые — не принадлежать ни одному. Их сочетание — это наш сильный обучающий алгоритм. Способ, которым сильный алгоритм делает прогнозы, заключается в том, что он позволяет слабым голосовать. Для этого набора данных мы используем три слабых алгоритма. Поскольку набор данных содержит 18 точек, рассмотрим три подмножества по шесть точек данных в каждом (рис. 12.4).

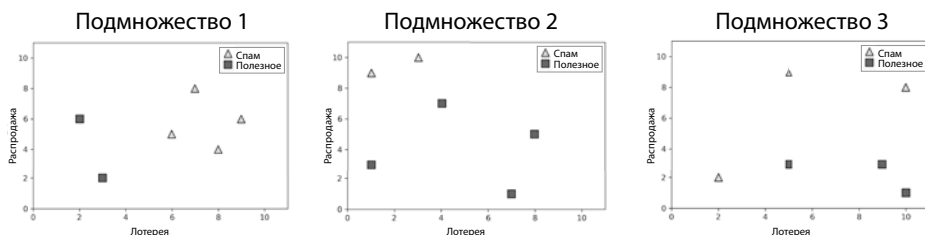


Рис. 12.4. Первый шаг при создании случайного леса — разделение данных на три подмножества (см. рис. 12.2)

Далее мы приступаем к построению трех слабых алгоритмов. Обучите дерево решений глубиной 1 для каждого из этих подмножеств. Вспомним из главы 9, что дерево решений глубиной 1 содержит только один узел и два листа. Его граница состоит из одной горизонтальной или вертикальной прямой, которая максимально разделяет набор данных. Слабые обучающие алгоритмы показаны на рис. 12.5.

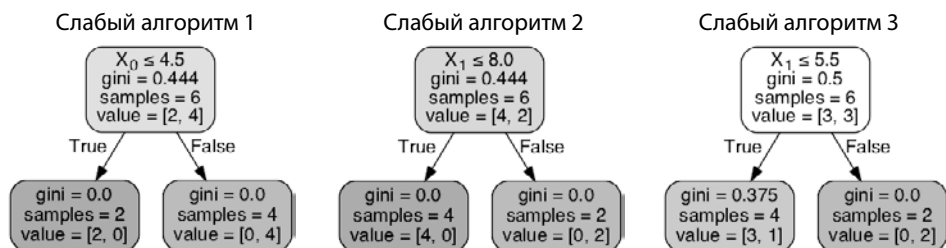


Рис. 12.5. Три слабых алгоритма, формирующих случайный лес, представляя собой деревья принятия решений глубиной 1. Каждое из них соответствует одному из трех подмножеств на рис. 12.4

Мы объединяем их в сильный обучающий алгоритм путем голосования. Другими словами, для любого ввода каждый из слабых алгоритмов прогнозирует значение 0 или 1. Прогноз, который делает сильный алгоритм, — это наиболее распространенный результат из трех. Эту комбинацию можно увидеть на рис. 12.6, где слабые алгоритмы расположены сверху, а сильные — внизу.

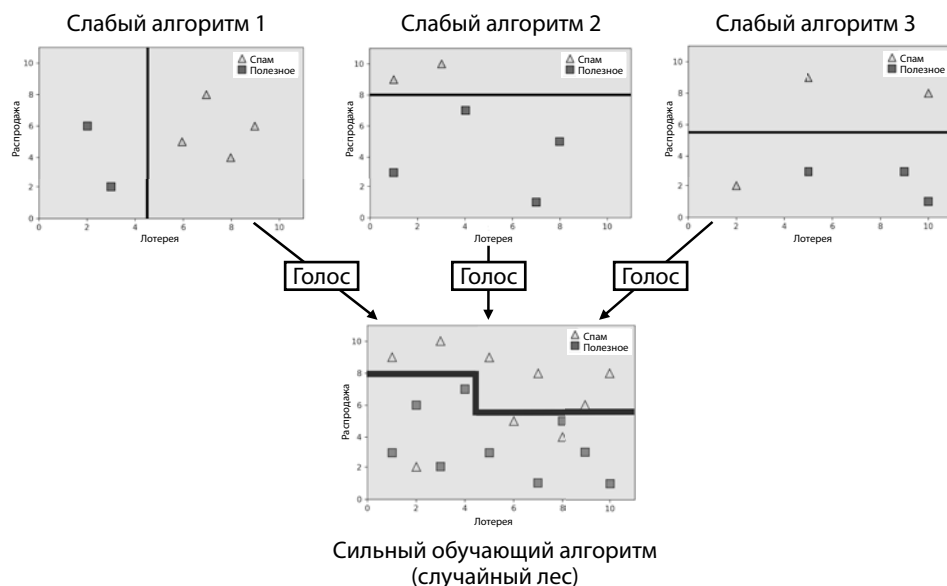


Рис. 12.6. Способ получения прогноза случайного леса заключается в объединении прогнозов трех слабых алгоритмов

Обратите внимание на то, что случайный лес является хорошим классификатором, поскольку правильно классифицирует большинство точек, но допускает несколько ошибок, чтобы не переобучиться. Однако нам не требуется обучать случайные леса вручную, потому что в Scikit-Learn для этого имеются функции, которые мы увидим в следующем разделе.

Обучение случайного леса в Scikit-Learn

В этом разделе рассмотрим, как обучить случайный лес с помощью Scikit-Learn. В следующем коде мы используем пакет `RandomForestClassifier`. У нас имеются данные в двух фреймах данных Pandas, называемых признаками и метками, как показано далее:

```
from sklearn.ensemble import RandomForestClassifier
random_forest_classifier = RandomForestClassifier(random_state=0,
        n_estimators=5, max_depth=1)
random_forest_classifier.fit(features, labels)
random_forest_classifier.score(features, labels)
```

В коде мы указали, что нам нужны пять слабых алгоритмов с гиперпараметром `n_estimators`. Эти слабые обучающие алгоритмы — деревья решений, и мы показали, что их глубина равна 1, с помощью гиперпараметра `max_depth`. График модели показан на рис. 12.7. Обратите внимание на то, что модель допускает некоторые ошибки, но ей удастся найти хорошую границу, где спам-письма — это те, в которых часто встречаются слова «лотерея» и «распродажа» (вверху справа от графика), а полезные — те, в которых они встречаются редко (внизу слева). Модель рассматривает две неправильно классифицированные точки как выбросы вместо того, чтобы попытаться классифицировать их правильно.

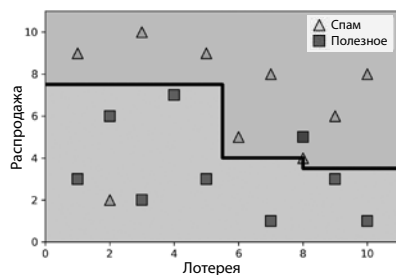
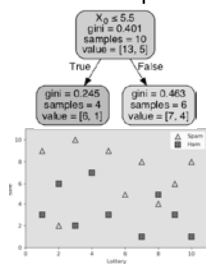


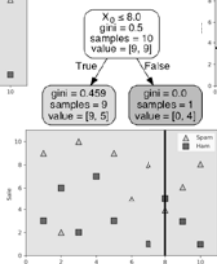
Рис. 12.7. Граница случайного леса, полученная с помощью Scikit-Learn

Scikit-Learn также позволяет нам визуализировать и отображать отдельные слабые алгоритмы (код см. в блокноте). Слабые алгоритмы показаны на рис. 12.8. Обратите внимание на то, что не все слабые обучающие алгоритмы полезны. Например, в первом случае каждая точка классифицируется как полезное письмо.

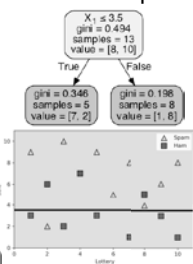
Слабый алгоритм 1



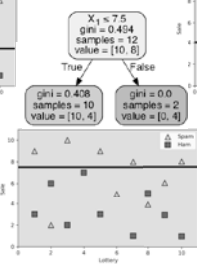
Слабый алгоритм 2



Слабый алгоритм 3



Слабый алгоритм 4



Слабый алгоритм 5

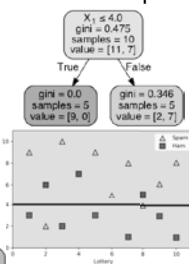


Рис. 12.8. Случайный лес сформирован пятью слабыми алгоритмами. Каждый из них представляет собой дерево решений глубиной 1. Они объединяются с целью формирования одного сильного, который показан на рис. 12.7

В этом разделе в качестве слабых обучающих алгоритмов мы использовали деревья решений глубины 1, но в целом они могут быть любой глубины. Попробуйте переподготовить эту модель, применяя деревья решений большей глубины, изменив гиперпараметр `max_depth`, и посмотрите, каким получится случайный лес.

AdaBoost. Разумное объединение слабых алгоритмов для формирования сильного

Бустинг похож на бэггинг в том смысле, что мы объединяем несколько слабых обучающих алгоритмов для создания сильного. Разница заключается в том, что слабые алгоритмы выбираются не случайным образом — они строятся путем концентрации на слабостях предыдущих обучающих алгоритмов. В этом разделе мы изучаем мощную технику бустинга под названием AdaBoost, разработанную Фрейндом и Шапиром в 1997 году (для справки см. приложение В). AdaBoost — это сокращение от *adaptive boosting*, эта техника подходит как для регрессии, так и для классификации. Однако мы будем использовать ее в примере классификации, который очень четко проиллюстрирует алгоритм обучения.

В AdaBoost, как и в случайных лесах, каждый слабый алгоритм представляет собой дерево решений глубиной 1. В отличие от случайных лесов, каждый слабый алгоритм обучается на всем наборе данных, а не на его части. Единственная оговорка заключается в том, что после обучения каждого слабого обучающего алгоритма мы модифицируем набор данных, увеличивая точки, которые были неправильно классифицированы, чтобы будущие слабые алгоритмы уделяли им больше внимания. В двух словах AdaBoost работает так, как описано далее.

Псевдокод для обучения модели AdaBoost

- Обучить первый слабый обучающий алгоритм на первом наборе данных.
- Повторить следующие шаги для каждого нового слабого алгоритма:
 - после обучения слабого алгоритма точки, которые неправильно классифицированы, увеличиваются;
 - новый слабый алгоритм обучается на измененном наборе данных.

В данном разделе мы более подробно рассмотрим этот псевдокод на примере. Используемый набор данных состоит из двух классов (треугольники и квадраты) и показан на рис. 12.9.

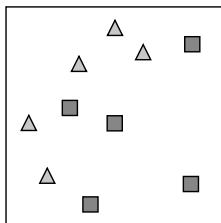


Рис. 12.9. Набор данных, который будем классифицировать с помощью AdaBoost

Общая картина AdaBoost. Построение слабых обучающих алгоритмов

В следующих двух разделах мы рассмотрим, как построить модель AdaBoost, соответствующую набору данных, показанному на рис. 12.9. Сначала создаем слабые алгоритмы, которые затем объединяем в один сильный.

Первый шаг — присвоение каждой из точек веса, равного 1 (рис. 12.10, *слева*). Затем на основе этого набора данных создаем слабый обучающий алгоритм. Напомним, что слабые алгоритмы — это деревья решений глубиной 1. Такое дерево решений соответствует горизонтальной или вертикальной прямой, которая наилучшим образом разделяет точки. С этой задачей справляются несколько таких деревьев, но мы выберем одно — вертикальную прямую, показанную на рис. 12.10 *в середине*. Оно правильно классифицирует два треугольника слева от него и пять квадратов справа и неправильно — три треугольника справа. Следующий шаг — увеличить три неправильно классифицированных пункта, чтобы придать им большее значение в глазах будущих слабых обучающих алгоритмов. Говоря об увеличении, напомним, что каждая точка изначально имеет вес 1. Мы определяем коэффициент масштабирования данного слабого алгоритма как количество правильно классифицированных точек, разделенное на количество неправильно классифицированных. В нашем случае коэффициент масштабирования $7/3 = 2,33$. Перейдем к масштабированию каждой неправильно классифицированной точки с помощью этого коэффициента масштабирования (рис. 12.10, *справа*).

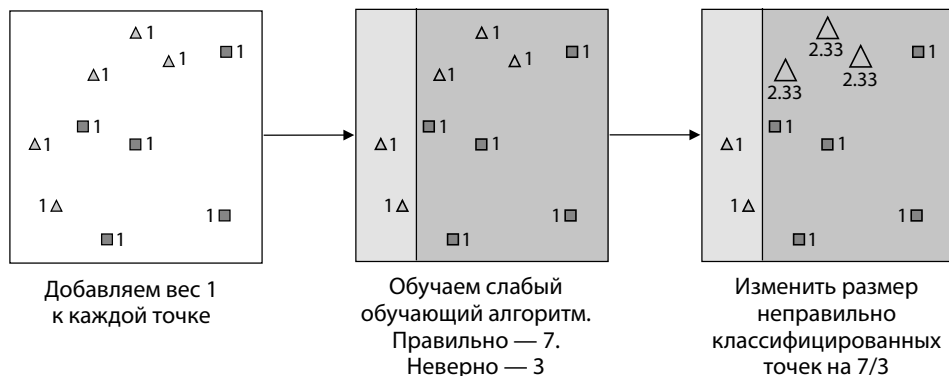


Рис. 12.10. Обучение первого слабого алгоритма модели AdaBoost

Теперь, когда у нас есть первый слабый обучающий алгоритм, создадим следующие таким же образом. Второй слабый алгоритм показан на рис. 12.11. Слева расположен масштабированный набор данных. Второй слабый обучающий алгоритм — тот, который лучше всего подходит для этого набора данных. Рассмотрим, что имеется в виду. Поскольку точки имеют разный вес, нам нужен слабый алгоритм, у которого сумма весов правильно классифицированных баллов наибольшая. Этот слабый обучающий алгоритм — горизонтальная прямая на среднем фрагменте рис. 12.11. Теперь перейдем к вычислению коэффициента

масштабирования. Нужно немного изменить его определение, потому что точки теперь имеют веса. Коэффициент масштабирования — это отношение между суммами весов правильно и неправильно классифицированных точек. Первая составляет $2,33 + 2,33 + 2,33 + 1 + 1 + 1 + 1 = 11$, вторая — $1 + 1 + 1 = 3$. Таким образом, коэффициент масштабирования $11/3 = 3,67$. Переходим к умножению весов трех неправильно классифицированных точек на коэффициент $3,67$ (рис. 12.11 *справа*).



Рис. 12.11. Пример второго слабого алгоритма модели AdaBoost

Процесс продолжается до тех пор, пока мы не построим столько слабых алгоритмов, сколько требуется. Для этого примера создаем три слабых обучающих алгоритма. Третий слабый алгоритм — это вертикальная прямая, изображенная на рис. 12.12. Слева — масштабированный набор данных с рис. 12.11. Справа — слабый обучающий алгоритм, который лучше всего подходит для масштабированного набора данных.

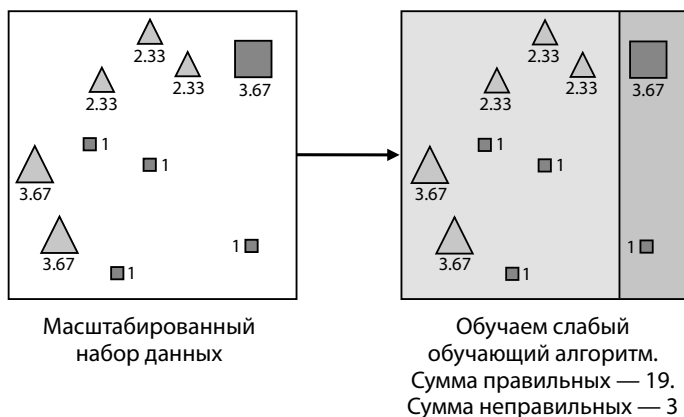


Рис. 12.12. Построение третьего слабого алгоритма модели AdaBoost

Именно так мы строим слабые алгоритмы. Теперь нужно объединить их в один сильный. Мы делали нечто похожее со случайными лесами, но теперь используем чуть больше математики, как показано в следующем разделе.

Объединение слабых обучающих алгоритмов в сильный

Теперь, когда мы создали слабые алгоритмы, узнаем эффективный способ объединить их в сильный. Суть состоит в том, чтобы заставить классификаторы голосовать, как это было в классификаторе случайных лесов, только на этот раз голос хороших обучающих алгоритмов имеет больший вес, чем плохих. Если классификатор по-настоящему плохой, его голос фактически будет отрицательным.

Чтобы понять это, представьте, что у нас есть три друга: правдивая Тереза, непредсказуемый Умберт и лживый Ленни. Правдивая Тереза почти всегда говорит правду, лживый Ленни почти всегда лжет, а непредсказуемый Умберт примерно в половине случаев говорит правду, а в другой половине лжет. Кто из этих трех друзей наименее полезен?

На мой взгляд, правдивая Тереза очень надежна, потому что почти всегда говорит правду. Иными словами, ей можно доверять. Из двух других я выберу лживого Ленни. Если он почти всегда лжет, отвечая «да» или «нет», мы просто принимаем за правду противоположное тому, что он говорит. И окажемся правы в большинстве случаев! А вот непредсказуемый Умберт совершенно бесполезен, если мы понятия не имеем, говорит он правду или лжет. Если бы нужно было присвоить оценку тому, что говорит каждый из них, я бы поставил правдивой Терезе высокий положительный балл, лживому Ленни — высокий отрицательный балл, а непредсказуемому Умберту — нулевой.

Теперь представьте, что наши трое друзей — слабые алгоритмы, обученные на наборе данных с двумя классами. Правдивая Тереза — классификатор с очень высокой достоверностью, лживый Ленни — классификатор с очень низкой, а непредсказуемый Умберт — классификатор с достоверностью, близкой к 50 %. Мы хотим создать сильный обучающий алгоритм, в котором прогноз будет получаться путем взвешенного голосования трех слабых алгоритмов. Таким образом, каждому из слабых алгоритмов мы присваиваем оценку, и именно такая доля голоса обучающего алгоритма будет учитываться при окончательном голосовании. Присваивать эти оценки мы будем так.

- Классификатор правдивая Тереза получает высокую положительную оценку.
- Оценка непредсказуемого Умберта близка к нулю.
- Лживый Ленни получает высокую отрицательную оценку.

Другими словами, оценка слабого алгоритма — это число, обладающее следующими свойствами.

1. Оно положительно, когда достоверность алгоритма превышает 0,5.
2. Оно равно 0, когда достоверность модели 0,5.
3. Оно отрицательно, когда достоверность алгоритма меньше 0,5.
4. Это большое положительное число, когда достоверность алгоритма близка к 1.
5. Это большое отрицательное число, когда достоверность алгоритма близка к 0.

Чтобы слабый обучающий алгоритм, который удовлетворяет данным свойствам, получил хорошую оценку, применим популярную концепцию вероятности, называемую логитом, или логарифмом отношения шансов, которую обсудим далее.

Вероятность, шансы и логарифмы отношения шансов

Возможно, вы знаете, что в азартных играх вероятности никогда не упоминаются, а речь всегда идет о шансах. Что же это за шансы? В некотором смысле они напоминают вероятность: если мы проводим эксперимент много раз и записываем, сколько раз получился конкретный результат, вероятность последнего равна количеству раз, когда он произошел, деленному на общее количество экспериментов. Вероятность такого исхода равна количеству раз, когда он произошел, деленному на количество раз, когда он не произошел.

Например, вероятность выпадения 1 при броске кубика равна $1/6$, но шансы — $1/5$. Если определенная лошадь выигрывает в трех из каждых четырех заездов, тогда вероятность ее победы в заезде равна $3/4$, а шансы — $3/1 = 3$. Формула для коэффициентов проста: если вероятность события равна x , то шансы равны $x/(1-x)$. Например, в примере с игрой в кости вероятность равна $1/6$, а шансы —

$$\frac{1/6}{1-1/6} = \frac{1}{5}.$$

Обратите внимание: поскольку вероятность — это число от 0 до 1, то шансы — это число от 0 до ∞ .

Теперь вернемся к первоначальной задаче. Мы ищем функцию, которая удовлетворяет приведенным ранее свойствам 1–5. Функция шансов почти подходит, но не совсем, так как выдает только положительные значения. Способ превратить шансы в функцию, удовлетворяющую этим свойствам, заключается в использовании логарифма. Таким образом, мы получаем логарифм отношения шансов, также называемый логитом:

$$\text{log-odds}(x) = \ln \frac{x}{1-x}.$$

На рис. 12.13 показан график логарифма отношения шансов $y = \ln \left(\frac{x}{1-x} \right)$. Обратите внимание на то, что эта функция удовлетворяет свойствам 1–5.

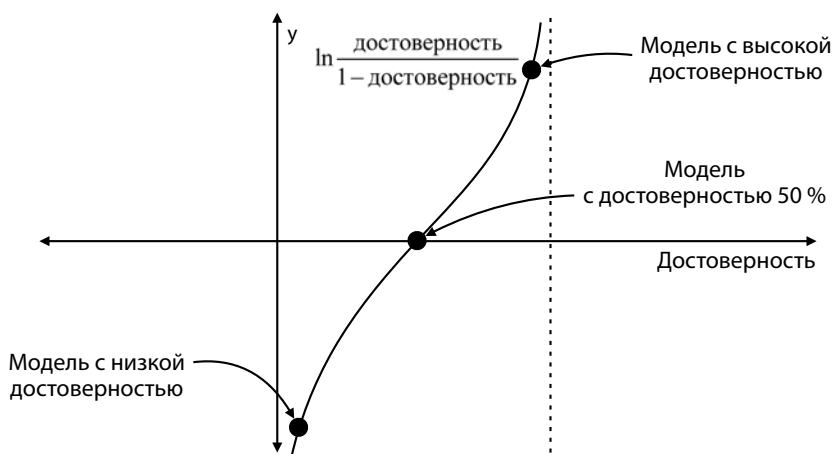


Рис. 12.13. Кривая — это график функции логарифма отношения шансов к достоверности

Поэтому нам нужно лишь использовать ее для вычисления оценки каждого из слабых алгоритмов. Мы применим логарифм отношения шансов к достоверности. В табл. 12.2 приведены несколько значений достоверности слабого алгоритма и логарифм отношения шансов к этой достоверности. Обратите внимание на то, что модели с высокой достоверностью имеют высокие положительные оценки, модели с низкой достоверностью — высокие отрицательные оценки, а модели с достоверностью, близкой к 0,5, — оценки, близкие к 0.

Таблица 12.2. Несколько значений достоверности слабого классификатора с соответствующей оценкой, рассчитанной с использованием логарифма отношения шансов

Достоверность	Логарифм отношения шансов (оценка слабого алгоритма)
0,01	–4,595
0,10	–2,197
0,20	–1,386
0,50	0,000
0,80	1,386
0,90	2,197
0,99	4,595

Объединение классификаторов

Теперь, когда мы приняли логарифм отношения шансов в качестве способа определения оценки всех слабых обучающих алгоритмов, можем соединить их,

чтобы создать сильный алгоритм. Напомним, что достоверность слабого алгоритма равна сумме оценок правильно классифицированных точек, деленной на сумму баллов всех оценок, как показано на рис. 12.10–12.12.

- Слабый алгоритм 1:
 - достоверность 7/10;
 - оценка $\ln(7/3) = 0,847$.
- Слабый алгоритм 2:
 - достоверность 11/14;
 - оценка $\ln(11/3) = 1,299$.
- Слабый алгоритм 3:
 - достоверность: 19/22;
 - оценка $\ln(19/3) = 1,846$.

Прогноз, который делает сильный обучающий алгоритм, получается путем взвешенного голосования слабых классификаторов, где голос каждого классификатора равен его оценке. Простой способ проверки — изменить предсказания слабых классификаторов с 0 и 1 на -1 и 1 , умножив каждый прогноз на оценку слабого алгоритма и сложив их. Если результирующий прогноз больше или равен нулю, то сильный классификатор прогнозирует 1, а если он отрицательный, то прогнозирует 0. Процесс голосования показан на рис. 12.14, а прогнозы — на рис. 12.15.

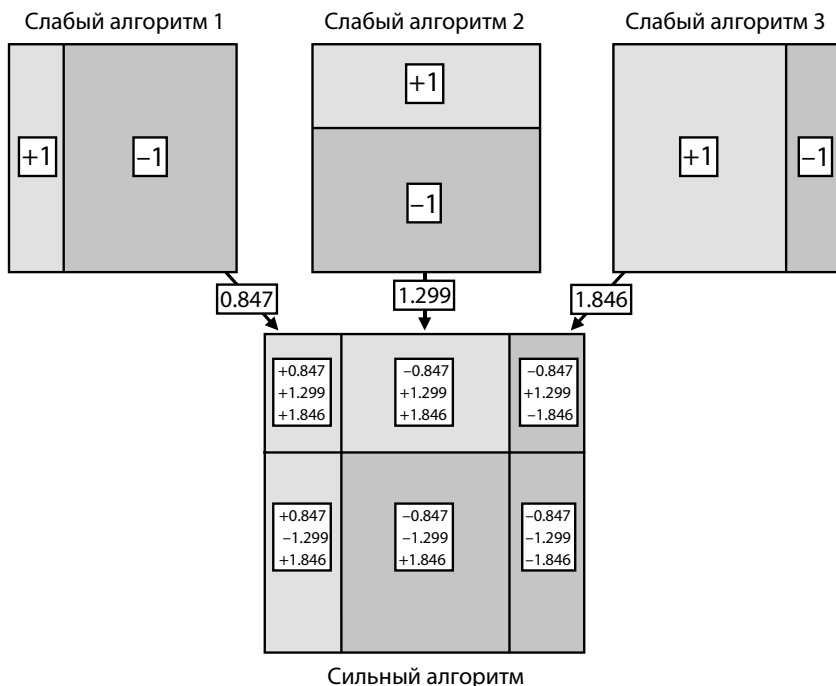


Рис. 12.14. Объединение слабых классификаторов в сильные в модели AdaBoost

Обратите внимание на то, что на рис. 12.15 результирующий классификатор правильно классифицировал каждую точку в наборе данных. После суммирования оценок, полученных от слабых классификаторов (см. рис. 12.14), мы присваиваем прогнозу значение 1, если сумма оценок больше или равна 0, и прогноз 0 в противном случае.

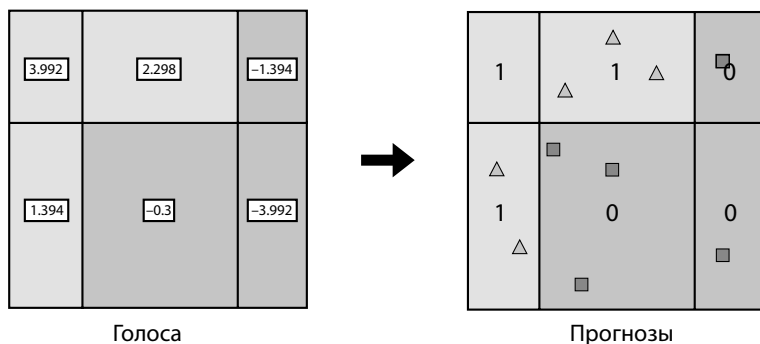


Рис. 12.15. Получение прогнозов для модели AdaBoost

Кодирование AdaBoost в Scikit-Learn

В этом разделе мы увидим, как использовать Scikit-Learn для обучения модели AdaBoost. Обучим ее на том же наборе данных нежелательной почты, который применялся в разделе «Обучение случайного леса вручную» и показан на рис. 12.16. Это тот же набор данных о спаме, что и в разделе «Бэггинг. Создание случайных наборов...», где признаком служит количество появлений слов «лотерея» и «спам». Спам-письма представлены треугольниками, а полезные — квадратами. Мы продолжаем использовать следующий блокнот из предыдущих разделов.

- **Блокнот** `Random_forests_and_AdaBoost.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_12_Ensemble_Methods/Random_forests_and_AdaBoost.ipynb.

Набор данных находится в двух фреймах данных Pandas, называемых признаками и метками. Обучение проводится с помощью пакета `AdaBoostClassifier` в Scikit-Learn. Мы указываем, что в этой модели будут использоваться шесть слабых обучающих алгоритмов, с помощью гиперпараметра `n_estimators`:

```
from sklearn.ensemble import AdaBoostClassifier
adaboost_classifier = AdaBoostClassifier(n_estimators=6)
adaboost_classifier.fit(features, labels)
adaboost_classifier.score(features, labels)
```

Граница результирующей модели показана на рис. 12.17. Обратите внимание на то, что классификатор хорошо подходит для набора данных и не слишком переобучен.

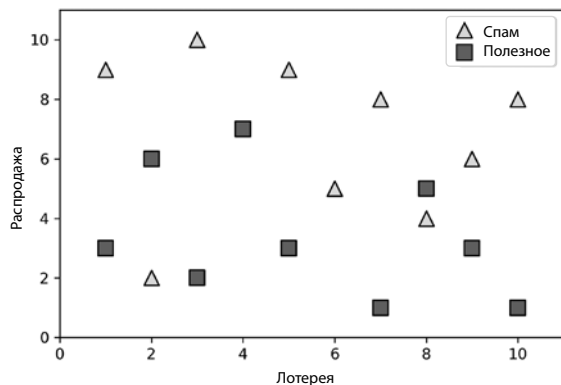


Рис. 12.16. На этом наборе данных обучаем классификатор AdaBoost с помощью Scikit-Learn

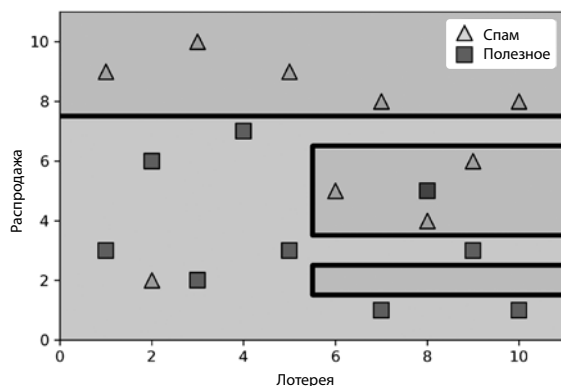


Рис. 12.17. Результат работы классификатора AdaBoost в наборе данных о спаме (см. рис. 12.16)

Мы можем пойти немного дальше и изучить шесть слабых алгоритмов (каждый представляет собой дерево решений глубиной 1) и их оценки (код см. в блокноте). Их границы отмечены на рис. 12.18, и, как видно из блокнота, оценки всех слабых классификаторов равны 1.

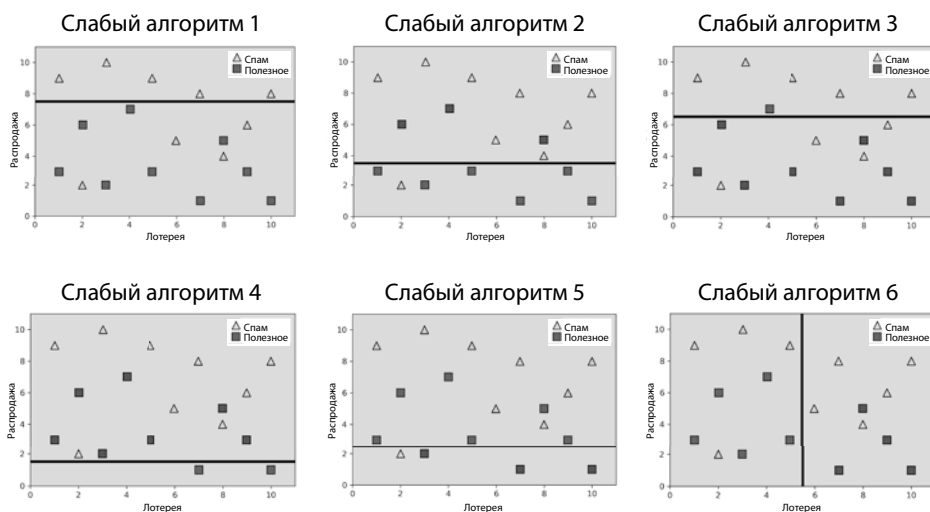


Рис. 12.18. Шесть слабых классификаторов в модели AdaBoost

Обратите внимание на то, что сильный алгоритм, изображенный на рис. 12.17, получается присвоением оценки 1 каждому из слабых алгоритмов, приведенных на рис. 12.18, и предоставлением им возможности проголосовать.

Градиентный бустинг. Использование деревьев решений для формирования сильных обучающих алгоритмов

В этом разделе мы обсудим градиентный бустинг — одну из самых популярных и успешных моделей машинного обучения современности. Градиентный бустинг похож на AdaBoost в том смысле, что слабые алгоритмы — это деревья решений и цель каждого слабого алгоритма — учиться на ошибках предшественников. Одно из различий градиентного бустинга и AdaBoost заключается в том, что при градиентном бустинге мы допускаем деревья решений глубиной более 1. Градиентный бустинг можно использовать как для регрессии, так и для классификации, но для наглядности рассмотрим пример регрессии. Чтобы задействовать его для классификации, потребуется внести лишь небольшие изменения. Узнать об этом подробнее можно, ознакомившись со ссылками на видео и материалы для чтения в приложении В. Код для этого раздела приведен далее.

- **Блокнот** Gradient_boosting_and_XGBoost.ipynb:
https://github.com/luisguiserrano/manning/blob/master/Chapter_12_Ensemble_Methods/Gradient_boosting_and_XGBoost.ipynb.

Мы рассмотрим тот же пример, что и в разделе «Деревья решений для регрессии» главы 9, в котором изучалась интенсивность применения приложения пользователями. Признак — это возраст пользователя, а метка — количество дней в неделю, в течение которых он взаимодействует с приложением (табл. 12.3). График набора данных показан на рис. 12.19. Обучение проведем с помощью градиентного бустинга.

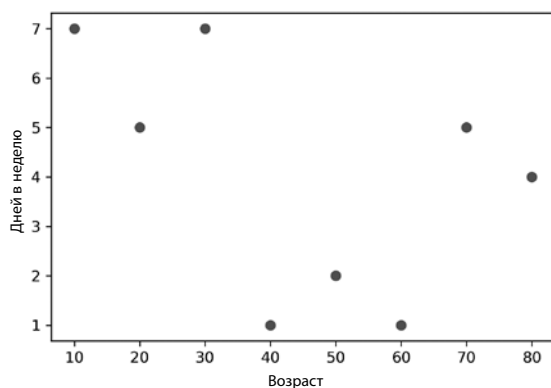


Рис. 12.19. График набора данных о вовлеченности пользователей из табл. 12.3

Таблица 12.3. Набор данных для восьми пользователей

Признак (возраст, лет)	Метка (вовлеченность, дней в неделю)
10	7
20	5
30	7
40	1
50	2
60	1
70	5
80	4

Суть градиентного бустинга состоит в том, что мы создаем последовательность деревьев, которые соответствуют набору данных. Используем два гиперпараметра: количество деревьев (установим равным 5) и скорость обучения (установим равной 0,8). Первый слабый классификатор прост — это дерево решений глубиной 0, которое наилучшим образом соответствует набору данных. Дерево решений глубиной 0 — это просто узел, который присваивает одну и ту же метку каждой точке в наборе данных. Поскольку функция ошибки, которую мы минимизируем, представляет собой среднюю

квадратическую ошибку, то это оптимальное значение прогноза является средним значением меток. Среднее значение меток этого набора данных равно 4, поэтому первый слабый алгоритм — это узел, который присваивает каждой точке прогноз, равный 4.

Следующий шаг — вычисление разности, которая представляет собой разницу между меткой и прогнозом, сделанным первым слабым классификатором, и подгонка нового дерева решений к этим разностям. Как можно видеть, происходит обучение дерева решений и заполнение пробелов, оставленных первым деревом. Метки, прогнозы и разности приведены в табл. 12.4.

Таблица 12.4. Прогнозы первого слабого алгоритма представляют собой среднее значение меток. Второй слабый алгоритм обучается так, чтобы соответствовать разностям первого слабого алгоритма

Признак (возраст, лет)	Метка (вовлеченность, дней в неделю)	Прогноз слабого алгоритма 1	Разность	Прогноз слабого алгоритма 2
10	7	4	3	3,000
20	5	4	2	2,000
30	7	4	3	2,000
40	1	4	-3	-2,667
50	2	4	-2	-2,667
60	1	4	-3	-2,667
70	5	4	1	0,500
80	4	4	0	0,500

Второй слабый алгоритм — это дерево, которое соответствует этим разностям. Оно может быть любой нужной глубины, в этом примере все слабые алгоритмы будут иметь глубину не более 2. Дерево показано на рис. 12.20 (вместе с его границей), а прогнозы приведены в крайнем правом столбце табл. 12.4. Получено дерево с помощью Scikit-Learn (процедуру см. в блокноте).

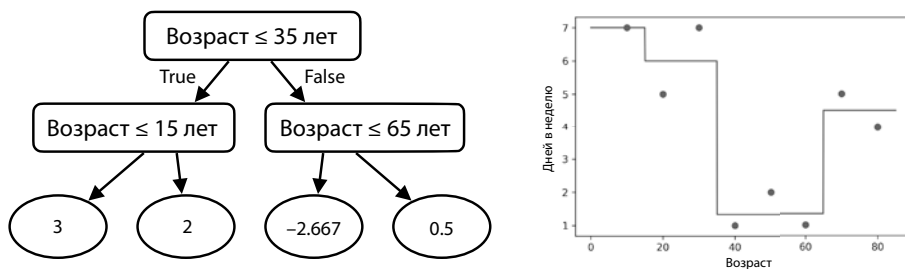


Рис. 12.20. Второй слабый алгоритм в модели градиентного бустинга: слева — дерево решений глубины 2; справа — прогноз слабого алгоритма

Далее мы продолжаем в том же духе, вычисляя новые разности и обучая новый слабый алгоритм, чтобы он соответствовал этим разностям. Однако есть небольшая тонкость: чтобы рассчитать прогноз первых двух слабых алгоритмов, мы сначала умножаем прогноз второго слабого алгоритма на скорость обучения. Напомним, что у нас скорость обучения равна 0,8. Таким образом, объединенный прогноз первых двух слабых алгоритмов равен прогнозу первого (4) плюс прогноз второго, умноженный на 0,8. Мы делаем это, потому что не хотим переобучить алгоритм, слишком хорошо обучив его на своих данных. Наша цель — имитировать алгоритм градиентного спуска, постепенно приближаясь к решению, и именно это обеспечивает умножение прогноза на скорость обучения. Новые разности — это исходные метки за вычетом объединенных прогнозов первых двух слабых алгоритмов. Они в табл. 12.5.

Таблица 12.5. Расчет новых разностей

Метка	Прогноз				Разность
	Слабого алгоритма 1	Слабого алгоритма 2	Слабого алгоритма 2, умноженный на скорость обучения	Слабых алгоритмов 1 и 2	
7	4	3,000	2,40	6,40	0,60
5	4	2,000	1,60	5,60	-0,60
7	4	2,000	1,60	5,60	1,40
1	4	-2,667	-2,13	1,87	-0,87
2	4	-2,667	-2,13	1,87	0,13
1	4	-2,667	-2,13	1,87	-0,87
5	4	0,500	0,40	4,40	0,60
4	4	0,500	0,40	4,40	-0,40

Теперь можем приступить к подгонке нового слабого алгоритма к новым разностям и вычислить объединенный прогноз первых двух слабых классификаторов. Мы добиваемся этого, складывая прогноз для первого слабого алгоритма и скорость обучения (0,8), умноженную на сумму прогнозов второго и третьего слабых алгоритмов. Повторяем этот процесс для каждого слабого алгоритма. Вместо того чтобы делать это вручную, мы можем использовать пакет GradientBoostingRegressor в Scikit-Learn (код находится в блокноте). Следующие несколько строк кода показывают, как обучать модель и делать прогнозы. Обратите внимание: мы установили глубину деревьев не более 2, количество деревьев — 5, а скорость обучения — 0,8. Для этого использованы гиперпараметры `max_depth`, `n_estimators` и `learning_rate`. Также обратите внимание на то, что, если нам нужно пять деревьев, мы должны присвоить гиперпараметру `n_estimators` значение 4, потому что первое дерево не учитывается:

```
from sklearn.ensemble import GradientBoostingRegressor
gradient_boosting_regressor = GradientBoostingRegressor(max_depth=2,
    n_estimators=4, learning_rate=0.8)
gradient_boosting_regressor.fit(features, labels)
gradient_boosting_regressor.predict(features)
```

График результирующего сильного алгоритма показан на рис. 12.21. Он хорошо справляется с предсказанием значений. Модель довольно хорошо соответствует набору данных.

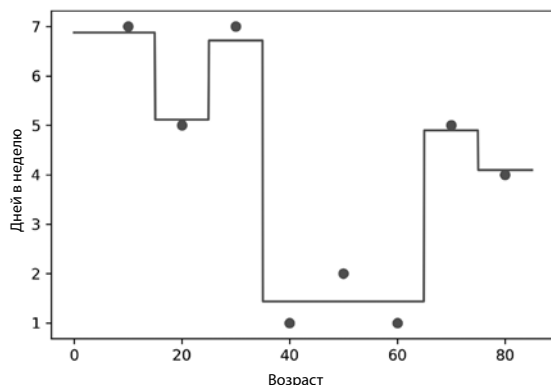


Рис. 12.21. График прогнозов сильного алгоритма в регрессоре градиентного бустинга

Однако мы можем пойти чуть дальше и построить график пяти полученных слабых алгоритмов. Подробная информация об этом приведена в блокноте, а пять слабых алгоритмов показаны на рис. 12.22. Первый — это дерево решений глубиной 0, которое всегда предсказывает среднее значение меток. Каждый последующий слабый классификатор представляет собой дерево решений глубиной не более 2, которое соответствует разностям от прогноза, данного предыдущими слабыми алгоритмами. Обратите внимание на то, что прогнозы последних слабых классификаторов намного меньше, чем первых, потому что каждый слабый алгоритм прогнозирует ошибку предыдущих и с каждым шагом эти ошибки становятся все меньше и меньше.

Наконец, мы можем использовать Scikit-Learn или выполнить расчеты вручную и убедиться, что прогнозы получаются следующие:

- возраст 10 лет, прогноз 6,87;
- возраст 20 лет, прогноз 5,11;
- возраст 30 лет, прогноз 6,71;
- возраст 40 лет, прогноз 1,43;
- возраст 50 лет, прогноз 1,43;
- возраст 60 лет, прогноз 1,43;
- возраст 70 лет, прогноз 4,90;
- возраст 80 лет, прогноз 4,1.

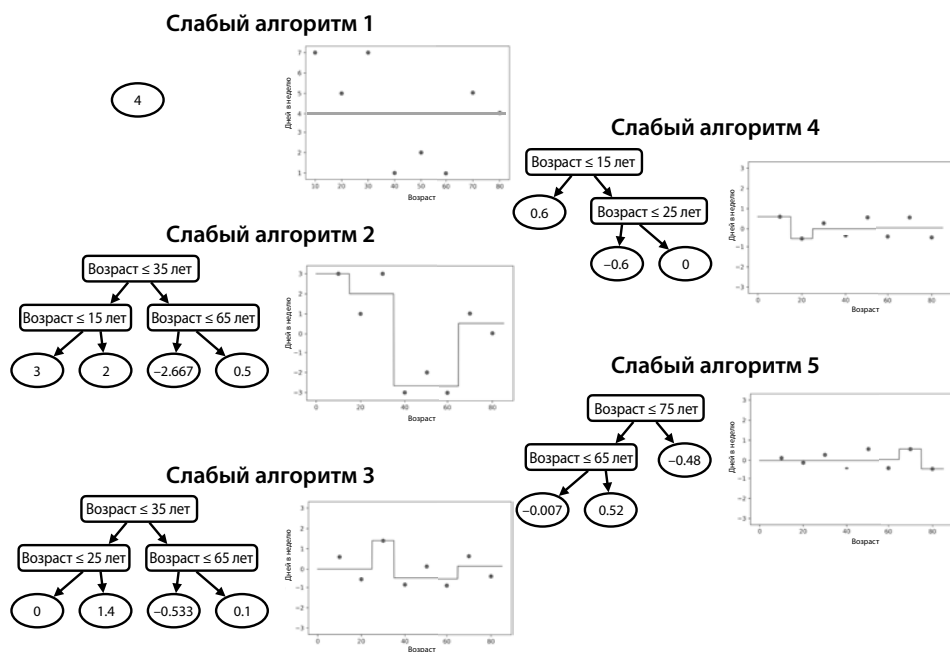


Рис. 12.22. Пять слабых алгоритмов в модели градиентного бустинга

XGBoost — экстремальный метод градиентного бустинга

XGBoost, что расшифровывается как *extreme gradient boosting*, — одна из самых популярных, мощных и эффективных реализаций градиентного бустинга. Созданные Тяньци Ченом и Карлосом Гестрином в 2016 году (для справки см. приложение В), модели XGBoost часто превосходят другие модели классификации и регрессии. В этом разделе мы обсудим, как работает XGBoost, с помощью примера регрессии из раздела «Градиентный бустинг. Использование деревьев решений для формирования сильных обучающих алгоритмов».

XGBoost применяет деревья решений в качестве слабых алгоритмов, и, как и в изученных ранее методах бустинга, каждый слабый алгоритм спроектирован так, чтобы принимать в расчет слабости предыдущих. Более конкретно, каждое дерево строится так, чтобы соответствовать разностям прогнозов предыдущих деревьев. Однако есть и кое-какие различия. Например, при построении деревьев используется показатель, называемый *показателем подобия*. Кроме того, для предотвращения переобучения мы добавляем этап подрезки, при котором удаляем ветви деревьев, если они не удовлетворяют определенным условиям. В данном разделе рассмотрим этот процесс подробнее.

Показатель подобия XGBoost — новый эффективный способ измерения подобия в наборе

В этом подразделе мы рассмотрим основной элемент XGBoost, который представляет собой способ измерения того, насколько схожи элементы набора. Он удачно назван показателем подобия. Прежде чем перейти к нему, выполним небольшое упражнение. Какой среди следующих трех наборов имеет наибольшее подобие, а какой — наименьшее?

- **Набор 1** {10, -10, 4}.
- **Набор 2** {7, 7, 7}.
- **Набор 3** {7}.

Если вы ответили, что набор 2 имеет наибольшее подобие, а набор 1 — наименьшее, то ваше предположение верно. В наборе 1 элементы сильно отличаются друг от друга, поэтому он имеет наименьшее подобие. Между наборами 2 и 3 выбор не так очевиден, так как оба содержат один и тот же элемент, но в разном количестве. В наборе 2 число 7 встречается три раза, а в наборе 3 — лишь один раз. Следовательно, в наборе 2 элементы более однородны или более подобны, чем в наборе 3.

Чтобы количественно оценить подобие, рассмотрим следующий показатель. Учитывая множество $\{a_1, a_2, \dots, a_n\}$, показатель подобия представляет собой квадрат суммы элементов, деленный на количество элементов, а именно:

$$\frac{(a_1 + a_2 + \dots + a_n)^2}{n}$$

Давайте рассчитаем показатель подобия для трех приведенных ранее наборов, как показано далее.

- **Набор 1** — показатель подобия $= \frac{(10 - 10 + 4)^2}{3} = 5,33$.
- **Набор 2** — показатель подобия $= \frac{(7 + 7 + 7)^2}{3} = 147$.
- **Набор 3** — показатель подобия $= \frac{7^2}{1} = 49$.

Обратите внимание: как и ожидалось, показатель подобия набора 2 получился самым высоким, а набора 1 — самым низким.

ОБРАТИТЕ ВНИМАНИЕ Данный показатель подобия неидеален. Можно утверждать, что множество {1, 1, 1} более подобно, чем множество {7, 8, 9}, однако показатель подобия {1, 1, 1} равен 3, а показатель подобия {7, 8, 9} — 192. Однако для нашего алгоритма эта оценка вполне подойдет. Основная цель показателя подобия — иметь возможность хорошо разделять большие и малые значения, и, как мы увидим в текущем примере, эта цель достигается.

Существует гиперпараметр λ , связанный с показателем подобия, который помогает предотвратить переобучение. При использовании он добавляется к знаменателю показателя подобия, что дает формулу

$$\frac{(a_1 + a_2 + \dots + a_n)^2}{n + \lambda}.$$

Таким образом, если $\lambda = 2$, то показатель подобия набора 1 теперь составляет:

$$\frac{(10 - 10 + 4)^2}{3 + 2} = 3,2.$$

Мы не будем использовать гиперпараметр λ в своем примере, но когда доберемся до кода, то увидим, как установить для него любое нужное значение.

Построение слабых классификаторов

В этом разделе мы увидим, как построить каждый слабый классификатор. Чтобы проиллюстрировать этот процесс, возьмем пример из раздела «Градиентный бустинг», приведенный в табл. 12.3. Для удобства этот набор данных пользователей приложения, в котором признаком служит их возраст, а меткой — количество дней в неделю, когда они взаимодействуют с приложением, показан в двух крайних левых столбцах табл. 12.6. Третий столбец содержит прогнозы первого слабого алгоритма в модели XGBoost. Все они по умолчанию равны 0,5. Последний столбец содержит разность, которая представляет собой разницу между меткой и прогнозом. График этого набора данных показан на рис. 12.19.

Таблица 12.6. Тот же набор данных, что и в табл. 12.3

Признак (возраст, лет)	Метка (вовлеченность, дней в неделю)	Прогноз от первого слабого алгоритма	Разность
10	7	0,5	6,5
20	5	0,5	4,5
30	7	0,5	6,5
40	1	0,5	0,5
50	2	0,5	1,5
60	1	0,5	0,5
70	5	0,5	4,5
80	4	0,5	3,5

Процесс обучения модели XGBoost аналогичен процессу обучения деревьев градиентного бустинга. Первый слабый алгоритм — это дерево, которое дает прогноз 0,5 для каждой точки данных. После его построения мы вычисляем разности, которые представляют собой различия между меткой и спрогнозированной меткой. Эти две величины можно увидеть в двух крайних правых столбцах табл. 12.6.

Прежде чем начать строить оставшиеся деревья, давайте решим, какую глубину им зададим. Чтобы не перегружать данный пример, давайте снова используем максимальную глубину 2. Это означает, что, добравшись до глубины 2, мы перестанем строить слабые классификаторы. Это гиперпараметр, который будет подробнее рассмотрен в разделе «Обучение модели XGBoost на Python».

Чтобы построить второй слабый алгоритм, нужно обучить дерево решений на разностях. Мы делаем это с помощью показателя подобия. Как обычно, в корневом узле содержится весь набор данных. Таким образом, мы начинаем с вычисления показателя подобия для всего набора данных:

$$\text{подобие} = \frac{(6,5 + 4,5 + 6,5 + 0,5 + 1,5 + 0,5 + 4,5 + 3,5)^2}{8} = 98.$$

Далее приступаем к разделению узла с использованием признака возраста всеми возможными способами, как мы это делали с деревьями решений. Для каждого разделения вычисляем показатель подобия подмножеств, соответствующих каждому из листьев, и складываем их. Это совокупный показатель подобия, соответствующий данному разделению. Показатели приведены далее.

Разделение для корневого узла с набором данных {6,5; 4,5; 6,5; 0,5; 1,5; 0,5; 4,5; 3,5} и показателем подобия 98:

- разделение на 15:
 - левый узел {6,5}, показатель подобия 42,25;
 - правый узел {4,5; 6,5; 0,5; 1,5; 0,5; 4,5; 3,5}, показатель подобия 66,04;
 - суммарный показатель подобия 108,29;
- разделение на 25:
 - левый узел {6,5; 4,5}, показатель подобия 60,5;
 - правый узел {6,5; 0,5; 1,5; 0,5; 4,5; 3,5}, показатель подобия 48,17;
 - суммарный показатель подобия 108,67;
- разделение на 35:
 - левый узел {6,5; 4,5; 6,5}, показатель подобия 102,08;
 - правый узел {0,5; 1,5; 0,5; 4,5; 3,5}, показатель подобия 22,05;
 - **суммарный показатель подобия 124,13;**
- разделение на 45:
 - левый узел {6,5; 4,5; 6,5; 0,5}, показатель подобия 81;
 - правый узел {1,5; 0,5; 4,5; 3,5}, показатель подобия 25;
 - суммарный показатель подобия 106;
- разделение на 55:
 - левый узел {6,5; 4,5; 6,5; 0,5; 1,5}, показатель подобия 76,05;
 - правый узел {0,5; 4,5; 3,5}, показатель подобия 24,08;
 - суммарный показатель подобия 100,13;
- разделение на 65:
 - левый узел {6,5; 4,5; 6,5; 0,5; 1,5; 0,5}, показатель подобия 66,67;
 - правый узел {4,5; 3,5}, показатель подобия 32;
 - суммарный показатель подобия 98,67;

- разделение на 75:
 - левый узел {6,5; 4,5; 6,5; 0,5; 1,5; 0,5; 4,5}, показатель подобия 85,75;
 - правый узел {3,5}, показатель подобия 12,25;
 - суммарный показатель подобия 98.

Как показано в этих расчетах, разделение с наилучшим совокупным показателем подобия происходит на возрасте 35 лет. Это будет разделение на корневом узле.

Далее мы приступаем к такому же разделению наборов данных на каждом из узлов. Разделение для левого узла с набором данных {6,5; 4,5; 6,5} и показателем подобия 102,08:

- разделение на 15:
 - левый узел {6,5}, показатель подобия 42,25;
 - правый узел {4,5; 6,5}, показатель подобия 60,5;
 - показатель подобия 102,75;
- разделение на 25:
 - левый узел {6,5; 4,5}, показатель подобия 60,5;
 - правый узел {6,5}, показатель подобия 42,25;
 - показатель подобия 102,75.

Оба разделения дают одинаковый суммарный показатель подобия, поэтому мы можем взять любой. Давайте воспользуемся разделением на 15. Теперь перейдем к правому узлу.

Разделение для правого узла с набором данных {0,5; 1,5; 0,5; 4,5; 3,5} и показателем подобия 22,05:

- разделение на 45:
 - левый узел {0,5}, показатель подобия 0,25;
 - правый узел {1,5; 0,5; 4,5; 3,5}, показатель подобия 25;
 - показатель подобия 25,25;
- разделение на 55:
 - левый узел {0,5; 1,5}, показатель подобия 2;
 - правый узел {0,5; 4,5; 3,5}, показатель подобия 24,08;
 - показатель подобия 26,08;
- разделение на 65:
 - левый узел {0,5; 1,5; 0,5}, показатель подобия 2,08;
 - правый узел {4,5; 3,5}, показатель подобия 32;
 - **показатель подобия 34,08;**
- разделение на 75:
 - левый узел {0,5; 1,5; 0,5; 4,5}, показатель подобия 12,25;
 - правый узел {3,5}, показатель подобия 12,25;
 - показатель подобия 24,5.

Отсюда делаем вывод, что лучшее разделение — на возрасте 65 лет. Теперь дерево имеет глубину 2, поэтому прекращаем его строить (мы так решили в начале алгоритма). Результирующее дерево вместе с показателями подобия в узлах показано на рис. 12.23. Для каждого из узлов показано разделение на основе возрастного признака, меток, соответствующих этому узлу, и показателя подобия для каждого набора меток. Разделение, выбранное для каждого узла, максимизирует совокупный показатель подобия листьев. Для каждого из листьев показаны соответствующие метки и их показатель подобия.

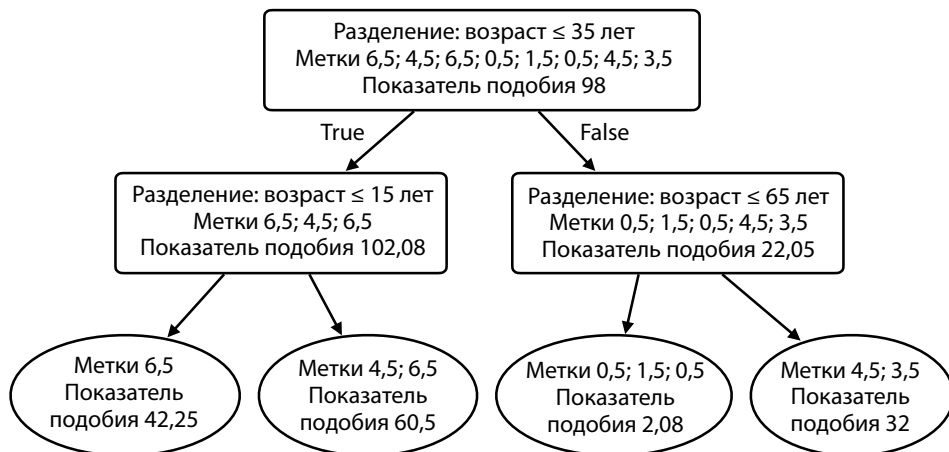


Рис. 12.23. Второй слабый алгоритм в классификаторе XGBoost

Это (почти) второй слабый алгоритм. Прежде чем продолжить создавать слабые алгоритмы, нам нужно сделать еще один шаг, чтобы уменьшить переобучение.

Подрезка деревьев — способ уменьшить переобучение за счет упрощения слабых алгоритмов

Отличительной особенностью XGBoost является то, что он не слишком переобучается. Для этого используются несколько гиперпараметров, которые подробно описаны в разделе «Обучение модели XGBoost на Python». Один из них, минимальная потеря при разделении, предотвращает разделение, если совокупные показатели подобия результирующих узлов незначительно превышают показатель подобия исходного. Это различие называется приростом подобия. Например, в корневом узле нашего дерева показатель подобия равен 98, а общий показатель подобия узлов — 124,13. Таким образом, прирост подобия составляет $124,13 - 98 = 26,13$. Аналогично прирост подобия левого узла равен 0,67, а прирост подобия правого — 12,03, как показано на рис. 12.24. Мы разрешаем только разбиения с приростом подобия выше 1 (гиперпараметр минимальных потерь при разбиении), поэтому одно из разбиений оказывается недопустимым. Результат — подрезанное дерево справа, которое теперь и становится слабым алгоритмом.

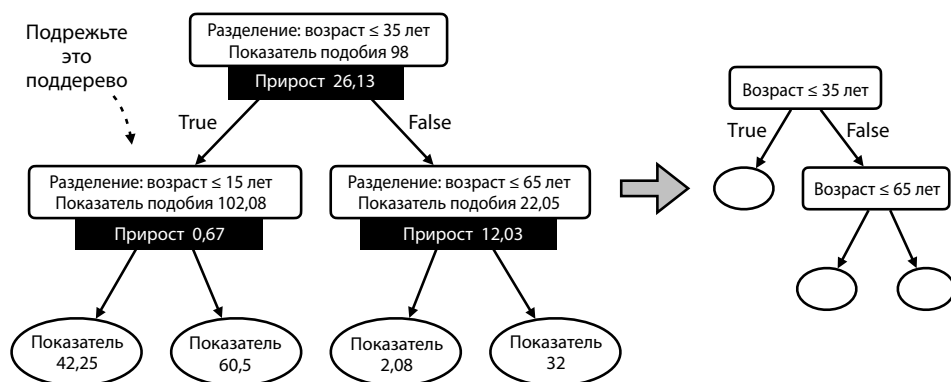


Рис. 12.24. Слева то же дерево, что и на рис. 12.23, с дополнительной информацией — приростом подобия

Установим минимальную потерю разделения равной 1. При этом значении предотвращается только разделение на левом узле (возраст ≤ 15 лет). Таким образом, второй слабый алгоритм выглядит так, как показано на рис. 12.24, *справа*.

Составление прогнозов

Теперь, когда создан второй слабый алгоритм, пришло время использовать его для прогнозов. Мы получаем предсказания так же, как и для любого другого дерева решений, а именно усреднением меток в соответствующем листе. Прогнозы второго слабого алгоритма отражены на рис. 12.25.

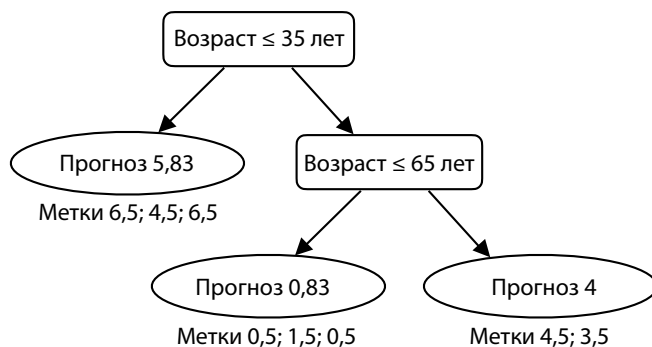


Рис. 12.25. Второй слабый алгоритм в модели XGBoost после подрезки. Это то же самое дерево, что и на рис. 12.24, с прогнозами

Теперь перейдем к вычислению совокупного прогноза для первых двух слабых алгоритмов. Чтобы избежать переобучения, используем ту же технику, что и при градиентном бустинге, которая заключается в умножении прогноза всех

слабых алгоритмов, кроме первого, на скорость обучения. Этим мы имитируем метод градиентного спуска, в котором медленно сходимся к хорошему прогнозу после нескольких итераций. Применим коэффициент обучения 0,7. Таким образом, объединенный прогноз первых двух слабых алгоритмов равен сумме прогноза первого слабого алгоритма (всегда равен 0,5) и прогноза второго, умноженного на 0,7. Например, для первой точки данных прогноз равен:

$$0,5 + 5,83 \times 0,7 = 4,58.$$

Пятая колонка табл. 12.7 содержит объединенный прогноз первых двух слабых алгоритмов. Разность представляет собой разницу между меткой и объединенным прогнозом.

Таблица 12.7. Метки, прогнозы первых двух слабых алгоритмов и разности

Метка (вовлеченность, дней в неделю)	Прогноз				Разность
	Слабого алгоритма 1	Слабого алгоритма 2	Слабого алгоритма 2, умноженный на скорость обучения	Объединенный	
7	0,5	5,83	4,08	4,58	2,42
5	0,5	5,83	4,08	4,58	0,42
7	0,5	5,83	4,08	4,58	2,42
1	0,5	0,83	0,58	1,08	−0,08
2	0,5	0,83	0,58	1,08	0,92
1	0,5	0,83	0,58	1,08	−0,08
5	0,5	4,00	2,80	3,30	1,70
4	0,5	4,00	2,80	3,30	0,70

Обратите внимание на то, что объединенные прогнозы ближе к меткам, чем прогнозы первого слабого алгоритма. Следующий шаг — повторение. Мы вычисляем новые разности для всех точек данных, подгоняем к ним дерево, подрезаем его, вычисляем новые объединенные прогнозы и продолжаем в том же духе. Количество деревьев — это еще один гиперпараметр, который можно указать в самом начале. Чтобы продолжить построение этих деревьев, используем пакет Python под названием xgboost.

Обучение модели XGBoost на Python

В этом разделе мы узнаем, как обучить модель, чтобы она соответствовала текущему набору данных, используя пакет xgboost Python. Код для этого раздела находится в том же блокноте, что и предыдущий.

- **Блокнот** Gradient_boosting_and_XGBoost.ipynb:
https://github.com/luiguiserrano/manning/blob/master/Chapter_12_Ensemble_Methods/Gradient_boosting_and_XGBoost.ipynb.

Прежде чем начать, еще раз пройдемся по гиперпараметрам, которые мы определили для этой модели.

ЧИСЛО ВЫЧИСЛИТЕЛЕЙ — количество слабых алгоритмов. (Примечание: в пакете `xgboost` первый слабый алгоритм не учитывается. Для этого примера установим его равным 3, что даст четыре слабых алгоритма.)

МАКСИМАЛЬНАЯ ГЛУБИНА — максимальная глубина, допустимая для каждого из деревьев решений (слабых алгоритмов). Устанавливаем его равным 2.

ЛЯМБДА-ПАРАМЕТР — число, добавляемое к знаменателю показателя подобия. Устанавливаем его равным 0.

МИНИМАЛЬНАЯ ПОТЕРЯ РАЗДЕЛЕНИЯ — минимальный прирост подобия, позволяющий выполнить разделение. Устанавливаем его равным 1.

СКОРОСТЬ ОБУЧЕНИЯ — прогнозы слабых алгоритмов от второго до последнего умножаются на скорость обучения. Устанавливаем его равным 0,7.

С помощью следующих строк кода мы импортируем пакет, создаем модель под названием `XGBRegressor` и обучаем ее на своем наборе данных:

```
import xgboost
from xgboost import XGBRegressor
xgboost_regressor = XGBRegressor(random_state=0,
                                  n_estimators=3,
                                  max_depth=2,
                                  reg_lambda=0,
                                  min_split_loss=1,
                                  learning_rate=0.7)
xgboost_regressor.fit(features, labels)
```

График модели показан на рис. 12.26. Обратите внимание на то, что он хорошо соответствует набору данных.

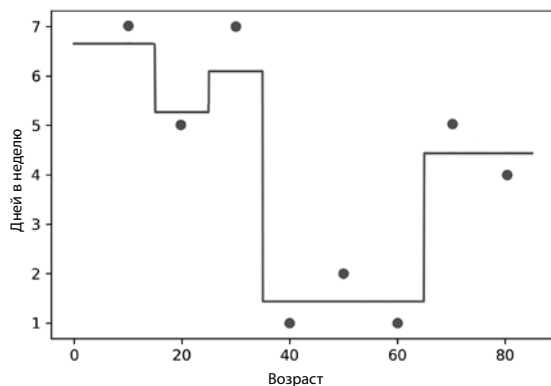


Рис. 12.26. График прогнозов модели XGBoost

Пакет `xgboost` также позволяет взглянуть на слабые алгоритмы (они показаны на рис. 12.24). Деревья, полученные таким образом, уже содержат метки, умноженные на скорость обучения 0,7, что становится очевидным при сравнении с прогнозами дерева, полученными вручную на рис. 12.25, и второго дерева слева на рис. 12.27. Обратите внимание на то, что первый алгоритм всегда прогнозирует значение 0,5. Остальные три очень похожи по форме, но это совпадение. Однако предсказания каждого из деревьев становятся меньше, потому что каждый раз мы обучаемся на меньших разностях.

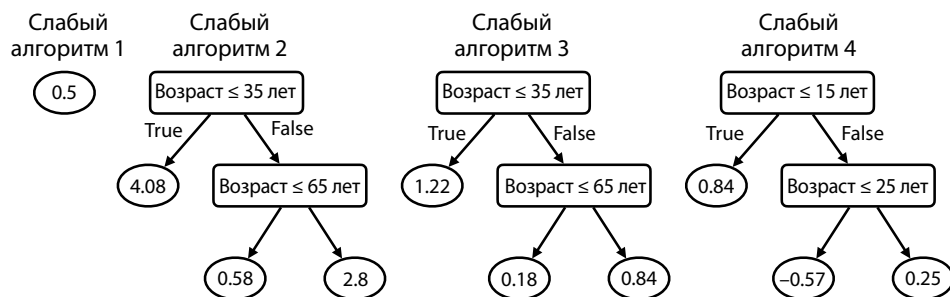


Рис. 12.27. Четыре слабых алгоритма, которые формируют сильный классификатор в модели XGBoost

Таким образом, чтобы получить прогнозы сильного алгоритма, нужно сложить только прогнозы каждого дерева. Например, для пользователя, которому 20 лет, прогнозы выглядят так:

- слабый алгоритм 1 — 0,5;
- слабый алгоритм 2 — 4,08;
- слабый алгоритм 3 — 1,22;
- слабый алгоритм 4 — -0,57.

Таким образом, прогноз равен $0,5 + 5,83 + 1,22 - 0,57 = 5,23$. Далее следуют прогнозы для других точек:

- возраст 10 лет, прогноз 6,64;
- возраст 20 лет, прогноз 5,23;
- возраст 30 лет, прогноз 6,05;
- возраст 40 лет, прогноз 1,51;
- возраст 50 лет, прогноз 1,51;
- возраст 60 лет, прогноз 1,51;
- возраст 70 лет, прогноз 4,39;
- возраст 80 лет, прогноз 4,39.

Применение методов ансамбля

Методы ансамбля — одни из наиболее полезных методов машинного обучения, используемых в настоящее время. Они демонстрируют высокий уровень производительности при относительно низкой стоимости. Одна из областей, где методы ансамбля применяются чаще всего, — это задачи машинного обучения, такие как Netflix Challenge. Netflix Challenge — это соревнование, организованное Netflix, в котором они анонимизировали некоторые данные и обнародовали их. Цель участников состояла в том, чтобы создать лучшую систему рекомендаций, чем у самого Netflix. Ее авторы выиграли 1 млн долларов. Команда-победитель использовала мощную комбинацию алгоритмов в ансамбле. Чтобы узнать подробности, ознакомьтесь с информацией, приведенной в приложении В.

Резюме

- Методы ансамбля состоят в обучении нескольких слабых алгоритмов и объединении их в сильный. Это эффективный способ построения мощных моделей, которые дают отличные результаты в процессе работы с реальными наборами данных.
- Методы ансамбля могут быть использованы как для регрессии, так и для классификации.
- Существуют два основных типа методов ансамбля — бэггинг и бустинг.
- Бэггинг состоит в построении последовательных алгоритмов на случайных подмножествах данных и объединении их в сильный алгоритм, который делает прогнозы на основе голосования.
- Бустинг заключается в построении последовательности алгоритмов, где каждый следующий фокусируется на слабостях предыдущего, и объединении их в сильный классификатор, который делает прогнозы на основе взвешенного голосования.
- AdaBoost, градиентный бустинг и XGBoost — это три продвинутых алгоритма бустинга, которые показывают отличные результаты на реальных наборах данных.
- Области применения методов ансамбля варьируются довольно широко — от рекомендательных алгоритмов до применения в медицине и биологии.

Упражнения

Упражнение 12.1

Сильный алгоритм бустинга L формируется из трех слабых, L_1 , L_2 и L_3 . Их веса равны 1; 0,4 и 1,2 соответственно. L_1 и L_2 прогнозируют, что метка конкретной точки положительна, а L_3 — что отрицательна. Каков окончательный прогноз, который сделает алгоритм L для этой точки?

Упражнение 12.2

Мы находимся в процессе обучения модели AdaBoost на наборе данных размером 100. Текущий слабый алгоритм правильно классифицирует 68 из 100 точек данных. Какой вес мы присвоим этому алгоритму в окончательной модели?

Применение изученного на практике. Реальный пример разработки данных и машинного обучения

13



В этой главе

- ✓ Очистка и предварительная обработка данных, делающая их удобочитаемыми для нашей модели.
- ✓ Использование Scikit-Learn для обучения и оценки нескольких моделей.
- ✓ Применение поиска по сетке для выбора подходящих гиперпараметров для модели.
- ✓ Применение k -кратной перекрестной проверки для получения возможности использовать данные для обучения и проверки одновременно.



Мы изучили ряд наиболее важных алгоритмов контролируемого обучения, и у нас была возможность закодировать их и использовать для прогнозирования на нескольких наборах данных. Однако процесс обучения модели на реальных данных требует нескольких дополнительных шагов, именно их мы обсудим в этой главе.

Одна из фундаментальных задач, стоящих перед специалистом по обработке данных, — очистка и предварительная обработка данных. Это крайне важно, потому что компьютер не в силах полностью справиться с ней. Чтобы правильно очистить данные, необходимо хорошо разбираться как в них самих, так и в решаемой проблеме. В этой главе мы рассмотрим некоторые из наиболее эффективных методов очистки и предварительной обработки данных. Затем более внимательно изучим признаки и применим конструирование признаков, что подготовит их для модели. В качестве следующего шага мы разделим модель на обучающий, проверочный и тестовый наборы, обучим несколько моделей на своем наборе данных и оценим их. Таким образом можно выбрать наиболее эффективную модель для набора данных. Наконец, мы изучим важные методы (например, поиск по сетке), чтобы найти наилучшие гиперпараметры для модели.

Далее выполним все эти шаги по отношению к популярному набору данных для изучения и применения методов машинного обучения, который называется «Титаник». На нем мы подробно остановимся в следующем разделе. В этой главе много кода. Два пакета Python, которые здесь использованы, — это Pandas и Scikit-Learn. Мы постоянно применяли их на протяжении всей книги. Пакет Pandas хорош для обработки данных, включая открытие файлов, загрузку данных и организацию их в виде таблиц, называемых фреймами данных. Пакет Scikit-Learn отлично подходит для обучения и оценки моделей, а также содержит надежные реализации большинства алгоритмов, изучаемых в книге.

Код и набор данных, с которыми мы будем работать на протяжении всей этой главы.

- **Блокнот** `End_to_end_example.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_13_End_to_end_example/End_to_end_example.ipynb.
- **Набор данных** `titanic.csv`.

Набор данных «Титаник»

В этом разделе мы загружаем и изучаем набор данных. Загрузка и обработка данных — важнейший для специалиста по обработке данных навык, поскольку успех модели в значительной степени зависит от предварительной обработки поступающих данных. Для этого воспользуемся пакетом `Pandas`.

На протяжении всей главы мы будем работать с популярным при изучении машинного обучения примером — набором данных «Титаник». На высоком уровне набор данных содержит информацию о множестве пассажиров «Титаника», включая имя, возраст, семейное положение, порт посадки и класс. И что более важно, в нем также есть сведения о том, выжил ли пассажир. Этот набор данных можно найти в `Kaggle` (www.kaggle.com) — популярном онлайн-сообществе с отличными наборами данных и конкурсами, которое я настоятельно рекомендую посетить.

ОБРАТИТЕ ВНИМАНИЕ Используемый набор данных является историческим и, соответственно, содержит множество социальных предубеждений, бытовавших в 1912 году. Исторические наборы данных не оставляют возможности для пересмотра или дополнительной выборки, позволяющей отразить текущие социальные нормы и понимание мира. Примерами могут служить различное отношение к пассажирам в зависимости от пола, социального положения и многие другие. Мы будем задействовать этот набор данных как числовую таблицу, потому что считаем, что это очень богатый и часто используемый для построения моделей и составления прогнозов набор данных. Однако, как специалисты по обработке данных, мы обязаны помнить о заложенных в данных предубеждениях, которые касаются расы, гендерной идентичности, сексуальной ориентации, социального статуса, способностей, национальности, убеждений и многих других вещей, и делать все, что в наших силах, чтобы гарантировать, что наши модели не будут увековечивать исторические предубеждения.

Особенности набора данных

Набор данных «Титаник» содержит имена 891 пассажира, взошедшего на борт «Титаника», и информацию о них, в том числе о том, выжили ли они при крушении судна. Столбцы набора данных.

- **PassengerID** — номер, идентифицирующий каждого пассажира, от 1 до 891.
- **Name** — полное имя пассажира.

- **Sex** — пол пассажира (мужчина или женщина).
- **Age** — возраст пассажира в виде целого числа.
- **Pclass** — класс, в котором путешествовал пассажир, — первый, второй или третий.
- **SibSP** — количество братьев, сестер и супругов пассажира (0, если он путешествовал один).
- **Parch** — количество родителей и детей пассажира (0, если он путешествовал один).
- **Ticket** — номер билета.
- **Fare** — стоимость проезда в британских фунтах стерлингов.
- **Cabin** — каюта, в которой путешествовал пассажир.
- **Embarked** — порт, в котором пассажир взшел на борт: «С» для Шербура, «Q» для Квинстауна и «S» для Саутгемптона.
- **Survived** — информация о том, выжил пассажир (1) или нет (0).

Использование Pandas для загрузки набора данных

В этом разделе мы узнаем, как открыть набор данных с помощью Pandas и загрузить его в `DataFrame`, который является объектом, применяемым Pandas для хранения таблиц данных. Я скачал данные из www.kaggle.com и сохранил в виде файла CSV (значения, разделенные запятыми) с именем `titanic.csv`. Прежде чем работать с Pandas, мы должны импортировать пакет с помощью следующей команды:

```
import pandas
```

Теперь, когда мы загрузили Pandas, загрузим сам набор данных. Для хранения наборов данных Pandas задействует два объекта: `DataFrame` и `Series`. По сути, это одно и то же, за исключением того, что `Series` используется для наборов данных только из одного столбца, а `DataFrame` — для наборов данных из более чем одного столбца.

Мы можем загрузить набор данных в виде фрейма данных с помощью следующей команды:

```
raw_data = pandas.read_csv('./titanic.csv', index_col="PassengerId")
```

Она сохраняет набор данных в фрейм данных Pandas с именем `raw_data`. Мы называем их необработанными данными, поэтому наша цель — впоследствии очистить их и предварительно обработать. Загрузив данные, мы видим, что первые строки выглядят как табл. 13.1. В общем, Pandas добавляет дополнительный столбец, нумерующий все элементы в наборе данных. Поскольку набор данных уже поставляется с нумерацией, мы можем установить данный индекс в качестве этого столбца, указав `index_col="PassengerId"`. Из-за этого в наборе данных строки индексируются с 1, а не с 0, как чаще встречается на практике.

Таблица 13.1. Набор данных с информацией о пассажирах «Титаника»

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...
890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

Сохранение и загрузка набора данных

Прежде чем приступить к изучению набора данных, нужно сделать небольшой шаг, который пригодится в дальнейшем. В конце каждого раздела мы сохраняем набор данных в CSV-файле и снова загружаем его в начале следующего. Это делается для того, чтобы можно было отложить книгу или выйти из блокнота Jupyter и позже вернуться к работе на любой контрольной точке, не повторяя все команды с самого начала. С таким небольшим набором данных повторный запуск команд не имеет большого значения, но представьте, что было бы, если бы мы обрабатывали большие объемы. Сериализация и сохранение данных очень важны, так как экономят время и вычислительную мощность.

Вот названия наборов данных, сохраненных в конце каждого раздела:

- набор данных «Титаник» — `raw_data`;
- очистка набора данных — `clean_data`;
- конструирование признаков — `preprocessed_data`.

Команды для сохранения и загрузки:

```
tablename.to_csv('./filename.csv', index=None)
tablename = pandas.read_csv('./filename.csv')
```

Когда Pandas загружает набор данных, он добавляет столбец индекса, который нумерует каждый из элементов. Мы можем игнорировать этот столбец, но, когда сохраняем набор данных, должны установить параметр `index=None`, чтобы избежать сохранения ненужных столбцов индекса.

В наборе данных уже содержится индексный столбец с именем `PassengerId`. Если бы мы хотели применить его в качестве индексного по умолчанию в Pandas, то могли бы указать `index_col='PassengerId'` при загрузке набора данных (но не будем этого делать).

Использование Pandas для изучения набора данных

В этом разделе я научу вас некоторым полезным методам изучения набора данных. Первый из них — это применение функции длины `len`. Она возвращает количество строк в наборе данных следующим образом:

```
len(raw_data)
```

Output: 891

Это означает, что набор данных содержит 891 строку. Для вывода имен столбцов используем свойство `columns` следующим образом:

```
raw_data.columns
```

Output: Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Employed'], dtype='object')

Теперь рассмотрим один из столбцов. Мы можем исследовать столбец `Survived` с помощью следующей команды:

```
raw_data['Survived']
```

Output:

```
0, 1, 1, 1, 0, ..., 0, 1, 0, 1, 0
```

Name: Survived, Length: 891, dtype: int64

Первый столбец — это индекс пассажира (от 1 до 891). Второй — 0, если пассажир утонул, и 1, если выжил. Однако, если нам нужны два столбца, например `Name` и `Age`, можем использовать следующую команду:

```
raw_data[['Name', 'Age']]
```

и она вернет `DataFrame` только с этими двумя столбцами.

Теперь предположим, что мы хотим выяснить, сколько пассажиров выжило. Для этого можем суммировать значения в столбце `Survived` с помощью функции `sum`:

```
sum(raw_data['Survived'])
```

Output: 342

Это указывает на то, что из 891 пассажира, сведения о которых имеются в наборе данных, выжили только 342.

Это лишь верхушка айсберга с точки зрения всей функциональности, которую Pandas предлагает для управления наборами данных. Для дополнительных подробностей обратитесь к документации на странице по адресу pandas.pydata.org.

Очистка набора данных. Отсутствующие значения и как с ними бороться

Теперь, когда мы выяснили, как обращаться с DataFrames, обсудим некоторые методы очистки набора данных. Почему это так важно? В реальной жизни данные могут быть беспорядочными, а ввод таких данных в модель обычно делает ее плохой. Важно, чтобы перед обучением моделей специалист по обработке данных хорошо изучил набор данных и провел определенную подготовительную очистку данных для моделей.

Первая проблема, с которой мы сталкиваемся, — это наборы данных, в которых отсутствуют значения. Из-за человеческих или компьютерных ошибок (или просто из-за проблем со сбором данных) наборы данных не всегда содержат все необходимые значения. Попытка подогнать модель к набору данных с отсутствием значений, вероятно, приведет к ошибке. Набор данных «Титаник» не исключение, когда речь заходит о недостающих данных. Например, взглянем на столбец `Cabin`, показанный здесь:

```
raw_data['Cabin']
Output:
0      NaN
1      C85
2      NaN
3      C123
4      NaN
...
886    NaN
887    B42
888    NaN
889    C148
890    NaN
Name: Cabin, Length: 891, dtype: object
```

В нем есть названия некоторых кают, например C123 или C148, но большинство значений — это NaN. NaN, или «not a number», означает, что запись либо отсутствует, либо нечитаема, либо того типа, который не может быть преобразован в число. Это могло произойти из-за канцелярских ошибок: можно предположить, что записи о «Титанике» старые и какая-то информация была утеряна или же номер каюты пассажира не записали с самого начала. В любом случае нам не нужны значения NaN в наборе данных. Мы находимся на этапе принятия решения: будем ли разбираться со значениями NaN или полностью удалим столбец? Сначала проверим, сколько значений NaN содержится в каждом столбце набора данных. Решение будет зависеть от ответа на этот вопрос.

Чтобы узнать, сколько значений NaN в каждом столбце, используем функцию `is_na` (или `is_null`). Функция `is_na` возвращает 1, если запись равна NaN, и 0 в противном случае. Следовательно, сложив эти значения, мы получим количество записей NaN в каждом столбце, как показано далее:

```
raw_data.isna().sum()
```

Output:

```

PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         2

```

В итоге мы видим, что единственными столбцами с отсутствующими данными являются **Age**, в котором отсутствуют 177 значений, **Cabin**, где нет 687 значений, и **Employed**, в котором нет двух значений. Мы можем решить проблему с недостающими данными несколькими методами и применим к разным столбцам этого набора данных разные методы.

Удаление столбцов с отсутствующими данными

Когда в столбце отсутствует слишком много значений, соответствующий признак может оказаться бесполезным для модели. В нашем случае **Cabin** не выглядит полезным признаком. Из 891 строки 687 не содержат значения. Этот признак должен быть удален. В Pandas можно сделать это с помощью функции **drop** следующим образом. Создадим новый **DataFrame** с именем **clean_data** для хранения данных, которые собираемся очистить:

```
clean_data = raw_data.drop('Cabin', axis=1)
```

Аргументы функции **drop**:

- имя столбца, который хотим удалить;
- параметр **axis**, который равен 1, когда хотим удалить столбец, и 0, когда строку.

Затем мы присваиваем выходные данные функции переменной **clean_data**, указывая, что хотим заменить старый фрейм данных, называемый **data**, на новый с удаленным столбцом.

Как не потерять весь столбец. Заполнение недостающих данных

Мы не всегда будем удалять столбцы, где отсутствуют данные, потому что в итоге можем потерять важную информацию. Вместо этого можно заполнить данные значениями, которые имели бы смысл. Например, взглянем на столбец возраста, показанный далее:

```
clean_data['Age'] Output:
```

```
0      22.0
1      38.0
2      26.0
3      35.0
4      35.0
... ..
886    27.0
887    19.0
888     NaN
889    26.0
890    32.0
```

```
Name: Age, Length: 891, dtype: float64
```

Как мы подсчитали ранее, в столбце **Age** отсутствуют только 177 значений из 891, что не так уж много. Этот столбец нам пригодится, поэтому не будем его удалять. Что же тогда делать с недостающими значениями? Есть несколько подходов, наиболее распространенный из которых — заполнение их средним или медианным значением других значений. Давайте сделаем последнее. Сначала мы вычисляем медиану, используя функцию медианы, и получаем 28. Далее применяем функцию **fillna**, которая заполняет недостающие значения тем, которое мы ей даем, как показано в следующем фрагменте кода:

```
median_age = clean_data["Age"].median()
clean_data["Age"] = clean_data["Age"].fillna(median_age)
```

Третьим столбцом, в котором нет некоторых значений, является столбец с двумя пропусками. Что делать здесь? Мы не можем использовать среднее значение, потому что это буквы, а не цифры. К счастью, только в двух строках из 891 отсутствует значение, так что здесь мы теряем не слишком много информации. Мое предложение состоит в том, чтобы объединить всех пассажиров, не имеющих значения, в столбце **Embarked**, в один класс. Можем назвать его **U**, что означает «Unknown». Это делает следующая строка кода:

```
clean_data["Embarked"] = clean_data["Embarked"].fillna('U')
```

Наконец, мы можем сохранить этот фрейм данных в CSV-файл с именем **clean_titanic_data** для использования в следующем разделе:

```
clean_data.to_csv('./clean_titanic_data.csv', index=None)
```

Конструирование признаков. Преобразование признаков в наборе данных перед обучением моделей

Теперь, очистив набор данных, мы намного ближе к тому, чтобы обучить модель. Однако нам все еще требуется выполнить некоторые важные манипуляции с данными, о которых узнаем в этом разделе. Первый заключается

в преобразовании типа данных из числового в категориальный и наоборот. Второй — это выбор признаков, при котором мы решаем, какие из них следует удалить вручную, чтобы улучшить обучение модели. Напомним (см. главу 2), что существуют два типа признаков: числовые и категориальные. Числовой признак хранится в виде чисел. В этом наборе данных такие характеристики, как возраст, стоимость проезда и класс, — это числа. Категориальный объект содержит несколько категорий или классов. Например, гендерный признак включает два класса: женский и мужской. Первый признак содержит три класса: С для Шербура, Q для Квинстауна и S для Саутгемптона.

Как мы видели на протяжении всей книги, модели машинного обучения в качестве входных данных принимают числа. Если это так, то как нам ввести слово «жен.» или букву «Q»? Нужен способ превратить категориальные признаки в числовые. Кроме того, хотите верить, хотите нет, но иногда для лучшего обучения может понадобиться рассматривать числовые признаки как категориальные, например разделить возраст на группы: 1–10, 11–20 и т. д. Подробнее об этом расскажем в разделе «Превращение числовых данных в категориальные...».

Более того, если говорить о таком признаке, как класс пассажира (называемый `Pclass`), действительно это числовой признак или же он категориальный? Должны ли мы думать о классе как о числе от одного до трех или как о трех классах — первом, втором и третьем? Ответим на все эти вопросы в данном разделе.

Здесь мы называем `DataFrame preprocessed_data`. Первые несколько строк этого набора данных показаны в табл. 13.2. Приступаем к предварительной обработке этих данных для обучения.

Таблица 13.2. Первые пять строк очищенного набора данных

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S
4	1	1	Futelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S

Преобразование категориальных данных в числовые. Однократное кодирование

Как упоминалось ранее, модели машинного обучения выполняют множество математических операций, и, чтобы этот процесс протекал успешно, нужно убедиться,

что все данные являются числовыми. Если есть какие-либо столбцы с категориальными данными, мы должны превратить их в числа. В этом разделе узнаем, как сделать это эффективно, используя технику, называемую *прямым кодированием*.

Но прежде, чем углубиться в прямое кодирование, ответим на вопрос: почему бы просто не присвоить каждому из классов номер? Например, если функция имеет 10 классов, почему бы не пронумеровать их 0, 1, 2... 9? Причина в том, что это приводит к упорядочению признаков, что не всегда полезно. Например, если столбец **Embarked** содержит три класса, C, Q и S, соответствующие Шербур, Квинстауну и Саутгемптону, присвоение им чисел 0, 1 и 2 неявно сообщит модели, что значение Квинстауна находится между значениями Шербура и Саутгемптона, что не обязательно так. Сложная модель может оказаться в состоянии справиться с этим неявным упорядочением, но более простые модели (например, линейные) пострадают. Мы хотели бы сделать эти значения более независимыми друг от друга, и именно здесь на помощь приходит прямое кодирование.

Работает оно следующим образом: сначала мы смотрим, сколько классов имеет признак, и создаем столько же новых столбцов. Например, столбец с двумя категориями, «женский» и «мужской», превратится в два столбца один для женщин и один для мужчин. Для ясности мы можем назвать эти столбцы **gender_male** и **gender_female**. Затем смотрим на каждого пассажира. Если это женщина, то в столбце **gender_female** будет 1, а в столбце **gender_male** — 0, если мужчина, то наоборот.

А что если у нас есть столбец с большим количеством классов, например **Embarked**? Поскольку он имеет три класса (C для Шербура, Q для Квинстауна и S для Саутгемптона), мы просто создаем три столбца с именами **embarked_c**, **embarked_q** и **embarked_s**. Таким образом, если пассажир сел, скажем, в Саутгемптоне, в третьем столбце будет 1, а в двух других — 0. Процесс проиллюстрирован на рис. 13.1. Слева расположены столбцы с категориальными характеристиками, такими как пол или порт посадки. Справа они превращены в числовые.

	gender		gender_female	gender_male
Passenger 1	F		1	0
Passenger 2	M		0	1
Passenger 3	M		0	1
Passenger 4	F		1	0

	embarked		embarked_c	embarked_q	embarked_s
Passenger 1	Q		0	1	0
Passenger 2	S		0	0	1
Passenger 3	C		1	0	1
Passenger 4	S		0	1	0

Рис. 13.1. Прямое кодирование данных превращает их в числа, которые может считывать модель машинного обучения

Функция Pandas `get_dummies` помогает с прямым кодированием. Мы используем ее для создания нескольких новых столбцов, затем присоединяем эти столбцы к набору данных. Исходный столбец нужно удалить, потому что эта информация является избыточной. Далее приведен код для прямого кодирования в столбцах `gender` и `embarked`:

```

gender_columns = pandas.get_dummies(data['Sex'], prefix='Sex')
embarked_columns = pandas.get_dummies(data["Pclass"], prefix="Pclass")

preprocessed_data = pandas.concat([preprocessed_data, gender_columns],
    axis=1)
preprocessed_data = pandas.concat([preprocessed_data, embarked_columns],
    axis=1)

preprocessed_data = preprocessed_data.drop(['Sex', 'Embarked'], axis=1)

```

Объединяет набор данных с только что созданными столбцами

Создает столбцы с результатами прямого кодирования

Удаляет старые столбцы из набора данных

Этот процесс может оказаться дорогостоящим. Представьте, что у вас есть столбец с 500 классами. Это добавит к таблице 500 новых столбцов! Более того, строки окажутся очень разреженными, поскольку будут содержать в основном нули. Теперь представьте, что у вас множество столбцов с сотнями классов в каждом — таблица стала бы слишком большой для обработки. В этом случае как специалист по обработке данных используйте собственные критерии для принятия решения. Если вычислительной мощности и места для хранения тысяч или, возможно, миллионов столбцов достаточно, то прямое кодирование не проблема. Если же ресурсы ограничены, стоит расширить классы, уменьшив число столбцов. Например, если имеется столбец со 100 видами животных, можно объединить их в шесть столбцов: млекопитающие, птицы, рыбы, амфибии, беспозвоночные и рептилии.

Можно ли использовать прямое кодирование для числовых характеристик?
Если да, то зачем это может понадобиться?

Очевидно, что если объект имеет категории (например, мужской и женский пол), лучшая стратегия — прямое кодирование. Но и для некоторых числовых характеристик прямое кодирование может пригодиться. Давайте посмотрим, например, на столбец `Pclass`. В нем цифрами 0, 1 и 2 обозначены первый, второй и третий класс. Должны ли мы сохранить его как числовой признак или же стоит применить прямое кодирование, получив признаки `Pclass1`, `Pclass2` и `Pclass3`? Вопрос непростой, и мы можем найти веские аргументы для обоих мнений. Кто-то может возразить, что не стоит излишне увеличивать набор данных, если это не дает потенциального улучшения производительности модели.

Существует эмпирическое правило, которое можно использовать, решая, следует ли разбивать столбец на несколько. Мы можем задаться вопросом: связан ли этот признак напрямую с результатом? Другими словами, делает ли увеличение значения признака более или менее вероятным выживание пассажира? Можно было бы предположить, что чем выше класс, тем больше вероятность выживания пассажира. Давайте узнаем, так ли это, выполнив некоторые подсчеты (см. код в блокноте):

- в первом классе выжили 62,96 % пассажиров;
- во втором классе выжили 47,28 % пассажиров;
- в третьем классе выжили 24,24 % пассажиров.

Обратите внимание на то, что наименьшая вероятность выживания у пассажиров третьего класса. Следовательно, будет верным утверждение, что увеличение (или уменьшение) класса автоматически повышает (понижает) шансы на выживание. Я предлагаю выполнить прямое кодирование этого признака:

```
categorized_pclass_columns = pd.get_dummies(preprocessed_data['Pclass'],
      prefix='Pclass')
preprocessed_data = pd.concat([preprocessed_data,
      categorized_pclass_columns], axis=1)
preprocessed_data = preprocessed_data.drop(['Pclass'], axis=1)
```

Превращение числовых данных в категориальные (и зачем это нужно?). Биннинг

В предыдущем разделе мы научились превращать категориальные данные в числовые. Здесь же увидим, как двигаться в обратном направлении. Зачем это вообще может понадобиться? Рассмотрим пример. Столбец **Age** — отличный числовой столбец. Модель машинного обучения отвечает на вопрос: «Насколько возраст определяет выживание на “Титанике”?» Представьте, что у нас есть линейная модель оценки выживания. Она в конечном итоге пришла бы к одному из следующих двух выводов:

- чем старше пассажир, тем больше у него шансов выжить;
- чем старше пассажир, тем меньше у него шансов выжить.

Однако всегда ли это так? Что, если связь между возрастом и выживаемостью не так однозначна? Что делать, если самая высокая вероятность выживания у группы от 20 до 30 лет, а у всех остальных возрастных групп она низкая? Что делать, если наименьшая вероятность выживания у возраста между 20 и 30 годами? Нам нужно предоставить модели полную свободу для прогнозирования того, какой возраст определяет шансы выжить. Что же делать?

Многие нелинейные модели способны справиться с этой задачей, но мы все равно должны изменить столбец возраста на что-то, что дает модели больше свободы в изучении данных. Полезный метод, который можно

использовать, — объединить возрасты, а именно разделить их на несколько групп. Например, мы можем превратить столбец возраста в следующее:

- от 0 до 10 лет;
- от 11 до 20 лет;
- от 21 года до 30 лет;
- от 31 года до 40 лет;
- от 41 года до 50 лет;
- от 51 года до 60 лет;
- от 61 года до 70 лет;
- от 71 года до 80 лет;
- 81 год или старше.

Это похоже на прямое кодирование в том смысле, что столбец возраста превращается в девять новых столбцов. Код приведен далее:

```
bins = [0, 10, 20, 30, 40, 50, 60, 70, 80]
categorized_age = pandas.cut(preprocessed_data['Age'], bins)
preprocessed_data['Categorized_age'] = categorized_age
preprocessed_data = preprocessed_data.drop(["Age"], axis=1)
```

Отбор признаков — избавляемся от ненужных

В разделе «Удаление столбцов с отсутствующими данными» мы исключали из таблицы некоторые столбцы, потому что в них было слишком много пропущенных значений. Однако мы должны отбросить еще несколько столбцов, так как они не нужны для нашей модели или, что еще хуже, способны полностью ее разрушить! В этом разделе мы обсудим, какие признаки следовало бы удалить. Но перед этим взгляните на признаки и подумайте, какие из них способны сослужить нашей модели плохую службу. Еще раз, вот они:

- **PassengerID** — уникальный номер, соответствующий пассажиру;
- **Name** — полное имя пассажира;
- **Sex (две категории)** — пол пассажира (мужчина или женщина);
- **Age (несколько категорий)** — возраст пассажира в виде целого числа;
- **Pclass (несколько категорий)** — класс, в котором путешествовал пассажир: первый, второй или третий;
- **SibSP** — количество братьев, сестер и супругов пассажира (0, если он путешествовал один);
- **Parch** — количество родителей и детей пассажира (0, если он путешествовал один);
- **Ticket** — номер билета;
- **Fare** — стоимость проезда в британских фунтах стерлингов;
- **Cabin** — каюта, в которой путешествовал пассажир;

- **Embarked** — порт, в котором пассажир взошел на борт: «С» для Шербура, «Q» для Квинстауна и «S» для Саутгемптона;
- **Survived** — информация о том, выжил пассажир (1) или нет (0).

Прежде всего, посмотрим на признак с именем. Должны ли мы учитывать его в модели? Ни в коем случае, и вот в чем причина: у всех пассажиров разные имена (возможно, за очень редкими и несущественными исключениями). Таким образом, модель просто обучилась бы запоминать имена выживших пассажиров, но не смогла бы ничего рассказать о новых, чьих имен она не встречала. Модель лишь запоминает данные — она не узнает ничего значимого об их признаках. Это означает, что она серьезно переобучена, следовательно, мы должны полностью избавиться от столбца **Name**.

Признаки **ticket** и **PassengerID** имеют ту же проблему, что и **name**, поскольку для каждого пассажира они уникальны. Мы удаляем и эти два столбца. В этом поможет функция **drop**:

```
preprocessed_data = preprocessed_data.drop(['Name', 'Ticket',
      'PassengerId'], axis=1)
```

А как насчет признака **Survived** — не следует ли избавиться и от него тоже? Определенно! Сохранение столбца **Survived** в наборе данных во время обучения будет излишним, потому что модель будет просто применять эту функцию, чтобы определить, выжил ли пассажир. Это все равно что проходить тест, глядя на решение. Но пока не будем удалять его из набора данных. Исключим его позже, когда разделим набор на признаки и метки для обучения.

Как обычно, сохраним этот набор данных в CSV-файле **preprocessed_titanic_data.csv** для использования в следующем разделе.

Обучение моделей

Теперь, когда данные предварительно обработаны, можем начать на их основе обучение различных моделей. Какие из изученных моделей нам следует выбрать: деревья решений, машины опорных векторов, логистические классификаторы? Ответ на этот вопрос заключается в оценке моделей. В этом разделе мы рассмотрим, как обучить несколько различных моделей, оценить их на основе набора данных для проверки и выбрать наилучшую модель, соответствующую набору данных.

Как обычно, загружаем данные из файла, в котором сохранили их в предыдущем разделе, как показано далее (назовем их **data**):

```
data = pandas.read_csv('preprocessed_titanic_data.csv')
```

Первые несколько строк предварительно обработанных данных содержатся в табл. 13.3. Обратите внимание на то, что здесь отображаются не все столбцы, потому что всего их 21. Дополнительные столбцы были созданы, когда применили прямое кодирование и использовали группировку для нескольких признаков.

Таблица 13.3. Первые пять строк предварительно обработанных данных, готовых к вводу в модель

Survived	SibSp	Parch	Fare	Sex_female	Sex_male	Pclass_C	Pclass_Q	Pclass_S	Pclass_U	...	Categorized_age_(10, 20]
0	1	0	7.2500	0	1	0	0	1	0	...	0
1	1	0	71.2833	1	0	1	0	0	0	...	0
1	0	0	7.9250	1	0	0	0	1	0	...	0
1	1	0	53.1000	1	0	0	0	1	0	...	0
0	0	0	8.0500	0	1	0	0	1	0	...	0

ОТВЛЕЧЕМСЯ Если вы запустите код из блокнота, то можете получить другие значения.

Разделение данных на признаки и метки, а также обучающие и проверочные наборы

Наш набор данных представляет собой совокупную таблицу с признаками и метками. Требуется выполнить два разделения. Во-первых, нужно отделить объекты от меток, чтобы передать все это в модель. Далее следует сформировать обучающий и тестовый набор. Все это мы сделаем в данном разделе.

Чтобы разделить набор данных на две таблицы, **features** и **labels**, используем функцию **drop**:

```
features = data.drop(["Survived"], axis=1)
labels = data["Survived"]
```

Затем разделяем данные на обучающие и проверочные наборы. Мы будем применять 60 % данных для обучения, 20 % — для проверки и 20 % — для тестирования. Для разделения данных мы задействуем функцию Scikit-Learn **train_test_split**. В ней с помощью параметра **test_size** указываем процент данных, которые хотим использовать для проверки. Результатом будут четыре таблицы: **features_train**, **features_test**, **labels_train**, **labels_test**.

Если бы мы хотели разделить данные на 80 % обучающих и 20 % для тестирования, то применили бы следующий код:

```
from sklearn.model_selection import train_test_split
features_train, features_test, labels_train, labels_test =
    train_test_split(features, labels, test_size=0.2)
```

Однако, поскольку нам требуется 60 % для обучения, 20 % для проверки и 20 % для тестирования, придется использовать `train_test_split` дважды: один раз для отделения обучающих данных и один раз для разделения наборов для проверки и тестирования:

```
features_train, features_validation_test, labels_train,
    labels_validation_test = train_test_split(features, labels,
    test_size=0.4)
features_validation, features_test, labels_validation,
    labels_test = train_test_split(features_validation_test,
    labels_validation_test, test_size=0.5)
```

ОТВЛЕЧЕМСЯ Вы могли заметить, что в блокноте мы указали фиксированное значение `random_state` для этой функции. Причина в том, что `train_test_split` перемешивает данные при разделении. Мы фиксируем это случайное состояние, чтобы всегда получать одно и то же разделение.

Мы можем проверить длину этих DataFrame и заметить, что длина обучающего набора равна 534, проверочного — 178, а тестового — 179. Теперь вспомните золотое правило из главы 4, которое предписывает никогда не использовать данные для тестирования при обучении или принятии решений по моделям. Поэтому мы придержим тестовый набор до момента, когда решим, какую модель применять. Будем использовать обучающий набор для обучения моделей, а набор для проверки — для принятия решений о том, какую модель выбрать.

Обучение нескольких моделей на наборе данных

Наконец-то мы добрались до самого интересного — обучения моделей! В этом разделе узнаем, как обучить несколько различных моделей в Scikit-Learn с помощью всего нескольких строк кода.

Начнем с обучения модели логистической регрессии. Мы можем сделать это, создав экземпляр `LogisticRegression` и используя метод `fit` следующим образом:

```
from sklearn.linear_model import LogisticRegression
lr_model = LogisticRegression()
lr_model.fit(features_train, labels_train)
```

Давайте также обучим дерево решений, наивную байесовскую модель, машину опорных векторов, случайный лес, дерево с градиентным бустингом и модель AdaBoost, как показано в следующем коде:

```
from sklearn.tree import DecisionTreeClassifier, GaussianNB, SVC,
    RandomForestClassifier, GradientBoostingClassifier,
AdaBoostClassifier

dt_model = DecisionTreeClassifier()
dt_model.fit(features_train, labels_train)
```

```

nb_model = GaussianNB()
nb_model.fit(features_train, labels_train)

svm_model = SVC()
svm_model.fit(features_train, labels_train)

rf_model = RandomForestClassifier()
rf_model.fit(features_train, labels_train)

gb_model = GradientBoostingClassifier()
gb_model.fit(features_train, labels_train)

ab_model = AdaBoostClassifier()
ab_model.fit(features_train, labels_train)

```

Какая же модель лучше? Оценка моделей

Теперь, когда мы обучили несколько моделей, нужно выбрать из них лучшую. В этом разделе для их оценки используем различные показатели и проверочный набор. Напомним, что в главе 7 мы изучили достоверность, полноту, точность и F_1 -меру. Чтобы освежить вашу память, далее приводим определения.

ДОСТОВЕРНОСТЬ — соотношение между количеством правильно помеченных точек и их общим количеством.

ПОЛНОТА — доля правильно классифицированных точек с положительными метками. Другими словами, $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$, где TP — количество истинно положительных результатов, FN — количество ложноотрицательных результатов.

ТОЧНОСТЬ — доля правильно классифицированных точек среди тех, что классифицированы как положительные. Другими словами, $\text{precision} = \text{TP} / (\text{TP} + \text{FP})$, где FP — количество ложноположительных результатов.

F1-МЕРА — среднее гармоническое значение точности и полноты. Это число находится между точностью и полнотой, но ближе к меньшему из двух.

Проверка достоверности каждой модели

Начнем с оценки достоверности моделей. В Scikit-Learn для этого имеется функция `score`:

```

lr_model.score(features_validation, labels_validation)
Output:
0.7932960893854749

```

При ее вычислении для всех остальных моделей получаем следующие результаты, которые я округлил до двух цифр (всю процедуру см. в блокноте).

Достоверность:

- логистическая регрессия — 0,77;
- дерево решений — 0,78;
- наивная байесовская модель — 0,72;
- SVM — 0,68;
- случайный лес — 0,7875;
- градиентный бустинг — 0,81;
- AdaBoost — 0,76.

Это указывает на то, что лучшей моделью в этом наборе данных является дерево с градиентным бустингом, потому что именно оно показывает самую высокую достоверность в проверочном наборе (81 %, что хорошо для набора данных «Титаник»). Это неудивительно, потому что данный алгоритм обычно работает очень хорошо.

Вы можете выполнить аналогичную процедуру для расчета полноты, точности и F_1 -меры. Я оставляю вам полноту и точность, а вот вычисление для F_1 -меры мы сделаем вместе.

Тестирование F_1 -меры каждой модели

Вот как мы проверяем F_1 -меру. Вначале мы должны вывести прогнозы модели с помощью функции `predict`, а затем использовать функцию `f1_score` следующим образом:

```
lr_predicted_labels = lr_model.predict(features_validation)
f1_score(labels_validation, lr_predicted_labels)
```

Output:
0.6870229007633588

Использует модель для составления прогнозов

Вычисляет F_1 -меру

Как и ранее, мы можем проделать это для всех моделей, чтобы получить следующее.

 F_1 -мера:

- логистическая регрессия — 0,69;
- дерево решений — 0,71;
- наивная байесовская модель — 0,63;
- машина опорных векторов — 0,42;
- случайный лес — 0,68;
- градиентный бустинг — 0,74;
- AdaBoost — 0,69.

И снова дерево с градиентным бустингом победило с F_1 -мерой 0,74. Учитывая тот факт, что его показатели были намного выше, чем у других моделей, можем с уверенностью заключить, что среди восьми моделей деревьев с градиентным бустингом лучшие. Обратите внимание на то, что древовидные модели в целом показали хорошие результаты, что неудивительно, учитывая высокую нелинейность

набора данных. Было бы интересно обучить нейронную сеть и модель XGBoost на этом наборе данных, и я призываю вас сделать это самостоятельно!

Тестирование модели

После сравнения моделей с использованием набора для проверки мы наконец приняли решение и выбрали дерево с градиентным бустингом в качестве лучшей модели для этого набора данных. Не удивляйтесь, деревья с градиентным бустингом (и их близкие родственники XGBoost) выигрывают большинство соревнований. Но чтобы увидеть, действительно ли мы преуспели или просто случайно переобучились, нужно провести итоговый тест этой модели — проверить ее на тестовом наборе, к которому мы еще не прикасались.

Во-первых, оценим достоверность:

```
gb_model.score(features_test, labels_test)
```

Output:

```
0.8324022346368715
```

А теперь посмотрим на F_1 -меру, как показано далее:

```
gb_predicted_test_labels = gb_model.predict(features_test)
```

```
f1_score(labels_test, gb_predicted_test_labels)
```

Output:

```
0.8026315789473685
```

Эти оценки довольно хороши для набора данных «Титаник». Таким образом, мы можем с уверенностью сказать, что наша модель хороша.

Однако мы обучили эти модели, не касаясь их гиперпараметров, что означает: Scikit-Learn выбрал для них некоторые стандартные гиперпараметры. Есть ли способ отыскать наилучшие гиперпараметры для модели? В следующем разделе мы узнаем, как это делается.

Настройка гиперпараметров для поиска наилучшей модели. Поиск по сетке

В предыдущем разделе мы обучили несколько моделей и обнаружили, что дерево градиентного бустинга работает лучше всего. Однако не касались множества различных комбинаций гиперпараметров, поэтому у нас все еще есть возможность для совершенствования обучения. В этом разделе рассмотрим полезную технику поиска среди множества комбинаций гиперпараметров с целью обнаружения хорошей модели для наших данных.

Производительность дерева с градиентным бустингом была почти такой высокой, какую вообще можно получить для набора данных «Титаник», так что давайте оставим его в покое. А слабая SVM пришла последней с достоверностью 69 % и F_1 -мерой 0,42. Однако мы верим в SVM, потому что она представляет собой весьма мощную модель машинного обучения. Возможно, плохая

производительность этой SVM связана с использованными гиперпараметрами. Вероятно, существует более удачная их комбинация, которая и работает лучше.

ОТВЛЕЧЕМСЯ На протяжении всего этого раздела мы выбираем параметры. Некоторые из них основаны на опыте, некоторые — на стандартной практике, а некоторые произвольны. Я призываю вас проделать аналогичную процедуру с любым выбором, который решите сделать, и попытаться превзойти текущие оценки моделей!

Чтобы повысить производительность нашей SVM, используем метод, называемый поиском по сетке, который состоит в обучении модели несколько раз по различным комбинациям гиперпараметров и выборе того, который работает лучше всего на проверочном наборе.

Начнем с выбора ядра. На практике я обнаружил, что ядро RBF (радиальная базисная функция) работает скорее хорошо, поэтому выберем его. Напомним (см. главу 9), что гиперпараметром, который входит в ядро RBF, служит γ (gamma), которая является действительным числом. Давайте попробуем обучить SVM с двумя значениями γ , а именно 1 и 10. Почему 1 и 10? Обычно, когда ищем гиперпараметры, мы, как правило, выполняем экспоненциальный поиск, поэтому пробовали бы такие значения, как 0,1, 1, 10, 100, 1000 и т. д., но не 1, 2, 3, 4, 5. Такой экспоненциальный поиск охватывает большее пространство и дает больше шансов найти хорошие гиперпараметры, поэтому выполнение такого типа поиска — стандартная практика у специалистов по обработке данных.

Вспомним также (еще раз см. главу 9), что другим гиперпараметром, связанным с SVM, служит параметр C. Давайте попробуем обучить модели с $C = 1$ и $C = 10$. Это дает нам следующие четыре возможные модели для обучения:

- **модель 1:** ядро — RBF, $\gamma = 1$, $C = 1$;
- **модель 2:** ядро — RBF, $\gamma = 1$, $C = 10$;
- **модель 3:** ядро — RBF, $\gamma = 10$, $C = 1$;
- **модель 4:** ядро — RBF, $\gamma = 10$, $C = 10$.

Мы можем легко обучить их все на своем обучающем наборе с помощью следующих восьми строк кода:

```
svm_1_1 = SVC(kernel='rbf', C=1, gamma=1)
svm_1_1.fit(features_train, labels_train)

svm_1_10 = SVC(kernel='rbf', C=1, gamma=10)
svm_1_10.fit(features_train, labels_train)

svm_10_1 = SVC(kernel='rbf', C=10, gamma=1)
svm_10_1.fit(features_train, labels_train)

svm_10_10 = SVC(kernel='rbf', C=10, gamma=10)
svm_10_10.fit(features_train, labels_train)
```

Теперь оцениваем их, используя достоверность (еще один произвольный выбор — мы также могли бы применить F_1 -меру, точность или полноту). Оценки записаны в табл. 13.4.

Таблица 13.4. Метод поиска по сетке полезен для поиска по множеству комбинаций гиперпараметров и выбора наилучшей модели

γ	$C = 1$	$C = 10$
0,1	0,69	0,72
1	0,70	0,70
10	0,67	0,65

Обратите внимание: наилучшую точность 0,72 показывает модель с $\gamma = 0,1$ и $C = 1$. Это шаг вперед по сравнению с 0,68, которую мы получили ранее, когда не указывали никаких гиперпараметров.

Если бы у нас было гораздо больше параметров, мы бы просто составили из них сетку и обучили все возможные модели. Обратите внимание на то, что по мере исследования все большего числа вариантов количество моделей стремительно растет. Например, если бы мы хотели изучить пять значений для γ и четыре значения для C , нам пришлось бы обучить 20 моделей (5×4). Мы также можем добавить больше гиперпараметров — например, если бы имелся третий гиперпараметр, который мы хотели бы попробовать, и для него существовало семь значений, то пришлось бы обучить в общей сложности 140 моделей ($5 \times 4 \times 7$). Поскольку количество моделей растет очень быстро, важно выбирать варианты так, чтобы они хорошо покрывали пространство гиперпараметров без обучения огромного количества моделей.

Scikit-Learn предлагает для этого простой способ — использование объекта `GridSearchCV`. Сначала мы определяем гиперпараметры в виде словаря, где ключ — это имя параметра, а значение, соответствующее этому ключу, — список значений, которые мы хотим опробовать для гиперпараметра. В нашем случае рассмотрим следующие комбинации гиперпараметров:

- **Kernel:** RBF;
- **C:** 0,01, 0,1, 1, 10, 100;
- **gamma:** 0,01, 0,1, 1, 10, 100.

В этом поможет следующий код:

```
svm_parameters = {'kernel': ['rbf'],
                  'C': [0.01, 0.1, 1, 10, 100],
                  'gamma': [0.01, 0.1, 1, 10, 100]}
svm = SVC()
svm_gs = GridSearchCV(estimator = svm,
                      param_grid = svm_parameters)
svm_gs.fit(features_train, labels_train)
```

← Обычная SVM без гиперпараметров

← Словарь с гиперпараметрами и значениями, которые мы хотим попробовать

← Объект `GridSearchCV`, где мы передаем SVM и словарь гиперпараметров

← Подгоняем модель `GridSearchCV` таким же образом, как и обычную модель в Scikit-Learn

Он обучает 25 моделей со всеми комбинациями гиперпараметров, приведенных в словаре гиперпараметров. Теперь мы выбираем лучшую из этих моделей и называем ее `svm_winner`. Рассчитаем ее достоверность на наборе проверки следующим образом:

```
svm_winner = svm_gs.best_estimator_  
svm_winner.score(features_validation, labels_validation)
```

Output:

```
0.7303370786516854
```

Победившая модель достигла достоверности 0,73, что лучше, чем исходные 0,68. Мы все еще можем улучшить ее, выполнив более широкий поиск по гиперпараметру, и я рекомендую вам сделать это самостоятельно. А пока рассмотрим, какие гиперпараметры использовались последней победившей моделью SVM:

```
svm_winner
```

Output:

```
SVC(C=10, break_ties=False, cache_size=200, class_weight=None, coef0=0.0,  
    decision_function_shape='ovr', degree=3, gamma=0.01, kernel='rbf',  
    max_iter=-1, probability=False, random_state=None, shrinking=True,  
    tol=0.001, verbose=False)
```

Победившая модель использовала ядро RBF с $\gamma = 0,01$ и $C = 10$.

ЗАДАЧА Я призываю вас попробовать применить поиск по сетке на других моделях и посмотреть, насколько вы сможете улучшить достоверность и F_1 -меру модели-победителя! Если у вас хороший результат, запустите его в наборе данных Kaggle и отправьте свои прогнозы по этой ссылке: <https://www.kaggle.com/c/titanic/submit>.

Еще одно: что это за CV в конце GridSearchCV? Это означает *cross-validation*, о которой мы узнаем в следующем разделе.

Применение k -кратной перекрестной проверки для повторного использования данных в качестве обучающих и проверочных

В этом разделе мы изучаем альтернативу традиционному методу обучения — тестированию, которое применяли в этой главе. Речь идет о k -кратной перекрестной проверке, которая полезна во многих ситуациях, особенно когда набор данных невелик.

В рассмотренном примере мы использовали 60 % данных для обучения, 20 % — для проверки и последние 20 % — для тестирования. На практике это работает, но, похоже, мы теряем некоторые данные, не так ли? В конечном итоге мы обучаем модель, используя только 60 % данных, что может повредить ей, особенно когда набор данных невелик. k -кратная перекрестная проверка — это

способ использовать все данные для обучения и тестирования, повторно задействуя их несколько раз. Работает она следующим образом.

1. Разделите данные на k равных (или почти равных) частей.
2. Обучите модель k раз, используя объединение $k - 1$ частей в качестве обучающего набора, а оставшуюся часть — в качестве проверочного.
3. Итоговая оценка этой модели представляет собой среднее значение оценок проверки на k этапах.

На рис. 13.2 показана картина четырехкратной перекрестной проверки. Вверху — классическое разделение на обучение и проверку. Внизу проиллюстрирована процедура проверки, в которой данные разделены на четыре равные (или почти равные) части. Затем мы обучаем модель четыре раза. Каждый раз выбираем три части в качестве обучающего набора, а оставшуюся — в качестве проверочного.

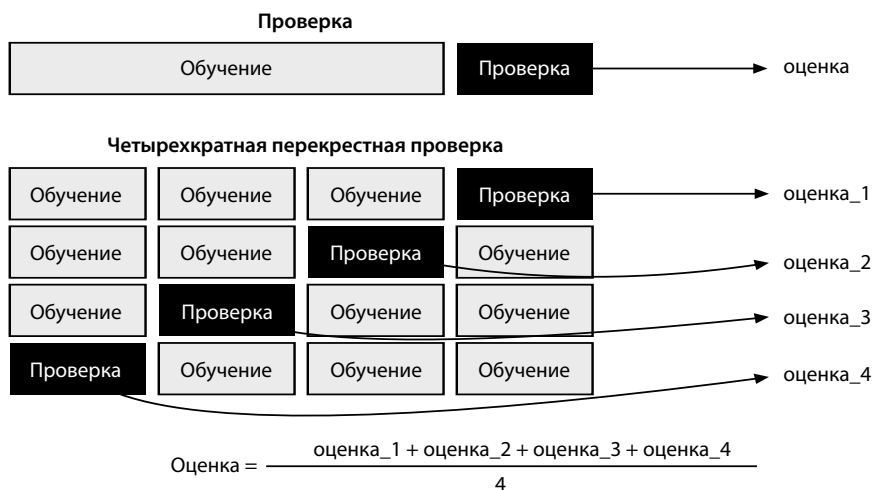


Рис. 13.2. k -кратная перекрестная проверка — полезный метод, который повторно использует данные как для обучения, так и для проверки

Этот метод используется в GridSearchCV, и результаты процесса можно увидеть, набрав `svm_gs.cv_results_`. Мы не будем показывать результаты здесь, потому что они очень объемные, но вы можете взглянуть на них в блокноте.

Резюме

- Pandas — это полезный пакет Python для открытия, обработки и сохранения наборов данных.
- Очистка данных необходима, поскольку у них могут иметься такие проблемы, как пропущенные значения.

- Признаки могут быть числовыми или категориальными. Числовые признаки — это числа, такие как возраст. Категориальные признаки — это категории или типы, такие как собака/кошка/птица.
- Модели машинного обучения принимают только числа, поэтому, чтобы передать в модель категориальные данные, мы должны превратить их в числовые. Один из способов сделать это — прямое кодирование.
- В некоторых ситуациях требуется рассматривать числовые признаки как категориальные, чего можно добиться с помощью биннинга.
- Важно правильно выбрать и удалить ненужные признаки из данных.
- Scikit-Learn — это полезный пакет для обучения, тестирования и оценки моделей машинного обучения.
- Перед обучением модели нужно разделить данные на обучающие, проверочные и тестовые. Для этого существуют функции Pandas.
- Поиск по сетке — это метод, используемый для поиска наилучших гиперпараметров для модели. Он состоит из обучения нескольких моделей по набору гиперпараметров (иногда большому).
- k -кратная перекрестная проверка — это метод повторного применения данных в качестве обучающих и проверочных. Он заключается в обучении и тестировании нескольких моделей на разных участках данных.

Упражнения

Упражнение 13.1

Репозиторий содержит файл с именем `test.csv`. В нем имеются сведения о других пассажирах «Титаника», но нет столбца `Survived`.

1. Предварительно обработайте данные в файле, как делалось в этой главе.
2. Используйте любую из моделей для прогнозирования меток в этом наборе данных. Согласно вашей модели, сколько пассажиров выжили?
3. Сравнивая характеристики всех моделей, описанных в этой главе, как вы думаете, сколько пассажиров из тестовой группы действительно выжили?



Глава 2. Типы машинного обучения

Что касается вопросов, заданных в этой главе, то ваши ответы не обязательно должны совпадать с моими. Если у вас есть другие идеи для моделей, используемых в приложениях, они могут оказаться замечательными! Я рекомендую вам поискать их в литературе и, если их пока не существует, попробовать реализовать.

Упражнение 2.1

Примером контролируемого или неконтролируемого обучения является каждый из следующих сценариев? Объясните свои ответы. В случае неоднозначности ответа выберите один из вариантов и объясните, почему выбрали именно его.

- А. Система рекомендаций в социальной сети, которая показывает пользователю потенциальных друзей.
- Б. Система на новостном сайте, которая разделяет новости на темы.
- В. Функция автоматического завершения предложений в Google.
- Г. Система рекомендаций в интернет-магазине, которая формирует список предлагаемых товаров, основываясь на истории покупок.
- Д. Созданная для компании, выпускающей кредитные карты, система, которая фиксирует мошеннические транзакции.

Решение

В зависимости от того, как вы интерпретировали задачу и набор данных, их все можно рассматривать как пример контролируемого или неконтролируемого обучения. Совершенно нормально (и очень хорошо!) иметь разные ответы, если за ними стоит правильная аргументация.

- А. Это пример как контролируемого, так и неконтролируемого обучения. Контролируемое обучение: для конкретного пользователя мы можем построить классификационную модель, в которой метка каждого другого пользователя будет положительной, если он является потенциальным другом, и отрицательной, если это не так. Неконтролируемое обучение: мы можем объединить пользователей в группы по сходству демографических или поведенческих особенностей. Конкретному пользователю можем рекомендовать других пользователей из их кластера в качестве потенциальных друзей.
- Б. Это тоже пример как контролируемого, так и неконтролируемого обучения. Контролируемое обучение: модель классификации, в которой меткой каждой новостной статьи является тема, например политика, спорт или наука. Неконтролируемое обучение: мы можем сгруппировать статьи, а затем вручную проверить, похожи ли темы в каждом кластере. Если да, то можем вручную пометить каждый кластер наиболее распространенной темой. Есть несколько более продвинутых методов неконтролируемого обучения, таких как скрытое распределение Дирихле, с которыми вы можете ознакомиться в видео: <https://www.youtube.com/watch?v=T05t-SqKaRY>.
- В. Это скорее задача контролируемого обучения. Мы можем построить классификационную модель, в которой признаки — это последние несколько слов, введенных пользователем, а метка — следующее слово, которое он введет. Таким образом, прогноз модели — это слово, которое мы предложим пользователю.
- Г. Эта задача похожа на А, и ее можно рассматривать как задачу контролируемого или неконтролируемого обучения. Контролируемое обучение: для конкретного пользователя мы можем построить классификационную модель для всех продуктов, где для каждого продукта прогнозируем, купит ли его пользователь. Мы также можем построить регрессионную модель, в которой предсказываем, сколько денег он потратит на конкретный продукт. Неконтролируемое обучение: можем сгруппировать пользователей. Если пользователь купил продукт, мы можем рекомендовать его другим входящим в кластер. Также можем сгруппировать продукты и, если пользователь купил продукт, рекомендуем продукты из того же кластера.
- Д. Это скорее задача контролируемого обучения. Мы можем построить классификационную модель, которая предсказывает, мошенническая определенная транзакция или нет, основываясь на характеристиках транзакции. Это также можно рассматривать как задачу неконтролируемого обучения, в которой мы группируем транзакции, и те, которые являются выбросами, имеют более высокую вероятность оказаться мошенническими.

Упражнение 2.2

Регрессию или классификацию необходимо применять для любого из следующих применений машинного обучения? Объясните свои ответы. В случае неоднозначности ответа выберите один из вариантов и объясните, почему выбрали именно его.

- А. Интернет-магазин, предсказывающий, сколько денег пользователь потратит на сайте.
- Б. Голосовой помощник, декодирующий голос и превращающий его в текст.
- В. Продажа или покупка акций определенной компании.
- Г. Рекомендации видео от YouTube.

Решение

- А. Регрессия, потому что мы пытаемся предсказать количество денег, которые тратит пользователь, а это числовой признак.
- Б. Классификация, потому что мы пытаемся предсказать, адресовано ли произнесенное пользователем предложение Alexa, а это категориальный признак.
- В. Это может быть регрессия или классификация. Если мы пытаемся предсказать ожидаемую прибыль или ожидаемый риск, чтобы принять решение, это регрессия. Если же пробуем предсказать, следует ли нам покупать акции, это классификация.
- Г. Это снова может быть регрессия или классификация. Если мы пытаемся предсказать, сколько времени пользователь потратит на просмотр видео, чтобы рекомендовать его, это регрессия. Если мы пробуем предсказать, будет ли пользователь смотреть видео, это классификация.

Упражнение 2.3

Ваша задача — построить беспилотный автомобиль. Приведите по крайней мере три примера проблем машинного обучения, которые вам пришлось бы при этом решить. В каждом из них объясните, какое обучение вы применяете, контролируемое или неконтролируемое. Если контролируемое, то применяете регрессию или классификацию? Если используете другие типы машинного обучения, объясните, какие и почему.

Решение

- Модель классификации, которая на основе изображения определяет, есть ли на нем пешеходы, знаки остановки, полосы движения, другие автомобили и т. д., — это обширная область машинного обучения, называемая компьютерным зрением, которую я настоятельно рекомендую изучить!
- Аналогичная предыдущей модели классификации, которая определяет, какие объекты находятся вокруг автомобиля, на основе сигналов от различных датчиков машины (лидар и т. д.).
- Модель машинного обучения, которая находит кратчайший путь к желаемому месту назначения. Это не на 100 % контролируемое или неконтролируемое обучение. Здесь можно использовать еще несколько классических алгоритмов искусственного интеллекта, таких как поиск по A^* (A-star).

Глава 3. Прямая вплотную к точкам. Линейная регрессия

Упражнение 3.1

Веб-сайт обучил модель линейной регрессии для прогнозирования времени, которое пользователь проведет на сайте. Полученная формула выглядит так:

$$\hat{t} = 0,8d + 0,5m + 0,5y + 0,2a + 1,5,$$

где $\hat{t}$ — прогнозируемое время в минутах, d , m , y и a — переменные индикатора (они принимают только значения 0 или 1), определенные следующим образом:

- d — это переменная, указывающая, применяет ли пользователь ПК;
- m — переменная, указывающая, задействует ли пользователь мобильное устройство;
- y — переменная, указывающая, молодой ли пользователь (моложе 21 года);
- a — это переменная, указывающая, взрослый ли пользователь (21 год или старше).

Пример: если пользователю 30 лет и он работает на ПК, то $d = 1$, $m = 0$, $y = 0$ и $a = 1$.

Если 45-летний пользователь просматривает веб-сайт с телефона, каким будет ожидаемое время, которое он проведет на сайте?

Решение

В этом случае значения переменных следующие:

- $d = 0$, потому что пользователь не работает на ПК;
- $m = 1$, потому что пользователь зашел на сайт с мобильного телефона;
- $y = 0$, потому что пользователь не младше 21 года;
- $a = 1$, потому что пользователю больше 21 года.

Включив их в формулу, мы получаем:

$$\hat{t} = 0,8 \times 0 + 0,5 \times 1 + 0,5 \times 0 + 0,2 \times 1 + 1,5 = 2,2.$$

То есть модель прогнозирует, что этот пользователь проведет на веб-сайте 2,2 минуты.

Упражнение 3.2

Представьте, что мы обучили модель линейной регрессии на медицинском наборе данных. Она предсказывает ожидаемую продолжительность жизни пациента. Каждому из свойств в наборе данных модель будет присваивать определенный вес.

- А. Вес, присвоенный следующим величинам, — это положительное число, отрицательное число или ноль? (*Примечание: если считаете, что вес — очень маленькое положительное или отрицательное число, то можете сказать, что это ноль.*)
- 1) Количество часов, которые пациент затрачивает на физические упражнения в неделю.
 - 2) Количество сигарет, которые пациент выкуривает в неделю.
 - 3) Количество у пациента членов семьи с проблемами с сердцем.
 - 4) Количество у пациента братьев и сестер.
 - 5) Был пациент госпитализирован или нет.
- Б. У модели есть смещение. Как вы думаете, оно положительное, отрицательное или нулевое?

Решение

- А. Сделаем некоторые обобщения, основанные на общих медицинских знаниях. Для конкретного пациента следующее не обязательно верно, но мы сделаем предположение, что это верно для населения в целом.
- 1) Ожидается, что пациент, который много тренируется, проживет дольше, чем тот, который этого не делает. Таким образом, этот вес должен быть положительным числом.
 - 2) Ожидается, что пациент, который выкуривает много сигарет в неделю, проживет меньше, чем похожий пациент, который этого не делает. То есть этот вес должен быть отрицательным числом.
 - 3) Пациент, в семье у которого многие испытывают проблемы с сердцем, имеет более высокую вероятность возникновения подобных проблем. Следовательно, ожидается, что он проживет меньше, чем аналогичный пациент, у которого их нет. Таким образом, этот вес должен быть отрицательным числом.
 - 4) Количество братьев и сестер, как правило, не влияет на ожидаемую продолжительность жизни, поэтому мы предполагаем, что этот вес будет очень небольшим или вообще нулевым числом.
 - 5) Пациент, который был госпитализирован в прошлом, скорее всего, ранее имел проблемы со здоровьем. Таким образом, его ожидаемая продолжительность жизни меньше, чем у аналогичного пациента, который прежде не попадал в больницу. Следовательно, этот вес должен быть отрицательным числом. Конечно, госпитализация может произойти по причине, не влияющей на ожидаемую продолжительность жизни (например, перелом ноги), но в среднем можем сказать, что, если пациент был в больнице в прошлом, у него более высокая вероятность возникновения проблем со здоровьем.
- Б. Смещение — это прогноз для пациента, для которого все характеристики равны нулю (то есть пациент, который не курит, не занимается спортом,

у которого нет членов семьи с заболеваниями сердца, нет братьев и сестер и который никогда не был госпитализирован). Поскольку ожидается, что жизнь этого пациента будет продолжительной, смещение модели должно быть положительным числом.

Упражнение 3.3

Далее приведен набор данных домов с размерами (в квадратных футах) и ценами (в долларах).

Дом	Размер (s)	Вознаграждение (p)
1	100	200
2	200	475
3	200	400
4	250	520
5	325	735

Предположим, мы обучили модель, в которой прогноз цены дома, основанный на размере, выглядит следующим образом:

$$\hat{p} = 2s + 50.$$

- А. Вычислите прогнозы, которые эта модель делает для набора данных.
- Б. Вычислите среднюю абсолютную погрешность этой модели.
- В. Вычислите среднюю квадратическую погрешность модели.

Решение

А. Прогнозируемые цены, основанные на модели:

- дом 1: $\hat{p} = 2 \times 100 + 50 = 250$;
- дом 2: $\hat{p} = 2 \times 200 + 50 = 450$;
- дом 3: $\hat{p} = 2 \times 200 + 50 = 450$;
- дом 4: $\hat{p} = 2 \times 250 + 50 = 550$;
- дом 5: $\hat{p} = 2 \times 325 + 50 = 700$.

Б. Средняя абсолютная погрешность составляет:

$$\begin{aligned} & 1/5 (|200 - 250| + |475 - 450| + |400 - 450| + |520 - 550| + |735 - 700|) \\ & = 1/5 (50 + 25 + 50 + 30 + 35) = 38. \end{aligned}$$

В. Средняя квадратическая ошибка равна:

$$\begin{aligned} & 1/5 ((200 - 250)^2 + (475 - 450)^2 + (400 - 450)^2 + (520 - 550)^2 + (735 - 700)^2) = \\ & = 1/5 (2500 + 625 + 2500 + 900 + 1225) = 1550. \end{aligned}$$

Следовательно, средняя квадратическая погрешность составляет: $\sqrt{1550} = 39,37$.

Упражнение 3.4

Наша цель состоит в том, чтобы переместить линию $\hat{y} = 2x + 3$ ближе к точке $(x, y) = (5, 15)$, используя приемы, которые освоили в этой главе. Для следующих двух задач примите скорость обучения $\eta = 0,01$.

- А. Примените абсолютный подход, чтобы передвинуть линию выше — и она оказалась ближе к точке.
- Б. Примените квадратический подход, чтобы передвинуть линию выше — и она оказалась ближе к точке.

Решение

Прогноз, который эта модель делает в данной точке, равен $\hat{y} = 2 \times 5 + 3 = 13$.

- А. Поскольку прогноз равен 13, что меньше метки 15, точка находится под прямой.

В этой модели наклон равен $m = 2$, а y -пересечение равно $b = 3$. Абсолютный метод включает в себя добавление $x\eta = 5 \times 0,01 = 0,05$ к наклону и $\eta = 0,01$ к y -пересечению и, таким образом, получение модели, соответствующей уравнению:

$$\hat{y} = 2,05x + 3,01.$$

- Б. Квадратический метод включает в себя прибавление $(y - \hat{y})x\eta = (15 - 13) \times 5 \times 0,01 = 0,1$ к наклону и $(y - \hat{y})\eta = (15 - 13) \times 0,01 = 0,02$ к y -пересечению, что дает модель, соответствующую уравнению:

$$\hat{y} = 2,1x + 3,02.$$

Глава 4. Оптимизация процесса обучения. Недообучение, переобучение, тестирование и регуляризация

Упражнение 4.1

Мы обучили четыре модели на одном и том же наборе данных с разными гиперпараметрами. В следующей таблице записали ошибки обучения и тестирования для каждой из моделей.

Модель	Ошибка обучения	Ошибка тестирования
1	0,1	1,8
2	0,4	1,2
3	0,6	0,8
4	1,9	2,3

- А. Какую модель вы бы выбрали для заданного набора данных?
- Б. Какая модель выглядит так, будто она недостаточно хорошо соответствует данным?
- В. Какая модель выглядит так, будто она переобучена?

Решение

- А. Лучшая модель — это модель с наименьшей ошибкой тестирования, то есть 3.
- Б. Модель 4 выглядит недообученной, потому что в ней допущены серьезные ошибки при обучении и тестировании.
- В. Модели 1 и 2 выглядят так, как будто они переобучены, потому что в них есть небольшие ошибки обучения, но большие ошибки тестирования.

Упражнение 4.2

Имеется следующий набор данных.

x	y
1	2,0
2	2,5
3	6,0
4	14,5
5	34,0

Мы обучаем модель полиномиальной регрессии, которая предсказывает значение y как $\hat{y}$, где

$$\hat{y} = 2x^2 - 5x + 4.$$

Если параметр регуляризации $\lambda = 0,1$, а функция ошибки, которую мы использовали для обучения этого набора данных, представляет собой среднее абсолютное значение (MAE), определите следующее.

- А. Ошибку регрессии лассо модели (с помощью нормы L1).
- Б. Ошибку гребневой регрессии модели (с использованием нормы L2).

Решение

Сначала нужно найти прогнозы, чтобы вычислить среднюю абсолютную ошибку модели. В следующей таблице мы можем найти прогноз, рассчитанный по формуле $\hat{y} = 2x^2 - 5x + 4$, и абсолютное значение разницы между прогнозом и меткой $|y - \hat{y}|$.

x	y	$\hat{y}$	$ y - \hat{y} $
1	2,0	1	1,0
2	2,5	2	0,5
3	6,0	7	1,0
4	14,5	16	1,5
5	34,0	29	5,0

Таким образом, средняя абсолютная ошибка представляет собой среднее значение чисел в четвертой строке, а именно:

$$1/5 (1 + 0,5 + 1 + 1,5 + 5) = 1,8.$$

- А. Сначала нужно найти норму L1 многочлена. Это сумма абсолютных значений непостоянных коэффициентов, а именно, $|2| + |-5| = 7$. Чтобы найти стоимость L1-регуляризации модели, мы складываем среднюю абсолютную ошибку и норму L1, умноженную на параметр регуляризации, и получаем $1,8 + 0,1 \times 7 = 2,5$.
- Б. Аналогично находим норму L1 многочлена, складывая квадраты непостоянных коэффициентов, чтобы получить $2^2 + (-5)^2 = 29$. Как и прежде, стоимость L2-регуляризации модели составляет $1,8 + 0,1 \times 29 = 4,7$.

Глава 5. Использование прямых для разделения точек. Алгоритм персептрона

Упражнение 5.1

Возьмем данные о пациентах с положительным или отрицательным тестом на COVID-19. У них наблюдаются следующие симптомы: кашель (C), температура (F), затрудненное дыхание (B) и утомляемость (T).

Пациент	Кашель (C)	Температура (F)	Затрудненное дыхание (B)	Утомляемость (T)	Диагноз (D)
1		X	X	X	Болен
2	X	X		X	Болен
3	X		X	X	Болен
4	X	X	X		Болен
5	X			X	Здоров
6		X	X		Здоров
7		X			Здоров
8				X	Здоров

Постройте модель персептрона для классификации такого набора данных.

Подсказка: можно использовать алгоритм персептрона. Но, возможно, вам удастся найти хорошую модель персептрона, которая сработает.

Решение

Если мы подсчитаем, сколько симптомов у каждого пациента, то заметим, что у больных наблюдается три симптома или более, в то время как у здоровых пациентов их два или менее. Таким образом, для прогнозирования диагноза D подойдет следующая модель:

$$\hat{D} = \text{step}(C + F + B + T - 2,5).$$

Упражнение 5.2

Подумайте над моделью персептрона, которая сопоставляет с точкой (x_1, x_2) прогноз $\hat{y} = \text{step}(2x_1 + 3x_2 - 4)$. Эта модель имеет пограничную прямую, описываемую уравнением $2x_1 + 3x_2 - 4 = 0$. У нас имеется точка $p = (1, 1)$ с меткой 0.

- А. Убедитесь, что точка p классифицирована моделью неверно.
- Б. Рассчитайте ошибку персептрона, которую дает модель в точке p .
- В. Используйте метод персептрона, чтобы получить новую модель, которая по-прежнему неправильно классифицирует p , но с меньшей ошибкой. В качестве скорости обучения можно взять $\eta = 0,01$.
- Г. Получите прогноз, данный новой моделью в точке p , после чего убедитесь, что полученная ошибка персептрона меньше предыдущей.

Решение

- А. Прогноз для точки p :

$$\hat{y} = \text{step}(2x_1 + 3x_2 - 4) = \text{step}(2 \times 1 + 3 \times 1 - 4) = \text{step}(1) = 1.$$

Поскольку метка точки равна 0, точка неправильно классифицирована.

- Б. Ошибка персептрона — это абсолютное значение оценки. Оценка составляет $2x_1 + 3x_2 - 4 = 2 \times 1 + 3 \times 1 - 4 = 1$, таким образом, ошибка персептрона равна 1.
- В. Веса модели равны 2, 3 и -4 , а координаты точки — $(1, 1)$. Метод персептрона выполняет следующее:
 - заменяет 2 на $2 - 0,01 \times 1 = 1,99$;
 - заменяет 3 на $3 - 0,01 \times 1 = 2,99$;
 - заменяет -4 на $-1 - 0,01 \times 1 = -4,01$.

Таким образом, новая модель — та, которая делает прогноз

$$\hat{y} = \text{step}(1,99x_1 + 2,99x_2 - 4,01).$$

- Г. Обратите внимание на то, что в нашей точке новый прогноз составляет $\hat{y} = \text{step}(1,99x_1 + 2,99x_2 - 4,01) = \text{step}(0,97) = 0$, что означает: модель по-прежнему неправильно классифицирует точку. Однако новая ошибка персептрона $|1,99 \times 1 + 2,99 \times 1 - 4,01| = 0,97$ меньше 1, предыдущей ошибки.

Упражнение 5.3

Перцептроны особенно часто используются для реализации логических вентилей, таких как AND или OR.

- А. Постройте перцептрон, моделирующий вентиль AND. Иными словами, создайте перцептрон, удовлетворяющий приведенному в таблице набору данных (где x_1, x_2 — это признаки, а y — метка).

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

- Б. Аналогичным образом построьте перцептрон, моделирующий OR, используя следующий набор данных.

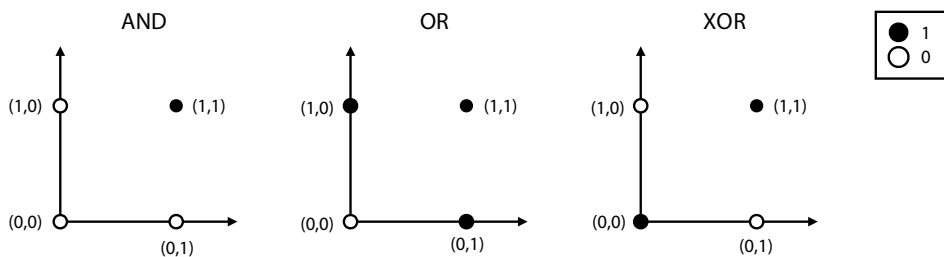
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

- В. Продемонстрируйте, что не существует перцептрона, который моделирует вентиль XOR, заданный следующим набором данных.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

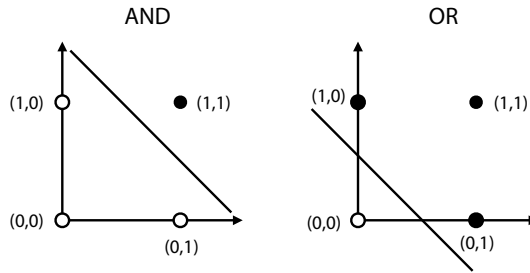
Решение

Для простоты нанесем точки данных на следующий график.



Обратите внимание на то, что классификатор персептрона — это именно та прямая, которая разделила бы черные и белые точки на приведенных графиках.

Для наборов данных AND и OR можно легко разделить черные и белые точки прямой, как показано далее.



- А. Многие уравнения подойдут для прямой, разделяющей набор данных AND. Мы же выберем прямую $x_1 + x_2 = 1,5$. Таким образом, персептрон, который классифицирует этот набор данных, делает прогноз $\hat{y} = \text{step}(x_1 + x_2 - 1,5)$.
- Б. Аналогично многие уравнения работают и для набора данных OR, и мы выбираем прямую $x_1 + x_2 = 0,5$. Уравнение для прогноза имеет вид $\hat{y} = \text{step}(x_1 + x_2 - 0,5)$.
- В. Обратите внимание на то, что набор данных для XOR невозможно разделить с помощью одной прямой. Таким образом, не существует модели персептрона, которая бы идеально соответствовала набору данных XOR. Однако разделить этот набор данных способна комбинация персептронов. Их также называют многослойными персептронами, или нейронными сетями, и мы рассмотрим их в главе 10. Если вам интересно, взгляните на упражнение 10.2.

Глава 6. Непрерывный подход к разделению точек. Логистические классификаторы

Упражнение 6.1

Стоматолог обучил логистический классификатор на наборе данных пациентов, чтобы спрогнозировать, есть ли у них больные зубы. Модель определила, что вероятность наличия больного зуба у пациента равна:

$$\sigma(d + 0,5c - 0,8),$$

где d — переменная, показывающая, был ли у пациента в прошлом другой больной зуб, c — переменная, указывающая, любит ли он сладости.

Например, если пациент любит конфеты, то $c = 1$, а если нет, то $c = 0$. Какова вероятность того, что у пациента, который ест конфеты и лечил больной зуб в прошлом году, имеется больной зуб?

Решение

Если пациент ест конфеты, то $c = 1$. Если пациент лечил кариес в прошлом году, то $d = 1$. Таким образом, согласно модели, вероятность того, что у пациента имеется разрушенный зуб:

$$\sigma(1 + 0,5 \times 1 - 0,8) = \sigma(0,7) = 0,668.$$

Упражнение 6.2

Рассмотрим логистический классификатор, который присваивает точке (x_1, x_2) прогноз $\hat{y} = \sigma(2x_1 + 3x_2 - 4)$, и точку $p = (1, 1)$ с меткой 0.

- А. Вычислите прогноз $\hat{y}$, который модель даст точке p .
- Б. Вычислите логарифмическую потерю, которую модель производит в точке p .
- В. Примените логистический метод, чтобы получить новую модель, которая приводит к меньшей логарифмической потере. Вы можете использовать значение скорости обучения $\eta = 0,1$.
- Г. Найдите прогноз, данный новой моделью в точке p , и убедитесь, что полученная логарифмическая потеря меньше исходной.

Решение

А. Прогноз $\hat{y} = \sigma(2 \times 1 + 3 \times 1 - 4) = \sigma(1) = 0,731$.

Б. Логарифмическая потеря составляет:

$$\begin{aligned} \text{логарифмическая потеря} &= -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y}) = \\ &= -0 \ln(0,731) - (1 - 0) \ln(1 - 0,731) = 1,313. \end{aligned}$$

В. Напомним, что метод персептрона для модели логистической регрессии с прогнозом $\hat{y} = \sigma(w_1x_1 + w_2x_2 + b)$ дает следующие новые веса:

- $w'_i = w_i + \eta(y - \hat{y})x_i$ при $i = 1, 2$;
- $b' = b + \eta(y - \hat{y})$ при $i = 1, 2$.

Вот значения, которые необходимо включить в предыдущие формулы:

- $y = 0$;
- $\hat{y} = 0,731$;
- $w_1 = 2$;
- $w_2 = 3$;
- $b = -4$;
- $\eta = 0,1$;
- $x_1 = 1$;
- $x_2 = 1$.

Мы получаем следующие новые веса для классификатора:

- $w'_1 = 2 + 0,1 \times (0 - 0,731) \times 1 = 1,9269$;
- $w'_2 = 3 + 0,1 \times (0 - 0,731) \times 1 = 2,9269$;
- $b' = -4 + 0,1 \times (0 - 0,731) = -4,0731$.

Таким образом, новый классификатор — тот, который делает прогноз $\hat{y} = \sigma(1,9269x_1 + 2,9269x_2 - 4,0731)$.

Прогноз в точке p равен $\hat{y} = \sigma(1,9269 \times 1 + 2,9269 \times 1 - 4,0731) = 0,686$.

Обратите внимание: поскольку метка равна 0, прогноз улучшился с исходного значения 0,731 до фактического значения 0,686.

- Г. Логарифмические потери для этого прогноза составляют $-y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y}) = -0 \ln(0,686) - (1 - 0) \ln(1 - 0,686) = 1,158$. Обратите внимание на то, что это меньше, чем первоначальная логарифмическая потеря 1,313.

Упражнение 6.3

Используя первую модель в упражнении 6.2, постройте точку, для которой прогноз равен 0,8.

ПОДСКАЗКА Сначала найдите оценку, которая даст прогноз 0,8, и вспомните, что прогноз равен $\hat{y} = \sigma(\text{мера})$.

Решение

Во-первых, нужно найти такую оценку, чтобы $\sigma(\text{мера}) = 0,8$. Это эквивалентно:

$$\frac{e^{\text{мера}}}{1 + e^{\text{мера}}} = 0,8;$$

$$e^{\text{мера}} = 0,8(1 + e^{\text{мера}});$$

$$e^{\text{мера}} = 0,8 + 0,8e^{\text{мера}};$$

$$0,2e^{\text{мера}} = 0,8;$$

$$e^{\text{мера}} = 4;$$

$$\text{мера} = \ln(4) = 1,386.$$

Напомним, что для точки (x_1, x_2) оценка равна $2x_1 + 3x_2 - 4$. Многие точки (x_1, x_2) дают оценку 1,386, но для удобства давайте выберем ту, в которой $x_2 = 0$. Нам нужно решить уравнение $2x_1 + 3 \times 0 - 4 = 1,386$, решение которого — $x_1 = 2,693$. Таким образом, точка, дающая прогноз 0,8, — это точка $(2,693, 0)$.

Глава 7. Как оценивать классификационные модели? Достоверность и ее друзья

Упражнение 7.1

Видеохостинг установил, что конкретному пользователю нравятся видео с животными, и только они. На рисунке мы видим рекомендации, которые такой пользователь получает, заходя на сайт.



Если это все данные, которыми мы располагаем согласно модели, ответьте на следующие вопросы.

- А. Какова достоверность модели?
- Б. Какова полнота модели?
- В. Какова точность модели?
- Г. Какова F_1 -мера модели?
- Д. Можно ли сказать, что это хорошая рекомендательная модель?

Решение

Прежде всего напомним матрицу неточностей. В этом случае мы помечаем видеоролики, посвященные животным, как *положительные*, а рекомендуемые видео — как *прогнозируемые положительные*.

- Есть четыре рекомендуемых видео. Три из них посвящены животным, что значит — это хорошие рекомендации. Оставшееся не о животных, так что это ложноположительный результат.
- Есть шесть видеороликов, которые не рекомендуются. Два из них посвящены животным и должны были быть рекомендованы. Таким образом, это ложноотрицательные результаты. Остальные четыре не о животных, поэтому было правильно не рекомендовать их.

Таким образом, матрица неточностей выглядит, как показано в таблице.

	Прогнозируемые положительные (рекомендуются)	Прогнозируемые отрицательные (не рекомендуются)
Положительные (о животных)	3	2
Отрицательные (не о животных)	1	4

Теперь мы можем рассчитать показатели.

А. Достоверность — $7/10 = 0,7$.

Б. Полнота — $3/5 = 0,6$;

В. Точность — $3/4 = 0,75$;

Г. F_1 -мера — $\frac{2(3/4)(3/5)}{(3/4)+(3/5)} = \frac{2}{3} = 0,67$.

Д. Это субъективный ответ. Медицинская модель с такими показателями может оказаться недостаточно хорошей. Однако, если модель рекомендаций обладает приличной достоверностью, точностью и полнотой, она считается хорошей, потому что допустить пару ошибок в модели рекомендаций не так уж важно.

Упражнение 7.2

Найдите чувствительность и избирательность медицинской модели со следующей матрицей неточностей.

	Прогноз	
	«Болен»	«Здоров»
Болен	120	22
Здоров	63	795

Решение

Чувствительность — это количество правильно предсказанных больных, разделенное на общее количество больных. Это $120/142 = 0,845$.

Избирательность — это количество правильно спрогнозированных здоровых людей, разделенное на общее их число. Это $795/858 = 0,927$.

Упражнение 7.3

Для следующих моделей установите, какая ошибка более серьезна — ложноположительная или ложноотрицательная. Исходя из этого, определите, на каком из двух показателей — точности или полноте — мы должны сделать акцент при оценке моделей.

1. Система рекомендаций по фильмам, которая предсказывает, будет ли пользователь смотреть фильм.
2. Модель распознавания изображений, применяемая в беспилотных автомобилях и определяющая, есть ли на изображении пешеход.
3. Голосовой помощник для дома, который прогнозирует, отдал ли пользователь ему приказ.

Решение

ОБРАТИТЕ ВНИМАНИЕ Во всех следующих моделях ложноотрицательный и ложноположительный результаты являются плохими, и мы стремимся их избежать. Однако мы приводим аргумент в пользу того, какой из двух вариантов хуже. Все это концептуальные вопросы, поэтому если у вас есть другая идея, то, пока вы способны хорошо ее аргументировать, она работает! Именно такие дискуссии возникают в команде специалистов по обработке данных, и важно иметь здравые мнения и аргументы, подтверждающие каждую точку зрения.

1. В этой модели мы помечаем фильмы, которые пользователь хочет посмотреть, как положительные. Ложноположительный результат возникает каждый раз, когда мы рекомендуем фильм, который он не хочет смотреть. Ложноотрицательный результат возникает каждый раз, когда пользователь хочет посмотреть фильм, но его нет в рекомендациях. Что хуже — ложноотрицательный или ложноположительный результат? Поскольку на главной странице отображается много рекомендаций и пользователь игнорирует большинство из них, эта модель имеет много ложноотрицательных прогнозов, которые несильно влияют на пользовательский опыт. Однако, если есть отличный фильм, который человек хотел бы посмотреть, очень важно порекомендовать его. Следовательно, в этой модели ложноотрицательный результат хуже ложноположительного, поэтому мы должны оценить ее с помощью **полноты**.
2. В этой модели мы обозначаем наличие пешехода как положительное. Ложноположительный результат возникает, когда пешехода нет, но автомобиль думает, что он есть. Ложноотрицательный результат возникает, когда автомобиль не видит пешехода, который находится перед автомобилем. В случае ложноотрицательного результата автомобиль может сбить пешехода. В случае ложноположительного прогноза он может затормозить без необходимости, что способно как привести, так и не привести к аварии. Хотя и то и другое серьезно, гораздо хуже сбить пешехода. Следовательно, в этой модели ложноотрицательный результат хуже ложноположительного, поэтому мы должны оценить ее с помощью **полноты**.
3. В этой модели мы помечаем голосовую команду как положительную. Ложное срабатывание возникает, когда пользователь не разговаривает с голосовым помощником, а он отвечает. Ложноотрицательный результат возникает, когда пользователь разговаривает с голосовым помощником, а тот не отвечает. Лично я предпочитаю лишний раз повторить что-то своему голосовому помощнику, чем слушать его разговор со мной ни с того ни с сего. Таким образом, в этой модели ложноположительный результат хуже, чем ложноотрицательный, поэтому следует оценивать ее с помощью **точности**.

Упражнение 7.4

У нас имеются следующие модели.

1. Модель для беспилотного автомобиля, обнаруживающая пешехода на основе изображения с камеры машины.
2. Медицинская модель для диагностики смертельной болезни на основе симптомов.
3. Система рекомендаций, основанная на ранее просмотренных фильмах.
4. Голосовой помощник, который определяет, нужна ли пользователю помощь, на основе голосовой команды.
5. Модель обнаружения нежелательной почты, которая определяет, является ли электронное письмо спамом, на основе слов, из которых оно состоит.

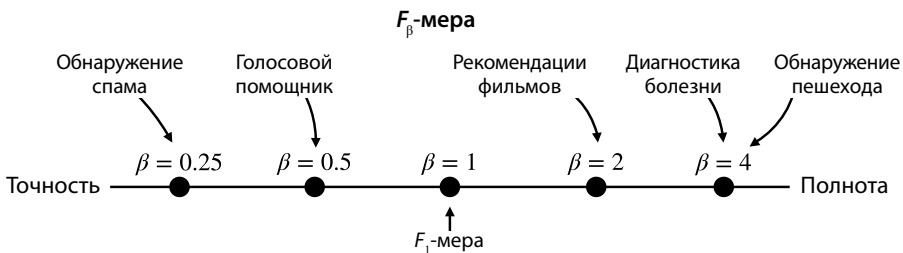
Перед нами стоит задача оценить данные модели с использованием F_β -меры. Вместе с тем значения β не даны. Какое значение β вы бы взяли для оценки каждой из моделей?

Решение

Помните, что для моделей, в которых точность важнее, чем отзыв, мы используем F_β -меру с небольшим значением β . И напротив, для моделей, в которых полнота важнее точности, применяем F_β -меру с большим значением β .

ОБРАТИТЕ ВНИМАНИЕ Если вы получили оценки, не такие, как здесь, это вполне нормально, когда у вас есть веские аргументы в пользу выбора между точностью и полнотой, а также в пользу собственного значения β .

- Для беспилотных автомобилей и медицинских моделей полнота чрезвычайно важна, потому что нужно получить как можно меньше ложно-отрицательных результатов. Таким образом, я бы использовал большое значение β , например 4.
- Для модели обнаружения спама важна точность, потому что хочется, чтобы было как можно меньше ложных срабатываний. Таким образом, я бы взял небольшое значение β , например 0,25.
- Для системы рекомендаций полнота более важна (см. упражнение 7.3), хотя точность также имеет значение. Таким образом, я бы использовал большое значение β , например 2.
- Для голосового помощника точность важнее, хотя полнота также имеет значение (см. упражнение 7.3). Таким образом, я бы взял небольшое значение β , например 0,5.



Глава 8. Как использовать вероятность по максимуму. Наивная байесовская модель

Упражнение 8.1

Для каждой пары событий А и В определите, зависимые они или независимые. Для п. А–Г приведите математическое обоснование. Для п. Д и Е дайте устное обоснование.

Честно три раза подбрасываем монету.

А: первый бросок — орел. Б: третий бросок — решка.

Б: первый бросок — орел. Б: среди трех бросков нечетное количество — орел.

Выбрасывание двух кубиков.

В: на первом кубике 1. Б: на втором кубике 2.

Г: на первом кубике 3. Б: на втором выпало более высокое значение, чем на первом.

Для следующего примера приведите устное обоснование. Предположим, мы живем там, где есть смена времен года.

Д: на улице идет дождь. Б: сегодня понедельник.

Е: на улице идет дождь. Б: сейчас июнь.

Решение

Некоторые из следующих выводов можно сделать с помощью интуиции. Однако иногда она подводит при определении того, являются ли два события независимыми. По этой причине, если события не явно независимые, мы будем придерживаться проверки, являются ли два события А и В независимыми, если $P(A \cap B) = P(A)P(B)$.

А. Поскольку А и В соответствуют подбрасыванию разных монет, они являются независимыми событиями.

Б. $P(A) = 1/2$, потому что подбрасывание монеты приводит к двум одинаково вероятным сценариям. Для вычисления $P(B)$ мы будем использовать обозначение h для орлов и t для решек. Таким образом, событие hth соответствует результатам «орел, решка, орел». Если мы подбросим три монеты, восемь одинаково вероятных возможностей будут таковы: {hhh, hht, hth, htt, thh, tht, tth, ttt}. $P(B) = 4/8 = 1/2$, потому что среди восьми одинаково вероятных возможностей (hhh, hht, hth, htt, thh, tht, tth, ttt) только четыре имеют нечетное число орлов, а именно {hhh, hth, tht, tth}. $P(A \cap B) = 2/8 = 1/4$, потому что среди восьми вероятностей только две удовлетворяют тому, что первая показывает на орел, и есть нечетное количество решек, а именно {hhh, htt}. $P(A)P(B) = 1/4 = P(A \cap B)$, поэтому события А и В независимые.

- В. Поскольку А и Б соответствуют выбрасыванию разных кубиков, это независимые события.
- Г. $P(A) = 1/6$, потому что это соответствует броску кубика и получению определенного значения.
 $P(B) = 5/12$ по следующей причине. Обратите внимание на то, что 36 одинаково вероятных возможностей для точек на двух кубиках равны {11, 12, 13... 56, 66}. В шести из них два кубика показывают одинаковое значение. Остальные 30 соответствуют 15, в которых первое значение выше, и 15, в которых второе значение выше. Таким образом, существуют 15 сценариев, в которых второй кубик показывает более высокое значение, чем третий, поэтому $P(B) = 15/36 = 5/12$.
 $P(A \cap B) = 1/2$ по следующей причине. Если первый кубик выпадет на 3, у нас будет в общей сложности шесть одинаково вероятных сценариев, а именно {31, 32, 33, 34, 35, 36}. Второе число выше в трех из них. Таким образом, $P(A \cap B) = 3/6 = 1/2$. Поскольку $P(A)P(B) \neq P(A \cap B)$, события А и Б зависят друг от друга.
- Д. Для этой задачи предположим, что А и Б независимы, то есть погода не зависит от дня недели. Это справедливое предположение, учитывая наши знания о погоде, но если бы мы хотели быть более уверенными, то могли бы посмотреть на наборы данных о погоде и проверить это, вычислив соответствующие вероятности.
- Е. Поскольку мы условились считать, что живем там, где есть смена времен года, июнь — это лето в северном полушарии и зима в южном. В зависимости от того, где мы живем, зимой или летом может идти больше дождей. Таким образом, можно предположить, что события А и Б зависят друг от друга.

Упражнение 8.2

Представим офис, куда мы регулярно ходим за рабочими бумагами. В офисе работают два клерка, Аиша и Бето. Мы знаем, что Аиша работает там три дня в неделю, а Бето — два оставшихся, однако расписание меняется каждую неделю, поэтому мы никогда не знаем, в какие три дня на месте Аиша, а в какие два — Бето.

- А. Если мы придем в офис в случайный день, какова вероятность того, что клерком будет Аиша?
 Мы смотрим в окно и замечаем, что клерк одет в красный свитер, хотя не можем точно рассмотреть, кто именно. Мы часто посещаем этот офис, поэтому знаем, что Бето носит красное чаще, чем Аиша. На самом деле Аиша носит красное в один день из трех (треть времени), а Бето — в один день из двух (половину времени).
- Б. Какова вероятность того, что сегодня работает Аиша, при условии, что клерк одет в красное?

Решение

Используем следующие обозначения для событий:

- A — случай, когда клерк — Аиша;
- B — случай, когда клерк — Бето;
- R — случай, когда клерк одет в красное.

А. Поскольку Аиша работает в офисе три дня, а Бето — два, вероятность того, что Аиша — сегодняшний клерк, равна $P(A) = 3/5$, или 60 %. Кроме того, вероятность того, что клерком будет Бето, составляет $P(B) = 2/5$, или 40 %.

Б. Интуиция говорит нам: поскольку Бето носит красное чаще, чем Аиша, можно предположить, что вероятность того, что клерк — Аиша, ниже, чем в п. А. Проверим, согласна ли с этим математика. Мы знаем, что клерк одет в красное, поэтому нужно найти вероятность того, что клерк — Аиша, *зная, что* клерк одет в красное. Это $P(A | R)$.

Вероятность того, что Аиша наденет красное, равна $1/3$, поэтому $P(R | A) = 1/3$. Вероятность того, что Бето наденет красное, составляет $1/2$, так что $P(R | B) = 1/2$.

Мы можем использовать теорему Байеса для получения

$$P(A | R) = \frac{P(R | A)P(A)}{P(R | A)P(A) + P(R | B)P(B)} = \frac{(1/3)(3/5)}{(1/3)(3/5) + (1/2)(2/5)} = \frac{1/5}{1/5 + 1/5} = \frac{1}{2},$$

или 50 %.

Аналогичный расчет показывает, что вероятность того, что клерком является Бето, равна $P(B | R) = 1/2$, или 50 %.

По сути, вероятность того, что клерк — Аиша, меньше, чем вероятность, полученная в п. А, так что наша интуиция оказалась права.

Упражнение 8.3

Возьмем данные о пациентах с положительным или отрицательным тестом на COVID-19. У них следующие симптомы: кашель (C), температура (F), затрудненное дыхание (B) и утомляемость (T).

Пациент	Кашель (C)	Температура (F)	Затрудненное дыхание (B)	Утомляемость (T)	Диагноз
1		X	X	X	Болен
2	X	X		X	Болен
3	X		X	X	Болен
4	X	X	X		Болен
5	X			X	Здоров
6		X	X		Здоров
7		X			Здоров
8				X	Здоров

Цель этого упражнения — построить наивную байесовскую модель, которая предсказывает диагноз по симптомам. Используйте наивный алгоритм Байеса, чтобы найти следующие вероятности.

ОБРАТИТЕ ВНИМАНИЕ Для последующих вопросов: симптомы, которые не упомянуты, нам совершенно неизвестны. Например, если мы знаем, что у пациента кашель, но ничего не говорится о жаре, это не значит, что у пациента нет жара.

- А. Вероятность того, что пациент болен, учитывая, что у него кашель.
- Б. Вероятность того, что пациент болен, учитывая, что он не утомлен.
- В. Вероятность того, что пациент болен, учитывая, что у него кашель и жар.
- Г. Вероятность того, что пациент болен, учитывая, что у него кашель и жар, но нет затрудненного дыхания.

Решение

Для этой задачи у нас есть следующие события:

- С — у пациента кашель;
- F — у пациента высокая температура;
- В — у пациента трудности с дыханием;
- Т — пациент утомлен;
- S — пациенту поставлен диагноз «болен»;
- Н — пациенту поставлен диагноз «здоров».

Кроме того, A^c обозначает дополнение (противоположность) события А. Так, например, T^c представляет событие, при котором пациент не устал.

Сначала вычислим $P(S)$ и $P(H)$. Обратите внимание: поскольку набор данных содержит четверых здоровых и четверых больных пациентов, обе эти (априорные) вероятности составляют $1/2$, или 50 %.

- А. Поскольку у четырех пациентов кашель и трое из них больны, $P(S|C) = 3/4$, или 75 %.

Мы можем использовать теорему Байеса следующим образом: сначала вычисляем $P(S|C) = 3/4$, заметив, что есть четыре больных пациента и у троих из них кашель. Замечаем также, что $P(C|H) = 1/4$, потому что есть четыре здоровых пациента и только один из них кашляет.

Теперь можем применить формулу:

$$P(S|C) = \frac{P(C|S)P(S)}{P(C|S)P(S) + P(C|H)P(H)} = \frac{(3/4)(1/2)}{(3/4)(1/2) + (1/4)(1/2)} = \frac{3}{4}.$$

- Б. Поскольку три пациента не утомлены и только один из них болен, $P(S|T^c) = 1/3$, или 33,3 %.

Мы так же, как и раньше, можем использовать теорему Байеса. Обратите внимание на то, что $P(T^c|S) = 1/4$, потому что только один из четверых

заболевших пациентов не утомлен. Кроме того, $P(T^c | H) = 2/4$, потому что двое из четверых здоровых пациентов не утомлены. По теореме Байеса,

$$P(S | T^c) = \frac{P(T^c | S)P(S)}{P(T^c | S)P(S) + P(T^c | H)P(H)} = \frac{(1/4)(1/2)}{(1/4)(1/2) + (2/4)(1/2)} = \frac{1/8}{1/8 + 2/8} = \frac{1}{3}.$$

- В. $C \cap F$ представляет собой событие, когда у пациента кашель и температура, поэтому нам нужно рассчитать $P(S | C \cap F)$.

Напомним, что, согласно п. А, $P(S | C) = 3/4$ и $P(C | H) = 1/4$.

Теперь нам нужно вычислить $P(F | S)$ и $P(F | H)$. Обратите внимание: поскольку имеются четверо больных пациентов и у троих из них высокая температура, $P(F | S) = 3/4$. Аналогично, у двоих из четверых здоровых пациентов наблюдается жар, поэтому $P(F | H) = 2/4 = 1/2$.

Мы готовы использовать наивный алгоритм Байеса для оценки вероятности того, что пациент болен, учитывая, что у него кашель и высокая температура. С помощью формулы из раздела «А как быть, если слова два? Наивный байесовский алгоритм» главы 8 получаем:

$$\begin{aligned} P(S | C \cap F) &= \frac{P(C | S)P(F | S)P(S)}{P(C | S)P(F | S)P(S) + P(C | H)P(F | H)P(H)} = \\ &= \frac{(3/4)(3/4)(1/2)}{(3/4)(3/4)(1/2) + (1/4)(2/4)(1/2)} = \frac{9/32}{9/32 + 2/32} = \frac{9}{11}, \end{aligned}$$

или 81,82 %.

- Г. Для этого упражнения нам нужно найти $P(S | C \cap F \cap B^c)$.

Поскольку есть четыре больных пациента и только у одного из них нет затруднения дыхания, то $P(B^c | S) = 1/4$. Аналогично есть четыре здоровых пациента, и у троих из них нет трудностей с дыханием, поэтому $P(B^c | H) = 3/4$.

Как и прежде, можем использовать наивный алгоритм Байеса:

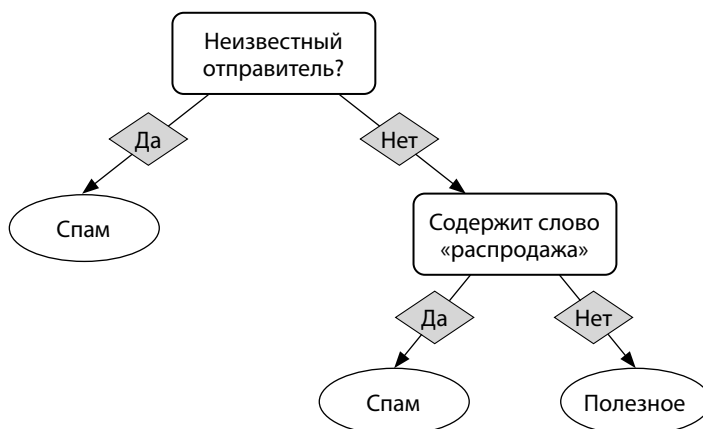
$$\begin{aligned} P(S | C \cap F \cap B^c) &= \\ &= \frac{P(C | S)P(F | S)P(B^c | S)P(S)}{P(C | S)P(F | S)P(B^c | S)P(S) + P(C | H)P(F | H)P(B^c | H)P(H)} = \\ &= \frac{(3/4)(3/4)(1/4)(1/2)}{(3/4)(3/4)(1/4)(1/2) + (1/4)(2/4)(3/4)(1/2)} = \frac{9/32}{9/32 + 6/32} = \frac{9}{15}, \end{aligned}$$

или 60 %.

Глава 9. Разбиение данных согласно ответам на вопросы. Деревья решений

Упражнение 9.1

В следующей модели дерева решений по обнаружению спама определите, будет ли электронное письмо от вашей мамы с темой «Пожалуйста, зайди в магазин, там распродажа» классифицироваться как спам.



Решение

Сначала мы проверяем, известен ли нам отправитель. Поскольку это наша мама, отправитель известен. Таким образом, берем ветвь справа. Мы должны проверить, содержит ли электронное письмо слово «распродажа». Содержит, поэтому классификатор неправильно классифицирует его как спам.

Упражнение 9.2

Наша цель — построить модель дерева решений, определяющую, являются ли транзакции по кредитным картам мошенническими. Мы используем приведенный далее набор данных транзакций по кредитным картам со следующими признаками:

- **сумма** — сумма транзакции;
- **надежный поставщик** — у компании, выпускающей кредитные карты, есть список надежных поставщиков. Эта переменная указывает, входит ли поставщик в данный список.

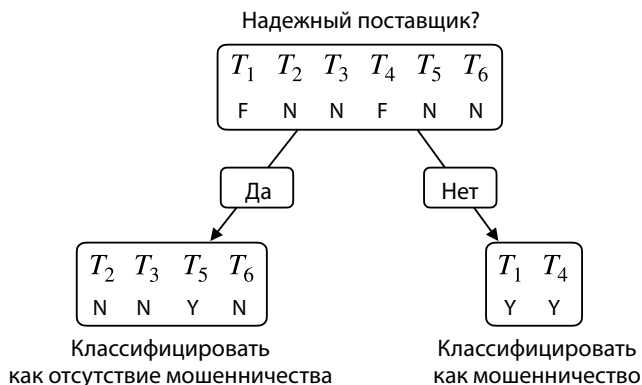
Транзакция	Сумма, в долларах	Поставщик	Мошенничество
1	100	Ненадежный	Да
2	100	Надежный	Нет
3	10 000	Надежный	Нет
4	10 000	Ненадежный	Да
5	5000	Надежный	Да
6	100	Надежный	Нет

Создайте первый узел дерева решений в соответствии со следующими спецификациями.

- А. Применение индекса примесей Джини.
- Б. Использование энтропии.

Решение

В обоих случаях наилучшее разделение достигается с использованием признака «Надежный поставщик», как показано на рисунке.



Назовем транзакции T_1, T_2, T_3, T_4, T_5 и T_6 . Вначале посмотрим на все разделения, которые можем сделать. Разделение с использованием надежного поставщика не сложно, потому что это категориальная переменная с двумя категориями. Столбец суммы более сложен — мы можем задействовать его для разделения данных двумя возможными способами. Один — это когда пороговое значение составляет от 100 до 5000 долларов, а другой — когда оно составляет от 5000 до 10 000 долларов. Подводя итог, можно сказать, что все возможные разделения таковы:

- **сумма 1**, где предельное значение составляет от 100 до 5000 долларов. Двумя классами здесь будут $\{T_1, T_2, T_6\}$ и $\{T_3, T_4, T_5\}$;
- **сумма 2**, где предельное значение составляет некоторую величину от 5000 до 10 000 долларов. Двумя классами здесь являются $\{T_1, T_2, T_5, T_6\}$ и $\{T_3, T_4\}$;
- **надежный поставщик** — два класса, «надежный» и «ненадежный», что эквивалентно $\{T_2, T_3, T_5, T_6\}$ и $\{T_1, T_4\}$.

А. Рассчитаем индекс примеси Джини для каждого из следующих трех разделений.

- **Сумма 1** — предельная стоимость от 100 до 5000 долларов.

Для первого класса $\{T_1, T_2, T_6\}$ метки в столбце «Мошенничество» — это {«да», «нет», «нет»}. Индекс примеси Джини этого разделения составляет $1 - (1/3)^2 - (2/3)^2 = 4/9$. Для второго класса $\{T_3, T_4, T_5\}$ метки в столбце «Мошенничество» — это {«нет», «да», «да»}. Индекс примеси Джини этого разделения составляет: $1 - (2/3)^2 - (1/3)^2 = 4/9$. Таким образом, взвешенный индекс примесей Джини для этого разделения составляет: $(3/6)(4/9) + (3/6)(4/9) = 4/9 = 0,444$.

- **Сумма 2** — предельная сумма от 5000 до 10 000 долларов.

Для первого класса $\{T_1, T_2, T_5, T_6\}$ метки в столбце «Мошенничество» — это {«да», «нет», «да», «нет»}. Индекс примеси Джини этого разделения составляет: $1 - (2/4)^2 - (2/4)^2 = 1/2$. Для второго класса $\{T_3, T_4\}$ метки в столбце «Мошенничество» — это {«нет», «да»}. Индекс примеси Джини этого разделения составляет: $1 - (1/2)^2 - (1/2)^2 = 1/2$. Таким образом, взвешенный индекс примесей Джини для этого разделения составляет: $(4/6)(1/2) + (2/6)(1/2) = 1/2 = 0,5$.

- **Надежный поставщик.** Для первого класса $\{T_2, T_3, T_5, T_6\}$ метки в столбце «Мошенничество» — {«нет», «нет», «да», «нет»}. Индекс примеси Джини этого разделения составляет: $1 - (3/4)^2 - (1/4)^2 = 6/16$. Для второго класса $\{T_1, T_4\}$ метки в столбце «Мошенничество» — {«да», «да»}. Индекс примеси Джини этого разделения равен: $1 - 1^2 = 0$. Таким образом, взвешенный индекс примесей Джини для этого разделения составляет: $(4/6)(6/16) + (2/6) \times 0 = 1/4 = 0,25$.

Обратите внимание на то, что из этих трех значений наименьшее — 0,25, что соответствует столбцу «Поставщик». Это означает, что лучший способ разделить эти данные — использовать признак «Надежный поставщик».

Б. На данный момент мы проделали большую часть тяжелой работы. Далее станем придерживаться той же процедуры, что и в п. А, за исключением вычисления на каждом этапе энтропии вместо индекса примеси Джини.

- **Сумма 1** — предельная стоимость от 100 до 5000 долларов США.

Энтропия множества {«да», «нет», «нет»} составляет:

$$-\frac{1}{3} \log_2 \left(\frac{1}{3} \right) - \frac{2}{3} \log_2 \left(\frac{2}{3} \right) = 0,918.$$

Энтропия множества {«нет», «да», «да»} также:

$$-\frac{2}{3} \log_2 \left(\frac{2}{3} \right) - \frac{1}{3} \log_2 \left(\frac{1}{3} \right) = 0,918.$$

Таким образом, взвешенная энтропия для этого разделения: $(3/6) \times 0,918 + (3/6) \times 0,918 = 0,918$.

- **Сумма 2** — предельная сумма от 5000 до 10 000 долларов США.

Энтропия множества {«да», «нет», «да», «нет»} составляет:

$$-\frac{2}{4}\log_2\left(\frac{2}{4}\right) - \frac{2}{4}\log_2\left(\frac{2}{4}\right) = 1.$$

Энтропия множества {«нет», «да»} составляет:

$$-\frac{1}{2}\log_2\left(\frac{1}{2}\right) - \frac{1}{2}\log_2\left(\frac{1}{2}\right) = 1.$$

Таким образом, взвешенная энтропия для этого разделения равна:
 $(4/6) \times 1 + (2/6) \times 1 = 1.$

- **Надежный поставщик.**

Энтропия множества {«нет», «нет», «да», «нет»} составляет:

$$-\frac{1}{4}\log_2\left(\frac{1}{4}\right) - \frac{3}{4}\log_2\left(\frac{3}{4}\right) = 0,811.$$

Энтропия множества {«да», «да»} составляет: $-\frac{2}{2}\log_2\left(\frac{2}{2}\right) = 0.$

Таким образом, взвешенная энтропия для этого разделения равна:
 $(4/6) \times 0,811 + (2/6) \times 0 = 0,541.$

Обратите внимание: наименьшая из трех энтропия равна 0,541, что соответствует столбцу «Поставщик». Таким образом, лучший способ разделить эти данные — использовать признак «Надежный поставщик».

Упражнение 9.3

Далее приведен набор данных о пациентах, у которых был положительный или отрицательный результат теста на COVID-19. У них следующие симптомы: кашель (C), температура (F), затрудненное дыхание (B) и утомляемость (T).

Пациент	Кашель (C)	Температура (F)	Затрудненное дыхание (B)	Утомляемость (T)	Диагноз
1		X	X	X	Болен
2	X	X		X	Болен
3	X		X	X	Болен
4	X	X	X		Болен
5	X			X	Здоров
6		X	X		Здоров
7		X			Здоров
8				X	Здоров

Используя достоверность, постройте дерево решений высотой 1 (пенек решений), которое классифицирует эти данные. Какова достоверность этого классификатора на наборе данных?

Решение

Давайте назовем пациентов P1–P8. Больные пациенты будут обозначаться буквой s , а здоровые — буквой h .

Сначала обратим внимание на то, что первым разделением может быть любой из четырех признаков, С, F, В или Т. Сначала рассчитаем точность классификатора, полученного разделением данных по признаку С, а именно того, который строим на основе вопроса: «Есть ли у пациента кашель?»

Разделение на основе функции С:

- пациенты с кашлем — $\{P_2, P_3, P_4, P_5\}$, их метки — $\{s, s, s, h\}$;
- пациенты без кашля — $\{P_1, P_6, P_7, P_8\}$, их метки — $\{s, h, h, h\}$.

Глядя на них, мы видим, что наиболее точным классификатором (только на основе функции С) является тот, который классифицирует каждого человека с кашлем как больного, а каждого человека без кашля — как здорового. Этот классификатор правильно классифицирует шестерых из восьми пациентов (трое больных и трое здоровых), поэтому его достоверность составляет $6/8$, или 75 %.

Теперь проделаем ту же процедуру с тремя другими признаками.

Разделение на основе функции F:

- пациенты с жаром — $\{P_1, P_2, P_4, P_6, P_7\}$, их метки — $\{s, s, s, h, h\}$;
- пациенты без жара — $\{P_3, P_5, P_8\}$, их метки — $\{s, h, h\}$.

Мы видим, что наиболее точным классификатором (только на основе функции F) является тот, который определяет каждого пациента с температурой как больного, а каждого пациента без температуры — как здорового. Он правильно классифицирует пятерых из восьми пациентов (трое больных и двое здоровых), поэтому его достоверность составляет $5/8$, или 62,5 %.

Разделение на основе функции В:

- пациенты с затрудненным дыханием — $\{P_1, P_3, P_4, P_5\}$, их метки — $\{s, s, s, h\}$;
- пациенты без затрудненного дыхания — $\{P_2, P_6, P_7, P_8\}$, их метки — $\{s, h, h, h\}$.

Мы видим, что наиболее точным классификатором (основанным только на признаке В) является тот, который определяет каждого пациента, испытывающего трудности с дыханием, как больного, а каждого без затруднений с дыханием — как здорового. Он правильно классифицирует шестерых из восьми пациентов (трое больных и трое здоровых), поэтому его достоверность составляет $6/8$, или 75 %.

Разделение на основе функции Т:

- пациенты с утомляемостью — $\{P_1, P_2, P_3, P_5, P_8\}$, их метки — $\{s, s, s, h, h\}$;
- пациенты без утомляемости — $\{P_4, P_6, P_7\}$, их метки — $\{s, h, h\}$.

Мы видим, что наиболее точным классификатором (основанным только на функции Т) является тот, который определяет каждого пациента с утомляемостью как больного, а каждого пациента без утомляемости — как здорового. Он правильно классифицирует пятерых из восьми пациентов (трое больных и двое здоровых), поэтому его достоверность составляет $5/8$, или 62,5 %.

Обратите внимание на то, что два признака, которые дают наилучшую достоверность, — это С (кашель) и В (затрудненное дыхание). Дерево решений выберет один из них случайным образом. Давайте возьмем первый, С. Разделив данные с помощью признака С, мы получим следующие два набора данных:

- пациенты с кашлем — $\{P_2, P_3, P_4, P_5\}$, их метки — $\{s, s, s, h\}$;
- пациенты без кашля — $\{P_1, P_6, P_7, P_8\}$, их метки — $\{s, h, h, h\}$.

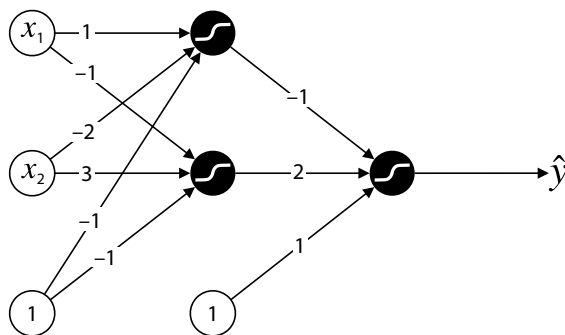
Это дает нам дерево глубиной 1, которое классифицирует данные с достоверностью 75 %. Дерево изображено на рисунке.



Глава 10. Комбинирование ради усиления. Нейронные сети

Упражнение 10.1

На рисунке показана нейронная сеть, в которой все активации являются сигмоидными функциями.



Каким будет прогноз этой нейронной сети для входных данных (1, 1)?

Решение

Назовем выходы средних узлов h_1 и h_2 . Они рассчитываются следующим образом:

$$\begin{aligned} h_1 &= \sigma(1x_1 - 2x_2 - 1); \\ h_2 &= \sigma(-1x_1 + 3x_2 - 1). \end{aligned}$$

Подставляя $x_1 = 1$ и $x_2 = 1$, получаем следующее:

$$\begin{aligned} h_1 &= \sigma(-2) = 0,119; \\ h_2 &= \sigma(1) = 0,731. \end{aligned}$$

Последний слой — это

$$\hat{y} = \sigma(-1h_1 + 2h_2 + 1).$$

Заменяя значения, рассчитанные ранее для h_1 и h_2 , получим:

$$\hat{y} = \sigma(-0,119 + 2 \times 0,731 + 1) = \sigma(2,343) = 0,912.$$

Таким образом, выходной сигнал нейронной сети равен 0,912.

Упражнение 10.2

Как мы узнали из упражнения 5.3, невозможно построить персептрон, имитирующий элемент XOR. Другими словами, невозможно сопоставить следующий набор данных с персептроном и получить 100%-ную достоверность.

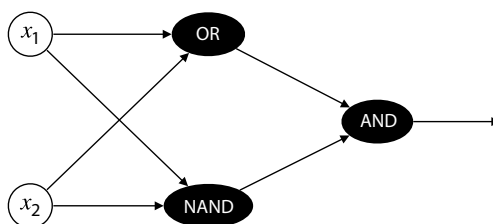
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Это связано с тем, что набор данных не является линейно разделяемым. Используя нейронную сеть глубиной 2, постройте персептрон, который имитирует элемент XOR, показанный ранее. Для получения дискретных выходных данных в качестве функций активации примените ступенчатую функцию вместо сигмоидной.

ПОДСКАЗКА Это трудно сделать с помощью метода обучения, вместо этого попробуйте внимательно следить за весами. Попробуйте сами (или найдите способ в Интернете) создать элемент XOR с помощью элементов AND, OR и NOT и используйте результаты упражнения 5.3 для облегчения задачи.

Решение

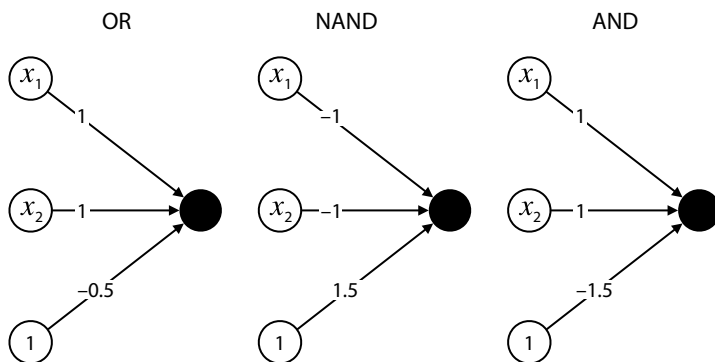
Обратите внимание на то, что следующая комбинация элементов AND, OR и NOT образует элемент XOR, где элемент NAND представляет собой комбинацию элемента AND и элемента NOT.



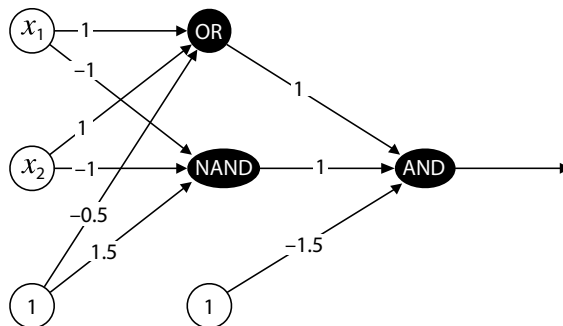
Это иллюстрирует следующая таблица истинности.

x_1	x_2	$h_1 = x_1 \text{ OR } x_2$	$h_2 = x_1 \text{ NAND } x_2$	$h_1 \text{ AND } h_2$	$x_1 \text{ XOR } x_2$
0	0	0	1	0	0
0	1	1	1	1	1
1	0	1	1	1	1
1	1	1	0	0	0

Как и в упражнении 5.3, здесь представлены персептроны, имитирующие элементы OR, NAND и AND. Элемент NAND получается сменой знака всех весов в элементе AND.



Соединив их, мы получим нейронную сеть, показанную на следующем рисунке.



Я прошу вас самостоятельно убедиться в том, что эта сеть действительно имитирует логический элемент XOR. Это делается вводом четырех векторов: $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$ — в сеть и проверкой того, что выходные данные равны 0, 1, 1, 0.

Упражнение 10.3

В конце раздела «Графическое представление нейронных сетей» мы увидели, что нейронная сеть, изображенная на рис. 10.13, с функцией активации не соответствует набору данных, приведенному в табл. 10.1, поскольку точка $(1, 1)$ неправильно классифицирована.

А. Убедитесь, что это так и есть.

Б. Измените веса таким образом, чтобы нейронная сеть правильно классифицировала каждую точку.

Решение

А. Для точки $(x_a, x_b) = (1, 1)$ прогнозы следующие:

$$C = \sigma(6 \times 1 + 10 \times 1 - 15) = \sigma(1) = 0,731;$$

$$F = \sigma(10 \times 1 + 6 \times 1 - 15) = \sigma(1) = 0,731;$$

$$\hat{y} = \sigma(1 \times 0,731 + 1 \times 0,731 - 1,5) = \sigma(-0,39) = 0,404.$$

Поскольку прогноз ближе к 0, чем к 1, точка классифицируется неправильно.

Б. Достаточно уменьшить смещение в конечном узле до чего-либо меньшего, чем $2 \times 0,731 = 1,461$. Например, если бы это смещение составляло 1,4, прогноз в точке $(1, 1)$ был бы выше 0,5. В качестве упражнения рекомендую вам убедиться, что эта новая нейронная сеть правильно прогнозирует метки для оставшихся точек.

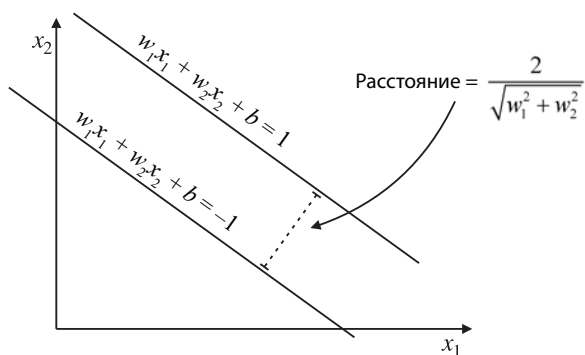
Глава 11. Нахождение границ со стилем. Машины опорных векторов и метод ядра

Упражнение 11.1

(Завершает вычисления, необходимые в разделе «Функция ошибки расстояния...»)

Покажите, что расстояние между прямыми:

$$w_1 x_1 + w_2 x_2 + b = 1 \text{ и } w_1 x_1 + w_2 x_2 + b = -1 \text{ — равно } \frac{2}{\sqrt{w_1^2 + w_2^2}}.$$



Решение

Сначала назовем прямые следующим образом:

- L_1 — прямая $w_1x_1 + w_2x_2 + b = 1$;
- L_2 — прямая $w_1x_1 + w_2x_2 + b = -1$.

Обратите внимание на то, что мы можем переписать уравнение: $w_1x_1 + w_2x_2 + b = 0$ как $x_2 = \frac{w_1x_1}{w_2} - \frac{b}{w_2}$, где $(-w_1/w_2)$ — наклон. Любой перпендикуляр к этой прямой имеет наклон (w_2/w_1) . В частности, прямая $x_2 = (w_2/w_1)x_1$ перпендикулярна как L_1 , так и L_2 . Назовем эту прямую L_3 .

Далее находим решение для точек пересечения L_3 с прямыми L_1 и L_2 . Точка пересечения L_1 и L_3 является решением следующих уравнений:

- $w_1x_1 + w_2x_2 + b = 1$;
- $x_2 = (w_2/w_1)x_1$.

Мы можем подставить второе уравнение в первое и получить:

$$w_1x_1 + w_2 \frac{w_2}{w_1} x_1 + b = 1,$$

а далее решить для x_1 , чтобы получить:

$$x_1 = (1 - b) \frac{w_1}{w_1^2 + w_2^2}.$$

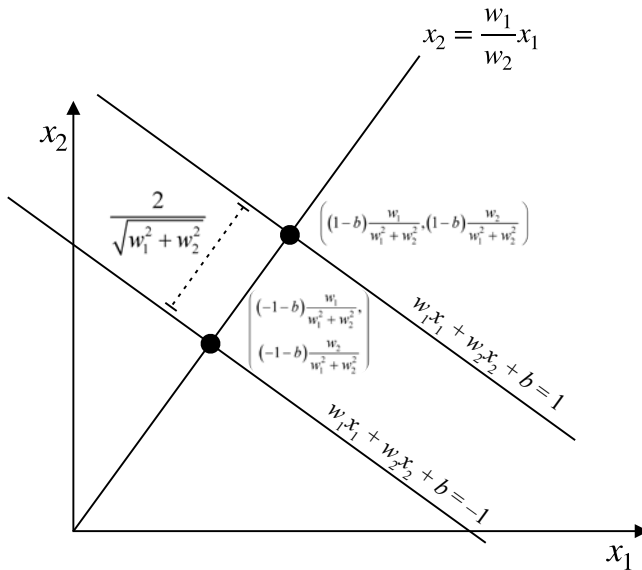
Следовательно, поскольку каждая точка в L_2 имеет вид $\left(x, \frac{w_2}{w_1}x\right)$, точка пересечения L_1 и L_3 — это точка с координатами $\left((1 - b) \frac{w_1}{w_1^2 + w_2^2}, (1 - b) \frac{w_2}{w_1^2 + w_2^2}\right)$.

Аналогичный расчет покажет, что точка пересечения L_2 и L_3 является точкой с координатами $\left((-1 - b) \frac{w_1}{w_1^2 + w_2^2}, (-1 - b) \frac{w_2}{w_1^2 + w_2^2}\right)$.

Чтобы найти расстояние между этими двумя точками, можно использовать теорему Пифагора. Это расстояние составляет:

$$\sqrt{\left((-1-b)\frac{w_1}{w_1^2+w_2^2}, (-1-b)\frac{w_2}{w_1^2+w_2^2}\right)^2 + \left((1-b)\frac{w_1}{w_1^2+w_2^2}, (1-b)\frac{w_2}{w_1^2+w_2^2}\right)^2} = \sqrt{\frac{4w_1^2+4w_2^2}{(w_1^2+w_2^2)^2}},$$

что и требовалось доказать.



Упражнение 11.2

Как мы узнали из упражнения 5.3, невозможно построить модель персептрона, имитирующую элемент XOR. Другими словами, невозможно сопоставить следующий набор данных (со 100%-ной точностью) с моделью персептрона.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Это связано с тем, что набор данных не является линейно разделяемым. У SVM та же проблема, так как она является линейной моделью. Однако мы можем использовать ядро, чтобы обойти это ограничение. Какое ядро следует задействовать, чтобы превратить этот набор данных в линейно разделяемый? Как будет выглядеть результирующая SVM?

ПОДСКАЗКА Обратитесь к примеру 2, приведенному в разделе «Использование полиномиальных уравнений в своих интересах...», который решает очень похожую задачу.

Решение

Взяв ядро полинома второй степени, мы получаем следующий набор данных.

x_1	x_2	x_1^2	$x_1 x_2$	x_2^2	y
0	0	0	0	0	0
0	1	0	0	1	1
1	1	1	0	0	1
1	1	1	1	1	0

Несколько классификаторов работают с этим измененным набором данных. Например, классификатор $\hat{y} = \text{step}(x_1 + x_2 - 2x_1x_2 - 0,5)$ правильно классифицирует данные.

Глава 12. Комбинирование моделей для достижения максимальных результатов. Ансамблевое обучение

Упражнение 12.1

Сильный алгоритм бустинга L формируется из трех слабых, L_1 , L_2 и L_3 . Их веса равны 1, 0,4 и 1,2 соответственно. L_1 и L_2 прогнозируют, что метка конкретной точки положительна, а L_3 — что отрицательна. Каков окончательный прогноз, который сделает алгоритм L для этой точки?

Решение

Поскольку L_1 и L_2 спрогнозировали, что метка положительная, а L_3 — что отрицательная, сумма голосов равна $1 + 0,4 - 1,2 = 0,2$. Этот результат положительный, что означает: сильный алгоритм прогнозирует, что метка этой точки положительна.

Упражнение 12.2

Мы находимся в процессе обучения модели AdaBoost на наборе данных размером 100. Текущий слабый алгоритм правильно классифицирует 68 из 100 точек данных. Какой вес мы присвоим этому алгоритму в окончательной модели?

Решение

Этот вес представляет собой логарифм отношения шансов, или натуральный логарифм шансов. Шансы равны $68/32$, потому что классификатор правильно классифицирует 68 точек и неправильно — оставшиеся 32. Следовательно, вес, присвоенный этому слабому алгоритму, составляет:

$$\text{вес} = \ln(68/32) = 0,754.$$

Глава 13. Применение изученного на практике. Реальный пример разработки данных и машинного обучения

Упражнение 13.1

Репозиторий содержит файл с именем `test.csv`. В нем имеются сведения о других пассажирах «Титаника», но нет столбца `Survived`.

1. Предварительно обработайте данные в этом файле, как делалось в этой главе.
2. Используйте любую из моделей для прогнозирования меток в этом наборе данных. Согласно вашей модели, сколько пассажиров выжили?
3. Сравнивая характеристики всех моделей, описанных в этой главе, как вы думаете, сколько пассажиров из тестовой группы действительно выжили?

Решение

Решение находится в конце следующего блокнота: https://github.com/luisguiserrano/manning/tree/master/Chapter_13_End_to_end_example.

Б

Приложение. Математика, лежащая в основе градиентного спуска. Спуск с горы с использованием производных и склонов



В этом приложении рассмотрим математические детали градиентного спуска. Это приложение чисто техническое, и для его понимания не требуется вникать в основной текст книги. Оно приводится для того, чтобы дать ощущение полноты знаний тем читателям, которые хотят понять внутреннюю работу некоторых основных алгоритмов машинного обучения. Математические знания, необходимые для этого приложения, должны быть выше, чем для остальной части книги. Если конкретно, то требуется понимание векторов, производных и цепного правила.

В главах 3, 5, 6, 10 и 11 мы использовали градиентный спуск, чтобы минимизировать функции ошибок в наших моделях. Если более конкретно, то применили градиентный спуск, чтобы минимизировать следующие функции ошибок:

- функции абсолютной и квадратической ошибок в модели линейной регрессии (глава);
- функцию ошибки персептрона в модели персептрона (глава 5);
- логарифмические потери в логистическом классификаторе (глава 6);
- потерю логарифма в нейронной сети (глава 10);
- ошибку классификации (персептрона) и ошибку расстояния (регуляризации) в SVM (глава 11).

Как мы узнали из глав 3, 5, 6, 10 и 11, функция ошибок измеряет, насколько плохо работает модель. Таким образом, нахождение минимального значения для этой функции ошибок — или по крайней мере действительно небольшого значения, даже если оно не минимально, — будет способствовать поиску хорошей модели.

Аналогия, которую мы использовали, заключалась в спуске с горы. Ошибочной, показанной на рис. Б.1. Сценарий следующий: вы находитесь где-то на вершине горы и хотели бы добраться до подножия. Здесь очень облачно, так что

видимость не очень хорошая. Лучшее, что вы можете сделать, — это спускаться с горы шаг за шагом. Вы спрашиваете себя: «Если бы я сделал только один шаг, то в каком направлении следует шагнуть, чтобы спуститься быстрее всего?» Вы находите это направление и делаете шаг. Затем снова задаете тот же вопрос, делаете еще один шаг и повторяете процесс много раз. Можно себе представить, что если вы всегда делаете один шаг, который помогает спуститься ниже всего, то должны добраться до какого-то низкого места. Возможно, вам понадобится немного удачи, чтобы действительно спуститься к подножию горы, вместо того чтобы застрять в долине, но мы разберемся с этим позже в разделе «Застревание на локальных минимумах...».

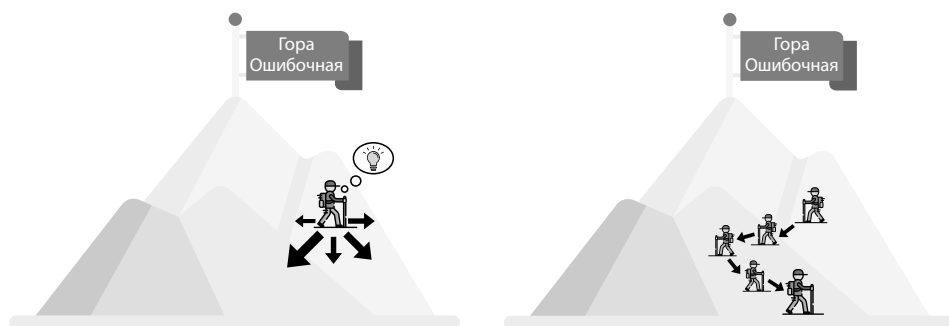


Рис. Б.1. На этапе градиентного спуска мы хотим спуститься с горы Ошибочной

В следующих разделах мы опишем математику, лежащую в основе градиентного спуска, и будем задействовать ее в обучении нескольких алгоритмов машинного обучения, уменьшая их функции ошибок.

Использование градиентного спуска для уменьшения функций

Математический формализм градиентного спуска следующий: допустим, вы хотите минимизировать функцию $f(x_1, x_2, \dots, x_n)$ по n переменным $x_1, x_2, \dots, x_n$. Мы предполагаем, что функция непрерывна и дифференцируема по каждой из n переменных.

В настоящее время мы находимся в точке p с координатами $(p_1, p_2, \dots, p_n)$ и хотим найти направление, в котором функция уменьшается быстрее всего, чтобы сделать этот шаг (рис. Б.2). Чтобы найти направление, в котором функция уменьшается быстрее всего, используем градиент функции. Он представляет собой n -мерный вектор, образованный частными производными от f по каждой из переменных $x_1, x_2, \dots, x_n$. Этот градиент обозначается ∇f и определяется следующим образом:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right).$$

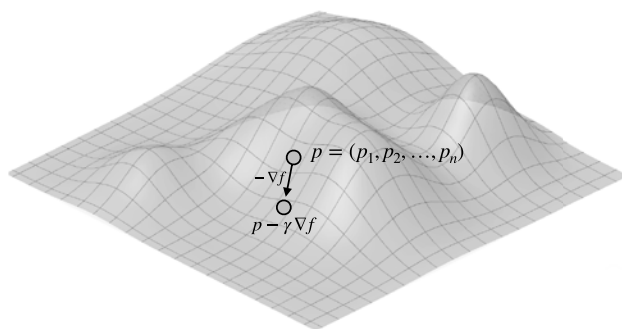


Рис. Б.2. Первоначально мы находились в точке p . Делаем шаг в направлении отрицательного градиента и оказываемся в новой точке. Это направление, в котором функция уменьшается быстрее всего (источник — изображение, созданное с помощью Grapher™ от Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>)

Градиент — это вектор, который указывает в направлении наибольшего роста — в том, в котором функция *увеличивается* быстрее всего. Таким образом, отрицательное значение градиента — это направление, в котором функция *уменьшается* быстрее всего. Именно этот шаг мы хотим сделать. Размер шага определяем, используя *скорость обучения*, которую мы изучили в главе 3 и которую обозначаем η . Шаг градиентного спуска состоит в выполнении шага длиной $\eta|\nabla f|$ в направлении отрицательного значения градиента ∇f . Таким образом, если исходной точкой была p , то после применения шага градиентного спуска получаем точку $p - \nabla f$. Рисунок Б.2 иллюстрирует шаг, который мы предприняли для уменьшения функции f .

Теперь, когда мы знаем, как сделать один шаг, чтобы немного уменьшить функцию, можем просто повторить этот процесс много раз и минимизировать функцию. Таким образом, псевдокод алгоритма градиентного спуска выглядит следующим образом.

Псевдокод для алгоритма градиентного спуска

Цель — минимизировать функцию f .

Гиперпараметры:

- количество периодов (повторений) N ;
- скорость обучения η .

Процесс (рис. Б.3).

- Выберите случайную точку p_1 .
- Для $i = 0, \dots, N - 1$:
 - вычислите градиент $\nabla f(p_i)$;
 - выберите точку $p_{i+1} = p_i - \eta \nabla f(p_i)$.
- Закончите на точке p_n .

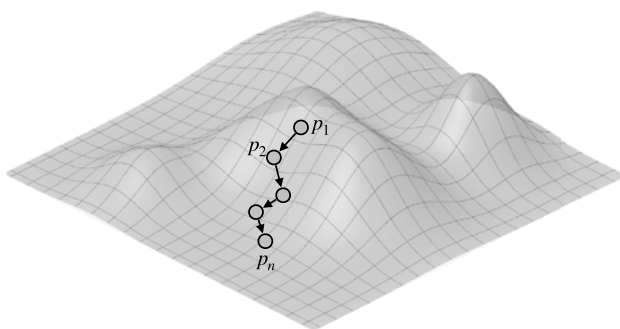


Рис. Б.3. Если повторить шаг градиентного спуска много раз, мы с высокой вероятностью найдем минимальное значение функции (источник — изображение, созданное с помощью Grapher™ от Golden Software, LLC; <https://www.goldensoftware.com/products/grapher>)

Всегда ли этот процесс находит минимум функции? К сожалению, нет. При попытке минимизировать функцию с помощью градиентного спуска может возникнуть несколько проблем, таких как застревание на локальном минимуме (долина). Мы изучим очень полезную технику решения этой проблемы в разделе «Застревание на локальных минимумах...».

Использование градиентного спуска для обучения моделей

Теперь, зная, как градиентный спуск помогает минимизировать (или по крайней мере находить небольшие значения) функции, мы увидим, как применять его для обучения некоторых моделей машинного обучения. Модели, которые мы будем обучать:

- линейная регрессия (из главы 3);
- персептрон (из главы 5);
- логистический классификатор (из главы 6);
- нейронная сеть (из главы 10);
- регуляризация (из глав 4 и 11). Это не модель, но мы сможем увидеть влияние шага градиентного спуска на модель, применяющую регуляризацию.

Способ, которым мы используем градиентный спуск для обучения модели, заключается в том, чтобы сделать f функцией ошибки модели и с помощью градиентного спуска минимизировать ее. Значение функции ошибки вычисляется по набору данных. Однако, как мы видели в разделах «Одна точка за раз или несколько?..» главы 3, «Стохастический, мини-пакетный и пакетный градиентный спуск» главы 6 и «Гиперпараметры» главы 10, если набор данных

слишком велик, можно ускорить обучение путем разделения набора данных на мини-пакеты примерно одинакового размера и на каждом шаге выбирать другой мини-пакет для вычисления функции ошибки.

Вот некоторые обозначения, которые мы будем использовать в этом приложении (большинство терминов введены в главах 1 и 2):

- **размер** набора данных, или количество строк, m ;
- **размерность** набора данных, или количество столбцов, n ;
- набор данных формируется с помощью **признаков** и **меток**;
- **признаками** служат m векторов $x_i = (x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)})$, где $i = 1, 2, \dots, m$;
- **метки** y_i , где $i = 1, 2, \dots, m$;
- **модель** задается вектором из n весов $w = (w_1, w_2, \dots, w_n)$ и смещением b (скаляр) (за исключением случаев, когда модель представляет собой нейронную сеть, которая будет иметь больше весов и смещений);
- **прогнозы** $\hat{y}_i$, где $i = 1, 2, \dots, m$;
- **скорость обучения** модели составляет η ;
- **мини-пакеты данных** — это $B_1, B_2, \dots, B_l$ для некоторого числа l . Каждый мини-пакет имеет длину q . Точки в одном мини-пакете (для удобства) обозначаются $x^{(1)}, \dots, x^{(q)}$, а метки — $y_1, \dots, y_q$.

Алгоритм градиентного спуска, который мы будем использовать для обучения моделей, выглядит следующим образом.

Алгоритм градиентного спуска для обучения моделей машинного обучения

Гиперпараметры:

- количество периодов (повторений) N ;
- скорость обучения η .

Процесс:

- Выберите случайные веса $w_1, w_2, \dots, w_n$ и случайное смещение b .
- Для $i = 0, \dots, N - 1$:
 - для каждого из мини-пакета $B_1, B_2, \dots, B_l$:
 - вычислите функцию ошибки $f(w, b)$;
 - вычислите градиент $\nabla f(w_1, \dots, w_n, b) = \left(\frac{\partial f}{\partial w_1}, \dots, \frac{\partial f}{\partial w_n}, \frac{\partial f}{\partial b} \right)$;
 - замените веса и смещение следующим образом:
 - w_i на $w'_i = w_i - \eta \frac{\partial f}{\partial w_i}$;
 - b на $b' = b - \eta \frac{\partial f}{\partial b}$.

В следующих подразделах мы будем детально выполнять этот процесс для каждой из следующих моделей и функций ошибок:

- модели линейной регрессии с функцией средней абсолютной ошибки (следующий раздел);
- модели линейной регрессии с функцией средней квадратической ошибки (следующий раздел);
- модели персептрона с функцией ошибок персептрона (раздел «Использование градиентного спуска для обучения моделей классификации»);
- модели логистической регрессии с функцией логарифмических потерь (раздел «Использование градиентного спуска для обучения моделей классификации»);
- нейронной сети с функцией логарифмических потерь (раздел «Использование градиентного спуска для обучения нейронных сетей»);
- модели с регуляризацией (раздел «Использование градиентного спуска для регуляризации»).

Использование градиентного спуска для обучения моделей линейной регрессии

В этом разделе мы применим градиентный спуск для обучения модели линейной регрессии, используя обе функции ошибок, которые изучили ранее: среднюю абсолютную ошибку и среднюю квадратическую ошибку. Напомним из главы 3, что в линейной регрессии прогнозы $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_q$ задаются следующей формулой:

$$\hat{y}_i = \sum_{j=1}^n w_j x_j^{(i)} + b.$$

Цель регрессионной модели — найти веса $w_1, \dots, w_n$, которые дают прогнозы, действительно близкие к меткам. Таким образом, функция ошибки пригодится для измерения того, как далеко $\hat{y}$ находится от y для определенного набора весов. Как мы видели в разделах «Абсолютная погрешность» и «Квадратическая погрешность» главы 3, есть два разных способа вычисления этого расстояния. Первый — это абсолютное значение $|\hat{y} - y|$, а второй — квадрат разности $(\hat{y} - y)^2$. Первый приводит к средней абсолютной ошибке, а второй — к средней квадратической. Изучим их по отдельности.

Обучение модели линейной регрессии с использованием градиентного спуска для уменьшения средней абсолютной ошибки

Здесь мы рассчитаем градиент функции средней абсолютной ошибки и задействуем его для применения градиентного спуска и обучения модели линейной регрессии. Средняя абсолютная ошибка — это способ определения того, как далеко друг от друга находятся $\hat{y}$ и y . Впервые он был определен в разделе «Абсолютная погрешность» главы 3, его формула выглядит следующим образом:

$$\text{MAE}(w, b, x, y) = \frac{1}{q} \sum_{i=1}^q |\hat{y}_i - y_i|.$$

Для удобства сократим $\text{MAE}(w, b, x, y)$ до MAE . Чтобы использовать градиентный спуск для уменьшения MAE , нужно вычислить градиент ∇MAE , который представляет собой вектор, содержащий $n + 1$ частных производных MAE относительно $w_1, \dots, w_n, b$:

$$\nabla \text{MAE} = \left(\frac{\partial \text{MAE}}{\partial w_1}, \dots, \frac{\partial \text{MAE}}{\partial w_n}, \frac{\partial \text{MAE}}{\partial b} \right).$$

Мы вычислим эти частные производные, используя цепное правило. Прежде всего, обратите внимание на то, что

$$\frac{\partial \text{MAE}}{\partial w_j} = \frac{1}{q} \sum_{i=1}^q \frac{\partial |\hat{y}_i - y_i|}{\partial w_j}.$$

Производная от $f(x) = |x|$ является знаковой функцией $\text{sgn}(x) = \frac{|x|}{x}$, которая равна $+1$, когда переменная x положительна, и -1 , когда x отрицательна (она не определена в 0 , но для удобства мы можем определить ее равной 0). Таким образом, можно переписать предыдущее уравнение как

$$\frac{\partial \text{MAE}}{\partial w_j} = \frac{1}{q} \sum_{i=1}^q \text{sgn}(\hat{y}_i - y_i) \frac{\partial \hat{y}_i}{\partial w_j}.$$

Чтобы вычислить это значение, сосредоточимся на заключительной части уравнения, а именно на $\frac{\partial \hat{y}_i}{\partial w_j}$. Поскольку

$$\hat{y}_i = \sum_{j=1}^n w_j x_j^{(i)},$$

то

$$\frac{\partial \hat{y}_i}{\partial w_j} = \sum_{j=1}^n \frac{\partial (w_j x_j^{(i)})}{\partial w_j} = x_j^{(i)}.$$

Это связано с тем, что производная от w_j по отношению к w_i равна 1 , если $j = i$, и 0 в противном случае. Таким образом, замена на производную дает следующее:

$$\frac{\partial \text{MAE}}{\partial w_j} = \frac{1}{q} \sum_{i=1}^q \text{sgn}(\hat{y}_i - y_i) x_j^{(i)}.$$

Используя аналогичный анализ, мы можем вычислить производную $\text{MAE}(w, b)$ по отношению к b как

$$\frac{\partial \text{MAE}}{\partial b} = \frac{1}{q} \sum_{i=1}^q \text{sgn}(\hat{y}_i - y_i).$$

Этап градиентного спуска заключается в следующем.

Шаг градиентного спуска. Заменить (w, b) на (w', b') , где:

- $w'_j = w_j + \frac{1}{q} \sum_{i=1}^q \eta \text{sgn}(y_i - \hat{y}_i) x_j^{(i)}$ при $i = 1, 2, \dots, n$;
- $b' = b + \eta \sum_{i=1}^q \text{sgn}(y_i - \hat{y}_i)$.

Обратите внимание на кое-что интересное: если мини-пакет имеет размер $q = 1$ и состоит только из точки $x = (x_1, x_2, \dots, x_n)$ с меткой y и прогнозом $\hat{y}$, то шаг определяется следующим образом: заменить (w, b) на (w', b') , где:

- $w'_j = w_j + \eta \operatorname{sgn}(y - \hat{y})x_j$;
- $b' = b + \eta \operatorname{sgn}(y - \hat{y})$.

Это именно тот простой трюк, который мы применяли в разделе «Псевдокод для алгоритма линейной регрессии» главы 3 для обучения алгоритма линейной регрессии.

Обучение модели линейной регрессии с использованием градиентного спуска для уменьшения средней квадратической ошибки

В этом подразделе мы рассчитаем градиент функции средней квадратической ошибки и будем использовать его для применения градиентного спуска и обучения модели линейной регрессии. Средняя квадратическая ошибка — это еще один способ определить, насколько далеко друг от друга находятся $\hat{y}$ и y . Впервые она была определена в разделе «Квадратическая погрешность...» главы 3, и ее формула такова:

$$\text{MSE}(w, b, x, y) = \frac{1}{2m} \sum_{i=1}^q (\hat{y}_i - y_i)^2.$$

Для удобства мы будем сокращать $\text{MSE}(w, b, x, y)$ до MSE . Чтобы вычислить градиент ∇MSE , мы можем придерживаться той же процедуры, что и для средней абсолютной ошибки, описанной ранее, за исключением того, что производная от $f(x) = x^2$ равна $2x$. Следовательно, производная от MSE по отношению к w_j составляет:

$$\frac{\partial \text{MSE}}{\partial w_j} = \frac{1}{2q} \sum_{i=1}^q \frac{\partial (\hat{y}_i - y_i)^2}{\partial w_j} = \frac{1}{2q} \sum_{i=1}^q 2(\hat{y}_i - y_i) \frac{\partial \hat{y}_i}{\partial w_j} = \frac{1}{q} \sum_{i=1}^q (\hat{y}_i - y_i) \frac{\partial \hat{y}_i}{\partial w_j}.$$

Аналогично производная от $\text{MSE}(w, b)$ по отношению к b составляет:

$$\frac{\partial \text{MSE}}{\partial b} = \frac{1}{2q} \sum_{i=1}^q \frac{\partial (\hat{y}_i - y_i)^2}{\partial b} = \frac{1}{2q} \sum_{i=1}^q 2(\hat{y}_i - y_i) \frac{\partial \hat{y}_i}{\partial b} = \frac{1}{q} \sum_{i=1}^q (\hat{y}_i - y_i) \frac{\partial \hat{y}_i}{\partial b}.$$

Шаг градиентного спуска. Заменить (w, b) на (w', b') , где:

- $w'_j = w_j + \eta \frac{1}{q} \sum_{i=1}^q (y_i - \hat{y}_i) x_j^{(i)}$ при $i = 1, 2, \dots, n$;
- $b' = b + \eta \frac{1}{q} \sum_{i=1}^q (y_i - \hat{y}_i)$.

Еще раз обратите внимание на то, что если мини-пакет имеет размер $q = 1$ и состоит только из точки $x = (x_1, x_2, \dots, x_n)$ с меткой y и прогнозом $\hat{y}$, то шаг определяется следующим образом: заменить (w, b) на (w', b') , где:

- $w'_j = w_j + \eta (y - \hat{y}) x_j$;
- $b' = b + \eta (y - \hat{y})$.

Это именно тот квадратический метод, который мы применяли в разделе «Квадратический подход...» главы 3 для обучения алгоритма линейной регрессии.

Использование градиентного спуска для обучения моделей классификации

В этом разделе мы узнаем, как с помощью градиентного спуска обучать модели классификации. Две модели, которые мы будем обучать, — это модель персептрона (см. главу 5) и модель логистической регрессии (см. главу 6). Каждая из них имеет собственную функцию ошибки, поэтому будем рассматривать их по отдельности.

Обучение модели персептрона с помощью градиентного спуска для уменьшения ошибки персептрона

Далее рассчитаем градиент функции ошибок персептрона и используем его для применения градиентного спуска и обучения модели персептрона. В модели персептрона прогнозы равны $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_q$, где каждый $\hat{y}_i$ равен 0 или 1. Чтобы рассчитать прогнозы, нам придется вспомнить ступенчатую функцию $\text{step}(x)$, представленную в главе 5. Она принимает в качестве входных данных любое действительное число x и выдает 0, если $x < 0$, и 1, если $x \geq 0$. Ее график показан на рис. Б.4.

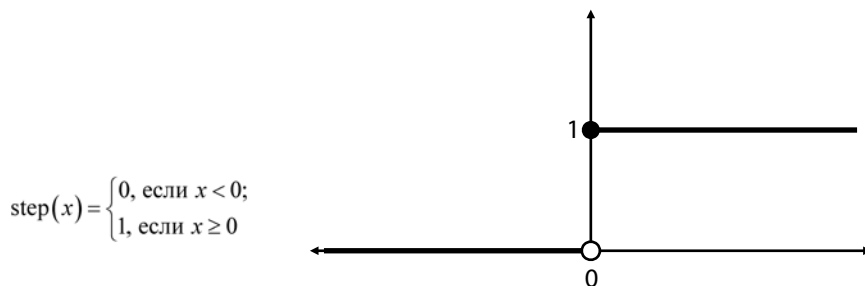


Рис. Б.4. Ступенчатая функция для отрицательных чисел выводит 0, а для неотрицательных — 1

Модель присваивает каждой точке *оценку*. Оценка, которую модель с весами $(w_1, w_2, \dots, w_n)$ и смещением b дает точке $x^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)})$ — это $\text{score}(w, b, x^{(i)}) = \sum_{j=1}^n w_j x_j^{(i)} + b$. Прогнозы $\hat{y}$ задаются по следующей формуле:

$$\hat{y}_i = \text{step}(\text{score}(w, b, x^{(i)})) = \text{step}\left(\sum_{j=1}^n w_j x_j^{(i)} + b\right).$$

Другими словами, прогноз равен 1, если оценка положительная, и 0 — в противном случае.

Функцию ошибки персептрона $\text{PE}(w, b, x, y)$ мы будем сокращать до PE. Впервые она была определена в разделе «Как сравнить классификаторы? Функция ошибки» главы 5. Это большое число, если модель сделала плохой прогноз,

и небольшое (в данном случае фактически 0), если хороший. Функция ошибки определяется следующим образом:

- $PE(w, b, x, y) = 0$, если $\hat{y} = y$;
- $PE(w, b, x, y) = |\text{score}(w, b, x)|$, если $\hat{y} \neq y$.

Другими словами, если точка классифицирована правильно, ошибка равна нулю. Если она классифицирована неправильно, то ошибка представляет собой абсолютное значение оценки. Таким образом, неправильно классифицированные точки с оценками с низким абсолютным значением дают малую ошибку, а с оценками с высоким абсолютным значением — большую. Так происходит потому, что абсолютное значение оценки точки пропорционально расстоянию между этой точкой и границей. Таким образом, точки с низкой погрешностью — это точки, которые расположены близко к границе, тогда как точки с высокой погрешностью находятся далеко от нее.

Чтобы вычислить градиент ∇PE , можно использовать то же правило, что и раньше. Мы лишь должны обратить внимание на то, что производная функции абсолютного значения $|x| = 1$, когда $x \geq 0$, и $|x| = 0$, когда $x < 0$. Эта производная не определена в 0, что становится проблемой для наших вычислений, но на практике можем произвольно определить ее как 1 без каких-либо проблем.

В главе 10 мы ввели функцию $\text{ReLU}(x)$ (блок линейной ректификации), которая равна 0, когда $x < 0$, и x , когда $x \geq 0$. Обратите внимание на то, что существуют два способа, с помощью которых точка может быть неправильно классифицирована:

- если $y = 0$ и $\hat{y} = 1$. Это означает $\text{score}(w, b, x) \geq 0$;
- если $y = 1$ и $\hat{y} = 0$. Это означает, что $\text{score}(w, b, x) < 0$.

Таким образом, мы можем для удобства переписать ошибку персептрона как:

$$PE = \sum_{i=1}^q y_i \text{ReLU}(-\text{score}(w, b, x)) + (1 - y_i) \text{ReLU}(\text{score}(w, b, x))$$

или более подробно как:

$$PE = \sum_{i=1}^q y_i \text{ReLU}\left(-\sum_{j=1}^n w_j x_j^{(i)} - b\right) + (1 - y_i) \text{ReLU}\left(\sum_{j=1}^n w_j x_j^{(i)} + b\right).$$

Теперь можно приступить к вычислению градиента ∇PE , применяя цепное правило. Важное наблюдение, которое мы будем использовать и которое читатель может проверить, состоит в том, что производная $\text{ReLU}(x)$ является ступенчатой функцией $\text{step}(x)$. Этот градиент составляет:

$$\frac{\partial PE}{\partial w_j} = \sum_{i=1}^q y_i \text{step}\left(-\sum_{j=1}^n w_j x_j^{(i)} - b\right) (-x_j^{(i)}) + (1 - y_i) \text{step}\left(\sum_{j=1}^n w_j x_j^{(i)} + b\right) x_j^{(i)},$$

что мы можем переписать как:

$$\frac{\partial PE}{\partial w_j} = \sum_{i=1}^q -y_i x_j^{(i)} \text{step}(-\text{score}(w, b, x)) + (1 - y_i) x_j^{(i)} \text{step}(\text{score}(w, b, x)).$$

Это только выглядит сложным. Давайте проанализируем каждое слагаемое из правой части выражения. Обратите внимание на то, что $\text{step}(\text{score}(w, b, x))$ равно 1 тогда и только тогда, когда $\text{score}(w, b, x) > 0$, а в противном случае равно 0. Это происходит именно тогда, когда $\hat{y} = 1$. Аналогично $\text{step}(-\text{score}(w, b, x))$ равен 1 тогда и только тогда, когда $\text{score}(w, b, x) < 0$, а в противном случае равен 0. Это происходит именно тогда, когда $\hat{y} = 0$. Следовательно:

- если $\hat{y}_i = 0$ и $y_i = 0$, то

$$-y_i x_j^{(i)} \text{step}(-\text{score}(w, b, x)) + (1 - y_i) x_j^{(i)} \text{step}(\text{score}(w, b, x)) = 0;$$
- если $\hat{y}_i = 1$ и $y_i = 1$, то

$$-y_i x_j^{(i)} \text{step}(-\text{score}(w, b, x)) + (1 - y_i) x_j^{(i)} \text{step}(\text{score}(w, b, x)) = 0;$$
- если $\hat{y}_i = 0$ и $y_i = 1$, то

$$-y_i x_j^{(i)} \text{step}(-\text{score}(w, b, x)) + (1 - y_i) x_j^{(i)} \text{step}(\text{score}(w, b, x)) = -x_j^{(i)};$$
- если $\hat{y}_i = 1$ и $y_i = 0$, то

$$-y_i x_j^{(i)} \text{step}(-\text{score}(w, b, x)) + (1 - y_i) x_j^{(i)} \text{step}(\text{score}(w, b, x)) = x_j^{(i)}.$$

Это означает, что при расчете $\frac{\partial \text{PE}}{\partial w_j}$ только слагаемые, исходящие из неправильно классифицированных точек, добавляют оценку.

Аналогично

$$\frac{\partial \text{PE}}{\partial b} = \sum_{i=1}^q -y_i \text{step}(-\text{score}(w, b, x)) + (1 - y_i) \text{step}(\text{score}(w, b, x)).$$

Итак, шаг градиентного спуска определяется следующим образом.

Шаг градиентного спуска. Заменить (w, b) на (w', b') , где

- $w'_j = w_j + \eta \sum_{i=1}^q -y_i x_j^{(i)} \text{step}(-\text{score}(w, b, x)) + (1 - y_i) x_j^{(i)} \text{step}(\text{score}(w, b, x));$
- $b' = b + \eta \sum_{i=1}^q -y_i \text{step}(-\text{score}(w, b, x)) + (1 - y_i) \text{step}(\text{score}(w, b, x)).$

И снова, глядя на правую часть предыдущего выражения:

- если $\hat{y}_i = 0$ и $y_i = 0$, то

$$-y_i \text{step}(-\text{score}(w, b, x)) + (1 - y_i) \text{step}(\text{score}(w, b, x)) = 0;$$
- если $\hat{y}_i = 1$ и $y_i = 1$, то

$$-y_i \text{step}(-\text{score}(w, b, x)) + (1 - y_i) \text{step}(\text{score}(w, b, x)) = 0;$$
- если $\hat{y}_i = 0$ и $y_i = 1$, то

$$-y_i \text{step}(-\text{score}(w, b, x)) + (1 - y_i) \text{step}(\text{score}(w, b, x)) = -1;$$
- если $\hat{y}_i = 1$ и $y_i = 0$, то

$$-y_i \text{step}(-\text{score}(w, b, x)) + (1 - y_i) \text{step}(\text{score}(w, b, x)) = 1.$$

Все это может быть не так уж важно, но можно это закодировать, чтобы вычислить все элементы градиента. Еще раз обратите внимание на то, что если мини-пакет имеет размер $q = 1$ и состоит только из точки $x = (x_1, x_2, \dots, x_n)$ с меткой y и предсказанием $\hat{y}$, то шаг определяется следующим образом.

Шаг градиентного спуска

- Если точка классифицирована правильно, не меняйте w и b .
- Если точка имеет метку $y = 0$ и классифицируется как $\hat{y} = 1$:
 - замените w на $w' = w - \eta x$;
 - замените b на $b' = w - \eta$.
- Если точка имеет метку $y = 1$ и классифицируется как $\hat{y} = 0$:
 - замените w на $w' = w + \eta x$;
 - замените b на $b' = w + \eta$.

Это именно тот метод персептрона, который описан в разделе «Метод персептрона — способ немного улучшить персептрон» главы 5.

Обучение модели логистической регрессии с использованием градиентного спуска для уменьшения логарифмических потерь

В этом подразделе мы рассчитаем градиент функции логарифмических потерь и будем задействовать его для применения градиентного спуска и обучения модели логистической регрессии. В модели логистической регрессии прогнозы равны $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_q$, где каждый $\hat{y}_i$ — действительное число в диапазоне от 0 до 1. Чтобы вычислить прогнозы, сначала нужно вспомнить сигмоидную функцию $\sigma(x)$, введенную в главе 6. Эта функция принимает в качестве входных данных любое действительное число x и выводит некоторое число от 0 до 1. Если x — большое положительное число, то $\sigma(x)$ близка к 1, если x — большое отрицательное число, то $\sigma(x)$ близка к 0. Формула сигмоидной функции имеет вид:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

График $\sigma(x)$ показан на рис. Б.5.

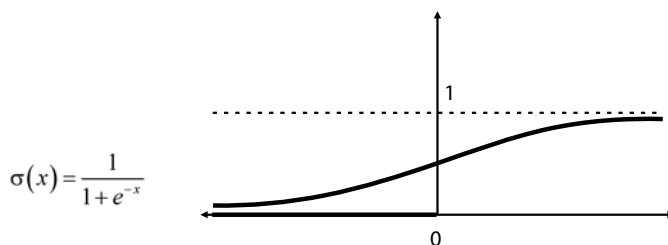


Рис. Б.5. Сигмоидная функция всегда выводит число от 0 до 1.

Выходные данные для отрицательных чисел близки к 0, а выходные данные для положительных чисел близки к 1

Предсказания модели логистической регрессии в точности являются результатом сигмоидной функции, то есть для $i = 1, 2, \dots, q$ они определяются следующим образом:

$$\hat{y}_i = \sigma(\text{score}(w, b, x^{(i)})) = \sigma\left(\sum_{j=1}^n w_j x_j^{(i)} + b\right).$$

Логарифмическая потеря обозначается $\text{LL}(w, b, x, y)$, что мы будем сокращать до LL . Эта функция ошибок была впервые определена в разделе «Набор данных и прогнозы» главы 6. Это похоже на функцию ошибок персептрона, потому что по своей конструкции это большое число, если модель сделала плохой прогноз, и небольшое число, если прогноз хороший. Функция логарифмической потери определяется как

$$\text{LL} = -\sum_{i=1}^q y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i).$$

Перейдем к вычислению градиента ∇LL , используя цепное правило. Перед этим отметим, что производная сигмоидной функции может быть записана как $\sigma'(x) = \sigma(x)|1 - \sigma(x)|$. Детали последнего вычисления могут быть выведены с помощью частного правила для дифференцирования, их мы оставим на усмотрение читателя. Используя это, можно вычислить производную от $\hat{y}_i$ по отношению к w_j . Поскольку $\hat{y}_i = \sigma\left(\sum_{j=1}^n (w_j x_j^{(i)} + b)\right)$, то согласно цепному правилу:

$$\frac{\partial \hat{y}_i}{\partial w_j} = \sigma\left(\sum_{j=1}^n (w_j x_j^{(i)} + b)\right) \left[1 - \sigma\left(\sum_{j=1}^n (w_j x_j^{(i)} + b)\right)\right] x_j^{(i)} = y_i (1 - y_i) x_j^{(i)}.$$

Теперь перейдем к разработке логарифмической потери. Снова используя цепное правило, мы получаем:

$$\frac{\partial \text{LL}}{\partial w_j} = -\sum_{i=1}^q y_i \frac{1}{\hat{y}_i} \frac{\partial \hat{y}_i}{\partial w_j} - (1 - y_i) \frac{-1}{1 - \hat{y}_i} \frac{\partial \hat{y}_i}{\partial w_j}.$$

И согласно предыдущему расчету для $\frac{\partial \hat{y}_i}{\partial w_j}$ получаем:

$$\frac{\partial \text{LL}}{\partial w_j} = \sum_{i=1}^q -y_i \frac{1}{\hat{y}_i} \hat{y}_i (1 + \hat{y}_i) x_j^{(i)} + (1 - y_i) \frac{-1}{1 - \hat{y}_i} \hat{y}_i (1 - \hat{y}_i) x_j^{(i)}.$$

Упрощая, мы получаем:

$$\frac{\partial \text{LL}}{\partial w_j} = \sum_{i=1}^q -y_i (1 - \hat{y}_i) x_j^{(i)} + (1 - y_i) \hat{y}_i x_j^{(i)},$$

что упрощается еще больше как:

$$\frac{\partial \text{LL}}{\partial w_j} = \sum_{i=1}^q (\hat{y}_i - y_i) x_j^{(i)}.$$

Аналогично, беря производную по отношению к b , получаем:

$$\frac{\partial \text{LL}}{\partial b} = \sum_{i=1}^q (\hat{y}_i - y_i).$$

Таким образом, шаг градиентного спуска становится следующим.

Шаг градиентного спуска. Заменить (w, b) на (w', b') , где:

- $w' = w + \eta \sum_{i=1}^q (y_i - \hat{y}_i) x^{(i)}$;
- $b' = b + \eta \sum_{i=1}^q (y_i - \hat{y}_i)$.

Обратите внимание на то, что когда мини-пакеты имеют размер 1, то для нахождения шага градиентного спуска заменяем (w, b) на (w', b') , где:

- $w' = w + \eta (y - \hat{y}) x^{(i)}$;
- $b' = b + \eta (y - \hat{y})$.

Именно этот метод логистической регрессии мы освоили в разделе «Как найти хороший логистический классификатор?..» главы 6.

Использование градиентного спуска для обучения нейронных сетей

В разделе «Обратное распространение...» главы 10 мы рассмотрели обратное распространение — процесс обучения нейронной сети. Он состоит из повторения шага градиентного спуска для минимизации логарифмических потерь. Здесь же увидим, как на самом деле вычисляются производные для выполнения этого шага градиентного спуска. Реализуем процесс в нейронной сети глубиной 2 (один входной, один скрытый и один выходной слой), этот пример достаточно велик для того, чтобы показать, как производные вычисляются в целом. Кроме того, мы применим градиентный спуск к ошибке только для одной точки, другими словами, выполним стохастический градиентный спуск. Тем не менее я рекомендую вам разработать производные для нейронной сети с большим количеством слоев и использованием мини-пакетов точек (градиентный спуск мини-пакета).

В нашей нейронной сети входной слой состоит из m входных узлов, скрытый слой — из n скрытых узлов, а выходной слой — из одного выходного узла. Обозначения в этом разделе для простоты отличаются от использованных в других и выглядят следующим образом (рис. Б.6):

- входными данными является точка с координатами $x_1, x_2, \dots, x_m$;
- первый скрытый слой имеет веса V_{ij} и смещения b_j при $i = 1, 2, \dots, m$ и $j = 1, 2, \dots, n$;
- второй скрытый слой имеет веса W_j при $j = 1, 2, \dots, n$ и смещение c .

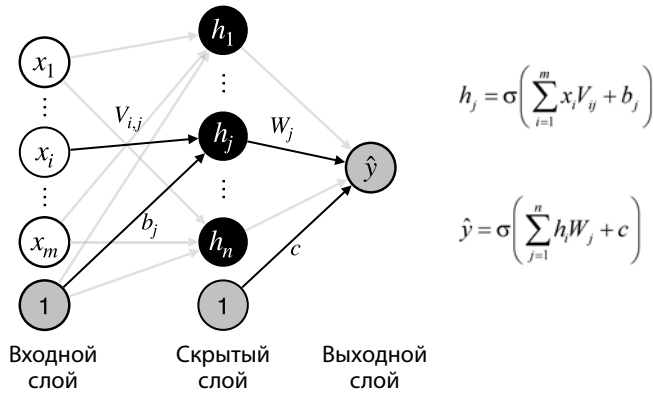


Рис. Б.6. Процесс вычисления прогноза с использованием нейронной сети с одним скрытым слоем и сигмоидными функциями активации

Способ вычисления выходных данных заключается в применении следующих уравнений:

- $h_j = \sigma \left(\sum_{i=1}^m x_i V_{ij} + b_j \right);$
- $\hat{y} = \sigma \left(\sum_{j=1}^n h_j W_j + c \right).$

Чтобы упростить вычисление производных, используем следующие вспомогательные переменные r_j и s :

- $r_j = \sum_{i=1}^m x_i V_{ij} + b_j;$
- $h_j = \sigma(r_j);$
- $s = \sum_{j=1}^n h_j W_j + c;$
- $\hat{y} = \sigma(s).$

Таким образом, можем вычислить следующие частные производные (вспомогательные, что производная сигмоидной функции равна $\sigma'(x) = \sigma(x)[1 - \sigma(x)]$ и что логарифмические потери равны $L(y, \hat{y}) = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$, — оставим L для удобства).

1. $\frac{\partial L}{\partial \hat{y}} = -y \frac{1}{\hat{y}} - (1 - y) \frac{-1}{1 - \hat{y}} = \frac{-(y - \hat{y})}{\hat{y}(1 - \hat{y})}.$
2. $\frac{\partial \hat{y}}{\partial s} = \sigma(s)[1 - \sigma(s)] = y(1 - y).$
3. $\frac{\partial s}{\partial W_j} = h_j$ и $\frac{\partial s}{\partial b_j} = 1.$

$$4. \frac{\partial s}{\partial h_j} = W_j.$$

$$5. \frac{\partial h_j}{\partial r_j} = \sigma(r_j)[1 - \sigma(r_j)] = h_j(1 - h_j).$$

$$6. \frac{\partial r_j}{\partial V_{ij}} = x_i \text{ и } \frac{\partial r_j}{\partial b_j} = 1.$$

Обратите внимание: если мы перемножим уравнения 1 и 2 и используем цепное правило, чтобы упростить вычисления, то получим:

$$7. \frac{\partial L}{\partial s} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial s} = \frac{-(y - \hat{y})}{\hat{y}(1 - \hat{y})} \hat{y}(1 - \hat{y}) = -(y - \hat{y}).$$

Теперь мы можем использовать цепное правило и уравнения 3–7 для вычисления производных логарифмических потерь по весам и смещениям следующим образом.

$$8. \frac{\partial L}{\partial W_j} = \frac{\partial L}{\partial s} \frac{\partial s}{\partial W_j} = -(y - \hat{y})h_j.$$

$$9. \frac{\partial L}{\partial c} = \frac{\partial L}{\partial s} \frac{\partial s}{\partial c} = -(y - \hat{y}).$$

$$10. \frac{\partial L}{\partial V_{ij}} = \frac{\partial L}{\partial s} \frac{\partial s}{\partial h_j} \frac{\partial h_j}{\partial r_j} \frac{\partial r_j}{\partial V_{ij}} = -(y - \hat{y})W_j h_j(1 - h_j)x_i.$$

$$11. \frac{\partial L}{\partial b_j} = \frac{\partial L}{\partial s} \frac{\partial s}{\partial h_j} \frac{\partial h_j}{\partial r_j} \frac{\partial r_j}{\partial b_j} = -(y - \hat{y})W_j h_j(1 - h_j).$$

Шаг градиентного спуска на основе предыдущих уравнений выглядит следующим образом.

Шаг градиентного спуска для нейронных сетей

- Заменить V_{ij} на $V_{ij} - \eta \frac{\partial L}{\partial V_{ij}} = V_{ij} + \eta(y - \hat{y})W_j h_j(1 - h_j)x_i$.
- Заменить b_j на $b_j - \eta \frac{\partial L}{\partial b_j} = b_j + \eta(y - \hat{y})W_j h_j(1 - h_j)$.
- Заменить W_j на $W_j + \eta \frac{\partial L}{\partial W_j} = W_j + \eta(y - \hat{y})h_j$.
- Заменить c на $c + \eta \frac{\partial L}{\partial c} = c + \eta(y - \hat{y})$.

Эти уравнения довольно сложны, как и уравнения обратного распространения для нейронных сетей с еще большим количеством слоев. К счастью, мы можем использовать PyTorch, TensorFlow и Keras для обучения нейронных сетей, не вычисляя все производные.

Использование градиентного спуска для регуляризации

В разделе «Изменение функции ошибки для решения проблемы...» главы 4 мы изучили регуляризацию как способ уменьшения переобучения в моделях машинного обучения. Регуляризация состоит в добавлении члена регуляризации к функции ошибки, что помогает уменьшить переобучение. Этот член может быть нормой L1 или L2 полинома, используемого в модели. В разделе «Методы обучения нейронных сетей...» главы 10 мы ввели термин «регуляризация» и узнали, как применить регуляризацию для обучения нейронных сетей. Позже, в разделе «Функция ошибки расстояния...» главы 11, мы изучили функцию ошибки расстояния для SVM, которая гарантировала, что две строки в классификаторе располагаются близко друг к другу. Функция ошибки расстояния имела ту же форму, что и член регуляризации L2.

Однако в разделе «Интуитивный способ понять регуляризацию» главы 4 мы узнали о более интуитивном способе взглянуть на регуляризацию. Если коротко, то каждый шаг градиентного спуска, использующий регуляризацию, уменьшает значения коэффициентов модели на небольшую величину. Давайте рассмотрим математику, лежащую в основе этого явления.

Для модели с весами $w_1, w_2, \dots, w_n$ условия регуляризации были следующими:

- регуляризация L1 — $W_1 = |w_1| + |w_2| + \dots + |w_n|$;
- регуляризация L2 — $W_2 = w_1^2 + w_2^2 + \dots + w_n^2$.

Напомним, что для того, чтобы не изменять коэффициенты слишком резко, член регуляризации умножается на параметр регуляризации λ . Таким образом, когда мы применяем градиентный спуск, коэффициенты изменяются следующим образом:

- регуляризация L1 — w_i заменяется на $w_i - \nabla W_1$;
- регуляризация L2 — w_i заменяется на $w_i - \nabla W_2$.

Здесь ∇ обозначает градиент члена регуляризации. Другими словами, $\nabla(b_1, \dots, b_n) = \left(\frac{\partial b_1}{\partial a_1}, \dots, \frac{\partial b_n}{\partial a_n} \right)$. Поскольку $\frac{\partial |w_i|}{\partial w_i} = \text{sgn}(w_i)$ и $\frac{\partial w_i^2}{\partial w_i} = 2w_i$, шаг градиентного спуска выглядит так.

Шаг градиентного спуска для регуляризации:

- регуляризация L1 — замените a_i на $a_i - \lambda \frac{\partial a_i}{\partial a_i} = a_i - \lambda \text{sgn}(a_i)$;
- регуляризация L2 — замените a_i на $a_i - \lambda \frac{\partial a_i}{\partial a_i} = a_i - 2\lambda a_i = (1 - 2\lambda)a_i$.

Обратите внимание на то, что этот шаг градиентного спуска всегда уменьшает абсолютное значение коэффициента a_i . При регуляризации L1 мы вычитаем небольшое значение из a_i , если он a_i положительный, и добавляем небольшое значение, если отрицательный. При регуляризации L2 умножаем a_i на число, которое немного меньше 1.

Застревание на локальных минимумах: как оно происходит и как его обойти

В начале этого приложения упоминалось, что алгоритм градиентного спуска не обязательно находит минимальное значение функции. В качестве примера посмотрите на рис. Б.7. Допустим, мы хотим найти минимум функции, используя градиентный спуск. Поскольку первый шаг в градиентном спуске — начать со случайной точки, начнем с точки, обозначенной как начальная.

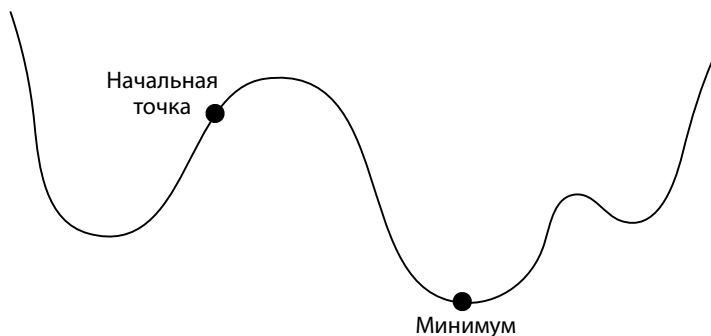


Рис. Б.7. Мы в начальной точке. Минимальное значение функции тоже обозначено. Сможем ли мы достичь его с помощью градиентного спуска?

На рис. Б.8 показан путь, который алгоритм градиентного спуска выберет для нахождения минимума. Обратите внимание на то, что ему удастся найти ближайший к начальной точке локальный минимум, но он пропускает глобальный минимум справа.

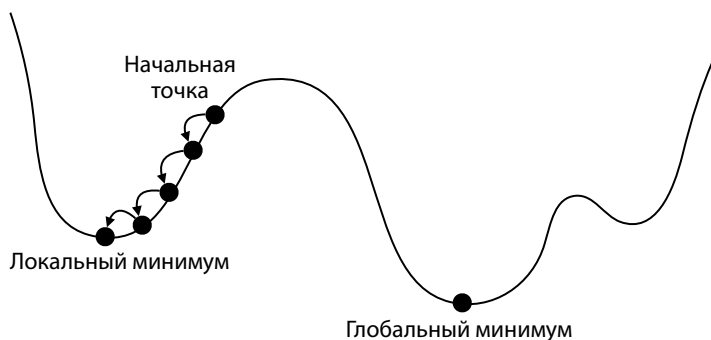


Рис. Б.8. К сожалению, градиентный спуск не помог найти минимальное значение этой функции. Нам удалось спуститься, но мы застряли на локальном минимуме (долина)

Как решить эту проблему? Можно использовать множество методов, и в этом разделе мы изучим распространенный метод, называемый *случайным перезапуском*. Решение состоит в том, чтобы просто запустить алгоритм несколько раз, всегда начиная с другой случайной точки и выбирая минимальное значение, которое было найдено в целом. На рис. Б.9 мы задействуем случайный перезапуск, чтобы найти глобальный минимум функции (обратите внимание на то, что эта функция определена только на изображенном интервале и, таким образом, наименьшее значение в интервале — действительно глобальный минимум). Мы выбрали три случайные начальные точки, одна из которых обозначена кружком, другая — квадратом и третья — треугольником. Если мы используем градиентный спуск в каждой из этих трех точек, квадрату удастся найти глобальный минимум функции.

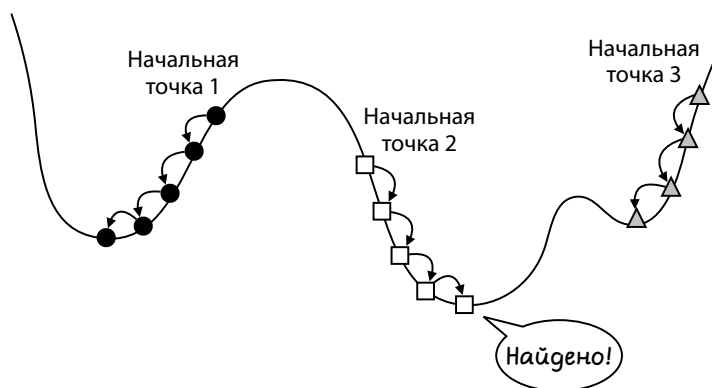


Рис. Б.9. Метод случайного перезапуска

Этот метод все еще не гарантирует нахождения глобального минимума, потому что нам может не повезти и мы выберем только точки, которые застрянут в долинах. Однако при достаточном количестве случайных начальных точек появится гораздо больше шансов отыскать глобальный минимум. И даже если мы не сможем обнаружить его, то все равно сможем найти достаточно хороший локальный минимум, который поможет обучить хорошую модель.



Эти ссылки можно найти также в <https://serrano.academy/grokking-machine-learning/>.

Общие ссылки

- Репозиторий GitHub: www.github.com/luisguiserrano/manning.
- Видео на YouTube: www.youtube.com/c/LuisSerrano.
- Общая информация: <https://serrano.academy>.
- Информация о книге: <https://serrano.academy/grokking-machine-learning>.

Курсы

- Наноуровень машинного обучения Udacity: <http://mng.bz/4KE5>.
- Курс машинного обучения Coursera: <https://www.coursera.org/learn/machine-learning>.
- Специализация Coursera по машинному обучению (Вашингтонский университет): <http://mng.bz/Xryl>.
- Сквозное машинное обучение: <https://end-to-end-machine-learning.teachable.com/courses>.

Блоги и каналы на YouTube

- Видео Брэндона Ропера о машинном обучении: <https://www.youtube.com/user/BrandonRohrer>.
- StatQuest с Джошем Стармером: <https://www.youtube.com/user/joshstarmar>.
- Блог Криса Олаха: <https://colah.github.io/>.
- Блог Джея Аламмара: <https://jalammr.github.io/>.
- Блог Алексис Кук: <https://alexisbcook.github.io/>.
- Блог Дхрува Партхасаратхи: <https://medium.com/@dhrupv>.
- 3Blue1Brown: <http://youtube.com/c/3blue1brown>.
- Мастерство машинного обучения: <https://machinelearningmastery.com>.
- Блог Андрея Карпати: <http://karpathy.github.io/>.

Книги

- *Bishop C. M. Pattern Recognition and Machine Learning*: <http://mng.bz/g1DZ>.

Глава 1

Видео

- Общие видеоролики по машинному обучению: <https://serrano.academy/general-machine-learning/>.
- Мягкое введение в машинное обучение: www.youtube.com/watch?v=IpGxLWOIZy4.
- Скетч Монти Пайтона о спаме: www.youtube.com/watch?v=zLih-WQwBSc.

Глава 2

Видео

- Видео о контролируемом машинном обучении: <https://serrano.academy/linear-models/>.
- Видео о неконтролируемом машинном обучении: <https://serrano.academy/unsupervised-learning/>.
- Видео о генеративном машинном обучении: <https://serrano.academy/generative-models>.
- Обучение с подкреплением: <https://serrano.academy/reinforcement-learning>.
- Видео о глубоком обучении: <https://serrano.academy/neural-networks>.

Книги

- *Morales M. Grokking Deep Reinforcement Learning*¹: <http://mng.bz/5Zy4>.

Курсы

- *Сильвер Д.* Курс UCL по обучению с подкреплением: <https://www.davidsilver.uk/teaching/>.
- Программа Udacity по глубокому обучению с подкреплением в Nano-degree: <http://mng.bz/6mMG>.

Глава 3

Код

- Репозиторий GitHub: <http://mng.bz/o8lN>.

Наборы данных

- Набор данных о жилье в Хайдарабаде:
 - создатель — Ручи Бхатия;
 - дата — 2020/08/27;
 - версия 4;

¹ *Моралес М.* Грокаем глубокое обучение с подкреплением. — СПб.: Питер, 2023.

- получено из <http://mng.bz/nrdv>;
- лицензия CC0 — общественное достояние.

Видео

- Видео о линейной регрессии: <http://mng.bz/v4Rx>.
- Видео о полиномиальной регрессии: <https://www.youtube.com/watch?v=HmKAEFaW0>.

Глава 4

Код

- Репозиторий GitHub: <http://mng.bz/4KXB>.

Видео

- Машинное обучение, показатели тестирования и ошибок: <https://www.youtube.com/watch?v=aDW44NPhNw0>.
- Регрессия лассо (L1) (StatQuest): <https://www.youtube.com/watch?v=NGf0voTmIcs>.
- Гребневая регрессия (L2) (StatQuest): <https://www.youtube.com/watch?v=Q81RR3yKn30>.

Глава 5

Код

- Репозиторий GitHub: <http://mng.bz/Qqpm>.

Видео

- Логистическая регрессия и алгоритм персептрона (видео): <https://www.youtube.com/watch?v=jbluHIGBmBo>.
- Скрытые модели Маркова (видео): <https://www.youtube.com/watch?v=kqSzLo9fenk>.

Глава 6

Код

- Репозиторий GitHub: <http://mng.bz/Xr9Y>.

Наборы данных

- Обширный набор отзывов о фильмах IMDB:
 - создатель Стефано Леоне;
 - дата 2019/11/24;
 - версия 2;
 - получено из <https://www.kaggle.com/stefanoleone992/imdb-extensive-dataset>;
 - лицензия CC0 — общественное достояние.

Видео

- Видео о логистической регрессии и алгоритме персептрона: <https://www.youtube.com/watch?v=jbluHIgBmBo>.
- Перекрестная энтропия (StatQuest): <https://www.youtube.com/watch?v=6ArSys5qHAU>.
- Перекрестная энтропия (Орельен Жерон): <https://www.youtube.com/watch?v=ErfnhcEV108>.

Глава 7

Видео

- Машинное обучение, показатели тестирования и ошибок: <https://www.youtube.com/watch?v=aDW44NPhNw0>.

Глава 8

Код

- Репозиторий GitHub: <http://mng.bz/yJRJ>.

Наборы данных

- Набор данных спам-фильтра:
 - создатель Картик Вееракумар;
 - дата 2017/07/14;
 - версия 1;
 - получено из <https://www.kaggle.com/karthickveerakumar/spam-filter>;
 - видимость публичная.

Видео

- Наивный Байес: <https://www.youtube.com/watch?v=Q8l0Vip5YUw>.

Глава 9

Код

- Репозиторий GitHub: <http://mng.bz/MvM2>.

Наборы данных

- Набор данных о поступлении:
 - создатель Мохан С. Ачарья;
 - дата 2018/03/03;
 - версия 2;
 - получено из <http://mng.bz/aZIJ>;
 - статья: *Acharya M. S., Armaan A., Antony A. S. A Comparison of Regression Models for Prediction of Graduate Admissions* // IEEE International Conference on Computational Intelligence in Data Science, 2019;
 - лицензия CC0 — общественное достояние.

Видео

- Деревья принятия решений (StatQuest): <https://www.youtube.com/watch?v=7VeUPuFGJHk>.
- Деревья регрессионных решений (StatQuest): <http://www.youtube.com/watch?v=g9c66TUylZ4>.
- Деревья принятия решений (Брэндон Ропер): <https://www.youtube.com/watch?v=9w16p4QmkAI>.
- Индекс примесей Джини: <http://www.youtube.com/watch?v=u4IxOk2ijSs>.
- Энтропия Шеннона и прирост информации: <https://www.youtube.com/watch?v=9r7FIXEAGvs>.

Сообщения в блоге

- Shannon entropy, information gain and picking balls from buckets (Энтропия Шеннона, прирост информации и доставание мячей из ведер): <http://mng.bz/g1lR>.

Глава 10

Код

- Репозиторий GitHub: <http://mng.bz/ePAJ>.

Наборы данных

- Набор данных MNIST. *Deng, L.* The MNIST Database of Handwritten Digit images for Machine Learning Research // IEEE Signal Processing Magazine 29, 2012, no. 6: 141–142. <http://yann.lecun.com/exdb/mnist/>.
- Набор данных о жилье в Хайдарабаде (см. ссылки на главу 3).

Видео

- Глубокое обучение и нейронные сети: <http://www.youtube.com/watch?v=BR9h47Jtqyw>.
- Сверточные нейронные сети: <https://www.youtube.com/watch?v=2-Ol7ZB0MmU>.
- Рекуррентные нейронные сети: <https://www.youtube.com/watch?v=UNmqTiOnRfg>.
- Как работают нейронные сети (Брэндон Ропер): <https://www.youtube.com/watch?v=ILsA4nyG7I0>.
- Рекуррентные нейронные сети (RNN) и долгая кратковременная память (LSTM) (Брэндон Ропер): <https://www.youtube.com/watch?v=WCUNPb-5EYI>.

Книги

- *Trask A.* Grokking Deep Learning¹: <https://www.manning.com/books/grokking-deep-learning>.
- *Goodfellow I., Bengio Y., Courville A.* Deep Learning²: <https://www.deeplearningbook.org/>.

¹ *Траск Э.* Грокаем глубокое обучение. — СПб.: Питер, 2022.

² *Гудфеллоу Я., Бенджио И., Курвилль А.* Глубокое обучение.

Курсы

- Курс глубокого обучения Udacity: <http://mng.bz/p9lP>.

Сообщения в блогах

- *Cook A.* Using Transfer Learning to Classify Images with Keras (Использование технологии переноса для классификации изображений с помощью Keras): <http://mng.bz/OQgP>.
- *Cook A.* Global Average Pooling Layers for Object Localization (Глобальные средние слои для локализации объектов): <http://mng.bz/Ywj7>.
- *Parthasarathy D.* A Brief History of CNNs in Image Segmentation: From R-CNN to Mask R-CNN (Краткая история CNN в сегментации изображений: от R-CNN до маски R-CNN): <http://mng.bz/GONN>.
- *Olah C.* Neural networks, Manifolds and Topology (Нейронные сети, многообразия и топология): <http://mng.bz/zERZ>.
- *Olah C.* Understanding LSTM Networks (Понимание сетей LSTM): <http://mng.bz/01nz>.
- *Alammar J.* How GPT3 Works: Visualizations and Animations (Как работает GPT3: визуализации и анимация): <http://mng.bz/KoXn>.
- *Brownlee J.* How to Configure the Learning Rate When Training Deep Learning Neural Networks (Как настроить алгоритм при обучении нейронных сетей глубокого обучения): <http://mng.bz/9ae8>.
- *Jordan J.* Setting the Learning Rate of Your Neural Network (Настройка скорости обучения вашей нейронной сети): <http://mng.bz/WBKX>.
- *Gad A.* Selecting the Best Architecture for Artificial Neural Networks (Выбор лучшей архитектуры для искусственных нейронных сетей): <http://mng.bz/WBKX>.
- *Karpathy A.* A Recipe for Training Neural Networks (Рецепт для обучения нейронных сетей): <http://mng.bz/80gg>.

Инструменты

- TensorFlow playground: <https://playground.tensorflow.org/>.

Глава 11

Код

- Репозиторий GitHub: <http://mng.bz/ED6r>.

Видео

- Машины опорных векторов: https://www.youtube.com/watch?v=Lpr__X8zuE8.
- Ядро полинома (StatQuest): <https://www.youtube.com/watch?v=Toet3EiSfCM>.
- Радиальное ядро (RBF) (StatQuest): https://www.youtube.com/watch?v=Qc5IyLW_hns.

Статья в блоге

- *Sicotte X. B.* Kernels and Feature Maps: Theory and Intuition (Ядра и карты объектов: теория и интуиция): <http://mng.bz/N4aX>.

Глава 12

Код

- Репозиторий GitHub: <http://mng.bz/DK50>.

Видео

- Случайные леса (StatQuest): https://www.youtube.com/watch?v=J4Wdy0Wc_xQ.
- AdaBoost (statquest): <https://www.youtube.com/watch?v=LsK-xG1cLYA>.
- Градиентный бустинг (StatQuest): <https://www.youtube.com/watch?v=3CC4N4z3GJc>.
- XGBoost (StatQuest): <https://www.youtube.com/watch?v=OtD8wVaFm6E>.

Статьи и сообщения в блогах

- *Freund Y., Shapire R.* A Decision-Theoretic Generalization of Online Learning and an Application to Boosting // Journal of Computer and System Sciences, 55, 119–139: <http://mng.bz/l9Bz>.
- Explaining AdaBoost, by Robert Schapire (Шапир Р. Объяснение AdaBoost): <http://rob.schapire.net/papers/explaining-adaboost.pdf>.
- *Chen T., Guestrin C.* XGBoost: A Scalable Tree Boosting System // KDD '16: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, August 2016, 785–794: <https://doi.org/10.1145/2939672.2939785>.
- *Chen E.* Winning the Netflix Prize: A Summary (Победа в премии Netflix: краткое содержание): <http://mng.bz/B1jq>.

Глава 13

Код

- Репозиторий GitHub: <http://mng.bz/drlz>.

Наборы данных

- Набор данных «Титаник»: получено из <https://www.kaggle.com/c/titanic/data>.

Луис Серрано
Грокаем машинное обучение

Перевел с английского Р. Чикин

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Ведущий редактор	<i>Н. Гринчик</i>
Литературный редактор	<i>Н. Рощина</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>Т. Никифорова, Л. Галаганова</i>
Верстка	<i>Е. Цыцен</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».
Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,
Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 12.2023.

Наименование: книжная продукция.

Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014,
58.11.12 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск,
ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 02.11.23. Формат 70×100/16. Бумага офсетная.
Усл. п. л. 41,280. Тираж 2000. Заказ 0000.